

TMS320DM270 ***CPU and Peripherals***

Technical Reference Manual ***Version 1.0***

IMPORTANT NOTICE

Texas Instruments and its subsidiaries (TI) reserve the right to make changes to their products or to discontinue any product or service without notice, and to advise customers to obtain the latest version of relevant information to verify, before placing orders, that information being relied on is current and complete. All products are sold subject to the terms and conditions of sale supplied at the time of order acknowledgment, including those pertaining to warranty, patent infringement, and limitation of liability.

TI warrants performance of its semiconductor products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are utilized to the extent TI deems necessary to support this warranty. Specific testing of all parameters of each device is not necessarily performed, except those mandated by government requirements.

Customers are responsible for their applications using TI components. In order to minimize risks associated with the customer's applications, adequate design and operating safeguards must be provided by the customer to minimize inherent or procedural hazards.

TI assumes no liability for applications assistance or customer product design. TI does not warrant or represent that any license, either express or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right of TI covering or relating to any combination, machine, or process in which such semiconductor products or services might be or are used. TI's publication of information regarding any third party's products or services does not constitute TI's approval, warranty or endorsement thereof.

Mailing Address:

Texas Instruments
Post Office Box 655303
Dallas, Texas 75265

Copyright © 2003, Texas Instruments Incorporated

Document Revision History

Revision	Date	Notes
1.0	16 July 2003	First release

Read This First

About This Manual

This user's guide serves as a reference for the Texas Instruments TMS320DM270 low-power, enhanced-architecture, dual-core digital signal processor (DSP), and is intended to assist hardware and software engineers in developing applications using this device. It describes both cores of the chip and their coprocessors—the TMS320C54x DSP CPU, the ARM7TDMI microcontroller unit (MCU), the Variable Length Coder and the Imaging Extension Coprocessor—and peripherals, together with the memory and peripheral interface associated with each core.

Throughout this book, the TMS320DM270 dual-core device is referred to as the DM270. Where the ARM7TDMI MCU core is referred to separately, the alphanumeric designation is shortened to ARM. Information about the two-processor cores in relevant chapters is provided separately, rather than combining similar features where applicable.

Notational Conventions

This book uses the following conventions.

- **Chip Support Library** (CSL) macro notation is used throughout this book. Refer Appendix for description about this notation
- Program listings and program examples are shown in a special typeface. Here is a segment of a program listing:

```
OSD_FSET(VIDWINADH, V1AH, vwin1_start >> 16 );
OSD_RSET(VINWIN1AL, vwin1_start & 0xFFFF );

while ( MMCSD_FGET( DMASTAT1, RUNST ) == 1 );
```

- Instruction sets

The TMS320DM270 DSP CPU can use either of two forms of the instruction set: a mnemonic form or an algebraic form. This book uses the mnemonic form of the instruction set. For information about the mnemonic form of the instruction set, see TMS320C54x DSP Reference Set, Volume 2: Mnemonic Instruction Set. For information about the algebraic form of the instruction set, see TMS320C54x DSP Reference Set, Volume 3: Algebraic Instruction Set. These references are both listed in the section titled Related Documentation From Texas Instruments.

The ARM7TDMI MCU CPU uses its own instruction set. For information about the MCU's instruction set, see TMS470R1x User's Guide, also shown in Related Documentation From Texas Instruments.

Related DM270 Documentation From Texas Instruments

Please also refer to the following documents for additional information about DM270

- o DM270 Datasheet
- o DM270 EVM User's Guide

Related DSP Documentation From Texas Instruments

The following books provide related documentation for the TMS320DM270. To obtain a copy of any of these TI documents, call the Texas Instruments Literature Response Center at (800) 477-8924. When ordering, please identify the book by its title and literature number. Many of these documents are located on the Internet at <http://www.ti.com/>.

- o TMS320C54x DSP Reference Set, Volume 1: CPU and Peripherals (literature number SPRU131F) provides information for developing hardware and software applications using the TMS320C54x
- o TMS320C54x DSP Reference Set, Volume 2: Mnemonic Instruction Set (literature number SPRU172) describes the TMS320C54x digital signal processor mnemonic instructions individually. Also includes a summary of instruction set classes and cycles.
- o TMS320C54x DSP Reference Set, Volume 3: Algebraic Instruction Set (literature number SPRU179) describes the TMS320C54x digital signal processor algebraic instructions individually. Also includes a summary of instruction set classes and cycles.
- o TMS470R1x User's Guide (literature number SPNU134) describes the TMS470R1x RISC microcontroller, its architecture (including registers), the ICEBreaker module, interfaces (memory, coprocessor, and debugger), 16-bit and 32-bit instruction sets, and electrical specifications.
- o TMS320C54x DSKplus User's Guide (literature number SPRU191) describes the TMS320C54x digital signal processor starter kit (DSK), which allows you to execute custom C54x. code in real time and debug it line by line. Covered are installation procedures, a description of the debugger and the assembler, customized applications, and initialization routines.
- o TMS320C54x Assembly Language Tools User's Guide (literature number SPRU102) describes the assembly language tools (assembler, linker, and other tools used to develop assembly language code), assembler directives, macros, common object file format, and symbolic debugging directives for the C54x generation of devices.
- o TMS320C54x C Source Debugger User's Guide (literature number SPRU099) tells you how to invoke the C54x emulator, evaluation module, and simulator versions of the C source debugger interface. This book discusses various aspects of the debugger interface, including window management, command entry, code execution, data management, and breakpoints. It also includes a tutorial that introduces basic debugger functionality.
- o TMS320C54x Code Generation Tools Getting Started Guide (literature number SPRU147) describes how to install the TMS320C54x assembly

language tools and the C compiler for the C54x devices. The installation for MS-DOS., OS/2., SunOS., Solaris., and HP-UX. 9.0x systems is covered.

- o TMS320C54x Evaluation Module Technical Reference (literature number SPRU135) describes the C54x evaluation module, its features, design details and external interfaces.
- o TMS320C54x Optimizing C/C++ Compiler User's Guide (literature number SPRU103) describes the C54x C compiler. This C compiler accepts ANSI standard C source code and produces TMS320. assembly language source code for the C54x generation of devices.
- o TMS320C54x Simulator Getting Started Guide (literature number SPRU137) describes how to install the TMS320C54x simulator and the C source debugger for the C54x. The installation for MS-DOS., PCDOS ., SunOS., Solaris., and HP-UX. systems is covered.
- o TMS320 Third-Party Support Reference Guide (literature number SPRU052) alphabetically lists over 100 third parties that provide various products that serve the family of TMS320 digital signal processors. A myriad of products and applications are offered—software and hardware development tools, speech recognition, image processing, noise cancellation, modems, etc.
- o TMS320 DSP Development Support Reference Guide (literature number SPRU011) describes the TMS320_ DSP family of digital signal processors and the tools that support these devices. Included are code-generation tools (compilers, assemblers, linkers, etc.) and system integration and debug tools (simulators, emulators, evaluation modules, etc.). Also covered are available documentation, seminars, the university program, and factory repair and exchange.

Trademarks

TMS320C54x, C54x, TMS320, ICECrusher, and MicroStar BGA are trademarks of Texas Instruments.

ARM7TDMI, ARM, Thumb, and Multi-ICE are registered trademarks of ARM Limited.

ICEBreaker, ARM7, and EmbeddedICE are trademarks of ARM Limited.

°C is a registered trademark of PHILLIPS

MS-DOS is a registered trademark of Microsoft Corporation.

OS/2 and PC-DOS are trademarks of International Business Machines Corporation.

SunOS and Solaris are trademarks of Sun Microsystems, Inc.

HP-UX is a trademark of Hewlett-Packard Company.

Other trademarks are the property of their respective owners.

Table Of Contents

READ THIS FIRST	4
TABLE OF CONTENTS	7
1 INTRODUCTION	13
1.1 KEY FEATURES	14
1.2 BLOCK DIAGRAM	15
1.3 FEATURES OF ARM SUBSYSTEM	16
1.4 FEATURES OF THE DSP SUBSYSTEM BLOCK	18
1.5 FEATURES OF THE IMAGING PERIPHERAL BLOCK	21
1.6 FEATURES OF THE MEMORY TRAFFIC CONTROLLER BLOCK	23
1.7 DM270 SYSTEM DATA FLOWS	24
2 ARM SUBSYSTEM	28
2.1 INTRODUCTION	28
2.2 ARM7TDMI RISC PROCESSOR CORE	29
2.3 ARM SUBSYSTEM BUS CONTROLLER	32
3 INSTRUCTION CACHE	42
3.1 FEATURES	42
3.2 I-CACHE CONFIGURATION	43
3.3 CACHE OPERATION EXAMPLE	44
3.4 I-CACHE CONTROL	45
3.5 I-CACHE OPERATION NOTES	45
3.6 INSTRUCTION CACHE REGISTER MAP (ICACHE)	47
3.7 INSTRUCTION CACHE REGISTERS	47
4 CLOCK CONTROLLER	48
4.1 INTRODUCTION	48
4.2 INPUT CLOCK SELECTION	51
4.3 SETTING PLL CLOCK OUTPUT FREQUENCY	52
4.4 PLL BYPASS MODE	53
4.5 MODULE CLOCK DISTRIBUTION	53
4.6 GIO CLOCK OUTPUT	54
4.7 PULSE WIDTH MODULATION	54
4.8 CLOCK CONTROLLER REGISTER MAP (CLKC)	55
4.9 CLOCK CONTROLLER REGISTERS	56
5 INTERRUPT CONTROLLER	69
5.1 INTRODUCTION	69
5.2 INTERRUPT SOURCE LIST	69
5.3 INTC METHODOLOGY	70
5.4 HARDWARE CONFIGURABLE INTERRUPT ENTRY TABLE	74
5.5 EXTERNAL INTERRUPTS	75
5.6 INTERRUPT CONTROLLER REGISTER MAP (INTC)	76
5.7 INTERRUPT CONTROLLER REGISTERS	77

6 TIMERS	114
6.1 INTRODUCTION	114
6.2 CONFIGURING THE TIMERS	114
6.3 TIMER REGISTER MAP (TMRx)	116
6.4 TIMER REGISTERS	117
7 WATCH DOG TIMER (WDT)	119
7.1 INTRODUCTION	119
7.2 USING THE WATCH DOG TIMER	119
7.3 SETTING THE CONFIGURABLE DETECTION TIME	120
7.4 WATCH DOG TIMER REGISTER MAP (WDT)	121
7.5 WATCH DOG TIMER REGISTERS	122
8 ADDRESS TRACE MODULE	124
8.1 INTRODUCTION	124
8.2 ADDRESS TRACE MODULE REGISTER MAP (ATM)	125
8.3 ADDRESS TRACE MODULE REGISTERS	126
9 CCD CONTROLLER	128
9.1 FEATURES	128
9.2 SIGNAL INTERFACE	129
9.3 CCDC RAW DATA INPUT INTERFACE	130
9.4 TIMING GENERATOR, FRAME SIZE SETUP	132
9.5 CCD CONTROLLER PROCESSING	133
9.6 CCD INTERRUPTS	141
9.7 SHUTTER CONTROL	141
9.8 YCBCR SIGNAL PROCESSING	142
9.9 INPUT/OUTPUT PORT CONFIGURATION	143
9.10 CCIR-656 4:2:2 PARALLEL INTERFACE	144
9.11 CCDC REGISTER UPDATE TIMING	148
9.12 RELATION BETWEEN CCD RAW DATA AND MODESET REGISTER	149
9.13 CCD CONTROLLER REGISTER MAP (CCDC)	151
9.14 CCD CONTROLLER REGISTERS	152
10 PREVIEW ENGINE	172
10.1 INTRODUCTION	172
10.2 PREVIEW ENGINE BLOCK DIAGRAM	173
10.3 MODULE DESCRIPTION	174
10.4 PREVIEW ENGINE OPERATION GUIDE	187
10.5 FRACTIONAL NOTATION	203
10.6 PREVIEW ENGINE REGISTER MAP (PREV)	204
10.7 PREVIEW ENGINE REGISTERS	205
11 HARDWARE 3A	231
11.1 FEATURES	231
11.2 INPUT DATA FOR H3A ENGINE	233
11.3 AUTO FOCUS (AF) ENGINE	233
11.4 AUTO EXPOSURE, AUTO WHITE-BALANCE ENGINE (AEWB)	239
11.5 HARDWARE 3A REGISTER UPDATE TIMING	242
11.6 HARDWARE 3A REGISTER MAP (H3A)	243
11.7 HARDWARE 3A REGISTERS	244
12 OSD - ON-SCREEN DISPLAY	263
12.1 INTRODUCTION	263
12.2 FEATURES	263

12.3	OSD BLOCK DIAGRAM	264
12.4	OSD WINDOWS AND THEIR PRIORITIES	264
12.5	OSD DATA STORAGE FORMAT	265
12.6	OSD CONFIGURATION AND CONTROL	267
12.7	BITMAP WINDOW	272
12.8	RECTANGULAR HARDWARE CURSOR	278
12.9	WINDOW EXPANSION (VGA TO NTSC/PAL CONVERSION)	279
12.10	MISCELLANEOUS FEATURES	279
12.11	ON SCREEN DISPLAY REGISTER MAP (OSD)	281
12.12	ON SCREEN DISPLAY REGISTERS	282
13	VIDEO ENCODER	321
13.1	INTRODUCTION	321
13.2	VIDEO ENCODER OUTPUT MODES	323
13.3	NTSC/PAL ENCODER	324
13.4	SYNCHRONOUS MODE	328
13.5	DIGITAL RGB/YUV INTERFACE	331
13.6	VIDEO ENCODER REGISTER MAP (VENC)	350
13.7	VIDEO ENCODER INTERFACE REGISTERS	350
14	GENERAL PURPOSE INPUT OUTPUT (GIO).....	377
14.1	INTRODUCTION	377
14.2	GIO PORT STATE CONTROL	377
14.3	EXTERNAL INTERRUPT SETTINGS	378
14.4	GIO FUNCTIONAL SETTINGS	378
14.5	GENERAL I/O REGISTER MAP (GIO)	381
14.6	GENERAL I/O REGISTERS	382
15	SERIAL PORT INTERFACE (SPI)	393
15.1	INTRODUCTION	393
15.2	SERIAL INTERFACE SIGNAL PINS	394
15.3	BAUD RATE	394
15.4	MASTER MODE SIF	395
15.5	SLAVE MODE SIF	396
15.6	TRANSMISSION AND RECEPTION BUFFERS	397
15.7	SERIAL PORT MODES	397
15.8	INTERRUPTS	397
15.9	TRANSFER USING THE DIRECT MEMORY ACCESS	398
15.10	SERIAL PORT REGISTER MAPS (SPx)	400
15.11	SERIAL PORT REGISTERS	401
16	UART.....	405
16.1	INTRODUCTION	405
16.2	UART REGISTER MAPS (UARTx).....	408
16.3	UART REGISTERS	409
17	MMC/SD INTERFACE	414
17.1	INTRODUCTION	414
17.2	FEATURES	414
17.3	MMC/SD MODE.....	422
17.4	SPI MODE	438
17.5	DMA FUNCTION	449
17.6	SDIO CARD FUNCTION	450
17.7	MMC/SD CARD INTERFACE REGISTER MAP (MMCSd)	451
17.8	MMC/SD CARD INTERFACE REGISTERS	452

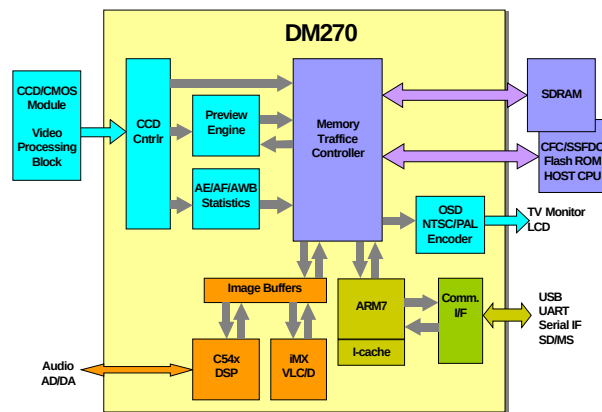
18	USB INTERFACE	483
18.1	INTRODUCTION TO USB INTERFACE	483
18.2	REFERENCES	483
18.3	SPECIFICATIONS	483
18.4	BLOCK DIAGRAM	484
18.5	INITIALIZATION FLOW	485
18.6	FIFO SIZE SELECTION	485
18.7	FIFO BASE ADDRESS SELECTION.....	486
18.8	INTERRUPT THRESHOLD	486
18.9	INTERRUPT CONTROL	487
18.10	PROTOCOL ENGINE RESET	487
18.11	DEVICE CONFIGURATION	487
18.12	DEVICE INITIALIZATION	488
18.13	POWER MANAGEMENT	490
18.14	USB ENDPOINT	491
18.15	USB STATUS	491
18.16	ENDPOINT CONTROL	492
18.17	UNIVERSAL SERIAL BUS INTERFACE REGISTER MAP (USB)	494
18.18	USB REGISTERS	495
19	I²C INTERFACE	534
19.1	INTRODUCTION	534
19.2	PSEUDO-I ² C INTERFACE	534
19.3	I ² C BUS CONFIGURATION	535
19.4	DATA TRANSFER SEQUENCE	536
19.5	I2C BUS INTERFACE REGISTER MAP (I2C).....	546
19.6	I2C BUS INTERFACE REGISTERS	547
20	MEMORY TRAFFIC CONTROLLER (MTC).....	550
20.1	INTRODUCTION	551
20.2	REGISTER SETTINGS	553
20.3	ACCESS TO EMIF AREAS	554
20.4	EXTERNAL MEMORY INTERFACE REGISTER MAP (EMIF)	571
20.5	EXTERNAL MEMORY INTERFACE REGISTERS	572
20.6	SDRAM CONTROLLER	596
20.7	SDRAM CONTROLLER REGISTER MAP (SDRC).....	601
20.8	SDRAM CONTROLLER REGISTERS	602
20.9	EMIF DMA CONTROLLER	623
20.10	SDRAM & EXTERNAL MEMORY I/F PIN SHARING	626
21	EXTERNAL HOST INTERFACE (EHIF)	631
21.1	INTRODUCTION	631
21.2	MEMORY SPACE	632
21.3	BUS ACCESS IN EXTERNAL HOST MODE.....	633
21.4	INTERRUPTS IN EXTERNAL HOST MODE.....	633
21.5	SDRAM ACCESS	634
21.6	EHIF CONTROL REGISTER MAP.....	644
21.7	DETAILED REGISTER DESCRIPTION	644
22	ARM BOOT-UP MODES	652
22.1	SELF BOOT-UP MODE	652
22.2	SERIAL BOOT-UP MODE	652
22.3	EXTERNAL HOST BOOT-UP MODE	656
23	ARM POWER SAVING MODES.....	658

23.1	DISABLING MODULE CLOCKS	658
23.2	USB POWER SAVING MODES	658
23.3	SLEEP MODE	659
23.4	POWER DOWN MODE	659
23.5	DISABLING DAC POWER	660
23.6	DISABLING POWER TO SDRAM	660
23.7	BYPASS BITS IN BYP (0x3088A)	661
24	DSP SUBSYSTEM	662
24.1	INTRODUCTION	662
24.2	DSP CORE	662
24.3	DSP PERIPHERALS	663
24.4	DSP SUBSYSTEM MEMORY	669
24.5	DSP INTERNAL MEMORY	671
24.6	DSP - POWER SAVING MODES	676
25	DSP COPROCESSOR SUBSYSTEM	678
25.1	INTRODUCTION TO COPROCESSOR SUBSYSTEM	678
25.2	COPROCESSOR SUBSYSTEM : PERIPHERAL REGISTER MAP	681
25.3	COPROCESSOR SUBSYSTEM : CLOCK CONTROL	681
25.4	SHARED MEMORY ACCESS SWITCHING AND MEMORY MAP	683
25.5	IMAGE BUFFER DMA CONTROLLER	683
25.6	IMAGE BUFFER DMA (IMGBUF)	687
25.7	IMAGE BUFFER DMA REGISTERS	688
25.8	IMAGING EXTENSION ENGINE (IMX)	695
25.9	IMX CONTROL REGISTER MAP (IMX)	698
25.10	IMX CONTROL REGISTERS	698
25.11	VARIABLE LENGTH CODER/DECODER (VLCD)	703
25.12	VARIABLE LENGTH CODER DECODER REGISTER MAP (VLCD)	706
25.13	VLCD REGISTERS	707
25.14	VLC – HUFFMAN TABLE CONFIGURATION	742
25.15	VLC – HEADER INSERTION	744
25.16	VLD – HUFFMAN TABLE CONFIGURATION	745
25.17	QIQ – SCANNING PATHS AND QUANTIZATION MATRICES USED	748
25.18	VLC TABLES USED	750
25.19	VLD TABLES USED	752
26	ARM-DSP COMMUNICATION	753
26.1	INTRODUCTION	753
26.2	HPI-16 MODE	754
26.3	MEMORY ACCESS	754
26.4	INTERRUPTS BETWEEN ARM AND DSP	755
26.5	BOOT LOADER	755
26.6	ARM-SIDE HPI CONTROL AND STATUS REGISTERS	757
26.7	ARM SUBSYSTEM - DSP CONTROLLER (DSPC)	759
26.8	ARM SUBSYSTEM - DSP CONTROLLER REGISTERS	760
26.9	DSP SUBSYSTEM - HPIB DMAC	762
26.10	DSP SUBSYSTEM - HOST PORT INTERFACE BRIDGE REGISTER MAP (HPIB)	763
26.11	DSP SUBSYSTEM - HOST PORT INTERFACE BRIDGE REGISTERS	764
27	APPENDIX	769
27.1	APPENDIX A : TMX320DM270GHK PIN ASSIGNMENT	769
27.2	APPENDIX B : CHIP SUPPORT LIBRARY (CSL) MACRO NOTATION	781

1 Introduction

The DM270 is a highly integrated, programmable platform for the digital still camera and mobile system. Designed to offer camera manufacturers the ability to produce affordable DSC products with high picture quality, the DM270 combines programmable image processing capability with a highly integrated imaging peripheral set. The chip's programmability comes from a DSP-based imaging co-processor that enables camera manufacturers to implement their own proprietary image processing algorithms in software. The DM270 also enables seamless interface to most additional external devices required for a complete digital camera implementation. The interface is flexible enough to support various types of CCD and CMOS sensors, signal conditioning circuits, power management, SDRAM, shutter, Iris and auto-focus motor controls. The DM270 will allow camera manufacturers to meet customer demands by fulfilling both image quality and low cost expectations required in the volume point and shoot digital still camera segment.

Figure 1. DM270 System Architecture



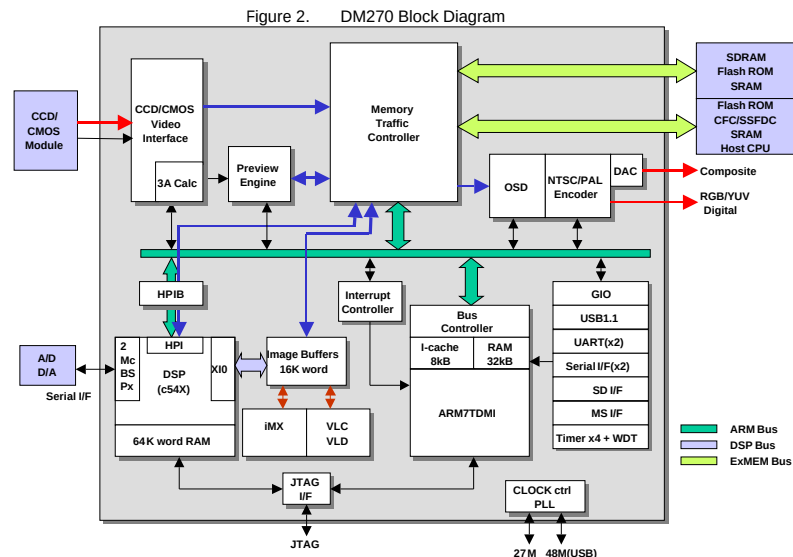
1.1 Key Features

- 288-pin micro star BGA(μ *BGA) 16x16mm (0.8mm pitch)
- Supports 102 MHz SDRAM up to 64Mbytes with 32/16-bit interface
- Supports up to 14-bit AFE (Analog Front End)
- Supports 8/16bit CCIR601/656 video format
- Supports Motion JPEG compression/decompression video at VGA resolution up to 30fps
- Supports MPEG4 I+P frame video at CIF resolution up to 30fps
- On-chip 88MHz ARM7 32 bit RISC microprocessor with I-cache (8 Kbytes) and on-chip program/data memory (32 Kbytes)
- On-chip 100MHz Texas Instruments C54X Digital Signal Processor(DSP) core with on-chip program/data memory (128Kbytes)
- On-chip image processing buffers (16Kbytes)
- On-chip 180MHz programmable SIMD image processing engine (iMX) for programmable image processing
- Variable Length Coding and Decoding coprocessor (VLC/VLD) for image/video compression and decompression.
- Preview engine for real-time live-view and digital zoom
- Versatile On Screen Display(OSD)
- Digital interface for color LCD
- Seamless interface Compact Flash, Smart Media, MMC/SD, and Memory Stick cards
- Supports flash memory up to 16MB
- Supports 27MHz, 48MHz (USB) crystal input to generate all clocks
- Four Timers + One WDT
- Two UARTs
- 1-channel 10-bit DAC for NTSC/PAL composite video output
- 2-channel serial audio codec interface (McBSP)
- Two sets of SPI serial interface with master/slave mode
- Up to 34 General-purpose IO pins available
- Limited I²C bus master capability
- On-chip USB 1.1 Function Controller
- 2 channel programmable PWM (pulse width modulation) outputs.

- Externally downloadable using SIF (serial interface) or HOST bus interface.
- External HOST interface capability to allow external HOST processor to interface with and control DM270

1.2 Block Diagram

A detailed DM270 Block Diagram is given below.



The blocks that make the DM270 can be grouped into four entities:

- The ARM subsystem and its peripherals
- The DSP subsystem with its two co-processors and memories
- The imaging peripheral block
- The memory traffic controller block

The features of each of these blocks are given in the subsequent section.

1.3 Features of ARM Subsystem

1.3.1 ARM7 Core

The ARM7 32-bit RISC processor contained in the DM270 acts as the system controller. The ARM7 CPU performs general system control tasks such as system initialization, configuration, user interface and user command implementation.

The embedded ARM processor features are:

- 32-bit RISC processor
- Maximum operating frequency 88Mhz (Refer DM270 datasheet for exact value)
- 32-Kbyte built-in RAM
- 8KB I-cache
- 32/16-bit command sets
- Memory format: Little Endian Format fixed
- 2 interrupt systems (FIQ, IRQ)
- 15 external terminal interrupt source
- 4 hardware timers
- 1 Watchdog timer
- JTAG-based emulation/debug capability
- External memory interface

1.3.2 USB Controller

The USB controller built into the DM270 conforms to USB Specification Revision 1.1. It corresponds to full speed devices and works in slave mode. The USB has a built-in transceiver, its own 2K bytes of RAM for FIFO, a programmable packet size and supports remote wakeup and DMA transfer to/from SDRAM.

1.3.3 UART ports

The DM270 supports two-channel start-stop synchronization serial ports (UART). One channel (UART0) supports a full set of UART communications. The UARTs contain separate 32-byte FIFOs for both the transmit side and receive functions and interrupts can be generated due to causes such as errors or various states of FIFO. The UARTs support 8 or 7 data bits, even, odd or no parity bit, and 1 or 2 stop bits. Transmission/reception timing uses the ARM clock, and any bit rate can be set in integral multiples of 1/16 of the input clock.

1.3.4 Serial Ports

DM270 has two clock-synchronous serial interfaces. These are simple three-line serial interfaces consisting of clock, output data, and input data signals. One of the serial interfaces supports SDRAM DMA transfers. Transmission is in 8-bit units, and each port has 8-bit buffers on each of the transmitting and receiving

sides. Either LSB first or MSB first can be selected for transmitted/received data. An interrupt to the ARM is generated when 8-bit transmission (reception) is complete. In addition, the clock state when there is no transfer active can be selected. It supports both master mode as well as slave mode of operation

1.3.5 General Purpose Input Output Pins

DM270 has 34 general-purpose input/output pins (GIO) available. Using the GIO ports, the logic levels of the ports of an externally assigned device are read, and can be controlled. These pins are often multiplexed with other functions.

1.3.6 I²C Interface

DM270 includes a 2-line serial interface that can perform I²C bus protocol-compliant synchronous serial communication as bus master. This module has following features.

- Supports only single bus master
- Supports transmit, receive and mixed transfer using the slave address of 7bits or 10bits. (CBUS address is unavailable)
- Supports transfer rate of 100kbps or 400kbps.

1.3.7 MMC/SD Card Interface

The MMC/SD interface passes data between the ARM and the DMA controller on one side and a MMC/SD memory card or cards on the other side. The ARM or the DMA controller can read from or write to the registers in the MMC Controller. As necessary, the ARM and/or the DMA controller can store or retrieve data in memory.

The MMC Controller initiates transfers between itself and the memory card(s). The communication follows either the controller's native MMC protocol or a protocol that is based on the SPI standard. The native protocol can use one bi-directional data line (for MMC) or four parallel data lines (for SD cards). The SPI protocol uses two serial data lines: one for storing data to the card and one for retrieving data from the card.

The MMC Controller includes the following features:

- Support for a Multimedia Card (an MMC) or a Secure Digital Memory Card (an SD card)
- The capability to use either the MMC/SD protocol or the SPI protocol
- A programmable frequency for the clock that controls the timing of transfers between the MMC Controller and the memory card.
- Support for DMA transfer between MMC Controller and SDRAM

1.3.8 Memory Stick Interface

The DM270 supports Memory Stick communications; it contains a Memory Stick controller. With the Memory Stick controller, the DM270's ARM core can read from or write to a Memory Stick.

IMPORTANT: Customer needs to have a separate license from Sony for Memory Stick. DM270 Memory Stick Documentation can be provided only to customers who have Memory Stick license

1.3.9 External CPU interface

An external processor can gain access to DM270 internal ARM/DSP memory, SDRAM by setting the EXTHOST pin to high. In this mode, the EMIF bus is used to connect the external HOST with the DM270. DM270 ARM7, DSP can still operate as usual in this mode. Dynamic WAIT control can be performed by the interface with the external HOST controller.

1.3.10 ARM Boot-up Modes

DM270 provides three different ARM boot-up sequences. There are

- Self boot-up mode,
- Serial boot-up mode and
- External HOST boot-up mode

Serial boot-up mode and external HOST boot-up mode makes it possible to download an ARM boot code externally.

1.4 Features of the DSP subsystem Block

1.4.1 16-bit Fixed Point Digital Signal Processor (c5409)

The DSP subsystem is constructed around the TMS320C5409 DSP core from Texas Instruments catalog DSP. The DSP subsystem includes 32Kwords of Dual-Port SRAM for both program and data, 16Kwords of SARAM for data and 16Kwords of SARAM for program, 16Kwords of ROM, an imaging extension coprocessor (iMX), a Quantization/Inverse-Quantization (QIQ) coprocessor, and a variable length coding/decoding coprocessor (VLCD). The iMX accelerates many common image processing algorithms with regular processing – FIR filters, array scaling, array multiplication, etc. The iMX includes separate dedicated memories for program and coefficient data. The QIQ and VLCD coprocessor support the JPEG, MPEG1, H.263 and MPEG4 standards. In addition, the coprocessors share two 4Kword Image Buffers as operand storage, which supports flexible 1-d/2-d SDRAM DMA transfers.

The TI C54x core is a high-performance, low-power DSP core. The DSP software development tools for the C54x are very mature and fully compatible with the DM270 DSP subsystem.

To implement image processing, image compression, and voice/audio processing, the DSP controls SDRAM DMA transfers to/from external memory, and iMX/QIQ/VLCD accelerators.

The embedded DSP processor features are:

- o 16-bit fixed-point digital signal processor (based on the 'C5409)
- o Maximum operating frequency 100 MHz (refer to Data sheet for actual value)
- o 32K x 16-bit internal DARAM for program and data
- o 16K x 16-bit internal SARAM for data
- o 16K x 16-bit internal SARAM for program
- o 16K x 16-bit internal ROM
- o 2 Multi-Channel Buffered Serial Ports (McBSP)
- o Host Port Interface that allows ARM access to DSP internal RAM
- o JTAG-based emulation/debug

1.4.2 DSP Hardware Accelerators (iMX/VLCD)

The DSP Hardware Accelerators are part of the DSP subsystem and are designed to enhance the DSP's capability in image processing and other computation-intensive applications. All components reside in DSP's memory and/or I/O space and are controlled by the C5409 DSP.

The iMX is a parallel MAC engine with flexible control and memory interface for extending the image processing performance of TI's programmable DSPs. The iMX works well in a shared memory configuration with the DSP, so that regular block-based image processing tasks can be accelerated by iMX, and DSP can handle irregular processing as well as task-level control.

The co-processors can run at a maximum speed of about 180Mhz (Refer DM270 datasheet for exact values)

1.4.2.1 Imaging Extensions (iMX)

The iMX is a parallel MAC engine for expanding DSP image processing performance. It has a flexible control, a memory interface, and four MACs. The iMX is effective for fast computation of image processing in regular block units.

For digital cameras, the iMX can be used to accelerate:

- o 2D FIR filter
- o CFA interpolation filter
- o Color space conversion
- o Chroma down-sampling
- o Edge enhancement
- o False color suppression

- o DCT (discrete cosine transform) and IDCT
- o Table look-up
- o Motion estimation

This list is not exhaustive and includes many more functions.

1.4.2.2 Variable Length Coder / Decoder

Variable Length Coder / Decoder consists of Quantizer / Inverse Quantizer Engine and Variable Length Coder / Decoder Engine

The Quantizer / Inverse Quantizer (Q/IQ) module is a high-performance, intelligent and highly programmable hardware accelerator for the quantization and inverse quantization of DCT coefficients. The Q/IQ is highly autonomous and is capable of carrying out almost all the steps of quantization and inverse quantization process with little or no help from the DSP. Q/IQ supports the quantization and inverse quantization of the following standards:

- o JPEG
- o H.263
- o MPEG 1
- o MPEG 4
- o MPEG2

The VLCD coprocessor handles Huffman encode and decode operations for image compression algorithms. It operates with Huffman tables preloaded by DSP, via shared memory blocks. The VLCD is highly autonomous and is capable of carrying out all the steps of Huffman coding and decoding process with little or no help from the DSP. VLCD supports the Huffman encode/decode for the following standards:

- o JPEG
- o H.263
- o MPEG 1
- o MPEG 4
- o MPEG2

1.5 Features of the Imaging Peripheral Block

1.5.1 CCD Controller

The CCD Controller captures and processes the physical data from the CCD sensor. Note, that the A/D converter and Timing Generator for the CCD sensor are external to the DM270. The CCD controller can provide synchronizing HD/VD signals to an external timing generator or can synchronize to externally generated HD/VD inputs from an external timing generator. The CCD Controller supports pixels clocks up to 40 MHz, imager sizes up to 4096x4096, and data widths from 8 to 14 bits. The CCD Controller can also implement the following common image processing tasks:

- o Optical black clamping
- o DC offset,
- o Median filtering,
- o White balance gain/offset,
- o A-law compression (10-bit to 8-bit)
- o Lowpass filtering
- o Downsampling/culling

The CCD Controller can also be used to receive 8/18-bit YcbCr 4:2:2 as input. It can also receive REC656 compatible 8/10-bit YcbCr 4:2:2 as input.

1.5.2 Preview Engine

The DM270 Preview Engine is a hardware video processor module which generates an image data with YCbCr-4:2:2 format from CCD raw data in either RGB or Complementary color space. It is designed to provide a high quality video image for both the LCD preview function in a digital camera and for the source images for the video recording functions. In addition, the Preview Engine can process data from SDRAM as well as the CCD Controller input and can seamlessly resize YUV 4:2:2 data stored in SDRAM. The Preview Engine supports the following image processing functions:

- o Noise filter
- o Digital gain
- o White balance
- o 2 Step Smoothing
- o Seamless horizontal and vertical down sampling (x1/64~x1)
- o Seamless horizontal and vertical zoom (x1~x4)*
- o RGB2RGB blend matrix
- o Gamma correction

- o Chroma offset & suppression
- o 1 shot preview
- o Resize only mode

1.5.3 On-Screen Display (OSD)

The On-Screen Display module of the DM270 manages the display and blending of multiple windows containing video and bitmap data. This module features the following:

- o Two Video windows (YUV 4:2:2)
- o Two bitmap windows, or 1 bitmap window and 1 attribute window
- o Blending of bitmap with video windows – 8 blending ratios
- o One rectangular cursor window
- o Background color of each window can be specified – background can be set to transparent
- o Video-Window0 and Video-Window1 are displayed by YCbCr and OSD-Window0, OSD-Window1, are displayed by bitmap
- o Bitmap windows can be 1,2,4, or 8 bit color depth
- o Bitmap Color Look Up Table (CLUT) in ROM or user defined table in RAM
- o Interlaced and non-interlaced modes are supported
- o Built in VGA to NTSC/PAL conversion circuit (supports non square pixel)

1.5.4 Hardware 3A Engine

DM270 provides real-time 3A (AE/AWB/AF) statistics function. ARM uses the statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated by using up to 192 windows. The windows are the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by using up to 36 windows. The position and the size of each window can be independently set by software.

1.5.5 Video Encoder Interface

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8-bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization.

1.6 Features of the Memory Traffic Controller Block

1.6.1 SDRAM Controller

The SDRAM controller block acts as the primary interface between SDRAM and all functional blocks such as the processors (ARM, DSP), CCD Controller, OSD and DMA controller. This block supports SDRAM timing of a maximum of 102 MHz, and provides continuous data access with low overhead.

- Supports 16/64/128/256M-bit SDRAM
- Supports both 32-bit SDRAM or 16-bit SDRAM
- Maximum 102 MHz operation (refer to the data sheet for actual value)
- Access in WORD, HALF WORD, or BYTE units (ARM only)
- Mode setting, power down and SELF refresh commands
- Programmable refresh interval
- Column Access Strobe (CAS) latency 2 or 3 can be selected
- SDRAM size of up to 64 Mbytes can be used
- DMA transfer with peripheral modules
- SDRAM access priority is user configurable

1.6.2 External Memory Interface

The External Memory Interface (EMIF) provides a flexible interface circuit to external Flash memory (CS0), CompactFlash (CS1) and SmartMedia (CS2) interfaces and 2 other user configurable memory mapped areas (CS3, CS4). Each area has a separate chip select output terminal and DMA support

- Data width of 8 or 16 bits can be selected for each CS regions (CS0-4), except CS2.
- Using BUS_REQ and BUS_ACK signals, handshakes of open request and acknowledgment of bus rights with the external controller is possible.
- External wait can be applied by using the EM_WAIT terminal when accessing CS3 and CS4.
- IDLE can be inserted in write after read operation, and in access to other CS regions.
- All buses are in hi-impedance during reset and when set in bus open register from ARM.
- In normal mode, after reset is canceled, ARM jumps to address 0x0 of CS0.
- 8-bit/16bit access CF/CF+ cards
- Dynamic bus sizing (CF/CF+ IO mode) by the IOIS16 terminal
- Support for DMA transfer to External Memory Area

1.7 DM270 System Data Flows

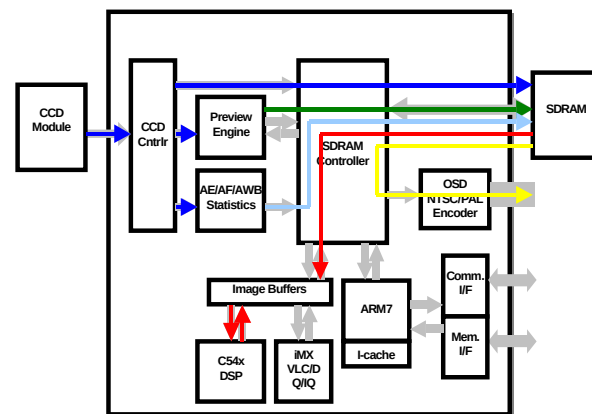
1.7.1 Preview Mode

In the preview mode, CCD or CMOS imager data comes in through the CCD controller to the preview engine. CCD controller has fault pixel correction, optical black clamping, and required timing control and interface. Preview engine contains a simplified image processing pipeline for AE/AWB, CFA interpolation, down sampling, and gamma lookups. The CCD/CMOS sensor should be configured to draft mode output, and preview engine handles further down-sampling, if necessary, to produce LCD/video resolution. Preview output is written to SDRAM.

DM270 has hardwired 3A (AF/AE/AWB) calculator. Each result is stored in SDRAM, ARM can read its data and control AFE.

The display data is read out by the OSD module, and merged/blended with the OSD data. The display data is sent to video encoder (VENC) or LCD controller, which sends composite analog or digital LCD data to the external display device.

Figure 3. Preview Mode Data Flow



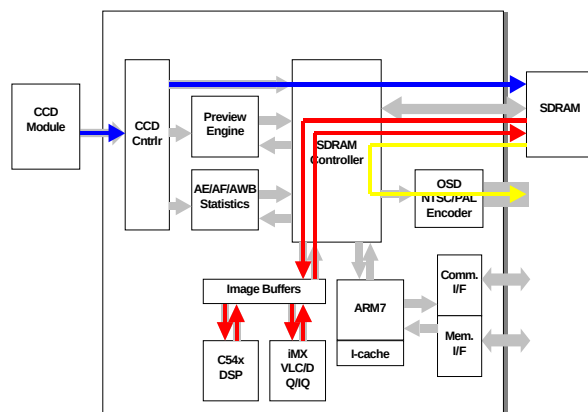
1.7.2 Still Picture Capture Mode

In the still picture capture mode, CCD or CMOS imager data comes in through the CCD controller and SDRAM controller directly to SDRAM first.

When there is a sufficient amount of data to start processing, which could be 20-24 lines with non-interlaced data and slightly over half-frame for interlaced data, ARM is notified via an interrupt signal from CCD controller. ARM then coordinates with iMX, VLCD, and DMA controller to transfer in the data from SDRAM into image buffer and process the data. The processing flow is software programmable and customer-specific. Typically compressed JPEG bitstream is the final output, and it's written back into SDRAM.

ARM then handles media storage card file system setup, sector header setup, and triggers DMA to write data from SDRAM into the card.

Figure 4. Still Picture Capture mode Data Flow



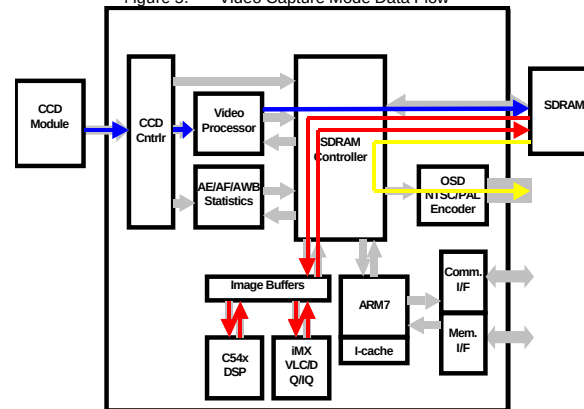
1.7.3 Video Capture Mode

Video capture is similar to still picture capture, except for the input stage. For video capture, we use Preview Engine to down sample the CCD/CMOS imager data to the video capture size (VGA, CIF, QCIF etc) before writing into SDRAM. This offloads much processing in the programmable engines and reduces power consumption due to processing and traffic.

Once sufficient amount of the video data is written into SDRAM, again ARM gets an interrupt from CCD controller and triggers the processing steps. MPEG-1 or MPEG-4 compression is performed with the coordination of ARM, iMX, VLCD and the DMA controller. The compressed MPEG bitstream is written back to SDRAM.

The video data produced by Preview Engine are also pulled out by OSD module to produce display data for composite analog video or for digital LCD interface.

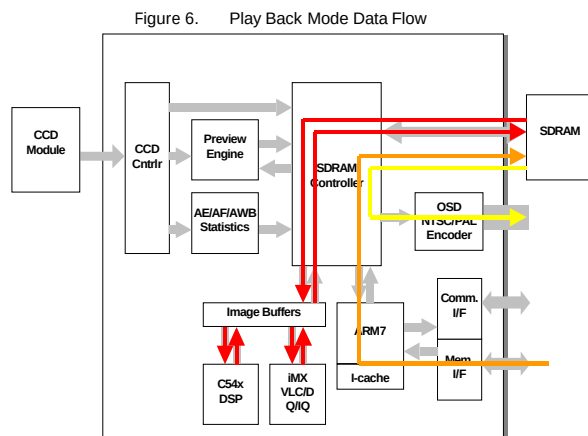
Figure 5. Video Capture Mode Data Flow



1.7.4 Still Picture and Video Playback

Still pictures or video clips can be played back on the digital camera. Typically the compressed data is first read out of the media card into SDRAM. Next the compressed data is processed and transported by ARM, coprocessors and DMA to uncompressed and, if necessary, resized into display resolution, and stored back into SDRAM.

The display data is then pulled out by OSD to supply VENC or LCD controller for analog composite video or digital LCD interface.



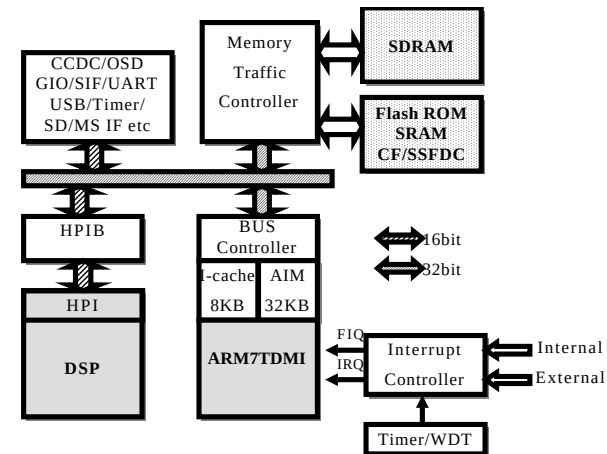
2 ARM Subsystem

2.1 Introduction

The ARM subsystem is made up of the ARM7TMDI RISC processor core, static RAM memory, and peripherals. The ARM controls the various peripheral modules including the timers, interrupt controller, I-cache, on-screen display, USB device, and serial ports.

In a typical system, the ARM is responsible for handling many system functions such as system-level initialization, configuration, user interface, user command execution, connectivity functions, and overall system control. The general purpose ARM handles these functions because it has a larger program memory space and better context switching capability, and is thus more suitable for complex, multi-tasking, and general purpose control tasks compared to the on-chip DSP.

Figure 7. ARM Subsystem



2.1.1 Features

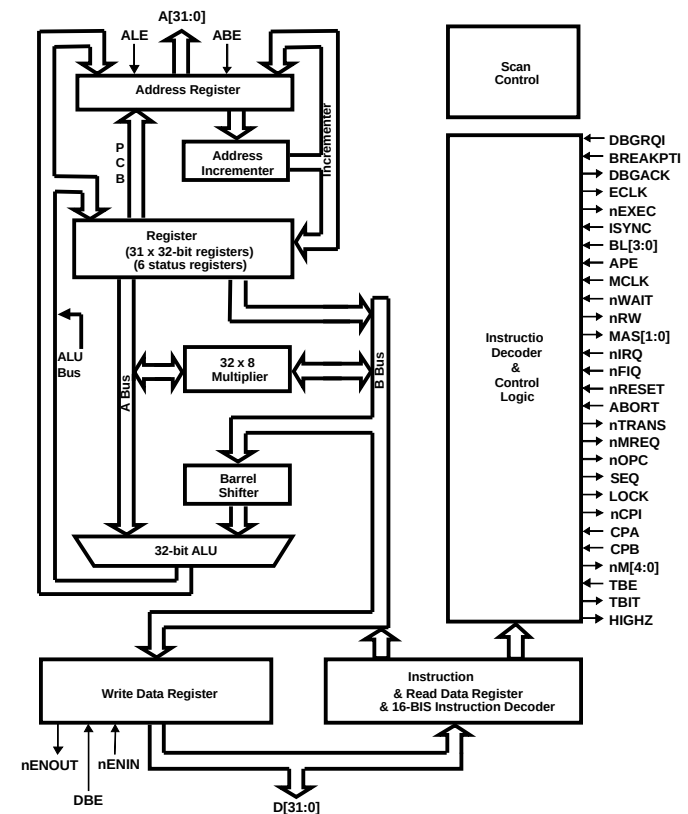
- 32-bit RISC processor
- Maximum operating frequency ~88 MHz (see datasheet for exact operating frequency)
- 32-Kbyte built-in RAM
- 8-K byte Instruction cache
- 32/16-bit command sets
- Memory format: fixed Little Endian Format
- 2 interrupt systems (FIQ, IRQ)
- 15 external terminal interrupt systems
- x4 hardware timers
- Watchdog timer
- JTAG base emulation
- External memory interface

2.2 ARM7TDMI RISC Processor Core

The ARM7TDMI core is integrated in the DM270 chip. The ARM7 core that you can find in the DM270 is based on the TMS470R1x microcontroller from Texas Instruments. The ARM7 core is specified to run at up to about 88 MHz (see datasheet for exact operating frequency). The ARM subsystem has its own 32 Kbytes of local static RAM (single cycle access).

The ARM7 processor employs a unique architectural strategy known as the 16-Bit Instruction Set (16-BIS), which makes it ideally suited to high-volume applications with memory restrictions, or applications where code density is an issue. A 32 bits instruction set (32-BIS) mode is also available as well to run complex algorithms.

Figure 8. ARM7 Core Block Diagram



2.2.1 The 16-BIS/32-BIS Concept

The key idea behind 16-BIS is that of a super-reduced instruction set. Essentially, the TMS470R1x processor has two instruction sets:

- ARM mode or 32-BIS: the standard 32-bit instruction set
- Thumb mode or 16-BIS: a 16-bit instruction set.

The 16-BIS 16-bit instruction length allows it to approach twice the density of standard 32-BIS code while retaining most of the 32-BIS's performance advantage over a traditional 16-bit processor using 16-bit registers. This is

possible because 16-BIS code operates on the same 32-bit register set as 32-BIS code. 16-bit code is able to provide up to 65% of the code size of the 32-bit code, and 160% of the performance of an equivalent 32-BIS processor connected to a 16-bit memory system.

2.2.1.1 16-BIS/32-BIS Advantages

16-bit instructions operate with the standard 32-bit register configuration, allowing excellent interoperability between 32-BIS and 16-BIS states. Each 16-bit instruction has a corresponding 32-bit instruction with the same effect on the processor model. The major advantage of a 32-bit architecture over a 16-bit architecture is its ability to manipulate 32-bit integers with single instructions, and to address a large address space efficiently. When processing 32-bit data, a 16-bit architecture will take at least two instructions to perform the same task as a single 32-bit instruction. However, not all the code in a program will process 32-bit data (for example, code that performs character string handling), and some instructions, like branches, do not process any data at all. If a 16-bit architecture only has 16-bit instructions, and a 32-bit architecture only has 32-bit instructions, then overall, the 16-bit architecture will have better code density, and better than one half the performance of the 32-bit architecture. Clearly 32-bit performance comes at the cost of code density. The 16-bit instruction breaks this constraint by implementing a 16-bit instruction length on a 32-bit architecture, making the processing of 32-bit data efficient with a compact instruction coding. This provides far better performance than a 16-bit architecture, with better code density than a 32-bit architecture. The 16-BIS also has a major advantage over other 32-bit architectures with 16-bit instructions. This is the ability to switch back to full 32-bit code and execute at full speed. Thus critical loops for applications such as

- o fast interrupts
- o DSP algorithms

can be coded using the full 32-BIS, and linked with 16-BIS code. The overhead of switching from 16-bit code to 32-bit code is folded into sub-routine entry time. Various portions of a system can be optimized for speed or for code density by switching between 16-BIS and 32-BIS execution as appropriate.

2.2.2 ARM7 Core Reference Documentation

For more information on the ARM7 core, refer to the user's guide of the TMS470R1x 32-bit RISC Micro-controller Family (SPNU134).

2.3 ARM Subsystem Bus Controller

2.3.1 Introduction

The ARM Bus Controller (BUSE) serves as the interface between the address/data bus of the ARM7TDMI core and the rest of the DM270 system. These Bus Controller functions are transparent to the user. However, there are some discrete registers in the Bus Controller that provide bit and byte reversal functions as well as a read-only register containing the DM270 Device Revision Number. This section gives information on these registers as well as describes the way the bus controller interfaces with peripherals and the external memory controller.

2.3.2 Features

- o The number of wait-state are inserted automatically according to the peripheral module accessed by ARM.
- o Internal RAM which can contain RTOS and the application program of ARM can be accessed by 32 bits data bus.
- o All registers of peripheral module are accessed with 16 bits data bus in no wait accessing except USB, SD/MMC IF, UART, MS and MG module.
- o Support the converting endian of data.
- o Alignment fault is not supported on addressing. Example when reading a 32-bit word the word address must be 32-bit aligned
- o Support three type of data access mode, e.g. a byte, a half-word and a word which is 8 bits, 16 bits and 32 bits respectively during data or instruction access to the external memories like SDRAM, Flash Memory and so on.
- o Outputs Chip Select (CS) for external memory area. The partition of the external memory area is controlled by Memory Traffic Controller Module (MTC). The MTC is suitable for various combination of external memory.
- o To reduce latency and power consumption while accessing program memory, memory bus is separated from peripheral bus.

2.3.3 Data access modes

The memory, which is addressed by BUSE module, can be accessed with a byte, a half-word and a word length.

The data access modes are shown in the following table.

Table 1. Data Access Mode

Data Mode	Bit Width
Byte	8
Half-word	16
Word	32

During a half-word or a word access to peripheral, the address from ARM has to be half-word or word aligned respectively. Hence, when ARM accesses memory

in a half-word or a word data mode, BUSC won't use address bits A [0] and A [1:0] respectively during address decoding.

2.3.4 Memory Map

NOTE: Refer section titled External HOST interface for memory map in external HOST interface mode

ARM can access all peripherals registers, memory area. The BUS Controller is responsible to automatically generate the number of wait states required, according to the peripheral module accessed by the ARM. The internal RAM (AIM) of the ARM can be access through a 32 bits width data bus. It is recommended to store the main RTOS within this memory to get the best throughput. All registers of peripheral modules, except for the USB/SD/MS/UART modules, are accessed through a 16 bits width data bus with no wait-state.

The BUSC module supports three type of data access mode, e.g. a byte, a half-word and a word which is 8 bits, 16 bits and 32 bits respectively during data or instruction access to the external memories like SDRAM, Flash Memory and so on. It supports dynamic allocation of external memory area. The maximum number of partitioning area is six. This feature is explained in the memory traffic controller/EMIF chapter.

In addition, DSP memory can be also accessed by ARM through HPI Bridge.

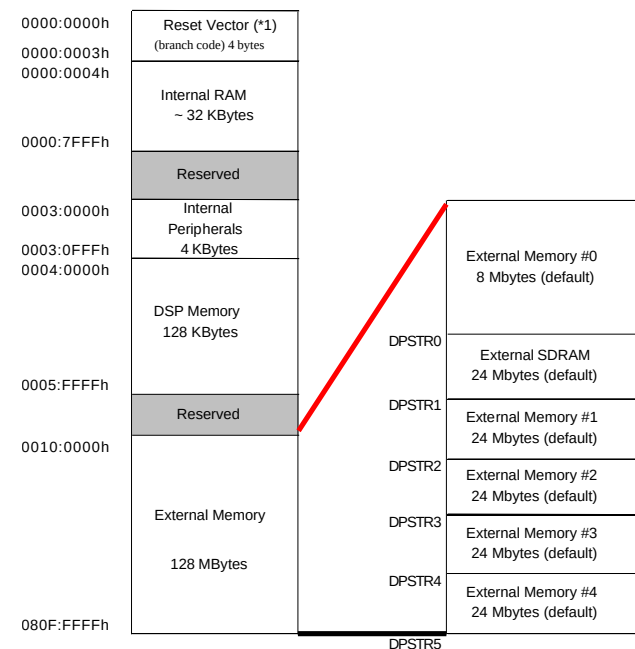
The characteristics of the different memory spaces accessible by the ARM are summarized in the Table 2. and Figure 9 below.

Table 2. ARM Memory Map

Area Name	Start address	End address	Size in bytes	Access Mode
Reset vector ROM (*1)	0x0000:0000	0x0000:0003	4	B/H/W
ARM Internal RAM	0x0000:0004	0x0000:7FFF	~32 K	B/H/W
Internal Peripheral	0x0003:0000	0x0003:0FFF	4 K	H
DSP Memory	0x0004:0000	0x0005:FFFF	128 K	H
External Memory	0x0010:0000	0x080F:FFFF	128 M	B/H/W

(*1): Reset vector ROM is used when BTSEL pin is set '0'. In this case, the first 4 byte of RAM block 0 (0000:0000h-0000:0004h) is not used. On the other hand, When BTSEL = '1', the area of Reset vector ROM is replaced by the first 4 byte of the RAM.

Figure 9. ARM Memory Map



(*1): Reset vector ROM is used when BTSEL pin is set '0'. In this case, the first 4 byte of RAM block 0 (0000:0000h-0000:0004h) is not used. On the other hand, When BTSEL = '1', the area of Reset vector ROM is replaced by the first 4 byte of the RAM.

2.3.4.1 Reset Vector ROM

In when BTSEL=0, reset Vector ROM (RVR), which is defined at 0x0000:0000 of ARM internal memory space, is hard-coded in BUSC. ARM accesses this area to fetch instruction after power on reset. BUSC will provide ARM with the following code. The code means that jump to the address 0x0010:0000.

```
0000:0000    EA 03 FF FE    ; RVR: B    00100000h
```

After the reset signal is cancelled, DM270 accesses external memory address 0 (CS0). Therefore the program must be written here in advance. (See chapter "ARM Boot-up Modes" for boot-up mode involving BTSEL=H)

2.3.4.2 ARM Internal RAM

This is ARM internal memory (IRAM). Since ARM can access this memory region in 1 cycle (no wait state), the RTOS (real time OS) or application programs can be operated more quickly here. Contents of this memory region can be access as byte (8 bit), half word (16 bit) or word (32 bit).

2.3.4.3 ARM Peripheral Registers

The ARM controls the various peripheral modules including the timers, interrupt controller, I-cache, on-screen display, USB device, and serial ports. BUSC can access all internal peripheral registers with 16 bits data width. A read and a write access can be achieved within one cycle of ARM clock except for USB/SD/MS/UART register access. The number of access cycles for USB/SD/MS/UART register access varies depending on the internal state, and several cycles are required. Details about each peripheral listed in the below figure are given in subsequent chapters

Figure 10. ARM Peripheral Memory Map

Peripheral Name	Start address	End address	Access Cycle Number
Timer 0	0x0003:0000	0x0003:007F	1
Timer 1	0x0003:0080	0x0003:00FF	1
Timer 2	0x0003:0100	0x0003:017F	1
Timer 3	0x0003:0180	0x0003:01FF	1
Serial 0	0x0003:0200	0x0003:027F	1
Serial 1	0x0003:0280	0x0003:02FF	1
UART 0	0x0003:0300	0x0003:037F	2+n ^(*)
UART 1	0x0003:0380	0x0003:03FF	2+n ^(*)
WDT	0x0003:0400	0x0003:047F	1
SD/MMC IF	0x0003:0480	0x0003:04FF	1+n ^(*)
Interrupt Controller	0x0003:0500	0x0003:057F	1
GIO	0x0003:0580	0x0003:05FF	1
DSP Controller	0x0003:0600	0x0003:067F	1
OSD	0x0003:0680	0x0003:06FF	1
CCD Controller	0x0003:0700	0x0003:077F	1
Preview	0x0003:0780	0x0003:07FF	1
NTSC/PAL Encoder	0x0003:0800	0x0003:087F	1
Clock Controller	0x0003:0880	0x0003:08FF	1
Bus Controller	0x0003:0900	0x0003:097F	1
SDRAM Controller	0x0003:0980	0x0003:09FF	1
External Memory I/F (EM IF,CFC/SSF IF)	0x0003:0A00	0x0003:0A7F	1
USB0	0x0003:0A80	0x0003:0AFF	2+n ^(*)
USB1	0x0003:0B00	0x0003:0B7F	2+n ^(*)
3A	0x0003:0B80	0x0003:0BFF	1
Memory Stick1	0x0003:0C00	0x0003:0C7F	2
Memory Stick2	0x0003:0C80	0x0003:0CFF	1+n ^(*)
ATM	0x0003:0D00	0x0003:0D7F	1
I2C	0x0003:0D80	0x0003:0DFF	1
ICACHE	0x0003:0E00	0x0003:0E7F	1

(*) n: This is variable, depending on access condition

ARM Peripherals can be grouped according to their functionality as follows:

- o ARM Internal Peripherals
 - n Timer 0-3
 - n Watch Dog Timer
 - n Interrupt Controller
 - n Clock Controller
 - n Bus Controller
 - n Address Trace Module
 - n I-cache
- o Imaging Peripherals
 - n OSD
 - n CCD Controller
 - n Preview Engine
 - n Video Encoder
 - n Hardware 3A
- o I/O peripherals
 - n Serial Port 0, 1
 - n UART 0, 1
 - n MMC/SD
 - n GIO
 - n USB
 - n Memory Stick
 - n I2C
- o Memory traffic controller
 - n SDRAM Controller
 - n External Memory Interface (EMIF)

2.3.4.4 DSP Memory

When the ARM accesses the DSP internal memory through the HPI bridge module (HPIB), one wait-state is inserted automatically. In addition to it, extra wait cycles will be inserted depending on the wait signal that is generated in HPIB module. ARM has no access to the image buffer memories.

The DSP internal memory (32-Kwords of DARAM,32-Kwords of SARAM) can be read and written by the ARM or an external host through the HPI-Bridge.

Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0004:0000-0005:FFFF in the ARM memory map.

Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0002:0000-0003:FFFF in the external host memory map.

The HPIB memory maps of the ARM and external HOST are shown in Table 3. and Table 4. , respectively. When the endianness of the ARM and DSP do not match, EXCHG in HPIBCTL (0x30600) can be used to swap the upper and lower 8-bits of data transferred between the ARM and DSP.

Table 3. ARM HPIB Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x40000-0x4FFFF	0x0-0x7FFF	Data & Program(Page0)	64k
0x50000-0x57FFF	0x8000-0xBFFF	Data	32k
0x58000-0x5FFFF	0xC000-0xFFFF	Program(Page1)	32k

Table 4. External HOST HPIB Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x20000-0x2FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x30000-0x37FFF	0x8000-0xBFFF	Data	32k
0x38000-0x3FFFF	0xC000-0xFFFF	Program(Page1)	32k

2.3.4.5 External Memory

As external memory interfaces, DM270 has an SDRAM interface and a memory card or general-purpose memory interface. ARM can access SDRAM via the SDRAM controller module. Also, it can access memory card or flash ROM via the external memory interface module. In access from ARM to external memory, a wait state is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The SDRAM memory access is controlled by the SDRAM controller. When ARM accesses SDRAM memory an asynchronous transfer is achieved between SDRAMC and BUSC.

The dynamically allocated external memory, which is directly accessible by the ARM, is a maximum of 128 Mbytes, and can be divided into a maximum of 6 regions (SDRAM, and CS0 to CS4). By changing the memory map dynamically, external memory of a different capacity can be used efficiently. Each of the 6 regions can be used by setting a boundary address. Byte (8-bit), half-word (16-bit) and word (32-bit) types of accesses are possible.

2.3.5 Endian Conversion

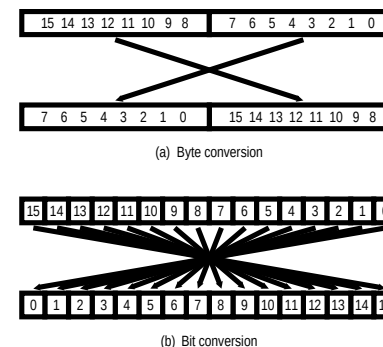
The endian can be converted by a use of Endian Convert Register ([ECR](#); 0x30900).

Writing data to be converted into ECR register once allows user to obtain either data which is converted by byte unit and by bit unit.

The converted data are stored to Endian Byte Converted Register ([EBYTER](#); 0x30902) and Endian Bit Converted Register ([EBITR](#); 0x30904).

The following figure shows this endian conversion.

Figure 11. Endian Conversion



2.3.6 Revision code Register

User can read the ES revision code from [REVR](#) register (0x30906). For example, if ES revision is ES1.2, 0x0012 can be read from REVR register.

2.3.7 Bus Controller Register Map (BUSE)

Address	Register	Description
0003:0900	ECR	Endian Conversion Register
0003:0902	EBYTER	Endian Byte Reverse Register
0003:0904	EBITR	Endian Bit Reverse Register
0003:0906	RCVR	Device Revision Register

2.3.8 Bus Controller Registers

2.3.8.1 ECR - BUSE Endian Conversion Register

ECR		0003:0900				offset: 0x00				default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	ECR[15]	ECR[14]	ECR[13]	ECR[12]	ECR[11]	ECR[10]	ECR[9]	ECR[8]	ECR[7]	ECR[6]	ECR[5]	ECR[4]	ECR[3]	ECR[2]	ECR[1]	ECR[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15:0	ECR			Endian conversion register. The endianness of data written to this register is converted and stored in the EBYTER and EBITR registers

2.3.8.2 EBYTER - BUSE Endian Byte Reverse Register

EBYTER		0003:0902				offset: 0x02				default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	EBYT[15]	EBYT[14]	EBYT[13]	EBYT[12]	EBYT[11]	EBYT[10]	EBYT[9]	EBYT[8]	EBYT[7]	EBYT[6]	EBYT[5]	EBYT[4]	EBYT[3]	EBYT[2]	EBYT[1]	EBYT[0]			
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	EBYT	0	R	Endian Byte Conversion register. Contains byte-swapped contents of data written to ECR register

2.3.8.3 EBITR - BUSE Endian Bit Reverse Register

EBITR		0003:0904				offset: 0x04				default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	EBIT[15]	EBIT[14]	EBIT[13]	EBIT[12]	EBIT[11]	EBIT[10]	EBIT[9]	EBIT[8]	EBIT[7]	EBIT[6]	EBIT[5]	EBIT[4]	EBIT[3]	EBIT[2]	EBIT[1]	EBIT[0]			
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	EBIT	0	R	Endian Bit Conversion register. Contains bit-swapped contents of data written to ECR register

2.3.8.4 REVR - BUSC Device Revision Register

REVR	0003:0906				offset: 0x06				default: 0x0010							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REV[7]	REV[6]	REV[5]	REV[4]	REV[3]	REV[2]	REV[1]	REV[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	REV	16	R	Reset value » 00010000: (0x10) ES1.0 « » 00010001: (0x11) ES1.1 «

3 Instruction Cache

DM270 has 8k-byte instruction cache (I-cache). I-cache copies instructions, that are to be executed into internal I-cache RAM and uses this instruction when it is to be used again. I-cache increases instruction fetch performance. I-cache can cache instructions stored in external memory area, which include SRAM, FLASH, and SDRAM.

3.1 Features

I-cache in DM270 contains 8KB RAM for caching instruction. I-cache selects only instructions mapped into external memory (0x100000-0x80FFFF). Given below are I-cache features.

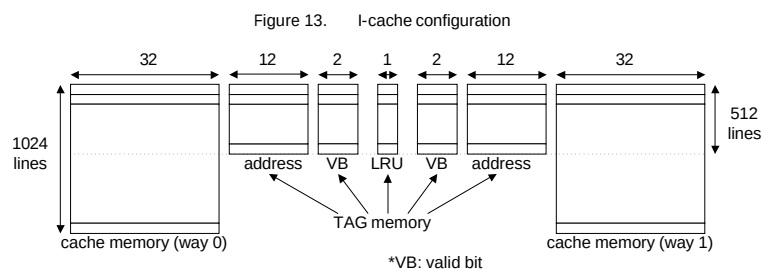
- o 4-word (128bit) line size
- o Total 512-line (256-line x 2way)
- o Cache data update by LRU (Least Recently Used) algorithm
- o Check tag-RAM according to address from ARM and execute action in one wait cycle.
- o When HIT, ARM reads the data stored in RAM in I-cache
- o When MISS, external memory access starts.
- o No Dirty bit in tag-RAM (no support write)
- o Support I-cache disable(bypass) mode
- o Selectable 1word/4word access (4word burst is supported only SDRAM access)
- o Support cache clear function

3.2 I-cache Configuration

I-cache consists of cache memory of 1K lines x 32-bit x 2 way (= 8Kbyte) and TAG memory, which is 512 lines x 45-bit. Cache memory includes copied instruction code. One line of TAG memory has two sets of address information, valid bits and one LRU bit. Figure 12 shows one line of TAG memory configuration and Figure 13 shows I-cache configuration of DM270.

Figure 12. TAG Memory Entry

44	43-24	23-22	21-2	1:0
LRU	way0 address[31:12]	valid bit[1:0]	way1 address[31:12]	valid bit[1:0]



Internal 32-bit ARM address (absolute address, internal ARM address[31:0]) is used for cache operation. Internal ARM address[11:3] is used for selection of TAG memory. For example, when ARM issues address 0x00000000, first line of TAG memory is selected. When ARM issues address 0x00000008, second line is chosen.

Each line of TAG memory is used for two lines (even and odd lines) of cache memory. To determine even or odd, internal ARM address[2] is used. Two sets of "Valid Bits", VBs, are present in one line of TAG memory. Each VB is for each way of cache memory. VB indicates that even or odd line of instruction codes in cache memory is valid or not. For example, when VB = "01", even line of instruction is valid. When VB="11", even and odd line are valid and so on.

Least recent used (LRU) bit is used to select way of cache memory. LRU indicates last used way of cache memory.

3.3 Cache Operation Example

Below is example of I-cache operation.

- ARM fetches code A from address 100000h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	x	0	0100f	1b	0b	xxb	xxxxxxh
1	x	x	1	xxxxxxh	xxb	xb	xxb	xxxxxxh
2	x	x	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code B from address 100004h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	x	0	00100h	1b	0b	xxb	xxxxxxh
1	B	x	1	xxxxxxh	xxb	xb	xxb	xxxxxxh
2	x	x	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code C from address 100008h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	x	0	00100h	11b	0b	xxb	xxxxxxh
1	B	x	1	0100f	1b	0b	xxb	xxxxxxh
2	C	x	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code D from address 101000h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	D	0	00100h	11b	1b	1b	0101f
1	B	x	1	00100h	01b	0b	xxb	xxxxxxh
2	C	x	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code E from address 101004h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	D	0	00100h	11b	1b	1b	00101h
1	B	E	1	00100h	01b	0b	xxb	xxxxxxh
2	C	x	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code F from address 101008h

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	A	D	0	00100h	11b	1b	11b	00101h
1	B	E	1	00100h	01b	1b	1b	0101f
2	C	F	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxxh	xxb	xb	xxb	xxxxxxh

- ARM fetches code G from address 102000h (replace occurs)

#	way0 Memory	way1 Memory	#	TAG	VB	LRU	VB	TAG
0	G	D	0	0102f	1b	1b	11b	00101h
1	B	E	1	00100h	01b	1b	01b	00101h
2	C	F	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:
1023	x	x	511	xxxxxh	xxb	xb	xxb	xxxxxh

- ARM fetches code D from address 101000h. This is "cache hit". The instruction code D is fetched from way1 cache memory.

3.4 I-cache Control

After DM270 power-on reset I-cache is disabled. Writing '1' to ICAENB in register ICAMODE – Instruction Cache Mode Register, enable I-cache. ICACLR is used to clear I-cache. After reset, cache clear is required to ensure proper operation. REQ4W bit enables 4 words burst access (when in ARM mode) for instruction code in SDRAM. Do not change REG4W bit when I-cache is enabled.

To ensure proper I-cache operation, below sequence is required when ICACHE is enabled.

- Disable cache (writes "0" to ICAENB)
- Clear cache (writes "1" to ICACLR)
- Wait cache clear completion (wait until ICACLR is "0")
- Enable cache (writes "1" to ICAENB)
- If 4 word burst is selected, REG4W bit and ICAENB bit should be set to "1" simultaneously.

3.5 I-cache Operation Notes

Given below are operation notes when I-cache is used.

- I-cache is can cache instructions from external memory from address 0x100000 to 0x80FFFF. (SDR_CS, FLSH_CE, EM_CE0, EM_CE1 area)
- When ARM is in thumb mode (16-BIS), 32-bit access occur to fill 32-bit cache line.

- When internal ARM writes instruction to memory area, I-cache contents of written area will be invalid.
- If another master device (e.g. external host) updates cache area, I-cache must be cleared to keep cache coherency.
- Do not re-enable when I-cache is enabled. If re-enable is required, I-cache should be disabled and cleared first.
- Do not change REQ4W bit when I-cache is enabled. If REQ4W bit change is required, I-cache should be disabled and cleared first.
- Breakpoint can be used when I-cache is enabled as well as when I-cache is disabled.
- When I-cache hit occurs, instruction execution happens with one wait state operation.

3.6 Instruction Cache Register Map (ICACHE)

Address	Register	Description
0003:0E00	ICAMODE	I-cache Mode Register

3.7 Instruction Cache Registers

3.7.1 ICAMODE – Instruction Cache Mode Register

ICAMODE	0003:0E00												offset:	0x00				default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REQ4W	RSV	RSV	RSV	RSV	RSV	RSV	ICACLR	ICAENB						
R/W	-	-	-	-	-	-	-	W	-	-	-	-	-	-	R/W	R/W						
Default	-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	-						

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	REQ4W	0	W	Enable 4 burst instruction read from SDRAM » 1: 4 word burst access request « » 0: Single word access request «
7:2	RSV	0	R/W	Reserved
1	ICACLR			Cache Clear When write '1' to this bit, ICAENB MUST be '0'. » 1: ICACHE Clear start « » 0: ICACHE Clear done «
0	ICAENB			Cache Enable » 1: ICACHE is enable « » 0: ICACHE is disable (bypass) «

§ When ICACLR bit is set to 1, clearing of cache starts. When this bit is read, it will return 1 while the clearing operation is in progress and 0 after clearing is complete. ICAENB must be 0 when writing 0 to ICACLR.

- After reset, ensure that the cache is cleared by writing 1 to ICACLR for initialization of TAG RAM.

§ When the program located at external memory is updated, the cache should be cleared by writing 1 to ICACLR.

§ REQ4W enables burst transfer (4 burst) from SDRAM to ICACHE, when the program is located in SDRAM.

§ (When 32-bit SDRAM is used) REQ4W can be set only when ICAENB is 1. (REQ4W and ICAENB can be set to 1 at same time.)

4 Clock Controller

Please also refer to the latest DM270 datasheet for supplementary PLL, clock controller related information like maximum ARM, DSP, AXL, SDRAM frequencies.

4.1 Introduction

The Clock Controller module generates multiple clock frequencies from a single crystal input. This module generates an internal interim frequency as a multiplier of the input crystal, then divides this frequency down to obtain the necessary clocks to drive the ARM7, IMX, Coprocessors, USB and SDRAM. The input frequency is user definable within the allowable range of 20-35MHz, however under normal conditions one would need to be 27MHz to allow for accurate NTSC/PAL video output. The internal interim frequency is the direct output from the PLL and can be divided up to 4 (x1,1/2,1/3,1/4) of the PLL output frequency. Up to four kind of output frequencies can be generated from this interim internal frequency.

As shown in Figure 14, the CLKC has two analog PLLs built into it, and you can freely set the frequencies of the ARM clock, SDRAM clock, Coprocessor (IMX, VLCD, Image Buffers) and DSP clock using these PLLs. To reduce system power, each clock frequency can be slowed down and each clock can be disabled.

The Clock Controller has the following external inputs:

- o PCLK: External pixel clock input
- o SYSCLK: External system clock input(optional)
- o MXI: Externally attached crystal oscillator input
- o M48XI: Externally attached 48MHz crystal oscillator input
- o TEST2: ARM Clock Selection

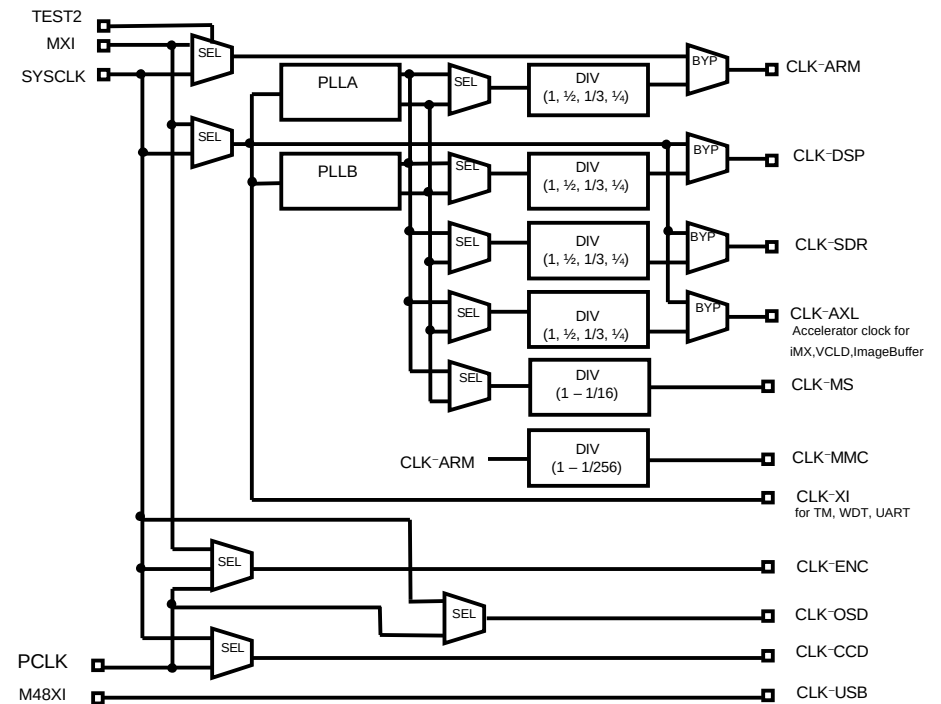
In normal operation, a 27MHz signal is applied to MXI and a 48MHz signal is applied to M48XI. The CCD Controller (CCDC) pixel clock is connected to PCLK. The maximum pixel clock frequency is 40MHz, since the CCDC operates at a maximum of 40MHz (refer to the section titled "CCD Controller" for details). The DM270 passes-through MXI and M48XI to output signals MXO and M48XO, respectively.

The Clock Controller generates the following 11 signals:

- CLK_ARM: ARM clock for ARM core and its related modules
- CLK_DSP: DSP clock for DSP core and its related modules
- CLK_AXL: Accelerator clock for iMX, VLCD and Image Buffers
- CLK_SDR: SDR clock for external SDRAM and its related modules
- CLK_XI: X'tal clock such as TMR, WDT, UART
- CLK_CCD: CCD Controller clock
- CLK_ENC: Video Encoder clock
- CLK_OSD: OSD clock
- CLK_USB: USB clock
- CLK_MMC: Multimedia Card / Secure Digital clock
- CLK_MS: Memory Stick clock

As shown in Figure 14, the input signal TEST2 is used to select which clock will be used as the ARM clock prior to power-on and reset. By default, the ARM PLL is bypassed during power-on and reset, and either MXI or SYSCLK are used as the ARM clock. When the state of TEST2 is low, MXI is used and when the state of TEST2 is high, SYSCLK is used. After the ARM is rebooted, the ARM can configure several clocks by setting various registers in the Clock controller module.

Figure 14. Clock Controller Block Diagram



4.2 Input Clock Selection

As shown in Figure 14 there are several selectors, which can be used to configure the clock controller. Based on these selectors input clock source to different modules can be selected. Input clock source can be selected on basis of signal level on TEST2 pin or via register CLKC - CLKC Input clock Source Selection, register SEL - CLKC Input PLL Source Selection. The input clock source and their relationship with different modules is shown in Table 5.

Table 5. Input Clock Selection

Module	Clock Selection
ARM clock selection during power-on and reset	The ARM PLL is bypassed during power-on and reset, and either MXI or SYSCLK are used as the ARM clock. When the state of TEST2 is low, MXI is used and when the state of TEST2 is high, SYSCLK is used. After the ARM is rebooted, the ARM can configure several clocks by setting various registers in the Clock controller module.
PLL input clock	Input clock to PLL can be either MXI or SYSCLK depending on the state of TEST2 and register CLKC - CLKC Input clock Source Selection. By default when the state of TEST2 is low, MXI is used and when the state of TEST2 is high, SYSCLK is used.
Video Encoder clock	Input clock to video encoder module can be either MXI(default) or SYSCLK or PCLK. This is selected via register CLKC - CLKC Input clock Source Selection
OSD clock	Input clock to OSD module can be either CLK_ENC/2(default) or CLK_CCD. This is selected via register CLKC - CLKC Input clock Source Selection
CCD Clock	Input Clock to CCD can be either PCLK(default) or SYSCLK. This is selected via register CLKC - CLKC Input clock Source Selection

Module	Clock Selection
ARM Clock (after power-on reset)	After power-on reset input clock to ARM can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL - CLKC Input PLL Source Selection. PLL can be bypassed using register BYP - CLKC PLL Bypass Configuration
DSP Clock	Input clock to DSP can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL - CLKC Input PLL Source Selection. PLL can be bypassed using register BYP - CLKC PLL Bypass Configuration
SDRAM Clock	Input clock to SDRAM can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL - CLKC Input PLL Source Selection. PLL can be bypassed using register BYP - CLKC PLL Bypass Configuration
AXL Clock	Input clock to AXL (iMX, VLCD, Image Buffers) can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL - CLKC Input PLL Source Selection. PLL can be bypassed using register BYP - CLKC PLL Bypass Configuration
UART, Timer Clock	Input clock to UART0-1, Timer 0-3 can be selected independently as either PLL input clock or ARM clock output(default)

4.3 Setting PLL Clock Output Frequency

The ARM, DSP, AXL and SDRAM clock frequencies are generated by one of two PLLs. The output frequency of each of the two PLL's can be set separately. Figure 15 shows the PLL structure. The PLL is a built-in analog PLL constructed of a charge pump, loop filter and VCO.

The ARM, DSP, AXL and SDRAM clock frequencies are setup by configuring, selecting the PLLs, and the divider connected to the input of ARM, DSP, AXL, SDRAM module.

The PLL output frequency is determined by setting the division ratio of the counter N, which divides the input clock, and the counter M, which divides the PLL feedback output. The PLL output is divided by the value of the divider. The PLL output frequency is given by the following formula:

$$PLL_{out} = [Fin * (M/N)]$$

Where N = 1...4 and M = 1...31

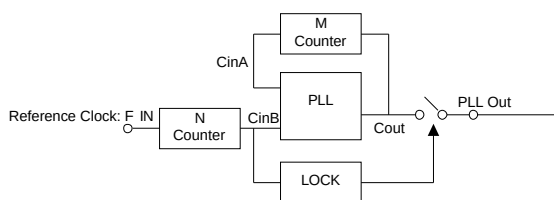
Fin is the PLL input clock signal (see Table 5. Input Clock Selection). M and N can be set in via the register PLLA - CLKC PLLA Configuration and register PLLB - CLKC PLLB Configuration

The PLL output can be used as input to ARM, DSP, AXL, SDRAM module. The divider, DIV, allows you to further divide this PLL output clock without modifying the PLL configuration. This is advantageous because, modifying M and/or N forces the clock to switch to FIN while the PLL locks. The divider can be set separately for ARM, DSP, AXL, SDRAM via register DIV - CLKC PLL Clock Divisor Settings

When the values of M and N are changed, the PLL is in an unstable state until the PLL output is stable (locked state). During this time, the clock is switched from the PLL output clock to FIN so that erroneous operation while PLL is unstable does not occur. The PLL lock in period is 500µsec.

IMPORTANT: The user must wait for 500µsec after setting PLL registers in order for the PLL to lock (see section "OSC Interface and Clock Timing" in DM270 datasheet)

Figure 15. PLL Structure



4.4 PLL Bypass Mode

It is possible to bypass PLL for ARM, DSP, AXL, SDRAM clock by using register BYP - CLKC PLL Bypass Configuration

4.5 Module Clock distribution

Clock supply to individual clock module can be turned ON or OFF using registers MOD0 - CLKC Module Clock Enables #0, register MOD1 - CLKC Module Clock Enables #1 and register MOD2 - CLKC Module Clock Enables #2

Note, image buffer, iMX, VLCD clocks can also be enabled by DSP using I/O port 0xFFFF. These clocks are enabled when either corresponding bits in register MOD1 - CLKC Module Clock Enables #1 are set to '1' **OR** when corresponding bits in I/O port 0xFFFF are set to '1'

4.6 GIO Clock Output

GIO18, GIO17 and GIO16 can be used as clock output signals. Clock source for these clock outputs can be selected separately. ARM, DSP, AXL, PLL input, M48XI clock output can be used as clock source for these GIO clock output signals. Before outputting the clock a divisor can be applied to divide the input clock.

When using GIO18, GIO17, GIO16 as clock output signal, the corresponding GIO must be set as output and dedicated function mode must be selected (GIO register DIR0-1, FSEL0-1). Input clock source is selected using register OSEL - CLKC Output Clock Selector. Divisor value is selected using register O0DIV - CLKC Output Clock #0 Divider, register O1DIV - CLKC Output Clock #1 Divider and register O2DIV - CLKC Output Clock #2 Divider

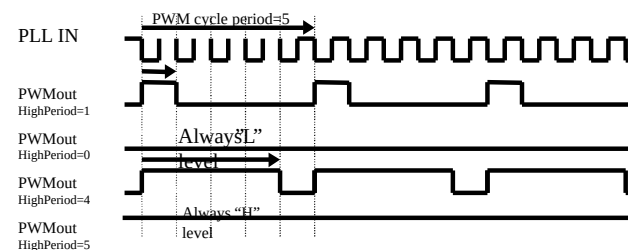
4.7 Pulse Width Modulation

GIO29, GIO30 can be used as pulse width modulation (PWM) signals. PLL input clock is used as reference clock for PWM. PWM cycle width, high level period can be set separately for GIO29, GIO30.

When using GIO29, GIO30 as PWM signal, the corresponding GIO must be set as output and dedicated function mode must be selected (GIO register DIR0-1, FSEL0-1). PWM cycle width is selected using register PWM0C - CLKC PWM #0 Cycle Count for GIO29 and register PWM1C - CLKC PWM #1 Cycle Count for GIO30. PWM high level period is selected using register PWM0H - CLKC PWM #0 High Period for GIO29 and register PWM1H - CLKC PWM #1 High Period for GIO30

Figure 16 shows the relationship between PWM cycle and PWM high level signal

Figure 16. Pulse Width Modulation



4.8 Clock Controller Register Map (CLKC)

Address	Register	Description
0003:0880	PLLA	PLLA Configuration
0003:0882	PLLB	PLLB Configuration
0003:0884	CLKC	Input clock Source Selection
0003:0886	SEL	Input clock Source Selection
0003:0888	DIV	PLL Clock Divisor Settings
0003:088A	BYP	PLL Bypass Configuration
0003:088C	MMCLK	MMC Clock Divisor Settings
0003:088E	CTEST	Test Register. Do not use.
0003:0890	MOD0	Module Clock Enables #0
0003:0892	MOD1	Module Clock Enables #1
0003:0894	MOD2	Module Clock Enables #2
0003:0896	LPCTL0	Low Power Control #0
0003:0898	LPCTL1	Low Power Control #1
0003:089A	OSEL	Output Clock Selector
0003:089C	O0DIV	Output Clock #0 Divider
0003:089E	O1DIV	Output Clock #1 Divider
0003:08A0	O2DIV	Output Clock #2 Divider
0003:08A2	PWMOC	PWM #0 Cycle Count
0003:08A4	PWMOH	PWM #0 High Period
0003:08A6	PWM1C	PWM #1 Cycle Count
0003:08A8	PWM1H	PWM #1 High Period

4.9 Clock Controller Registers

4.9.1 PLLA - CLKC PLLA Configuration

PLLA		0003:0880										offset: 0x00		default: 0x0080					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	TEST0	TEST1	PWRDN	RSV	TEST2[2]	TEST2[1]	TEST2[0]	PMA[3]	PMA[2]	PMA[1]	PMA[0]	RSV	PNA[2]	PNA[1]	PNA[0]			
R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W			
Default	-	0	0	0	0	-	0	0	0	1	0	0	0	-	0	0			

Bit	Name	Reset Value	R/W	Function
15	RSV			
14	TEST0	0	R/W	Test bit for PLLA, set to 0
13	TEST1	0	R/W	Bypass Test bit for PLLA, set to 0
12	PWRDN	0	R/W	Powerdown for PLLA » 0: normal operation « » 1: powerdown «
11	RSV			
10:8	TEST2	0	R/W	Test bits for PLLA, set to 000
				PLLA Numerator
				M=PMA+1 N=PNA+1
				PLLA = Fin*M/N
7:4	PMA	8	R/W	Note: In ES1.0, PLL output clock frequency 200 MHz < Fin*M/N < 351 MHz In ES1.1 and higher, PLL output clock frequency 110 MHz < Fin*M/N < 351 MHz
3	RSV			
				PLLA Denominator
				M=PMA+1 N=PNA+1
				PLLA = Fin*M/N
2:0	PNA	0	R/W	Note: In ES1.0, PLL output clock frequency 200 MHz < Fin*M/N < 351 MHz In ES1.1 and higher, PLL output clock frequency 110 MHz < Fin*M/N < 351 MHz Note: In ES1.0 default value is 1 In ES1.1 default value is 0

4.9.2 PLLB - CLKC PLLB Configuration

PLLB 0003:0882 offset: 0x02 default: 0x0081																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	TEST0	TEST1	PWRDN	RSV	TEST2[2]	TEST2[1]	TEST2[0]	PMB[3]	PMB[2]	PMB[1]	PMB[0]	RSV	PNB[2]	PNB[1]	PNB[0]
R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W
Default	-	0	0	0	-	0	0	0	1	0	0	0	-	0	0	1

Bit	Name	Reset Value	R/W	Function
15	RSV			
14	TEST0	0	R/W	Test bit for PLLB, set to 0
13	TEST1	0	R/W	Bypass Test bit for PLLB, set to 0
12	PWRDN	0	R/W	Powerdown for PLLB » 0: normal operation « » 1: powerdown «
11	RSV			
10:8	TEST2	0	R/W	Test bits for PLLB, set to 000
7:4	PMB	8	R/W	PLLB Numerator M=PMB+1 N=PNB+1 PLLB = Fin*M/N Note: In ES1.0, PLL output clock frequency 200 MHz < Fin*M/N < 351 MHz In ES1.1 and higher, PLL output clock frequency 110 MHz < Fin*M/N < 351 MHz
3	RSV			
2:0	PNB	1	R/W	PLLB Denominator M=PMB+1 N=PNB+1 PLLB = Fin*M/N Note: In ES1.0, PLL output clock frequency 200 MHz < Fin*M/N < 351 MHz In ES1.1 and higher, PLL output clock frequency 110 MHz < Fin*M/N < 351 MHz

4.9.3 CLKC - CLKC Input clock Source Selection

CLKC 0003:0884 offset: 0x04 default: 0x4000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	CMCIV	CCDIV	CENIV	TEST0	CUAIS	CUA0S	CTM3S	CTM2S	CTM1S	CTM0S	COSDS	CCCDs	CENS1	CENS0	CFINS
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			
14	CMCIV	1	R/W	MMC Clock Invert change only after clock is disabled » 0: normal « » 1: invert «
13	CCDIV	0	R/W	CCD Pixel Clock (CCD_CLK) Invert change only after clock is disabled » 0: normal « » 1: invert «
12	CENIV	0	R/W	Encoder Clock Invert change only after clock is disabled » 0: normal « » 1: invert «
11	TEST0	0	R/W	Test bit - fix at 0
10	CUAIS	0	R/W	UART1 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
9	CUA0S	0	R/W	UART0 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
8	CTM3S	0	R/W	TMR3 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
7	CTM2S	0	R/W	TMR2 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
6	CTM1S	0	R/W	TMR1 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
5	CTM0S	0	R/W	TMR0 Clock Select change only after clock is disabled » 0: ARM Clock « » 1: PLLIN «
4	COSDS	0	R/W	OSD Clock Select change only after clock is disabled » 0: SYSClk « » 1: PCLK «
3	CCCDs	0	R/W	CCD Clock Select change only after clock is disabled » 0: PCLK « » 1: SYSClk «
2	CENS1	0	R/W	ENC1 Clock Select change only after clock is disabled » 0: MXI/SYSClk « » 1: PCLK «
1	CENS0	0	R/W	ENC0 Clock Select change only after clock is disabled » 0: MXI « » 1: SYSClk «
0	CFINS	0	R/W	PLL input clock select Setting of PLL should be conducted again after change in TEST2 In case of TEST2 = L » 0: MXI (TES2='0') or SYSClk (TEST2='1') « » 0: SYSClk (TES2='0') or MXI (TEST2='1') «

4.9.4 SEL - CLKC Input PLL Source Selection

SEL		0003:0886				offset: 0x06		default: 0x0100											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	CAXLS	RSV	RSV	RSV	CSDRS	RSV	RSV	RSV	CDSPS	RSV	RSV	RSV	CARMS			
R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W			
Default	-	-	-	0	-	-	-	1	-	-	-	0	-	-	-	0			

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	CAXLS	0	R/W	Accelerator Clock Select » 0: PLL A « » 1: PLL B «
11:9	RSV			
8	CSDRS	1	R/W	SDRAM Clock Select » 0: PLL A « » 1: PLL B «
7:5	RSV			
4	CDSPS	0	R/W	DSP Clock Select » 0: PLL A « » 1: PLL B «
3:1	RSV			
0	CARMS	0	R/W	ARM Clock Select » 0: PLL A « » 1: PLL B «

4.9.5 DIV - CLKC PLL Clock Divisor Settings

DIV		0003:0888				offset: 0x08		default: 0x0003											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	CAXDIV[1]	CAXDIV[0]	RSV	RSV	CSDIV[1]	CSDIV[0]	RSV	RSV	CDDIV[1]	CDDIV[0]	RSV	RSV	CADIV[1]	CADIV[0]			
R/W	-	-	R/W	R/W	-	-	R/W	R/W	-	-	R/W	R/W	-	-	R/W	R/W			
Default	-	-	0	0	-	-	0	0	-	-	0	0	-	-	1	1			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	CAXDIV	0	R/W	Accelerator Clock Division » 00: 1 « » 01: 2 « » 10: 3 « » 11: 4 «
11:10	RSV			Reserved
9:8	CSDIV	0	R/W	SDRAM Clock Division » 00: 1 « » 01: 2 « » 10: 3 « » 11: 4 «
7:6	RSV			Reserved
5:4	CDDIV	0	R/W	DSP Clock Division » 00: 1 « » 01: 2 « » 10: 3 « » 11: 4 «
3:2	RSV			Reserved
1:0	CADIV	3	R/W	ARM Clock Division » 00: 1 « » 01: 2 « » 10: 3 « » 11: 4 «
Note: In ES1.0 default value is 01b In ES1.1 default value is 11b				

4.9.6 BYP - CLKC PLL Bypass Configuration

BYP				0003:088A				offset: 0x0A				default: 0x0001				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	CMB	RSV	RSV	RSV	CSB	RSV	RSV	RSV	CDB	RSV	RSV	RSV	CAB
R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-	1

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	CMB	0	R/W	Accelerator Clock Bypass » 0: Normal « » 1: Bypass «
11:9	RSV			Reserved
8	CSB	0	R/W	SDRAM Clock Bypass » 0: Normal « » 1: Bypass «
7:5	RSV			Reserved
0:00	CDB	0	R/W	DSP Clock Bypass » 0: Normal « » 1: Bypass «
3:1	RSV			Reserved
0	CAB	1	R/W	ARM Clock Bypass » 0: Normal « » 1: Bypass « It is 0 by default in external host boot mode (EXTHOST=H and BTSEL=H) and 1 by default in normal mode.
				In ES1.0, Therefore, when 27Mhz is set to input frequency of PLL in the external host boot mode, the clock having (9/2)/2 times frequency 60.75Mhz by default is given as ARM clock after PLL locking In ES1.1, Therefore, when 27Mhz is set to input frequency of PLL in the external host boot mode, the clock having (9/1)/4 times frequency 60.75Mhz by default is given as ARM clock after PLL locking

4.9.7 MMCCLK - CLKC MMC Clock Divisor Settings

MMCCLK		0003:088C								offset: 0x0C		default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CMDIV[7]	CMDIV[6]	CMDIV[5]	CMDIV[4]	CMDIV[3]	CMDIV[2]	CMDIV[1]	CMDIV[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	CMDIV	0	R/W	MMC Clock Division MMC Clock rate = ARM Clock/(CMDIV +1)

4.9.8 CTEST - CLKC Test Register - do not use

CTEST		0003:088E				offset: 0x0E										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	

Bit	Name	Reset Value	R/W	Function
15:0	RSV			Reserved. Do not use

4.9.9 MOD0 - CLKC Module Clock Enables #0

MOD0					0003:0890					offset: 0x10					default: 0x07E7				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	TEST	RSV	RSV	RSV	RSV	CHPIB	C5409	CEHIF	CSDRC	CEMIF	CINTC	CATM	CADMAC	CICA	CBUSC	CACORE			
R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	-	-	-	-	1	1	1	1	1	1	0	0	1	1	1			

Bit	Name	Reset Value	R/W	Function
15	TEST	0	R/W	Test bit. Do not use
14:11	RSV			Reserved
10	CHP1B	1	R/W	HPI Bridge, » 0: disable «, » 1: enable «
9	C5409	1	R/W	DSP Controller, » 0: disable «, » 1: enable «
8	CEHIF	1	R/W	Ext Host I/F, » 0: disable «, » 1: enable «
				SDRAM Controller, » 0: disable «, » 1: enable «
				Note: In ES1.0 default value is always 1
7	CSDRC	1	R/W	In ES1.1, default value depends on FLSHSEL signal - when FLSHSEL=0, default value = 0 - when FLSHSEL=1, default value = 1 Also automatic setting at the time of wakeup is performed only when FLSHSEL=1
6	CEMIF	1	R/W	EMIF, » 0: disable «, » 1: enable «
5	CINTC	1	R/W	Interrupt Controller, » 0: disable «, » 1: enable «
4	CATM	0	R/W	ARM Trace, » 0: disable «, » 1: enable «
3	CADMAC	0	R/W	ARM DMA Controller, » 0: disable «, » 1: enable «
2	CICA	1	R/W	I-cache, » 0: disable «, » 1: enable «
1	CBUSC	1	R/W	BUS Controller, » 0: disable «, » 1: enable «
0	CACORE	1	R/W	ARM core, » 0: disable «, » 1: enable «

4.9.10 MOD1 - CLKC Module Clock Enables #1

MOD1 0003:0892															offset: 0x12																	default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CIMG	CIMX	CVLC	CDAC	CVENC	COSD	CPRV	CH3A	CCCDC																	
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0																	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	CIMG	0	R/W	Image Buffer, » 0: disable «, » 1: enable «
7	CIMX	0	R/W	IMX, » 0: disable «, » 1: enable «
6	CVLC	0	R/W	VLCD, » 0: disable «, » 1: enable «
5	CDAC	0	R/W	DAC, » 0: disable «, » 1: enable «
4	CVENC	0	R/W	Video Encoder, » 0: disable «, » 1: enable «
3	COSD	0	R/W	OSD, » 0: disable «, » 1: enable «
2	CPRV	0	R/W	Preview Engine, » 0: disable «, » 1: enable «
1	CH3A	0	R/W	Hardware 3A, » 0: disable «, » 1: enable «
0	CCCDC	0	R/W	CCD Controller, » 0: disable «, » 1: enable «

4.9.11 MOD2 - CLKC Module Clock Enables #2

MOD2		0003:0894										offset: 0x14		default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	CIGC	TEST	CMMC	CSIF1	CSIF0	CUAT1	CUAT0	CUSB	GGIO	CTMR3	CTMR2	CTMR1	CTMR0	CWDT	
R/W	-	-	RW	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	CI2C	0	R/W	I2C, » 0: disable «, » 1: enable «
12	TEST	0	R/W	Test Bit - do not use
11	CMMC	0	R/W	MMC, » 0: disable «, » 1: enable «
10	CSIF1	0	R/W	Serial IF #1, » 0: disable «, » 1: enable «
9	CSIF0	0	R/W	Serial IF #0, » 0: disable «, » 1: enable «
8	CUAT1	0	R/W	UART #1, » 0: disable «, » 1: enable «
7	CUAT0	0	R/W	UART #0, » 0: disable «, » 1: enable «
6	CUSB	0	R/W	USB, » 0: disable «, » 1: enable «
5	CGIO	0	R/W	GIO, » 0: disable «, » 1: enable «
4	CTMR3	0	R/W	Timer #3, » 0: disable «, » 1: enable «
3	CTMR2	0	R/W	Timer #2, » 0: disable «, » 1: enable «
2	CTMR1	0	R/W	Timer #1, » 0: disable «, » 1: enable «
1	CTMR0	0	R/W	Timer #0, » 0: disable «, » 1: enable «
0	CWDT	0	R/W	Watch Dog Timer, » 0: disable «, » 1: enable «

4.9.12 LPCTL0 - CLKC Low Power Control #0

LPCTL0 0003:0896 offset: 0x16 default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SLPMD
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
				Sleep mode 0 : Normal 1 : Sleep
0	SLPMD	0	R/W	Wake-up from Sleep mode must be done through external interrupt,GI00, and this bit can be cleared only by external interrupt signal, GI00.

4.9.13 LPCTL1 - CLKC Low Power Control #1

LPCTL1	0003:0898				offset:	0x18								default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PDMD	OSC48		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0		

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
				Auto Power Down Mode 0: Normal 1: Auto Power Down Mode
1	PDMD	0	R/W	When PDMD=0, Power Down mode is entered by, 1. Setting LPCTL0:bit 0 = 1. This stops PLL and clock for all modules 2. Pull PWDWN=H. This stops oscillation of crystal connected to MXI, M48XI pins When PDMD=1, Power Down mode is entered by, Pulling PWDWN=H. This automatically stops PLL and clock for all modules. After which it stops oscillation of crystal connected to MXI, M48XI pins
0	OSC48	0	R/W	OSC48 Cell Power Down » 0: Cell enable « » 1: Cell disable «

4.9.14 OSEL - CLKC Output Clock Selector

OSEL	0003:089A												offset:	0x1A				default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	RSV	RSV	RSV	RSV	RSV	O2SEL[2]	O2SEL[1]	O2SEL[0]	RSV	O1SEL[2]	O1SEL[1]	O1SEL[0]	RSV	O0SEL[2]	O0SEL[1]	O0SEL[0]						
R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W						
Default	-	-	-	-	-	0	0	0	-	0	0	0	-	0	0	0						

Bit	Name	Reset Value	R/W	Function
15: 11	RSV			General-purpose clock output 2 (GI018) source select. » 000: ARM clock « » 001: DSP clock « » 010: SDR clock « » 011: AXI clock « » 100: PLL input clock « » 101: M48XI «
7	RSV			General-purpose clock output 1 (GI017) source select. » 000: ARM clock « » 001: DSP clock « » 010: SDR clock « » 011: AXI clock « » 100: PLL input clock « » 101: M48XI «
6: 4	O1SEL	0	R/W	
3	RSV			General-purpose clock output 0 (GI016) source select. » 000: ARM clock « » 001: DSP clock « » 010: SDR clock « » 011: AXI clock « » 100: PLL input clock « » 101: M48XI «
2: 0	O0SEL	0	R/W	

4.9.15 O0DIV - CLKC Output Clock #0 Divider

O0DIV	0003:089C										offset:	0x1C	default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	O0DV[15]	O0DV[14]	O0DV[13]	O0DV[12]	O0DV[11]	O0DV[10]	O0DV[9]	O0DV[8]	O0DV[7]	O0DV[6]	O0DV[5]	O0DV[4]	O0DV[3]	O0DV[2]	O0DV[1]	O0DV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	O0DV	0	R/W	General-purpose clock output 0 (GI016) Divide Ratio Clock source selected by O0SEL is divided by 2*(O0DV+1)

4.9.16 O1DIV - CLKC Output Clock #1 Divider

O1DIV	0003:089E										offset:	0x1E	default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	O1DV[15]	O1DV[14]	O1DV[13]	O1DV[12]	O1DV[11]	O1DV[10]	O1DV[9]	O1DV[8]	O1DV[7]	O1DV[6]	O1DV[5]	O1DV[4]	O1DV[3]	O1DV[2]	O1DV[1]	O1DV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	O1DV	0	R/W	General-purpose clock output 0 (GI017) Divide Ratio Clock source selected by O1SEL is divided by 2*(O1DV+1)

4.9.17 O2DIV - CLKC Output Clock #2 Divider

O2DIV	0003:08A0										offset:	0x20	default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	O2DV[3]	O2DV[2]	O2DV[1]	O2DV[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:4	RSV			
3:0	O2DV	0	R/W	General-purpose clock output 0 (GI018) Divide Ratio Clock source selected by O2SEL is divided by 2*(O2DV+1)

4.9.18 PWM0C - CLKC PWM #0 Cycle Count

PWM0C				0003:08A2				offset:		0x22				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	P0C[15]	P0C[14]	P0C[13]	P0C[12]	P0C[11]	P0C[10]	P0C[9]	P0C[8]	P0C[7]	P0C[6]	P0C[5]	P0C[4]	P0C[3]	P0C[2]	P0C[1]	P0C[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	P0C	0	R/W	Cycle of PWM output 0 Number of counts on PLL input clock base. PWM output 0 is output selecting from GI029

4.9.19 PWM0H - CLKC PWM #0 High Period

PWM0H				0003:08A4				offset:		0x24										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	P0H[15]	P0H[14]	P0H[13]	P0H[12]	P0H[11]	P0H[10]	P0H[9]	P0H[8]	P0H[7]	P0H[6]	P0H[5]	P0H[4]	P0H[3]	P0H[2]	P0H[1]	P0H[0]					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:0	P0H	0	R/W	High level period of PWM output 0 Number of counts on PLL input clock basis

4.9.20 PWM1C - CLKC PWM #1 Cycle Count

PWM1C	0003:08A6								offset:	0x26								default:	0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	P1C[15]	P1C[14]	P1C[13]	P1C[12]	P1C[11]	P1C[10]	P1C[9]	P1C[8]	P1C[7]	P1C[6]	P1C[5]	P1C[4]	P1C[3]	P1C[2]	P1C[1]	P1C[0]				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:0	P1C	0	R/W	Cycle of PWM output 1 Number of counts on PLL input clock base. PWM output 0 is output selecting from GI030

4.9.21 PWM1H - CLKC PWM #1 High Period

PWM1H	0003:08A8								offset:	0x28								default:	0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	P1H[15]	P1H[14]	P1H[13]	P1H[12]	P1H[11]	P1H[10]	P1H[9]	P1H[8]	P1H[7]	P1H[6]	P1H[5]	P1H[4]	P1H[3]	P1H[2]	P1H[1]	P1H[0]				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:0	P1H	0	R/W	High level period of PWM output 1 Number of counts on PLL input clock basis

5 Interrupt Controller

5.1 Introduction

The Interrupt Controller (INTC) has 39 interrupt channels: 23 internal peripheral interrupt channels and 16 external interrupt channels. Section 5.2 Interrupt Source List lists each interrupt and the interrupt's corresponding function.

The INTC has the following characteristics:

- o 39 interrupt channels (16 external channels)
- o Interrupt mask each for channel
- o Each interrupt channel can be mapped to an FIQ or IRQ type interrupt
- o Configurable hardware interrupt priority
- o Configurable interrupt entry table

The ARM core supports two interrupt types: FIQ and IRQ. See the ARM7 User's Manual for detailed information about the ARM's FIQ and IRQ interrupts. Each interrupt channel can be mapped to an FIQ or IRQ type interrupt, and each channel can be enabled or disabled. The INTC supports hardware configurable interrupt priority, and interrupt entry address generation. Entry address generation minimizes time spent for jumping to interrupt vectors. When an interrupt occurs the corresponding highest priority interrupt vector is stored in the INTC's ENTRY register. The interrupt service routine can read the ENTRY register and jump to the interrupt vector. Thus the ARM does not have to execute any parsing software to associate the interrupt with the correct interrupt vector.

5.2 Interrupt Source List

Table 6. shows the various sources that can interrupt the ARM7. See respective sections for cause of each interrupt

Table 6. Interrupt Source List

Interrupt ID	Tag	Name
0	TMR0	Timer 0 Interrupt
1	TMR1	Timer 1 Interrupt
2	TMR2	Timer 2 Interrupt
3	TMR3	Timer 3 Interrupt
4	CCDVD0	CCD VD Interrupt #0
5	CCDVD1	CCD VD Interrupt #1
6	CCDVD2	CCD VD Interrupt #2. Occurs at the falling edge of C_WEN signal
7	OSD	OSD Interrupt. Occurs at the falling edge of OSD_VSYNC

Interrupt ID	Tag	Name
8	SP0	Serial I/F 0 Interrupt Occurs at the end of 8-bit data transfer(non-DMA mode) or at the end of DMA transfer
9	SP1	Serial I/F 1 Interrupt Occurs at the end of 8-bit data transfer(non-DMA mode)
10	EXTHOST	External Host Interrupt
11	DSPHINT	DSP HPI Interrupt
12	UART0	UART 0 Interrupt
13	UART1	UART 1 Interrupt
14	USB	USB Interrupt
15	MTC0	Memory Traffic Controller Interrupt #0 Occurs at the end of EMIF DMA transfer
16	MTC1	Memory Traffic Controller Interrupt #1 Occurs due to conditions specified register BUSINTEN
17	MMCS0	MMS/SD I/F Interrupt #0
18	MMCS1	MMS/SD I/F Interrupt #1
19	EXT0	External Interrupt #0 (GIO0)
20	EXT1	External Interrupt #1 (GIO1)
21	EXT2	External Interrupt #2 (GIO2)
22	EXT3	External Interrupt #3 (GIO3)
23	EXT4	External Interrupt #4 (GIO4)
24	EXT5	External Interrupt #5 (GIO5)
25	EXT6	External Interrupt #6 (GIO6)
26	EXT7	External Interrupt #7 (GIO7)
27	EXT8	External Interrupt #8 (GIO8)
28	EXT9	External Interrupt #9 (GIO9)
29	EXT10	External Interrupt #10 (GIO10)
30	EXT11	External Interrupt #11 (GIO11)
31	EXT12	External Interrupt #12 (GIO12)
32	EXT13	External Interrupt #13 (GIO13)
33	EXT14	External Interrupt #14 (GIO14)
34	EXT15	External Interrupt #15 (GIO15)
35	PREV	Preview Engine Interrupt
36	WDT	Watch Dog Timer Interrupt
37	I2C	I2C Interrupt
38	CLKC	Clock Controller Interrupt
39	RSVINT	Reserved Interrupt

5.3 INTC Methodology

Figure 17 illustrates the INTC methodology. As shown in Figure 17, interrupts are mapped to FIQ or IRQ type interrupts by setting the register FISELn.

- When an interrupt occurs the status is reflected in either the FIQn or IRQn registers.
- Interrupts are enabled or disabled (masked), by setting the EINTn register. Note: even if an interrupt is masked, the status of each interrupt is still reflected in the FIQn and IRQn registers.
- When interrupt from any interrupt channel occurs, IRQ or FIQ interrupt is generated to ARM7 core (depending on whether the interrupt channel is mapped to IRQ or FIQ interrupt). The ARM7 then branches to the IRQ or FIQ interrupt service routine.
- The INTCC generates the entry address of the pending interrupt with the highest priority and stores the entry address in the FIQENTRY or IRQENTRY register depending on whether the interrupt is mapped to IRQ or FIQ interrupt. The IRQ or FIQ ISR can then read this interrupt entry address and thus branch to the ISR of the interrupt channel.

Interrupt priority can be configured in the INTPRIn registers. The entry address is equal to a base address plus an offset, and the base address can be set in the EABASEn registers.

Figure 17. INTCC Methodology

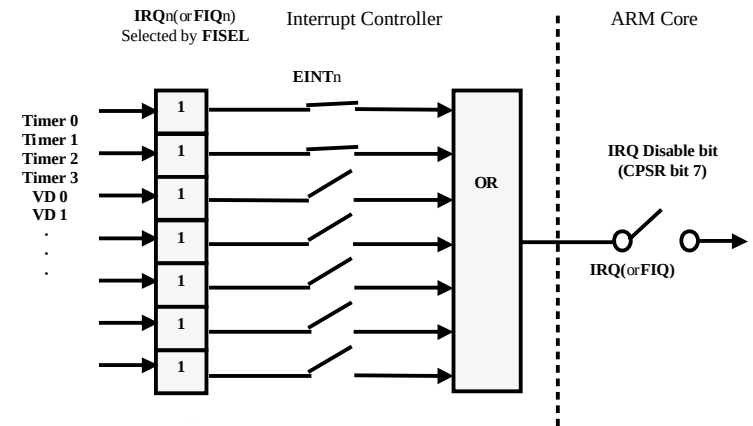
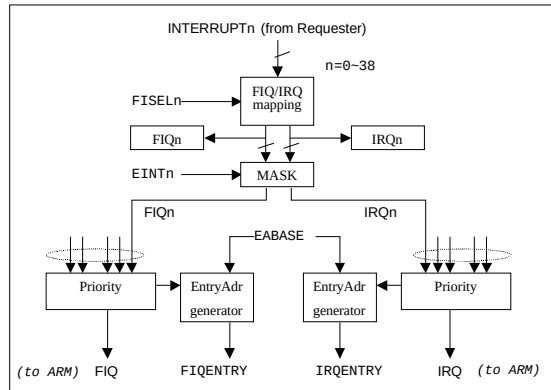


Table 7. summarizes the registers that are used to configure the interrupt controller. Refer to section 5.7 Interrupt Controller Registers for details

Table 7. Interrupt Controller Register Summary

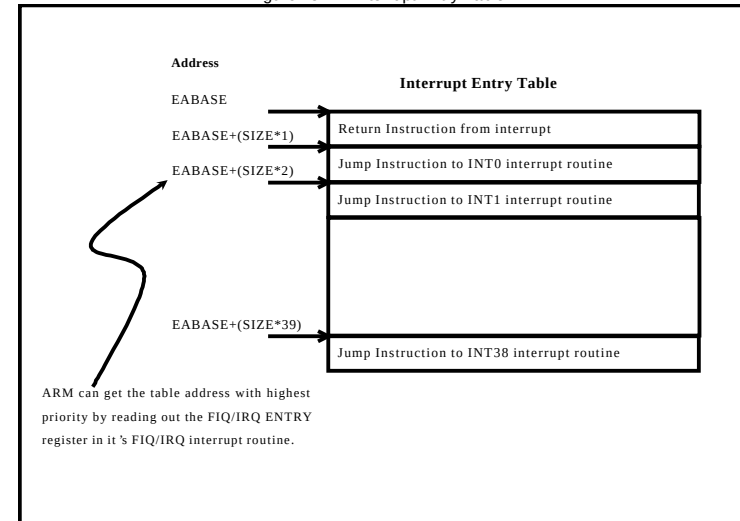
Register	Description
FISELn	<u>IRQ/FIQ Selection</u> Select interrupt channel as FIQ or IRQ (See 5.7.11, 5.7.12, 5.7.13)
EINTn	<u>Interrupt Masking</u> Enable or disable a interrupt channel Note: even if an interrupt is masked (disabled), the status of each interrupt is still reflected in the FIQn and IRQn registers (See 5.7.14, 5.7.15, 5.7.16)
EABASE	<u>Interrupt Entry Table</u> Set base address and size of each entry in the interrupt entry table (See 5.7.18, 5.7.19)

Register	Description
FIQENTRY, IRQENTRY	Interrupt entry address of highest pending IRQ, FIQ interrupt (See 5.7.7, 5.7.9)
INTRAW	Selects whether a masked interrupt is reflected in FIQENTRY or IRQENTRY (See 5.7.17)
FIQn	Shows status of interrupt mapped to ARM7 FIQ interrupt. Clear/acknowledge pending interrupt. When the register is read, '0', in a bit position indicates that the corresponding interrupt has occurred. '1', in a bit position indicates that the corresponding interrupt has NOT occurred. When the register is written, writing '0', in a bit position has no effect writing '1', in a bit position clears the pending interrupt (See 5.7.1, 5.7.2, 5.7.3)
IRQn	Shows status of interrupt mapped to ARM7 IRQ interrupt. Clear/acknowledge pending interrupt When the register is read, '0', in a bit position indicates that the corresponding interrupt has occurred. '1', in a bit position indicates that the corresponding interrupt has NOT occurred. When the register is written, writing '0', in a bit position has no effect writing '1', in a bit position clears the pending interrupt (See 5.7.4, 5.7.5, 5.7.6)
INTPRIn	Set priority for each interrupt channel. Highest priority interrupt entry address is reflect in FIQENTRY or IRQENTRY register. By default, Timer 0 (INT0) has priority and CLKC interrupt (INT38) has lowest priority (See 5.7.20 - 5.7.39)

5.4 Hardware Configurable Interrupt Entry Table

The hardware configurable interrupt entry table can be used to minimize the time spent in jumping to interrupt service routines. The entry table, created in software, can be stored anywhere in memory, but it must be on an 8-byte boundary. The structure of the entry table is shown in Figure 18

Figure 18. Interrupt Entry Table



5.4.1 Initialization of the interrupt entry table:

- o Mask FIQ and IRQ
- o Mask all interrupt channels by EINTn register
- o Map each interrupt to FIQ or IRQ by FISELn register
- o Setup the priority of each interrupt by priority configuration register (INTPRIn)
- o Setup the interrupt entry table base register (EABASE)
- o Enable the corresponding interrupts
- o Enable the FIQ and IRQ

5.4.2 Using the entry table to process interrupts

- ARM receives common FIQ/IRQ interrupt signal
- ARM responds to interrupt signal and jumps to common FIQ or IRQ vector (depending on whether an FIQ or IRQ type interrupt occurred). In ARM7, FIQ vector is at address 0x001C and IRQ vector is at address 0x0018
- The software located in memory at the common interrupt vector reads the FIQ entry or the IRQ entry registers to get the interrupt service routine address, and then jumps to this address, which is the interrupt handler, corresponding to INTx.
- The interrupt is cleared by setting the bit corresponding to INTx in FIQ, IRQ status register
- Interrupt handler is executed
- After this control returns to the common interrupt vector routine from the interrupt handler
- After this control returns to the state of program execution that existed prior to when the interrupt occurred from the common interrupt vector routine

5.5 External Interrupts

Interrupts ID 19-34 (EXT0-EXT15) are external interrupts. These interrupts are shared with pins GPIO15-0. To use any one of these interrupts, the shared GPIO port must be setup as an interrupt port by setting the corresponding GPIO register (IRQPORT, IRQEDGE). Refer to the section on GPIO for more information on configuring GPIO port as interrupt ports.

The interrupt control registers samples interrupt signals on the raising edge of the ARM clock. Therefore interrupt signal should be at least one ARM cycle long to be recognized (refer to the section on the Clock Controller for information on the ARM clock).

5.6 Interrupt Controller Register Map (INTC)

Address	Register	Description
0003:0500	FIQ0	FIQ Interrupt Flag Register #0
0003:0502	FIQ1	FIQ Interrupt Flag Register #1
0003:0504	FIQ2	FIQ Interrupt Flag Register #2
0003:0508	IRQ0	IRQ Interrupt Flag Register #0
0003:050A	IRQ1	IRQ Interrupt Flag Register #1
0003:050C	IRQ2	IRQ Interrupt Flag Register #2
0003:0510	FIQENTRY0	FIQ Entry Address Register #0
0003:0512	FIQENTRY1	FIQ Entry Address Register #1
0003:0518	IRQENTRY0	IRQ Entry Address Register #0
0003:051A	IRQENTRY1	IRQ Entry Address Register #1
0003:0520	FISEL0	FIQ select register #0
0003:0522	FISEL1	FIQ select register #1
0003:0524	FISEL2	FIQ select register #2
0003:0528	EINT0	Interrupt Enable Register #0
0003:052A	EINT1	Interrupt Enable Register #1
0003:052C	EINT2	Interrupt Enable Register #2
0003:0530	INTRAW	Interrupt Raw Register
0003:0538	EABASE0	Entry Table Base Address Register #0
0003:053A	EABASE1	Entry Table Base Address Register #1
0003:0540	INTPRI00	Interrupt Priority Register #0
0003:0542	INTPRI01	Interrupt Priority Register #1
0003:0544	INTPRI02	Interrupt Priority Register #2
0003:0546	INTPRI03	Interrupt Priority Register #3
0003:0548	INTPRI04	Interrupt Priority Register #4
0003:054A	INTPRI05	Interrupt Priority Register #5
0003:054C	INTPRI06	Interrupt Priority Register #6
0003:054E	INTPRI07	Interrupt Priority Register #7
0003:0550	INTPRI08	Interrupt Priority Register #8
0003:0552	INTPRI09	Interrupt Priority Register #9
0003:0554	INTPRI10	Interrupt Priority Register #10
0003:0556	INTPRI11	Interrupt Priority Register #11
0003:0558	INTPRI12	Interrupt Priority Register #12
0003:055A	INTPRI13	Interrupt Priority Register #13
0003:055C	INTPRI14	Interrupt Priority Register #14
0003:055E	INTPRI15	Interrupt Priority Register #15
0003:0560	INTPRI16	Interrupt Priority Register #16
0003:0562	INTPRI17	Interrupt Priority Register #17
0003:0564	INTPRI18	Interrupt Priority Register #18
0003:0566	INTPRI19	Interrupt Priority Register #19

5.7 Interrupt Controller Registers

5.7.1 FIQ0 - FIQ Interrupt Flag Register #0

FIQ0 0003:0500 offset: 0x00 default: 0xFFFF																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	MTC0	USB	UART1	UART0	DSPHINT	EXTHOST	SP1	SP0	OSD	CCDVD2	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	MTC0	1	R/W	<u>Memory Traffic Controller #0 Interrupt Flag</u> Rd 0: active Wr 1: cleared
14	USB	1	R/W	<u>USB Interrupt Flag</u> Rd 0: active Wr 1: cleared
13	UART1	1	R/W	<u>UART1 Interrupt Flag</u> Rd 0: active Wr 1: cleared
12	UART0	1	R/W	<u>UART0 Interrupt Flag</u> Rd 0: active Wr 1: cleared
11	DSPHINT	1	R/W	<u>DSP HINT Interrupt Flag</u> Rd 0: active Wr 1: cleared
10	EXTHOST	1	R/W	<u>External HOST Interrupt Flag</u> Rd 0: active Wr 1: cleared
9	SP1	1	R/W	<u>Serial Port 1 Interrupt Flag</u> Rd 0: active Wr 1: cleared
8	SP0	1	R/W	<u>Serial Port 0 Interrupt Flag</u> Rd 0: active Wr 1: cleared
7	OSD	1	R/W	<u>OSD Interrupt Flag</u> Rd 0: active Wr 1: cleared
6	CCDVD2	1	R/W	<u>CCD VD Interrupt #2 Flag</u> Rd 0: active Wr 1: cleared
5	CCDVD1	1	R/W	<u>CCD VD Interrupt #1 Flag</u> Rd 0: active Wr 1: cleared
4	CCDVD0	1	R/W	<u>CCD VD Interrupt #0 Flag</u> Rd 0: active Wr 1: cleared
3	TMR3	1	R/W	<u>Timer 3 Interrupt Flag</u> Rd 0: active Wr 1: cleared
2	TMR2	1	R/W	<u>Timer 2 Interrupt Flag</u> Rd 0: active Wr 1: cleared
1	TMR1	1	R/W	<u>Timer 1 Interrupt Flag</u> Rd 0: active Wr 1: cleared
0	TMR0	1	R/W	<u>Timer 0 Interrupt Flag</u> Rd 0: active Wr 1: cleared

5.7.2 FIQ1 - FIQ Interrupt Flag Register #1

FIQ1															0003:0502		offset: 0x02																	default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																			
Name	EXT12	EXT11	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCS01	MMCS00	MTC1																			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																			
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1																			

Bit	Name	Reset Value	R/W	Function
15	EXT12	1	R/W	<u>External Interrupt #12 Flag</u> Rd 0: active Wr 1: cleared
14	EXT11	1	R/W	<u>External Interrupt #11 Flag</u> Rd 0: active Wr 1: cleared
13	EXT10	1	R/W	<u>External Interrupt #10 Flag</u> Rd 0: active Wr 1: cleared
12	EXT9	1	R/W	<u>External Interrupt #9 Flag</u> Rd 0: active Wr 1: cleared
11	EXT8	1	R/W	<u>External Interrupt #8 Flag</u> Rd 0: active Wr 1: cleared
10	EXT7	1	R/W	<u>External Interrupt #7 Flag</u> Rd 0: active Wr 1: cleared
9	EXT6	1	R/W	<u>External Interrupt #6 Flag</u> Rd 0: active Wr 1: cleared
8	EXT5	1	R/W	<u>External Interrupt #5 Flag</u> Rd 0: active Wr 1: cleared
7	EXT4	1	R/W	<u>External Interrupt #4 Flag</u> Rd 0: active Wr 1: cleared
6	EXT3	1	R/W	<u>External Interrupt #3 Flag</u> Rd 0: active Wr 1: cleared
5	EXT2	1	R/W	<u>External Interrupt #2 Flag</u> Rd 0: active Wr 1: cleared
4	EXT1	1	R/W	<u>External Interrupt #1 Flag</u> Rd 0: active Wr 1: cleared
3	EXT0	1	R/W	<u>External Interrupt #0 Flag</u> Rd 0: active Wr 1: cleared
2	MMCS01	1	R/W	<u>MMC/SD Interrupt #1 Flag</u> Rd 0: active Wr 1: cleared
1	MMCS00	1	R/W	<u>MMC/SD Interrupt #0 Flag</u> Rd 0: active Wr 1: cleared
0	MTC1	1	R/W	<u>Memory Traffic Controller #1 Interrupt Flag</u> Rd 0: active Wr 1: cleared

5.7.3 FIQ2 - FIQ Interrupt Flag Register #2

FIQ2		0003:0504				offset: 0x04		default: 0x00FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSVINT	CLKC	I2C	WDT	PREV	EXT15	EXT14	EXT13	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	RSVINT	1	R/W	Reserved Interrupt Flag Rd 0: active Wr 1: cleared
6	CLKC	1	R/W	Clock Controller Interrupt Flag Rd 0: active Wr 1: cleared
5	I2C	1	R/W	I2C Interrupt Flag Rd 0: active Wr 1: cleared
4	WDT	1	R/W	Watch Dog Timer Interrupt Flag Rd 0: active Wr 1: cleared
3	PREV	1	R/W	Preview Engine Interrupt Flag Rd 0: active Wr 1: cleared
2	EXT15	1	R/W	External Interrupt #15 Flag Rd 0: active Wr 1: cleared
1	EXT14	1	R/W	External Interrupt #14 Flag Rd 0: active Wr 1: cleared
0	EXT13	1	R/W	External Interrupt #13 Flag Rd 0: active Wr 1: cleared

5.7.4 IRQ0 - IRQ Interrupt Flag Register #0

IRQ0	0003:0508												offset: 0x08						default: 0xFFFF			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	MTC0	USB	UART1	UART0	DSPHINT	EXTHOST	SP1	SP0	OSD	CCDVD2	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0						
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W						
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1						

Bit	Name	Reset Value	R/W	Function
15	MTC0	1	R/W	Memory Traffic Controller #0 Interrupt Flag Rd 0: active Wr 1: cleared
14	USB	1	R/W	USB Interrupt Flag Rd 0: active Wr 1: cleared
13	UART1	1	R/W	UART1 Interrupt Flag Rd 0: active Wr 1: cleared
12	UART0	1	R/W	UART0 Interrupt Flag Rd 0: active Wr 1: cleared
11	DSPHINT	1	R/W	DSP HINT Interrupt Flag Rd 0: active Wr 1: cleared
10	EXTHOST	1	R/W	MMC/SD and MS Interrupt Flag Rd 0: active Wr 1: cleared
9	SP1	1	R/W	Serial Port 1 Interrupt Flag Rd 0: active Wr 1: cleared
8	SP0	1	R/W	Serial Port 0 Interrupt Flag Rd 0: active Wr 1: cleared
7	OSD	1	R/W	OSD Interrupt Flag Rd 0: active Wr 1: cleared
6	CCDVD2	1	R/W	CCD VD Interrupt #2 Flag Rd 0: active Wr 1: cleared
5	CCDVD1	1	R/W	CCD VD Interrupt #1 Flag Rd 0: active Wr 1: cleared
4	CCDVD0	1	R/W	CCD VD Interrupt #0 Flag Rd 0: active Wr 1: cleared
3	TMR3	1	R/W	Timer 3 Interrupt Flag Rd 0: active Wr 1: cleared
2	TMR2	1	R/W	Timer 2 Interrupt Flag Rd 0: active Wr 1: cleared
1	TMR1	1	R/W	Timer 1 Interrupt Flag Rd 0: active Wr 1: cleared
0	TMR0	1	R/W	Timer 0 Interrupt Flag Rd 0: active Wr 1: cleared

5.7.5 IRQ1 - IRQ Interrupt Flag Register #1

IRQ1		0003:050A		offset: 0x0A		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	EXT12	EXT11	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCS01	MMCS00	MT1			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15	EXT12			<u>External Interrupt #12 Flag</u> Rd 0: active Wr 1: cleared
14	EXT11	0	R/W	<u>External Interrupt #11 Flag</u> Rd 0: active Wr 1: cleared
13	EXT10			<u>External Interrupt #10 Flag</u> Rd 0: active Wr 1: cleared
12	EXT9			<u>External Interrupt #9 Flag</u> Rd 0: active Wr 1: cleared
11	EXT8			<u>External Interrupt #8 Flag</u> Rd 0: active Wr 1: cleared
10	EXT7			<u>External Interrupt #7 Flag</u> Rd 0: active Wr 1: cleared
9	EXT6			<u>External Interrupt #6 Flag</u> Rd 0: active Wr 1: cleared
8	EXT5			<u>External Interrupt #5 Flag</u> Rd 0: active Wr 1: cleared
7	EXT4			<u>External Interrupt #4 Flag</u> Rd 0: active Wr 1: cleared
6	EXT3			<u>External Interrupt #3 Flag</u> Rd 0: active Wr 1: cleared
5	EXT2			<u>External Interrupt #2 Flag</u> Rd 0: active Wr 1: cleared
4	EXT1			<u>External Interrupt #1 Flag</u> Rd 0: active Wr 1: cleared
3	EXT0			<u>External Interrupt #0 Flag</u> Rd 0: active Wr 1: cleared
2	MMCS01			<u>MMC/SD Interrupt #1 Flag</u> Rd 0: active Wr 1: cleared
1	MMCS00			<u>MMC/SD Interrupt #0 Flag</u> Rd 0: active Wr 1: cleared
0	MT1			<u>Memory Traffic Controller #1 Interrupt Flag</u> Rd 0: active Wr 1: cleared

5.7.6 IRQ2 - IRQ Interrupt Flag Register #2

IRQ2		0003:050C		offset: 0x0C		default: 0x00FF													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSVINT	CLKC	I2C	WDT	PREV	EXT15	EXT14	EXT13			
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1			

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	RSVINT	1	R/W	Reserved Interrupt Flag Rd 0: active Wr 1: cleared
6	CLKC	1	R/W	<u>Clock Controller Interrupt Flag</u> Rd 0: active Wr 1: cleared
5	I2C	1	R/W	<u>I2C Interrupt Flag</u> Rd 0: active Wr 1: cleared
4	WDT	1	R/W	<u>Watch Dog Timer Interrupt Flag</u> Rd 0: active Wr 1: cleared
3	PREV	1	R/W	Preview Engine Interrupt Flag Rd 0: active Wr 1: cleared
2	EXT15	1	R/W	<u>External Interrupt #15 Flag</u> Rd 0: active Wr 1: cleared
1	EXT14	1	R/W	<u>External Interrupt #14 Flag</u> Rd 0: active Wr 1: cleared
0	EXT13	1	R/W	<u>External Interrupt #13 Flag</u> Rd 0: active Wr 1: cleared

5.7.7 FIQENTRY0 - FIQ Entry Address Register #0

[illegible][illegible]

5.7.8 FIQENTRY1 - FIQ Entry Address Register #1

[illegible][illegible]

5.7.9 IRQENTRY0 - IRQ Entry Address Register #0

[illegible][illegible]

5.7.10 IRQENTRY1 - IRQ Entry Address Register #1

[illegible][illegible]

5.7.11 FISEL0 - FIQ select register #0

FISEL0	0003:0520										offset:	0x20	default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	MTC0	USB	UART1	UART0	DSPHINT	EXTHOST	SP1	SP0	OSD	CCDVD2	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	MTC0	0	R/W	<u>Memory Traffic Controller FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
14	USB	0	R/W	<u>USB FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
13	UART1	0	R/W	<u>UART1 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
12	UART0	0	R/W	<u>UART0 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
11	DSPHINT	0	R/W	<u>DSP HINT FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
10	EXTHOST	0	R/W	<u>MMC/SD and MS FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
9	SP1	0	R/W	<u>Serial Port 1 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
8	SP0	0	R/W	<u>Serial Port 0 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
7	OSD	0	R/W	<u>OSD FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
6	CCDVD2	0	R/W	<u>CCD VD #2 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
5	CCDVD1	0	R/W	<u>CCD VD #1 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
4	CCDVD0	0	R/W	<u>CCD VD #0 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
3	TMR3	0	R/W	<u>Timer 3 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
2	TMR2	0	R/W	<u>Timer 2 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
1	TMR1	0	R/W	<u>Timer 1 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «
0	TMR0	0	R/W	<u>Timer 0 FIQ Interrupt Select</u> » 0: IRQ « » 1: FIQ «

5.7.12 FISEL1 - FIQ select register #1

FISEL1	0003:0522										offset:	0x22	default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	EXT12	EXT11	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCS01	MMCS00	MTC1		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	EXT12	0	R/W	<u>External Interrupt #12 FIQ Select</u> » 0: IRQ « » 1: FIQ «
14	EXT11	0	R/W	<u>External Interrupt #11 FIQ Select</u> » 0: IRQ « » 1: FIQ «
13	EXT10	0	R/W	<u>External Interrupt #10 FIQ Select</u> » 0: IRQ « » 1: FIQ «
12	EXT9	0	R/W	<u>External Interrupt #9 FIQ Select</u> » 0: IRQ « » 1: FIQ «
11	EXT8	0	R/W	<u>External Interrupt #8 FIQ Select</u> » 0: IRQ « » 1: FIQ «
10	EXT7	0	R/W	<u>External Interrupt #7 FIQ Select</u> » 0: IRQ « » 1: FIQ «
9	EXT6	0	R/W	<u>External Interrupt #6 FIQ Select</u> » 0: IRQ « » 1: FIQ «
8	EXT5	0	R/W	<u>External Interrupt #5 FIQ Select</u> » 0: IRQ « » 1: FIQ «
7	EXT4	0	R/W	<u>External Interrupt #4 FIQ Select</u> » 0: IRQ « » 1: FIQ «
6	EXT3	0	R/W	<u>External Interrupt #3 FIQ Select</u> » 0: IRQ « » 1: FIQ «
5	EXT2	0	R/W	<u>External Interrupt #2 FIQ Select</u> » 0: IRQ « » 1: FIQ «
4	EXT1	0	R/W	<u>External Interrupt #1 FIQ Select</u> » 0: IRQ « » 1: FIQ «
3	EXT0	0	R/W	<u>External Interrupt #0 FIQ Select</u> » 0: IRQ « » 1: FIQ «
2	MMCS01	0	R/W	<u>MMC/SD Interrupt #1 FIQ Select</u> » 0: IRQ « » 1: FIQ «
1	MMCS00	0	R/W	<u>MMC/SD Interrupt #0 FIQ Select</u> » 0: IRQ « » 1: FIQ «
0	MTC1	0	R/W	<u>Memory Traffic Controller #1 Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «

5.7.13 FISEL2 - FIQ select register #2

FISEL2				0003:0524				offset:		0x24				default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSVINT	CLKC	I2C	WDT	PREV	EXT15	EXT14	EXT13					
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	RSVINT	0	R/W	<u>Reserved Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «
6	CLKC	0	R/W	<u>Clock Controller Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «
5	I2C	0	R/W	<u>I2C Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «
4	WDT	0	R/W	<u>Watch Dog Timer Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «
3	PREV	0	R/W	<u>Preview Engine Interrupt FIQ Select</u> » 0: IRQ « » 1: FIQ «
2	EXT15	0	R/W	<u>External Interrupt #15 FIQ Select</u> » 0: IRQ « » 1: FIQ «
1	EXT14	0	R/W	<u>External Interrupt #14 FIQ Select</u> » 0: IRQ « » 1: FIQ «
0	EXT13	0	R/W	<u>External Interrupt #13 FIQ Select</u> » 0: IRQ « » 1: FIQ «

5.7.14 EINT0 - Interrupt Enable Register #0

EINT0	0003:0528												offset:		0x28				default:				0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	MTC0	USB	UART1	UART0	DSPHINT	EXTHOST	SP1	SP0	OSD	CCDVD2	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0								
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W								
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0								

Bit	Name	Reset Value	R/W	Function
15	MTC0	0	R/W	<u>Memory Traffic Controller #0 Interrupt Enable</u> » 0: disable « » 1: enable «
14	USB	0	R/W	<u>USB Interrupt Enable</u> » 0: disable « » 1: enable «
13	UART1	0	R/W	<u>UART1 Interrupt Enable</u> » 0: disable « » 1: enable «
12	UART0	0	R/W	<u>UART0 Interrupt Enable</u> » 0: disable « » 1: enable «
11	DSPHINT	0	R/W	<u>DSP HINT Interrupt Enable</u> » 0: disable « » 1: enable «
10	EXTHOST	0	R/W	<u>MMC/SD and MS Interrupt Enable</u> » 0: disable « » 1: enable «
9	SP1	0	R/W	<u>Serial Port 1 Interrupt Enable</u> » 0: disable « » 1: enable «
8	SP0	0	R/W	<u>Serial Port 0 Interrupt Enable</u> » 0: disable « » 1: enable «
7	OSD	0	R/W	<u>OSD Interrupt Enable</u> » 0: disable « » 1: enable «
6	CCDVD2	0	R/W	<u>CCD_VD Interrupt#2 Enable</u> » 0: disable « » 1: enable «
5	CCDVD1	0	R/W	<u>CCD_VD Interrupt#1 Enable</u> » 0: disable « » 1: enable «
4	CCDVD0	0	R/W	<u>CCD_VD Interrupt#0 Enable</u> » 0: disable « » 1: enable «
3	TMR3	0	R/W	<u>Timer 3 Interrupt Enable</u> » 0: disable « » 1: enable «
2	TMR2	0	R/W	<u>Timer 2 Interrupt Enable</u> » 0: disable « » 1: enable «
1	TMR1	0	R/W	<u>Timer 1 Interrupt Enable</u> » 0: disable « » 1: enable «
0	TMR0	0	R/W	<u>Timer 0 Interrupt Enable</u> » 0: disable « » 1: enable «

5.7.15 EINT1 - Interrupt Enable Register #1

EINT1 0003:052A															offset: 0x2A	default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	EXT12	EXT11	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCS01	MMCS00	MT1		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	EXT12	0	R/W	<u>External Interrupt #12 Enable</u> » 0: disable « » 1: enable «
14	EXT11	0	R/W	<u>External Interrupt #11 Enable</u> » 0: disable « » 1: enable «
13	EXT10	0	R/W	<u>External Interrupt #10 Enable</u> » 0: disable « » 1: enable «
12	EXT9	0	R/W	<u>External Interrupt #9 Enable</u> » 0: disable « » 1: enable «
11	EXT8	0	R/W	<u>External Interrupt #8 Enable</u> » 0: disable « » 1: enable «
10	EXT7	0	R/W	<u>External Interrupt #7 Enable</u> » 0: disable « » 1: enable «
9	EXT6	0	R/W	<u>External Interrupt #6 Enable</u> » 0: disable « » 1: enable «
8	EXT5	0	R/W	<u>External Interrupt #5 Enable</u> » 0: disable « » 1: enable «
7	EXT4	0	R/W	<u>External Interrupt #4 Enable</u> » 0: disable « » 1: enable «
6	EXT3	0	R/W	<u>External Interrupt #3 Enable</u> » 0: disable « » 1: enable «
5	EXT2	0	R/W	<u>External Interrupt #2 Enable</u> » 0: disable « » 1: enable «
4	EXT1	0	R/W	<u>External Interrupt #1 Enable</u> » 0: disable « » 1: enable «
3	EXT0	0	R/W	<u>External Interrupt #0 Enable</u> » 0: disable « » 1: enable «
2	MMCS01	0	R/W	<u>MMC/SD Interrupt #1 Enable</u> » 0: disable « » 1: enable «
1	MMCS00	0	R/W	<u>MMC/SD Interrupt #0 Enable</u> » 0: disable « » 1: enable «
0	MT1	0	R/W	<u>Memory Traffic Controller #1 Interrupt Enable</u> » 0: disable « » 1: enable «

5.7.16 EINT2 - Interrupt Enable Register #2

EINT2 0003:052C offset: 0x2C default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSVINT	CLKC	I2C	WDT	PREV	EXT15	EXT14	EXT13
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	RSVINT	0	R/W	<u>Reserved Interrupt Enable</u> » 0: disable « » 1: enable «
6	CLKC	0	R/W	<u>Clock Controller Interrupt Enable</u> » 0: disable « » 1: enable «
5	I2C	0	R/W	<u>I2C Interrupt Enable</u> » 0: disable « » 1: enable «
4	WDT	0	R/W	<u>Watch Dog Timer Interrupt Enable</u> » 0: disable « » 1: enable «
3	PREV	0	R/W	<u>Preview Engine Interrupt Enable</u> » 0: disable « » 1: enable «
2	EXT15	0	R/W	<u>External Interrupt #15 Enable</u> » 0: disable « » 1: enable «
1	EXT14	0	R/W	<u>External Interrupt #14 Enable</u> » 0: disable « » 1: enable «
0	EXT13	0	R/W	<u>External Interrupt #13 Enable</u> » 0: disable « » 1: enable «

5.7.17 INTRAW - Interrupt Raw Register

INTRAW		0003:0530				offset: 0x30		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	IRQ	FIQ			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0			

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	IRQ	0	R/W	Set to see if the interrupt masked in IRQENTRY1,IRQENTRY0 register is reflected or not. » 0: Does not reflect « » 1: Reflects «
0	FIQ	0	R/W	Set to see if the interrupt masked in the FIQENTRY0,FIQENTRY1 register is reflected or not. » 0: Does not reflect « » 1: Reflects «

5.7.18 EABASE0 - Entry Table Base Address Register #0

EABASE0		0003:0538				offset: 0x38		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	EABL[12]	EABL[11]	EABL[10]	EABL[9]	EABL[8]	EABL[7]	EABL[6]	EABL[5]	EABL[4]	EABL[3]	EABL[2]	EABL[1]	EABL[0]	RSV	TSIZE[1]	TSIZE[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	-	0	0			

Bit	Name	Reset Value	R/W	Function
15:3	EABL	0	R/W	Entry Address Base of Interrupt Vector Table - Low - Bits 15:3 of the lower 16-bits of the entry address of interrupt vector table. - Bits 2:0 are interpreted as zero - interrupt entry table is aligned on an 8-byte boundary.
2	RSV			Reserved
1:0	TSIZE	0	R/W	Table Size The size of each interrupt entry in the table » 00: 4 Bytes « » 01: 8 Bytes « » 10: 16 Bytes « » 11: 32 Bytes «

5.7.19 EABASE1 - Entry Table Base Address Register #1

EABASE1	0003:053A			offset:	0x3A			default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	EABH[12]	EABH[11]	EABH[10]	EABH[9]	EABH[8]	EABH[7]	EABH[6]	EABH[5]	EABH[4]	EABH[3]	EABH[2]	EABH[1]	EABH[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:0	EABH	0	R/W	Entry Address Base of Interrupt Vector Table - High Bits 28:16 of the entry address of interrupt vector table

5.7.20 INTPRI00 - Interrupt Priority Register #0

INTPRI00	0003:0540			offset:	0x40			default: 0x0100										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI01[5]	PRI01[4]	PRI01[3]	PRI01[2]	PRI01[1]	PRI01[0]	RSV	RSV	PRI00[5]	PRI00[4]	PRI00[3]	PRI00[2]	PRI00[1]	PRI00[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	0	0	0	0	1	-	-	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 2nd highest priority				
» 000000: TMR0 « ; » 000001: TMR1 « ;				
» 000010: TMR2 « ; » 000011: TMR3 « ;				
» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;				
» 000110: CCDVD2 « ; » 000111: OSD « ;				
» 001000: SP0 « ; » 001001: SP1 « ;				
» 001010: EXTHOST « ; » 001011: DSPHINT « ;				
» 001100: UART0 « ; » 001101: UART1 « ;				
» 001110: USB « ;				
» 001111: MTC0 « ; » 010000: MTC1 « ;				
» 010001: MMCS0 « ; » 010010: MMCS01 « ;				
» 010011: EXT0 « ; » 010100: EXT1 « ;				
» 010101: EXT2 « ; » 010110: EXT3 « ;				
» 010111: EXT4 « ; » 011000: EXT5 « ;				
» 011001: EXT6 « ; » 011010: EXT7 « ;				
» 011011: EXT8 « ; » 011100: EXT9 « ;				
» 011101: EXT10 « ; » 011110: EXT11 « ;				
» 011111: EXT12 « ; » 100000: EXT13 « ;				
» 100001: EXT14 « ; » 100010: EXT15 « ;				
» 100011: PREV « ; » 100100: WDT « ;				
» 100101: I2C « ; » 100110: CLKC « ;				
» 100111: RSVINT «				
7:6	RSV			Reserved
Interrupt Factor with highest priority				
» 000000: TMR0 « ; » 000001: TMR1 « ;				
» 000010: TMR2 « ; » 000011: TMR3 « ;				
» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;				
» 000110: CCDVD2 « ; » 000111: OSD « ;				
» 001000: SP0 « ; » 001001: SP1 « ;				
» 001010: EXTHOST « ; » 001011: DSPHINT « ;				
» 001100: UART0 « ; » 001101: UART1 « ;				
» 001110: USB « ;				
» 001111: MTC0 « ; » 010000: MTC1 « ;				
» 010001: MMCS0 « ; » 010010: MMCS01 « ;				
» 010011: EXT0 « ; » 010100: EXT1 « ;				
» 010101: EXT2 « ; » 010110: EXT3 « ;				
» 010111: EXT4 « ; » 011000: EXT5 « ;				
» 011001: EXT6 « ; » 011010: EXT7 « ;				
» 011011: EXT8 « ; » 011100: EXT9 « ;				
» 011101: EXT10 « ; » 011110: EXT11 « ;				
» 011111: EXT12 « ; » 100000: EXT13 « ;				
» 100001: EXT14 « ; » 100010: EXT15 « ;				
» 100011: PREV « ; » 100100: WDT « ;				
» 100101: I2C « ; » 100110: CLKC « ;				
» 100111: RSVINT «				

5.7.21 INTPRI01 - Interrupt Priority Register #1

INTPRI01				0003:0542								offset:		0x42								default:		0x0302	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	PRI03[5]	PRI03[4]	PRI03[3]	PRI03[2]	PRI03[1]	PRI03[0]	RSV	RSV	PRI02[5]	PRI02[4]	PRI02[3]	PRI02[2]	PRI02[1]	PRI02[0]									
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W									
Default	-	-	0	0	0	0	0	0	1	-	0	0	0	0	1	0									

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 4th highest priority				
13:8	PRI 03	3	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 3rd highest priority				
5:0	PRI 02	2	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.22 INTPRI02 - Interrupt Priority Register #2

INTPRI02	0003:0544										offset:	0x44						default:						0x0504	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	RSV	PRI05[5]	PRI05[4]	PRI05[3]	PRI05[2]	PRI05[1]	PRI05[0]	RSV	RSV	PRI04[5]	PRI04[4]	PRI04[3]	PRI04[2]	PRI04[1]	PRI04[0]								
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W								
Default	-	-	-	0	0	0	1	0	1	-	-	0	0	0	1	0	0								

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 6th highest priority				
13:8	PRI 05	5	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 5th highest priority				
5:0	PRI 04	4	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.23 INTPRI03 - Interrupt Priority Register #3

INTPRI03		0003:0546				offset:		0x46								default:		0x0706	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PRI07[5]	PRI07[4]	PRI07[3]	PRI07[2]	PRI07[1]	PRI07[0]	RSV	RSV	PRI06[5]	PRI06[4]	PRI06[3]	PRI06[2]	PRI06[1]	PRI06[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	0	0	1	1	1	-	-	0	0	0	1	1	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 8th highest priority				
13:8	PRI 07	7	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 7th highest priority				
5:0	PRI 06	6	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.24 INTPRI04 - Interrupt Priority Register #4

INTPRI04																0003:0548		offset: 0x48																		default: 0x0908	
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0						
Name	RSV		RSV		PRI09[5]		PRI09[4]		PRI09[3]		PRI09[2]		PRI09[1]		PRI09[0]		RSV		RSV		PRI08[5]		PRI08[4]		PRI08[3]		PRI08[2]		PRI08[1]		PRI08[0]						
R/W	-		-		R/W		R/W		R/W		R/W		R/W		R/W		-		-		R/W		R/W		R/W		R/W		R/W		R/W						
Default	-		-		0		0		1		0		0		1		-		-		0		0		1		0		0		0						

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 10th highest priority				
13:8	PRI 09	9	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 9th highest priority				
5:0	PRI 08	8	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.25 INTPRI05 - Interrupt Priority Register #5

INTPRI05		0003:054A				offset:		0x4A								default:		0x0B0A	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PRI1[5]	PRI1[4]	PRI1[3]	PRI1[2]	PRI1[1]	PRI1[0]	RSV	RSV	PRI0[5]	PRI0[4]	PRI0[3]	PRI0[2]	PRI0[1]	PRI0[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	0	1	0	1	1	-	-	0	0	1	0	1	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 12th highest priority				
13:8	PRI 11	11	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 11th highest priority				
5:0	PRI 10	10	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.26 INTPRI06 - Interrupt Priority Register #6

INTPRI06														0003:054C				offset: 0x4C								default: 0x0D0C			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0													
Name	RSV	RSV	PRI1[5]	PRI1[4]	PRI1[3]	PRI1[2]	PRI1[1]	PRI1[0]	RSV	RSV	PRI2[5]	PRI2[4]	PRI2[3]	PRI2[2]	PRI2[1]	PRI2[0]													
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W													
Default	-	-	0	0	1	1	0	1	-	-	0	0	1	1	0	0													

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 14th highest priority				
13:8	PRI 13	13	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 13th highest priority				
5:0	PRI 12	12	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.27 INTPRI07 - Interrupt Priority Register #7

INTPRI07		0003:054E				offset: 0x4E												default: 0x0F0E	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PRI15[5]	PRI15[4]	PRI15[3]	PRI15[2]	PRI15[1]	PRI15[0]	RSV	RSV	PRI14[5]	PRI14[4]	PRI14[3]	PRI14[2]	PRI14[1]	PRI14[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	0	1	1	1	1	-	-	0	0	1	1	1	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
				Interrupt Factor with 16th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
13:8	PRI 15	15	R/W	
				Interrupt Factor with 15th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
				Interrupt Factor with 14th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
5:0	PRI 14	14	R/W	

5.7.28 INTPRI08 - Interrupt Priority Register #8

INTPRI08																0003:0550		offset: 0x50																		default: 0x1110	
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0						
Name	RSV		RSV		PRI17[5]		PRI17[4]		PRI17[3]		PRI17[2]		PRI17[1]		PRI17[0]		RSV		RSV		PRI16[5]		PRI16[4]		PRI16[3]		PRI16[2]		PRI16[1]		PRI16[0]						
R/W	-		-		R/W		R/W		R/W		R/W		R/W		R/W		-		-		R/W		R/W		R/W		R/W		R/W		R/W						
Default	-		-		0		1		0		0		0		1		-		-		0		1		0		0		0		0						

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
				Interrupt Factor with 18th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
13:8	PRI 17	17	R/W	
				Interrupt Factor with 17th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
				Interrupt Factor with 16th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
5:0	PRI 16	16	R/W	

5.7.29 INTPRI09 - Interrupt Priority Register #9

INTPRI09																0003:0552		offset: 0x52																		default: 0x1312	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																					
Name	RSV	RSV	PRI19[5]	PRI19[4]	PRI19[3]	PRI19[2]	PRI19[1]	PRI19[0]	RSV	RSV	PRI18[5]	PRI18[4]	PRI18[3]	PRI18[2]	PRI18[1]	PRI18[0]																					
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W																					
Default	-	-	0	1	0	0	1	1	-	-	0	1	0	0	1	0																					

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 20th highest priority				
13:8	PRI 19	19	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
» 011111: EXT12 « ; » 100000: EXT13 « ;				
» 100001: EXT14 « ; » 100010: EXT15 « ;				
» 100011: PREV « ; » 100100: WDT « ;				
» 100101: I2C « ; » 100110: CLKC « ;				
» 100111: RSVINT «				
7:6	RSV			Reserved
Interrupt Factor with 19th highest priority				
5:0	PRI 18	18	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.30 INTPRI10 - Interrupt Priority Register #10

INTPRI10															0003:0554		offset: 0x54																	default: 0x1514	
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0				
Name	RSV		RSV		PRI21[5]		PRI21[4]		PRI21[3]		PRI21[2]		PRI21[1]		PRI21[0]		RSV		RSV		PRI20[5]		PRI20[4]		PRI20[3]		PRI20[2]		PRI20[1]		PRI20[0]				
R/W	-		-		R/W		R/W		R/W		R/W		R/W		R/W		-		-		R/W		R/W		R/W		R/W		R/W		R/W				
Default	-		-		0		1		0		1		0		1		-		-		0		1		0		1		0		0				

Bit	Name	Reset Value	R/W	Function				
15: 14	RSV			Reserved				
Interrupt Factor with 22nd highest priority								
13: 8	PRI 21	21	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;				
				» 000010: TMR2 « ; » 000011: TMR3 « ;				
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;				
				» 000110: CCDVD2 « ; » 000111: OSD « ;				
				» 001000: SP0 « ; » 001001: SP1 « ;				
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;				
				» 001100: UART0 « ; » 001101: UART1 « ;				
				» 001110: USB « ;				
				» 001111: MTC0 « ; » 010000: MTC1 « ;				
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;				
				» 010011: EXT0 « ; » 010100: EXT1 « ;				
				» 010101: EXT2 « ; » 010110: EXT3 « ;				
				» 010111: EXT4 « ; » 011000: EXT5 « ;				
				» 011001: EXT6 « ; » 011010: EXT7 « ;				
				» 011011: EXT8 « ; » 011100: EXT9 « ;				
				» 011101: EXT10 « ; » 011110: EXT11 « ;				
				» 011111: EXT12 « ; » 100000: EXT13 « ;				
				» 100001: EXT14 « ; » 100010: EXT15 « ;				
				» 100011: PREV « ; » 100100: WDT « ;				
				» 100101: I2C « ; » 100110: CLKC « ;				
				» 100111: RSVINT «				
				7: 6	RSV			Reserved
				Interrupt Factor with 21st highest priority				
				5: 0	PRI 20	20	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
» 000010: TMR2 « ; » 000011: TMR3 « ;								
» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;								
» 000110: CCDVD2 « ; » 000111: OSD « ;								
» 001000: SP0 « ; » 001001: SP1 « ;								
» 001010: EXTHOST « ; » 001011: DSPHINT « ;								
» 001100: UART0 « ; » 001101: UART1 « ;								
» 001110: USB « ;								
» 001111: MTC0 « ; » 010000: MTC1 « ;								
» 010001: MMCS0 « ; » 010010: MMCS1 « ;								
» 010011: EXT0 « ; » 010100: EXT1 « ;								
» 010101: EXT2 « ; » 010110: EXT3 « ;								
» 010111: EXT4 « ; » 011000: EXT5 « ;								
» 011001: EXT6 « ; » 011010: EXT7 « ;								
» 011011: EXT8 « ; » 011100: EXT9 « ;								
» 011101: EXT10 « ; » 011110: EXT11 « ;								
» 011111: EXT12 « ; » 100000: EXT13 « ;								
» 100001: EXT14 « ; » 100010: EXT15 « ;								
» 100011: PREV « ; » 100100: WDT « ;								
» 100101: I2C « ; » 100110: CLKC « ;								
» 100111: RSVINT «								

5.7.31 INTPRI11 - Interrupt Priority Register #11

INTPRI11																0003:0556		offset:		0x56																		default:		0x1716	
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0										
Name	RSV		RSV		PRI2[5]		PRI2[4]		PRI2[3]		PRI2[2]		PRI2[1]		PRI2[0]		RSV		RSV		PRI2[5]		PRI2[4]		PRI2[3]		PRI2[2]		PRI2[1]		PRI2[0]										
R/W	-		-		R/W		R/W		R/W		R/W		R/W		R/W		-		-		R/W		R/W		R/W		R/W		R/W		R/W										
Default	-		-		0		1		0		1		1		1		1		-		-		0		1		0		1		1		0								

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 24th highest priority				
13:8	PRI 23	23	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 23rd highest priority				
5:0	PRI 22	22	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.32 INTPRI12 - Interrupt Priority Register #12

INTPRI12															0003:0558		offset: 0x58																	default: 0x1918	
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0				
Name	RSV		RSV		PRI25[5]		PRI25[4]		PRI25[3]		PRI25[2]		PRI25[1]		PRI25[0]		RSV		RSV		PRI24[5]		PRI24[4]		PRI24[3]		PRI24[2]		PRI24[1]		PRI24[0]				
R/W	-		-		R/W		R/W		R/W		R/W		R/W		R/W		-		-		R/W		R/W		R/W		R/W		R/W		R/W				
Default	-		-		0		1		1		0		0		1		-		-		0		1		1		0		0		0				

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 26th highest priority				
13:8	PRI 25	25	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 25th highest priority				
5:0	PRI 24	24	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.33 INTPRI13 - Interrupt Priority Register #13

INTPRI13		0003:055A				offset: 0x5A		default: 0x1B1A											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	PRI27[5]	PRI27[4]	PRI27[3]	PRI27[2]	PRI27[1]	PRI27[0]	RSV	RSV	PRI26[5]	PRI26[4]	PRI26[3]	PRI26[2]	PRI26[1]	PRI26[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	1	0	1	1	1	-	-	0	1	1	0	1	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 28th highest priority				
13:8	PRI 27	27	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 27th highest priority				
5:0	PRI 26	26	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.34 INTPRI14 - Interrupt Priority Register #14

INTPRI14		0003:055C				offset: 0x5C		default: 0x1D1C											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	PRI29[5]	PRI29[4]	PRI29[3]	PRI29[2]	PRI29[1]	PRI29[0]	RSV	RSV	PRI28[5]	PRI28[4]	PRI28[3]	PRI28[2]	PRI28[1]	PRI28[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	0	1	1	1	0	1	-	-	0	1	1	1	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 30th highest priority				
13:8	PRI 29	29	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 29th highest priority				
5:0	PRI 28	28	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS01 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.35 INTPRI15 - Interrupt Priority Register #15

INTPRI15		0003:055E				offset: 0x5E								default: 0x1F1E			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	1	1	1	1	1	-	-	0	1	1	1	1	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
				Interrupt Factor with 32nd highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
13:8	PRI 31	31	R/W	
				Interrupt Factor with 31st highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
				Interrupt Factor with 30th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
5:0	PRI 30	30	R/W	

5.7.36 INTPRI16 - Interrupt Priority Register #16

INTPRI16		0003:0560				offset: 0x60				default: 0x2120							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	1	0	0	0	0	1	-	-	1	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
				Interrupt Factor with 34th highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
13:8	PRI 33	33	R/W	
				Interrupt Factor with 33rd highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
				Interrupt Factor with 32nd highest priority
				» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
5:0	PRI 32	32	R/W	

5.7.37 INTPRI17 - Interrupt Priority Register #17

INTPRI17	0003:0562				offset: 0x62				default: 0x2322											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]				
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	1	0	0	0	1	1	-	-	1	0	0	0	1	0				

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 36th highest priority				
13:8	PRI 35	35	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 35th highest priority				
5:0	PRI 34	34	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.38 INTPRI18 - Interrupt Priority Register #18

INTPRI18	0003:0564				offset: 0x64				default: 0x2524											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]	RSV	RSV	PRI3[5]	PRI3[4]	PRI3[3]	PRI3[2]	PRI3[1]	PRI3[0]				
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	1	0	0	1	0	1	-	-	1	0	0	1	0	0				

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
Interrupt Factor with 38th highest priority				
13:8	PRI 37	37	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
Interrupt Factor with 37th highest priority				
5:0	PRI 36	36	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

5.7.39 INTPRI19 - Interrupt Priority Register #19

INTPRI19	0003:0566												offset: 0x66						default: 0x2726	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	PRI39[5]	PRI39[4]	PRI39[3]	PRI39[2]	PRI39[1]	PRI39[0]	RSV	RSV	PRI38[5]	PRI38[4]	PRI38[3]	PRI38[2]	PRI38[1]	PRI38[0]				
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	1	0	0	1	1	1	-	-	1	0	0	1	1	0				

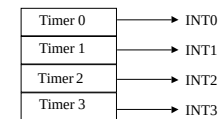
Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
				Interrupt Factor with 40th highest priority
13:8	PRI 39	39	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «
7:6	RSV			Reserved
				Interrupt Factor with 39th highest priority
5:0	PRI 38	38	R/W	» 000000: TMR0 « ; » 000001: TMR1 « ;
				» 000010: TMR2 « ; » 000011: TMR3 « ;
				» 000100: CCDVD0 « ; » 000101: CCDVD1 « ;
				» 000110: CCDVD2 « ; » 000111: OSD « ;
				» 001000: SP0 « ; » 001001: SP1 « ;
				» 001010: EXTHOST « ; » 001011: DSPHINT « ;
				» 001100: UART0 « ; » 001101: UART1 « ;
				» 001110: USB « ;
				» 001111: MTC0 « ; » 010000: MTC1 « ;
				» 010001: MMCS0 « ; » 010010: MMCS1 « ;
				» 010011: EXT0 « ; » 010100: EXT1 « ;
				» 010101: EXT2 « ; » 010110: EXT3 « ;
				» 010111: EXT4 « ; » 011000: EXT5 « ;
				» 011001: EXT6 « ; » 011010: EXT7 « ;
				» 011011: EXT8 « ; » 011100: EXT9 « ;
				» 011101: EXT10 « ; » 011110: EXT11 « ;
				» 011111: EXT12 « ; » 100000: EXT13 « ;
				» 100001: EXT14 « ; » 100010: EXT15 « ;
				» 100011: PREV « ; » 100100: WDT « ;
				» 100101: I2C « ; » 100110: CLKC « ;
				» 100111: RSVINT «

6 Timers

6.1 Introduction

The TMS320DM270 ARM subsystem has four timers. The output of each timer is input into an ARM interrupt, as shown in Figure 19. Timer 0 is connected to interrupt 0, timer 1 is connected to interrupt 1, timer 2 is connected to interrupt 2, and timer 3 is connected to interrupt 3. The clock provided to timer module decrements a counter. When the counter value reaches zero an interrupt is generated.

Figure 19. ARM timer interrupts



6.2 Configuring the Timers

All timers (Timer 0 to Timer 4) are identical in their function. In this section, 'x' can take on values from 0 through 3, indicating timer 0 through timer 3, respectively Figure 20 shows the function block diagram for timer 0.

Each of the general-purpose timers can run in one of two modes: one-shot mode and free-run mode.

In one-shot mode, an interrupt occurs only once when the timer count reaches zero and then the timer must be explicitly reset to begin the timing operation again.

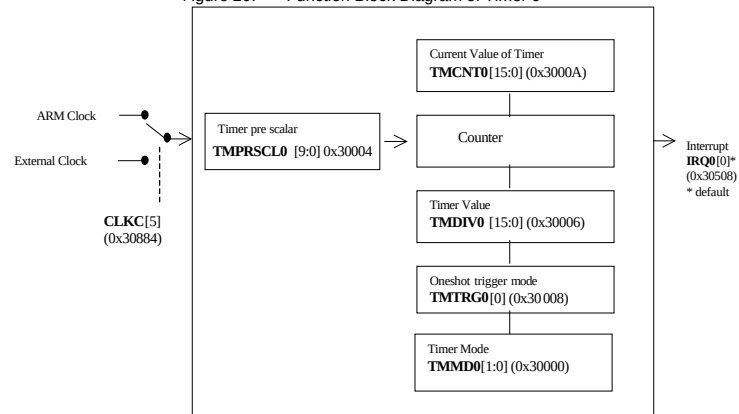
In free-run mode, when the timer generates an interrupt, the timer counter is automatically reloaded to start the count operation again.

Either the ARM clock or an external clock (PLL input clock) can be selected as the clock source of the timer. Bit field CTMxS in Register **CLKC** (0003:0880h) in the "Clock Controller" module can be used to select clock sources for timers. The actual count operation is equal to the value set in the bit field PRSCL (10-bits) of the register TMPRSCLx – Timer Pre-Scalar Registers times the value set in the bit field DIV (16-bits) of the register TMDIVx – Timer Divider Registers. DIV is the source clock frequency divide factor and PRSCL is used to scale the source clock frequency divide factor. Therefore frequency of a timer interrupt is given by the following equation:

$$\text{Timer Interrupt Frequency} = \frac{\text{Source Clock Frequency}}{(\text{PRSCL} + 1)(\text{DIV} + 1)}$$

When a timer is set to one-shot mode, the timer is started by writing a “1” to the bit TRG in the register TMTRGx – Timer Trigger Registers.

Figure 20. Function Block Diagram of Timer 0



6.3 Timer Register Map (TMRx)

6.3.1 Timer 0 Register Map

Address	Register	Description
0003:0000	TMMD0	Timer 0 Mode
0003:0002	TMRSV0	Reserved
0003:0004	TMPSCL0	Timer 0 Prescaler
0003:0006	TMDIV0	Timer 0 Divisor (count)
0003:0008	TMTRG0	Timer 0 One-Shot Trigger
0003:000A	TMCNT0	Timer 0 Count

6.3.2 Timer 1 Register Map

Address	Register	Description
0003:0000	TMMD1	Timer 1 Mode
0003:0002	TMRSV1	Reserved
0003:0004	TMPSCL1	Timer 1 Prescaler
0003:0006	TMDIV1	Timer 1 Divisor (count)
0003:0008	TMTRG1	Timer 1 One-Shot Trigger
0003:000A	TMCNT1	Timer 1 Count

6.3.3 Timer 2 Register Map

Address	Register	Description
0003:0100	TMMD2	Timer 2 Mode
0003:0102	TMRSV2	Reserved
0003:0104	TMPSCL2	Timer 2 Prescaler
0003:0106	TMDIV2	Timer 2 Divisor (count)
0003:0108	TMTRG2	Timer 2 One-Shot Trigger
0003:010A	TMCNT2	Timer 2 Count

6.3.4 Timer 3 Register Map

Address	Register	Description
0003:0180	TMMD3	Timer 3 Mode
0003:0182	TMRSV3	Reserved
0003:0184	TMPSCL3	Timer 3 Prescaler
0003:0186	TMDIV3	Timer 3 Divisor (count)
0003:0188	TMTRG3	Timer 3 One-Shot Trigger
0003:018A	TMCNT3	Timer 3 Count

6.4 Timer Registers

6.4.1 TMMDx – Timer Mode Registers

TMMD0	0003:0000	offset:	0x00													default:	0x0000
TMMD1	0003:0080	offset:	0x00														
TMMD2	0003:0100	offset:	0x00														
TMMD3	0003:0180	offset:	0x00														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TEST[5]	TEST[4]	TEST[3]	TEST[2]	TEST[1]	TEST[0]	MODE[1]	MODE[0]	
R/W	-	-	-	-	-	-	-	-	W	W	W	W	W	W	W	W	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	TEST	0	W	Test Bits - set to 0
1:0	MODE	0	W	<u>Timer Mode</u> » 00: stop « » 01: one shot « » 10: free run « » 11: n/a «
				One Shot: Count is started when 1 is written in one shot trigger bit. When set value is reached, interrupt occurs and count operation is stopped. Free Run: Count is repeated up to set value. Each time set value is reached, interrupt occurs.

6.4.2 TMPRSClX – Timer Pre-Scalar Registers

TMPSCL0	0003:0004	offset:	0x04													default:	0x0100
TMPSCL1	0003:0084	offset:	0x04														
TMPSCL2	0003:0104	offset:	0x04														
TMPSCL3	0003:0184	offset:	0x04														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSCl[9]	PRSCl[8]	PRSCl[7]	PRSCl[6]	PRSCl[5]	PRSCl[4]	PRSCl[3]	PRSCl[2]	PRSCl[1]	PRSCl[0]	
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	PRSCl	256	R/W	<u>Timer Prescaler Divide Value</u> Timer performs count operation with input clock frequency divided by (PRSCl+1)

6.4.3 TMDIVx – Timer Divider Registers

TMDIV0	0003:0006	offset:	0x06													default:	0x1FFF
TMDIV1	0003:0086	offset:	0x06														
TMDIV2	0003:0106	offset:	0x06														
TMDIV3	0003:0186	offset:	0x06														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	DIV	8191	R/W	Timer Division ratio Timer count value is (PRSCl+1)x(DIV+1)

6.4.4 TMTRGx – Timer Trigger Registers

TMTRG0	0003:0008	offset:	0x08													default:	0x0000
TMTRG1	0003:0088	offset:	0x08														
TMTRG2	0003:0108	offset:	0x08														
TMTRG3	0003:0188	offset:	0x08														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	TRG	0	W	<u>One Shot trigger</u> Trigger bit when timer is set to one shot mode. When 1 is written, timer count begins.

6.4.5 TMCNTx – Timer Count Registers

TMCNT0	0003:000A	offset:	0x0A													default:	0x0000
TMCNT1	0003:008A	offset:	0x0A														
TMCNT2	0003:010A	offset:	0x0A														
TMCNT3	0003:018A	offset:	0x0A														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	CNT[15]	CNT[14]	CNT[13]	CNT[12]	CNT[11]	CNT[10]	CNT[9]	CNT[8]	CNT[7]	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	CNT	0	R	Current counter value

7.4 Watch Dog Timer Register Map (WDT)

Address	Register	Description
0003:0400	WDTMD	Watch Dog Timer Mode
0003:0402	WDRST	Watch Dog Timer Reset
0003:0404	WDTPRCL	Watch Dog Timer Prescaler
0003:0406	WDTDIV	Watch Dog Timer Divisor (count)
0003:0408	WDEXRST	Watch Dog Timer External Reset

7.5 Watch Dog Timer Registers

7.5.1 WDTMD – WDT Mode Register

WDTMD		0003:0400								offset:		0x00								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TEST[4]	TEST[3]	TEST[2]	TEST[1]	TEST[0]	OPSEL	TEST0	ENABLE							
R/W	-	-	-	-	-	-	-	-	W	W	W	W	W	W	W	W							
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0							

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:3	TEST	0	W	Test Bits - set to 0 <u>Watch dog timer output selection</u> » 0: Generate WDT interrupt « » 1: Reset ARM «
2	OPSEL	0	W	Defines action taken if a write to WDRST is not performed within specified time.
1	TEST0	0	W	Test Bits - set to 0 <u>Watch Dog Timer enable</u> » 0: disable « » 1: enable «
0	ENABLE	0	W	

7.5.2 WDRST – WDT Status Register

WDRST	0003:0402												offset: 0x02				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	WDRST				
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W				
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0				

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	WDRST	0	W	<u>Watch Dog Timer Reset</u> » 0: n/a « » 1: Reset Watch Dog Timer «

7.5.3 WDTPRCL – WDT Pre-Scalar Register

WDTPRCL				0003:0404				offset: 0x04												default: 0x0100			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]							
R/W	-	-	-	-	-	-	W	W	W	W	W	W	W	W	W	W							
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0							

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	PRSC	256	W	<u>Watch Dog Timer Prescaler Divide Value</u> Timer performs count operation with 27 MHz clock divided by (PRSC+1)

7.5.4 WDTDIV – WDT Divider Register

WDTDIV		0003:0406												offset: 0x06		default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	DIV	0	R/W	Watch Dog Timer Timer Division ratio Timer's count is (PRSCl+1)x(DIV+1)

7.5.5 WDTXRST – WDT External Reset Register

WDTXRST					0003:0408					offset: 0x08					default: 0x0001				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	WERST			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1			

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	WERST	1	R/W	Watch Dog Timer External Reset External reset output pin (when GI025 MRST is set) » 0: External reset output pin is forced low « » 1: Reset signal to ARM output to Ext. Reset out pin «

8 Address Trace Module

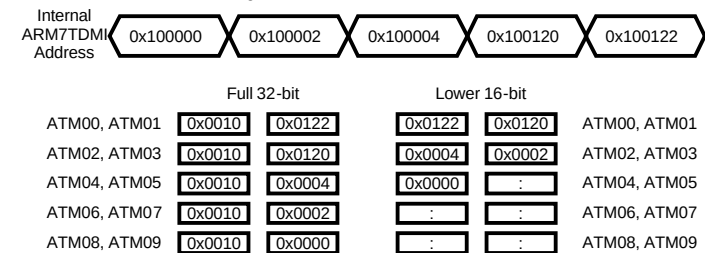
8.1 Introduction

Address Trace Module (ATM) provides trace function for internal ARM7TDMI processor. ATM has 62 sets of 16-bit registers and it stores address, which internal ARM7TDMI outputs, into these registers. The programmer can check history of internal ARM7TDMI address. Address, which is stored into ATM, is only the instruction address, including addresses for instructions pre-fetched by ARM7. Addresses for data is not taken. Buffer register size is 31 sets of full 32-bit address or 62 sets of lower 16-bit address. Stored address is last 31 or 62 sets of address. Figure 22 shows ATM behavior and section 0 shows registers for ATM in DM270.

Note,

- LSB of address (A0) in ATM registers should be ignored since internal ARM7TDMI outputs non-meaning logic level when in thumb-mode (16-BIS).
- ATM does not support correct history storage once the program execution hits a breakpoint. ATM should be stopped using STOP bit in register ATMCTRL - ATM Control register before you see ATM registers in debugger after break.
- ATM can be used regardless of I-cache enable/disable.

Figure 22. ATM Behaviour



8.2 Address Trace Module Register Map (ATM)

Address	Register	Description
00030000	ATMCTRL	ATM Control register
00030004	ATM0	ARM Program Address: N-0 (upper) / N-0 (lower)
00030006	ATM01	ARM Program Address: N-0 (lower) / N-1 (lower)
00030008	ATM02	ARM Program Address: N-1 (upper) / N-2 (lower)
0003000A	ATM03	ARM Program Address: N-1 (lower) / N-3 (lower)
0003000C	ATM04	ARM Program Address: N-2 (upper) / N-4 (lower)
0003000E	ATM05	ARM Program Address: N-2 (lower) / N-5 (lower)
00030010	ATM06	ARM Program Address: N-3 (upper) / N-6 (lower)
00030012	ATM07	ARM Program Address: N-3 (lower) / N-7 (lower)
00030014	ATM08	ARM Program Address: N-4 (upper) / N-8 (lower)
00030016	ATM09	ARM Program Address: N-4 (lower) / N-9 (lower)
00030018	ATM10	ARM Program Address: N-5 (upper) / N-10 (lower)
0003001A	ATM11	ARM Program Address: N-5 (lower) / N-11 (lower)
0003001C	ATM12	ARM Program Address: N-6 (upper) / N-12 (lower)
0003001E	ATM13	ARM Program Address: N-6 (lower) / N-13 (lower)
00030020	ATM14	ARM Program Address: N-7 (upper) / N-14 (lower)
00030022	ATM15	ARM Program Address: N-7 (lower) / N-15 (lower)
00030024	ATM16	ARM Program Address: N-8 (upper) / N-16 (lower)
00030026	ATM17	ARM Program Address: N-8 (lower) / N-17 (lower)
00030028	ATM18	ARM Program Address: N-9 (upper) / N-18 (lower)
0003002A	ATM19	ARM Program Address: N-9 (lower) / N-19 (lower)
0003002C	ATM20	ARM Program Address: N-10 (upper) / N-20 (lower)
0003002E	ATM21	ARM Program Address: N-10 (lower) / N-21 (lower)
00030030	ATM22	ARM Program Address: N-11 (upper) / N-22 (lower)
00030032	ATM23	ARM Program Address: N-11 (lower) / N-23 (lower)
00030034	ATM24	ARM Program Address: N-12 (upper) / N-24 (lower)
00030036	ATM25	ARM Program Address: N-12 (lower) / N-25 (lower)
00030038	ATM26	ARM Program Address: N-13 (upper) / N-26 (lower)
0003003A	ATM27	ARM Program Address: N-13 (lower) / N-27 (lower)
0003003C	ATM28	ARM Program Address: N-14 (upper) / N-28 (lower)
0003003E	ATM29	ARM Program Address: N-14 (lower) / N-29 (lower)
00030040	ATM30	ARM Program Address: N-15 (upper) / N-30 (lower)
00030042	ATM31	ARM Program Address: N-15 (lower) / N-31 (lower)
00030044	ATM32	ARM Program Address: N-16 (upper) / N-32 (lower)
00030046	ATM33	ARM Program Address: N-16 (lower) / N-33 (lower)
00030048	ATM34	ARM Program Address: N-17 (upper) / N-34 (lower)
0003004A	ATM35	ARM Program Address: N-17 (lower) / N-35 (lower)
0003004C	ATM36	ARM Program Address: N-18 (upper) / N-36 (lower)
0003004E	ATM37	ARM Program Address: N-18 (lower) / N-37 (lower)
00030050	ATM38	ARM Program Address: N-19 (upper) / N-38 (lower)
00030052	ATM39	ARM Program Address: N-19 (lower) / N-39 (lower)
00030054	ATM40	ARM Program Address: N-20 (upper) / N-40 (lower)
00030056	ATM41	ARM Program Address: N-20 (lower) / N-41 (lower)
00030058	ATM42	ARM Program Address: N-21 (upper) / N-42 (lower)
0003005A	ATM43	ARM Program Address: N-21 (lower) / N-43 (lower)
0003005C	ATM44	ARM Program Address: N-22 (upper) / N-44 (lower)
0003005E	ATM45	ARM Program Address: N-22 (lower) / N-45 (lower)
00030060	ATM46	ARM Program Address: N-23 (upper) / N-46 (lower)
00030062	ATM47	ARM Program Address: N-23 (lower) / N-47 (lower)
00030064	ATM48	ARM Program Address: N-24 (upper) / N-48 (lower)
00030066	ATM49	ARM Program Address: N-24 (lower) / N-49 (lower)
00030068	ATM50	ARM Program Address: N-25 (upper) / N-50 (lower)
0003006A	ATM51	ARM Program Address: N-25 (lower) / N-51 (lower)
0003006C	ATM52	ARM Program Address: N-26 (upper) / N-52 (lower)
0003006E	ATM53	ARM Program Address: N-26 (lower) / N-53 (lower)
00030070	ATM54	ARM Program Address: N-27 (upper) / N-54 (lower)
00030072	ATM55	ARM Program Address: N-27 (lower) / N-55 (lower)
00030074	ATM56	ARM Program Address: N-28 (upper) / N-56 (lower)
00030076	ATM57	ARM Program Address: N-28 (lower) / N-57 (lower)
00030078	ATM58	ARM Program Address: N-29 (upper) / N-58 (lower)
0003007A	ATM59	ARM Program Address: N-29 (lower) / N-59 (lower)
0003007C	ATM60	ARM Program Address: N-30 (upper) / N-60 (lower)
0003007E	ATM61	ARM Program Address: N-30 (lower) / N-61 (lower)

8.3 Address Trace Module Registers

8.3.1 ATMCTRL - ATM Control register

ATMCTRL	0003:0D00								offset: 0x00				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	LOW16	RSV	RSV	RSV	RSV	RSV	RSV	RSV	STOP
R/W	-	-	-	-	-	-	-	R/W	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	LOW16	0	R/W	Save only the lower 16-bits of the ARM program addresses » 0: save 32-bit ARM addresses « » 1: save only lower 16-bits of ARM addresses « Example: The last ARM address before setting STOP : 0x100940 The 2nd last ARM address before setting STOP : 0x10093C [LOW16=0] – a pair of ATMxx registers shows 32bit width ARM address. ATM00 : 0x0910 ATM01 : 0x0940 [LOW16=1] – ATMxx registers shows lower 16bit of ARM address. ATM00 : 0x0940 (upper 16-bits omitted) ATM01 : 0x093C (upper 16-bits omitted)
7:1	RSV			
0	STOP	0	R/W	» 0: stores current ARM address into register « » 1: stop storing current ARM address « User program must set STOP bit at check point. When using debugger tool, STOP must set before opening any memory window to the registers because memory window read will overwrite the data in ATM00-ATM61 with it's unique command code.

8.3.2 ATM00 - ARM Program Address: N-0 (upper) / N-0 (lower)

ATM00		0003:0D04				offset: 0x04								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	ATM00[15]	ATM00[14]	ATM00[13]	ATM00[12]	ATM00[11]	ATM00[10]	ATM00[9]	ATM00[8]	ATM00[7]	ATM00[6]	ATM00[5]	ATM00[4]	ATM00[3]	ATM00[2]	ATM00[1]	ATM00[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	ATM00	0	R	ARM address history [LOW16=0] - Upper 16-bits of last N-0 ARM program address before setting STOP [LOW16=1] - Lower 16-bits of last N-0 ARM program address before setting STOP - Only the Instruction address is stored in this register - I-cache on/off does NOT affect the stored ARM address

8.3.3 ATM01 - ARM Program Address: N-0 (lower) / N-1 (lower)

ATM01	0003:0D06												offset: 0x06				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	ATM01[15]	ATM01[14]	ATM01[13]	ATM01[12]	ATM01[11]	ATM01[10]	ATM01[9]	ATM01[8]	ATM01[7]	ATM01[6]	ATM01[5]	ATM01[4]	ATM01[3]	ATM01[2]	ATM01[1]	ATM01[0]				
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:0	ATM01	0	R	ARM address history [LOW16=0] - Lower 16-bits of last N-0 ARM program address before setting STOP [LOW16=1] - Lower 16-bits of last N-1 ARM program address before setting STOP - Only the Instruction address is stored in this register - I-cache on/off does NOT affect the stored ARM address

NOTE: ATM02 – ATM61 registers have the same structure as ATM00, ATM01 registers

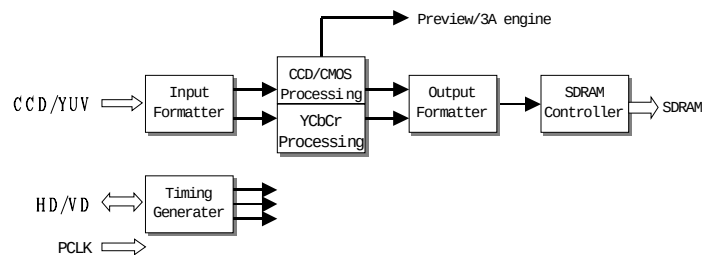
9 CCD Controller

The CCD Controller (CCDC) provides the necessary logic to interface CCD and CMOS imagers. In a typical DSC system, digital data from a CCD or CMOS imager is fed into the DSC engine for processing and manipulation. The CCD Controller captures and processes the physical data from the CCD sensor. The A/D converter and Timing Generator for CCD sensor are external to DM270 chip. The CCD controller can provide HD/VD to external timing generator or synchronize to HD/VD given from external timing generator.

9.1 Features

- Generates timing signals VD/HD and field ID
- Can synchronize on externally generated HD/VD signals
- Supports progressive scan and interlaced CCD
- Generates optical black clamping control signals
- Programmable culling pattern
- Maximal supported pixel clock is 40 MHz
- Maximal supported CCD imager size 4096 X 4096
- Shutter signal control
- Digital clamping
- Black level compensation
- Median filtering
- Gain and offset control
- 10-to-8 bit A-law compression table inside
- Supports CCIR601/656 Digital YCbCr 4:2:2 (8/16bit) interface

Figure 23. CCD Controller Block Diagram



9.2 Signal Interface

The CCD Controller accepts pixel data via the C17-C10, Y17-Y10 input pins. The definition of each pixel input is determined by the input format selected in the input formatter. The definition for signals in each mode is shown in Table 8. The interface consists of a set of signals used to transfer raw CCD data from an imager to the CCD Controller. Additionally, the CCD controller can be configured to operate in a mode that adheres to the CCIR 601/656 interface specification. The CCIR 601/656 specification provides a standard method, to transfer 8/16-bit, YCbCr-4:2:2 formatted video data.

Table 8. CCDC Data Input Formats

Pin Name	CCD	16-bit YCbCr	8-bit YCbCr
C17		Cb7,Cr7	Y7,Cb7,Cr7
C16		Cb6,Cr6	Y6,Cb6,Cr6
C15	C_DATA13	Cb5,Cr5	Y5,Cb5,Cr5
C14	C_DATA12	Cb4,Cr4	Y4,Cb4,Cr4
C13	C_DATA11	Cb3,Cr3	Y3,Cb3,Cr3
C12	C_DATA10	Cb2,Cr2	Y2,Cb2,Cr2
C11	C_DATA9	Cb1,Cr1	Y1,Cb1,Cr1
C10	C_DATA8	Cb0,Cr0	Y0,Cb0,Cr0
Y17	C_DATA7	Y7	Y7,Cb7,Cr7
Y16	C_DATA6	Y6	Y6,Cb6,Cr6
Y15	C_DATA5	Y5	Y5,Cb5,Cr5
Y14	C_DATA4	Y4	Y4,Cb4,Cr4
Y13	C_DATA3	Y3	Y3,Cb3,Cr3
Y12	C_DATA2	Y2	Y2,Cb2,Cr2
Y11	C_DATA1	Y1	Y1,Cb1,Cr1
Y10	C_DATA0	Y0	Y0,Cb0,Cr0

The CCDC supports 8 to 14-bit wide raw data signals. When the CCD has fewer than 14 data lines, connect the CCD to the lower or upper data lines C_DATA[13]-C_DATA[0], and ground the upper or lower, unused lines.

CCDC input mode is selected using field IMPMOD in register MODESET - CCDC Mode Setup

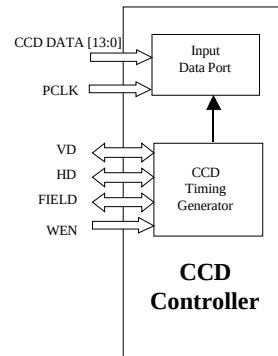
9.3 CCDC Raw Data Input Interface

Timing and control of the CCD interface is done using the HD, VD, field ID, pixel clock, and write enable signals. As shown in Table 9, the CCDC controller can either provide HD, VD, and field signals to the CCD imager, or it can use the HD, VD, and field ID signals provided by the CCD imager. The pixel clock clocks data into the CCDC at a maximum rate of 40MHz. When the CCDC is configured to write data to SDRAM, the write enable signal allows an external device to control whether-or-data the current data will be written to SDRAM.

Table 9. CCDC Signal Interface

Name	I/O	Function
C_DATA[13]-C_DATA[0]	I	<u>Raw Image data loaded by CCD.</u> Bit width can be configured to 8 to 14 bits. The polarity of the input image data can be reversed.
C_VSYNC (VD)	I/O	<u>VSYNC. Vertical synch signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the VD signal. When configured as an output the DM270 will supply the VD signal. The polarity of VD can be reversed.
C_HSYNC (HD)	I/O	<u>HSYNC. Horizontal synch signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the HD signal. When configured as an output the DM270 will supply the HD signal. The polarity of HD can be reversed.
C_FIELD	I/O	<u>Field identification signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the field identification signal. When configured as an output the DM270 will supply the field identification signal. When the field identification signal is set to be an input to the DM270, this signal can be configured to be latched by the VD signal. The polarity of the field identification signal can be reversed.
C_PCLK	I	<u>Pixel clock.</u> This signal is the pixel clock used to load image data into the CCDC. The Clock controller can configure to trigger on the rising or falling edge of the PCLK signal. The maximum pixel clock rate is 40MHz.
C_WEN	I	<u>CCD Controller Write Enable.</u> C_WEN is selected by setting register FSEL1(0x305A6), bit 3 in GIO module.

Figure 24. CCD/CMOS Camera and DM270 Interface

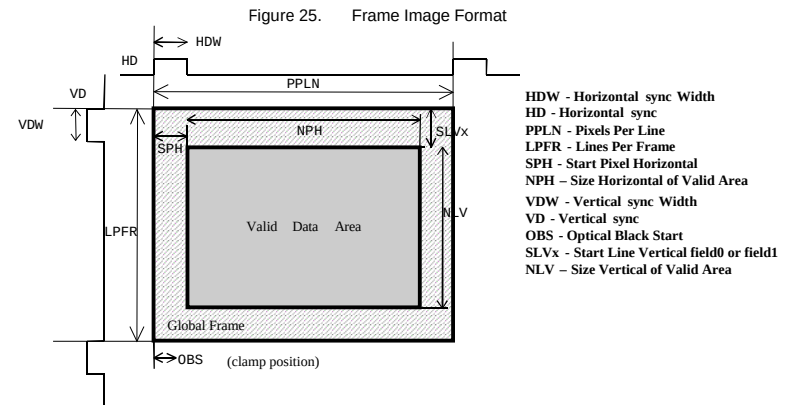


Related registers are

- o SYNCEN - CCDC Synchronization Enable
- o MODESET - CCDC Mode Setup
- o HDWIDTH - CCDC HD pulse width
- o VDWIDTH - CCDC VD pulse width

9.4 Timing Generator, Frame Size Setup

The timing generator provides the use of external sync signals (HD/VD) or internally generated timing signals. The ARM controls width, polarity, position and direction of internal generated signals. Figure below shows various CCDC register settings related to the timing. The shaded area is the physical imager size and the gray area is the valid data area. The image data in this valid data area will be processed and stored to external SDRAM. The vertical start position for even and odd fields can be configured independently.



Related registers are as follows,

- o HDWIDTH - CCDC HD pulse width
- o VDWIDTH - CCDC VD pulse width
- o PPLN - CCDC Pixels per line
- o LPFR - CCDC Lines per frame
- o SPH - CCDC Start pixel horizontal
- o NPH - CCDC Number of pixels horizontal
- o SLV0 - CCDC Start line vertical - field 0
- o SLV1 - CCDC Start line vertical - field 1
- o NLV - CCDC Number of lines vertical

- o HSIZE - CCDC Horizontal size
- o STADRH - CCDC SDRAM Address #1 - high
- o STADRL - CCDC SDRAM Address #2 - low

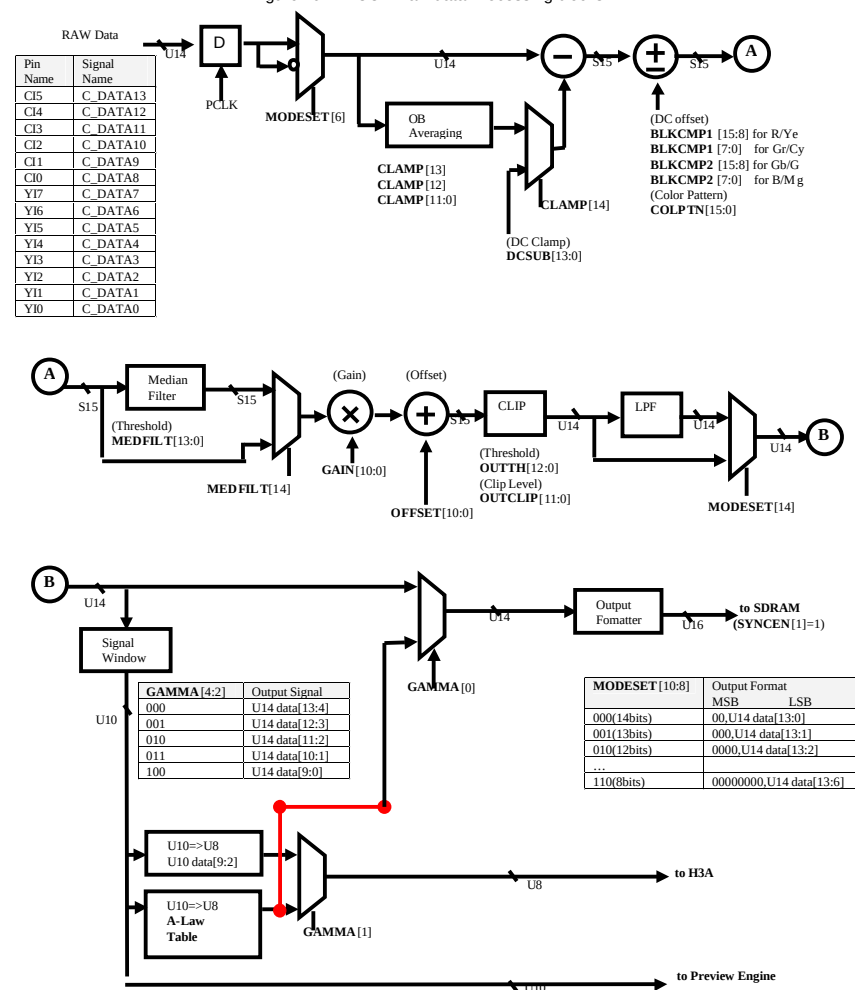
All the above registers are latched by VD (see also 9.11 CCDC Register Update Timing)

9.5 CCD Controller Processing

As shown in Figure 26, the CCDC consists of several data processing modules that format and enhance raw CCD data. There are three destinations for the processed output data, SDRAM, H3A Engine and/or Preview Engine. The following sections describe the modules that perform the following functions.

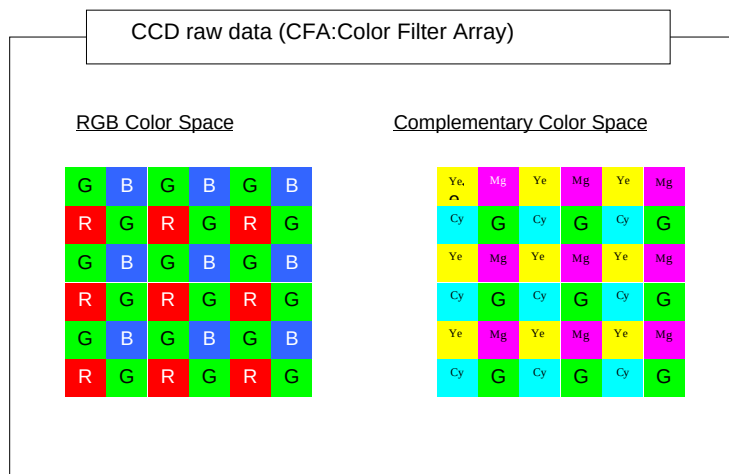
- o Digital clamping
- o Black level compensation
- o Median Filtering
- o Gain and Offset
- o A-LAW compression
- o Output Formatter

Figure 26. CCD Raw data Processing blocks



9.5.1 Arrangement of CCD Raw Data

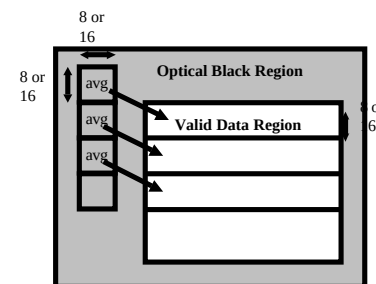
CCD controller can accept RGB raw data or complementary color raw data. The color pattern can be decided by the user using register COLPTN - CCDC CCD Color Pattern



9.5.2 Digital Clamp

The first sub-module is the digital clamp block. The image data is latched at the rising or falling edge of the input pixel clock (PCLK). The averaging circuit takes an average of masked (black) pixel values from the image sensor and this value is subtracted from the image data. The user controls the position of the black pixels, the number of pixels (8 or 16) in each line averaged, and the number of lines (8 or 16) averaged.

Figure 27. Digital Clamp Block Diagram



Alternately, the user can disable black clamp averaging and select a constant black value for subtraction, instead of using the calculated average value for subtraction.

Related registers are

- Interrupt Controller Registers
- DCSUB - CCDC DC Clamp

9.5.3 Black Level Compensation

After the Digital Clamp is applied to the data, Black Level Compensation is applied to the R/Ye, Gr/Cy, Gb/G, and B/Mg signals. The offset applied to the current data sample is selected according to the even/odd phase and the color specified for each phase.

Related registers are

- BLKCOMP1 - CCDC Black Compensation #1
- BLKCOMP2 - CCDC Black Compensation #2

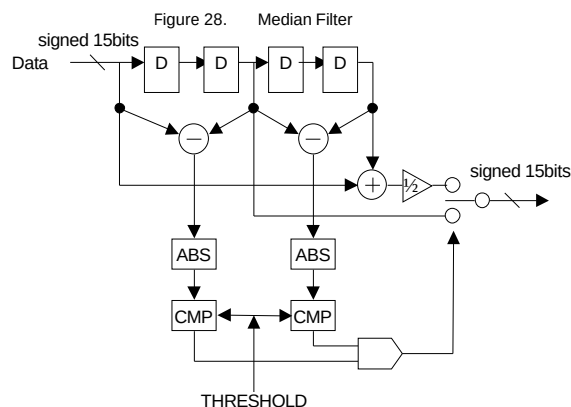
9.5.4 Median Filter

The Median Filter, shown in Figure 28 calculates the absolute difference between the current pixel (i) and pixel (i-2) and the absolute difference between the current pixel (i) and pixel (i+2). If the absolute differences exceeds a threshold, the average of pixel (i-2) and pixel (i+2) replaces pixel (i). A line of

CCD data is input into the Media Filter, so by operating on pixels (i-2), (i), and (i+2) the filter is applied to same color pixels in a CCD data CFA pattern. The Median Filter's threshold is configurable and the Median Filter can be enabled or disabled.

```
for (i=0, i< length_pixel, i++){
    sub1 = input_data(i) - input_data(i-2);
    sub2 = input_data(i) - input_data(i+2);

    if ((abs(sub1) > THRESHOLD && abs(sub2) > THRESHOLD) &&
        ((sub1<0 && sub2<0) || (sub1>0 && sub2>0))) {
        output_data(i) = ( input_data(i-2) +
                           input_data(i+2))/2;
    } else
        output_data(i) = input_data(i);
}
```



Related register is

- MEDFILT - CCDC CCD Median Filter

9.5.5 Gain , Offset, Clip and Low Pass Filter

A gain factor from 0 to 7.9921875 can be applied to the output of the Median Filter. After multiplying the data by the gain factor an offset can be added to the result. The offset can range from 0 to 1023. The result CCD data of gain control is clipped to signed 15bit data, and finally clipped to unsigned 14bit data after offset control.

The low-pass filter consists of a simple three-tap ($\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$) filter.

Related registers are

- GAIN - CCDC CCD Gain Adjustment
- OFFSET - CCDC CCD Offset Adjustment
- OUTTH - CCDC Output Clipping Threshold
- OUTCLIP - CCDC Output Clipping Value
- MODESET - CCDC Mode Setup (field LPF, bit 14)

9.5.6 A-Law

A-Law function can be used for the data compression for the improvement in efficiency of SDRAM storage space.

It is not a substitute to gamma characteristic.

When A-Law table compression is used, 10-bit data can be stored in SDRAM as 8-bit packed data, thus reducing SDRAM storage space required by 50% (Note, 10-bit raw data requires 16-bit space in SDRAM for storage)

Please refer to ITU-T Recommendation G.711 for details about of A-Law.

Related registers are

- GAMMA - CCDC CCD Gamma

9.5.7 Output Formatter

The output formatter block provides options for applying an anti-aliasing filter for horizontal culling.

In the data compression (A-law) mode, any 10 bits of 14bit CCD data are compressed to 8 bits and every four pixels are stored to SDRAM in a packed manner. This format can requires less SDRAM capacity. Please refer to ITU-T Recommendation G.711 for details about of A-Law.

9.5.7.1 Decimation

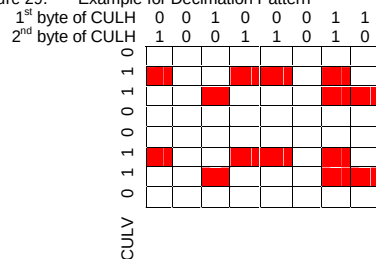
The final stage in CCDC raw data processing is a programmable decimation function along both the horizontal and vertical directions.

Two registers control the horizontal and vertical decimation of CCD Data. The 16-bit CULH register sets the horizontal culling for even and odd lines. The 8-bit CULV register sets the culling along the vertical direction. The LSB of CULV represents the top line of the CCD, the MSB is the 8th line.

Figure below is an example showing how CULH and CULV apply a decimation pattern to the data

In this case CULH = 0x9A23 and CULV = 0x0066

Figure 29. Example for Decimation Pattern



Related registers are

- o CULH - CCDC Culling - horizontal
- o CULV - CCDC Culling - vertical

9.5.7.2 Output Format

The bits of data from each pixel are stored in the upper bits of a 16-bit SDRAM word, and the unused bit positions are filled with zeros. The SDRAM data format is shown in the below table.

Table 10. SDRAM Data Format(CCD)

	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
14bit	Pix0	0	Pix1	0
13bit	Pix0	0	Pix1	0
12bit	Pix0	0	Pix1	0
11bit	Pix0	0	Pix1	0
10bit	Pix0	0	Pix1	0
9bit	Pix0	0	Pix1	0
8bit	Pix0	0	Pix1	0
8bit pack	Pix0	Pix1	Pix2	Pix3

The processed CCD data is transferred to the SDRC in units of 32 bytes. The CCDC has an internal, 16-bit by 16 FIFO. Data is transferred from the FIFO to the SDRC in 32-byte bursts.

Data is only written to SDRAM when WEN in SYNCEN is set to "1".

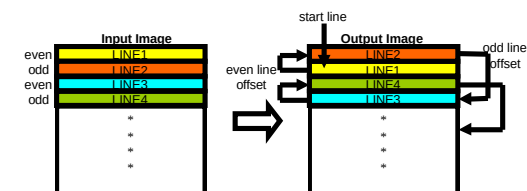
Figure below shows how to construct a frame format in SDRAM.

Related registers are,

- o MODESET - CCDC Mode Setup, fields IMPMOD, PACK8, DATSIZ
- o SYNCEN - CCDC Synchronization Enable, field WEN

Further, the output formatter can be configured for any image format by using SDRAM line offset register, and offset control registers(See [SDOFST](#)[14:0] 0x3071C).

Figure 30. Frame Image format conversion



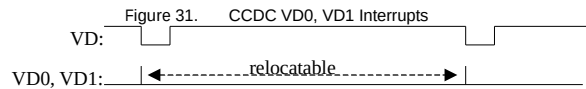
9.6 CCD Interrupts

The CCDC can generate three different interrupt VD0, VD1, and VD2.

9.6.1 Interrupts VD0 and VD1

As shown in the below figure, the VD0 and VD1 interrupts occur relative to the VD pulse. VD0 and VD1 will occur after receiving the number of horizontal lines (horizontal pulse signals) set in [VDINT0](#) and [VDINT1](#), respectively.

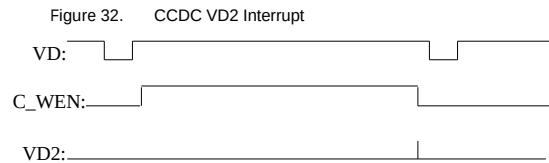
VD0 and VD1 correspond to INT4 and INT5, respectively. Please refer to the section on the Interrupt Controller for information on INT4 and INT5.



9.6.2 Interrupt VD2

In addition to the VD0 and VD1 interrupts, the CCDC also has an interrupt called the VD2 interrupt. The VD2 interrupt always occurs at the falling edge of the C_WEN signal. Note, C_WEN signal is shared with GIO26, to use GIO26 as C_WEN set FSEL1:bit 3 = 1. Also make sure GIO26 is configured as input. Refer GIO section for details.

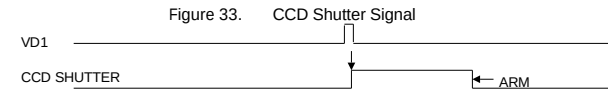
VD2 corresponds to INT6. Please refer to the section on the Interrupt Controller for information on INT6. There are no registers in the CCDC module to configure this interrupt.



9.7 Shutter Control

The CCD Shutter signal is synchronized with the VD1 interrupt. The CCD Shutter signal is immediately set high when VD1 becomes active. The ARM can set the CCD Shutter signal low by writing to the [SHUT\[0\]\(0x 3073A\)](#) register. The CCD Shutter signal is connected to GIO27 pin, so the CCD Shutter signal

can be connected to an imager's external shutter control logic through GIO27 pin. To use GIO27 as shutter signal set FSEL1:bit 4 = 1. Also make sure GIO27 is configured as output. Refer GIO module for details.



9.8 YCbCr signal Processing

Figure below shows a YCbCr signal processing flow in the CCD controller block. The CCD controller accepts 4:2:2 sampled YCbCr input data. The luminance and color difference signals are each 8 bits, scaled 0 to 255. The color difference signals are multiplexed into one 8-bit bus beginning with a Cb sample. The Y and CbCr busses may be input parallel(16-bit mode) or may be time multiplexed and input as a single bus(8-bit mode). The single bus may also contain SAV and EAV video timing reference codes(ITU-R BT.656 mode). In CCIR656 mode, the CCD controller is controlled by the SAV(Start Active Video) time code in the 8-bit pixel data. All timing information is transmitted using a 4-byte synchronization pattern. A synchronization pattern is sent immediately before and after each line during active picture and retrace. DM270 decodes SAV code and is synchronized at this timing.

The 16-bit or 8-bit YCbCr data is stored in SDRAM as 4:2:2 format. Table 11. shows data format in SDRAM. Y data typically has a range of 16 to 235, however, it is possible to subtract a DC value to Y signal.

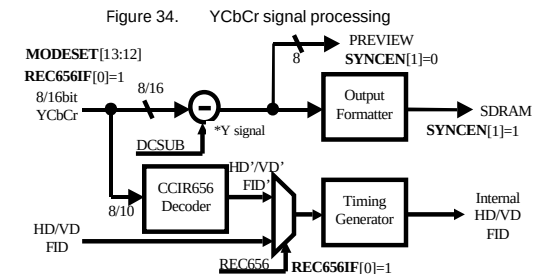


Table 11. SDRAM Data Format

SDRAM Address	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
N	Cb0	Y0	Cr0	Y1
N+1	Cb2	Y2	Cr2	Y3
N+2	Cb4	Y4	Cr4	Y5

Related registers are,

- MODESET - CCDC Mode Setup, field IMPMOD
- REC656IF - CCDC CCIR 656 Control

9.9 Input/Output port configuration

DM270 allows to have several port configuration as shown in the Figure 35 below. Byte swapping of input/output port or input data through mode is configurable by setting CCDC registers. 0 shows a example of system configuration. Register CCDCFG - CCDC CCD Configuration is used for Input/Output port configuration

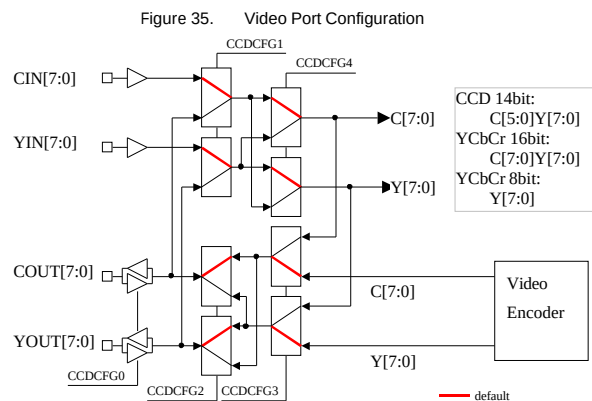
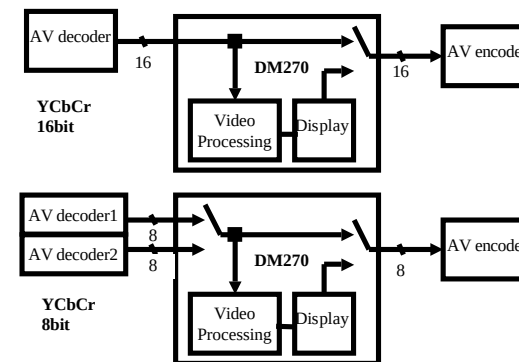


Figure 36. Example of Video System Configuration



9.10 CCIR-656 4:2:2 Parallel Interface

The CCIR-656 signal interface is shown in Figure 38. CCIR-656 is a specification that provides a method to transfer YCbCr-4:2:2 formatted, digital video data over an 8/10-bit wide interface. Note: the CCIR-656 specification is for 525-line and 625-line, digital component video signals in compliance with CCIR Rec. 601. Data and timing codes are transferred over the same 8/10-bit interface. Please refer to CCIR document Rec. 656-1 for detailed information on the interface.

To enable CCIR-656 mode, set R656E in REC656IF (0x3073E). When in CCIR-656 mode, only the data lines and clock signal are connected between the external device and the CCDC. An NTSC/PAL decoder is an example of an external device that may be connected to the CCIR-656 interface.

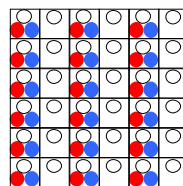
Data lines Y7-Y0 are used for 8-bit YCbCr data and data lines C1-C0, Y7-Y0 are used for 10-bit YCbCr data. The video timing signals, HD, VD, and FIELD, are generated internally by the CCDC.

Two timing reference codes synchronize HD, VD, and FIELD to the video data. At the start and end of each video data block the device sends a unique timing reference code. The start code is called the Start of Active Video signal (SAV), and the end code is called the End of Active Video signal (EAV). The SAV and EAV codes proceed and follow valid data as shown in Figure 38. HD, VD, and FIELD are generated internally by the CCDC based on the SAV and EAV codes. Register settings allow the user to control when the CCDC will read valid data. The delay between the end of the HD pulse and the start of valid data can be

configured by setting [SPH](#) (0x3070C) and the length of valid data can be configured by setting [NPH](#) (0x3070E).

Both timing reference signals, SAV and EAV, consist of a four word sequence in the following format: FF 00 00 XY, where FF 00 00 are a set preamble and the fourth word defines the field identification, the state of vertical field blanking, the state of horizontal line blanking, and error correction codes. The bit format of the fourth word is shown in Table 12. and the definitions for bits, F, V, and H, are given in Table 13. F, V, and H are used in place of the usual horizontal sync, vertical sync, and blank timing control signals. Bits P3, P2, P1, and P0 are error correction bits for F, V, and H. The relationship between F, V, and H and the error correction bits is given in Table 14. To enable error correction, set bit ECCFVH in [RECC656IE](#) (0x3073E). The CCDC will automatically detect and apply error correction when ECCFVH is enabled.

Figure 37. YCbCr 4:2:2 Image data



○ Y
● Cb
● Cr

Figure 38. CCIR-656 Signal Interface

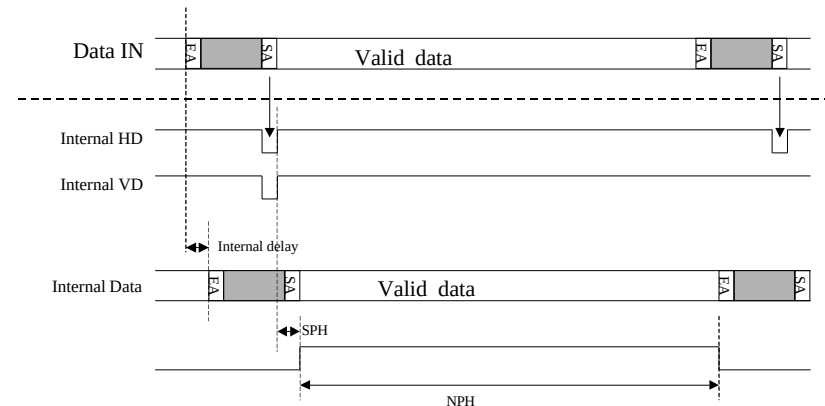


Table 12. Video Timing Reference Codes for SAV and EAV

Data Bit Number	First Word	Second Word	Third Word	Fourth Word
9 (MSB)	1	0	0	1
8	1	0	0	F
7	1	0	0	V
6	1	0	0	H
5	1	0	0	P1
4	1	0	0	P2
3	1	0	0	P3
2	1	0	0	P4
1	1	0	0	0
0	1	0	0	0

Table 13. F, V, H Signal Descriptions

Signal	Value	Command
F	0	Field 1
	1	Field 2
V	0	0
	1	Vertical blank
H	0	SAV
	1	EAV

Table 14. F, V, H Error Correction Bits

F	V	H	P3	P2	P1	P0
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	1	0	1	1
0	1	1	1	1	1	0
1	0	0	0	1	1	1
1	0	1	0	0	1	0
1	1	0	1	1	0	0
1	1	1	1	0	0	1

When operating in CCIR-656 mode, data is stored in SDRAM according to the format shown in the figure below.

Figure 39. CCIR-656 mode data format in SDRAM

8bit	Pixel3 (Y1)	Pixel2 (Cr0)	Pixel1 (Y0)	Pixel0 (Cb0)
------	-------------	--------------	-------------	--------------

9.11 CCDC Register Update Timing

Many of the CCDC registers contain bits that are latched by the VD signal. Data written to a latched bit does not take affect until the next VD pulse is received. This allows registers that control the CCDC, such as the SDRAM write address location, HD width, VD width, etc. to be changed in between successive VD pulses without corrupting the current frame of CCD data. The CCDC registers, latched by the VD signal are,

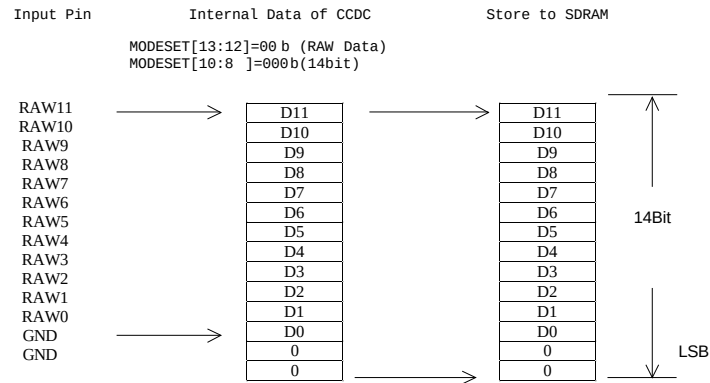
- SYNCEN - CCDC Synchronization Enable
- LPF, bit 14 in MODESET - CCDC Mode Setup
- HDWIDTH - CCDC HD pulse width
- VDWIDTH - CCDC VD pulse width
- PPLN - CCDC Pixels per line
- LPFR - CCDC Lines per frame
- SPH - CCDC Start pixel horizontal
- NPH - CCDC Number of pixels horizontal
- SLV0 - CCDC Start line vertical - field 0
- SLV1 - CCDC Start line vertical - field 1
- NLV - CCDC Number of lines vertical
- CULH - CCDC Culling - horizontal
- CULV - CCDC Culling - vertical
- HSIZE - CCDC Horizontal size
- SDOFST - CCDC SDRAM Line Offset
- STADRH - CCDC SDRAM Address #1 - high
- STADRL - CCDC SDRAM Address #2 - low
- CLAMPEN, bit 14 in Interrupt Controller Registers
- GAMMA - CCDC CCD Gamma

Other CCDC registers are asynchronously set up with VD

9.12 Relation between CCD RAW data and MODESET Register

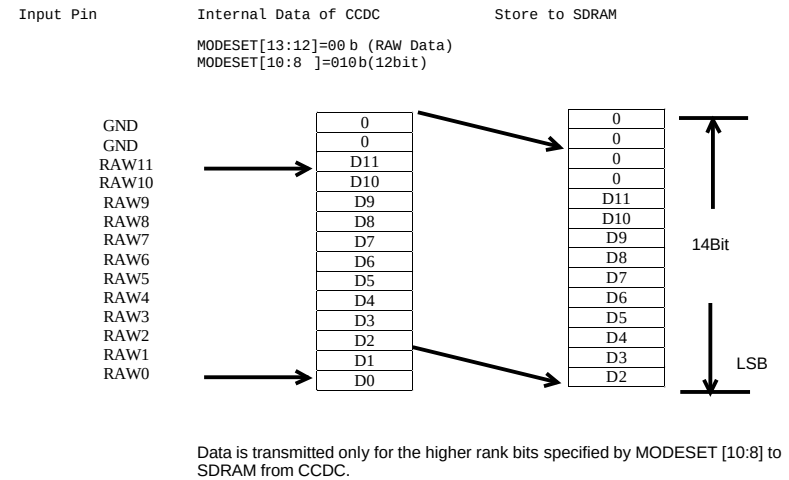
The example is of 12bit RAW Input Data is as follows.

Case 1:



If CCDC outputs data to SDRAM as RAW data of 14Bit(s) by this method, higher order 10Bit of RAW Data can be transmitted to Preview Engine.

Case 2:



9.13 CCD Controller Register Map (CCDC)

Address	Register	Description
0003:0700	SYNEN	Synchronization Enable
0003:0702	MODESET	Mode Setup
0003:0704	HDWDTH	HD pulse width
0003:0706	VDWDTH	VD pulse width
0003:0708	PPLN	Pixels per line
0003:070A	LPFR	Lines per frame
0003:070C	SPH	Start pixel horizontal
0003:070E	NPH	Number of pixels horizontal
0003:0710	SLVO	Start line vertical - field 0
0003:0712	SLV1	Start line vertical - field 1
0003:0714	NLV	Number of lines vertical
0003:0716	CULH	Culling - horizontal
0003:0718	CULV	Culling - vertical
0003:071A	HSIZE	Horizontal size
0003:071C	SDOFS	SDRAM Line Offset
0003:071E	STADRH	SDRAM Address #1 - high
0003:0720	STADRL	SDRAM Address #2 - low
0003:0722	CLAMP	CCD Data Clamping
0003:0724	DCSUB	DC Clamp
0003:0726	COLPTN	CCD Color Pattern
0003:0728	BLKCM1	Black Compensation #1
0003:072A	BLKCM2	Black Compensation #2
0003:072C	MEDFIL	CCD Median Filter
0003:072E	GAIN	CCD Gain Adjustment
0003:0730	OFFSET	CCD Offset Adjustment
0003:0732	OUTH	Output Clipping Threshold
0003:0734	OUTCLIP	Output Clipping Value
0003:0736	VDINT0	VD Interrupt #0
0003:0738	VDINT1	VD Interrupt #1
0003:073A	SHUT	CCD Shutter
0003:073C	GAMMA	CCD Gamma
0003:073E	REC6561F	CCIR 656 Control
0003:0740	CDCFG	CCD Configuration

9.14 CCD Controller Registers

9.14.1 SYNCEN - CCDC Synchronization Enable

[illegible][illegible]

9.14.2 MODESET - CCDC Mode Setup

[illegible]

Bit	Name	Reset Value	R/W	Function
0:00	FLDSTAT	0	R	<u>Field Status</u> » 0: odd field « » 1: even field «
14	LPF	0	R/W	<u>2-tap Low-Pass (anti-aliasing) Filter</u> » 0: off « » 1: on «
13:12	IMPMOD	0	R/W	<u>Setting of data input mode</u> » 0: CCD RAW data « » 1: YCbCr 16-bit « » 2: YCbCr 8-bit « » 3: Reserved «
11	PACK8	0	R/W	Pack to 8-bits/pixel » 0: normal (16 bits/pixel) « » 1: pack to 8-bits/pixel «
10:8	DATSI Z	0	R/W	<u>CCD Data Width</u> » 000: 14-bits « » 001: 13-bits « » 010: 12-bits « » 011: 11-bits « » 100: 10-bits « » 101: 9-bits « » 110: 8-bits « » 111: Reserved «
7	FLDMODE	0	R/W	<u>Sensor Field Mode</u> » 0: non-interlaced (progressive) « » 1: interlaced «
6	DATAPOL	0	R/W	<u>CCD Data Polarity</u> » 0: normal (no change) « » 1: one's complement «
5	EXWEN	0	R/W	<u>External WEN Selection</u> » 0: do not use external WEN (Write Enable) « » 1: use external WEN (Write Enable) «
4	FLDPOL	0	R/W	<u>Field Indicator Polarity</u> » 0: positive « » 1: negative «
3	HDPOL	0	R/W	<u>HD Sync Polarity</u> » 0: positive « » 1: negative «
2	VDPOL	0	R/W	<u>VD Sync Polarity</u> » 0: positive « » 1: negative «
1	FLDOUT	0	R/W	<u>Field ID Direction</u> » 0: input « » 1: output «
0	VDHDOUT	0	R/W	<u>VD/HD Sync Direction</u> » 0: input « » 1: output «

9.14.3 HDWIDTH - CCDC HD pulse width

HDWIDTH										0003:0704		offset:		0x04												default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0													
Name	RSV	RSV	RSV	RSV	HDW[11]	HDW[10]	HDW[9]	HDW[8]	HDW[7]	HDW[6]	HDW[5]	HDW[4]	HDW[3]	HDW[2]	HDW[1]	HDW[0]													
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W													
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0													

[illegible]

9.14.4 VDWIDTH - CCDC VD pulse width

[illegible][illegible]

9.14.5 PPLN - CCDC Pixels per line

[illegible][illegible]

9.14.6 LPFR - CCDC Lines per frame

[illegible][illegible]

9.14.7 SPH - CCDC Start pixel horizontal

[illegible][illegible]

9.14.8 NPH - CCDC Number of pixels horizontal

[illegible][illegible]

9.14.9 SLV0 - CCDC Start line vertical - field 0

SLV0				0003:0710				offset:				0x10				default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	SLV0[11]	SLV0[10]	SLV0[9]	SLV0[8]	SLV0[7]	SLV0[6]	SLV0[5]	SLV0[4]	SLV0[3]	SLV0[2]	SLV0[1]	SLV0[0]							
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W							
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0							

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	SLV0	0	R/W	Start Line, Vertical (Field 0) Sets line at which data output to SDRAM will begin, measured from the start of VD. *This bit field is latched by VD.

9.14.10 SLV1 - CCDC Start line vertical - field 1

SLV1	0003:0712				offset:	0x12								default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	SLV1[11]	SLV1[10]	SLV1[9]	SLV1[8]	SLV1[7]	SLV1[6]	SLV1[5]	SLV1[4]	SLV1[3]	SLV1[2]	SLV1[1]	SLV1[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	SLV1	0	R/W	Start Line, Vertical (Field 1) Sets line at which data output to SDRAM will begin, measured from the start of VD. *This bit field is latched by VD.

9.14.11 NLV - CCDC Number of lines vertical

NLV	0003:0714				offset:	0x14								default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	NLV[11]	NLV[10]	NLV[9]	NLV[8]	NLV[7]	NLV[6]	NLV[5]	NLV[4]	NLV[3]	NLV[2]	NLV[1]	NLV[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	NLV	0	R/W	Number of lines, vertical Sets number of vertical lines that will be output to SDRAM. The number of lines output to SDRAM = (NLV + 1). *This bit field is latched by VD.

9.14.12 CULH - CCDC Culling - horizontal

CULH				0003:0716				offset: 0x16				default: 0xFFFF				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CULHEVN[7]	CULHEVN[6]	CULHEVN[5]	CULHEVN[4]	CULHEVN[3]	CULHEVN[2]	CULHEVN[1]	CULHEVN[0]	CULHODD[7]	CULHODD[6]	CULHODD[5]	CULHODD[4]	CULHODD[3]	CULHODD[2]	CULHODD[1]	CULHODD[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	CULHEVN	255	R/W	Horizontal Culling Pattern for Even Line, 8-bit mask: 0: cull, 1:retain LSB is first pixel, MSB is 8th pixel, then pattern repeats. *This bit field is latched by VD.
7:0	CULHODD	255	R/W	Horizontal Culling Pattern for Even Line, 8-bit mask: 0: cull, 1:retain LSB is first pixel, MSB is 8th pixel, then pattern repeats. *This bit field is latched by VD.

9.14.13 CULV - CCDC Culling - vertical

CULV	0003:0718								offset:	0x18								default: 0x00FF			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CULV[7]	CULV[6]	CULV[5]	CULV[4]	CULV[3]	CULV[2]	CULV[1]	CULV[0]					
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1					

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	CULV	255	R/W	Vertical Culling Pattern, 8-bit mask: 0: cull, 1:retain LSB is first line, MSB is 8th line, then pattern repeats. *This bit field is latched by VD.

9.14.14 HSIZE - CCDC Horizontal size

HSIZE	0003:071A												offset:	0x1A				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	LNOFST[8]	LNOFST[7]	LNOFST[6]	LNOFST[5]	LNOFST[4]	LNOFST[3]	LNOFST[2]	LNOFST[1]	LNOFST[0]					
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8:0	LNOFST	0	R/W	Address offset for each line. Sets size of line in SDRAM, units: 32 bytes Either 16 or 32 pixels depending on setting of PACK8 *This bit field is latched by VD.

9.14.15 SDOFST - CCDC SDRAM Line Offset

SDOFST		0003:071C				offset: 0x1C				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	FINV	FOFST[1]	FOFST[0]	LOFST0[1]	LOFST0[0]	LOFST0[0]	LOFST1[2]	LOFST1[1]	LOFST1[0]	LOFST2[2]	LOFST2[1]	LOFST2[0]	LOFST3[2]	LOFST3[1]	LOFST3[0]	
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	FINV	0	R/W	Field identification signal inverse » 0: Non inverse « » 1: Inverse «
13: 12	FOFST	0	R/W	Line offset value of field ID = 1 » 00: +1 line « » 01: +2 line « » 10: +3 line « » 11: +4 line «
11: 9	LOFST0	0	R/W	Line offset values of even line and field ID = 1 » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
8: 6	LOFST1	0	R/W	Line offset values of odd line and field ID = 1 » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
5: 3	LOFST2	0	R/W	Line offset values of even line and field ID = 1 (0?) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
2: 0	LOFST3	0	R/W	Line offset values of odd line and field ID = 1 (0?) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «

9.14.16 STADRH - CCDC SDRAM Address #1 - high

STADRH		0003:071E				offset: 0x1E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]	
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15: 6	RSV			Reserved
5: 0	ADRH	0	R/W	Upper 6 bits of the SDRAM starting address for CCDC output The address is specified offset from the SDRAM base address in units of 32 bytes. *This bit field is latched by VD.

9.14.17 STADRL - CCDC SDRAM Address #2 - low

STADRL		0003:0720				offset: 0x20												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	ADRL	0	R/W	Lower 16 bits of the SDRAM starting address for CCDC output The address is specified offset from the SDRAM base address in units of 32 bytes. *This bit field is latched by VD.

9.14.18 CLAMP - CCDC CCD Data Clamping

CLAMP				0003:0722				offset: 0x22								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	CLAMPEN	OBSLEN	OBSLN	OBSL[11]	OBSL[10]	OBSL[9]	OBSL[8]	OBSL[7]	OBSL[6]	OBSL[5]	OBSL[4]	OBSL[3]	OBSL[2]	OBSL[1]	OBSL[0]			
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	CLAMPEN	0	R/W	<u>Clamp Enable:</u> » 0: disable « » 1: enable « Enables clamping of CCD data based on the calculated average of Optical Black Samples. *This bit is latched by VD.
13	OBSLEN	0	R/W	<u>Optical Black Sample Length:</u> » 0: 8 pixels « » 1: 16 pixels « Number of Optical Black Sample pixels per line to include in the average calculation.
12	OBSLN	0	R/W	<u>Optical Black Sample Lines:</u> » 0: 8 lines « » 1: 16 lines « Number of Optical Black Sample lines to include in the average calculation.
11:0	OBSL	0	R/W	<u>Start Pixel of Optical Black Samples</u> Start pixel position of Optical Black Samples, specified from the start of HD in pixel clocks.

9.14.19 DCSUB - CCDC DC Clamp

DCSUB		0003:0724				offset: 0x24										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	DCSUB[13]	DCSUB[12]	DCSUB[11]	DCSUB[10]	DCSUB[9]	DCSUB[8]	DCSUB[7]	DCSUB[6]	DCSUB[5]	DCSUB[4]	DCSUB[3]	DCSUB[2]	DCSUB[1]	DCSUB[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:0	DCSUB	0	R/W	<u>DC level to subtract from CCD data</u> The DC value set here is subtracted from the CCD data when OBS clamping is disabled - CCLAMP:CLEN

9.14.20 COLPTN - CCDC CCD Color Pattern

COLPTN		0003:0726		offset: 0x26		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	EPEL1[1]	EPEL1[0]	OPEL1[1]	OPEL1[0]	EPOL1[1]	EPOL1[0]	OPOL1[1]	OPOL1[0]	EPEL0[1]	EPEL0[0]	OPEL0[1]	OPEL0[0]	EPOL0[1]	EPOL0[0]	OPOL0[1]	OPOL0[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	EPEL1	0	R/W	Color Pattern for Even Pixel, Even Line (Field 1) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg « Not used in non-interlaced mode
13:12	OPEL1	0	R/W	Color Pattern for Odd Pixel, Even Line (Field 1) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg « Not used in non-interlaced mode
11:10	EPOL1	0	R/W	Color Pattern for Even Pixel, Odd Line (Field 1) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg « Not used in non-interlaced mode
9:8	OPOL1	0	R/W	Color Pattern for Odd Pixel, Odd Line (Field 1) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg « Not used in non-interlaced mode
7:6	EPEL0	0	R/W	Color Pattern for Even Pixel, Even Line (Field 0) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
5:4	OPEL0	0	R/W	Color Pattern for Odd Pixel, Even Line (Field 0) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
3:2	EPOL0	0	R/W	Color Pattern for Even Pixel, Odd Line (Field 0) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
1:0	OPOL0	0	R/W	Color Pattern for Odd Pixel, Odd Line (Field 0) » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «

9.14.21 BLKCMP1 - CCDC Black Compensation #1

BLKCMP1		0003:0728		offset: 0x28		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	R_YE[7]	R_YE[6]	R_YE[5]	R_YE[4]	R_YE[3]	R_YE[2]	R_YE[1]	R_YE[0]	GR_CY[7]	GR_CY[6]	GR_CY[5]	GR_CY[4]	GR_CY[3]	GR_CY[2]	GR_CY[1]	GR_CY[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	R_YE	0	R/W	Black level compensation for R/Ye pixels (-128:+127) 2's complement, MSB is sign bit
7:0	GR_CY	0	R/W	Black level compensation for Gr/Cy pixels (-128:+127) 2's complement, MSB is sign bit

9.14.22 BLKCMP2 - CCDC Black Compensation #2

BLKCMP2		0003:072A		offset: 0x2A		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	GB_G[7]	GB_G[6]	GB_G[5]	GB_G[4]	GB_G[3]	GB_G[2]	GB_G[1]	GB_G[0]	B_MC[7]	B_MC[6]	B_MC[5]	B_MC[4]	B_MC[3]	B_MC[2]	B_MC[1]	B_MC[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	GB_G	0	R/W	Black level compensation for Gb/G pixels (-128:+127) 2's complement, MSB is sign bit
7:0	B_MC	0	R/W	Black level compensation for B/Mg pixels (-128:+127) 2's complement, MSB is sign bit

9.14.23 MEDFILT - CCDC CCD Median Filter

[illegible][illegible]

9.14.24 GAIN - CCDC CCD Gain Adjustment

[illegible][illegible]

9.14.25 OFFSET - CCDC CCD Offset Adjustment

[illegible][illegible]

9.14.26 OUTTH - CCDC Output Clipping Threshold

[illegible][illegible]

9.14.27 OUTCLIP - CCDC Output Clipping Value

[illegible][illegible]

9.14.28 VDINT0 - CCDC VD Interrupt #0

VINDT0										0003:0736										offset: 0x36										default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																							
Name	RSV	RSV	RSV	RSV	VINDT0[11]	VINDT0[10]	VINDT0[9]	VINDT0[8]	VINDT0[7]	VINDT0[6]	VINDT0[5]	VINDT0[4]	VINDT0[3]	VINDT0[2]	VINDT0[1]	VINDT0[0]																							
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																							
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0																							

[illegible]

9.14.29 VDINT1 - CCDC VD Interrupt #1

[illegible][illegible]

9.14.30 SHUT - CCDC CCD Shutter

[illegible][illegible]

9.14.31 GAMMA - CCDC CCD Gamma

GAMMA		0003 073C										offset:		0x3C				default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
R/W	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GDW[2]	GDW[1]	GDW[0]	H3ATBL	CCD1BL					
Default	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W					

[illegible]

9.14.32 REC656IF - CCDC CCIR 656 Control

REC656IF										0003:073E					offset: 0x3E							default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ECCFVH	R656ON							
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W							
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0							

[illegible]

9.14.33 CCDCFG - CCDC CCD Configuration

CCDCFG				0003:0740				offset: 0x40								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	VDLC	MSBINVO	MSBINVI	BSWD	Y8POS	EXTRG	TRGSEL	WENLOG	FIDMD[1]	FIDMD[0]	BW656	YCINSWP	INPTRU	YCPUTSWP	IDS	YCIO			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	VDLC	0	R/W	Enable latching function registers on internal VSYNC » 0: Latched on VSYNC « » 1: Not latched on VSYNC «
14	MSBINVO	0	R/W	MSB of Chroma signal output inverted » 0: normal « » 1: MSB inverted «
13	MSBINVI	0	R/W	MSB of Chroma input signal stored to SDRAM inverted » 0: normal « » 1: MSB inverted «
12	BSWD	0	R/W	Byte Swap Data stored to SDRAM » 0: normal « » 1: Swap Bytes «
11	Y8POS	0	R/W	Location of Y signal when YCbCr 8bit data is input » 0: even pixel « » 1: odd pixel «
10	EXTRG	0	R/W	External Trigger » 0: rising edge « » 1: falling edge «
9	TRGSEL	0	R/W	Signal that initializes SDRAM address when EXTRG = 1 » 0: DWEN bit (SYNCEN register) « » 1: FID input terminal «
8	WENLOG	0	R/W	In ES1.0, Sets data capture when WEN input terminal is used » 0: Data stored when WEN or frame definition is valid « » 1: Data stored when WEN and frame definition is valid « In ES1.1 Sets data capture when WEN input terminal is used » 0: Data stored when WEN and frame definition is valid « » 1: Data stored when WEN or frame definition is valid «
7:6	FIDMD	0	R/W	Setting of FID detection function » 00: FID input signal is captured at the head of VD « » 01: FID input signal is used directly « » 10: FID is 0 at rising of VD signal and 1 at falling « » 11: FID is 0 when VD and HD input signals are in phase, and 1 when they are out-of-phase «
5	BW656	0	R/W	The data width in CCIR656 input mode » 0: 8-bits « » 1: 10-bits «
4	YCINSWP	0	R/W	Y input (YIN[7:0]) and C input (CIN[7:0]) are swapped » 0: YIN[7:0] = Y signal / CIN[7:0] = C signal « » 1: YIN[7:0] = C signal / CIN[7:0] = Y signal «
3	INPTRU	0	R/W	Y output and C output are controlled. » 0: Video Encoder output « » 1: Input passthrough mode «
2	YCPUTSWP	0	R/W	Y output (YOUT[7:0]) and C output (COUT[7:0]) are swapped » 0: YOUT[7:0] = Y signal / COUT[7:0] = C signal « » 1: YOUT[7:0] = C signal / COUT[7:0] = Y signal «
1	IDS	0	R/W	Selection of Y signal / C signal input port » 0: YIN[7:0]/CIN[7:0] « » 1: YOUT[7:0]/COUT[7:0] «
0	YCIO	0	R/W	I/O setting of YOUT/COUT output terminal » 0: Output « » 1: Input «

10 Preview Engine

10.1 Introduction

The Preview Engine is a programmable hardware video processing module that generates image data in YCbCr-4:2:2 format from raw CCD data. The Preview Engine can also be configured to operate in a resize only mode, which allows the YCbCr-4:2:2 format data to be resized without applying the processing of every module in the Preview Engine.

As shown in Figure 41, several internal modules are used to process CCD raw data into YCbCr-4:2:2 data. These modules are briefly mentioned in this section and described in detail in the sections that follow.

The input interface module has two input ports, which are connected to the CCD Controller (CCDC) and the SDRAM Controller (SDRC).

The input interface extracts valid raw data from the CCD raw data, and then various modules in the Preview Engine process the raw CCD data.

The noise filter module reduces impulse noise in the raw data.

The white balance module applies two gain adjustments to the data: a digital gain and a white balance gain.

The CFA interpolation module implements CFA interpolation, as well as smoothing and a resizing. The output from the CFA interpolation module is always RGB formatted data regardless of the type of color space used by the CCD.

The black adjustment module independently adds an offset value to each RGB component. The offset adjusts the black level of image data.

The RGB2RGB blending module applies a 3x3 matrix transform to the RGB data generated by the CFA interpolation module.

The gamma correction module independently applies gamma correction to each RGB component. This module can be configured to use one of two gamma tables, which are stored in the Preview Engine's internal ROM.

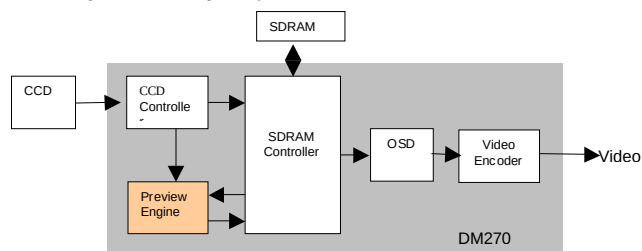
The RGB2YCbCr conversion module applies 3x3 matrix transformation to the RGB data to convert it to YCbCr data. This module also implements offset, contrast, brightness, and pseudo color suppression.

The 4:2:2 conversion module downsamples Cb and Cr, so that the Preview Engine output data is in YCbCr-4:2:2 format.

The output interface module transfers data from the Preview Engine to SDRAM. Data is transferred from the Preview Engine in burst of 16, 16bit words.

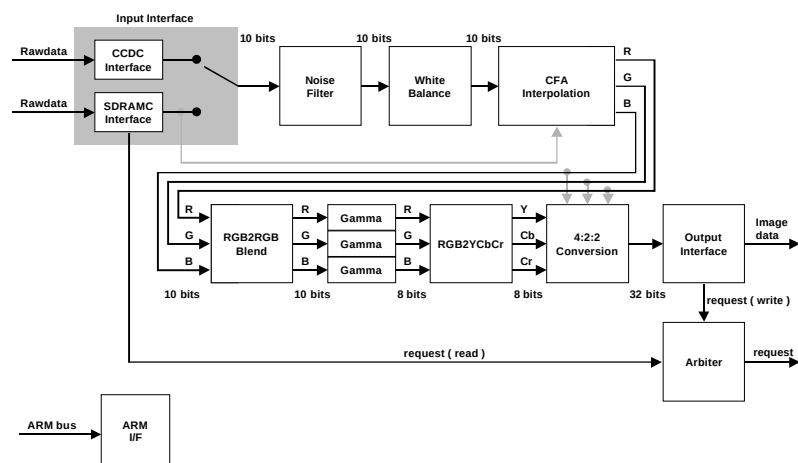
Figure 40 shows Image Peripheral Architecture containing Preview Engine.

Figure 40. Image Peripheral Architecture



10.2 Preview Engine Block Diagram

Figure 41. Preview Engine Block Diagram



As shown in the Preview Engine block diagram is shown in Figure 41., the Preview Engine has the following characteristics:

- Dual data input ports (CCD controller and SDRAM controller)
- Support for RGB and Complementary CFA input data patterns
- Noise filter
- Digital gain
- White balance
- Smoothing filter
- Horizontal and vertical seamless down sampling (x1/64~x1)
- Horizontal and vertical seamless zoom (x1~x4),
- Black adjustment
- Programmable RGB2RGB blending transformation
- Gamma correction
- Luminance offset, brightness, and contrast
- Chrominance offset and pseudo-color suppression
- Programmable RGB-to-YCbCr conversion coefficients
- One-shot preview
- Inverse A-Law table
- Performs YCbCr-4:2:2 image resizing

10.3 Module Description

This section provides a detailed description of the modules that implement the Preview Engine's image pipeline. The modules convert raw CCD data to YCbCr-4:2:2 data. The raw data input can come from the CDC or from SDRAM. The YCbCr-4:2:2 output is always written to SDRAM.

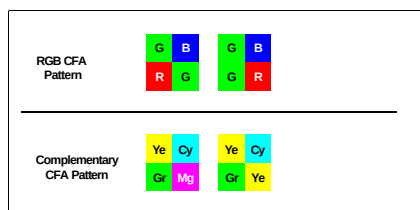
See section 10.5 Fractional Notation for description of fractional notation used in this chapter

10.3.1 Input data and Color filter Array

A CCD sensor generates a color image by measuring the intensity of light that passes through a color filter mask. This color filter mask is called a Color Filter Array (CFA). The masking pattern of the CFA array as well as the filter color primaries vary between different manufactures. In digital camera applications, the filter color primaries are often components of the RGB or Complementary color space. The Preview engine supports 2x2 RGB and Complementary CFA patterns. All possible 2x2 RGB or Complementary patterns are supported. Figure 42 depicts two RGB CFA patterns and two Complementary CFA patterns.

Setting bits 7-0 of the register PVSET2 - Preview Setup Register #2 configures the position of each RGB or Complementary color component within the 2x2 CFA pattern. These bits should be set according to the CFA pattern used by the CCD module.

Figure 42. CFA Pattern



10.3.2 Noise Filter Module

The noise filter is the first stage of the Preview Engine's image-processing pipeline. The noise filter is a high pass filter that reduces noise in raw CCD data. The filter equation is shown in Figure 43. In the formula, X_0 , is the current pixel and 'a' is a configurable constant. The noise filter operates on pixels of the same color component, i.e. X_{-2} , X_0 , and X_2 . The value of the constant is defined by the NFRT field, in the register NFILT - Noise Filter Definition Register. Noise filter is enabled using the NFEN field in the same register.

Figure 43. Noise Filter Equation

$$x_0 = X_0 - a \frac{X_{-2} - 2X_0 + X_2}{4}$$

$$\left(a = \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1 \right)$$

10.3.3 White Balance Module

The preview engine white balance module has two gain adjusters, a digital gain adjuster and a white balance adjuster. In the digital gain adjuster, the raw data is multiplied by a fixed value gain regardless of the color pixel to be processed. In the white balance gain adjuster, the raw data is multiplied by a selected gain corresponding to the color of the processed pixel. Figure 44, Figure 45 shows this example for R, G, B raw data.

Figure 44. Digital Gain

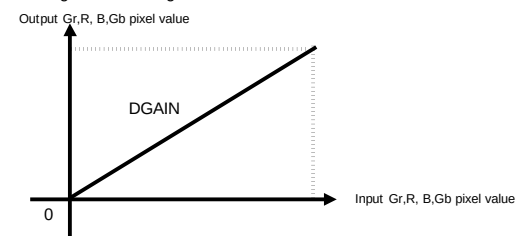


Figure 45. White Balance

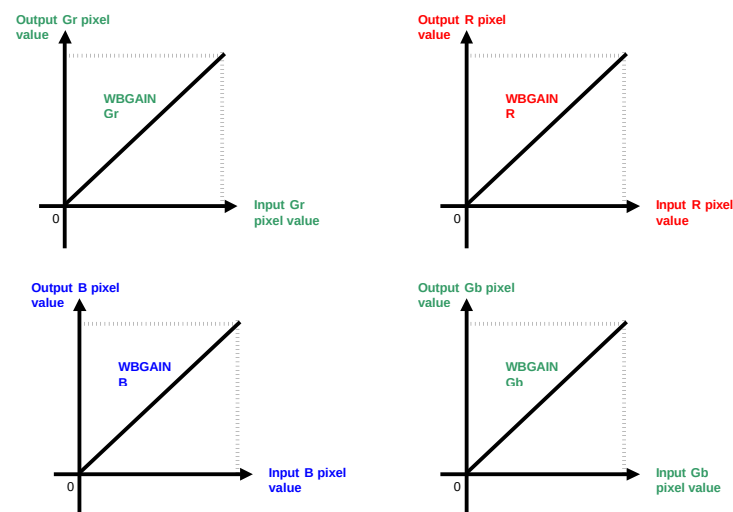
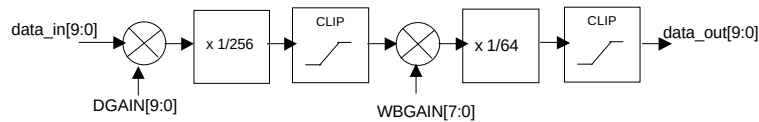


Figure 46 shows the block diagram of the white balance module. It is composed of two multipliers for the digital gain and the white balance gain.

Figure 46. White Balance Module Block Diagram



The registers used for white balance and their precision is shown below,

See section 10.5 Fractional Notation for description of fractional notation used in this chapter

- Digital Gain
Register: DGAIN - Digital Gain Register
Precision: U10Q8

DGAIN[9:0]	Value
11 1111 1111	3.99609375 (3+255/256)
01 0000 0000	1.0
00 1000 0000	0.5
...	
00 0000 0001	0.00390625 (1/256)
00 0000 0000	0

- White balance
Register:
WBGAIN0 - White Balance Gain register #0,
WBGAIN1 - White Balance Gain register #1
Precision: U8Q6

WBGAIN[8:0]	Value
1111 1111	3.984375 (3+63/64)
0100 0000	1.0
0010 0000	0.5
...	
0000 0001	0.015625 (1/64)
0000 0000	0

10.3.4 CFA Interpolation

The Preview Engine's CFA interpolation module includes many important sub-modules. The sub-modules perform functions such as, horizontal and vertical interpolation, smoothing, horizontal and vertical resizing, color adjustment, and conversion from complementary to RGB. Figure 47 shows a block diagram of the CFA interpolation module. In Figure 47, the data path taken during normal mode is shown by the dark lines, and the data path taken in resize only mode is shown by the light lines.

When the Preview Engine is operating in resize only mode, some of the CFA interpolation modules are bypassed. In resize only mode, only smoothing and horizontal and vertical resizing are applied to the data. The data path taken in resize only mode is shown by the dark lines in Figure 48.

The horizontal interpolation sub-module applies horizontal interpolation using a two tap or a five-tap interpolation filter.

The smoother sub-module applies a low pass filter.

The horizontal resampler sub-module applies a seamless horizontal down-sampling or up-sampling (zoom) by x4 over-sampling.

The vertical interpolation and resampler sub-module performs a vertical interpolation and a seamless vertical down-sampling or up-sampling by x4 over-sampling using two line memories.

The color selector sub-module extracts the appropriate color from the 4 inputs of the vertical interpolation and resampler sub-module.

The color adjustment sub-module executes color adjustment on the interpolated data. This module is bypassed when the Preview Engine is in RGB CCD mode.

The comp2RGB conversion sub-module converts the data from the complementary color space to the RGB color space. The output of the CFA interpolation module is RGB data regardless of a type of the color space of used by the CCD imager.

Figure 47. CFA Interpolation module (Normal Mode)

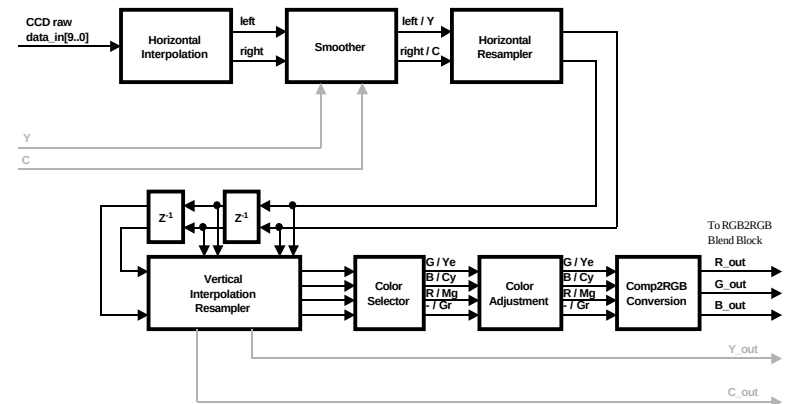
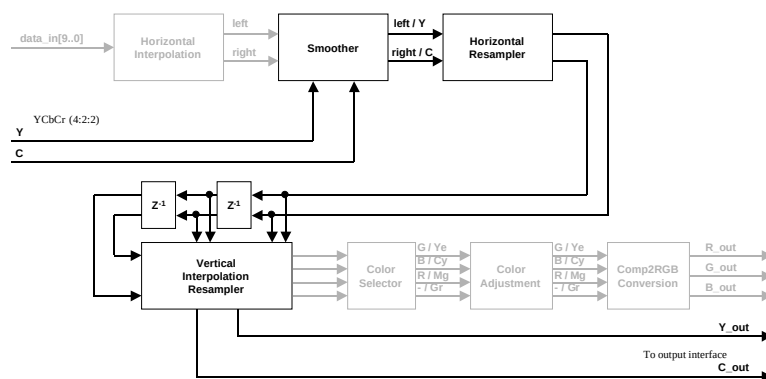


Figure 48. CFA Interpolation module (Resize only Mode)



10.3.4.1 Horizontal Interpolation : CFA Sub-module

The horizontal interpolation sub-module performs horizontal interpolation with either a two-tap or a five-tap interpolation filter to estimate the missing color components. As shown in the following equations, the 2-tap interpolation filter uses a simple average of the previous and next pixels to calculate the interpolated pixel value, while the 5-tap interpolation filter utilizes the values of neighboring pixels (same color) to calculate the interpolated pixel value. By including neighboring pixels, the 5-tap filter reduces false color artifacts.

$$x_0 = \begin{cases} \frac{X_{-1} + X_1}{2} & (2 \text{ taps}) \\ \frac{-X_{-2} + 2X_{-1} + 2X_0 + 2X_1 - X_2}{4} & (5 \text{ taps}) \end{cases}$$

The type of horizontal CFA filter is selected with the HINT bit of the register PVSET2 - Preview Setup Register #2.

10.3.4.2 Smoother : CFA Sub-module

The smoother sub-module is an anti-aliasing filter. The sub-module actually consists of two FIR filters. One of the filters can be used independently or both filters can be used in combination to provide two levels of horizontal low pass filtering. The following equations shows the formulas for the two levels of filtering as well as for the bypass configuration.

$$x_0 = \begin{cases} X_0 & (\text{bypass}) \\ \frac{X_{-1} + 2X_0 + X_1}{4} \equiv x'_0 & (\text{weak}) \\ \frac{x'_{-2} + 2x'_0 + x'_2}{4} & (\text{strong}) \end{cases}$$

Smoother is setup using register SMTH - Smoothing Definition register

10.3.4.3 Horizontal Resampler : CFA Sub-module

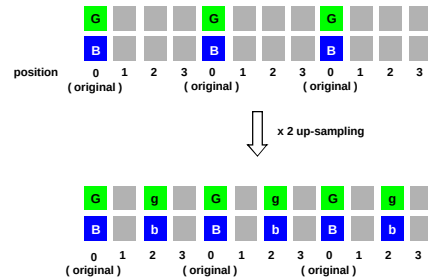
The horizontal resampler sub-module performs a seamless horizontal down-sampling or up-sampling (zoom). Note: up-sampling is only possible when operating in SDRAM mode, i.e. when bit INMODE of register PVSET1 - Preview Setup Register #1 is set to '1'. (see also section 10.4.2 Preview Engine Input Modes).

This seamless resampling processing is done using a bi-linear interpolation algorithm on data from the smoother sub-module

The horizontal resampler uses the following equations to calculate the zoom, and Figure 49 shows an example of this resampling process used to apply x2 zoom.

$$x_0 = \begin{cases} X_0 & (\text{position 0}) \\ \frac{X_0 + 3X_1}{4} & (\text{position 1}) \\ \frac{X_0 + X_1}{2} & (\text{position 2}) \\ \frac{3X_0 + X_1}{4} & (\text{position 3}) \end{cases}$$

Figure 49. x2 Zoom Resampling



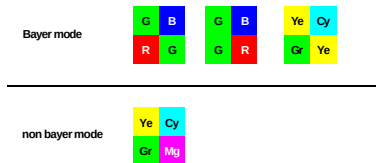
See also section 10.4.3 Defining Output Image Size for details about setting resizing registers.

10.3.4.4 Vertical Interpolation and Resampler : CFA Sub-module

The vertical interpolation and resampler sub-module performs vertical interpolation and seamless vertical down-sampling or up-sampling. Note: up-sampling is only possible when operating in SDRAM mode, i.e. when bit INMODE of register PVSET1 - Preview Setup Register #1 is set to '1' (see also section 10.4.2 Preview Engine Input Modes).

Like the horizontal resampler module, this vertical resampling applies a bi-linear interpolation algorithm on data from the horizontal interpolation sub-module. This sub-module executes two different algorithms depending on the CCD's CFA pattern. Figure 50 shows an example of two types of the CFA patterns.

Figure 50. Type of CFA pattern



If the CFA pattern is of the type that has one color component in both lines, then this sub-module operates in bayer mode. Otherwise, this sub-module operates in non-bayer mode. In bayer mode, the vertical interpolation and resampling processing is based on the main color (G or Ye in Figure 50). As shown in Figure 50, in bayer mode one main color (G or Ye in this figure) exists in every

line, and the other colors are calculated based on the main color. (see section 10.4.3 Defining Output Image Size for details about setting resizing registers)

10.3.4.5 Color Selector : CFA Sub-module

The output of the vertical resampler sub-module is the input for the color selector. The color selector sub-module extracts an appropriate color from the four input signals which the vertical interpolation and resampler sub-module generates.

Similar to other modules within the Preview Engine, the color selector sub-module, uses bits 7-0 of the register PVSET2 - Preview Setup Register #2 to control which pixels in the input data correspond to G/Ye, B/Cy, R/Mg, or Gr.

10.3.4.6 Color Adjustment for the Complementary Color Space : CFA Sub-module

After applying the interpolation algorithm, data in the Complementary color space can contain artifacts. The color adjustment sub-module performs color adjustment on the each of the four Complementary components, such that the following equation is satisfied.

$$Ye + Cy = Gr + Mg$$

To enable the color adjustment module set field CCDMOD in the register PVSET2 - Preview Setup Register #2

10.3.4.7 Comp2RGB conversion : CFA Sub-module

The comp2RGB conversion sub-module transforms data from the Complementary color space to the RGB color space. The comp2RGB sub-module is bypassed when operating in RGB mode (field CCDMOD of register PVSET2 - Preview Setup Register #2 is cleared).

The green blend ratio, 'g', is used to adjust the blending ratio between the original green value, G, and the new green value, 'g', calculated using the other three colors, Ye, Cy, and Mg.

The green blend ratio can be set to 0, 1/4, 1/2, 3/4, or 1 by setting GBLND[2]-GBLND[0] in the register PVSET2 - Preview Setup Register #2. Similar to the color adjustment sub-module, the comp2RGB sub-module is enabled by setting CCDMD in PVSET2.

10.3.5 Black Adjustment Sub-Module

After CFA interpolation, black adjustment can be applied to each color component, R, G, and B. A different offset can be applied to each component. The color adjustment sub-module performs the following calculation on each color component.

$$\text{data_out} = \text{data_in} + \text{bl_offset}$$

Black adjustment offset is specified as a 8-bit 2's complement value and is set using register BLOFST0 - Black Level Offset Adjust #0, register BLOFST1 - Black Level Offset Adjust #1

10.3.6 RGB2RGB Blending Module

The RGB2RGB blending module transforms the RGB data generated by the CFA interpolation module using a 3x3 square matrix transformation in combination with an added offset. The RGB2RGB blending module computes the following calculation.

$$\begin{bmatrix} R_{out} \\ G_{out} \\ B_{out} \end{bmatrix} = \begin{bmatrix} MTX_RR & MTX_GR & MTX_BR \\ MTX_RG & MTX_GG & MTX_BG \\ MTX_RB & MTX_GB & MTX_BB \end{bmatrix} \begin{bmatrix} R_{in} \\ G_{in} \\ B_{in} \end{bmatrix} + \begin{bmatrix} R_offset \\ G_offset \\ B_offset \end{bmatrix}$$

The registers used for RGB2RGB blending are as follows:

- o MTXGAIN0 - Matrix Gain register #0
- o MTXGAIN1 - Matrix Gain register #1
- o MTXGAIN2 - Matrix Gain register #2
- o MTXGAIN3 - Matrix Gain register #3
- o MTXGAIN4 - Matrix Gain register #4
- o MTXOFST0 - R_offset register
- o MTXOFST1 - G_offset register
- o MTXOFST2 - B_offset register

The precision of MTXGAINx register is S8Q6. Offset is specified as a 8-bit 2's complement number

MTXGAINx[7:0]	Gain value
0111 1111	1.984375 (1+63/64)
0100 0000	1.0
0000 0001	0.015625 (1/64)
0000 0000	0.0
1111 1111	-0.015625 (-1/64)
1100 0000	-1.0
1000 0000	-2.0

See section 10.5 Fractional Notation for description of fractional notation used in this chapter

10.3.7 Gamma Correction Module

The gamma correction module applies gamma correction to each color component, R, G, and B. Two gamma tables are stored in the Preview Engine's internal ROM. Set TBSEL bit in register GAMTSBYP - GAMMA table bypass register to select gamma table 1, and clear TBSEL bit to select gamma table 0. Set the bypass bits (BYPR, BYPG, or BYPB) in register GAMTSBYP - GAMMA table bypass register to bypass the gamma tables. The gamma tables can be bypassed for each color component, independently. When the gamma table is bypassed the input data is divided by four.

Gamma Correction module exists independently for each color so that the independent setting is possible.

Figure 51 shows a block diagram of the gamma correction module. It is composed of two ROM tables and one selector. When the BYPASS bit is asserted, the input data is only divided by four. Figure 52, Figure 53 show the characteristic of two gamma tables. See Appendix for exact gamma table values

Figure 51. Gamma Correction Module Block Diagram

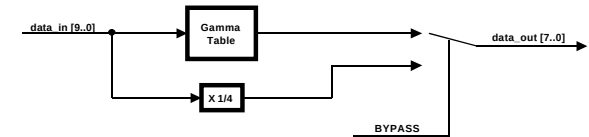


Figure 52. Preview Engine ROM Gamma Table 0 transformation function

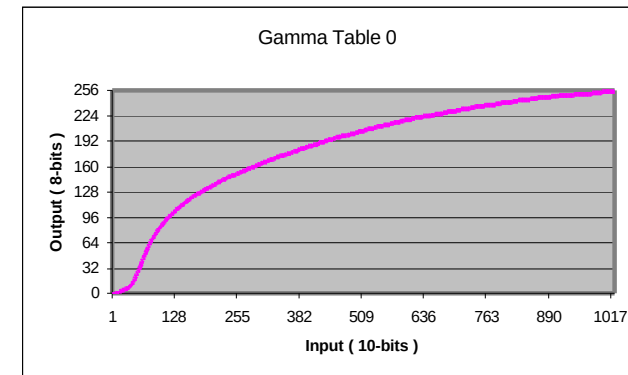
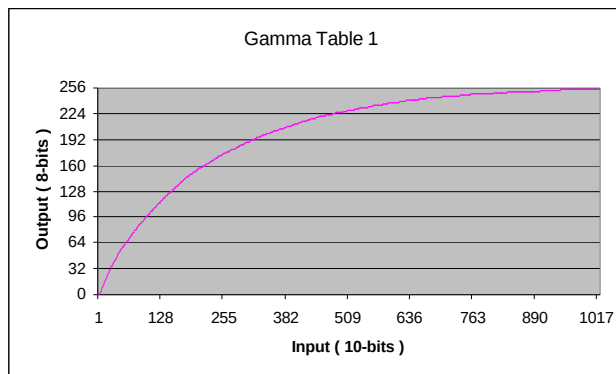


Figure 53. Preview Engine ROM Gamma Table 1 transformation function



10.3.8 RGB2YCbCr Conversion Module

The RGB to YCbCr conversion module converts RGB formatted data to YCbCr formatted data and adjusts Cb and Cr with offset values. The following matrix calculation is used to calculate Y, Cb, and Cr.

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} CSC_RY & CSC_GY & CSC_BY \\ CSC_RCB & CSC_GCB & CSC_BCB \\ CSC_RCR & CSC_GCR & CSC_BCR \end{bmatrix} \begin{bmatrix} Rin \\ Gin \\ Bin \end{bmatrix} + \begin{bmatrix} y_offset \\ Cb_offset \\ Cr_offset \end{bmatrix}$$

The registers used for RGB2YcbCr blending are as follows:

- o CSC0 - Color Space Conversion Coefficient register #0
- o CSC1 - Color Space Conversion Coefficient register #1
- o CSC2 - Color Space Conversion Coefficient register #2
- o CSC3 - Color Space Conversion Coefficient register #3
- o CSC4 - Color Space Conversion Coefficient register #4
- o YOFST - Y offset register
- o COFST - Cb and Cr offset register
- o CNTBRT - Contrast and brightness adjustment register

The precision of CSCx register is S8Q6

CSCx[7:0]	Gain value
0111 1111	1.984375 (1+63/64)
0100 0000	1.0
0000 0001	0.015625 (1/64)
0000 0000	0.0
1111 1111	-0.015625 (-1/64)
1100 0000	-1.0
1000 0000	-2.0

See section 10.5 Fractional Notation for description of fractional notation used in this chapter

In addition to this color space operation, contrast (scaling/multiplication) and brightness (offset/addition) adjustments can be made on the luminance data and pseudo-color suppression can be performed for chrominance data.

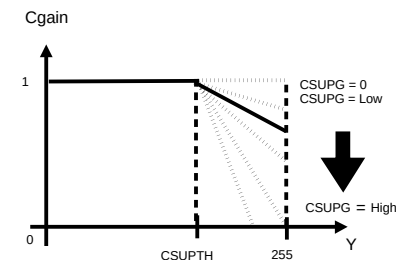
10.3.8.1 Pseudo-color suppression

The gain factor applied to Cb and Cr during pseudo-color suppression is determined using a configurable function mapping. The functional mapping is shown in Figure 54. The pseudo-color suppression can be enabled using register CSUP0 - Chroma Suppression Register #0. The pseudo-color suppression threshold and gain can be set in the register CSUP1 - Chroma Suppression Register #1. Use CSUPTH to set the threshold constant and use CSUPG to set the gain constant.

Cgain is suppressed smaller than one while luminance signal level is higher than CSUPTH.

```
IF ( Y > CSUPTH )
    Cgain = 1 - ( Y - CSUPTH ) / 256 * CSUPG
ELSE
    Cgain = 1
```

Figure 54. Psuedo-color suppression



Precision of CSUG is U8Q8

CSUPG[7:0]	Value
1111 1111	0.99609375
1111 1110	0.99218750
...	...
0000 0001	0.00390625
0000 0000	0.00000000

See section 10.5 Fractional Notation for description of fractional notation used in this chapter

10.3.9 Output Clipping Module

Before outputting data to the SDRAM, Preview Engine performs clipping on the data to output to SDRAM. The minimum and maximum threshold values are specified using register SETUPY - Y max and min setup register, register SETUPC - Cb and Cr max and min setup register. By default, these values are all zero, so user needs to set appropriate value in order to get output from the preview engine. For example 0xFF for maximum Y, Cb, Cr value and 0 for minimum Y, Cb, Cr value means no clipping of output data

This setting must be done for all modes of operation including normal CCD raw-data processing mode, one-shot mode, resize only mode, SDRAM input mode

10.3.10 4:2:2 Conversion and Output Interface Module

The 4:2:2 conversion module converts the image data to YCbCr-4:2:2 format by discarding every other Cb and Cr component in the horizontal direction. After converting the data to YCbCr-4:2:2, the image data is transferred to the SDRAM controller (SDRC) in units of 32 bytes. The SDRC manages the transfer of the image data to SDRAM. The Preview Engine has an internal 16-bit by 16 FIFO that is used buffer the data. Data is transferred from the FIFO to the SDRC in 32-byte bursts. Figure 55 shows how the YCbCr image data is formatted in SDRAM.

Figure 55. YCbCr-4:2:2 Format

MSB				LSB			
31	24	23	16	15	8	7	0
Y1				Cr0			
Y0				Cb0			

10.4 Preview Engine Operation Guide

Except for selecting the SDRAM data bus width (32-bits or 16-bits), activation of the preview engine is determined by registers, which are written through the ARM Bus (SDRAM data bus width is determined by a special input signal from SDRAMC). This section explains how to configure and use the Preview Engine.

See section 0 Preview Engine Registers for details of each register.

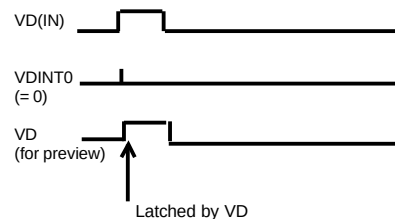
Some Preview Engine register bits are latched by the VD signal. The VD signal is the vertical synch pulse generated by the CCDC or input to the CCDC from an external CCD imager. Data written to latched registers does not take affect until the VD pulse is received. This allows bits that control the Preview Engine, such as the Preview Engine enable bit and the SDRAM data source and destination bit field, to be changed in between successive VD pulses without corrupting the current image being processed by the Preview Engine. The Preview Engine register descriptions marked with (*) indicate that the corresponding bits are latched by the VD signal.

Incase of VD input mode (In below chart, VDPOL and VDINT0 are set at CCDC module), preview engine registers are always latched on rising edge of the VD signal, irrespective of VD polarity, as shown in the below figure.

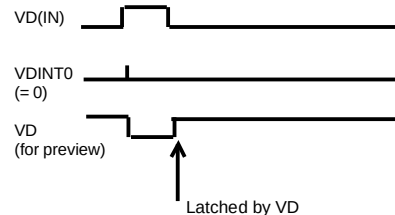
When VD is output by CCDC, CCDC internally always generate VD signal with high polarity and before outputting the signal it inverts the VD based on VDPOL bit. Thus, when VD is output by CCDC, preview engine registers will always be latched at the start of VD.

Figure 56. VD Latch timing

In case of VDPOL = 0



In case of VDPOL = 1



Registers latched by VD signal are as follows.

- o PVEN - Preview Enable Register
- o RSONLY bit in PVSET1 - Preview Setup Register #1
- o RADRH - SDRAM Read Address - high order bits
- o RADRL - SDRAM Read Address - low order bits
- o WADRH - SDRAM Write Address - high order bits
- o WADRL - SDRAM Write Address - low order bits
- o HSTART - Start pixel horiz. (for CCD input) or Horiz. Size in SDRAM (for SDRAM input)
- o HSIZE - Number of pixels horizontal
- o VSTART - Start line vertical
- o VSIZE - Number of pixels vertical

10.4.1 Starting the Preview Engine

Setting bit 0 of register PVEN - Preview Enable Register activates the Preview Engine. Clearing this bit deactivates the Preview Engine. The status of the preview engine can be verified by reading PVEN. Also Preview Engine interrupt is generated when the preview engine changes from enabled state to disabled state. PVEN is latched by VD. PVEN timing is shown in Figure 57.

When bit 0 of PVEN is set, the Preview Engine becomes active at the next VD. When bit 0 of PVEN is cleared the Preview Engine becomes inactive after completing the last burst transfer of processed image data.

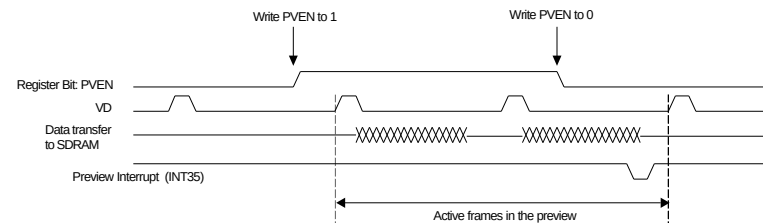
Furthermore when the Preview Engine becomes inactive the Preview Engine interrupt signal is issued. This interrupt always triggers after completing the last burst transfer of processed image data. This interrupt corresponds to INT35 in the Interrupt Controller module. Refer to the "Interrupt Controller" section for more information about this interrupt.

IMPORTANT: It is recommended that the preview engine is made inactive by clearing bit 0 of PVEN register when the following registers are changed.

PVSET1, HSRT, HSIZE, VSRT, VSIZE, SMTH, HRSZ, VRSZ

NOTE: Preview Engine does not trigger Preview Engine interrupt after processing of each and every frame of data. It is triggered when 1) Preview Engine is disabled and last burst transfer of the last frame of processed image data is done, 2) when in one-shot mode, last burst transfer of the processed image data is done

Figure 57. Preview Enable/disable timing



10.4.2 Preview Engine Input Modes

The preview engine has two selectable input ports, which are connected to the CCDC and the SDRAMC. There are some differences of available functions between the CCDC input mode and the SDRAMC input mode. shows the main difference of available functions between the two input modes. Normal mode means RAW data input mode and Resize Only mode means YCbCr data input mode

IMPORTANT: When Preview Engine is configured to take input from SDRAM (YCbCr input data during YCbCr resize or raw data input), in order for Preview Engine to operate the following conditions must be satisfied

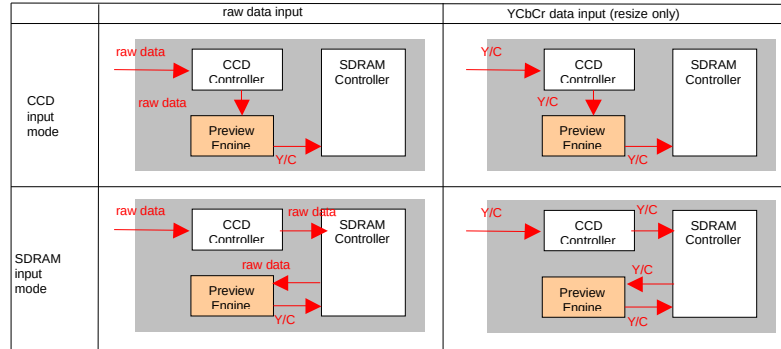
- o The clock to CCD (CLK_CCD signal, see Clock Controller section) must be active even if CCD controller module is not being used. The clock to CCD (CLK_CCD) can either be externally provided pixel clock (PCLK) or external reference clock input (SYSCLK), (see Clock Controller Section for details about select CCD clock)
- o CCD controller HD/VD signal must be active (VDHDEN field in register SYNCEN in CCD controller).

Figure 58. Preview Engine : Input Mode Comparison

Functions	CCDC input mode		SDRC input mode	
	Normal	Resize Only	Normal	Resize Only
Noise filter	O	X	O	X
Digital gain	O	X	O	X
White balance	O	X	O	X
Smoothing	O	O	O	O
Down sample	O	O	O	O
Zoom(Up sample)	X	X	O	O
RGB blend	O	X	O	X
Gamma	O	X	O	X
Chroma suppress	O	X	O	X
Chroma offset	O	X	O	X
1 shot preview	O	O	O	O

O = available, X = not available

Figure 59. Data Flow in CCDC, SDRAM and Preview Engine



- Clearing the INMODE bit in register PVSET1 - Preview Setup Register #1 configures the Preview Engine to be in CCDC mode, and setting the INMODE bit configures the Preview Engine to be in SDRAMC mode.
- When operating in CCDC or SDRAMC mode, the user must specify an SDRAM destination. This is the location in SDRAM where the Preview Engine will write the processed image data.
- When operating in SDRAM input mode, the user must also specify an SDRAM input data address. This is the location in SDRAM where the Preview Engine will read the data to be processed.
- The SDRAM read/write address is specified as offset from start of SDRAM and in units of 32 bytes, since data is transferred between the Preview Engine and SDRAMC in bursts of 32 bytes.

For example suppose the SDRAM address 0x911000 is to be set as the Preview Engine write address then the value that is to be programmed in the Preview Engine destination address registers (WADRL, WADRH) is calculated as follows:

$$(0x911000 - 0x900000)/32 = (0x11000)/32 = 0x880$$

- When Preview Engine is configured to take input from SDRAM, the register HSTART - Start pixel horiz. (for CCD input) or Horiz. Size in SDRAM (for SDRAM input), must be set with the pixel width of input data present in the SDRAM.
- Furthermore, if the BSTAL bit is set to 1 in register PVSET1 - Preview Setup Register #1, then the horizontal size of pixel data transferred to the SDRAMC will be aligned on a 32-byte boundary, i.e. if the horizontal size is not a multiple of 32-bytes, the excess data will not be transferred to the SDRAMC.
- MAXY, MINY fields in register SETUPY - Y max and min setup register, SETUPC - Cb and Cr max and min setup register, must be set for all modes of operation of the Preview Engine, which includes raw data input from CCD, raw data input from SDRAM, YUV resize only mode, one-shot mode. These registers specify the clipping to be applied to the Y, Cb, Cr pixel data before it is output to SDRAM.

10.4.2.1 SDRAM Input mode - input pixel width selection

The preview engine has three selectable input modes in SDRAM input mode.

- 10bits mode (8BITS = 0 and ALAW = 0 and INMODE = 1 in PVSET1 register)
- 8bits mode (8BITS = 1 and ALAW = 0 and INMODE = 1 in PVSET1 register)
- Inverse A-law mode (8BITS = 1 and ALAW = 1 and INMODE = 1 in PVSET1 register).

NOTE: Inverse A-law mode is available only when preview engine is in SDRAM input mode. 10bits mode and 8bits mode is available in both SDRAM input mode as well as CCDC input mode

10.4.2.1.1 10bits mode

In 10bits mode, raw data must be stored in SDRAM as shown below,

31	24	23	16	15	8	7	0
x x x x	Pixel N+1 (25:16)				x x x x	Pixel N (9:0)	

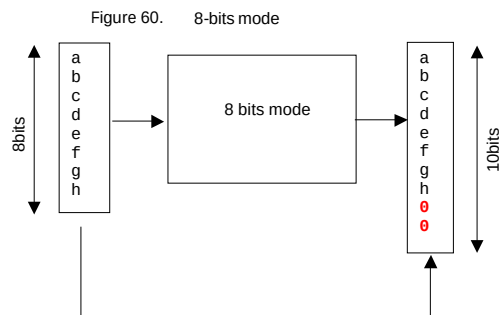
X = invalid bits

10.4.2.1.2 8bits mode

In this mode each input raw data pixel has width of 8bits. The raw data stored in SDRAM as shown below

31	24	23	16	15	8	7	0
Pixel N+3		Pixel N+2		Pixel N+1		Pixel N	

Internally preview engine always processes raw data with 10 bits per pixel. When processing the 8-bit raw data, the preview engine expands the data from 8 bits to 10 bits by adding "00" in LSB side as shown in the below figure.



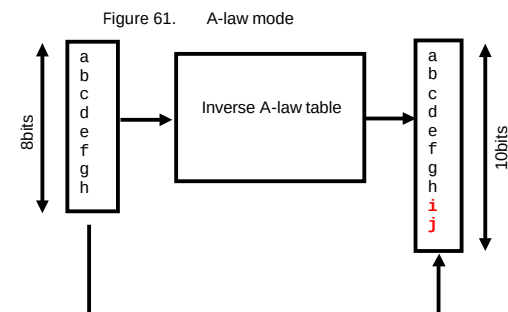
10.4.2.1.3 A-law Mode

In this mode each input raw data pixel has width of 8bits. The raw data stored in SDRAM as shown below

31	24	23	16	15	8	7	0
Pixel N+3		Pixel N+2		Pixel N+1		Pixel N	

Internally preview engine always processes raw data with 10 bits per pixel. When processing the 8-bit raw data, the preview engine expands the data from 8 bits to 10 bits by using inverse A-law table. Refer to ITU-T G.711 specification for A-law/Inverse A-law characteristics

NOTE: Inverse A-law mode is available only when preview engine is in SDRAM input mode.



10.4.2.2 14-bit control bit

This control bit is assigned to bit 15 of register SMTH - Smoothing Definition register.

Figure 62. CCDC SDRAM Data Format

	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
14bit	Pixel0		Pixel1	
13bit	Pixel0		Pixel1	
12bit	Pixel0		Pixel1	
11bit	Pixel0		Pixel1	
10bit	Pixel0		Pixel1	
9bit	Pixel0		Pixel1	
8bit	Pixel0		Pixel1	
8bit pack	Pixel0		Pixel2	Pixel3

The CCDC SDRAM data format is shown above.

On ES1.0 chip, when preview engine is in SDRAM fetch mode (INMODE bit of PVSET1 is 1), preview engine fetches pixel1[9:0] and pixel0[9:0] (lower part data) regardless of CCDC store mode. Thus, when CCDC store mode is 14bit mode, MSB 4bits of stored pixel data are discarded.

This is a problem since preview engine SDRAM fetch mode can be used only when CCDC store mode is 10 bit.

On ES1.1 or later chip, preview engine can select fetch location of stored data by using 10BITS bit of register SMTH - Smoothing Definition register when it is in SDRAM fetch mode.

When SMTH:10BITS is,

0: pixel1[9:0] and pixel0[9:0] is read by preview engine

1: pixel1[13:4] and pixel0[13:4] is read by preview engine

So now preview engine can support greater than 10-bit store mode of CCDC in SDRAM fetch mode.

10.4.3 Defining Output Image Size

The size of the processed Preview Engine output image is determined by the following registers.

- o HSIZE - Number of pixels horizontal
- o VSIZE - Number of pixels vertical
- o HRSZ - Horizontal Resize Register
- o VRSZ - Vertical Resize Register

IMPORTANT: Up-sampling is only possible when operating in SDRAM mode, i.e. when bit INMODE of PVSET1 register is set to 1.

IMPORTANT: Depending on the values of the HRSZ and the VRSZ, the preview engine may send an extra burst (32-bytes) to SDRAM. Please allow an SDRAM work area that is at least 32-bytes larger than the expected output image size.

10.4.3.1 Burst Alignment

The Preview Engine always transfers data to the SDRC in bursts of 32-bytes. If you clear BSTAL in PVSET1 register to 0, and configure the Preview Engine such that number of horizontal pixels output (Hout), is not a multiple of 32-bytes (16 pixels), then the Preview Engine will transfer Hout in addition to some excess data in the final 32-byte transfer per horizontal line. Please be aware that in this case you must reserve additional bytes of SDRAM for the excess data, in-order-to avoid unexpected overwriting of an important segment of SDRAM.

If you set BSTAL in PVSET1 register to 1 and configure the Preview Engine such that Hout is a multiple of 32-bytes (16 pixels), the horizontal size of transferred data will be burst aligned, and excess pixel data will be discarded.

```
i.e, when BSTAL=0,
if(Hresizeout%16 != 0)
    Hout = Hresizeout + 16 - Hresizeout%16
else
```

Hout = Hresizeout

and when BSTAL=1, Hout = Hresizeout - (Hresizeout%16)

where, Hout, is number of pixels per line output to SDRAM
and Hresizeout, is number of pixels per line after resizing
NOTE: 1 pixel = 16 -bits

10.4.3.2 Output Image Size Calculation

The size of the processed output image when operating in SDRAM mode and CCDC mode are given by the equation show below. The equation for the input from SDRAM mode and the input from CCDC mode are identical, except for the minimum value for HRSZD and VRSZD, since up-sampling is not supported in the input for CCDC mode.

NOTE : When Preview Engine is configured to take input from SDRAM, the register HSTART - Start pixel horiz. (for CCD input) or Horiz. Size in SDRAM (for SDRAM input) must be set with the pixel width of input data present in the SDRAM. See figure below

Figure 63. SDRAM Input Mode Output Size Calculation

When INMODE=1,

$$H_{out} = \begin{cases} HSIZE \times \frac{16}{HRSZD} & (\text{if } HRSZD \geq 0x010 \text{ and } HRSZD \leq 0x2FF) \\ (HSIZE - 1) \times \frac{16}{HRSZD} & (\text{if } HRSZD \geq 0x004 \text{ and } HRSZD < 0x010) \\ \text{meaningless} & (\text{if } HRSZD < 0x004 \text{ or } HRSZD > 0x2FF) \end{cases}$$

When INMODE = 1,

$$V_{out} = \begin{cases} VSIZE \times \frac{16}{VRSZD} & (\text{if } VRSZD \geq 0x004 \text{ and } VRSZD \leq 0x2FF) \\ \text{meaningless} & (\text{if } VRSZD < 0x004 \text{ or } VRSZD > 0x2FF) \end{cases}$$

**INMODE = 1 (input from SDRAM)
register settings**

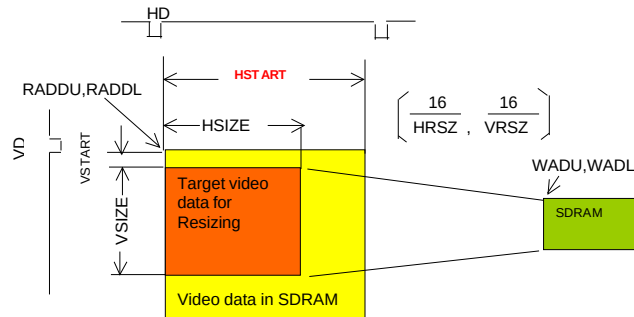


Figure 64. CCDC Input Mode Output Size Calculation

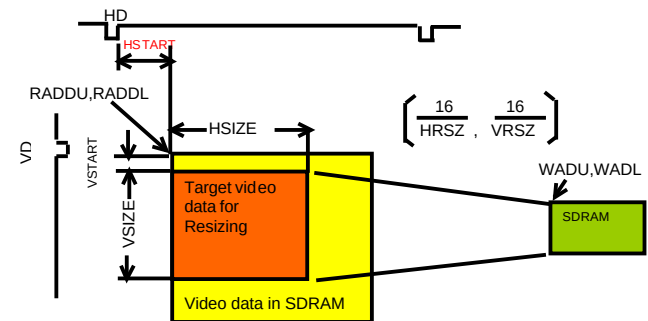
When INMODE = 0,

$$H_{out} = \begin{cases} HSIZE \times \frac{16}{HRSZD} & (\text{if } HRSZD \geq 0x010 \text{ and } HRSZD \leq 0x2FF) \\ \text{meaningless} & (\text{if } HRSZD < 0x010 \text{ or } HRSZD > 0x2FF) \end{cases}$$

When INMODE = 0,

$$V_{out} = \begin{cases} VSIZE \times \frac{16}{VRSZD} & (\text{if } VRSZD \geq 0x010 \text{ and } VRSZD \leq 0x2FF) \\ \text{meaningless} & (\text{if } VRSZD < 0x010 \text{ or } VRSZD > 0x2FF) \end{cases}$$

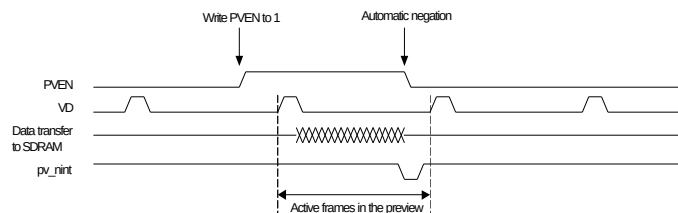
**INMODE = 0 (input from CCDC)
register settings**



10.4.4 One-shot preview

When the PVOS bit in the register PVSET1 - Preview Setup Register #1 is set, the Preview Engine will process only one frame of data, and then automatically deactivate. Figure below shows signal timings in one-shot mode.

Figure 65. One-shot Mode Timings



10.4.5 Resize only mode

When the RSONLY bit in the register PVSET1 - Preview Setup Register #1 is set, the Preview Engine will operate in resize only mode. See the following sections for details:

- o 10.3.4 CFA
- o 10.4.2 Preview Engine Input Modes
- o 10.4.3 Defining Output Image Size
- o 10.4.6 Preview Engine Operation Examples, Example 2.

10.4.6 Preview Engine Operation Examples

This section gives few examples on configuration and control of preview engine. Refer to "Appendix" for description about the CSL macro notation used in this section.

10.4.6.1 Example 1.

CCD input raw data size is 1536x240, 10-bits per pixel
Required Preview engine output size is 720x240
Preview Engine data output address is 0x97AB00

```
Uint16 input_width = 1536;
Uint16 input_height = 240;
Uint16 output_width = 720;
Uint16 output_height = 240;
Uint32 write_addr = 0x97AB00;
```

```
PREV_RSET(PVEN, 0); // disable Preview Engine
while(PREV_RGET(PVEN)) // wait till preview is disabled
;
```

```
PREV_FSET(PVSET1, 8BITS, 0); // 10 bits input mode
PREV_FSET(PVSET1, ALAW, 0); // Normal Mode
PREV_FSET(PVSET1, PVOS, 0); // Normal Mode -continuous processing
PREV_FSET(PVSET1, RSONLY, 0); // Normal Mode -raw CCD data
PREV_FSET(PVSET1, INMODDE, 0); // CCDC Input Mode
PREV_FSET(PVSET1, BSTAL, 1); // burst align mode
PREV_FSET(PVSET1, WEN, 0); // ignore WEN signal
```

```
write_addr = (write_addr - 0x900000)/32;
PREV_RSET( WADRH, write_addr >> 16 );
PREV_RSET( WADRL, write_addr & 0xFFFF );
```

```
PREV_RSET( HSTART, . . . ); // depends on CCD module
PREV_RSET( VSTART, . . . ); // depends on CCD module
```

```
prev_hsize = input_width;
prev_vsize = input_height;
```

```
prev_hrsz = ( prev_hsize * 16 ) / output_width;
prev_vsize = ( prev_vsize * 16 ) / output_height;
```

```
// since for horizontal upscaling Hout = (HSIZE -1)*16/HRSZ
if(output_width > input_width )
    prev_hsize++;
```

```
prev_vrsz = ( prev_vsize* 16 ) / output_height;
prev_hrsz = ( prev_hrsz * 16 ) / output_width;
```

```
/**
Since 1536/720 = 2.13333 resize ratio is not supported by preview
engine, only 1530 pixels are taken as input so that (1530*16)/34 =
720 pixels will be output
**/
PREV_RSET( HSIZE, prev_hsize); // HSIZE = 1530
PREV_RSET( HRSZ, prev_hrsz); // HRSZ = 34
PREV_RSET( VSIZE, prev_vsize); // VSIZE = 240
PREV_RSET( VRSZ, prev_vrsz); // VRSZ = 16
```

```
// setup other registers depending on CCD raw data properties

// no clipping of Y, C output data
PREV_RSET( SETUPY, 0x00FF);
PREV_RSET( SETUPC, 0x00FF);

PREV_RSET( PVEN, 1 ); // enable Preview Engine
```

10.4.6.2 Example 2.

SDRAM input raw data size is 1536x240, 10-bits per pixel
 Required Preview engine output size is 720x480
 Preview Engine data output address is 0x97AB00
 Preview Engine data input address is 0xA7AB00

One-shot mode processing is required.

NOTE: CCD clock must be active and CCDC, HD/VD must be enabled in order for Preview Engine to operate in SDRAM input mode

```
UInt16 input_width = 1536;
UInt16 input_height = 240;
UInt16 output_width = 720;
UInt16 output_height = 480;
UInt32 write_addr = 0x97AB00;
UInt32 read_addr = 0xA7AB00;

PREV_RSET(PVEN, 0); // disable Preview Engine
while(PREV_RGET(PVEN)) // wait till preview is disabled
;

PREV_FSET(PVSET1, 8BITS, 0 ); // 10 bits input mode
PREV_FSET(PVSET1, ALAW, 0); // Normal Mode
PREV_FSET(PVSET1, PVOS, 1); // Enable one-shot mode
PREV_FSET(PVSET1, RSONLY, 0); // Normal Mode -raw CCD data
PREV_FSET(PVSET1, INMODE, 1); // SDRAM Input Mode
PREV_FSET(PVSET1, BSTAL, 1); // burst align mode
PREV_FSET(PVSET1, WEN, 0); // ignore WEN signal

write_addr = (write_addr - 0x900000)/32;
PREV_RSET( WADRH, write_addr >> 16 );
PREV_RSET( WADRL, write_addr & 0xFFFF );

read_addr = (read_addr - 0x900000)/32;
PREV_RSET( RADRH, read_addr >> 16 );
PREV_RSET( RADRL, read_addr & 0xFFFF );

// horizontal size of data in SDRAM, units of pixels
PREV_RSET( HSTART, input_width);

// read data from 1st line in SDRAM
PREV_RSET( VSTART, 0);

prev_hsize = input_width;
prev_vsize = input_height;

prev_hrsz = ( prev_hsize * 16 ) / output_width;
prev_hsize = ( prev_hrsz * output_width ) / 16;

// since for horizontal upscaling Hout = (HSIZE -1)*16/HRSZ
if(output_width > input_width )
    prev_hsize++;
```

```
prev_vrsz = ( prev_vsize* 16 ) / output_height;
prev_vsize = ( prev_vrsz * output_height ) / 16;

/**
Since 1536/720 = 2.13333 resize ratio is not supported by preview
engine, only 1530 pixels are taken as input so that (1530*16)/34 =
720 pixels will be output
***/

PREV_RSET( HSIZE, prev_hsize); // HSIZE = 1530
PREV_RSET( HRSZ , prev_hrsz); // HRSZ = 34
PREV_RSET( VSIZE, prev_vsize); // VSIZE = 240
PREV_RSET( VRSZ , prev_vrsz); // VRSZ = 8

// setup other registers depending on CCD raw data properties

// no clipping of Y, C output data
PREV_RSET( SETUPY, 0x00FF);
PREV_RSET( SETUPC, 0x00FF);

// enable Preview Engine, start one shot mode
PREV_RSET( PVEN, 1 );

/**
wait for preview engine interrupt to indicate completion of one
shot mode
***/
```

10.5 Fractional Notation

The fractional notation used in the subsequent sections is as follows:

$$(S | U) \times Q_y$$

is used to denote a fractional numbers.

Here,

'x', represents the total number of bits in the real number including the fractional part

'y', represents the number of bits allocated to the fractional part. i.e (x - y) is the number of bits allocated to the integer part

'S', indicates that the number is signed, with negative numbers represented in 2's complement format

'U', indicates that the number is unsigned.

For example,

U8Q5, means that the value can range from 0 to 7.96875

S8Q6, means that the value can range from -2 to 1.984375

10.6 Preview Engine Register Map (PREV)

Address	Register	Description
0003:0780	PVEN	Preview Enable Register
0003:0782	PVSET1	Preview Setup Register #1
0003:0784	RADRH	SDRAM Read Address - high order bits
0003:0786	RADRL	SDRAM Read Address - low order bits
0003:0788	WADRH	SDRAM Write Address - high order bits
0003:078A	WADRL	SDRAM Write Address - low order bits
0003:078C	HSTART	Start pixel horizontal (for CCD input) Horizontal Size in SDRAM (for SDRAM input)
0003:078E	HSIZE	Number of pixels horizontal
0003:0790	VSTART	Start line vertical
0003:0792	VSIZ	Number of pixels vertical
0003:0794	PVSET2	Preview Setup Register #2
0003:0796	NFLT	Noise Filter Definition Register
0003:0798	DGAIN	Digital Gain Register
0003:079A	WBGAIN0	White Balance Gain register #0
0003:079C	WBGAIN1	White Balance Gain register #1
0003:079E	SMTH	Smoothing Definition register
0003:07A0	HRSZ	Horizontal Resize Register
0003:07A2	VRSZ	Vertical Resize Register
0003:07A4	BLOFST0	Black Level Offset Adjust #0
0003:07A6	BLOFST1	Black Level Offset Adjust #1
0003:07A8	MTXGAIN0	Matrix Gain register #0
0003:07AA	MTXGAIN1	Matrix Gain register #1
0003:07AC	MTXGAIN2	Matrix Gain register #2
0003:07AE	MTXGAIN3	Matrix Gain register #3
0003:07B0	MTXGAIN4	Matrix Gain register #4
0003:07B2	MTXOFST0	R_offset register
0003:07B4	MTXOFST1	G_offset register
0003:07B6	MTXOFST2	B_offset register
0003:07B8	GAMTSBYP	GAMMA table bypass register
0003:07BA	CSC0	Color Space Conversion Coefficient register #0
0003:07BC	CSC1	Color Space Conversion Coefficient register #1
0003:07BE	CSC2	Color Space Conversion Coefficient register #2
0003:07C0	CSC3	Color Space Conversion Coefficient register #3
0003:07C2	CSC4	Color Space Conversion Coefficient register #4
0003:07C4	YOFST	Y offset register
0003:07C6	COFST	Cb and Cr offset register
0003:07C8	CNTBRT	Contrast and brightness adjustment register
0003:07CA	CSUP0	Chroma Suppression Register #0
0003:07CC	CSUP1	Chroma Suppression Register #1
0003:07CE	SETUPY	Y max and min setup register
0003:07D0	SETUPC	Cb and Cr max and min setup register

10.7 Preview Engine Registers

10.7.1 PVEN - Preview Enable Register

[illegible][illegible]

10.7.2 PVSET1 - Preview Setup Register #1

PVSET1											0003:0782				offset: 0x02								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0										
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	8BITS	ALAW	PVOS	RSONLY	INMODE	BSTAL	WEN										
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W										
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0										

Bit	Name	Reset Value	R/W	Function
15: 7	RSV			Reserved
6	8BIT S	0	R/W	<u>Data from CCDC is 8-bits width for each color</u> » 0: 10-bit mode « » 1: 8-bit mode «
5	ALAW	0	R/W	<u>Inverse A-LAW mode</u> » 0: Normal mode « » 1: Inverse A-LAW mode « *This bit is latched by VD.
4	PVOS	0	R/W	<u>Preview One Shot</u> » 0: normal (continuous processing) « » 1: one frame only «
3	RSONLY	0	R/W	<u>Re-Size Only - Select type of input data</u> » 0: Normal (raw CCD data) « » 1: Resizing only mode (YCbCr-4:2:2 data input) « *This bit is latched by VD.
2	INMODE	0	R/W	<u>Input Mode</u> » 0: CCD Controller Interface « » 1: SDRAM Controller Interface «
1	BSTAL	0	R/W	<u>Burst Aligned</u> » 0: normal « » 1: burst aligned mode « Apply a burst aligned transfer of output image to SDRAMC. If it is set to 1, the horizontal size is aligned to the burst length and surplus pixels are disregarded.
0	WEN	0	R/W	<u>Write Enable</u> » 0: ignore « » 1: valid « Select handling of a write enable signal from the CCDC. If set to 1 and the write enable signal is low, the preview engine is suspended in the frame.

10.7.3 RADRH - SDRAM Read Address - high order bits

RADR bit		0003:0784				offset: 0x04								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]	
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	

[illegible]

10.7.4 RADRL - SDRAM Read Address - low order bits

[illegible][illegible]

10.7.5 WADRH - SDRAM Write Address - high order bits

ADDRH		0003:7888				offset:		0x08												default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]							
Default	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W							

[illegible]

10.7.6 WADRL - SDRAM Write Address - low order bits

[illegible][illegible]

10.7.7 HSTART - Start pixel horiz. (for CCD input) or Horiz. Size in SDRAM (for SDRAM input)

[illegible][illegible]

10.7.8 HSIZE - Number of pixels horizontal

[illegible][illegible]

10.7.9 VSTART - Start line vertical

[illegible][illegible]

10.7.10 VSIZE - Number of pixels vertical

[illegible][illegible]

10.7.11 PVSET2 - Preview Setup Register #2

PVSET2	0003:0794				offset: 0x14				default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	HINT	VINT	VNFILT	CCDMOD	RSV	GBLND[2]	GBLND[1]	GBLND[0]	EPEL[1]	EPEL[0]	OPEL[1]	OPEL[0]	EPOL[1]	EPOL[0]	OPOL[1]	OPOL[0]			
R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	-	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	HI NT	0	R/W	<u>Horizontal Interpolation</u> » 0: Hue Interpolation (5 taps) « » 1: Simple Interpolation (2 taps) «
14	VI NT	0	R/W	<u>Vertical Interpolation</u> » 0: Hue Interpolation « » 1: Simple Interpolation «
13	VNFILT	0	R/W	<u>Vertical Noise Filter</u> » 0: disable « » 1: enable «
12	CCDMOD	0	R/W	<u>CCD Sensor Mode</u> » 0: RGB color space « » 1: Complementary color space «
11	RSV			Reserved
10: 8	GBLND	0	R/W	<u>Green Blend</u> In complementary CCD mode, adjust blending ratio between original green and a calculated green from the other colors. Goutput = rGorg + (1-r)(Ye+Cy-Mg) » 000: r=1 « » 001: r=3/4 « » 010: r=1/2 « » 011: r=1/4 « » 100: r=0 «
7: 6	EPEL	0	R/W	<u>Color Pattern for Even Pixel, Even Line</u> » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
5: 4	OPEL	0	R/W	<u>Color Pattern for Odd Pixel, Even Line</u> » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
3: 2	EPOL	0	R/W	<u>Color Pattern for Even Pixel, Odd Line</u> » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
1: 0	OPOL	0	R/W	<u>Color Pattern for Odd Pixel, Odd Line</u> » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «

10.7.12 NFILT - Noise Filter Definition Register

NFILT	0003:0796				offset: 0x16				default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	NFEN	RSV	RSV	NFRT[1]	NFRT[0]			
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	0			

Bit	Name	Reset Value	R/W	Function
15: 5	RSV			Reserved
4	NFEN	0	R/W	<u>Noise Filter Enable</u> » 0: disable « » 1: enable «
3: 2	RSV			Reserved
1: 0	NFRT	0	R/W	<u>Strength of Noise Filter</u> » 00: weak « » 01: a little weak « » 10: a little strong « » 11: strong «

10.7.13 DGAIN - Digital Gain Register

DGAIN	0003:0798				offset: 0x18				default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	DGAIN[9]	DGAIN[8]	DGAIN[7]	DGAIN[6]	DGAIN[5]	DGAIN[4]	DGAIN[3]	DGAIN[2]	DGAIN[1]	DGAIN[0]			
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 10	RSV			Reserved
9: 0	DGAIN	0	R/W	Digital Gain value Precision is U10Q8, i.e., DGAIN/(2^8)

10.7.14 WBGAIN0 - White Balance Gain register #0

[illegible][illegible]

10.7.15 WBGAIN1 - White Balance Gain register #1

[illegible][illegible]

10.7.16 SMTH - Smoothing Definition register

[illegible][illegible]

10.7.17 HRSZ - Horizontal Resize Register

[illegible][illegible]

10.7.18 VRSZ - Vertical Resize Register

[illegible][illegible]

10.7.19 BLOFST0 - Black Level Offset Adjust #0

[illegible][illegible]

10.7.20 BLOFST1 - Black Level Offset Adjust #1

[illegible][illegible]

10.7.21 MTXGAIN0 - Matrix Gain register #0

[illegible][illegible]

10.7.22 MTXGAIN1 - Matrix Gain register #1

[illegible][illegible]

10.7.23 MTXGAIN2 - Matrix Gain register #2

[illegible][illegible]

10.7.24 MTXGAIN3 - Matrix Gain register #3

[illegible][illegible]

10.7.25 MTXGAIN4 - Matrix Gain register #4

MTXGIN4		0003.07B0				offset:		0x30								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	MTXB[7]	MTXB[6]	MTXB[5]	MTXB[4]	MTXB[3]	MTXB[2]	MTXB[1]	MTXB[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-			
Default	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-			

[illegible]

10.7.26 MTXOFST0 - R_offset register

[illegible][illegible]

10.7.27 MTXOFST1 - G offset register

[illegible][illegible]

10.7.28 MTXOFST2 - B_offset register

[illegible][illegible]

10.7.29 GAMTSBYP - GAMMA table bypass register

GAMTSBYP													0003:07B8				offset:		0x38								default:		0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TBSEL	BYPR	BYPG	BYPB																
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W																
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0																

[illegible]

10.7.30 CSC0 - Color Space Conversion Coefficient register #0

[illegible][illegible]

10.7.31 CSC1 - Color Space Conversion Coefficient register #1

[illegible][illegible]

10.7.32 CSC2 - Color Space Conversion Coefficient register #2

[illegible][illegible]

10.7.33 CSC3 - Color Space Conversion Coefficient register #3

[illegible][illegible]

10.7.34 CSC4 - Color Space Conversion Coefficient register #4

CS4								0003.07C2								offset: 0x42		default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	SCBCR[7]	SCBCR[6]	SCBCR[5]	SCBCR[4]	SCBCR[3]	SCBCR[2]	SCBCR[1]	SCBCR[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV									
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-									
Default	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-									

[illegible]

10.7.35 YOFST - Y offset register

[illegible][illegible]

10.7.36 COFST - Cb and Cr offset register

[illegible][illegible]

10.7.37 CNTBRT - Contrast and brightness adjustment register

[illegible][illegible]

10.7.38 CSUP0 - Chroma Suppression Register #0

[illegible][illegible]

10.7.39 CSUP1 - Chroma Suppression Register #1

CSUP1								0003:07CC								offset:		0x4C										default:		0x0000	
Bit	15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0														
Name	CSUPH[7]	CSUPH[6]	CSUPH[5]	CSUPH[4]	CSUPH[3]	CSUPH[2]	CSUPH[1]	CSUPH[0]		CSUGP[7]	CSUGP[6]	CSUGP[5]	CSUGP[4]	CSUGP[3]	CSUGP[2]	CSUGP[1]	CSUGP[0]														
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W														
Default	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0														

[illegible]

10.7.40 SETUPY - Y max and min setup register

[illegible][illegible]

10.7.41 SETUPC - Cb and Cr max and min setup register

[illegible][illegible]

11 Hardware 3A

DM270 provides real-time 3A (AE/AWB/AF) statistics function. ARM uses the statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated by using up to 192 windows. The windows are the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by using up to 36 windows. The position and the size of each window can be independently set by software.

11.1 Features

11.1.1 Auto Exposure, Auto White-Balance (AE, AWB)

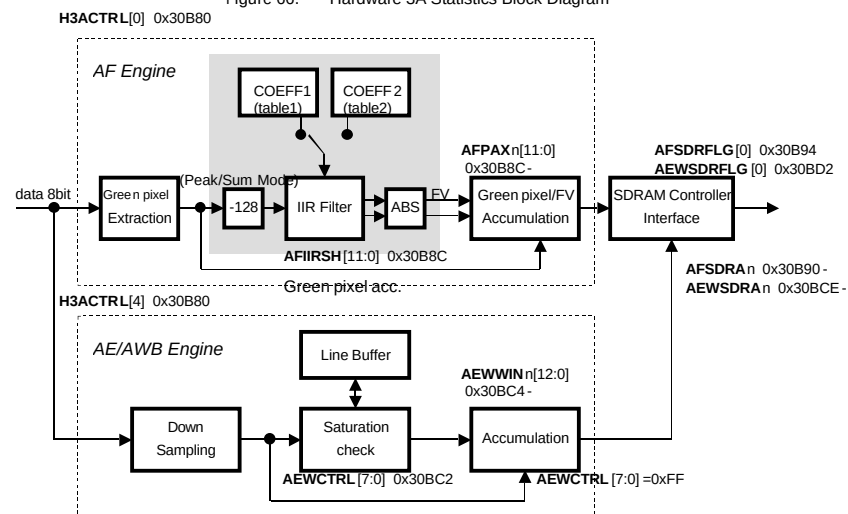
- Total number of windows is 192 and is organized as a m-by-n matrix.
- The base position for AE/AWB statistics window matrix can be configured by software
- Number of windows in horizontal/vertical direction is from 1 to 12(H), 1 to 8(V)
- Total number of pixels per window in horizontal/vertical direction is from 2 to 256(H), 4 to 64(V)
- Sampling step for calculation in horizontal/vertical direction is from 2 to 32. Total number of 2x2 blocks in a window is up to 256.
- Outputs for each window are the accumulated R, Gr, Gb and B. The results are 16-bit integers stored in SDRAM readable by ARM

11.1.2 Auto-Focus (AF)

- Total number of windows is up to 36 and is organized as a m-by-n matrix
- The AF calculation can be performed only on green pixel data in the window.
- The base position for AF statistics window matrix can be configured by software

- Two biquad filters are used to filter the green or Y pixel data. The coefficients of both filters are 10-bit wide and can be set by software.
- Number of windows in horizontal/vertical direction is from 1 to 4(H), 1 to 9(V)
- Total number of green pixels per window in horizontal/vertical direction is from 1 to 256(H), 2 to 64(V)

Figure 66. Hardware 3A Statistics Block Diagram



AF Engine, performs the following calculations:

- Green pixel accumulation for each Poxel.
- Focus Value (FV) accumulation for each Poxel.
[Peak mode]
Accumulate the maximum FV of each line in a Poxel.

[Sum mode]
Accumulate FV in a Poxel.

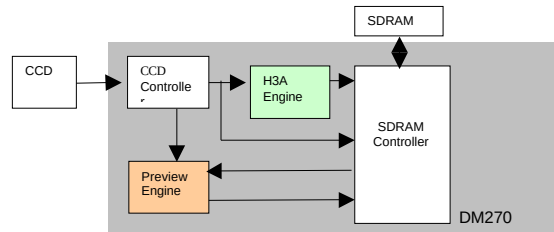
NOTE: Poxel in AF statistics is a window of green pixel data

AE/AWB Engine, performs the following calculations:

- o Accumulate the pixels in the sub-sampled 2x2 blocks for each Window (AVE1).
- o Accumulate the pixels which are clipped to the limit value in the sub-sampled 2x2 blocks (AVE2).
- o Count the number of the sub-sampled 2x2 blocks which are not saturating (CNT2).

Figure below shows the Image Peripheral Architecture containing Hardware 3A module

Figure 67. Image Peripheral Architecture



11.2 Input data for H3A Engine

The input signal to H3A engine is 8bit data selected with GWDI(2:0) of the GAMMA(0x3073C) register in CCDC. Note, GWDI(2:0) selects 10-bit raw data, the higher 8-bits are taken as input by the H3A engine (See also CCDC Block Diagram in chapter "CCD Controller").

11.3 Auto Focus (AF) Engine

NOTE: Poxel in AF statistics is a window of green pixel data

The input data to AF engine is the raw data output by the CCD Controller. The AF Engine extracts green pixel data from this raw data. The extracted green data are subtracted by 128, and fed to the filter block to calculate the Focus Value (FV), which is the absolute value of the IIR filter output. The maximum FV of each line in a Poxel is acquired if FV mode is set to 'Peak mode'. Value of the green pixels and either the FV or the maximum FV (of each line) are accumulated in the Poxel, and are stored in SDRAM.

The window configuration of the AF Engine is based on a frame formed by the extracted green pixels.

Figure 68. AF Engine Configuration Registers

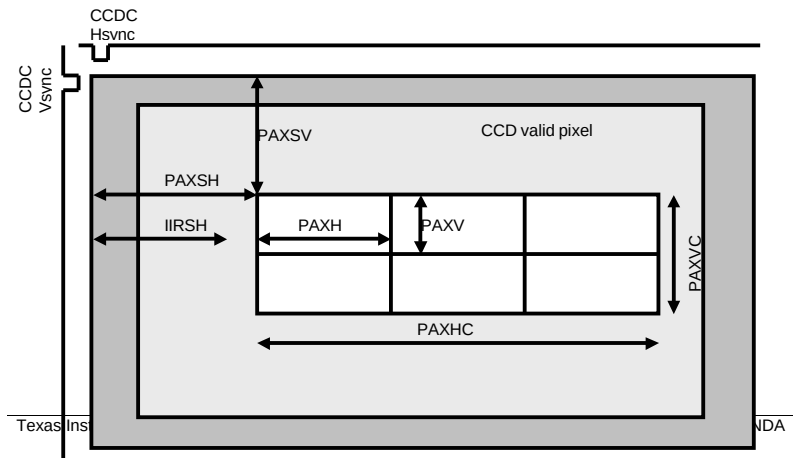
Register	Width	Description	Address
AFEN	1	AF Engine enable 0:disable 1:enable	H3ACTRL[0] 0x30B80
GPOS	4	Green pixel position in 2x2 block	AFCTRL[3:0] 0x30B82
PAXH	8	Poxel Width (PAXH+1) (1 – 256)	AFPAX1[7:0] 0x30B84
PAXV	6	Poxel Height (PAXV+1) (2 – 64)	AFPAX1[13:8] 0x30B84
PAXHC	4	Poxel Count for H direction (PAXHC+1) (1 – 9)	AFPAX2[3:0] 0x30B86
PAXVC	2	Poxel Count for V direction (PAXVC+1) (1 – 4)	AFPAX2[9:8] 0x30B86
PAXSH	12	Poxel start position (H) (0 – 4,095)	AFPAX3[11:0] 0x30B88
PAXSV	12	Poxel start position (V) (0 – 4,095)	AFPAX2[11:0] 0x30B8A
IIRSH	12	IIR filter start position (H) (0 – 4,095)	AFIIRSH[110] 0x30B8C
AFINCV	4	Line increments in a Poxel (AFINCV+1) (1 -16)	AFPAX5[11:8] 0x30B8E
FVMODE	1	Focus Value accumulation mode 0:Sum mode 1:Peak mode	AFCTRL[4] 0x30B82
COEFF1	11x11	Coefficient table1 for IIR filter (decimal 6bit)	AFCOEFF10[10:0] 0x30B96 -
COEFF2	11x11	Coefficient table2 for IIR filter (decimal 6bit)	AFCOEFF20[10:0] 0x30BAC -
AFSDRA	22	SDRAM destination address to store the results of AF engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.	AFSDRA1[5:0] 0x30B90 AFSDRA2[15:0] 0x30B92
AFSDRFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.	AFSDRFLG[0] 0x30B94

Note,

- o PAXSH must be equal to or greater than (IIRSH + 1).
- o Pixels cannot overlap the last pixel in a line. PAXSH, PAXW and PAXHC must be set accordingly.
- o Pixels can overlap either the first or the last line in a frame, but not both of them. PAXSV, PAXH and PAXVC must be set accordingly.

The figure below shows the arrangement of paxels within a frame of pixels data.

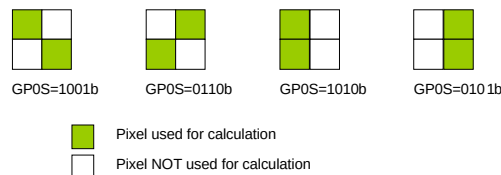
Figure 69. AF Poxel Configuration



11.3.1 Green pixel position (GPOS)

The input signal is 8bit data, and the green pixel data is within a 2x2 window is selected using GPOS(3:0). Selectable green patterns are shown in the figure below.

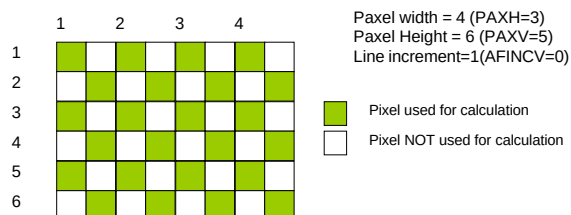
Figure 70. Sub-sampling Pattern



11.3.2 Poxel Setting

PAXH is used to set paxel width (horizontal), PAXV is used to set paxel height (vertical). Figure below shows an example of setting PAXH, PAXV. Note, paxel width is number of green pixels in horizontal direction. See also 11.3.4 Line increment

Figure 71. Poxel setting example

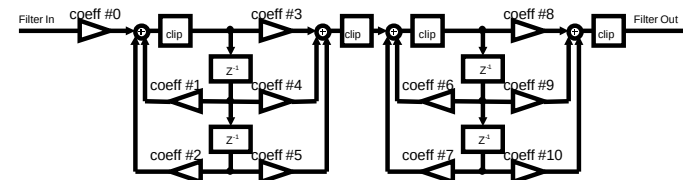


11.3.3 IIR Filter

There are two sets of programmable coefficient tables. FV calculation is performed using both tables. Shift registers of the IIR filter are cleared to zero once in a line, at the next position of IIRSH.

Figure 72. IIR Filter

	Precision	Range
Filter Input	[Value of a green pixel (8bit)] - 128	-128 to 127
Coefficient	Signed 11bit (decimal 6 bit)	-16 to 15+63/64
Filter Output	Signed 12bit (decimal 4 bit)	-128 to 127+15/16

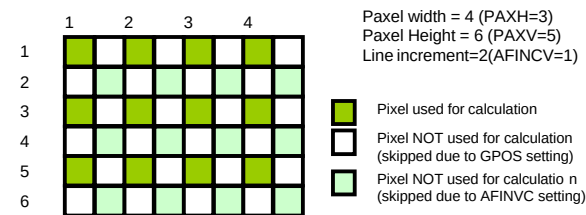


Coeff #n = AFCEFF n[10:0] 0x30B98 -

11.3.4 Line increment

Line increment within each paxel can be specified using AFINCV (0x30B8E). Figure below shows an example of line increment within a paxel. IIR calculation is performed for only the lines selected using line increment

Figure 73. Line increment setting Example



11.3.5 AF Engine Output

For each paxel, AF Engine outputs

- Green pixel accumulation for each Paxel. (22bit)
- Focus Value (FV) accumulation for each Paxel for IIR coefficient table 1, 2 (26bit, decimal 4bit)
[Peak mode]
Accumulate the maximum FV of each line in a Paxel.

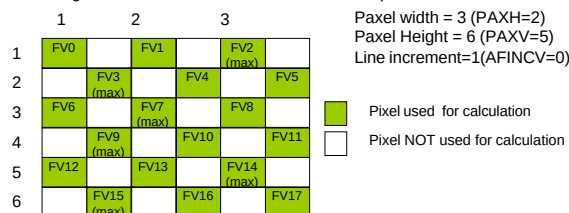
[Sum mode]
Accumulate FV in a Paxel.

Figure below shows an example of FV accumulation mode. Here FVn is FV value calculated by IIR filter using a coefficient table. FVn (max), is maximum FV value in a line within a paxel.

In sum mode, FV value output by AF engine for each paxel is $FV_{out} = FV_0 + FV_1 + FV_2 + \dots + FV_{17}$

In peak mode, FV value output by AF Engine for each paxel is sum of FVn max, i.e $FV_{out} = FV_2 + FV_3 + FV_7 + FV_9 + FV_{14} + FV_{15}$

Figure 74. FV Accumulation Mode Example



11.3.5.1 Data Transfer to SDRAM

Results of the two adjacent Paxels in a row are gathered and transferred to the SDRAM at the same time,

Therefore,
if PAXHC is set to 8, i.e number of paxels in horizontal direction is $8+1 = 9$ and PAXVC is set to 3, i.e number of paxels in vertical direction is $3+1 = 4$ there will be $\lceil (9+1)/2 \rceil \times 4 = 20$ sets of data transfer within an image.

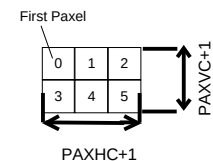
The results of the first paxel in an image are stored to the address specified in the register AFSDRA. The address is automatically incremented until two continuous buffers for two successive frames become full. SDRAM buffer flag (AFSDRFLG) indicates the buffer at which the results are stored.

NOTE: SDRAM Address is specified as offset from start of SDRAM in units of 32bytes

Example,

- (PAXHC, PAXVC) = (2,1)

Figure 75. Exampe Paxel Configuration



In Figure 76, the accumulation of green pixel values in the paxel is indicated as Green sum and the IIR filter output FV accumulated values of using coefficient table 1 and 2 are indicated as FV sum-1, FV sum-2. In addition, as AF result is output in sets of two paxels, when the number of horizontal paxel shown in Figure 75 is odd number, dummy data is output for a paxel. The part in gray in Figure 76 shows the dummy data output.

Figure 76. AF SDRAM Output Format

SDRAM address (byte address)	31	16	15	0	
AFSDRA	Green sum			(Paxel 0)	1 st transfer start
	FV sum -1			(Paxel 0)	
	FV sum -2			(Paxel 0)	
	All zero				
	Green sum			(Paxel 1)	
	FV sum -1			(Paxel 1)	
	FV sum -2			(Paxel 1)	
	All zero				1 st transfer end
AFSDRA + 32	Green sum			(Paxel 2)	2 nd transfer start
	FV sum -1			(Paxel 2)	
	FV sum -2			(Paxel 2)	
	All zero				
	Green sum			(Paxel 1)	
	FV sum -1			(Paxel 1)	
	FV sum -2			(Paxel 1)	
	All zero				2 nd transfer end
AFSDRA + 64	Green sum			(Paxel 3)	3 rd transfer start
	FV sum -1			(Paxel 3)	
	FV sum -2			(Paxel 3)	
	All zero				
	Green sum			(Paxel 4)	
	FV sum -1			(Paxel 4)	
	FV sum -2			(Paxel 4)	
	All zero				3 rd transfer end
AFSDRA + 96	Green sum			(Paxel 5)	4 th transfer start

FV sum -1	(Paxel 5)
FV sum -2	(Paxel 5)
Green sum	(Paxel 4)
FV sum -1	(Paxel 4)
FV sum -2	(Paxel 4)
All zero	4 th transfer end

11.3.5.1.1 Calculation of current SDRAM result buffer

Code below shows how to calculate the SDRAM buffer where the AF results are stored for the previous frame.

```

uint32 AF_getCurFrameDataAddress() {
    uint32 dataAddr;

    dataAddr = H3A_FGET(AFSDRA1, ADRH);
    dataAddr = dataAddr << 16;
    dataAddr |= H3A_RGET(AFSDRA2);
    dataAddr *= 32;
    dataAddr += (uint32)SDRAM_MEMORY_BASE;

    if(H3A_FGET(AFSDRFLG, AFSRFLG)) {
        if((H3A_FGET(AFPAX2, PAXHC) + 1) & 0x1) {
            dataAddr +=
                (H3A_FGET(AFPAX2, PAXHC) + 1 + 1) *
                H3A_FGET(AFPAX2, PAXVC) + 1) * 16;
        } else {
            dataAddr +=
                (H3A_FGET(AFPAX2, PAXHC) + 1) *
                (H3A_FGET(AFPAX2, PAXVC) + 1) * 16;
        }
    }
    return dataAddr;
}

```

11.4 Auto Exposure, Auto White -Balance Engine (AEWB)

The input data to AEWB engine is the raw data output by the CCD Controller. All color component pixels (R, Gr, Gb, B) are input to the AEWB engine. The pixels in the sub-sampled 2x2 blocks are accumulated in the window as AVE1. If any of the pixels in a 2x2 block are equal to or greater than the limit value (AVE2LMT), the block is treated as a saturated block. Only the blocks which are not saturating are counted into CNT2. The pixels which are clipped to AVE2LMT in the sub-sampled 2x2 blocks are accumulated in the window as AVE2, and are stored in SDRAM with AVE1.

Figure 77. AEWB Configuration Registers

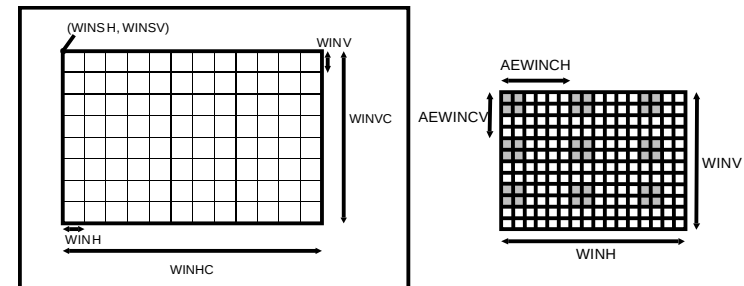
Register	width	Description	Address
AEWEN	1	AE/AWB Engine enable 0:disable 1:enable	H3ACTRL[4] 0x30B80
WINH	7	Window Width (even number) ((WINH +1) x2) (2 – 256)	AEWWIN1[6:0] 0x30BC4
WINV	5	Window Height (even number) ((WIN V +1) x2) (2 – 64)	AEWWIN1[12:8] 0x30BC4
WINHC	4	Window Count for H direction (WINWHC+1) (1 – 12)	AEWWIN2[3:0] 0x30BC6
WINVC	3	Window Count for V direction (WINWVC+1) (1 – 8)	AEWWIN2[10:8] 0x30BC6
WINSH	11	X position of the Window start point (WINSH x2)	AEWWIN3[10:0] 0x30BC8

Register	width	Description	Address
WINSV	11	(even number) Y position of the Window start point (WINSV x2) (even number)	AEWWIN4[10:0] 0x30BCA
AEWINCH	4	Sampling point increments (X) in a Window (even number) ((AEWINCH+1) x2) (2 -32)	AEWWIN5[3:0] 0x30BCC
AEWINCV	4	Sampling point increments (Y) in a Window (even number) ((AEWINCV+1) x2) (2 -32)	AEWWIN5[11:8] 0x30BCC
AVE2LMT	8	AVE2 limit value	AEWCTRL[7:0] 0x30BC2
AEWSDRA	22	SDRAM destination address to store the results of AE/AWB engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.	AEWSDRA1[5:0] 0x30BCE AEWSDRA2[15:0] 0x30BCE
AEWSDRFLG	1	SDRAM buffer flag to indicate the buffer at which the results are stored.	AEWSDRFLG[0] 0x30BD2

Note,

- The number of 2x2 blocks in a Window cannot exceed 256. WINH, WINV, AEWINCH and AEWINCV must be set accordingly.
- Windows can overlap either the first or the last line in a frame, but not both of them. WINSV, WINV and WINVC must be set accordingly.
- WINSH, WINSV are set with respect to CCD HD, VD signal respectively

Figure 78. AEWB Window Configuration



11.4.1 AE/AWB Engine Output

AE/AWB Engine outputs the following results,

- AVE1[i][j] – 16bit (Accumulated sub-sampled data clipped to 65,535)
- AVE2[i][j] – 16bit (Accumulated sub-sampled data which are not saturated clipped to 65,535)
- CNT2[i] – 16bit(9bit) (Number of sub-sampled data which are not saturated)

i : window number

j : pixel position in 2x2 blocks (0: upper left, 1: upper right, 2: lower left, 3: lower right)

11.4.1.1 Data transfer to SDRAM

Results of each window are transferred to the SDRAM at the same time,

Therefore,
if WINHC is set to 8, i.e number of windows in horizontal direction is $8+1 = 9$ and
WINVC is set to 3, i.e number of windows in vertical direction is $3+1 = 4$ there
will be $9 \times 4 = 36$ sets of data transfer within an image.

The results of the first Window in an image are stored to the address specified in the register AEWSDBA. The address is automatically incremented until two continuous buffers for two successive frames become full. SDRAM buffer flag (AEWSDBFLG) indicates the buffer at which the results are stored.

NOTE: SDRAM Address is specified as offset from start of SDRAM in units of 32bytes

Example,

- (WINHC, WINVC) = (1,1)

Figure 79. Example Window Configuration

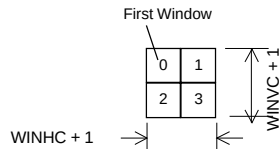


Figure 80. AEWB SDRAM Data Format

SDRAM address (byte address)	31	16	15	0	
AEWSDBA	AVE1[0][1]	AVE1[0][0]			1 st transfer start
	AVE1[0][3]	AVE1[0][2]			
	AVE2[0][1]	AVE2[0][0]			
	AVE2[0][3]	AVE2[0][2]			
	All zero	CNT2[0]			
	All zero				
	All zero				
	All zero				1 st transfer end
AEWSDBA + 32	AVE1[1][1]	AVE1[1][0]			2 nd transfer start
	AVE1[1][3]	AVE1[1][2]			
	AVE2[1][1]	AVE2[1][0]			
	AVE2[1][3]	AVE2[1][2]			
	All zero	CNT2[1]			
	All zero				
	All zero				
	All zero				2 nd transfer end
AEWSDBA + 64	AVE1[2][1]	AVE1[2][0]			3 rd transfer start
	AVE1[2][3]	AVE1[2][2]			
	AVE2[2][1]	AVE2[2][0]			

AEWSDBA + 96

AVE2[2][3]	AVE2[2][2]	
All zero	CNT2[2]	
All zero		
All zero		
All zero		3 rd transfer end
AVE1[3][1]	AVE1[3][0]	4 th transfer start
AVE1[3][3]	AVE1[3][2]	
AVE2[3][1]	AVE2[3][0]	
AVE2[3][3]	AVE2[3][2]	
All zero	CNT2[3]	
All zero		
All zero		
All zero		4 th transfer end

11.4.1.1.1 Calculation of current SDRAM result buffer

Code below shows how to calculate the SDRAM buffer for AEWB results of the previous frame.

```

uint32 AEWB_getCurFrameDataAddress () {
    uint32 dataAddr;

    dataAddr = H3A_FGET(AEWSDBA1, ADRH);
    dataAddr = dataAddr << 16;
    dataAddr |= H3A_RGET(AEWSDBA2);
    dataAddr *= 32;
    dataAddr += (uint32)SDRAM_MEMORY_BASE;

    if (H3A_FGET(AEWSDBFLG, AEWSDBFLG)) {
        dataAddr +=
            (H3A_FGET(AEWIN2, WINHC) + 1) *
            (H3A_FGET(AEWIN2, WINVC) + 1) * 32;
    }
    return dataAddr;
}

```

11.5 Hardware 3A Register Update timing

All registers in AF/AE/AWB Statistics Engine are latched by CCDC VD signal.

11.7.2 AFCTRL - AF Engine Control Register

AFCTRL		0003:0B82				offset: 0x02														default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	FVMODE	GPOSUL	GPOSUR	GPOSLL	GPOS LR					
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	-	-	-					

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	FVMODE	0	R/W	[AF Engine] Focus Value(FV) accumulation mode. » 0: Sum mode « » 1: Peak mode «
3	GPOSUL	0	R/W	[AF Engine] Green pixel extraction. upper left in 2x2 block (even pixel, even line). » 0: other than Green « » 1: Green «
2	GPOSUR			[AF Engine] Green pixel extraction. upper right in 2x2 block (odd pixel, even line). » 0: other than Green « » 1: Green «
1	GPOSLL			[AF Engine] Green pixel extraction. lower left in 2x2 block (even pixel, odd line). » 0: other than Green « » 1: Green «
0	GPOS LR			[AF Engine] Green pixel extraction. lower right in 2x2 block (odd pixel, odd line). » 0: other than Green « » 1: Green «

11.7.3 AFPAX1 - AF Engine Poxel Register #1

AFPAX1		0003:0B84				offset: 0x04												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PAXV[5]	PAXV[4]	PAXV[3]	PAXV[2]	PAXV[1]	PAXV[0]	PAXH[7]	PAXH[6]	PAXH[5]	PAXH[4]	PAXH[3]	PAXH[2]	PAXH[1]	PAXH[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PAXV	0	R/W	[AF Engine] Poxel Height (number of green lines). Poxel Height is (PAXV +1) and ranges: 2-64. A value greater than 0 must be set to PAXV.
7:0	PAXH	0	R/W	[AF Engine] Poxel Width (number of green pixels). Poxel Width is (PAXH +1) and ranges: 1-256. Setting a small value to PAXH might collapse the data transfer to SDRAM. SDRAM access time can not be longer than the time it takes for the Poxel Width (PAXH+1). SDRAM access time varies depending on CCD and SDRAM clock frequency settings, and also the status of the other peripherals' access to SDRAM.

11.7.4 AFPAX2 - AF Engine Paxel Register #2

AFFPAX2		0003:0B86				offset:		0x06								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	PAXVC[1]	PAXVC[0]	RSV	RSV	RSV	RSV	PAXHC[3]	PAXHC[2]	PAXHC[1]	PAXHC[0]			
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	0	0	-	-	-	-	0	0	0	0			

[illegible]

11.7.5 AFPAX3 - AF Engine Paxel Register #3

AFPAx3		0003.0B88										offset:		0x08												default:		0x0000																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					

[illegible]

11.7.6 AFPAX4 - AF Engine Paxel Register #4

[illegible][illegible]

11.7.7 AFIRSH - AF Engine IIR Shift register

[illegible][illegible]

11.7.8 AFPAX5 - AF Engine Paxel Register #5

AFPAx5				0003:0B8E				offset:		0x0E								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4		3	2	1	0				
Name	RSV	RSV	RSV	RSV	AFINCV[3]	AFINCV[2]	AFINCV[1]	AFINCV[0]	RSV	RSV	RSV	RSV		RSV	RSV	RSV	RSV				
Default	-	-	-	-	R/W	R/W	R/W	R/W	-	-	-	-		-	-	-	-				

[illegible]

11.7.9 AFSDRA1 - AF Engine SDRAM Address (high)

AFSDRA1				0003:0B90				offset:		0x10		default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]			
Raw	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:6	RSV			Reserved
5:0	ADRH	0	R/W	[AF Engine] Upper 6 bits of SDRAM destination address to store the results of AF engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on PAXHC and PAXVC. AFSDRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.10 AFSDRA2 - AF Engine SDRAM Address (low)

AFSDRA2		0003:0B92								offset:		0x12										default:		0x000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]									
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W									
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0									

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	[AF Engine] Lower 16 bits of SDRAM destination address to store the results of AF engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on PAXHC and PAXVC. AFSDBA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.11 AFSDRFLG - AF Engine SDRAM Buffer Flag

[illegible]

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	AFSDRFLG	0	R	[AF Engine] SDRAM buffer flag to indicate the buffer at which the results of the last frame are stored. » 0: Results stored in first buffer from AFSDRA « » 1: Results stored in second buffer «

11.7.12 AFCOEFF10 - AF Engine IIR Filter Table #1, Coefficient #0

AFCOEFF10					0003:0B96		offset: 0x16										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF10[10]	COEFF10[9]	COEFF10[8]	COEFF10[7]	COEFF10[6]	COEFF10[5]	COEFF10[4]	COEFF10[3]	COEFF10[2]	COEFF10[1]	COEFF10[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF10	0	R/W	[AF Engine] IIR filter Coefficient #0 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.13 AFCOEFF11 - AF Engine IIR Filter Table #1, Coefficient #1

AFCOEFF11 0003:0B98 offset: 0x18 default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF11[10]	COEFF11[9]	COEFF11[8]	COEFF11[7]	COEFF11[6]	COEFF11[5]	COEFF11[4]	COEFF11[3]	COEFF11[2]	COEFF11[1]	COEFF11[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF11	0	R/W	[AF Engine] IIR filter Coefficient #1 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.14 AFCOEFF12 - AF Engine IIR Filter Table #1, Coefficient #2

AFCOEFF12 0003:0B9A offset: 0x1A default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF12[10]	COEFF12[9]	COEFF12[8]	COEFF12[7]	COEFF12[6]	COEFF12[5]	COEFF12[4]	COEFF12[3]	COEFF12[2]	COEFF12[1]	COEFF12[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF12	0	R/W	[AF Engine] IIR filter Coefficient #2 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.15 AFCOEFF13 - AF Engine IIR Filter Table #1, Coefficient #3

AFCOEFF13 0003:0B9C offset: 0x1C default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF13[10]	COEFF13[9]	COEFF13[8]	COEFF13[7]	COEFF13[6]	COEFF13[5]	COEFF13[4]	COEFF13[3]	COEFF13[2]	COEFF13[1]	COEFF13[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF13	0	R/W	[AF Engine] IIR filter Coefficient #3 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.16 AFCOEFF14 - AF Engine IIR Filter Table #1, Coefficient #4

AFCOEFF14 0003:0B9E offset: 0x1E default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF14[10]	COEFF14[9]	COEFF14[8]	COEFF14[7]	COEFF14[6]	COEFF14[5]	COEFF14[4]	COEFF14[3]	COEFF14[2]	COEFF14[1]	COEFF14[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF14	0	R/W	[AF Engine] IIR filter Coefficient #4 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.17 AFCOEFF15 - AF Engine IIR Filter Table #1, Coefficient #5

AFCOEFF15 0003:0BA0 offset: 0x20 default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF15[10]	COEFF15[9]	COEFF15[8]	COEFF15[7]	COEFF15[6]	COEFF15[5]	COEFF15[4]	COEFF15[3]	COEFF15[2]	COEFF15[1]	COEFF15[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF15	0	R/W	[AF Engine] IIR filter Coefficient #5 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.18 AFCOEFF16 - AF Engine IIR Filter Table #1, Coefficient #6

AFCOEFF16 0003:0BA2 offset: 0x22 default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF16[10]	COEFF16[9]	COEFF16[8]	COEFF16[7]	COEFF16[6]	COEFF16[5]	COEFF16[4]	COEFF16[3]	COEFF16[2]	COEFF16[1]	COEFF16[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF16	0	R/W	[AF Engine] IIR filter Coefficient #6 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.19 AFCOEFF17 - AF Engine IIR Filter Table #1, Coefficient #7

AFCOEFF17 0003:0BA4 offset: 0x24 default: 0x0000																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF17[10]	COEFF17[9]	COEFF17[8]	COEFF17[7]	COEFF17[6]	COEFF17[5]	COEFF17[4]	COEFF17[3]	COEFF17[2]	COEFF17[1]	COEFF17[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF17	0	R/W	[AF Engine] IIR filter Coefficient #7 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.20 AFCEFF18 - AF Engine IIR Filter Table #1, Coefficient #8

AFCEFF18		0003:0BA6				offset: 0x26		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF18[1]	COEFF18[0]	COEFF18[7]	COEFF18[6]	COEFF18[5]	COEFF18[4]	COEFF18[3]	COEFF18[2]	COEFF18[1]	COEFF18[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF18	0	R/W	[AF Engine] IIR filter Coefficient #8 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.21 AFCEFF19 - AF Engine IIR Filter Table #1, Coefficient #9

AFCEFF19		0003:0BA8				offset: 0x28		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF19[1]	COEFF19[0]	COEFF19[7]	COEFF19[6]	COEFF19[5]	COEFF19[4]	COEFF19[3]	COEFF19[2]	COEFF19[1]	COEFF19[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF19	0	R/W	[AF Engine] IIR filter Coefficient #9 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.22 AFCEFF110 - AF Engine IIR Filter Table #1, Coefficient #10

AFCEFF110		0003:0BAA				offset: 0x2A		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF110[1]	COEFF110[0]	COEFF110[7]	COEFF110[6]	COEFF110[5]	COEFF110[4]	COEFF110[3]	COEFF110[2]	COEFF110[1]	COEFF110[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF110	0	R/W	[AF Engine] IIR filter Coefficient #10 <Table1>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.23 AFCEFF20 - AF Engine IIR Filter Table #2, Coefficient #0

AFCEFF20		0003:0BAC				offset: 0x2C		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF20[1]	COEFF20[0]	COEFF20[7]	COEFF20[6]	COEFF20[5]	COEFF20[4]	COEFF20[3]	COEFF20[2]	COEFF20[1]	COEFF20[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF20	0	R/W	[AF Engine] IIR filter Coefficient #0 <Table2>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.24 AFCEFF21 - AF Engine IIR Filter Table #2, Coefficient #1

AFCEFF21		0003:0BAE				offset: 0x2E		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF21[1]	COEFF21[0]	COEFF21[7]	COEFF21[6]	COEFF21[5]	COEFF21[4]	COEFF21[3]	COEFF21[2]	COEFF21[1]	COEFF21[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF21	0	R/W	[AF Engine] IIR filter Coefficient #1 <Table2>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.25 AFCEFF22 - AF Engine IIR Filter Table #2, Coefficient #2

AFCEFF22		0003:0BB0				offset: 0x30		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF22[1]	COEFF22[0]	COEFF22[7]	COEFF22[6]	COEFF22[5]	COEFF22[4]	COEFF22[3]	COEFF22[2]	COEFF22[1]	COEFF22[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF22	0	R/W	[AF Engine] IIR filter Coefficient #2 <Table2>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.26 AFCEFF23 - AF Engine IIR Filter Table #2, Coefficient #3

AFCEFF23		0003:0BB2				offset: 0x32		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF23[1]	COEFF23[0]	COEFF23[7]	COEFF23[6]	COEFF23[5]	COEFF23[4]	COEFF23[3]	COEFF23[2]	COEFF23[1]	COEFF23[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF23	0	R/W	[AF Engine] IIR filter Coefficient #3 <Table2>. Precision S11Q6: -160 0oeff0 5+63/64

11.7.27 AFCEFF24 - AF Engine IIR Filter Table #2, Coefficient #4

AFCEFF24					0003:0BB4		offset: 0x34										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF24[10]	COEFF24[9]	COEFF24[8]	COEFF24[7]	COEFF24[6]	COEFF24[5]	COEFF24[4]	COEFF24[3]	COEFF24[2]	COEFF24[1]	COEFF24[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF24	0	R/W	[AF Engine] IIR filter Coefficient #4 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.28 AFCEFF25 - AF Engine IIR Filter Table #2, Coefficient #5

AFCEFF25					0003:0BB6					offset: 0x36										default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RSV	RSV	RSV	RSV	RSV	COEFF25[1]	COEFF25[0]	COEFF25[8]	COEFF25[7]	COEFF25[6]	COEFF25[5]	COEFF25[4]	COEFF25[3]	COEFF25[2]	COEFF25[1]	COEFF25[0]								
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W								
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0								

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF25	0	R/W	[AF Engine] IIR filter Coefficient #5 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.29 AFCEFF26 - AF Engine IIR Filter Table #2, Coefficient #6

AFCEFF26					0003:0BB8					offset: 0x38					default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	COEFF26[1]	COEFF26[0]	COEFF26[8]	COEFF26[7]	COEFF26[6]	COEFF26[5]	COEFF26[4]	COEFF26[3]	COEFF26[2]	COEFF26[1]	COEFF26[0]			
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF26	0	R/W	[AF Engine] IIR filter Coefficient #6 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.30 AFCEFF27 - AF Engine IIR Filter Table #2, Coefficient #7

AFCEFF27					0003:0BBA					offset: 0x3A										default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RSV	RSV	RSV	RSV	RSV	COEFF27[10]	COEFF27[9]	COEFF27[8]	COEFF27[7]	COEFF27[6]	COEFF27[5]	COEFF27[4]	COEFF27[3]	COEFF27[2]	COEFF27[1]	COEFF27[0]								
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W								
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0								

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF27	0	R/W	[AF Engine] IIR filter Coefficient #7 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.31 AFCEFF28 - AF Engine IIR Filter Table #2, Coefficient #8

AFCEFF28																0003:0BBC		offset: 0x3C																		default: 0x0000																												
Bit	15				14				13				12				11				10				9				8				7				6				5				4				3				2				1				0			
Name	RSV				RSV				RSV				RSV				RSV				COEFF28[1]				COEFF28[0]				COEFF28[8]				COEFF28[7]				COEFF28[6]				COEFF28[5]				COEFF28[4]				COEFF28[3]				COEFF28[2]				COEFF28[1]				COEFF28[0]			
R/W	-				-				-				-				-				R/W				R/W				R/W				R/W				R/W				R/W				R/W				R/W				R/W				R/W							
Default	-				-				-				-				-				0				0				0				0				0				0				0				0				0				0				0			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF28	0	R/W	[AF Engine] IIR filter Coefficient #8 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.32 AFCEFF29 - AF Engine IIR Filter Table #2, Coefficient #9

AFCEFF29															0003:0BBE		offset: 0x3E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0														
Name	RSV	RSV	RSV	RSV	RSV	COEFF29[1]	COEFF29[0]	COEFF29[8]	COEFF29[7]	COEFF29[6]	COEFF29[5]	COEFF29[4]	COEFF29[3]	COEFF29[2]	COEFF29[1]	COEFF29[0]														
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W														
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0														

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF29	0	R/W	[AF Engine] IIR filter Coefficient #9 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.33 AFCEFF210 - AF Engine IIR Filter Table #2, Coefficient #10

AFCEFF210																0003:0BC0			offset: 0x40																			default: 0x0000										
Bit	15			14			13			12			11			10			9			8			7			6			5			4			3			2			1			0		
Name	RSV			RSV			RSV			RSV			RSV			COEFF210[1]			COEFF210[0]			COEFF210[8]			COEFF210[7]			COEFF210[6]			COEFF210[5]			COEFF210[4]			COEFF210[3]			COEFF210[2]			COEFF210[1]			COEFF210[0]		
R/W	-			-			-			-			-			R/W			R/W			R/W			R/W			R/W			R/W			R/W			R/W			R/W			R/W					
Default	-			-			-			-			-			0			0			0			0			0			0			0			0			0			0			0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF210	0	R/W	[AF Engine] IIR filter Coefficient #10 <Table2>. Precision S11Q6: -16 ₀ 0oeff ₀ 5+63/64

11.7.34 AEWCTRL - AE/AWB Engine Control Register

AEWCTRL		0003/0BC2								offset:		0x42								default:		0x0000											
Bit		15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0	
Name		RSV		RSV		RSV		RSV		RSV		RSV		RSV		RSV		R/VELMT[7]		R/VELMT[6]		R/VELMT[5]		R/VELMT[4]		R/VELMT[3]		R/VELMT[2]		R/VELMT[1]		R/VELMT[0]	
R/W		-		-		-		-		-		-		-		-		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W	
Default		-		-		-		-		-		-		-		-		0		0		0		0		0		0		0		0	

[illegible]

11.7.35 AEWWIN1 - AE/AWB Engine Window Register #1

AEWIN1										0003:0BC4										offset: 0x44										default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																							
Name	RSV	RSV	RSV	WINV[4]	WINV[3]	WINV[2]	WINV[1]	WINV[0]	RSV	WINH[6]	WINH[5]	WINH[4]	WINH[3]	WINH[2]	WINH[1]	WINH[0]																							
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W																							
Default	-	-	-	0	0	0	0	0	-	0	0	0	0	0	0	0																							

[illegible]

11.7.36 AEWWIN2 - AE/AWB Engine Window Register #2

AEWWIN2		0003.0BC6				offset: 0x46								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	WINVC[2]	WINVC[1]	WINVC[0]	RSV	RSV	RSV	RSV	WINHC[3]	WINHC[2]	WINHC[1]	WINHC[0]	
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	-	-	-	-	0	0	0	0	

[illegible]

11.7.37 AEWWIN3 - AE/AWB Engine Window Register #3

[illegible][illegible]

11.7.38 AEWWIN4 - AE/AWB Engine Window Register #4

[illegible][illegible]

11.7.39 AEWWIN5 - AE/AWB Engine Window Register #5

AEWIN5																0003:0BCC				offset:		0x4C				default: 0x000																																						
Bit	15				14				13				12				11				10				9				8				7				6				5				4				3				2				1				0			
Name	RSV				RSV				RSV				RSV				EWINC[V]				EWINC[V]				EWINC[V]				EWINC[V]				RSV				RSV				RSV				RSV				EWINC[V]				EWINC[V]				EWINC[V]				EWINC[V]			
R/W	-				-				-				-				R/W				R/W				R/W				R/W				-				-				-				-				R/W				R/W				R/W				R/W			
Default	-				-				-				-				0				0				0				0				-				-				-				-				0				0				0				0			

[illegible]

11.7.40 AEWSRA1 - AE/AWB Engine SDRAM Address (high)

AEWSDR1		00030BCE				offset:		0x4E								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]			
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 6	RSV			Reserved
5: 0	ADRH	0	R/W	[AE/AWB Engine] Upper 6 bits of SDRAM destination address to store the results of AE/AWB Engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on WINHC and WINVC. AEWSRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.41 AEWSRA2 - AE/AWB Engine SDRAM Address (low)

[illegible]

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	[AE/AWB Engine] Lower 16 bits of SDRAM destination address to store the results of AE/AWB Engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on WINHC and WINVC. AEWSRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.42 AEWSRFLG - AE/AWB Engine SDRAM Buffer Flag

[illegible]

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	AEWSDRFLG	0	R	[AE/AWB Engine] SDRAM buffer flag to indicate the buffer at which the results of the last frame are stored. » 0: Results stored in first buffer from AEWSDBA « » 1: Results stored in second buffer «

11.7.43 TEST3A - H3A Test Register

TEST3A		0003:0BF6										offset: 0x7E						default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	3ATST1	RSV	RSV	RSV	3ATST0			
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	-	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	0			

[illegible]

12 OSD - On-Screen Display

12.1 Introduction

The OSD graphic accelerator is responsible for managing OSD data from different OSD windows and blending it with the video. It reads OSD data from SDRAM, and outputs it to the on-chip Video Encoder. The OSD module defaults to standby mode, in which it simply sends video to the Video Encoder. After being configured and activated by the ARM CPU, the OSD reads OSD data and mixes it with the video output. The ARM is responsible for configuration and control of OSD operations and writing of the OSD data to the SDRAM.

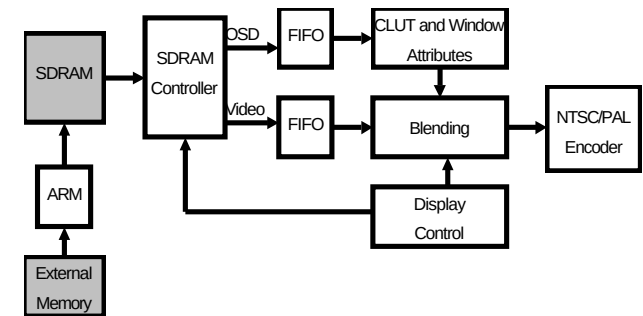
12.2 Features

- Can be configured for two video display windows and two OSD bitmap display windows, OR can be configured for two video display windows, one OSD bitmap display window, and one attribute window
- Configurable rectangular hardware cursor can be displayed, its size can be specified, and its color can be specified by a look-up table.
- Attribute window can provide pixel level blinking and blending
- The four windows and the one cursor can be displayed simultaneously
- The background color can be specified
- Video-Window0 and Video-Window1 use YCbCr data format, and OSD-Window0, OSD-Window1 use bitmap index data format
- Supports 1,2,4,8 bit wide bitmap data
- Eight blending ratios for the OSD-window can be selected
- Transparency for the OSD-windows
- Default bitmap look-up table in ROM available
- Programmable bit map look-up table in RAM available (256 colors, 24-bit per color)
- Interlaced and non-interlaced modes supported
- Zoom horizontal and vertical with factor x2 and x4 available
- VGA to NTSC/PAL conversion circuit built in

12.3 OSD Block Diagram

The figure below shows the OSD block diagram.

Figure 81. OSD Block Diagram



12.4 OSD Windows and their Priorities

OSD supports the following windows:

- OSD Video Window 0 (Main Video Window)– displays YCbCr 4:2:2 video data
- OSD Video Window 1 – displays YCbCr 4:2:2 video data
- OSD Bitmap Window 0 – displays 1,2,4,8-bit bitmap data using color lookup table
- OSD Bitmap Window 1 – displays 1,2,4,8-bit bitmap data using color lookup table OR
OSD Attribute Window – for pixel level blending of OSD Bitmap Window 0 and video window 0,1
- Rectangular Hardware cursor

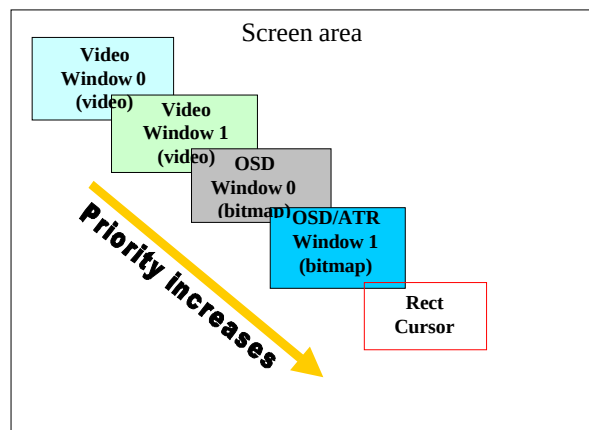
The individual windows can be enabled/disabled using the following registers:

- VIDWINMD - Video Window Mode Setup
- OSDWIN0MD - OSD Window 0 Mode Setup
- OSDWIN1MD - OSD Window 1 Mode Setup
- OSDATRMD - OSD Attribute Window Mode Setup
- RECTCUR - Rectangular Cursor Setup

If overlapping occurs between different windows, the color of each displayed pixel corresponds to the color of the window with the highest priority. The priority order is fixed. When OSD bitmap window 1 is configured to be in attribute mode, its priority does not change.

(highest) Rectangle Cursor > OSD Bitmap Window 1 > OSD Bitmap Window 0 > OSD Video Window 1 > OSD Video Window 0 (lowest)

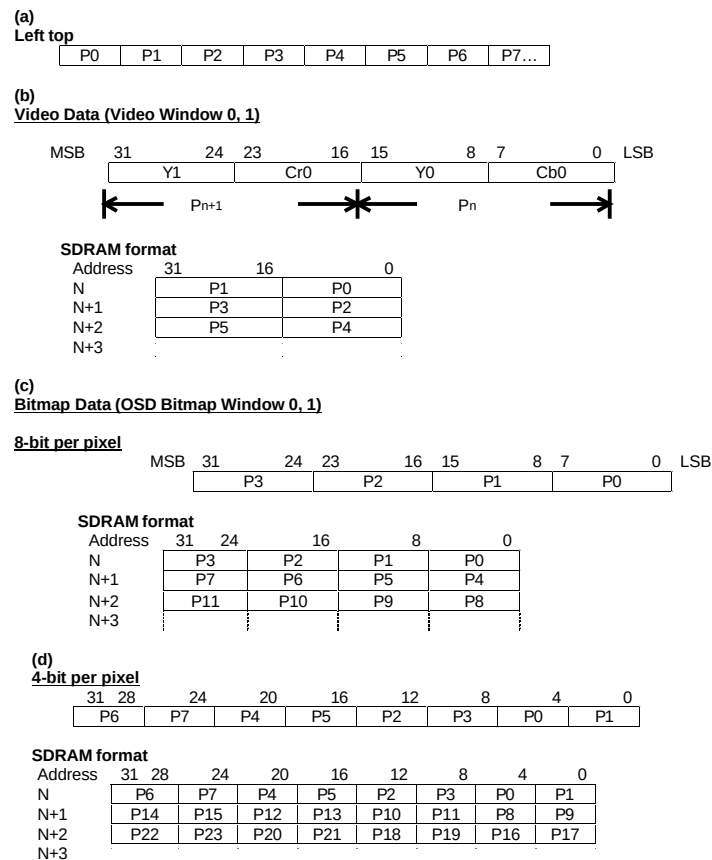
Figure 82. OSD Window Priority



12.5 OSD Data Storage Format

The OSD window data has variable size. In the bitmap window, each pixel can be 1, 2, 4, or 8 bits wide. In the YCbCr 4:2:2 window, each Y, Cb, Cr component takes 8-bit per components, and the components are arranged according to 4:2:2 interleaved format (Cb-Y-Cr-Y-...). In case RGB graphics data needs to be displayed using OSD, the application should perform software conversion to Y,Cr,Cb before storing it. The OSD data is always packed into 32-bit words and left justified. Starting from the upper left corner of the OSD window, all data will be packed into adjacent 32-bit words. Figure 83 shows data format of video, bitmap data in SDRAM.

Figure 83. OSD SDRAM Data Format



(e)

2-bit per pixel

	31	30	28	26	24	22	20	18	16	14	12	10	8	6	4	2	0
	P12	P13	P14	P15	P8	P9	P10	P11	P4	P5	P6	P7	P0	P1	P2	P3	
SDRAM format																	
Address	31	30	28	26	24	22	20	18	16	14	12	10	8	6	4	2	0
N	P12	P13	P14	P15	P8	P9	P10	P11	P4	P5	P6	P7	P0	P1	P2	P3	
N+1	P28	P29	P30	P31	P24	P25	P26	P27	P20	P21	P22	P23	P16	P17	P18	P19	
N+2	P44	P45	P46	P47	P40	P41	P42	P43	P36	P37	P38	P39	P32	P33	P34	P35	
N+3																	

(f)

1-bit per pixel

	31																0
P	P	P	P	P	P	P	P	P	P	P	P	P	P	P	P	P	P
2	2	2	2	2	3	3	1	1	1	1	2	2	2	2	8	9	1
4	5	6	7	8	9	0	1	6	7	8	9	0	1	2	3	0	1

SDRAM format

Address	31																0
N	P24	P25											P4	P5	P6	P7	
N+1	P56	P57											P36	P37	P38	P39	
N+2	P88	P89											P68	P69	P70	P71	
N+3																	

12.6 OSD Configuration and Control

Many of the OSD registers contain bits that are latched by the VD signal, which is the vertical synch pulse generated by the DM270's Video Encoder module. Data written to a latched bit does not take affect until the VD pulse is received. This allows registers that control the OSD, such as the SDRAM data address, display window size, display zoom configuration, etc. to be changed in between successive VD pulses without corrupting the current display. In the OSD register descriptions, bit fields marked with (*) are latched by the VD signal.

12.6.1 SDRAM location and offset

The location of data stored in SDRAM is defined by several memory mapped registers (see Table 15.). The SDRAM addresses are specified in units of 32 bytes, since data is transferred from SDRAM to the OSD hardware in bursts of 32 bytes. Also SDRAM address is specified as offset from start of SDRAM region.

For example, let the video window 0 video data be located at absolute SDRAM address 0x97AB00. The address register would be set as shown below,

```
uint32 window_addr;
```

```
window_addr = (0x97A B00 - 0x900000)/32
```

```
OSD_FSET( VIDWINADH, V0AH, window_addr >> 16);
OSD_RSET( VIDWIN0ADL, window_addr & 0xFFFF);
```

Thus SDRAM address must be 32byte aligned.

Table 15. SDRAM address location registers

OSD Window	SDRAM address location register
Video Window 0	VIDWINADH - Video Window 0/1 Address - High VIDWIN0ADL - Video Window 0 Address - Low
Video Window 1	VIDWINADH - Video Window 0/1 Address - High VIDWIN1ADL - Video Window 1 Address - Low
OSD Bitmap Window 0	OSDWINADH - OSD Window 0/1 Address - High OSDWIN0ADL - OSD Window 0 Address - Low
OSD Bitmap Window 1 / Attribute Window	OSDWINADH - OSD Window 0/1 Address - High OSDWIN1ADL - OSD Window 1 Address - Low

The offset registers allow a cropped segment of an image to be easily displayed. An offset register specifies the distance, in SDRAM, from the start of a displayed line to the start of the next displayed line. The offset is in units of 32 bytes.

For example, for OSD bitmap window 0, with 4-bit pixel width. If the width of the bitmap is 320 pixels. Since for 4-bit bitmap, two pixel is one byte, hence width of data stored in SDRAM would be (320x8)/16 bytes and offset register would be set as,

```
OSD_RSET(OSDWIN0FST, ((320*8)/16)/32 );
```

Thus width of data stored in SDRAM must be multiple of 32bytes. In case, the width of data in SDRAM is not multiple of 32bytes, then padding must be done to make it multiple of 32bytes. Note, the window display width however has no such restriction.

Table 16. SDRAM offset registers

OSD Window	SDRAM offset register
Video Window 0	VIDWIN0OFST - Video Window 0 Offset
Video Window 1	VIDWIN1OFST - Video Window 1 Offset
OSD Bitmap Window 0	OSDWIN0OFST - OSD Window 0 Offset
OSD Bitmap Window 1 / Attribute Window	OSDWIN1OFST - OSD Window 1 Offset

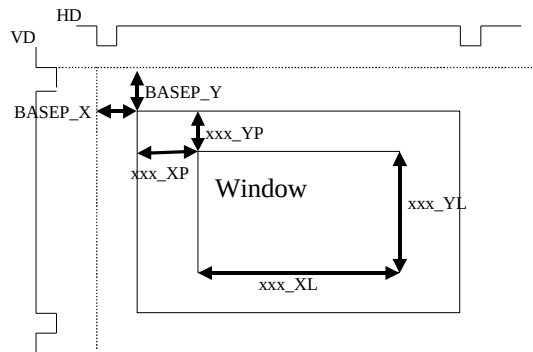
12.6.2 Windows positioning

The windows use a common reference pixel (base pixel). The position of this pixel is determined from the beginning of the Video Encoder HD and the beginning of the Video Encoder VD signal.

For each window (video or bitmap) and for the cursor, the location of the upper left corner can be specified. The display size of the window needs to be specified as well.

The relationship of the display positions of each window is shown below.

Figure 84. Window Positioning



Window start position is specified with respect to the BASE_X, BASE_Y position (see register BASEPX - Base Pixel X, register BASEPY - Base Pixel Y). Window start position in X-direction and window width is specified in units of pixels. Window start position in Y-direction and window height is specified in units of field lines. The table shows the register used for window positioning.

Table 17. Window positioning registers

OSD Window	Window positioning registers
Video Window 0	VIDWIN0XP - Video Window 0 X -Position VIDWIN0YP - Video Window 0 Y -Position VIDWIN0XL - Video Window 0 X -Size VIDWIN0YL - Video Window 0 Y -Size
Video Window 1	VIDWIN1XP - Video Window 1 X -Position VIDWIN1YP - Video Window 1 Y -Position VIDWIN1XL - Video Window 1 X -Size VIDWIN1YL - Video Window 1 Y -Size
OSD Bitmap Window 0	OSDWIN0XP - OSD Bitmap Window 0 X -Position OSDWIN0YP - OSD Bitmap Window 0 Y -Position OSDWIN0XL - OSD Bitmap Window 0 X -Size OSDWIN0YL - OSD Bitmap Window 0 Y -Size
OSD Bitmap Window 1 / Attribute Window	OSDWIN1XP - OSD Bitmap Window 1 X -Position OSDWIN1YP - OSD Bitmap Window 1 Y -Position OSDWIN1XL - OSD Bitmap Window 1 X -Size OSDWIN1YL - OSD Bitmap Window 1 Y -Size
Hardware cursor	CURXP - Rectangular Cursor Window X -Position CURYP - Rectangular Cursor Window Y -Position CURXL - Rectangular Cursor Window X -Size CURYL - Rectangular Cursor Window Y -Size

12.6.3 Window Mode

12.6.3.1 Field / Frame Mode

Each video, bitmap window has two display modes: field mode and frame mode. The video encoder reads data from the video, bitmap windows with the help of two fields : even field and odd field. When the window mode is field mode, each line is duplicated in both even field as well as odd field. When window mode is frame mode, the even field reads every even line and the odd field reads every odd line. This relationship is shown in the below figures,

Figure 85. Window Field Mode

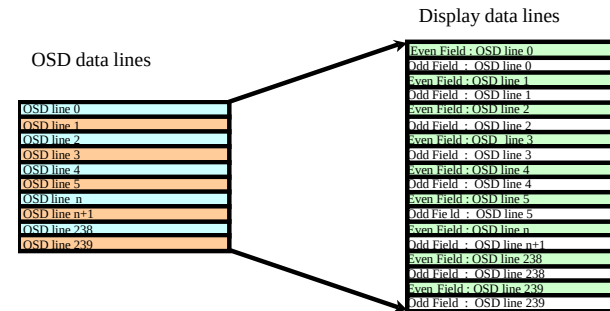
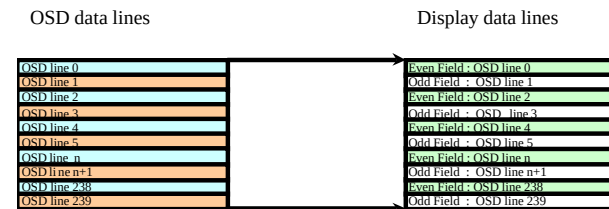


Figure 86. Window Frame Mode



Field/frame mode selection for different windows is done using the following registers :

- VIDWINMD - Video Window Mode Setup
- OSDWIN0MD - OSD Window 0 Mode Setup
- OSDWIN1MD - OSD Window 1 Mode Setup
- OSDATRMD - OSD Attribute Window Mode Setup

12.6.3.2 Window Zooming

The video window and OSD Bitmap window can be zoomed along their horizontal and vertical directions with a x2 or x4 expansion coefficients. When the video data is resized, a horizontal and vertical filter is applied to the Y information. There is no filter applied to the OSD Bitmap windows. The filter applied for Y information is shown below

x2:					
input	P0		P1		P2
output	P0	$(P0+P1)/2$	P1	$(P1+P2)/2$	P2
x4:					
input	P0			P1	
output	P0	P0	$(P0+P1)/2$	P1	$(P1+P2)/2$

The figure below shows the zoom process and the parameters that must be setup to execute a zoom. The user must set the starting SDRAM address of the area desired to be magnified, the offset, the zoom factor and the display window size. The OSD will take enough data, starting from the start address, to generate a magnified image that fits into the display size.

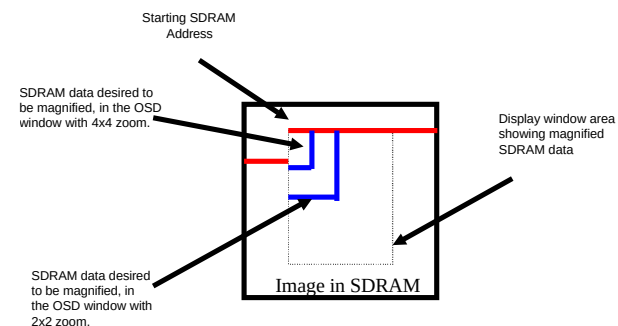
The offset is always the number of bursts in one horizontal line of SDRAM pixel data, as shown in the figure. Set the display position to the desired position of the window, as usual. Set the display height and display width to the desired magnified height and width. When zoom is enabled, a fraction, $\frac{1}{2}$ or $\frac{1}{4}$ in the horizontal and/or vertical directions of SDRAM data starting from the SDRAM start position will be magnified to fit into the display area specified by the display height and display width. For example, if the horizontal and vertical directions are set to 2x zoom and the display width and height are set 640x480, then a 320x240 block of data starting from the SDRAM start position will be magnified to 640x480 in the display window.

All of the following registers, used in the zoom process, are latched by the VD signal: SDRAM starting address, offset, display position, display height, display width, 2x/4x zoom. So you can safely update these registers for your zoom in between VD pulses.

OSD Zoom is set for various windows using the following registers:

- VIDWINMD - Video Window Mode Setup
- OSDWIN0MD - OSD Window 0 Mode Setup
- OSDWIN1MD - OSD Window 1 Mode Setup
- OSDATRMD - OSD Attribute Window Mode Setup

Figure 87. OSD Zoom Process



12.7 Bitmap Window

Bitmap window allows the user to display graphics and icons on the display unit. Upto two windows can be used as bitmap windows OR one window can be used as bitmap window and other can act as attribute window (see section 12.7.3 Attribute Window). OSD Bitmap Window 0 can act as a bitmap window. OSD Bitmap Window 1 can act as either bitmap window or attribute window. The bitmap window uses a color lookup table (CLUT) in order to display a color for a given bitmap pixel value. A total of 256 colors are available. The maximum width of bitmap pixel is thus 8-bit. However 1,2,4-bit bitmaps are also supported. See section 12.5 OSD Data Storage Format for SDRAM storage format of bitmap data. Bitmap pixel width is selected using the following registers

- OSDWIN0MD - OSD Window 0 Mode Setup
- OSDWIN1MD - OSD Window 1 Mode Setup

12.7.1 Color Lookup Table

The user has the option to select either the ROM CLUT or he can define his own color look table in RAM.

12.7.1.1 ROM Color Look Up Table

Figure 88. ROM CLUT Color values

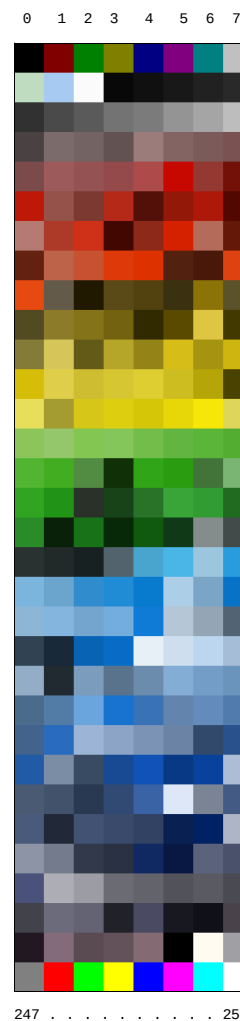
8 Bit Offset	R	G	B
0	00	00	00
1	80	00	00
2	00	80	00
3	80	80	00
4	00	00	80
5	80	00	80
6	00	80	80
7	C0	C0	C0
8	C0	DC	C0
9	A6	CA	F0
10	FB	FB	FB
11	08	08	08
12	10	10	10
13	18	18	18
14	21	21	21
15	29	29	29
16	31	31	31
17	4A	4A	4A
18	5A	5A	5A
19	73	73	73
20	7B	7B	7B
21	94	94	94
22	A5	A5	A5
23	BD	BD	BD
24	4A	42	42
25	7B	6B	6B
26	73	63	63
27	63	52	52
28	9C	7B	7B
29	84	63	63
30	7B	5A	5A
31	7B	52	52
32	7B	4A	4A
33	9C	5A	5A
34	94	52	52
35	94	4A	4A
36	AD	4A	4A
37	C6	08	08
38	94	39	31
39	73	10	08
40	BD	18	08
41	94	52	4A
42	7B	39	31
43	B5	29	18
44	52	10	08
45	94	18	08
46	AD	18	08
47	52	08	00
48	B5	7B	73
49	AD	39	29
50	CE	31	18
51	42	08	00
52	8C	29	18
53	D6	21	00
54	B5	6B	5A
55	63	18	08
56	63	21	10
57	BD	63	4A
58	C6	52	31
59	DE	39	08
60	DE	31	00
61	52	21	10
62	4A	18	08
63	DE	42	10

8 Bit Offset	R	G	B
64	E7	4A	10
65	63	5A	4A
66	21	18	00
67	5A	4A	18
68	52	42	10
69	39	31	10
70	8C	73	08
71	5A	52	29
72	52	4A	21
73	8C	7B	29
74	84	73	18
75	73	63	10
76	31	29	00
77	5A	4A	00
78	DE	C6	42
79	42	39	00
80	84	7B	39
81	D6	C6	5A
82	63	5A	18
83	B5	A5	29
84	94	84	18
85	D6	BD	18
86	A5	94	10
87	CE	B5	10
88	D6	BD	08
89	DE	CE	4A
90	CE	BD	31
91	D6	C6	31
92	DE	CE	31
93	CE	BD	21
94	B5	A5	08
95	4A	42	00
96	E7	DE	5A
97	A5	9C	31
98	D6	C6	18
99	DE	CE	10
100	D6	C6	08
101	E7	D6	08
102	F7	E7	08
103	DE	D6	5A
104	8C	C6	5A
105	94	C6	6B
106	84	C6	52
107	84	C6	5A
108	73	BD	4A
109	63	B5	42
110	5A	B5	39
111	52	AD	31
112	52	B5	31
113	42	AD	21
114	52	8C	42
115	10	31	08
116	31	A5	18
117	29	9C	10
118	42	73	39
119	7B	B5	73
120	31	A5	21
121	21	94	18
122	29	31	29
123	18	42	18
124	29	73	29
125	39	A5	39
126	31	9C	31
127	21	6B	21

8 Bit Offset	R	G	B
128	29	8C	29
129	08	21	08
130	18	73	18
131	08	29	08
132	10	5A	10
133	10	39	18
134	84	8C	8C
135	42	4A	4A
136	29	31	31
137	21	29	29
138	18	21	21
139	52	63	6B
140	4A	A5	CE
141	4A	B5	E7
142	9C	C6	DE
143	29	9C	DE
144	7B	B5	DE
145	6B	A5	CE
146	31	8C	CE
147	21	8C	D6
148	08	7B	CE
149	AD	CE	E7
150	7B	A5	C6
151	08	73	C6
152	8C	B5	D6
153	84	B5	DE
154	73	A5	CE
155	73	AD	DE
156	10	7B	D6
157	B5	C6	D6
158	94	A5	B5
159	52	63	73
160	31	42	52
161	18	29	39
162	08	63	B5
163	08	6B	C6
164	E7	EF	F7
165	CE	DE	EF
166	BD	D6	EF
167	A5	BD	D6
168	94	AD	C6
169	21	29	31
170	7B	9C	BD
171	5A	73	8C
172	6B	8C	AD
173	84	AD	D6
174	73	9C	C6
175	6B	94	BD
176	4A	6B	8C
177	52	7B	A5
178	6B	A5	DE
179	18	73	CE
180	39	73	B5
181	63	84	AD
182	63	8C	BD
183	52	7B	AD
184	42	63	9C
185	29	6B	BD
186	9C	B5	D6
187	8C	A5	C6
188	7B	94	B5
189	6B	84	A5
190	31	4A	6B
191	29	52	8C

8 Bit Offset	R	G	B
192	21	5A	A5
193	7B	8C	A5
194	39	4A	63
195	18	4A	94
196	10	52	B5
197	08	39	84
198	08	42	9C
199	AD	BD	D6
200	4A	5A	73
201	42	52	6B
202	29	39	52
203	31	4A	73
204	39	63	A5
205	DE	E7	F7
206	7B	84	94
207	42	5A	84
208	4A	5A	7B
209	21	29	39
210	42	52	73
211	39	4A	6B
212	31	42	63
213	08	21	52
214	00	21	63
215	AD	B5	C6
216	8C	94	A5
217	73	7B	8C
218	31	39	4A
219	29	31	42
220	10	29	63
221	08	18	42
222	5A	63	7B
223	4A	52	6B
224	4A	52	7B
225	AD	AD	B5
226	9C	9C	A5
227	6B	6B	73
228	63	63	6B
229	52	52	5A
230	5A	5A	63
231	4A	4A	52
232	42	42	4A
233	6B	6B	7B
234	63	63	73
235	21	21	29
236	4A	4A	63
237	18	18	21
238	10	10	18
239	4A	42	4A
240	21	18	21
241	84	6B	7B
242	5A	4A	52
243	63	52	5A
244	84	6B	73
245	00	00	00
246	FF	FB	F0
247	A0	A0	A4
248	80	80	80
249	FF	00	00
250	00	FF	00
251	FF	FF	00
252	00	00	FF
253	FF	00	FF
254	00	FF	FF
255	FF	FF	FF

Figure 89. ROM CLUT Color Chart



12.7.1.2 RAM Color Lookup Table

If you do not want to use the default ROM CLUT, you can create your own CLUT and store it within the internal OSD RAM memory.

To in setup the OSD RAM CLUT, you need to do the following:

- Wait for the CPBSY bit of register MISCCTL - Miscellaneous Control to be zero,
- Write into the register CLUTRAMYCB - CLUT RAM Y/Cb Setup. The first byte corresponds to the Y value of the color to be added to the CLUT, the second byte corresponds to the Cb value color to be added to the CLUT.
- Wait again for the CPBSY bit of register MISCCTL - Miscellaneous Control to return to zero,
- Write into the register CLUTRAMCR - CLUT RAM Cr/Mapping Setup. The first byte correspond to the Cr value of the color to be added to the CLUT, the second byte is the RAM offset address of the Y, Cr, Cb data to be added to the CLUT.

Note, after a device reset, the color values of the RAM CLUT are undetermined.

Sample code to write to RAM CLUT is given below,

```

uint8 lut[n*3];

/* Y, Cb, Cr value range = 0..255, n = number of LUT entries to be
added, 1..256 */

lut[0] = Y0;          // color for index 0
lut[1] = Cb0;
lut[2] = Cr0;
lut[3] = Y1;          // color for index 1
lut[4] = Cb1;
lut[5] = Cr1;
...
lut[(n-1)*3] = Y(n-1); // color for index 255
lut[(n-1)*3+1] = Cb(n-1);
lut[(n-1)*3+2] = Cr(n-1);

for(i=0; i<n; i++) {
    while( OSD_FGET( MISCCTL, CPBSY )
        ;
        OSD_RSET( CLUTRAMYCB,
            ((uint16)lut[i*3] << 8) |
            lut[i*3+1] ); // Y, Cb value
        OSD_RSET( CLUTRAMCR,
            ((uint16)lut[i*3+2] << 8) |
            i ); // address, Cr value
}

```

12.7.1.3 Selecting RAM/ROM CLUT

Selection between RAM or ROM CLUT can be done separately for OSD Bitmap Window 0, OSD Bitmap Window 1. For OSD Bitmap Window 0 use bit 12 in register OSDWIN0MD - OSD Window 0 Mode Setup. For OSD Bitmap Window 1 use bit 12 in register OSDWIN1MD - OSD Window 1 Mode Setup.

12.7.1.4 1,2,4-bit Bitmap and CLUT

When the bitmap size is 1,2 or 4-bit, the registers W0BMP01 - Window 0 Bitmap Value to Palette Map 0/1 to W1BMPEF - Window 1 Bitmap Value to Palette Map E/F can be used to map any of the 256 colors to a bitmap value. For example, for 1-bit bitmap data, any of the 256 colors in the CLUT can be mapped to bit value "0" and any of the 256 colors can be mapped to bitmap "1". This mapping can be specified separately for each OSD Bitmap window. The table below shows the register which can be used to select the color for a bitmap value

Table 18. 1,2,4-bit Bitmap Color Selection

Register : Field (x = 0,1 for OSD Bitmap Window 0,1 respectively)	4-bit bitmap Color corresponding to bitmap value	2-bit bitmap Color corresponding to bitmap value	1-bit bitmap Color corresponding to bitmap value
WxBMP01 : PAL00	0	0	0
WxBMP01 : PAL01	1	-	-
WxBMP23 : PAL02	2	-	-
WxBMP23 : PAL03	3	-	-
WxBMP45 : PAL04	4	-	-
WxBMP45 : PAL05	5	1	-
WxBMP67 : PAL06	6	-	-
WxBMP67 : PAL07	7	-	-
WxBMP89 : PAL08	8	-	-
WxBMP89 : PAL09	9	-	-
WxBMPAB : PAL10	10	2	-
WxBMPAB : PAL11	11	-	-
WxBMPAB : PAL12	12	-	-
WxBMPAB : PAL13	13	-	-
WxBMPEF : PAL14	14	-	-
WxBMPEF : PAL15	15	3	1

12.7.2 Blending and Transparency

The hardware supports a transparency mode with bitmap window. If transparency is enabled, then any pixel on the bitmap display that has a value of 0 will allow video to be displayed. In this case, blending factor would be applied before displaying the video. Essentially, 0 valued pixels are considered transparent, and the background color (after blending) will show through the bitmap. See section 12.10.1 OSD Mode Selection – Background Color, Cb/Cr format to set the background color.

Color blending at the pixel level is also supported. This feature is available for the bitmap displays only (OSD bitmap window 0, 1). If the window color blending is enabled, the amount of blending of each pixel is determined by the blending factor. As shown in the following table, the window blending supports 8 different levels, according to the selected blending factor.

Table 19. Blending and Transparency in OSD

Transparency	Blending Factor	OSD Window contribution	Video contribution
OFF	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8

ON	7	1	0
	If pixel value = 0		
	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0
	If pixel value != 0		
	X	1	0

Blending factor, transparency is set using register OSDWIN0MD - OSD Window 0 Mode Setup, register OSDWIN1MD - OSD Window 1 Mode Setup for OSD Bitmap window 0,1 respectively

12.7.3 Attribute Window

OSD bitmap window 1 can be configured as an attribute window. The attribute window allows you to apply blending and blinking to individual pixels in the display windows. In particular the attribute window allows you to:

- Blend pixels from OSD Window 0 with Video Window 0 or Video Window 1
- Blink pixels in OSD Window 0

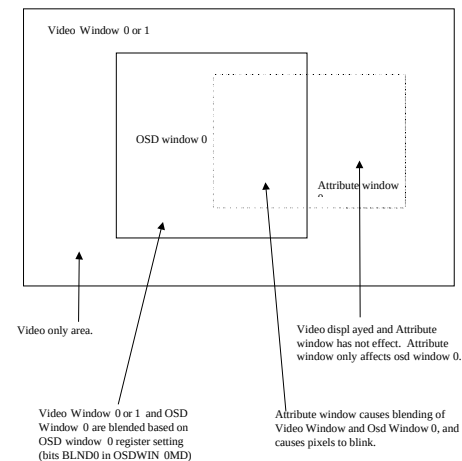
The SDRAM start address, position, and size of the attribute window are configured exactly as OSD window 1; however, the data in SDRAM is interpreted as shown in Figure 90. The first bit specifies whether or not to blink the corresponding pixel, and the last 3 bits specify the blending factor to apply to the corresponding pixel. The blend factor is defined in Table 19. .

Figure 90. Attribute Data Format

Bit 3	Bit 2	Bit 1	Bit 0
Blink	Blend Factor		

As shown in Figure 91 the attribute window applies blending and blinking to OSD window 0 depending on the position of the attribute window.

Figure 91. The Attribute Window's Relationship to the Other Windows



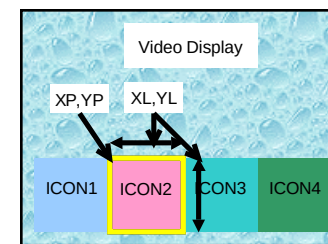
Attribute window is configured using register OSDATRMD - OSD Attribute Window Mode Setup

12.8 Rectangular Hardware Cursor

The rectangular hardware cursor always appears on top of other OSD Windows. The cursor's size, color, horizontal and vertical thickness can be specified. This cursor can also be configured to use the ROM or RAM CLUT.

Figure 92 shows an example usage of the rectangle cursor. Yellow Line shows the rectangle cursor and ICON1-4 are displayed by using OSD window 0 or1.

Figure 92. Rectangular Cursor



Rectangular cursor can be setup using the following registers:

- o RECTCUR - Rectangular Cursor Setup
- o CURXP - Rectangular Cursor Window X-Position
- o CURYP - Rectangular Cursor Window Y-Position
- o CURXL - Rectangular Cursor Window X-Size
- o CURYL - Rectangular Cursor Window Y-Size

12.9 Window Expansion (VGA to NTSC/PAL Conversion)

OSD supports conversion from VGA size image to NTSC/PAL image during display of video windows, bitmap windows. Table 20. shows the expansion ratios supported.

Table 20. Window Expansion		
	Horizontal	Vertical
Video Window	*x9/8	*x6/5
Bitmap Window	--	*x6/5

Use register OSDMODE - OSD Mode Setup for enabling this expansion filter. See also section 12.6.3.2 Window Zooming.

12.10 Miscellaneous Features

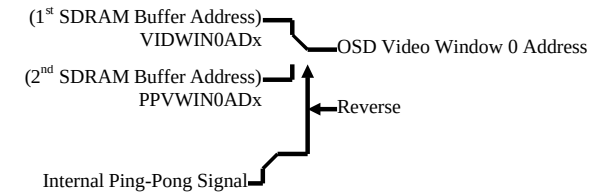
12.10.1 OSD Mode Selection – Background Color, Cb/Cr format

Background color selection and Cb/Cr format selection can be done using register OSDMODE - OSD Mode Setup.

12.10.2 OSD Ping Pong Buffer

The main video window (video window 0) supports ping-pong buffers. Initially the SDRAM source address for OSD Video window 0 can be set to the SDRAM address specified in register VIDWINADH - Video Window 0/1 Address - High and VIDWINADL - Video Window 0 Address - Low, and when the ping pong buffer toggle bit is cleared the SDRAM source address switches to the address specified in register PPVWINADH - Ping-Pong Video Window 0 Address (High) and register PPVWINADL - Ping-Pong Video Window 0 Address (Low). Ping Pong buffer switching is done using register MISCCTL - Miscellaneous Control

Figure 93. Ping-Pong buffers for the main video window (OSD Video Window 0)



12.12.2 VIDWINMD - Video Window Mode Setup

VIDWINMD 0003:0682 offset: 0x02 default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3
Name	RSV	RSV	VHZ1[1]	VHZ1[0]	VVZ1[1]	VVZ1[0]	VFF1	ACT1	RSV	RSV	VHZ0[1]	VHZ0[0]	VVZ0[1]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	-	-	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	VHZ1	0	R/W	<u>Video window1 horizontal direction zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
11:10	VVZ1	0	R/W	<u>Video window1 vertical direction zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9	VFF1	0	R/W	<u>Video window1 display mode</u> » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
8	ACT1	0	R/W	<u>Sets image display on/off video window1</u> » 0: Off « » 1: On « *This bit is latched by VD.
7:6	RSV			Reserved
5:4	VHZ0	0	R/W	<u>Video window0 horizontal direction zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
3:2	VVZ0	0	R/W	<u>Video window0 vertical direction zoom</u> » 0: x1 « » 1: x2 « » 0: x4 « » 1: Reserved (same as '00') « *This bit is latched by VD.
1	VFF0	0	R/W	<u>Video window0 display mode</u> » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	ACT0	0	R/W	<u>Sets image display on/off video window0</u> » 0: Off « » 1: On « *This bit is latched by VD.

12.12.3 OSDWIN0MD - OSD Window 0 Mode Setup

OSDWIN0MD 0003:0684 offset: 0x04 default: 0x0000															
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Name	RSV	RSV	RSV	CLUTS0	OHZ0[1]	OHZ0[0]	OVZ0[1]	OVZ0[0]	BMW0[1]	BMW0[0]	BLND0[2]	BLND0[1]	BLND0[0]	TE0	OFF0
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	CLUTS0	0	R/W	<u>CLUT select</u> » 0: ROM-look-up table « » 1: RAM-look-up table « Selects look-up table that is used for OSD Window0 *This bit is latched by VD.
11:10	OHZ0	0	R/W	<u>OSD Window0 Horizontal Zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9:8	OVZ0	0	R/W	<u>OSD Window0 Vertical Zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
7:6	BMW0	0	R/W	<u>Bitmap bit width for OSD window0</u> » 00: 1 « » 01: 2 « » 10: 4 « » 11: 8 « *This bit is latched by VD.
5:3	BLND0	0	R/W	<u>Blending Ratio for OSD Window0</u> Blending ratio of OSD Window 0 and Video window 0 [2:0] Win0 VidWin » 000: 0 1 « » 001: 1/8 7/8 « » 010: 2/8 6/8 « » 011: 3/8 5/8 « » 100: 4/8 4/8 « » 101: 5/8 3/8 « » 110: 6/8 2/8 « » 111: 1 0 « *This bit is latched by VD.
2	TE0	0	R/W	<u>Transparency Enable for OSD Window0</u> » 0: disable « » 1: enable « When transparency is off, entire window is blended with video window. When transparency is on, blending is only performed for pixels whose bitmap value is 0. *This bit is latched by VD.
1	OFF0	0	R/W	<u>OSD Window0 Display Mode</u> » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	OACT0	0	R/W	<u>OSD Window0 Active (displayed)</u> » 0: Off « » 1: On « *This bit is latched by VD.

12.12.4 OSDWIN1MD - OSD Window 1 Mode Setup

OSDWIN1MD 0003:0686 offset: 0x06 default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3
Name	OASW	RSV	RSV	CLUTS1	OHZ1[1]	OHZ1[0]	OVZ1[1]	OVZ1[0]	BMW1[1]	BMW1[0]	BLND1[2]	BLND1[1]	BLND1[0]
R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	-	-	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	OASW	0	R/W	<u>OSD Window1 Attribute Mode Enable</u> » 0: OSD Window1 « » 1: Attribute window « *This bit is latched by VD.
14:13	RSV			Reserved
12	CLUTS1	0	R/W	<u>CLUT Select</u> OASW=0 » 0: ROM-look-up table « » 1: RAM-look-up table « Selects look-up table that is used for OSD Window1
11:10	OHZ1	0	R/W	<u>OSD Window1 Horizontal Zoom</u> OASW=0 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9:8	OVZ1	0	R/W	<u>OSD Window1 Vertical Zoom</u> OASW=0 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
7:6	BMW1	0	R/W	<u>Bitmap Bit Width</u> OASW=0 Bitmap bit width for OSD Window1 » 00: 1-bit « » 01: 2-bits « » 10: 4-bits « » 11: 8-bits « *This bit is latched by VD.
5:3	BLND1	0	R/W	<u>Blending Ratio for OSD Window1</u> OASW=0 Sets blending ratio of OSD window 1 and Video Window 0 [2:0] Win0 VidWin » 000: 0 1 « » 001: 1/8 7/8 « » 010: 2/8 6/8 « » 011: 3/8 5/8 « » 100: 4/8 4/8 « » 101: 5/8 3/8 « » 110: 6/8 2/8 « » 111: 1 0 « *This bit is latched by VD.
2	TE1	0	R/W	<u>Transparency Enable for OSD Window0</u> OASW=0 » 0: disable « » 1: enable « When transparency is off, entire window is blended with video window. When transparency is on, blending is only performed for pixels whose bitmap value is 0. *This bit is latched by VD.
1	OFF1	0	R/W	<u>OSD Window0 Display Mode</u> OASW=0 » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	OACT1	0	R/W	<u>OSD Window0 Active (displayed)</u> OASW=0 » 0: Off « » 1: On « *This bit is latched by VD.

12.12.5 OSDATRMD - OSD Attribute Window Mode Setup

OSDATRMD 0003:0686 offset: 0x06 default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3
Name	OASW	RSV	RSV	NA1	OHZA[1]	OHZA[0]	OVZA[1]	OVZA[0]	BLNKINT1	BLNKINT0	NA2[2]	NA2[1]	NA2[0]
R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	-	-	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	OASW	0	R/W	<u>OSD Window1 Attribute Mode Enable</u> » 0: OSD Window1 « » 1: Attribute window « This bit enables attribute mode for OSD Window1. *This bit is latched by VD.
14:13	RSV			Reserved
12	NA1	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 *This bit is latched by VD.
11:10	OHZA	0	R/W	<u>OSD Attribute Window Horizontal Zoom</u> OASW=1 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9:8	OVZA	0	R/W	<u>Attribute Window Vertical Zoom</u> OASW=1 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
7:6	BLNK INT	0	R/W	<u>Blinking Interval</u> OASW=1 » 00: 1-unit « » 01: 2-units « » 10: 3-units « » 11: 4-units « Specifies the blinking interval of the attribute window in units of 8 VD pulses *This bit is latched by VD.
5:3	NA2	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 *This bit is latched by VD.
2	NA3	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 *This bit is latched by VD.
1	OFFA	0	R/W	<u>OSD Attribute Window Display Mode</u> OASW=1 » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	BLNK	0	R/W	<u>OSD Attribute Window Blink Enable</u> OASW=1 » 0: disable « » 1: enable « *This bit is latched by VD.

12.12.6 RECTCUR - Rectangular Cursor Setup

RECTCUR	0003:0688												offset:	0x08				default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	RCAD[7]	RCAD[6]	RCAD[5]	RCAD[4]	RCAD[3]	RCAD[2]	RCAD[1]	RCAD[0]	CLUTSR	RCHW[2]	RCHW[1]	RCHW[0]	RCVW[2]	RCVW[1]	RCVW[0]	RCACT						
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W						
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						

Bit	Name	Reset Value	R/W	Function
15:8	RCAD	0	R/W	Rectangular Cursor Color Palette Address <u>CLUT Select</u> » 0: ROM-look-up table « » 1: RAM-look-up table « *This bit is latched by VD.
7	CLUTSR	0	R/W	
6:4	RCHW	0	R/W	<u>Rectangular Cursor Horizontal Line Width</u> Width is 4xRCHW, with 0 interpreted as 1 pixel Range is: 1,4,8,16,20,24,28 pixels *This bit is latched by VD.
3:1	RCVW	0	R/W	Rectangular Cursor Vertical Line Width Width is 2xRCVW, with 0 interpreted as 1 line Range is: 1,2,4,6,8,10,12,14 lines *This bit is latched by VD.
0	RCACT	0	R/W	<u>Rectangular Cursor Active</u> (displayed) » 0: Off « » 1: On « *This bit is latched by VD.

12.12.7 RSV0 - Reserved

RSV0	0003:068A														offset:	0x0A		default:	0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV				
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-				
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-				

Bit	Name	Reset Value	R/W	Function
15:0	RSV			Reserved

12.12.8 VIDWIN0OFST - Video Window 0 Offset

[illegible][illegible]

12.12.9 VIDWIN1OFST - Video Window 1 Offset

[illegible][illegible]

12.12.10 OSDWIN0OFST - OSD Window 0 Offset

[illegible]

Bit	Name	Reset Value	R/W	Function
15: 9	RSV			Reserved
				<u>OSD Window 0 Line Offset</u>
				Number of burst transfers (32-bytes) in a horizontal line.
				This depends on OSD window bit depth:
				Line width in pixels * bitdepth / 256-bits/burst
				i.e., $64 * 8 / 256 = 2$
8: 0	00L0	0	R/W	Note: If line width and bitdepth settings for the window result in a non-integer value, round the value up to the next larger integer and organize the data in SDRAM so that each line begins on a burst boundary. *This bit is latched by VD.

12.12.11 OSDWIN1OFST - OSD Window 1 Offset

[illegible][illegible]

12.12.12 VIDWINADH - Video Window 0/1 Address - High

VIDWINADH		0003:0694		offset:		0x14												default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	VIAH[5]	VIAH[4]	VIAH[3]	VIAH[2]	VIAH[1]	VIAH[0]	RSV	RSV	VOAH[5]	VOAH[4]	VOAH[3]	VOAH[2]	VOAH[1]	VOAH[0]					
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	0	0	0	0	0	0	-	-	0	0	0	0	0	0					

[illegible]

12.12.13 VIDWIN0ADL - Video Window 0 Address - Low

[illegible][illegible]

12.12.14 **VIDWIN1ADL - Video Window 1 Address - Low**

[illegible][illegible]

12.12.15 OSDWINADH - OSD Window 0/1 Address - High

OSDWINADH				0003:069A				offset:				0x1A								default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0											
Name	RSV	RSV	O1AH[5]	O1AH[4]	O1AH[3]	O1AH[2]	O1AH[1]	O1AH[0]	RSV	RSV	O0AH[5]	O0AH[4]	O0AH[3]	O0AH[2]	O0AH[1]	O0AH[0]											
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W											
Default	-	-	0	0	0	0	0	0	-	-	0	0	0	0	0	0											

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
OSD Window 1 / Attribute Window SDRAM Source Address - High 6 MSBs of SDRAM source address				
13:8	O1AH	0	R/W	The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. *This bit is latched by VD.
7:6	RSV			Reserved
OSD Window 0 SDRAM Source Address - High 6 MSBs of SDRAM source address				
5:0	O0AH	0	R/W	The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. *This bit is latched by VD.

12.12.16 OSDWIN0ADL - OSD Window 0 Address - Low

OSDWIN0ADL				0003:069C				offset:				0x1C								default:				0x0000								
Bit	15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0	
Name	O0AL[15]		O0AL[14]		O0AL[13]		O0AL[12]		O0AL[11]		O0AL[10]		O0AL[9]		O0AL[8]		O0AL[7]		O0AL[6]		O0AL[5]		O0AL[4]		O0AL[3]		O0AL[2]		O0AL[1]		O0AL[0]	
R/W	R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W		R/W	
Default	0		0		0		0		0		0		0		0		0		0		0		0		0		0		0		0	

Bit	Name	Reset Value	R/W	Function
15:0	O0AL	0	R/W	OSD Window 0 SDRAM Source Address - Low 16 LSBs of SDRAM source address The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. *This bit is latched by VD.

12.12.17 OSDWIN1ADL - OSD Window 1 Address - Low

[illegible][illegible]

12.12.18 **BASEPX - Base Pixel X**

[illegible][illegible]

12.12.19 **BASEPY - Base Pixel Y**

[illegible][illegible]

12.12.20 **VIDWIN0XP - Video Window 0 X-Position**

[illegible][illegible]

12.12.21 VIDWIN0YP - Video Window 0 Y-Position

[illegible][illegible]

12.12.22 VIDWIN0XL - Video Window 0 X-Size

[illegible][illegible]

12.12.23 **VIDWIN0YL - Video Window 0 Y-Size**

[illegible][illegible]

12.12.24 **VIDWIN1XP - Video Window 1 X-Position**

[illegible][illegible]

12.12.25 **VIDWIN1YP - Video Window 1 Y-Position**

VIDWIN1YP		0003.06AE				offset:		0x2E										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	VLY[8]	VLY[7]	VLY[6]	VLY[5]	VLY[4]	VLY[3]	VLY[2]	VLY[1]	VLY[0]					
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0					

[illegible]

12.12.26 **VIDWIN1XL - Video Window 1 X-Size**

[illegible][illegible]

12.12.27 **VIDWIN1YL - Video Window 1 Y-Size**

[illegible][illegible]

12.12.28 **OSDWIN0XP - OSD Bitmap Window 0 X-Position**

[illegible][illegible]

12.12.29 OSDWIN0YP - OSD Bitmap Window 0 Y-Position

[illegible][illegible]

12.12.30 OSDWIN0XL - OSD Bitmap Window 0 X-Size

[illegible][illegible]

12.12.31 OSDWIN0YL - OSD Bitmap Window 0 Y-Size

[illegible][illegible]

12.12.32 OSDWIN1XP - OSD Bitmap Window 1 X-Position

[illegible][illegible]

12.12.33 OSDWIN1YP - OSD Bitmap Window 1 Y-Position

[illegible][illegible]

12.12.34 OSDWIN1XL - OSD Bitmap Window 1 X-Size

[illegible][illegible]

12.12.35 OSDWIN1YL - OSD Bitmap Window 1 Y-Size

[illegible][illegible]

12.12.36 CURXP - Rectangular Cursor Window X-Position

[illegible][illegible]

12.12.37 CURYP - Rectangular Cursor Window Y-Position

[illegible][illegible]

12.12.38 CURXL - Rectangular Cursor Window X-Size

[illegible][illegible]

12.12.39 CURL - Rectangular Cursor Window Y-Size

[illegible][illegible]

12.12.40 RSV1 - Reserved

[illegible][illegible]

12.12.41 RSV2 - Reserved

[illegible][illegible]

12.12.42 W0BMP01 - Window 0 Bitmap Value to Palette Map 0/1

[illegible][illegible]

12.12.43 W0BMP23 - Window 0 Bitmap Value to Palette Map 2/3

[illegible][illegible]

12.12.44 W0BMP45 - Window 0 Bitmap Value to Palette Map 4/5

[illegible][illegible]

12.12.45 **W0BMP67 - Window 0 Bitmap Value to Palette Map 6/7**

W0BMP67		0003_06D6				offset:		0x56										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	PAL0[7]	PAL0[6]	PAL0[5]	PAL0[4]	PAL0[3]	PAL0[2]	PAL0[1]	PAL0[0]	PAL0E[7]	PAL0E[6]	PAL0E[5]	PAL0E[4]	PAL0E[3]	PAL0E[2]	PAL0E[1]	PAL0E[0]					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					

[illegible]

12.12.46 W0BMP89 - Window 0 Bitmap Value to Palette Map 8/9

[illegible][illegible]

12.12.47 **W0BMPAB - Window 0 Bitmap Value to Palette Map A/B**

[illegible][illegible]

12.12.48 W0BMPCD - Window 0 Bitmap Value to Palette Map C/D

[illegible][illegible]

12.12.49 W0BMPEF - Window 0 Bitmap Value to Palette Map E/F

W0BMPF		0003_06DE				offset:		0x5E										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	PAL15[7]	PAL15[6]	PAL15[5]	PAL15[4]	PAL15[3]	PAL15[2]	PAL15[1]	PAL15[0]	PAL14[7]	PAL14[6]	PAL14[5]	PAL14[4]	PAL14[3]	PAL14[2]	PAL14[1]	PAL14[0]					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					

[illegible]

12.12.50 W1BMP01 - Window 1 Bitmap Value to Palette Map 0/1

[illegible][illegible]

12.12.51 W1BMP23 - Window 1 Bitmap Value to Palette Map 2/3

W1BMP23									0003:06E2									offset: 0x62											default: 0x0000			
Bit	15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0															
Name	PAL03[7]	PAL06[6]	PAL05[5]	PAL04[4]	PAL03[3]	PAL02[2]	PAL03[1]	PAL03[0]		PAL02[7]	PAL02[6]	PAL02[5]	PAL02[4]	PAL02[3]	PAL02[2]	PAL02[1]	PAL02[0]															
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W															
Default	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0															

[illegible]

12.12.52 W1BMP45 - Window 1 Bitmap Value to Palette Map 4/5

[illegible][illegible]

12.12.53 W1BMP67 - Window 1 Bitmap Value to Palette Map 6/7

W1BMP67		0003_06E6				offset:		0x66										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	PAL0[7]	PAL0[6]	PAL0[5]	PAL0[4]	PAL0[3]	PAL0[2]	PAL0[1]	PAL0[0]	PAL0[7]	PAL0[6]	PAL0[5]	PAL0[4]	PAL0[3]	PAL0[2]	PAL0[1]	PAL0[0]					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					

[illegible]

12.12.54 W1BMP89 - Window 1 Bitmap Value to Palette Map 8/9

[illegible][illegible]

12.12.55 **W1BMPAB - Window 1 Bitmap Value to Palette Map A/B**

W1BMPAB				0003:06EA				offset: 0x6A				default: 0x0000							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	PALL1[7]	PALL1[6]	PALL1[5]	PALL1[4]	PALL1[3]	PALL1[2]	PALL1[1]	PALL1[0]	PAL10[7]	PAL10[6]	PAL10[5]	PAL10[4]	PAL10[3]	PAL10[2]	PAL10[1]	PAL10[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

[illegible]

12.12.56 W1BMPCD - Window 1 Bitmap Value to Palette Map C/D

[illegible][illegible]

12.12.57 W1BMPEF - Window 1 Bitmap Value to Palette Map E/F

W1BMPF		0003_06EE				offset:		0x6E										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	PAL15[7]	PAL15[6]	PAL15[5]	PAL15[4]	PAL15[3]	PAL15[2]	PAL15[1]	PAL15[0]	PAL14[7]	PAL14[6]	PAL14[5]	PAL14[4]	PAL14[3]	PAL14[2]	PAL14[1]	PAL14[0]					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					

[illegible]

12.12.58 RSV3 - Reserved

[illegible][illegible]

12.12.59 **MISCCTL - Miscellaneous Control**

MISCTCTL		0003:06F4				offset: 0x74								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CPBSY	PPSW	PPRV	RSV	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	-	
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	-	

[illegible]

12.12.60 CLUTRAMYCB - CLUT RAM Y/Cb Setup

CLUTRAMYCB										0003.06F6										offset:		0x76				default:		0x0000	
Bit	15	14	13	12		11	10	9	8	7	6	5	4	3	2	1	0												
Name	Y[7]	Y[6]	Y[5]	Y[4]		Y[3]	Y[2]	Y[1]	Y[0]	C[B][7]	C[B][6]	C[B][5]	C[B][4]	C[B][3]	C[B][2]	C[B][1]	C[B][0]												
R/W	R/W	R/W	R/W	R/W		R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W												
Default	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0												

[illegible]

12.12.61 CLUTRAMCR - CLUT RAM Cr/Mapping Setup

[illegible][illegible]

12.12.62 PPVWIN0ADH - Ping-Pong Video Window 0 Address (High)

PPVWIN0ADH										0003:06FC		offset: 0x7C								default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	VOPAH[5]	VOPAH[4]	VOPAH[3]	VOPAH[2]	VOPAH[1]	VOPAH[0]					
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0					

[illegible]

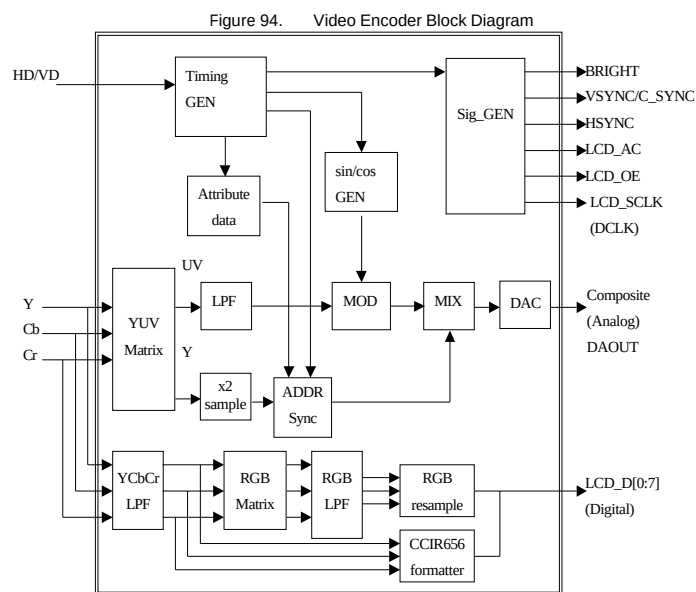
12.12.63 **PPVWIN0ADL - Ping-Pong Video Window 0 Address (Low)**

[illegible][illegible]

13 Video Encoder

13.1 Introduction

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8-bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization. The Video Encoder is shown in the below figure.



13.1.1 Composite Analog Video Encoder features (NTSC/PAL)

- Master Clock Input single 27MHz
- 10-bit D/A Output Interface with output control
- NTSC/PAL Output Selectable
- Line 20 Video Attribute Insertion
- Interface/Non-Interlace selected output
- Setup Selection for NTSC
- VSYNC/ HSYNC Output
- VSYNC/Composite SYNC Output Selectable
- 16-235/0-255 Input Amplitude Selectable
- Video Output Blanking
- Chrominance Low Pass Filter 1.5MHz/3MHz
- Internal Color Bar Generation (100/75%)

13.1.2 Digital RGB/YUV Interface features

- Programmable DCLK out for Digital-RGB Output
- YCbCr or RGB digital output selectable
- 8-bit Digital RGB Output for Digital LCD
- Internal Color Bar Generation (100/75%)
- Digital Low Pass Filter for RGB Output
- Digital VSYNC/HSYNC output
- Programmable HSYNC/VSYNC Width
- Programmable HSYNC/VSYNC Interval
- Programmable VSYNC Output Delay from Input VSYNC at DCLK
- Programmable HSYNC Output Delay from Input HSYNC at DCLK
- Programmable RGB Output Format of ODD/ Even Line (Digital RGB Output)
- Programmable Picture Start Position Horizontal (Digital RGB Output)
- Programmable Picture Start Position Vertical (Digital RGB Output)

- Programmable Picture Valid Length Horizontal (Digital RGB Output)
- Programmable Picture Valid Length Vertical (Digital RGB Output)
- Back Light Control Signal Output

13.2 Video Encoder Output Modes

DM270 video encoder supports the following video output modes.

13.2.1 NTSC/PAL standard mode (VID02:VIMD=00b, DLCD1:DRGBE=0)

This operates as NTSC/PAL encoder. Analog composite video signal and horizontal / vertical sync signals are output from DAOUT terminal, and HSYNC and VSYNC terminals respectively. Composite sync signal can be also output from VSYNC terminal by setting VID02:VSSW. Furthermore, ITU-R BT.656-compliant digital YCbCr output (with SAV/EAV processing) is available (see 13.5.3.4 Mode with SYNC processing).

13.2.2 NTSC/PAL digital extend mode (VID02:VIMD=00b, DLCD1:DRGBE=1)

This is to output not only analog composite but also digital data (RGB or YCbCr) data. Simultaneous usage of analog monitor and digital LCD is available. In this mode, the timings of digital data valid duration and output sync signal are compliant with NTSC/PAL standard, however these timings can be changed by register setting.

13.2.3 Nonstandard mode (VID02:VIMD=10b)

This is a dedicated mode for digital LCD. The sync signals are generated at given timing by register setting and digital data (RGB or YCbCr) is output. Programmable settings for cycle, data valid duration and pulse width are available.

Table 21. Video Encoder Output Modes

VID02[15:14]	DLCD1[15]	Video mode
00	0	NTSC/PAL standard mode
	1	NTSC/PAL digital extend mode
10	-	Nonstandard mode
01 or 11	-	Reserved

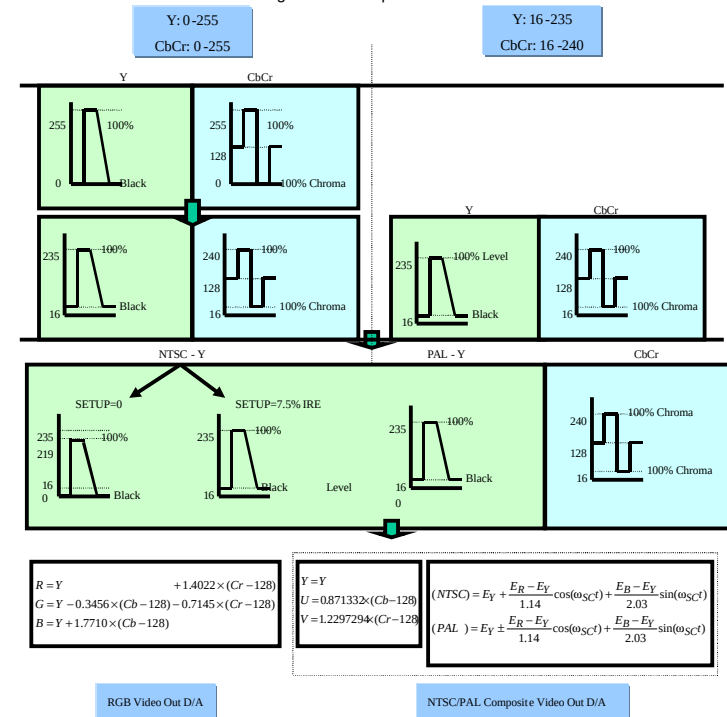
Select appropriate video output mode among above-mentioned three modes for digital/analog output.

13.3 NTSC/PAL Encoder

13.3.1 Signal Level

The NTSC/PAL Encoder accepts digital input signals in YCbCr format. The NTSC/PAL Encoder has a selection register determines the manner in which input signal levels are processed (INLVL in VID02(0x30802)). When the INLVL field value is 0, a value of 16 is processed as black level, and a value of 235 is processed as 100% white level. For chroma signal also, a value of 16 and a value of 240 is processed as 100% chroma level.

Figure 95. Input Video Data



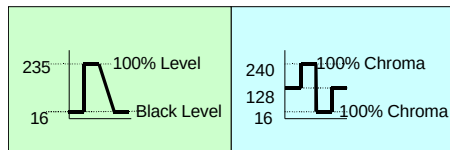
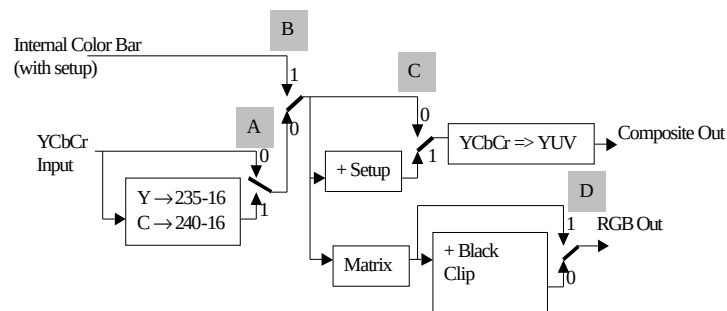


Figure 96. Video Encoder Configuration



13.3.2 Signal calculation

Each RGB or Composite video output is transformed by using following equations.

[RGB output]

$$R = Y + 1.4022 \times (Cr - 128)$$

$$G = Y - 0.3456 \times (Cb - 128) - 0.7145(Cr - 128)$$

$$B = Y + 1.7710 \times (Cb - 128)$$

[Y/C, Composite output]

$$Y = Y$$

$$U = 0.871332 \times (Cb - 128)$$

$$V = 1.2297294 \times (Cr - 128)$$

$$NTSC = E_y + \frac{E_R - E_y}{1.14} \cos(?_{sc}t) + \frac{E_B - E_y}{2.03} \sin(?_{sc}t)$$

$$PAL = E_y + \frac{E_B - E_y}{2.03} \cos(?_{sc}t) + \frac{E_R - E_y}{2.03} \sin(?_{sc}t)$$

13.3.3 Color signal cutoff frequency select

When output is Composite NTSC/PAL, the chroma signal is restricted to the frequency band by the low-pass filter. On the NTSC/PAL encoder, cutoff frequency is selected using the field CRCUT in [VID01\(0x30800\)](#). When CRCUT=0, cutoff frequency is 1.5MHz; When CRCUT=1, cutoff frequency is 3MHz.

13.3.4 Enable Internal Color Bars

The selection at point B in Figure 96 is used to select internal color bars as the input signal. Set CBEN in [VID01\(0x30800\)](#) to enable internal color bars. Two types of color bars can also be selected. Set CBTYP in [VID01\(0x30800\)](#) to select 100% color bars and clear CBTYP to select 75% color bars.

13.3.5 Enable 7.5% Setup

The selection at point C in Figure 3 is used to 7.5% setup for Composite signal. The effect of enabling 7.5% setup is also shown in level 3 of the signal pass diagram, Figure 95. Set SETUP in [VID01](#) to enable 7.5% setup.

13.3.6 Video attribute insertion

The NTSC/PAL encoder has capability to insert video information into the vertical blanking period. For example, the NTSC/PAL encoder can insert a video attribute which indicates the proper aspect ratio to the video receiver. For NTSC mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every field to conform to the EIAJ CPX-1024 Video Aspect Ratio ID specification. Attribute information should be set using [ATR1\(0x3083E\)](#) and [ATR0\(0x3083C\)](#).

registers. The [ATR2\(0x30840\)](#) register should be set with the 6bit CRC data that is calculated by the following equation:

$$G(X) = X^6 + X + 1 \text{ where } X^6, X \text{ are preset to } 1$$

For PAL mode, the NTSC/PAL encoder can insert 14-bit video information on line 20 of every frame to confirm to the ETS300-294 Wide Screen Signaling specification. Attribute information should be set in the [ATR1\(0x3083E\)](#) register and [ATR2\(0x30840\)](#) register.

13.4 Synchronous mode

DM270 video encoder supports two synchronous modes; Master and Slave. This can be used in all analog video modes and digital output modes. (Slave function is supported in ES1.1 or greater.)

13.4.1 Master mode

This operates in synchronization with horizontal / vertical sync signals generated in built-in sync signal generator.

13.4.2 Slave mode

This operates in synchronization with sync signals input from outside. When this mode is used, set [DOUTCTL](#):ESYEN to "1" and register FSEL1:bit 13 of GIO module to "1".

In this case GIO30 and GIO33 are horizontal and vertical sync signal inputs respectively. Also GIO30 bit and GIO33 of GIO module need to be set as input (refer GIO section for details on setting GIO pin as input)

Even though the polarity of input signal is active low by default, it is possible to invert it by DOUTCTL:ESYIV.

In case this mode is operated in NTSC/PAL mode, set DOUTCTL:ESGEN to "1" additionally.

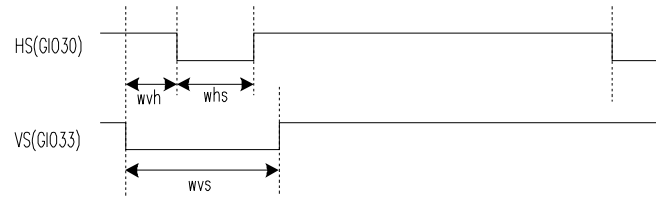
Furthermore, when FSEL1:bit13 is "0", this operates in synchronization with sync signals from CCD controller. At this moment, by setting DOUTCTL:EFEN to "1", field signal processing is conducted taking field signal generated in CCD controller.

DOUTCTL :ESYEN (bit 6)	FSEL1:bit13	Video mode
0	-	Master mode
1	0	Slave mode (CCD controller sync)
	1	Slave mode (External sync using GIO30,GIO33)

Pin Name	Function
GIO30	External horizontal sync signal input
GIO33	External vertical sync signal input

Figure 97 below shows the timing diagram of external signal input in slave operation. The parameters are in units of video encoder input clock (one of SYSCLK, MXI and PCLK).

Figure 97. External sync signal input timing



Parameter	MIN	MAX
whs	Horizontal sync signal pulse width	3 clk
wvs	Vertical sync signal pulse width	3 clk
wvh	Phase difference of horizontal and vertical sync signal s	0 clk

For convenience, DOUTCTL register is shown below

13.4.3 DOUTCTL - Digital Output Control

DOUTCTL																0003:084A		offset: 0x4A																		default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																					
Name	RSV	RSV	DOF[1]	DOF[0]	VCLKS	ESGEN	EFIV	EFEN	ESYIV	ESYEN	Y16O[1]	Y16O[0]	DOIV	DOMD[1]	DOMD[0]	DODIS																					
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																					
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0																					

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	DOF	0	R/W	DCLK Offset » 00: 0.0 « » 01: -0.5 « » 10: 0.5 «
11	VCLKS	0	R/W	VCLK Select » 0: DCLK « » 1: External Clock «
10	ESGEN	0	R/W	External Sync (analog synchronous mode) » 0: disable « » 1: enable «
9	EFIV	0	R/W	External Field Input Inversion » 0: normal « » 1: inverted «
8	EFEN	0	R/W	External Field Input Mode » 0: internal field generation « » 1: external field input «
7	ESYIV	0	R/W	External SYNC Input Inversion » 0: normal « » 1: inverted «
6	ESYEN	0	R/W	External SYNC Input Mode » 0: internal SYNC generation « » 1: external SYNC input «
5:4	Y16O	0	R/W	YCbCr 16-bit Mode Chroma Order » 00: Cb-Cr (based on 13.5MHz) « » 01: Cb-Cr (based on 27MHz) « » 10: Cr-Cb (based on 13.5MHz) « » 11: Cr-Cb (based on 27MHz) «
3	DOIV	0	R/W	Digital Output Inversion » 0: normal « » 1: inverted «
2:1	DOMD	0	R/W	Digital Output Mode » 00: Digital RGB « » 01: YUV 16-bit « » 10: Epson LCD « » 11: Casio LCD «
0	DODIS	0	R/W	Digital Output Disable » 0: Normal « » 1: Fixed LOW «

13.5 Digital RGB/YUV Interface

Select appropriate video output mode among three modes given in Table 21. for digital output.

13.5.1 Digital output mode

DM270 video encoder supports following digital output modes.

- o YCbCr16 Mode:
To output 16-bit YCbCr(4:2:2) in REC601(ITU-R BT.601) format.
- o YCbCr8 Mode:
To output 8-bit YCbCr(4:2:2). This supports REC656(ITU-R BT.656) output.
- o Standard RGB Mode:
To output signals for standard LCD.
- o RGB666 Mode:
To output 18-bit RGB(6:6:6).
- o EPSON:
To output signals for LCD manufactured by EPSON.
- o CASIO:
To output signals for LCD manufactured by CASIO.

Table below shows the output signals in each mode.

Table 22. Digital LCD mode output signals

Pin Name	YCbCr16	YCbCr8	Standard RGB	RGB666	RGB565	EPSON	CASIO
HSYNC	HSYNC	HSYNC	HSYNC	HSYNC	HSYNC	HSYNC	HSYNC
VSYNC	VSYNC	VSYNC	VSYNC	VSYNC	VSYNC	VSYNC	VSYNC
VCLK	13.5/27MHz	27MHz	DCLK	DCLK	DCLK	XSCL	CLK
YOUT7	Y7	Y/C7	Data7	R7	R7	DOUT5	D05
YOUT6	Y6	Y/C6	Data6	R6	R6	DOUT4	D04
YOUT5	Y5	Y/C5	Data5	R5	R5	DOUT3	D03
YOUT4	Y4	Y/C4	Data4	R4	R4	DOUT2	D02
YOUT3	Y3	Y/C3	Data3	R3	R3	DOUT1	D01
YOUT2	Y2	Y/C2	Data2	G7	G7	DOUT0	D00
YOUT1	Y1	Y/C1	Data1	G6	G6	GCP	STBYB
YOUT0	Y0	Y/C0	Data0	G5	G5	LP	RIT
COU7	C7	LCD_AC	LCD_AC	G4	G4	RES	CP
COU6	C6	LCD_OE	LCD_OE	G3	G3	XINH	STH
COU5	C5	BRIGHT	BRIGHT	G2	G2	YSCL	STB
COU4	C4	CSYNC	CSYNC	B7	B7	YSCLD	GPCK
COU3	C3	PWM_OUT	PWM_OUT	B6	B6	FRYS	GRES
COU2	C2	0	0	B5	B5	FRYP	POL
COU1	C1	0	0	B4	B4	FRX	0
COU0	C0	0	0	B3	B3	DY	GSRT
GIO25	-	-	-	R2	-	-	-
GIO27	-	-	-	B2	-	-	-
GIO28	-	-	-	LCD_OE	LCD_OE	-	-

13.5.2 YCbCr16 mode

This is to output 16-bit YCbCr(4:2:2) in REC601(ITU-R BT.601) format. This can be used in all video modes; NTSC/PAL standard mode, NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL](#):DOMD to "01" in addition to video mode setting.

Signal Name	Pin Name	Function
HSYNC	HSYNC	Horizontal sync signal
VSYNC	VSYNC	Vertical sync signal
13.5/27MHz	VCLK	Clock output
Y7-0	YOUT7-0	Y signal output
C7-0	COU7-0	C signal output

13.5.2.1 HSYNC/VSNC

Horizontal / Vertical sync signals. In nonstandard mode, pulse width, HD/VD interval, data valid duration and delay can be set by register [0x30822](#) to [0x30832](#) by the user. In NTSC/PAL digital extend mode, when [VID02](#):SYSW (bit 12) is "0", the timings of data valid duration and output sync signal are compliant

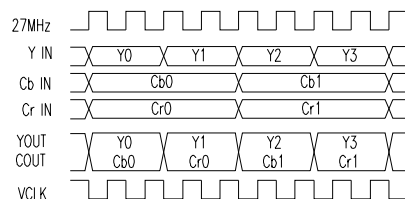
with NTSC/PAL standard. When [VID02](#):SYSW (bit 12) is set to "1", the timing except that of HD/VD interval can be changed.

13.5.2.2 13.5/27MHz Clock output.

When [DOUTCTL](#):VCLKS (bit 11) is set to "1", inverse clock of video encoder input clock is output from VCLK terminal. The polarity is fixed.

13.5.2.3 Output of 16-bit digital YCbCr data.

The data is output format is shown in the below figure.
Figure 98. YCbCr16 mode data output (27MHz)



The order of Cb and Cr is permutable by [DOUTCTL](#):Y16O (bit 5) register.

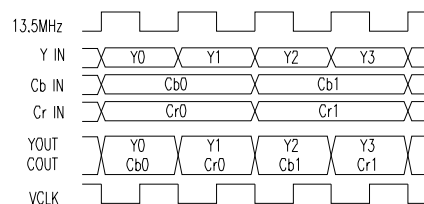
Color-bar output is available by [VID01](#):CBEN (bit 12).

Digital RGB output control by [DLCD2](#):DRGBDE (bit 15) is invalid in YCbCr16 mode.

Setting of low pass filter ([VID01](#)[7], [VID01](#)[13]), for 7.5% setup addition ([VID01](#)[10]), brightness signal and color difference signal can not be conducted. Furthermore, video attribute can not be set.

Input clock of video encoder is set to 27MHz in initial state. When 13.5MHz is used, set [DOUTCTL](#):Y16O to "10" or "00". Figure below shows the data timing in 13.5MHz input.

Figure 99. YCbCr16 mode data output (13.5MHz)



13.5.3 YCbCr8 mode

This is to output 8-bit YCbCr(4:2:2) supporting REC656(ITU-R BT.656) output. This can be used in all video modes; NTSC/PAL standard mode, NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL](#):DOMD to "00" and [REC656](#):RECE (bit 0) to "1" in addition to video mode setting.

Signal Name	Pin Name	Function
HSYNC	HSYNC	Horizontal sync signal
VSNC	VSNC	Vertical sync signal
27MHz	VCLK	Clock output
Y/C7-0	YOUT7-0	YCbCr signal output
LCD_AC	COUT7	AC signal
LCD_OE	COUT6	Data output enable
BRIGHT	COUT5	Brightness control signal
CSYNC	COUT4	Composite sync signal
PWM_OUT	COUT3	PWM signal

13.5.3.1 HSYNC/VSNC

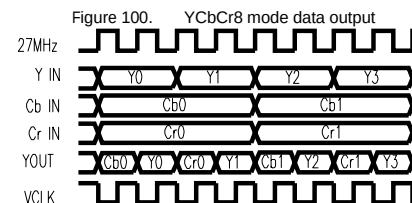
Horizontal / Vertical sync signal s. Same as YCbCr16 mode.

13.5.3.2 27MHz

Clock output. When [DOUTCTL](#):VCLKS (bit 11) is set to "1", inverse clock of video encoder input clock is output from VCLK terminal. The polarity is fixed.

13.5.3.3 Y/C7-0:Output of 8-bit digital YCbCr data.

Figure below shows the reference timing.



In this YCbCr8 mode, the data output mode is selectable between two modes; the mode with SYNC processing of REC656 and without it.

13.5.3.4 Mode with SYNC processing

Setting [REC656:SYNCE](#) (bit 1) to "1" in NTSC/PAL standard mode makes REC656 SYNC processing ON, and insertion of timing reference signals (SAV and EAV) to data stream and blank processing are executed. Lines of 1-10 in each field come to blank (black) signal automatically. Mode with SYNC processing can not be used in NTSC/PAL digital extend mode and nonstandard mode.

13.5.3.5 Mode without SYNC processing

In this case that [REC656:SYNCE](#) (bit 1) is set to "0". SAV/EAV processing and blank processing are never conducted and output data from OSD is output according to the order of REC656:CBCE. When display is needed to be blanked, background color should be "black". The order of Y/Cb/Cr is permutable by REC656:CBCE register. This can be used in all video modes; NTSC/PAL standard mode, NTSC/PAL digital extend mode and nonstandard mode. Except in NTSC/PAL standard mode, image data only within valid display range is output according to the settings of display starting location and display width register ([0x30822](#) to [0x30832](#)). "Black" data is displayed outside of valid display range. The pulse width ([HVPWIDTH](#), 0x30822) and cycle ([HINTERVL](#) (0x30824), [VINTERVL](#) (0x3082A)) can be set only in nonstandard mode.

13.5.3.6 Miscellaneous

The signals for REC656 is output as below.

OSD -> Select [0-255] or [16-235] / Conversion circuit -> REC656 circuit -> Select REC656 or RGB

When REC656 output is used, therefore, inputting to REC656 circuit after adjustment of amplitude with [VID02:INLVL](#) (bit 13) = 1 is recommended. (Do not this confuse with EAV/SAV)

Digital RGB output control by [DLCD2:DRGBDE](#) (bit 15) is valid in YCbCr8 mode. By setting this bit to "1", YOUT[7-0] terminal output is fixed to "HIGH".

Color bar is available by [VID01:CBEN](#) (bit 12). Setting of low pass filter (VID01[7], VID01[13]), for 7.5% setup addition (VID01[10]), brightness signal and color difference signal can not be conducted. Furthermore, video attribute can not be set.

In mode without SYNC processing, data enable LCD_OE is output. When data output and LCD_OE are out-of-sync, set [DLCD1:OEPH](#) (bit 12) to "1" (ES1.1 only).

Also refer to the next section (Standard RGB Mode) for the description of signals output from COUT[7-0] terminal (LCD_AC, LCD_OE, BRIGHT, CSYNC, PWM_OUT).

13.5.4 Standard RGB mode

This is to output the signals for general digital LCD. This can be used in NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL:DOMD](#) to "00" in addition to video mode setting.

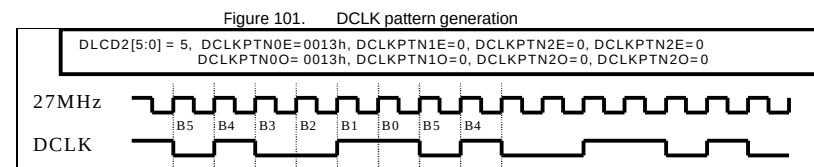
Signal Name	Pin Name	Function
HSYNC	HSYNC	Horizontal sync signal
VSNC	VSNC	Vertical sync signal
DCLK	VCLK	Data Clock
Data7-0	YOUT7-0	RGB digital data
LCD_AC	COUT7	AC signal
LCD_OE	COUT6	Data output enable
BRIGHT	COUT5	Brightness control signal
CSYNC	COUT4	Composite sync signal
PWM_OUT	COUT3	PWM signal

13.5.4.1 HSYNC/VSNC

Horizontal / Vertical sync signals. In NTSC/PAL digital extend mode, pulse width, data valid duration and delay can be set. In nonstandard mode, the cycle interval can be set additionally.

13.5.4.2 DCLK - Data clock.

Given waveform patterns in units of 64 cycle at the maximum can be generated to even and odd lines independently. The patterns are generated by setting bit patterns to [0x30808](#) to [0x3080E](#) registers for even lines, [0x30810](#) to [0x30816](#) is for odd lines and specifying pattern cycles by [DLCD2:PATW](#) register. An example for generation is shown in the below figure.



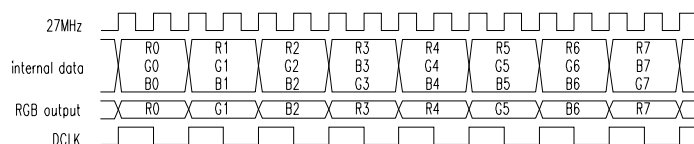
When [DLCD2:DCKME](#) (bit 7) is set to "1", the range outside of valid range of clock can be masked. When mask is used, specify the clock starting location for horizontal and vertical directions and clock valid range via [0x30818](#) to [0x30820](#) registers. In addition, starting location in horizontal direction can be set to even and odd lines independently.

Furthermore, the polarity of output clock can be inverted by [DLCD2:DCKP](#) (bit 6) register.

13.5.4.3 Data Output Format

8-bit RGB output data. RGB data is output in synchronization with rising edge of DCLK as shown in the figure below.

Figure 102. RGB Output Data



There are six types of RGB data output order. RGB output order can be set to even and odd lines independently. This setting is done [DLCD1:EVNS](#) and [DLCD1:ODDS](#) fields.

Register Field value	RGB order
0	R0-G1-B2
1	R0-B1-G2
2	G0-R1-B2
3	G0-B1-R2
4	B0-R1-G2
5	B0-G1-R2

Since RGB data is usually output in synchronization with rising of DCLK, DCLK frequency and picture rate are same. By setting [DLCD2:HFDCK](#) (bit 8) to "1", this comes to half rate mode and can output data at half rate of DCLK as shown in the figure below.

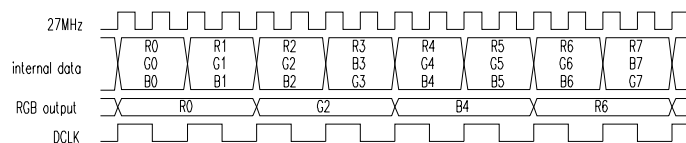
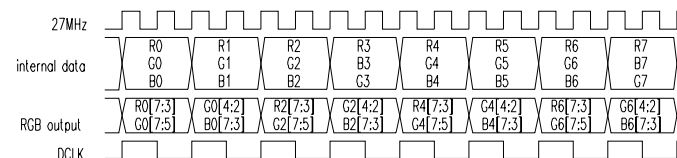


Figure 103. Half Rate Mode

IronMan mode is set by setting [DLCD1:IRONM](#) (bit 8) to "1" and RGB data is output in 5:6:5 format as shown in the below figure.

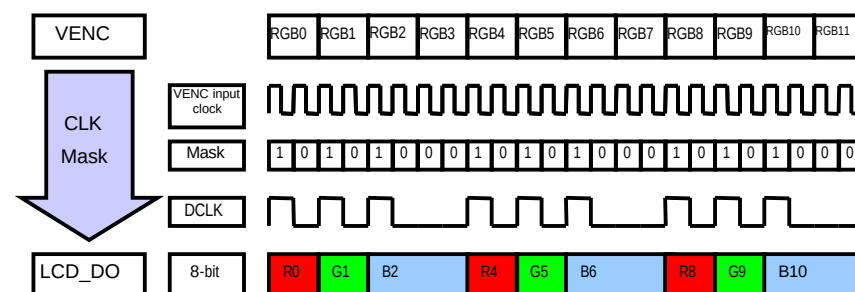
Figure 104. IronMan Mode



13.5.4.4 DCLK Masking and Output Format

Given below is a figure shows effect on data output format when used along with with DCLK masking. Here pattern width = 8 (PATW=7) and DCLK pattern = 10101000b (DCLKPTN0E = 0xA8, DCLKPTN0O = 0xA8, DCLKPTNxx = 0)

Figure 105. DCLK Masking and Data Output



13.5.4.5 LCD_AC

Optional AC signal. Use depending on LCD being interfaced. This is toggle-inverted signal per given field of from 1 to 8. Interval of switching is specified by [DLCD2:DACL](#) field. Set [DLCD2:ACE](#) (bit 9) to "1" to enable output of AC signal.

13.5.4.6 LCD_OE

Data output enable signal. This is asserted "HIGH" during data valid duration specified by sync signal valid duration registers; [0x30826](#), [0x30828](#), [0x3082C](#) and [0x3082E](#). The polarity is invertible by [DLCD2:DOEP](#) (bit 13) register. Set [DLCD2:DOE](#) (bit 14) to "1" when this signal is used.

13.5.4.7 BRIGHT

Optional brightness control signal. Use depending on LCD being interfaced. This outputs a pulse for a certain period from falling of horizontal sync signal. The polarity is invertible by [VID02:BRPL](#) (bit 8). The pulse width is specified by [VID02:BRWIDTH](#) field. Note that pulse width is counted by 2-dividing clock of video encoder input clock. Set [VID02:BREN](#) (bit 9) to "1" to enable this output of this signal.

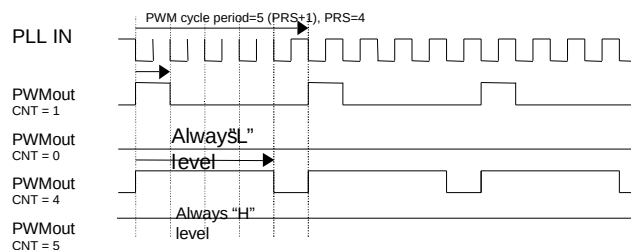
13.5.4.8 CSYNC

NTSC/PAL composite sync signal. This is invalid in nonstandard mode.

13.5.4.9 PWM_OUT

Optional PWM signal. Use depending on LCD being interfaced. The cycle and pulse width of PWM signal are programmable and set by [PWMCTRL:PRS](#) and [PWMCNT:CNT](#) register respectively. The polarity is invertible by [PWMCTRL:PWMP](#) (bit 14). Set [PWMCTRL:PWMP](#) (bit 15) to "1" when this signal is used.

Figure 106. Pulse Width Modulation



13.5.5 RGB666 mode

This is to output 18-bit RGB(6:6:6) parallel data. This can be used in NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL:DOMD](#) to "00" and [REC656:RGB565](#) (bit 5) to "1" separately from video mode setting.

Signal Name	Pin Name	Function
HSYNC	HSYNC	Horizontal sync signal
VSYSN	VSYSN	Vertical sync signal
DCLK	VCLK	Data clock
R[7]	YOUT7	R[7]
R[6]	YOUT6	R[6]
R[5]	YOUT5	R[5]
R[4]	YOUT4	R[4]
R[3]	YOUT3	R[3]
G[7]	YOUT2	G[7]
G[6]	YOUT1	G[6]
G[5]	YOUT0	G[5]
G[4]	COU7	G[4]
G[3]	COU6	G[3]
G[2]	COU5	G[2]
B[7]	COU4	B[7]
B[6]	COU3	B[6]
B[5]	COU2	B[5]
B[4]	COU1	B[4]
B[3]	COU0	B[3]
R[2]	GIO25	R[2]
B[2]	GIO27	B[2]
LCD_OE	GIO28	Data output enable

13.5.5.1 HSYNC/VSYSN

Horizontal / Vertical sync signal s. Same operation as in Standard RGB mode.

13.5.5.2 DCLK

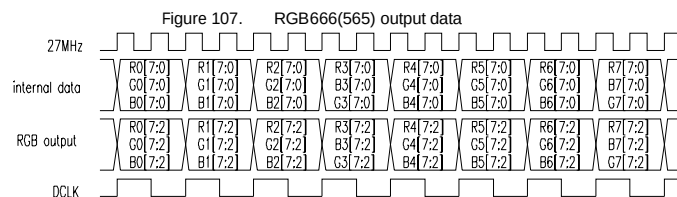
Data clock of LCD. Same operation as in Standard RGB mode.

13.5.5.3 RGB Output Data Format

RGB parallel data output. Data is output in synchronization with rising of DCLK as shown in the figure below. 16-bit RGB(5:6:5) data is output from YOUT and COUT pins.

To output 18-bit RGB(6:6:6), set register FSEL1.bit14 of GIO module to "1". In this case R[2] and B[2] can be output from GIO25 pin and GIO27 pin respectively.

Also GIO25 pin and GIO27 pin need to be set as output direction. Refer GIO section for setting GIO in output direction



13.5.5.4 LCD_OE

Data output enable signal. Same operation as in Standard RGB mode. Set GIO module, register FSEL1.bit 5 to "1" when this signal is used. In this case GIO28 should be set in output direction

13.5.6 Horizontal, Vertical Timing for YCbCr16, YCbCr8, RGB mode

Figure below shows Horizontal and Vertical timing for YCbCr16, YCbCr 8, and each RGB mode.

As shown in Table below, in YcbCr16 and YcbCr8 mode, there are an invalid output signals.

Figure 108. YCbCr16, YCbCr 8, and each RGB mode timing diagram

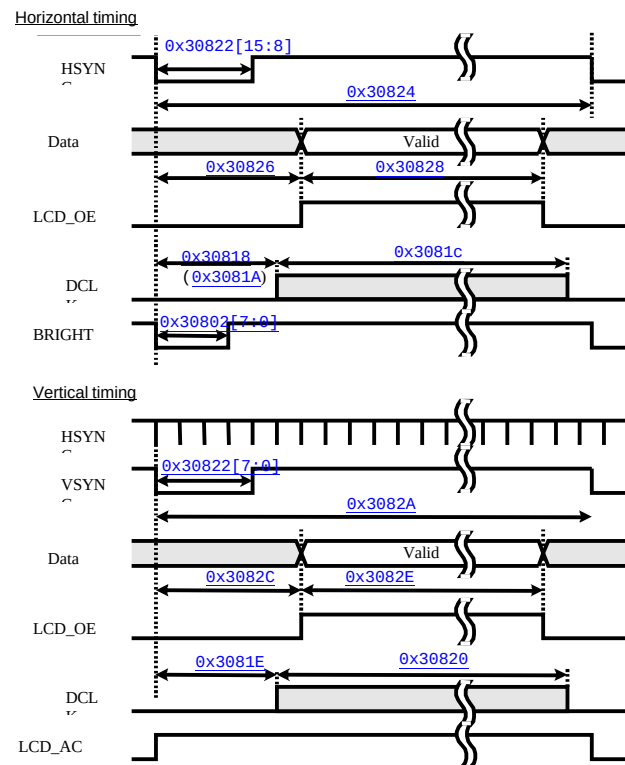


Table 23. Output signals in YcbCr16 and YcbCr8 mode

Mode Output signal	YCbCr16	YcbCr8(with SYNC)	YcbCr8(without SYNC)
HSYNC	O	X	O
LCD_OE	X	X	O
DCLK	13.5/27MHz	27MHz	27MHz
BRIGHT	X	O	O
VSYNC	O	X	O
LCD_AC	X	O	O
Note, X = signal invalid, O = signal valid			

13.5.7 EPSON mode

This mode is to output the signals for LCD manufactured by EPSON. This can be used in NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL](#):DOMD to "10" separately from video mode setting.

Signal Name	Pin Name
HSYNC	HSYNC
VSYNC	VSYNC
XSCL	VCLK
DOUT5-0	YOUT7-2
GCP	YOUT1
LP	YOUT0
RES	COU7
XINH	COU6
YSCL	COU5
YSCLD	COU4
FRYS	COU3
FRYP	COU2
FRX	COU1
DY	COU0

(refer to EPSON's data sheet for function of each signal)

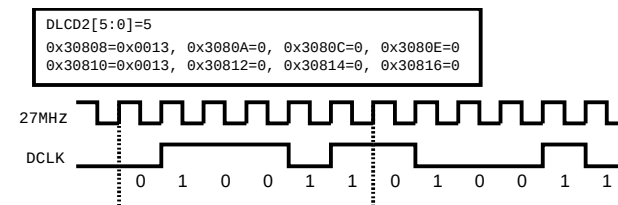
13.5.7.1 HSYNC/VSNC

Horizontal / Vertical sync signals. Same operation as in Standard RGB mode.

13.5.7.2 XSCL

This corresponds to data clock of general LCD. Generate the clock suitable for LCD in use by DCLK register. In addition, in this EPSON mode, setting method of DCLK pattern setting register ([0x30808-0x30816](#)) differs from others and NRZI encoded clock is output. An example for generation is shown in the figure below.

Figure 109. DCLK pattern generation in EPSON mode



Even though the DCLK pattern is set to switch per 1-line by default, switch per 3-line can be done by setting [EPSON_LCD:EP18DBT](#) (bit 4) to "1". This setting is for 18DBT-type LCD.

13.5.7.3 DOUT5-0

RGB data is output in synchronization with rising of DCLK similar to data output in Standard RGB mode. 6-bit RGB data is however output by dropping the lower 2 bits.

13.5.7.4 GCP

GCP pulse output. An output pattern can be selected from four types of ROM tables by [EPSON_LCD:GCPSL](#) register. Furthermore, by setting [EPSON_LCD:GCPRU](#) (bit 2) to "1", a given pattern read in internal RAM can be output.

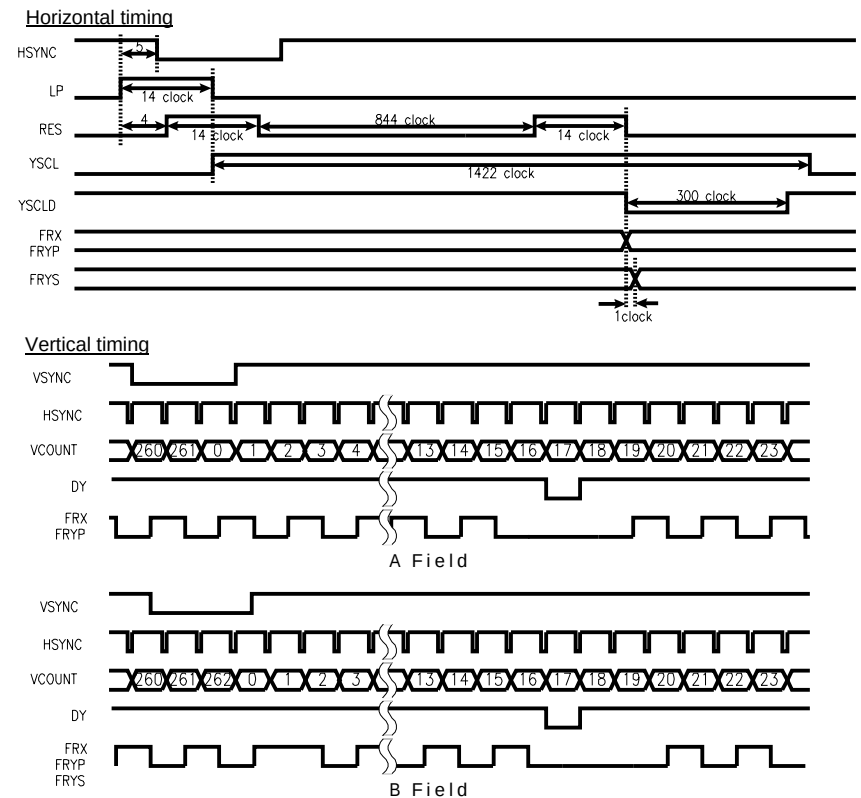
13.5.7.5 XINH

Fixed "HIGH" output.

13.5.7.6 Miscellaneous

Refer to figure below for the timing. Also it is possible to put reset timing of vertical direction in B Field ahead 1H by setting [EPSON_LCD:EPSYSC](#) (bit 10) to "1".

Figure 110. EPSON mode timing diagram



13.5.8 CASIO mode

This is to output the signals for LCD manufactured by CASIO. This can be used in NTSC/PAL digital extend mode and nonstandard mode. In case that this mode is used, set [DOUTCTL:DOMD](#) to "11" separately from video mode setting.

In addition, CASIO LCD has 3 types of timing specs: standard, NTSC and PAL. Select these modes by [CASIO:CLS\[1:0\]](#). (Mode setting by [CASIO:CLS\[1:0\]](#) is needed in only DM270 ES1.0. A mode is determined automatically by video mode setting in ES1.1. In addition, set [CASIO:CLS\[1:0\]](#) to "01" in ES1.1.)

Signal Name	Pin Name	Function
HSYNC	HSYNC	Horizontal sync signal
VSYNC	VSYNC	Vertical sync signal
CLK	VCLK	Data clock
D05-00	YOUT7-2	RGB digital data
STBYB	YOUT1	Source standby control signal
RIT	YOUT0	Horizontal flip control signal
CP	COUT7	Clock signal for charge pump
STH	COUT6	Sampling starting signal
STB	COUT5	Source drive control signal
GPCK	COUT4	Gate shift clock
GRES	COUT3	Gate driver control signal
POL	COUT2	Polarity selecting signal
GSRT	COUT0	Gate starting signal

13.5.8.1 HSYNC/VSYNC

Horizontal / Vertical sync signals. Same operation as in Standard RGB mode. When CASIO Standard mode is used, set the cycle and others in nonstandard mode.

13.5.8.2 CLK

6.75MHz clock. Output the 4-dividing clock of 27MHz by DCLK setting register.

13.5.8.3 D05-00

RGB data is output in synchronization with rising of DCLK similar to data output in Standard RGB mode. 6-bit RGB data is however output by dropping the lower 2 bits.

13.5.8.4 STBYB

The value of [CASIO:CTSB](#) (bit 2) is output. When CASIO:CTSB (bit 2) is set to "1", CP output is fixed "HIGH".

13.5.8.5 RIT

The value of [CASIO:CRIT](#) (bit 2) is output in synchronization with falling of horizontal sync signal.

13.5.8.6 Miscellaneous

The timings in NTSC and PAL modes are shown in the figures below.

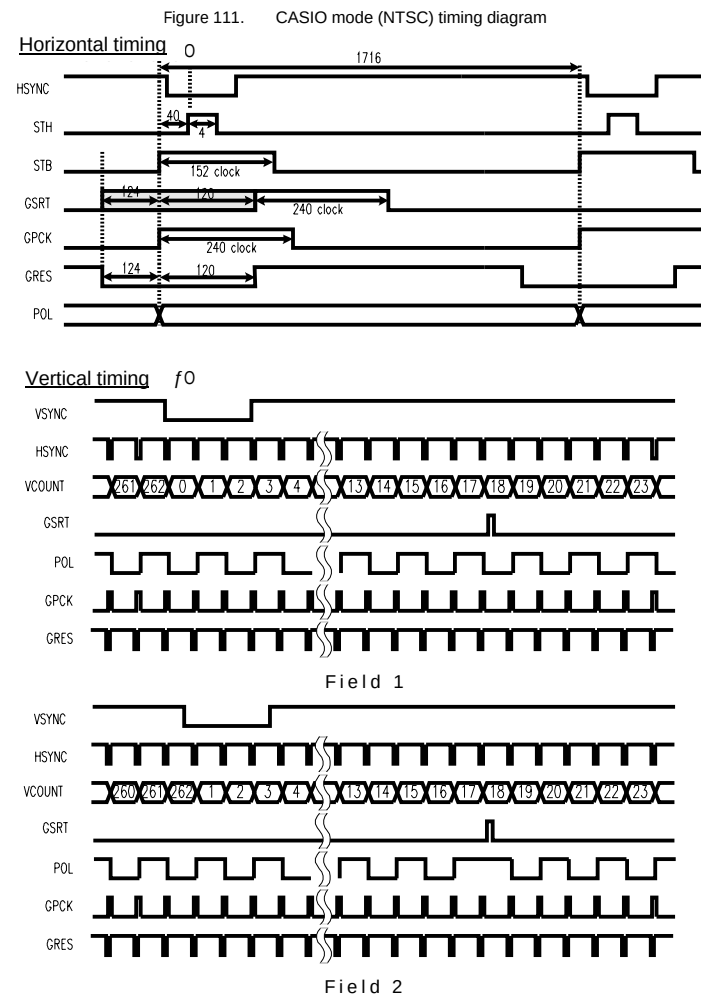
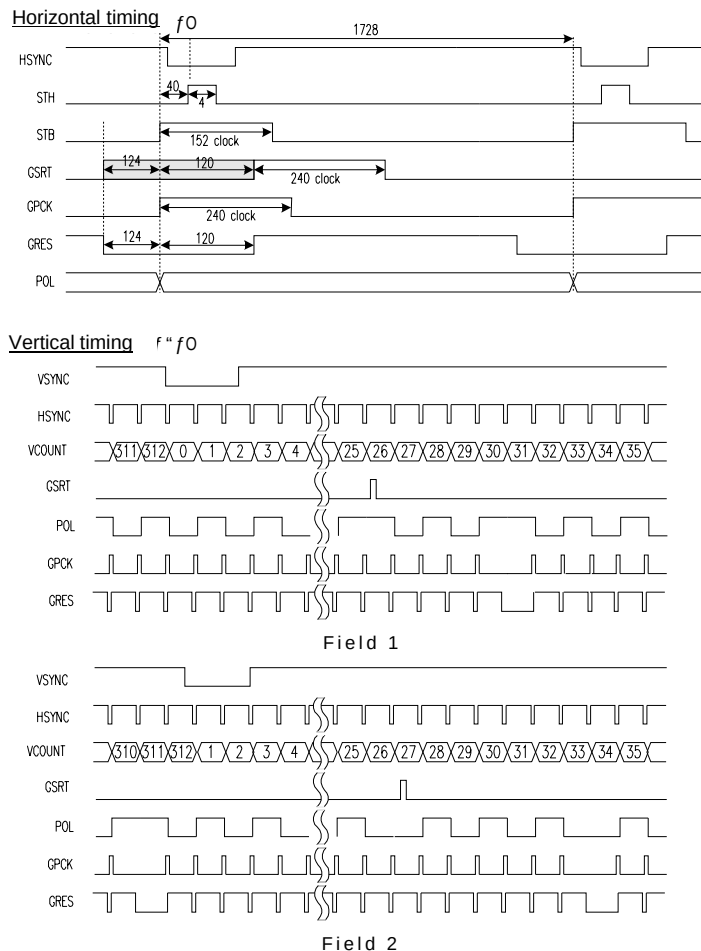


Figure 112. CASIO mode (PAL) timing diagram



13.6 Video Encoder Register Map (VENC)

Address	Register	Description
0003:0800	VID01	Video Mode Setup #1
0003:0802	VID02	Video Mode Setup #2
0003:0804	DLCD1	Digital LCD Mode Setup #1
0003:0806	DLCD2	Digital LCD Mode Setup #2
0003:0808	DCLKPTN0E	DCLK Pattern Register #0 - EVEN Line
0003:080A	DCLKPTN1E	DCLK Pattern Register #1 - EVEN Line
0003:080C	DCLKPTN2E	DCLK Pattern Register #2 - EVEN Line
0003:080E	DCLKPTN3E	DCLK Pattern Register #3 - EVEN Line
0003:0810	DCLKPTN0O	DCLK Pattern Register #0 - ODD Line
0003:0812	DCLKPTN1O	DCLK Pattern Register #1 - ODD Line
0003:0814	DCLKPTN2O	DCLK Pattern Register #2 - ODD Line
0003:0816	DCLKPTN3O	DCLK Pattern Register #3 - ODD Line
0003:0818	DCLKSTPHE	DCLK Enable Start Position in Horizontal - Even Line
0003:081A	DCLKSTPHO	DCLK Enable Start Position in Horizontal - Odd line
0003:081C	DCLKVLDH	DCLK Valid Area in Horizontal
0003:081E	DCLKSTPV	DCLK Enable Start Position in Vertical
0003:0820	DCLKVLDV	DCLK Valid Area in Vertical
0003:0822	HVPW DTH	Horizontal and Vertical Sync Pulse Width Control
0003:0824	HINTERVL	Horizontal Sync Interval Control
0003:0826	HSTART	Horizontal Start Position Control
0003:0828	HVALID	Horizontal Valid Area Control
0003:082A	VINTERVL	Vertical Sync Interval Control
0003:082C	VSTART	Vertical Start Position Control
0003:082E	VVALID	Vertical Valid Area Control
0003:0830	HDELAY	Horizontal Sync Delay
0003:0832	VDELAY	Vertical Sync Delay
0003:0834	CULLLINE	Line Culling Control
0003:0836	PWMCTRL	PWM Control
0003:0838	PWMCNT	PWM Count
0003:083A	RGBLEVEL	RGB Level Control
0003:083C	ATRO	Video Attribute Data #0
0003:083E	ATRI	Video Attribute Data #1
0003:0840	ATR2	Video Attribute Data #2
0003:0842	REC656	CCIR 656 Mode Setup
0003:0844	EPSON_LCD	EPSON LCD Control
0003:0846	GCPDATA	GCPDATA
0003:0848	CASIO	CASIO LCD Control
0003:084A	DOUTCTL	Digital Output Control
0003:084C	SCHSYNP	Sync Period

13.7 Video Encoder Interface Registers

13.7.1 VID01 - Video Mode Setup #1

VID01		0003:0800		offset: 0x00		default: 0x0000														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	NTPLS	SCMD	CRCUT	CBEN	CBTYP	SETUP	RGBFLT[1]	RGBFLT[0]	YFLT	DAPD	DAOE	RSV	RSV	VOUT	BLANK	VENC				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	-	-	0	0	0				

Bit	Name	Reset Value	R/W	Function
15	NTPLS	0	R/W	<u>NTSC/PAL Select</u> » 0: NTSC « » 1: PAL «
14	SCMD	0	R/W	<u>Scan Mode Select</u> » 0: Interlace « » 1: Non-interlace «
13	CRCUT	0	R/W	<u>Chroma Low-Pass Filter Select (Composite)</u> » 0: 1.5MHz cutoff « » 1: 1.3MHz cutoff «
12	CBEN	0	R/W	<u>Color Bars Enable</u> » 0: Normal « » 1: Enable built-in color bars «
11	CBTYP	0	R/W	<u>Color Bar Type</u> » 0: 75% « » 1: 100% «
10	SETUP	0	R/W	<u>Setup Select (Composite)</u> » 0: 0% setup « » 1: 7.5% setup «
9:8	RGBFLT	0	R/W	<u>RGB Low-Pass Filter (Horizontal)</u> » 00: Off « » 01: 3-tap LPF « » 10: 5-tap LPF « » 11: Off «
7	YFLT	0	R/W	<u>Y (luma) low Pass Filter (Horizontal)</u> » 0: Off « » 1: 3-tap LPF «
6	DAPD	0	R/W	<u>DAC Power Down</u> <u>In ES1.0</u> » 0: normal « » 1: powerdown « <u>In ES1.1</u> » 0: powerdown « » 1: normal «
5	DAOE	0	R/W	<u>DAC Output Enable</u> » 0: disable « » 1: enable «
4:3	RSV			<u>Reserved</u>
2	VOUT	0	R/W	<u>Video Output Enable (Analog Composite signals)</u> » 0: Disable analog video signals « » 1: Enable analog video signals « Use register DLC01 to enable/disable Digital LCD signals
1	BLANK	0	R/W	<u>External Sync Select</u> » 0: Normal output of the Y and Composite signals « » 1: Output only the sync signal, without video, at the Y and Composite output pins « When BLANK is set to 1, the video component of the Y and Composite signals is forced to black level. Thus, only the sync signal is output at the Y and Composite output pins.
0	VENC	0	R/W	<u>Video Encoder Enable</u> » 0: Disable « » 1: Enable «

13.7.2 VID02 - Video Mode Setup #2

VID02		0003:0802		offset: 0x02		default: 0x0000														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	VIMD[1]	VIMD[0]	INLVL	SYSW	VSSW	SYNE	BREN	BRPL	BRWDTH[7]	BRWDTH[6]	BRWDTH[5]	BRWDTH[4]	BRWDTH[3]	BRWDTH[2]	BRWDTH[1]	BRWDTH[0]				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:14	VIMD	0	R/W	<u>Video Mode Select</u> » 00: Composite « » 01: Reserved « » 10: Digital RGB « » 11: Reserved «
13	INLVL	0	R/W	<u>Input Signal Range Select</u> » 0: 16-253 « » 1: 0-255 «
12	SYSW	0	R/W	<u>Analog/Digital SYNC Output Select</u> » 0: Analog SYNC « » 1: Digital SYNC « Use SYSEL to output analog or digital sync pulses at the pins named VSYNC/C_SYNC and HSYNC.
11	VSSW	0	R/W	<u>VSYNC/C_SYNC Output Select</u> » 0: Vertical sync, VSYNC « » 1: Composite sync, C_SYNC «
10	SYNE	0	R/W	<u>SYNC Enable</u> » 0: Disable « » 1: Enable « Use SYEN to enable/disable the pins named VSYNC/C_SYNC and HSYNC.
9	BREN	0	R/W	<u>BRIGHT Enable</u> » 0: Disable « » 1: Enable «
8	BRPL	0	R/W	<u>BRIGHT Polarity:</u> » 0: Normal « » 1: Reverse polarity of BRIGHT «
7:0	BRWDTH	0	R/W	<u>BRIGHT Signal Width Control</u> Use BRWDTH to specify the width of the BRIGHT signal. BRIGHT width (in seconds) = BRWDTH*(2/CLK_VENC), where CLK_VENC is the frequency of the clock attached to Video Encoder.

13.7.3 DLCD1 - Digital LCD Mode Setup #1

DLCD1	0003:0804				offset: 0x04				default: 0x6000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DRGBE	VPL	HPL	OEPH	LRCE	EXLID	EXIDP	IRONM	RGBF	LINID	EVNS[2]	EVNS[1]	EVNS[0]	ODDS[2]	ODDS[1]	ODDS[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	DRGBE	0	R/W	<u>Digital RGB Enable</u> » 0: Disable « » 1: Enable «
14	VPL	1	R/W	<u>VSYNC Polarity</u> » 0: Active low « » 1: Active high « In ES1.0, default value is 0 In ES1.1, default value is 1
13	HPL	1	R/W	<u>HSYNC Polarity</u> » 0: Active low « » 1: Active high « In ES1.0, default value is 0 In ES1.1, default value is 1
12	OEPH	0	R/W	<u>Change timing of LCD OE assert.</u> Used for REC601 non-sync output mode. As ES1.0 has LCD_OE assert timing delay for REC656 non-sync mode. This bit can be used to avoid this problem. » 0: same as ES1.0 « » 1: 1cycle(27MHz) first « Note: This bit is reserved in ES1.0 and is available in ES1.1 and higher
11	LRCE	0	R/W	<u>Line Reverse Control Enable</u> » 0: Disable « » 1: Enable « Used when displaying 288 line PAL signal on an LCD display with 240 lines. Output lines to LCD must be culled in 5/6 ratio but color sequence depends on ODD/EVEN line on LCD panel. If lines are culled from original color sequence to 240 lines, then the color sequence changes every 6 lines. It is possible to control ODD/EVEN line identification reverse by this bit set "1".
10	EXLID	0	R/W	<u>External Line ID Enable</u> » 0: Ignore the External Line ID (EXT_LINE_ID) « » 1: Enable use of External ID (EXT_LINE_ID) « The External Line ID is input to the DM310 from an LCD panel on the pin named EXT_LINE_ID.
9	EXIDP	0	R/W	<u>External Line ID Polarity</u> » 0: Normal « » 1: Reverse polarity of EXT_LINE_ID «

Continued on next page ...

8	IRONM	0	R/W	<u>RGB565 Iron Man Enable</u> » 0: Disable « » 1: Enable «
7	RGBF	0	R/W	<u>RGB Line Cull</u> » 0: Normal « » 1: Remove 1 of every 6 lines «
6	LINID	0	R/W	<u>Line ID</u> » 0: Normal « » 1: Different « Controls whether odd/even line ID is same/different for odd/even-field selection
5:3	EVNS	0	R/W	<u>Digital RGB Output Color Sequence for Even Line</u> » 000: R0-G1-B2 « » 001: R0-B1-G2 « » 010: G0-R1-B2 « » 011: G0-B1-R2 « » 100: B0-R1-G2 « » 101: B0-G1-R2 «
2:0	ODDS	0	R/W	<u>Digital RGB Output Color Sequence for Odd Line</u> » 000: R0-G1-B2 « » 001: R0-B1-G2 « » 010: G0-R1-B2 « » 011: G0-B1-R2 « » 100: B0-R1-G2 « » 101: B0-G1-R2 «

13.7.4 DLCD2 - Digital LCD Mode Setup #2

[illegible]

Bit	Name	Reset Value	R/W	Function
15	DRGBDE	0	R/W	<u>Digital RGB Data Output Port Enable</u> » 0: Enable LCD_D0[7:0] « » 1: Disable LCD_D0[7:0] «
14	DOE	0	R/W	<u>LCD OE Enable</u> » 0: Disable « » 1: Enable «
13	DOEP	0	R/W	<u>LCD OE Polarity</u> » 0: Normal « » 1: Reverse polarity of LCD_OE «
12:10	DACI	0	R/W	<u>LCD AC Switching Interval</u> The frequency of LCD_AC is equal to (DACI + 1)*(VSYNC).
9	ACE	0	R/W	<u>LCD AC Signal Generator Control</u> » 0: Disable « » 1: Enable «
8	HFDCK	0	R/W	<u>Half Rate DCLK Mode Enable</u> » 0: Normal « » 1: Enable Half Rate Mode « When operating in normal mode, an 8-bit digital RGB data value is output during every LCD_SCLK cycle. When Half Rate Mode is enabled, an 8-bit digital RGB data value is output during two LCD_SCLK cycles.
7	DKME	0	R/W	<u>LCD SCLK Mask Enable</u> » 0: Disable Digital Clock (LCD_SCLK) masking « » 1: Enable Digital Clock (LCD_SCLK) masking « When DKME=1, the registers DCLKSTPHx(V), DCLKVLDH(V) are used to control the DCLK valid/invalid area. When DKME=0, the value set in registers DCLKSTPHx(V), DCLKVLD(V) are ignored and DCLK is always valid.
6	DCKP	0	R/W	<u>LCD SCLK Polarity</u> » 0: Normal « » 1: Reverse polarity of LCD_SCLK «
5:0	PATW	0	R/W	<u>Digital Clock (LCD SCLK) Mask Width</u> PATW + 1 is the width of the Digital Clock mask. The mask width can range from 1 to 64. The mask is specified in the Digital Clock Pattern Registers, which are DCLKPTN0, DCLKPTN1, DCLKPTN2, and DCLKPTN3. For example, If PATW = 15, then mask is 16-bits: DCLKPTN0(15-0) If PATW = 35, then mask is 36-bits: DCLKPTN2(3-0), DCLKPTN1(15-0), DCLKPTN0(15-0)

13.7.5 DCLKPTN0E - DCLK Pattern Register #0 - EVEN Line

[illegible][illegible]

13.7.6 DCLKPTN1E - DCLK Pattern Register #1 - EVEN Line

[illegible][illegible]

13.7.7 DCLKPTN2E - DCLK Pattern Register #2 - EVEN Line

[illegible][illegible]

13.7.8 DCLKPTN3E - DCLK Pattern Register #3 - EVEN Line

[illegible][illegible]

13.7.9 DCLKPTN0O - DCLK Pattern Register #0 - ODD Line

[illegible][illegible]

13.7.10 DCLKPTN1O - DCLK Pattern Register #1 - ODD Line

[illegible][illegible]

13.7.11 DCLKPTN20 - DCLK Pattern Register #2 - ODD Line

[illegible][illegible]

13.7.12 DCLKPTN3O - DCLK Pattern Register #3 - ODD Line

[illegible][illegible]

13.7.13 DCLKSTPHE - DCLK Enable Start Position in Horizontal - Even Line

[illegible][illegible]

13.7.14 DCLKSTPHO - DCLK Enable Start Position in Horizontal - Odd line

[illegible][illegible]

13.7.15 DCLKVLDH - DCLK Valid Area in Horizontal

[illegible][illegible]

13.7.16 DCLKSTPV - DCLK Enable Start Position in Vertical

[illegible][illegible]

13.7.17 DCLKVLDV - DCLK Valid Area in Vertical

[illegible][illegible]

13.7.18 HVPWIDTH - Horizontal and Vertical Sync Pulse Width Control

[illegible][illegible]

13.7.19 HINTERVL - Horizontal Sync Interval Control

[illegible][illegible]

13.7.20 HSTART - Horizontal Start Position Control

HSTART		0003.0826				offset:		0x26								default:		0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	HSTP[11]	HSTP[10]	HSTP[9]	HSTP[8]	HSTP[7]	HSTP[6]	HSTP[5]	HSTP[4]	HSTP[3]	HSTP[2]	HSTP[1]	HSTP[0]					
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0					

[illegible]

13.7.21 HVALID - Horizontal Valid Area Control

[illegible][illegible]

13.7.22 VINTERVL - Vertical Sync Interval Control

[illegible][illegible]

13.7.23 VSTART - Vertical Start Position Control

VSTART	0003:082C				offset: 0x2C		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	VSTP[11]	VSTP[10]	VSTP[9]	VSTP[8]	VSTP[7]	VSTP[6]	VSTP[5]	VSTP[4]	VSTP[3]	VSTP[2]	VSTP[1]	VSTP[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	VSTP	0	R/W	<u>Start Position of Valid Data in Vertical</u> The start position is measured from the start of VSYNC. The units are in number of HSYNC pulses.

13.7.24 VVALID - Vertical Valid Area Control

VVALID	0003:082E				offset: 0x2E		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	VVLD[11]	VVLD[10]	VVLD[9]	VVLD[8]	VVLD[7]	VVLD[6]	VVLD[5]	VVLD[4]	VVLD[3]	VVLD[2]	VVLD[1]	VVLD[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	VVLD	0	R/W	<u>Period of Valid Data in Vertical</u> The period is measured from the start position of valid data in the vertical direction. The units are in number of HSYNC pulses.

13.7.25 HDELAY - Horizontal Sync Delay

HDELAY	0003:0830				offset: 0x30		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	HDLY[11]	HDLY[10]	HDLY[9]	HDLY[8]	HDLY[7]	HDLY[6]	HDLY[5]	HDLY[4]	HDLY[3]	HDLY[2]	HDLY[1]	HDLY[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	HDLY	0	R/W	<u>Specifies the delay of HSYNC</u> HSYNC delay specified from the start of the DM310's internal HSYNC pulse and in units of CLK_VENC/2 cycles.

13.7.26 VDELAY - Vertical Sync Delay

VDELAY	0003:0832				offset: 0x32		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	VDLY[11]	VDLY[10]	VDLY[9]	VDLY[8]	VDLY[7]	VDLY[6]	VDLY[5]	VDLY[4]	VDLY[3]	VDLY[2]	VDLY[1]	VDLY[0]		
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	VDLY	0	R/W	<u>Specifies the delay of VSYNC</u> VSYNC delay specified from the start of the DM310's internal VSYNC pulse and in units of HSYNC pulses.

13.7.27 CULLLINE - Line Culling Control

[illegible][illegible]

13.7.28 PWMCTRL - PWM Control

[illegible][illegible]

13.7.29 PWMCNT - PWM Count

[illegible][illegible]

13.7.30 RGBLEVEL - RGB Level Control

RGBLEVEL		0003.083A				offset:		0x3A								default: 0xE820			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	CLIP[7]	CLIP[6]	CLIP[5]	CLIP[4]	CLIP[3]	CLIP[2]	CLIP[1]	CLIP[0]	RSV	RSV	BLK[5]	BLK[4]	BLK[3]	BLK[2]	BLK[1]	BLK[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	1	1	1	0	1	0	1	1	-	-	1	0	0	0	0	0			

[illegible]

13.7.31 ATR0 - Video Attribute Data #0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ATRO[7]	ATRO[6]	ATRO[5]	ATRO[4]	ATRO[3]	ATRO[2]	ATRO[1]	ATRO[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

[illegible]

13.7.32 ATR1 - Video Attribute Data #1

ATR1		0003:083E								offset: 0x3E		default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ATR1[7]	ATR1[6]	ATR1[5]	ATR1[4]	ATR1[3]	ATR1[2]	ATR1[1]	ATR1[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	

[illegible]

13.7.33 ATR2 - Video Attribute Data #2

[illegible][illegible]

13.7.34 REC656 - CCIR 656 Mode Setup

REC656	0003:0842												offset: 0x42				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	FIDPOL	RGB565	VOUTHLD	CBCR[1]	CBCR[0]	SYNCE	RECE				
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6	FIDPOL	0	R/W	Field ID Polarity - polarity of the BRIGHT terminal » 0: Normal « » 1: Reversed «
5	RGB565	0	R/W	RGB565/666 mode 0: 8bit output with DOMD[1:0]=00 1: RGB565 16bit/18bit mode with DOMD[1:0]=00 Changed to RGB parallel output mode select bit since ES1.1. Under the condition that DOUTCTL register (0x3084A) DOMD[1:0](bit[2:1])=00 and this bit is 1, RGB565/666 16bits/18bits output is enabled. Note: In ES1.0 this bit is reserved
4	VOUTHLD	0	R/W	Digital Output HOLD function » 1: Video output stopped at end of current line « » 0: Video output starts from start of next line «
3:2	CBCR	0	R/W	The output order of Y/Cb/Cr is specified. » 00: Cb-Y-Cr-Y « » 01: Y-Cr-Y-Cb « » 10: Cr-Y-Cb-Y « » 11: Y-Cb-Y-Cr «
1	SYNCE	0	R/W	SYNC process enable » 0: no SYNC processing « » 1: SYNC processing (REC656) «
0	RECE	0	R/W	REC656 Mode Select » 0: normal RGB out « » 1: REC656 output mode «

13.7.35 EPSON_LCD - EPSON LCD Control

EPSON_LCD				0003:0844				offset: 0x44								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	EPVM	EPVSYSC	EPRGBR	RSV	RSV	EPYFS[1]	EPYFS[0]	EP18DBT	GCPRD	GCPRU	GCPSL[1]	GCPSL[0]			
R/W	-	-	-	-	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	0	0	0	-	-	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	EPVM	0	R/W	Epson LCD Vertical Signal Mode » 0: Normal « » 1: Test Mode «
10	EPVSC	0	R/W	Epson LCD SYS Control » 0: Normal « » 1: B-field up «
9	EPRGBR	0	R/W	Epson LCD RGB filter operation rate » 0: 13.5 MHz « » 1: 27.0 MHz «
8:7	RSV			Reserved
6:5	EPYFS	0	R/W	Epson LCD Pre-Filter Selection » 00: No filter « » 01: 1+1 « » 10: 1+2+1 «
4	EP18DBT	0	R/W	Epson LCD 18DBT mode In normal mode, pattern of DCLK is toggled per line, however DCLK is altered per 3-line in 18DBT mode. » 0: Normal mode « » 1: 18DBT type «
3	GCPRD	0	R/W	GCP RAM Access Prohibit Controls Read/write to GCPDATA register » 0: Access permit « » 1: Access prohibit «
2	GCPRU	0	R/W	RAM is selected to GCP pulse.
1:0	GCPSL	0	R/W	ROM output pattern of GCP pulse is selected.

13.7.36 GCPDATA - GCPDATA

[illegible]

Bit	Name	Reset Value	R/W	Function
15:0	GCPD	0	R/W	<u>GCP RAM Port Data register</u> When GCPRD=0, RAM table is accessible Writing: To begin writing the GCP table, first set GCPRD to 1 and then return to 0 to clear the internal address counter. Then, successive writes to the GPCDATA register will set the GCP table values Reading: To begin reading the GCP table, first set GCPRD to 1 and then return to 0 to clear the internal address counter. Then, successive reads from the GPCDATA register will return the GCP table values The data written at the beginning of GCP output is output in MSB first.

13.7.37 CASIO - CASIO LCD Control

CASIO	0003:0848										offset: 0x48		default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CGRES	CDM	CRIT	CSTB	CLS[1]	CLS[0]	
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:6	RSV			Reserved
5	CGRES	0	R/W	Casio LCD GRES Port Control » 0: Normal « » 1: Fixed LOW «
4	CDM	0	R/W	Casio LCD Display Mode » 0: Normal « » 1: Vertical Inverse Mode «
3	CRIT	0	R/W	Casio LCD RIT Port Control » 0: ? « » 1: ? «
2	CSTB	0	R/W	Casio LCD STBYB Port Control » 0: ? « » 1: ? «
1:0	CLS	0	R/W	In ES1.0, Casio LCD Panel Selection » 00: Standard « » 01: NTSC « » 10: PAL « In ES1.1, bit 1 is reserved in ES1.1 Bit 0 is used for, Internal SYNC signal select bit for EPSON, CASIO LCD I/F 0: Analog SYNC (SSG) 1: Digital SYNC

13.7.38 DOUTCTL - Digital Output Control

[illegible]

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	DOF	0	R/W	DCLK Offset » 00: 0.0 « » 01: -0.5 « » 10: 0.5 «
11	VCLKS	0	R/W	<u>VCLK Select</u> » 0: DCLK « » 1: External Clock «
10	ESGEN	0	R/W	<u>External Sync (analog synchronous mode)</u> » 0: disable « » 1: enable «
9	EFIV	0	R/W	<u>External Field Input Inversion</u> » 0: normal « » 1: inverted «
8	EFEN	0	R/W	<u>External Field Input Mode</u> » 0: internal field generation « » 1: external field input «
7	ESYIV	0	R/W	<u>External SYNC Input Inversion</u> » 0: normal « » 1: inverted «
6	ESYEN	0	R/W	<u>External SYNC Input Mode</u> » 0: internal SYNC generation « » 1: external SYNC input «
5:4	Y160	0	R/W	<u>YCbCr 16-bit Mode Chroma Order</u> » 00: Cb-Cr (based on 13.5MHz) « » 01: Cb-Cr (based on 27MHz) « » 10: Cr-Cb (based on 13.5MHz) « » 11: Cr-Cb (based on 27MHz) «
3	DOI V	0	R/W	<u>Digital Output Inversion</u> » 0: normal « » 1: inverted «
2:1	DOMD	0	R/W	<u>Digital Output Mode</u> » 00: Digital RGB « » 01: YUV 16-bit « » 10: Epson LCD « » 11: Casio LCD «
0	DODI S	0	R/W	<u>Digital Output Disable</u> » 0: Normal « » 1: Fixed LOW «

13.7.39 SGHSYNP - Sync Period

[illegible][illegible]

14 General Purpose Input Output (GIO)

14.1 Introduction

The DM270 has 33 General-Purpose I/O (GIO) ports. GIO ports serve two purposes. One is to provide users a general-purpose programmable I/O that gives a convenient way of performing simple control to external devices. The other is to provide additional dedicated interface signals to external devices. With the additional signals, DM270 is able to either increase the performance or the functionality of the overall system. Since the GIO ports can't perform both functions at the same time, it is up to the user to trade off between general-purpose programming I/Os and dedicated feature for performance enhanced functions.

IMPORTANT: In ES1.1 and higher GIO0 is dedicated for WakeUp function from sleep or power-down modes or wakeup from reset during serial boot-up mode. GIO0 direction should not be set as output.

14.2 GIO Port State Control

This section describes the registers that allow you to use the GIO ports as regular I/O ports, i.e. how to read the current state and how to set the current state of each GIO port.

The port direction of each GIO can be set in the GIO Direction registers (DIR0- GIO Direction Register 0, DIR1- GIO Direction Register 1, and DIR2- GIO Direction Register 2). The direction can be set to input or output. To allow an external device to drive a particular GIO port, set the port direction to input. To allow a particular GIO port to drive an external device, set the port direction to output.

When set to input, the port is in a high impedance state, and an external device can drive the port. When set to output, the port can drive an external device.

The GIO Bit Set registers (BITSET0- GIO Bit Set Register 0, BITSET1- GIO Bit Set Register 1, and BITSET2- GIO Bit Set Register 2) and the GIO Bit Clear registers (BITCLR0- GIO Bit Clear Register 0, BITCLR1- GIO Bit Clear Register 1, and BITCLR2- GIO Bit Clear Register 2) are used to set the state of each GIO port and also to read the state of each GIO port.

GIO chatter control registers allow de-bouncing of the external input signal. Chatter control for each GIO port can be enabled or disabled using CHAT0- GIO Chatter Setting Register 0, CHAT1- GIO Chatter Setting Register 1, and CHAT2- GIO Chatter Setting Register 2. User should set the chatter width (NCH) in NCHAT- GIO Chatter Value Register. When chatter is enabled, the GIO port imports input data if the input is stable for more than $(NCH+1) \times \text{ARM clocks}$.

14.3 External Interrupt Settings

Several GIO ports can be used as external ARM interrupt ports. A maximum of 15 external interrupts can be specified by settings in registers in the GIO module. GIO1 through GIO15 can be used as external interrupt ports. These interrupt signals are supplied to the interrupt controller. Please refer to the section on the Interrupt Controller for information on how to configure the interrupt controller when using these interrupts.

IMPORTANT: In ES1.1 and higher GIO0 is dedicated for WakeUp function from sleep or power-down modes or wakeup from reset during serial boot-up mode. GIO0 direction should not be set as output. It cannot be used as an external interrupt.

To use a particular GIO port as an external interrupt you must set the appropriate bit in register IRQPORT- GIO IRQ Port Setting Register and you must set the GIO port direction to input. Furthermore, to trigger the interrupt on the rising edge of the external input signal, set the appropriate bit in one of the GIO Inversion registers (INV0- GIO Inversion Register 0, INV1- GIO Inversion Register 1, and INV2- GIO Inversion Register 2) to 1. When a GIO port's inversion bit in INV0, INV1 or INV2 is set to 0, the interrupt will trigger on the falling edge of the signal. It is possible to trigger "single-sided" interrupts (rising or falling edge as per INVx registers) or "dual sided" interrupts where in interrupts will be triggered for both edges of the external signal. Set IRQEDGE- GIO IRQ Edge Setting Register to configure a GIO port for "single sided" or "double sided" interrupts.

When using the GIO ports for external interrupts, the external signal should pulse for at least 2 ARM clock cycles.

Furthermore, even when GIO ports are configured as external interrupts, the corresponding bits in the Bit Set and Bit Clear registers (BITSET0, BITSET1, BITSET2, BITCLR0, BITCLR1, and BITCLR2) are still updated to reflect the port status.

14.4 GIO Functional Settings

GIO ports (GIO1-GIO33) can be used as dedicated function ports rather than general-purpose input/output ports. Dedicated ports can be selected by setting the proper bits in the FSEL0- GIO Function Select Register 0, FSEL1- GIO Function Select Register 1, and IRQPORT- GIO IRQ Port Setting Register. A table of GIO ports and signals that can be selected is shown in Table 24.

After a reset all GIO pins are defined as non-inverted, general-purpose input.

Table 24. GPIO port selection output

Pin name (primary)	Pin name (secondary)	I/O	Pin description	Register used for enabling secondary function
GIO0	EXT0	I	External interrupt #0	IRQPORT
GIO1	EXT1	I	External interrupt #1	IRQPORT
GIO2	EXT2	I	External interrupt #2	IRQPORT
GIO3	EXT3	I	External interrupt #3	IRQPORT
GIO4	EXT4	I	External interrupt #4	IRQPORT
GIO5	EXT5	I	External interrupt #5	IRQPORT
GIO6	EXT6	I	External interrupt #6	IRQPORT
GIO7	EXT7	I	External interrupt #7	IRQPORT
GIO8	DSP_BFSR1 EXT8	I/O I	DSP frame sync pulse input #1 or External interrupt #8	FSEL0 IRQPORT
GIO9	DSP_BCLKR1 EXT9	I/O I	DSP serial transfer clock input #1 or External interrupt #9	FSEL0 IRQPORT
GIO10	DSP_BDR1 EXT10	I I	DSP serial data input #1 or External interrupt #10	FSEL0 IRQPORT
GIO11	DSP_BFSX1 EXT11	I/O I	DSP frame sync pulse #1 or External interrupt #11	FSEL0 IRQPORT
GIO12	DSP_BCLKX1 EXT12	I/O I	DSP serial transfer clock #1 or External interrupt #12	FSEL0 IRQPORT
GIO13	DSP_BDX1 EXT13	O I	DSP serial data output #1 or External interrupt #13	FSEL0 IRQPORT
GIO14	DSP_CLKS1 EXT14	I I	DSP external clock input #1 or External interrupt #14	FSEL0 IRQPORT
GIO15	XF EXT15	O I	DSP External flag output or External interrupt #15	FSEL0 IRQPORT
GIO16	CLKOUT0	O	Clock output #0	FSEL0
GIO17	CLKOUT1	O	Clock output #1	FSEL0
GIO18	CLKOUT2	O	Clock output #2	FSEL0
GIO19	SDO1	O	Serial I/F#1 data output	FSEL0
GIO20	SDI1	O	Serial I/F#1 data input	FSEL0
GIO21	SCLK1	I/O	Serial I/F#1 clock input/output	FSEL0
GIO22	SDEN1	I/O	Serial I/F#1 transfer enable	FSEL0
GIO23	RXD1	I	UART #1 data input	FSEL0
GIO24	TXD1	O	UART #1 data output	FSEL1
GIO25	MIRQ MRST R[2]	O O O	ARM interrupt (IRQ) output or WDT reset signal output or R[2] in RGB666 mode	FSEL1 FSEL1 FSEL1 (bit 14)
GIO26	C_WEN	I	CCD controller write enable	FSEL1
GIO27	CCDC_SHUT G[2]	O O	CCD Controller shutter trigger or G[2] in RGB666 mode	FSEL1 FSEL1 (bit 14)
GIO28	LCD_OE	O	LCD_OE output in RGB666/R GB565 mode	FSEL1
GIO29	PWM1	O	PWM output #1	FSEL1
GIO30	PWM2 VENC_EXT_VD	O I	PWM output #2 or VENC external HD	FSEL1 FSEL1 (bit 13)
GIO31	SCL	O	I2C clock output	FSEL1
GIO32	SDA	I/O	I2C transmit data	FSEL1

Pin name (primary)	Pin name (secondary)	I/O	Pin description	Register used for enabling secondary function
GIO33	EXT_LINE_ID VENC_EXT_HD	I I	EVEN/ODD line control signal for Digital LCD I/F or VENC external VD	FSEL1 FSEL1 (bit 13)

14.5 General I/O Register Map (GIO)

Address	Register	Description
0003:0580	DIR0	GIO Direction Register 0
0003:0582	DIR1	GIO Direction Register 1
0003:0584	DIR2	GIO Direction Register 2
0003:0586	INV0	GIO Inversion Register 0
0003:0588	INV1	GIO Inversion Register 1
0003:058A	INV2	GIO Inversion Register 2
0003:058C	BITSET0	GIO Bit Set Register 0
0003:058E	BITSET1	GIO Bit Set Register 1
0003:0590	BITSET2	GIO Bit Set Register 2
0003:0592	BITCLR0	GIO Bit Clear Register 0
0003:0594	BITCLR1	GIO Bit Clear Register 1
0003:0596	BITCLR2	GIO Bit Clear Register 2
0003:0598	IRQPORT	GIO IRQ Port Setting Register
0003:059A	IRQEDGE	GIO IRQ Edge Setting Register
0003:059C	CHAT0	GIO Chatter Setting Register 0
0003:059E	CHAT1	GIO Chatter Setting Register 1
0003:05A0	CHAT2	GIO Chatter Setting Register 2
0003:05A2	NCHAT	GIO Chatter Value Register
0003:05A4	FSEL0	GIO Function Select Register 0
0003:05A6	FSEL1	GIO Function Select Register 1

14.6 General I/O Registers

14.6.1 DIR0- GIO Direction Register 0

DIR0	0003:0580				offset: 0x00								default: 0xFFFF			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15DIR	GIO14DIR	GIO13DIR	GIO12DIR	GIO11DIR	GIO10DIR	GIO9DIR	GIO8DIR	GIO7DIR	GIO6DIR	GIO5DIR	GIO4DIR	GIO3DIR	GIO2DIR	GIO1DIR	GIO0DIR
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	GI015DIR	1	R/W	GI015 Port Direction; » 0: output «, » 1: input «
14	GI014DIR	1	R/W	GI014 Port Direction; » 0: output «, » 1: input «
13	GI013DIR	1	R/W	GI013 Port Direction; » 0: output «, » 1: input «
12	GI012DIR	1	R/W	GI012 Port Direction; » 0: output «, » 1: input «
11	GI011DIR	1	R/W	GI011 Port Direction; » 0: output «, » 1: input «
10	GI010DIR	1	R/W	GI010 Port Direction; » 0: output «, » 1: input «
9	GI009DIR	1	R/W	GI009 Port Direction; » 0: output «, » 1: input «
8	GI008DIR	1	R/W	GI008 Port Direction; » 0: output «, » 1: input «
7	GI007DIR	1	R/W	GI007 Port Direction; » 0: output «, » 1: input «
6	GI006DIR	1	R/W	GI006 Port Direction; » 0: output «, » 1: input «
5	GI005DIR	1	R/W	GI005 Port Direction; » 0: output «, » 1: input «
4	GI004DIR	1	R/W	GI004 Port Direction; » 0: output «, » 1: input «
3	GI003DIR	1	R/W	GI003 Port Direction; » 0: output «, » 1: input «
2	GI002DIR	1	R/W	GI002 Port Direction; » 0: output «, » 1: input «
1	GI001DIR	1	R/W	GI001 Port Direction; » 0: output «, » 1: input «
0	GI000DIR	1	R/W	GI000 Port Direction; » 0: output «, » 1: input «

14.6.2 DIR1- GIO Direction Register 1

DIR1	0003:0582				offset: 0x02								default: 0xFFFF			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15DIR	GIO14DIR	GIO13DIR	GIO12DIR	GIO11DIR	GIO10DIR	GIO9DIR	GIO8DIR	GIO7DIR	GIO6DIR	GIO5DIR	GIO4DIR	GIO3DIR	GIO2DIR	GIO1DIR	GIO0DIR
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	GI031DIR	1	R/W	GI031 Port Direction; » 0: output «, » 1: input «
14	GI030DIR	1	R/W	GI030 Port Direction; » 0: output «, » 1: input «
13	GI029DIR	1	R/W	GI029 Port Direction; » 0: output «, » 1: input «
12	GI028DIR	1	R/W	GI028 Port Direction; » 0: output «, » 1: input «
11	GI027DIR	1	R/W	GI027 Port Direction; » 0: output «, » 1: input «
10	GI026DIR	1	R/W	GI026 Port Direction; » 0: output «, » 1: input «
9	GI025DIR	1	R/W	GI025 Port Direction; » 0: output «, » 1: input «
8	GI024DIR	1	R/W	GI024 Port Direction; » 0: output «, » 1: input «
7	GI023DIR	1	R/W	GI023 Port Direction; » 0: output «, » 1: input «
6	GI022DIR	1	R/W	GI022 Port Direction; » 0: output «, » 1: input «
5	GI021DIR	1	R/W	GI021 Port Direction; » 0: output «, » 1: input «
4	GI020DIR	1	R/W	GI020 Port Direction; » 0: output «, » 1: input «
3	GI019DIR	1	R/W	GI019 Port Direction; » 0: output «, » 1: input «
2	GI018DIR	1	R/W	GI018 Port Direction; » 0: output «, » 1: input «
1	GI017DIR	1	R/W	GI017 Port Direction; » 0: output «, » 1: input «
0	GI016DIR	1	R/W	GI016 Port Direction; » 0: output «, » 1: input «

14.6.3 DIR2- GIO Direction Register 2

[illegible][illegible]

14.6.4 INV0- GIO Inversion Register 0

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI015INV	0	R/W	GI015 Input Port Inversion; « 0: normal «, » 1: inverted « NOTE: when a GIO pin is set as output, corresponding INV bit setting has no effect
14	GI014INV	0	R/W	GI014 Input Port Inversion; « 0: normal «, » 1: inverted «
13	GI013INV	0	R/W	GI013 Input Port Inversion; « 0: normal «, » 1: inverted «
12	GI012INV	0	R/W	GI012 Input Port Inversion; « 0: normal «, » 1: inverted «
11	GI011INV	0	R/W	GI011 Input Port Inversion; « 0: normal «, » 1: inverted «
10	GI010INV	0	R/W	GI010 Input Port Inversion; « 0: normal «, » 1: inverted «
9	GI009INV	0	R/W	GI009 Input Port Inversion; « 0: normal «, » 1: inverted «
8	GI008INV	0	R/W	GI008 Input Port Inversion; « 0: normal «, » 1: inverted «
7	GI007INV	0	R/W	GI007 Input Port Inversion; « 0: normal «, » 1: inverted «
6	GI006INV	0	R/W	GI006 Input Port Inversion; « 0: normal «, » 1: inverted «
5	GI005INV	0	R/W	GI005 Input Port Inversion; « 0: normal «, » 1: inverted «
4	GI004INV	0	R/W	GI004 Input Port Inversion; « 0: normal «, » 1: inverted «
3	GI003INV	0	R/W	GI003 Input Port Inversion; « 0: normal «, » 1: inverted «
2	GI002INV	0	R/W	GI002 Input Port Inversion; « 0: normal «, » 1: inverted «
1	GI001INV	0	R/W	GI001 Input Port Inversion; « 0: normal «, » 1: inverted «
0	GI000INV	0	R/W	GI000 Input Port Inversion; « 0: normal «, » 1: inverted «

14.6.5 INV1- GIO Inversion Register 1

[illegible]

Bit	Name	Reset Value	R/W	Function
15	G0I3INV	0	R/W	G0I31 Input Port Inversion; » 0: normal «, » 1: inverted « NOTE: When a GIO pin is set as output, corresponding INV bit setting has no effect
14	G0I30INV	0	R/W	G0I30 Input Port Inversion; » 0: normal «, » 1: inverted «
13	G0I29INV	0	R/W	G0I29 Input Port Inversion; » 0: normal «, » 1: inverted «
12	G0I28INV	0	R/W	G0I28 Input Port Inversion; » 0: normal «, » 1: inverted «
11	G0I27INV	0	R/W	G0I27 Input Port Inversion; » 0: normal «, » 1: inverted «
10	G0I26INV	0	R/W	G0I26 Input Port Inversion; » 0: normal «, » 1: inverted «
9	G0I25INV	0	R/W	G0I25 Input Port Inversion; » 0: normal «, » 1: inverted «
8	G0I24INV	0	R/W	G0I24 Input Port Inversion; » 0: normal «, » 1: inverted «
7	G0I23INV	0	R/W	G0I23 Input Port Inversion; » 0: normal «, » 1: inverted «
6	G0I22INV	0	R/W	G0I22 Input Port Inversion; » 0: normal «, » 1: inverted «
5	G0I21INV	0	R/W	G0I21 Input Port Inversion; » 0: normal «, » 1: inverted «
4	G0I20INV	0	R/W	G0I20 Input Port Inversion; » 0: normal «, » 1: inverted «
3	G0I19INV	0	R/W	G0I19 Input Port Inversion; » 0: normal «, » 1: inverted «
2	G0I18INV	0	R/W	G0I18 Input Port Inversion; » 0: normal «, » 1: inverted «
1	G0I17INV	0	R/W	G0I17 Input Port Inversion; » 0: normal «, » 1: inverted «
0	G0I16INV	0	R/W	G0I16 Input Port Inversion; » 0: normal «, » 1: inverted «

14.6.6 INV2- GIO Inversion Register 2

[illegible][illegible]

14.6.7 BITSET0- GIO Bit Set Register 0

[illegible]

Bit	Name	Reset Value	R/W	Function
15	QI015SET	0	R/W	GIO15 Bit Set Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT (INV=NORMAL or INV=INVERT), set HIGH, Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	QI014SET	0	R/W	GIO14 Bit Set
13	QI013SET	0	R/W	GIO13 Bit Set
12	QI012SET	0	R/W	GIO12 Bit Set
11	QI011SET	0	R/W	GIO11 Bit Set
10	QI010SET	0	R/W	GIO10 Bit Set
9	QI009SET	0	R/W	GIO09 Bit Set
8	QI008SET	0	R/W	GIO08 Bit Set
7	QI007SET	0	R/W	GIO07 Bit Set
6	QI006SET	0	R/W	GIO06 Bit Set
5	QI005SET	0	R/W	GIO05 Bit Set
4	QI004SET	0	R/W	GIO04 Bit Set
3	QI003SET	0	R/W	GIO03 Bit Set
2	QI002SET	0	R/W	GIO02 Bit Set
1	QI001SET	0	R/W	GIO01 Bit Set
0	QI000SET	0	R/W	GIO00 Bit Set

14.6.8 BITSET1- GIO Bit Set Register 1

[illegible]

Bit	Name	Reset Value	R/W	Function
15	QI031SET	0	R/W	GIO31 Bit Set Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT (INV=NORMAL or INV=INVERT), set HIGH, Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	QI030SET	0	R/W	GIO30 Bit Set
13	QI029SET	0	R/W	GIO29 Bit Set
12	QI028SET	0	R/W	GIO28 Bit Set
11	QI027SET	0	R/W	GIO27 Bit Set
10	QI026SET	0	R/W	GIO26 Bit Set
9	QI025SET	0	R/W	GIO25 Bit Set
8	QI024SET	0	R/W	GIO24 Bit Set
7	QI023SET	0	R/W	GIO23 Bit Set
6	QI022SET	0	R/W	GIO22 Bit Set
5	QI021SET	0	R/W	GIO21 Bit Set
4	QI020SET	0	R/W	GIO20 Bit Set
3	QI019SET	0	R/W	GIO19 Bit Set
2	QI018SET	0	R/W	GIO18 Bit Set
1	QI017SET	0	R/W	GIO17 Bit Set
0	QI016SET	0	R/W	GIO16 Bit Set

14.6.9 BITSET2- GIO Bit Set Register 2

BITSET2		0003:0590								offset:		0x10										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO32SET	GIO32SET								
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W								
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0								

[illegible]

14.6.10 BITCLR0- GIO Bit Clear Register 0

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI015CLR	0	R/W	GI015 Bit Clr Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & (INV=NORMAL or INV=INVERT), set LOW Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	GI014CLR	0	R/W	GI014 Bit Clr
13	GI013CLR	0	R/W	GI013 Bit Clr
12	GI012CLR	0	R/W	GI012 Bit Clr
11	GI011CLR	0	R/W	GI011 Bit Clr
10	GI010CLR	0	R/W	GI010 Bit Clr
9	GI009CLR	0	R/W	GI009 Bit Clr
8	GI008CLR	0	R/W	GI008 Bit Clr
7	GI007CLR	0	R/W	GI007 Bit Clr
6	GI006CLR	0	R/W	GI006 Bit Clr
5	GI005CLR	0	R/W	GI005 Bit Clr
4	GI004CLR	0	R/W	GI004 Bit Clr
3	GI003CLR	0	R/W	GI003 Bit Clr
2	GI002CLR	0	R/W	GI002 Bit Clr
1	GI001CLR	0	R/W	GI001 Bit Clr
0	GI000CLR	0	R/W	GI000 Bit Clr

14.6.11 BITCLR1- GIO Bit Clear Register 1

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI031CLR	0	R/W	GIO31 Bit Clr Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & (INV= NORMAL or INV=INVERT), set LOW Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	GI030CLR	0	R/W	GIO30 Bit Clr
13	GI029CLR	0	R/W	GIO29 Bit Clr
12	GI028CLR	0	R/W	GIO28 Bit Clr
11	GI027CLR	0	R/W	GIO27 Bit Clr
10	GI026CLR	0	R/W	GIO26 Bit Clr
9	GI025CLR	0	R/W	GIO25 Bit Clr
8	GI024CLR	0	R/W	GIO24 Bit Clr
7	GI023CLR	0	R/W	GIO23 Bit Clr
6	GI022CLR	0	R/W	GIO22 Bit Clr
5	GI021CLR	0	R/W	GIO21 Bit Clr
4	GI020CLR	0	R/W	GIO20 Bit Clr
3	GI019CLR	0	R/W	GIO19 Bit Clr
2	GI018CLR	0	R/W	GIO18 Bit Clr
1	GI017CLR	0	R/W	GIO17 Bit Clr
0	GI016CLR	0	R/W	GIO16 Bit Clr

14.6.12 BITCLR2- GIO Bit Clear Register 2

[illegible][illegible]

14.6.13 IRQPORT- GIO IRQ Port Setting Register

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI 0151 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
14	GI 0141 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
13	GI 0131 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
12	GI 0121 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
11	GI 0111 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
10	GI 0101 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
9	GI 0091 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
8	GI 0081 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
7	GI 0071 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
6	GI 0061 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
5	GI 0051 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
4	GI 0041 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
3	GI 0031 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
2	GI 0021 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
1	GI 0011 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «
0	GI 0001 RQ	0	R/W	Enable External IRQ; » 0: normal «, » 1: external IRQ «

14.6.14 IRQEDGE- GIO IRQ Edge Setting Register

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI015IRE	0	R/W	Interrupt Edge » 0: Single-sided « GIOxxINV = 0/normal -> falling edge GIOxxINV = 1/invert -> rising edge
				» 1: Dual-sided « Both rising and falling edges
14	GI014IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
13	GI013IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
12	GI012IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
11	GI011IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
10	GI010IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
9	GI09IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
8	GI08IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
7	GI07IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
6	GI06IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
5	GI05IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
4	GI04IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
3	GI03IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
2	GI02IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
1	GI01IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «
0	GI00IRE	0	R/W	Interrupt Edge; » 0: Single-sided «; » 1: Dual-sided «

14.6.15 CHAT0- GIO Chatter Setting Register 0

[illegible]

Bit	Name	Reset Value	R/W	Function
15	GI015CHA	0	R/W	Chatter Disable (de-bounce) When enabled, input data is set only after input signal is stable for an interval longer than set in NCHAT » 0: Enable » 1: Disable «
14	GI014CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
13	GI013CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
12	GI012CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
11	GI011CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
10	GI010CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
9	GI09CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
8	GI08CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
7	GI07CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
6	GI06CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
5	GI05CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
4	GI04CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
3	GI03CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
2	GI02CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
1	GI01CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
0	GI00CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «

14.6.16 CHAT1- GIO Chatter Setting Register 1

[illegible]

Bit	Name	Reset Value	R/W	Function
15	CI 031CHA	0	R/W	Chatter Disable (de-bounce) When enabled, input data is set only after input signal is stable for an interval longer than set in NCHAT » 0: Enable » 1: Disable «
14	CI 030CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
13	CI 029CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
12	CI 028CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
11	CI 027CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
10	CI 026CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
9	CI 025CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
8	CI 024CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
7	CI 023CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
6	CI 022CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
5	CI 021CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
4	CI 020CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
3	CI 019CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
2	CI 018CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
1	CI 017CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «
0	CI 016CHA	0	R/W	Chatter Disable; » 0: Enable «; » 1: Disable «

14.6.17 CHAT2- GIO Chatter Setting Register 2

CHAT2				0003:05A0				offset:		0x20								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO33CHA	GIO32CHA				
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R	R				
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0				

[illegible]

14.6.18 NCHAT- GIO Chatter Value Register

NCHAT		0003:05A2				offset: 0x22				default: 0x0008							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	NCH[3]	NCH[2]	NCH[1]	NCH[0]	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	1	0	0	0	

[illegible]

14.6.19 FSEL0- GIO Function Select Register 0

FSEL0		0003:05A4				offset: 0x24								default: 0x0000			
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name		GIO23SEL	GIO22SEL	GIO21SEL	GIO20SEL	GIO19SEL	GIO18SEL	GIO17SEL	GIO16SEL	GIO15SEL	GIO14SEL	GIO13SEL	GIO12SEL	GIO11SEL	GIO10SEL	GIO9SEL	GIO8SEL
R/W		R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GI 023SEL	0	R/W	Function setting of GIO23 » 0: GIO23 « » 1: RXD1 «
14	GI 022SEL	0	R/W	Function setting of GIO22 » 0: GIO22 « » 1: SDEN1 «
13	GI 021SEL	0	R/W	Function setting of GIO21 » 0: GIO21 « » 1: SCLK1 «
12	GI 020SEL	0	R/W	Function setting of GIO20 » 0: GIO20 « » 1: SDI1 «
11	GI 019SEL	0	R/W	Function setting of GIO19 » 0: GIO19 « » 1: SDO1 «
10	GI 018SEL	0	R/W	Function setting of GIO18 » 0: GIO18 « » 1: CLKOUT2 «
9	GI 017SEL	0	R/W	Function setting of GIO17 » 0: GIO17 « » 1: CLKOUT1 « CLKOUT can output clock generated by OSEL and O'DIV registers in general clock output.
8	GI 016SEL	0	R/W	Function setting of GIO16 » 0: GIO16 « » 1: CLKOUT0 « CLKOUT can output clock generated by OSEL and O'DIV registers in general clock output.
7	GI 015SEL	0	R/W	Function setting of GIO15 » 0: GIO15 « » 1: DSP_XF «
6	GI 014SEL	0	R/W	Function setting of GIO14 » 0: GIO14 « » 1: DSP_CLKS1 «
5	GI 013SEL	0	R/W	Function setting of GIO13 » 0: GIO13 « » 1: DSP_BDX1 «
4	GI 012SEL	0	R/W	Function setting of GIO12 » 0: GIO12 « » 1: DSP_BCLKX1 «
3	GI 011SEL	0	R/W	Function setting of GIO11 » 0: GIO11 « » 1: DSP_BFSX1 «
2	GI 010SEL	0	R/W	Function setting of GIO10 » 0: GIO10 « » 1: DSP_BDR1 «
1	GI 09SEL	0	R/W	Function setting of GIO9 » 0: GIO9 « » 1: DSP_BCLKR1 «
0	GI 08SEL	0	R/W	Function setting of GIO8 » 0: GIO8 « » 1: DSP_BFSR1 «

14.6.20 FSEL1- GIO Function Select Register 1

FSEL1		0003:05A6				offset: 0x26		default: 0x0000									
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name		RSV	RGB666SEL	HDVDSSEL	RSV	RSV	GIO33SEL	GIO32SEL	GIO31SEL	GIO30SEL	GIO29SEL	GIO28SEL	GIO27SEL	GIO26SEL	GIO25SEL[1]	GIO25SEL[0]	GIO24SEL
R/W		-	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default		-	0	0	-	-	0	0	0	0	0	0	0	0	0	0	0

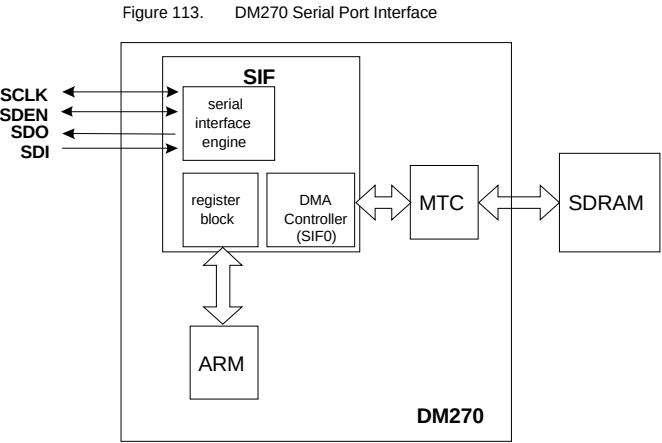
Bit	Name	Reset Value	R/W	Function
15	RSV	0		Reserved
14	RGB666SEL	0	R/W	RGB666 Select 0: No effect 1: GIO25 acts as R[2], GIO27 acts as B[2] in RGB565/666 mode Thus, RGB666SEL is a switch from RGB565 to RGB666 mode. Make sure GIO25 and GIO27 are set as output direction Note: This field is present in ES1.1 and higher. This field is reserved in ES1.0
13	HDVDSSEL	0	R/W	External HDVP Select 0: No effect 1: GIO30 is selected as external HD, GIO33 is selected as external VD. Make sure GIO30, GIO33 are set as input direction Note: This field is present in ES1.1 and higher. This field is reserved in ES1.0
12:11	RSV	0		Reserved
10	QO33SEL	0	R/W	Function setting of GIO33 » 0: GIO33 « » 1: EXT_LINE_ID «
9	QO32SEL	0	R/W	Function setting of GIO32 » 0: GIO32 « » 1: SDA «
8	QO31SEL	0	R/W	Function setting of GIO31 » 0: GIO31 « » 1: SCL «
7	QO30SEL	0	R/W	Function setting of GIO30 » 0: GIO30 « » 1: PWM1 «
6	QO29SEL	0	R/W	Function setting of GIO29 » 0: GIO29 « » 1: PWM0 «
5	QO28SEL	0	R/W	Function setting of GIO28 » 0: GIO28 « » 1: LCD_OE output in RGB565/666 mode «
4	QO27SEL	0	R/W	Function setting of GIO27 » 0: GIO27 « » 1: CCDC_SHUT «
3	QO26SEL	0	R/W	Function setting of GIO26 » 0: GIO26 « » 1: C_WEN «
2:1	QO25SEL	0	R/W	Function setting of GIO25 » 00: GIO25 « » 01: MIRQ « » 10: MRST « » 11: reserved « MIRQ: Interrupt signal input in ARM core MRST: Reset signal of ARM core generated in WDT
0	QO24SEL	0	R/W	Function setting of GIO24 » 0: GIO24 « » 1: TXD1 «

15 Serial Port Interface (SPI)

15.1 Introduction

The DM270 has a two-channel clock synchronized serial port interface (SIF or SPI) controller. The transfer unit is 8 bits each time, and it is a simple three-line serial interface consisting of a clock line, output data line, and input data line. Serial Port 0 has a DMA transfer function with SDRAM for transfer of large quantities of data. DM270 SPI has master mode and slave mode and which is selectable. The SPI can supply the serial clock when it is used in a master mode. The SPI also can be synchronized by the external serial clock input when it is used in a slave mode.

Both of the serial ports and the DMA channel are controlled by ARM registers. Refer Serial Port 0 for a list of control registers for serial port 0, and Serial Port 1 for a list of control registers for serial port 1.



15.2 Serial Interface Signal Pins

The SPI-0 uses the serial interface engine's hardware pins for external interface. SPI-1 on the other hand uses GIO pins for external interface. Table 25. shows the hardware pins used by the two Serial Interface channels for signal interfaces. When SPI-1 is used, set all GIO pins for usage in serial interface using the register FSEL0 (0003:05A4h) in the "GIO module".

Table 25. Serial Interface Signal Pins

Serial Interface Signal	SPI 0	SPI 1
Serial Data Input	SDI	GIO20
Serial Data Output	SDO	GIO19
Serial Data Enable	SDEN	GIO22
Serial Clock	SCLK	GIO21

15.3 Baud Rate

Bit rate is set in the RATE field of SIOMODEx – SP Mode Registers.

The bit rate is defined with the following formula:

$$\text{Bit Rate} = (\text{ARM clock}) / ((\text{RATE} + 1) \times 2)$$

where, RATE is the value set in SIOMODEx – SP Mode Registers

Note that the baud rate settings are meaningful only in master mode. In slave mode baud rate settings are invalid.

15.4 Master Mode SIF

In master mode, the serial port outputs both the Serial Clock (SCLK) and Serial Enable (SDEN) signals. Writing "0" to the bit MSSEL in SIOMODEx – SP Mode Registers configures the SPI in master mode.

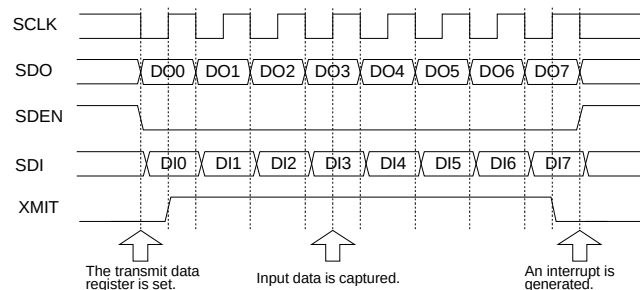
15.4.1 Data Transfer Procedure

Following section describes the data transfer procedure in master mode.

- Write "1" in bit SIOEN of SIOENx – SP Transmit Enable Registers
- Write data byte to be transferred to TXDATAx – SP Transmit Data Registers. This initiates the serial data transmission one bit at a time from SDO port in synchronization with the falling edge of serial clock output (SCLK).
- The data is read from SDI to RXDATAx – SP Receive Data Registers one bit at a time in synchronization with the rising edge of SCLK.
- The transfer flag XMIT in RXDATAx – SP Receive Data Registers is asserted at the first rising edge of SCLK after data transfer is initiated.
- The transfer flag XMIT is cleared automatically at the eighth rising edge of SCLK from the start of transfer.
- Upon the completion of data transfer (transmit/receive) of 8 bits, SDEN signal is asserted "Hi" and a transfer complete interrupt is generated.
- The receive data can be read from the RXDATAx – SP Receive Data Registers. When transfer is continued without reading the receive data, the RXDATA register is overwritten.

15.4.2 Data Transfer Timing Diagram

Figure 114. Timing Diagram for Master Mode Data Transfer



15.5 Slave Mode SIF

In slave mode, the serial port expects both the Serial Clock (SCLK) and Serial Enable (SDEN) signals as inputs. Writing "1" to the bit MSSEL in SIOMODEx – SP Mode Registers configures the SPI in slave mode. By default, both SPI-0 and SPI-1 are configured in slave mode.

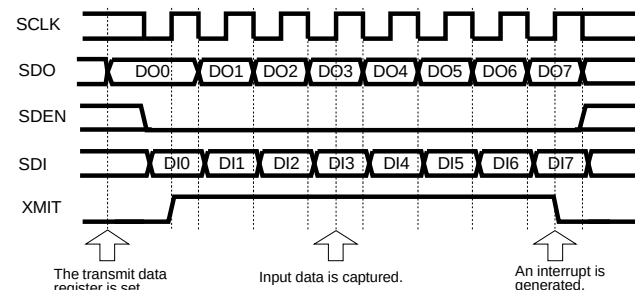
15.5.1 Data Transfer Procedure

Following section describes the data transfer procedure in slave mode.

- Write "1" in bit SIOEN of SIOENx – SP Transmit Enable Registers
- Write data byte to be transferred to TXDATAx – SP Transmit Data Registers. The first bit (Bit-0 in case of LSB first, and Bit-7 in case of MSB first) of the data set in TXDATAx – SP Transmit Data Registers is output to SDO port in synchronization with the falling edge of serial clock output (SCLK).
- When SDEN is asserted "Low", the transfer flag XMIT is asserted at the first rising edge of SCLK.
- Remaining 7 bits of the data set in TXDATAx – SP Transmit Data Registers is output to SDO port in synchronization with the falling edge of serial clock output (SCLK).
- The data is read from SDI to RXDATAx – SP Receive Data Registers one bit at a time in synchronization with the rising edge of SCLK.
- The transfer flag XMIT is cleared automatically and a transfer complete interrupt is generated at the eighth rising edge of SCLK from the start of transfer.
- The receive data can be read from the RXDATAx – SP Receive Data Registers. When transfer is continued without reading the receive data, the RXDATA register is overwritten.

15.5.2 Data Transfer Timing Diagram

Figure 115. Timing Diagram for Slave Mode Data Transfer



NOTE: If the serial clock meets AC Timing, it is not necessary to be 50 of duty ratio.

15.6 Transmission and Reception Buffers

Each serial port has an 8-bit buffer on each the transmitting and receiving sides. The data set in the transmission buffer is output in synchronization with the falling edge of SCLK.

These buffers correspond to the TXDATAx – SP Transmit Data Registers and RXDATAx – SP Receive Data Registers. The XMIT bit of the RXDATAx – SP Receive Data Registers is used by the system to indicate if a transmission or reception is in progress or not. When writing into the transmit data buffer, the transfer flag XMIT is automatically asserted. The transfer flag is cleared automatically at the end of the transmission or reception.

You need to check the status of XMIT before writing into the transmit register. If a transmission is in progress, the piece of data you write into TXDATAx – SP Transmit Data Registers will be ignored and lost. When a reception is in progress (XMIT=1), the data read from RXDATAx – SP Receive Data Registers will always be the previous one. Any new piece of data that arrives into the serial port will update the RXDATAx register. If the previous data hasn't been read before, it will be lost.

15.7 Serial Port Modes

In master mode of operation, the serial clock mode bit (SCLKM) of the SIOMODEx – SP Mode Registers can be used to set the polarity of the clock pin when the serial port is in idle mode. By default, this bit will be set to '0' and the SCLK clock signal will stay high when no data transfer is in progress.

The SIOEN bit in SIOENx – SP Transmit Enable Registers is used to enable or disable the serial port. When this bit is set to 0 (default value), the serial port is disabled. If a transmission or reception is in progress, writing to this bit will take effect only after the current transfer has been completed.

When the serial port is disabled, any new data written into the TXDATAx – SP Transmit Data Registers does not initiate any new transmission, any new data received is ignored and the RXDATAx – SP Receive Data Registers keeps its previous value. When in this mode, the pins SDO and SDEN output the logic level '1'. The status of the SCLK pin is dependent on the SCLKM bit.

The MSB control bit of the SIOMODEx – SP Mode Registers allows you to chose between the LSB or the MSB of the data being placed first. This bit affects the transmission and the reception.

15.8 Interrupts

When a transmission or reception is completed (DMA (SPI #0) as well as non-DMA mode), an interrupt can be sent to the ARM. The interrupts INT8 and INT9

are connected to the serial port 0 and serial port 1, respectively. The interrupt can be enabled or disabled by setting registers in the interrupt controller.

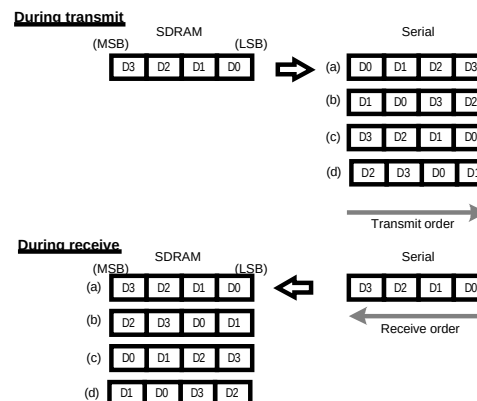
15.9 Transfer Using The Direct Memory Access

Serial I/F 0 is connected to SDRAM by a dedicated bus, and DMA data transfer to and from external devices is possible without going through ARM.

The SDRAM controller has two DMA channels, which are used to perform DMAs involving SDRAM as source or destination. A DMA channel between SDRAM and Serial I/F 0 must be selected prior to triggering Serial I/F 0 DMA. The DMA channel is selected using the REFCTL register(0x309A8). Also the DMA channel SDRAM access priority can be changed using the SDPRTY1-10 (0x309AA – 0x309BC) registers (see section "SDRAM priority access" in chapter Memory Traffic Controller (MTC) for details)

ARM executes DMA transfer by controlling the DMA controller in the serial I/F module. The SDRAM transfer start address, total number of transferred bytes and the transfer direction are set. When DMA transfer is complete, an interrupt is generated to ARM (INT8). The number of transfer bytes from SDRAM has a minimum unit of 4 bytes. The number of bytes transmitted over the serial port line has to be a multiple of 2. Endian conversion is possible at the time of DMA transfer. There are four conversion modes, as shown in Figure 116. The endian conversion is configured with the bits PRM1:0 in DMAMODE0 – SP0 DMA Mode Registers.

Figure 116. Endian Conversion During Transmission and Reception

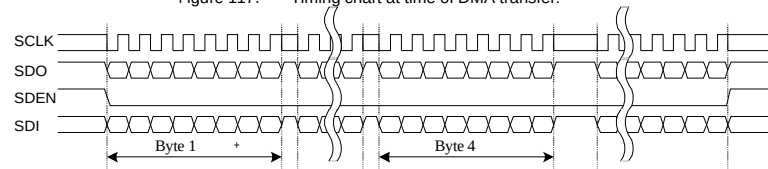


15.9.1 DMA transfer procedure

Below is an example of DMA transfer procedure:

- DMA channel is selected between serial interface and SDRAM by setting SDMA1 or SDMA2 field in REFCTL (0x309A8) register.
- The serial interface is set to DMA mode, furthermore transfer SDRAM start address and the number of transfer bytes are set.
NOTE: SDRAM address must be even byte aligned and transfer byte must be multiple of two bytes
- DMA transfer starts by setting the start bit (TRG) in the register DMATRG0 – SP0 DMA Trigger Registers
- With DMA start, the first piece of data is read from SDRAM 4 bytes at a time. The data is written in the SPI transfer register 1 byte at a time. The data transmitted is synchronized with the falling edge of the serial port clock.
- In master as well as slave mode, serial transfer is suspended during SDRAM access. That is serial transfer is suspended for 1 CPU clock after every byte transfer. In slave mode, one has to leave wide enough time-interval to ensure time for SDRAM access.
- The next 4 bytes are read from the SDRAM and transmitted.
- When the requested amount of bytes to be sent has been reached, an interruption occurs.

Figure 117. Timing chart at time of DMA transfer:



Because DMA transfer request occurs every 4 bytes, the serial transfer can look discontinued.

15.10 Serial Port Register Maps (SPx)

15.10.1 Serial Port 0

Address	Register	Description
0003:0200	TXDATA0	SP0 Transmit Data Register
0003:0202	RXDATA0	SP0 Receive Data Register
0003:0204	SI0EN0	SP0 Transmit Enable
0003:0206	SI0MODE0	SP0 Mode
0003:0208	DMATRG0	SP0 DMA Trigger
0003:020A	DMAMODE0	SP0 DMA Mode
0003:020C	DMASTADL0	SP0 DMA SDRAM Start Address - low
0003:020E	DMASTADH0	SP0 DMA SDRAM Start Address - high
0003:0210	DMASTAT0	SP0 DMA Status

15.10.2 Serial Port 1

Address	Register	Description
0003:0280	TXDATA1	SP1 Transmit Data Register
0003:0282	RXDATA1	SP1 Receive Data Register
0003:0284	SI0EN1	SP1 Transmit Enable
0003:0286	SI0MODE1	SP1 Mode

15.11 Serial Port Registers

15.11.1 TXDATAx – SP Transmit Data Registers

TXDATA0	0003:0200	offset:	0x00													default:	0x0000
TXDATA1	0003:0280	offset:	0x00														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TXD[7]	TXD[6]	TXD[5]	TXD[4]	TXD[3]	TXD[2]	TXD[1]	TXD[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	TXD	0	R/W	Transmit Data register When data is written to this register, transmission is started. Data output is performed on the falling edge of the clock. NOTE: - TXEN must be set to 1 to enable writes to this register - Cannot write to this register when RXDATA:XMIT is 1

15.11.2 RXDATAx – SP Receive Data Registers

RXDATA0	0003:0202	offset:	0x02													default:	0x0000
RXDATA1	0003:0282	offset:	0x02														
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	XMIT	RXD[7]	RXD[6]	RXD[5]	RXD[4]	RXD[3]	RXD[2]	RXD[1]	RXD[0]	
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	XMIT	0	R	Send/Receive flag » 0: Send/Receive complete « » 1: Send/Receive in progress «
7:0	RXD	0	R	Received Data register After writing output data to the TXDATA register and waiting for the send/receive to complete, received data can be read from this register. Data is sampled on the rising edge of the clock.

15.11.3 SIOENx – SP Transmit Enable Registers

SIOEN0	0003:0204										offset:	0x04						default: 0x0000					
SIOEN1	0003:0284										offset:	0x04											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SIOEN							
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W							
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0							

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	SIOEN	0	R/W	<u>Transmission/reception enable register</u> When this bit is set, transmission/reception is started by writing data into TXDATAx. When this bit is cleared, transmission/reception is prohibited, and data cannot be written into TXDATAx.

15.11.4 SIOMODEx – SP Mode Registers

SIOMODE0	0003:0206			offset:	0x06											
SIOMODE1	0003:0286			offset:	0x06		default: 0x0100									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	SLVEN	SIOCLR	SCLKM	MSB	MSEL	RATE[7]	RATE[6]	RATE[5]	RATE[4]	RATE[3]	RATE[2]	RATE[1]	RATE[0]
R/W	-	-	-	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	SLVEN	0	R	<u>External Enable</u> » 0: disable « » 1: enable « SP0: SDEN0, SP1:GIO22 (when SDEN1 is set)
11	SIOCLR	0	R/W	<u>Serial I/O Port Clear</u> Resets Serial Port, all registers settings are retained
10	SCLKM	0	R/W	<u>Serial Clock Mode</u> Sets level of clock when transmission is idle » 0: serial clock level = 1 when idle « » 1: serial clock level = 0 when idle «
9	MSB	0	R/W	Determines bit order of transmitted or received data » 0: LSB first « » 1: MSB first «
8	MSSEL	1	R/W	<u>Master/Slave Selection</u> » 0: Master « » 1: Slave «
7:0	RATE	0	R/W	<u>Transfer Bit Rate</u> The transfer rate is given by Bit rate = (CLK_ARM/2) / (2*(RATE+1))

15.11.5 DMATRG0 – SP0 DMA Trigger Registers

DMATRG0		0003:0208		offset: 0x08												default: 0x0000	
DMA is present in SP0 only																	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	TRG	0	W	DMA Transfer Start bit When set to 1, DMA transfer is started. After transfer ends, bit is automatically cleared to 0. Writing 0 has no meaning. If set to 1 during transfer, it has no meaning.

15.11.6 DMAMODE0 – SP0 DMA Mode Registers

DMAMODE0		0003:020A		offset: 0x0A												default: 0x0000	
DMA is present in SP0 only																	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	PRM[1]	PRM[0]	DIR	TRSZ[10]	TRSZ[9]	TRSZ[8]	TRSZ[7]	TRSZ[6]	TRSZ[5]	TRSZ[4]	TRSZ[3]	TRSZ[2]	TRSZ[1]	TRSZ[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	PRM	0	R/W	DMA transfer byte order » 00: D3D2D1D0 -> D3D2D1D0 « » 01: D3D2D1D0 -> D0D1D2D3 « » 10: D3D2D1D0 -> D2D3D0D1 « » 11: D3D2D1D0 -> D1D0D3D2 «
11	DIR	0	R/W	DMA transfer direction » 0: SIF -> SDRAM « » 1: SDRAM -> SIF «
10:0	TRSZ	0	R/W	Transfer size in bytes Only even numbers of bytes can be specified, i.e., bit 0 is forced to 0

15.11.7 DMASTADL0 – SP0 DMA SDRAM Start Address Low Registers

DMASTADL0

0003:020C

offset: 0x0C

DMA is present in SP0 only

default: 0x0000

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	Lower 16 bits of DMA start address of SDRAM. Only even numbers (word addresses) can be specified. NOTE: address specified as offset from start of SDRAM

15.11.8 DMASTADH0 – SP0 DMA SDRAM Start Address High Registers

DMASTADH0					0003:020E					offset: 0x0E												default: 0x0000				
DMA is present in SP0 only																										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0										
Name	RSV	RSV	RSV	RSV	RSV	ADRH[10]	ADRH[9]	ADRH[8]	ADRH[7]	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]										
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W										
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0										

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	ADRH	0	R/W	Upper 11 bits of DMA start address of SDRAM. Only even numbers (word addresses) can be specified. NOTE: address specified as offset from start of SDRAM

15.11.9 DMASTAT0 – SP0 DMA Status Registers

DMASTAT0		0003:0210		offset: 0x10												default: 0x0000	
DMA is present in SP0 only																	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RUN	RSV	RSV	RSV	RSV	REM[10]	REM[9]	REM[8]	REM[7]	REM[6]	REM[5]	REM[4]	REM[3]	REM[2]	REM[1]	REM[0]	
R/W	R	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

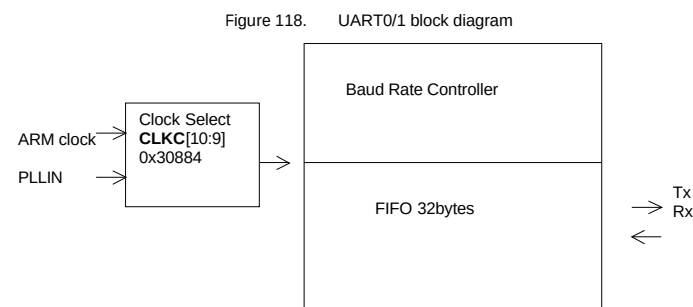
Bit	Name	Reset Value	R/W	Function
15	RUN	0	R	DMA transfer flag » 0: Transfer done « » 1: Transfer in progress «
14:11	RSV			Reserved
10:0	REM	0	R/W	Remaining number of DMA transfer bytes

16 UART

16.1 Introduction

DM270 supports a two-channel two-lines (RX, TX) start-stop synchronization serial port (UART). A 32-byte FIFO is built in for each the transmitting side and receiving side, and interrupts can be generated due to causes such as errors or various states of FIFO. Data is 8 or 7 bits. Even, odd or no parity bit can be selected, and 1 or 2 stop bits can be selected. Transmission/reception timing uses the ARM clock. The maximum baud rate achievable is a 1/16 of the ARM clock (in bps).

Figure 118 shows the DM270-UART block diagram



The UART module detects parity errors and overrun errors. Interrupt to ARM is generated when any of the following events occur:

- o Amount of data in receiving-side FIFO exceeds set value
- o Amount of data in transmitting-side FIFO exceeds set value
- o Word received has an error
- o The timeout period is reached

UART Register Maps (UARTx) lists all the registers in the UART module and their corresponding addresses. ARM controls the UART module by accessing the following internal registers.

DTRRx – UART Data Transmit/Receive Registers

BRSRx – UART Bit Rate Set Registers

MSRx – UART Mode Setup Registers

RFCRx – UART Receive FIFO Control Registers

TFCRx – UART Transmit FIFO Control Registers

LCRx – UART Line Control Registers

SRx – UART Status Registers

In addition to this reference clock for UART can either be ARM Clock or PLL input clock. This is selected using CLKC (0x30884) register in clock controller

The speed of UART ports is determined by BRSRx – UART Bit Rate Set Registers. The UART is clocked by the internal signal ARM_CLK (or PLLIN) divide by 16.

$$\text{UARTspeed} = \text{Reference Clock} / (16 * (\text{BRSRvalue} + 1))$$

Where, Reference Clock = ARM Clock or PLL input clock (refer “Clock Controller” section for details)

Each UART has 2 FIFOs, one 32-byte for transmit and one 32-byte for receive. However, there is only one register to access them. The DTRRx – UART Data Transmit/Receive Registers is used to access transmit and receive FIFOs. When this register is read, the oldest data stored in reception FIFO is returned. When you write into it, the data is stored into the transmission FIFO register and then transferred. You must make sure that the transmit FIFO is not full before writing into this register. When reading this register, the bits 12 to 8 indicates if the word read is valid or not. The data is valid only when bits 11 to 8 are equal to 0 and bit 12 is equal to 1.

The configuration of the UART is defined by the MSRx – UART Mode Setup Registers. The PSB and PEB bits select the parity used during a communication. When enabled, one or two parity bits are appended at the end of the data. The bit CLSx select the size of the word transmitted or received. When only 7 bits are used, the DTRx7 bit of the DTRRx – UART Data Transmit/Receive Registers is ignored during a write and always reads 0 during a read.

The RFTIEx bit enables the interrupt to be generated when the reception FIFO is above the trigger level.

The TFTIEx bit enables the interrupt to be generated when the transmission FIFO is below trigger level.

The REIEx bit enables the interrupt to be generated when a received word that has either parity error, overrun, frame error or break is present in reception FIFO.

The TOICx1:0 bits are used to configure the timeout interrupt. In status where at least one word of received data is stored in reception FIFO, if new data has not been transmitted even when the set time has elapsed, interrupt is generated. This interrupt is used to provide a trigger of reception data loading to ARM7TDMI when, at the end of data transfer, reception data up to the reception FIFO trigger level has not been stored in reception FIFO.

16.2 UART Register Maps (UARTx)

Address	Register	Description
0003:0300	DTRR0	UART0 Data Transmit/Receive Register
0003:0302	BRSR0	UART0 Bit Rate Set Register
0003:0304	MSR0	UART0 Mode Setup Register
0003:0306	RFCR0	UART0 Receive FIFO Control Register
0003:0308	TFCR0	UART0 Transmit FIFO Control Register
0003:030A	LCR0	UART0 Line Control Register
0003:030C	SR0	UART0 Status Register

Address	Register	Description
0003:0380	DTRR1	UART1 Data Transmit/Receive Register
0003:0382	BRSR1	UART1 Bit Rate Set Register
0003:0384	MSR1	UART1 Mode Setup Register
0003:0386	RFCR1	UART1 Receive FIFO Control Register
0003:0388	TFCR1	UART1 Transmit FIFO Control Register
0003:038A	LCR1	UART1 Line Control Register
0003:038C	SR1	UART1 Status Register

16.3 UART Registers

16.3.1 DTRRx – UART Data Transmit/Receive Registers

DTRR0	0003:0300				offset:		0x00									
DTRR1	0003:0380				offset:		0x00		default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RVF	BF	FE	ORF	PEF	DTR[7]	DTR[6]	DTR[5]	DTR[4]	DTR[3]	DTR[2]	DTR[1]	DTR[0]
R/W	-	-	-	R	R	R	R	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	RVF	0	R	<u>Received Word Valid Flag</u> » 0: Received word invalid « » 1: Received word valid «
11	BF	0	R	<u>Break Flag</u> » 0: No break « » 1: detected a break «
10	FE	0	R	<u>Framing Error</u> » 0: No Framing Error « » 1: There is framing error «
9	ORF	0	R	<u>Over Run Flag</u> » 0: No Overrun « » 1: Overrun «
8	PEF	0	R	<u>Parity Error Flag</u> » 0: No parity error « » 1: Parity error «
7:0	DTR	0	R/W	<u>Data Transmit/Receive</u> On a read, the oldest received byte stored in Receive FIFO is read along with the corresponding status bits and the byte read is deleted from the Receive FIFO. On a write, the data is stored in the transmission FIFO. Before writing, it is necessary to confirm that the transmission FIFO is not full.

16.3.2 BRSRx – UART Bit Rate Set Registers

BRSR0		0003:0302		offset:		0x02											
BRSR1		0003:0382		offset:		0x02		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	BRSR[15]	BRSR[14]	BRSR[13]	BRSR[12]	BRSR[11]	BRSR[10]	BRSR[9]	BRSR[8]	BRSR[7]	BRSR[6]	BRSR[5]	BRSR[4]	BRSR[3]	BRSR[2]	BRSR[1]	BRSR[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	BRSR	0	R/W	<u>Bit Rate Set Register</u> Sets data transfer bit rate: $BRSR = \{ (CLK/2) / (16 \cdot \text{bitrate}) \} - 1$ $UART\text{bitrate} = (CLK/2) / ((16 \cdot (BRSR + 1)))$

16.3.3 MSRx – UART Mode Setup Registers

MSR0		0003:0304				offset:		0x04									
MSR1		0003:0384				offset:		0x04		default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RFTIE	TFTIE	REIE	RSV	TOIC[1]	TOIC[0]	RSV	RSV	RSV	RSV	RSV	PEB	PSB	SBLS	RSV	CLS	
R/W	R/W	R/W	R/W	-	R/W	R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	
Default	0	0	0	-	0	0	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15	RFTIE	0	R/W	<u>Receive FIFO Trigger Interrupt Enable</u> » 0: disabled « » 1: enabled « Receive Interrupt set when amount of data in Receive FIFO passes above trigger level
14	TFTIE	0	R/W	<u>Transmit FIFO Trigger Interrupt Enable</u> » 0: disabled « » 1: enabled « Transmit Interrupt set when amount of data in Transmit FIFO falls below trigger level
13	REIE	0	R/W	<u>Receive Error Interrupt Enable</u> » 0: disabled « » 1: enabled « Receive Error Interrupt is set if - any received word that has a parity error, - there is a FIFO overrun, - there is a frame error, or - a break exists in the reception FIFO.
12	RSV			Reserved
11:10	TOIC	0	R/W	<u>Time Out Interrupt Control</u> Timeout Interrupt period: » 00: disabled « » 01: 3 words « » 10: 7 words « » 11: 15 words « Timeout Interrupt is set in case where at least one word of received data has been stored in reception FIFO and new data has not been transmitted even though the set time has elapsed. This interrupt can be used to provide a trigger for received data loading to ARM in the case where the received data has not been accumulated in the reception FIFO up to the reception FIFO trigger level at the end of data transfer.
9:5	RSV			Reserved
4	PEB		R/W	<u>Parity Enable Bit</u> » 0: disable (no parity) « » 1: enable (parity bit added to end of data byte) «
3	PSB		R/W	<u>Parity Selection Bit</u> » 0: even parity « » 1: odd parity «
2	SBLS		R/W	<u>Stop Bit Length Selection</u> » 0: 1 stop bit « » 1: 2 stop bits «
1	RSV			Reserved
0	CLS	0	R/W	<u>Character Length Selection</u> » 0: 8-bits « » 1: 7-bits « Specifies the data size

16.3.4 RFCRx – UART Receive FIFO Control Registers

RFCR0	0003:0306										offset:		0x06											
RFCR1	0003:0386										offset:		0x06		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RFCB	RDEF	RSV	RSV	RSV	RTL[2]	RTL[1]	RTL[0]	RSV	RSV	RWC[5]	RWC[4]	RWC[3]	RWC[2]	RWC[1]	RWC[0]								
R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R								
Default	0	0	-	-	-	0	0	0	-	-	0	0	0	0	0	0								

Bit	Name	Reset Value	R/W	Function
15	RFCB	0	R/W	<u>Receive FIFO Clear Bit</u> » 0: no action « » 1: Receive FIFO is cleared, set to 0 after clear «
14	RDEF	0	R/W	<u>Receive Data Error Flag</u> Set to 1 if data that causes receive error is in FIFO Cleared to 0 if no data FIFO causes a receive error This bit is the same as bit 9 of the SR register. When 1 is written to this bit, the receive block is reset and then the bit is cleared.
13:11	RSV			Reserved
10:8	RTL	0	R/W	<u>Receive Trigger Level</u> » 000: 1 word « » 001: 4 words « » 010: 8 words « » 011: 16 words « » 100: 24 words « » 101: 32 words « » 110: N/A « » 111: N/A « Sets Receive FIFO Trigger Interrupt level
7:6	RSV			Reserved
5:0	RWC	0	R	<u>Receive Word Count</u> Number of words in Receive FIFO

16.3.5 TFCRx – UART Transmit FIFO Control Registers

TFCR0	0003:0308										offset: 0x08							
TFCR1	0003:0388										offset: 0x08		default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	TFCB	RSV	RSV	RSV	RSV	TTL[2]	TTL[1]	TTL[0]	RSV	RSV	TWC[5]	TWC[4]	TWC[3]	TWC[2]	TWC[1]	TWC[0]		
R/W	R/W	-	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R		
Default	0	-	-	-	-	0	0	0	-	-	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	TFCB	0	R/W	<u>Transmit FIFO Clear Bit</u> » 0: no action « » 1: Transmit FIFO is cleared, set to 0 after clear «
14:11	RSV			Reserved
10:8	TTL	0	R/W	<u>Transmit Trigger Level</u> » 000: 1 word « » 001: 4 words « » 010: 8 words « » 011: 16 words « » 100: 24 words « » 101: 32 words « » 110: N/A « » 111: N/A « Sets Receive FIFO Trigger Interrupt level
7:6	RSV			Reserved
5:0	TWC	0	R	<u>Transmit Word Count</u> Number of words in Transmit FIFO

16.3.6 LCRx – UART Line Control Registers

LCR0	0003:030A										offset: 0x0A					
LCR1	0003:038A										offset: 0x0A					
	default: 0x0000															
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	UTST	RSV	RSV	RSV	RSV	RSV	BOC	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV
R/W	-	R/W	-	-	-	-	-	R/W	-	-	-	-	-	-	-	-
Default	-	0	-	-	-	-	-	0	-	-	-	-	-	-	-	-

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	UIST	0	R/W	Test mode setting - set to 0 for normal operation
13:9	RSV			Reserved
8	BOC	0	R/W	Break Output Control Controls the TX output: » 0: Normal data output « » 1: Transmit output is forced to 0 «
7:0	RSV			Reserved

16.3.7 SRx – UART Status Registers

SRO	0003:030C				offset:	0x0C								default: 0x0000			
SR1	0003:038C				offset:	0x0C											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RFTI	TFTI	RFER	TOIF	RSV	RSV	RSV	RSV	RSV	RFNEF	TFEF	TREF	
R/W	-	-	-	-	R	R	R	R	-	-	-	-	-	R	R	R	
Default	-	-	-	-	0	0	0	0	-	-	-	-	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	RFTI	0	R	Receive FIFO Trigger Indicator » 0: amount of data is below trigger level « » 1: amount of data is above trigger level «
10	TFTI	0	R	Transmit FIFO Trigger Indicator » 0: amount of data is above trigger level « » 1: amount of data is below trigger level «
9	RFER	0	R	Receive Data Error Flag Set to 1 if data that causes receive error is in FIFO Cleared to 0 if no data FIFO causes a receive error This bit is the same as bit 14 of the RFCR register.
8	TOIF	0	R	Time Out Interrupt Flag » 0: no timeout interrupt « » 1: timeout interrupt asserted «
7:3	RSV			Reserved
2	RFNEF	0	R	Receive FIFO Not Empty Flag » 0: No data in Receive FIFO « » 1: Data in Receive FIFO «
1	TFEF	0	R	Transmit FIFO Empty Flag » 0: Data in Transmit FIFO « » 1: No data in Transmit FIFO «
0	TREF	0	R	Transmit Register Empty Flag » 0: Data in either Transmit register or FIFO « » 1: No data in either Transmit register or FIFO «

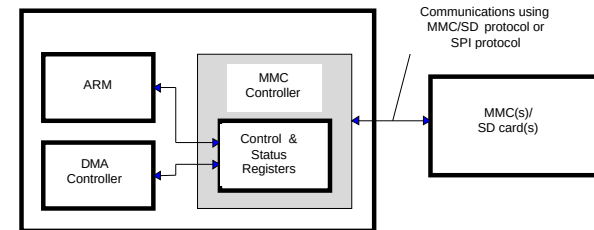
17 MMC/SD Interface

17.1 Introduction

The MMC Controller transfers data between the ARM or DMA controller on one side and a memory card or cards on the other side. (Refer to Figure 119). The ARM and DMA controller can read/write the data in memory by accessing the registers in the MMC Controller. (Refer MMC/SD Card Interface Register Map (MMCSA))

The communication between the MMC Controller and memory card(s) is performed by the MMC/SD protocol or the protocol based on the SPI standard. The MMC/SD protocol leverages one bi-directional data line (MMC) or four parallel data lines (SD Card). The SPI protocol uses two serial data lines for transmit/receive.

Figure 119. Role of the MMC Controller



17.2 Features

MMC Controller has following features.

- Supports a MultiMediaCard (an MMC) / Secure Digital Memory Card (an SD card).
- The ability to use either the MMC/SD protocol or SPI protocol.
- Software-oriented implementation for future extensibility.
- A programmable frequency of the clock that controls the timing of transfers between the MMC controller and memory card.
- Supports DMA transfer between the MMC controller and SDRAM.
- Supports SDIO card.

17.2.1 MMC Controller Pins

Table 26. shows the pins of the MMC Controller used in each mode.

The MMC/SD protocol uses the lines of clock, command (two-way communication between the MMC Controller and memory card) and data (one line for MMC, 4 lines for SD card).

SPI communications use four lines; one for a clock, one for card selection and two for data. The card select line is used to select one card to transmit/receive data. The card may be selected with any of the ARM's general-purpose I/O (GPIO) pins. Two data lines act as the DataIn line and DataOut line exchanging serial data with the selected card.

Table 26. MMC Controller Pins

Pin	Type [*]	Function		
		MMC Communications	SD Communications	SPI Communications
CLK	O	Clock line	Clock line	Clock line
CMD	I/O/Z	Command line	Command line	DataIn line (data to the card)
DAT0	I/O/Z	Data line 0	Data line 0	DataOut line (data from the card)
DAT1	I/O/Z	(Not used)	Data line 1	(Not used)
DAT2	I/O/Z	(Not used)	Data line 2	(Not used)
DAT3	I/O/Z	(Not used)	Data line 3	Card select line

^{*}I = Input to the MMC Controller ; O = Output from the MMC Controller; Z = High -impedance

17.2.2 Function Clock and Memory Clock

There are two clocks, the function clock and memory clock, in the MMC module and each clock must be set with the proper frequency.

The function clock frequency determines the operation speed of the MMC Controller. This function clock is generated by dividing the ARM clock by programmable clock divider in the Clock Controller Module (CLKC) (Refer to Figure 120). The divide-down value is set by the value in the CMCDIV field (0x3088C [7:0]) of the CLK's MMCFCLK register as in the following equation.

$$\text{Function clock frequency} = \text{ARM clock frequency} / (\text{CMDV} + 1)$$

The memory clock appears on the CLK pin of MMC Controller Interface. This clock controls the timing of communication between the MMC Controller and the connected memory card. Memory clock is generated by dividing the function clock in the MMC Controller (Refer to Figure 120). The divide-down value is set by CLKRT field of the MMCCLK - MMC Control Register as shown in the following equation.

$$\text{Memory clock frequency} = \text{function clock frequency} / (2^{(\text{CLKRT} + 1)})$$

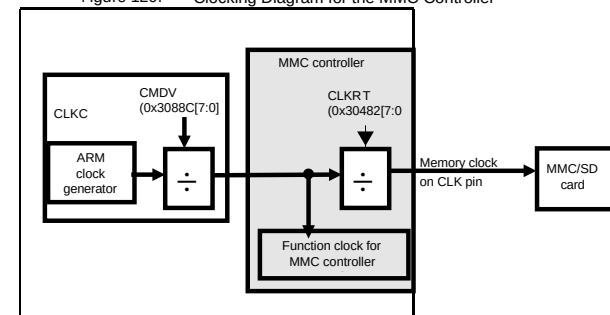
The following restrictions must be observed when the function clock and memory clock are determined.

- Ensure that CMCIV bit (0x30884 [14]) in the CLKC register is set to '1' (as default setting) when the MMCCCLK - MMC Control Register is set to '0'.
- Set the operating frequencies of the ARM and SDRAM, and memory clock frequency to meet the following equation when DMA transfer is used for the data transfer between this module and the SDRAM.

Operating clock frequency relationship among ARM, SDRG and MMC/SD memory clock frequency when using DMA transfer

$$\text{MMC/SD memory clock frequency} \geq \left(\frac{3}{\text{ARM clock frequency}} \right) \left(\frac{11}{\text{SDRC clock frequency}} \right)$$

Figure 120. Clocking Diagram for the MMC Controller



17.2.3 Command Registers (MMCCMD, MMCARGH and MMCARGL)

The argument and command index are included in command bit rows. Writing the command index to the CMD field in the MMCCMD - MMC Command Register, after setting upper 16 bits and lower 16 bits of the argument to the MMCARGH - MMC Argument Register High and MMCARGL - MMC Argument Register Low respectively, sends command bit rows from the command line in MMC/SD protocol mode and the DataOut line in the SPI protocol mode. In addition, the setting of other bits in the MMCCMD register is required to be changed depending on the type of command when the command index is written in the MMCCMD - MMC Command Register. Table 27. shows what the bits in the MMCCMD register individually represent.

Table 27. Descriptions of the MMCCMD register bits

Bit	Name	Description
15	DCLR	Setting '1' clears the shadow buffer for data transmit/receive. Set this when issuing a command to write the data to the card using DMA.
14	INITCK	Setting '1' sends 80 dummy clocks before sending the command data row.
13	WDATX	Specify if a command to be executed is accompanied by the data transfer. '0': No data transfer '1': With data transfer
12	STRMTP	When a command with a data transfer is executed, specify the transfer is either a block transfer or a stream transfer. '0': Block transfer '1': Stream transfer
11	DTRW	When a command with a data transfer is executed, set the destination for the data transfer. '0': Reading the data from the card '1': Writing the data to the card
10-9	RSPFMT1- RSPFMT0	Specify the type of the expected command response. "00": No command response "01": R1/R4/R5/R6 -type command response (48 bits length) "10": R2 -type command response (136 bits length) "11": R3 -type command response (48 bits length without CRC)
8	BSYEXP	Specify if the command to be executed requires the BUSY signal detect. '0': BUSY signal detect is not required. '1': BUSY signal detect is required.
7	PPLEN	Specify the type of the output buffer of command lines. '0': Nonuse of push-pull drivers (High level: Hi-Z, Low level: L drive) '1': Use of push-pull drivers
6	RSV	Reserved bit.
5-0	CMD5-CMD0	Specify the command index.

17.2.4 Command Response Registers (MMCCIDX and MMCRSP0-MMCRSP7)

After issuing a command, the card drives the response. The data rows of the response vary depending on the protocol mode and the type of the response.

In the MMC/SD protocol mode, one byte at the top of the response data row is stored into the MMCCIDX - MMC Command Index Register. The succeeding data rows are stored into the MMCRSP7 - MMC Response Register #7 to MMCRSP0 - MMC Response Register #0 in sequence.

In the SPI protocol mode, response data rows are stored into the MMCRSP7 - MMC Response Register #7 to MMCRSP5 - MMC Response Register #5 in sequence.

The following tables show how response data rows are stored into the command response registers.

Table 28. Response data assignment in case of R1 or R3 (R4, R5 or R6) response in MMC mode

Register Name	Bit position of command response	Description
MMCCIDX	[47-40]	
MMCRSP7	[39-24]	
MMCRSP6	[23-8]	
MMCRSP5-0	-	No assignment

Table 29. Response data assignment in case of R2 response in MMC mode

Register Name	Bit position of command response	Description
MMCCIDX	[135-128]	
MMCRSP7	[127-112]	
MMCRSP6	[111-96]	
MMCRSP5	[95-80]	
MMCRSP4	[79-64]	
MMCRSP3	[63-48]	
MMCRSP2	[47-32]	
MMCRSP1	[31-16]	
MMCRSP0	[15-0]	

Table 30. Response data assignment in case of R1 response in SPI mode

Register Name	Bit position of command response	Description
MMCRSP7	[7-0]	It is stored into LSB byte of MMCRSP7
MMCRSP6-0	-	No assignment

Table 31. Response data assignment in case of R2 response in SPI mode

Register Name	Bit position of command response	Description
MMCRSP7	[15-0]	
MMCRSP6-0	-	No assignment

Table 32. Response data assignment in case of R3 response in SPI mode

Register Name	Bit position of command response	Description
MMCRSP7	[39-22]	It is stored into LSB byte of MMCRSP7
MMCRSP6	[31-16]	
MMCRSP5	[15-0]	
MMCRSP4-0	-	No assignment

17.2.5 MMC/SD Interrupt Activity in the MMC Controllers

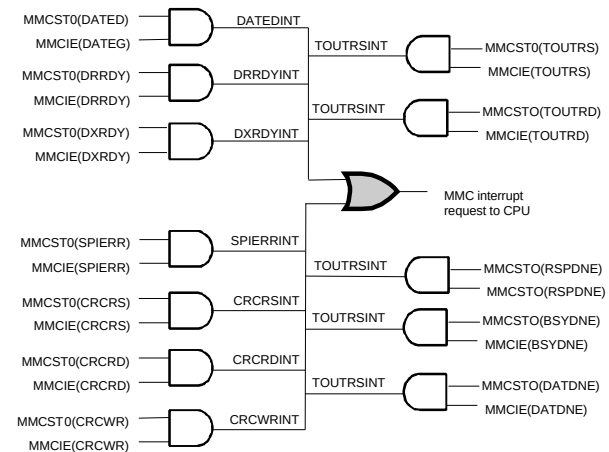
The MMC Controller can generate the interrupt requests described in Figure 121. When an interrupt event occurs, its flag bit is set in the MMCST0 - MMC Status Register #0. If enable bits corresponding to each flag are set in the MMCIE - MMC Interrupt Enable Register, an interrupt request is generated. All such requests are multiplexed to a single MMC interrupt request for the ARM, as shown in Figure 121.

The MMC interrupt is one of the maskable ARM interrupts (INT17). As with any maskable interrupt request, if it is properly enabled in the ARM, the ARM executes the corresponding interrupt service routine (ISR). The ISR for the MMC interrupt can determine the event that caused the interrupt by checking the bits in the MMCST0 - MMC Status Register #0. When the ARM reads MMCST0, all of the register's bits are automatically cleared except for DXRDY bit (MMCST0 [9]) and DRRDY bit (MMCST0 [10]), which are not cleared until the ARM writes 0 to them.

Table 33. Descriptions of the MMC Interrupt Requests

Interrupt Request	Interrupt Event
DATEDINT	An edge was detected on the DAT3 pin.
DRRDYINT	MMCDRR is ready to be read (data received).
DXRDYINT	MMCDXR is ready for new data (data transmitted).
SPIERRINT	A data error token was received in the SPI mode.
CRCRSINT	A CRC error was detected in a response from the memory card.
CRCRDINT	A CRC error was detected in the data read from the memory card.
CRCWRINT	A CRC error was detected in the data written to the memory card.
TOUTRSINT	A time-out occurred while the MMC Controller was waiting for a response to a command.
TOUTRDINT	A time-out occurred while the MMC Controller was waiting for the data from the memory card.
RSPDNEINT	For a command that requires a response: The MMC Controller has received the response without a CRC error. For a command that does not require a response: The MMC Controller has finished sending the command.
BSYDNEINT	The memory card stops or is no longer sending a busy signal when the MMC Controller is expecting a busy signal.
DATDNEINT	For read operations: The MMC Controller has received data without a CRC error. For write operations: The MMC Controller has finished sending data.

Figure 121. Enable Paths of the MMC Interrupt Requests



17.2.6 MMC Controller DMA

There is a DMA channel for transferring the data between the MMC Controller and SDRAM without involving ARM.

The EMIF, SIF, USB, MMC/SD and HPIB modules in DM270 share two DMA channels. Consequently, only up to two DMAs among five modules' DMAs can be used exclusively. The setting of the modules that use the DMA channel is performed in the SDMA field (0x309A8 [15:13] and [12:10]) of the REFCTL register. To use this module's DMA, set 0x3 to the SDMA1x or SDMA2x field.

The SDRAM transfer start address or the transfer destination is set in the MMC Controller's DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High registers. The DMA registers in MMC Controller also allow the user to suspend DMA transfers and to read transfer status information, such as the number of bytes transferred. If DATDNEINT is enabled by setting EDATDNE bit in MMCIE - MMC Interrupt Enable Register, the ARM is interrupted by INT17 after completing DMA transfer.

In addition, when DMA is used, the transfer destination/ source SDRAM address and the size of data transfer can be specified only in 4bytes units.

17.2.7 Data Flow in the Data Registers (MMCDRR and MMCDXR)

The ARM or DMA controller can read 16 bits at a time from MMCDRR - MMC Data Receive Register and write 16 bits at a time to the MMCDXR - MMC Data Transmit Register. However, the memory card is 8-bit device; it transmits or receives one byte at a time. Figure 122 and Figure 123 show how the data-size difference is handled via the data registers.

In most cases, once MMCDRR - MMC Data Receive Register is filled with two bytes data, the MMC Controller issues a data receive ready (RRDY) event. If an odd number of bytes data is received, the last byte data is stored in the lower byte in MMCDRR - MMC Data Receive Register and then a RRDY event is generated.

During transmit, the ARM typically loads 2 bytes data to MMCDXR - MMC Data Transmit Register. When the second byte data leaves MMCDXR, a data transmit ready (XRDY) event is generated. In case that an odd number of bytes is transferred, the ARM writes the last byte in right-aligned (Refer to Figure 123. The transmission of the last byte data causes a XRDY event.

Figure 122. Data Flow in the Data Receive Register (MMCDRR)

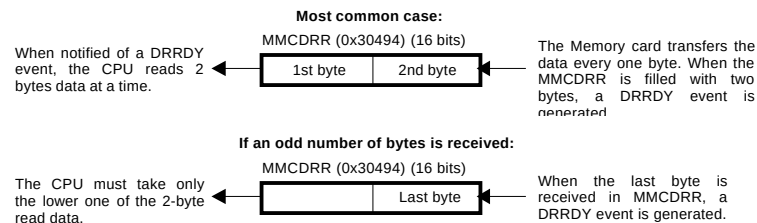
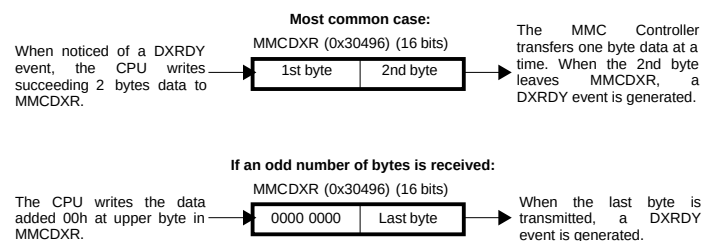


Figure 123. Data Flow in the Data Receive Register (MMCDRR)



17.3 MMC/SD Mode

The interface of the MMC Controller can use an MMC/SD protocol or an SPI protocol. To use the MMC/SD protocol, select the MMC/SD mode by clearing the SPIEN bit in the MMCCTL - MMC Function Clock Control Register.

17.3.1 MMC/SD Mode Interface

Figure 124 summarizes the MMC/SD mode interface.

In the MMC/SD mode:

- The MMC Controller supports one or more MultiMediaCards (MMCs) / Secure Digital Memory Cards (SD cards). When multiple cards are connected, the MMC Controller selects one by using identification broadcast on the data line.

The following MMC Controller pins are used:

- CMD: This pin is used for two-way communication between the connected card and the MMC Controller. The MMC Controller transmits commands to the card by using a command line and the memory card drives responses to the commands.
- DAT0 or DAT[3:0]: An MMC leverages only one data line (DAT0) and an SD Card requires four data lines. The number of DAT pins (the data bus width) can be set with the WIDTH field in the MMCCTL - MMC Function Clock Control Register (Refer to Figure 125).
- CLK: This pin provides a clock to the memory card from the Controller.
- As the data and command lines are separate, the next command to the card may be sent at the same time as data associated with the previous command. Thus, the MMC/SD mode allows sequential and multiple-block read/write operations.

Figure 124. MMC/SD Mode Interface

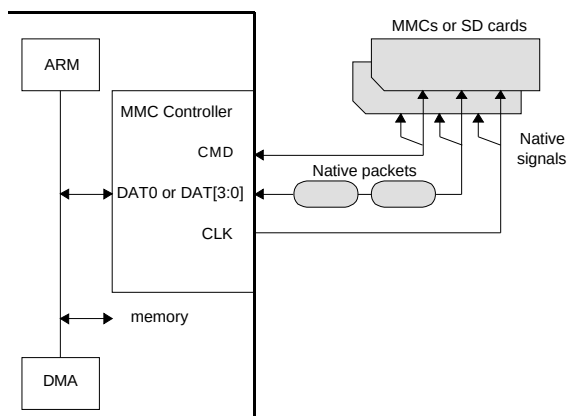
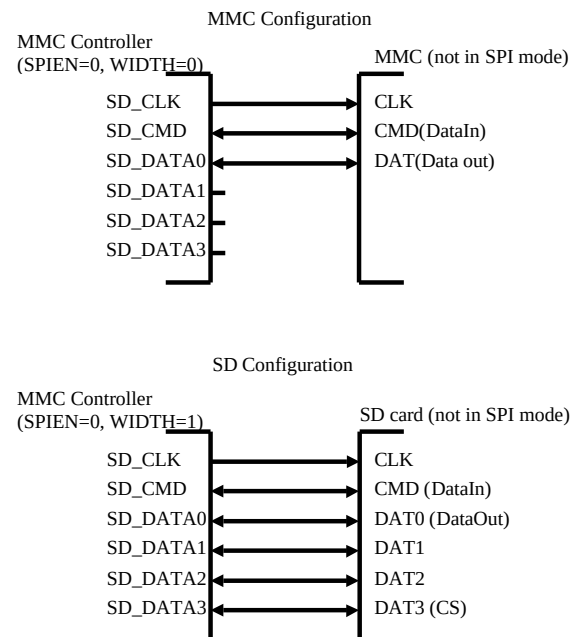


Figure 125. MMC Configuration Versus SD Configuration



17.3.2 MMC/SD Mode Write Sequence

Figure 126 and Table 34. show the signal activity when the MMC Controller is in the MMC/SD mode and is writing data to a memory card. The same block length must be defined in the MMC Controller and in the card. In a successful write sequence:

- The Controller sends a write command to the card.
- The card receives the command and sends responses to the command.
- The Controller sends a block of data to the card.
- The card sends the CRC status to the Controller.
- The card sends low BUSY bit until all the data has been programmed into the flash memory inside the card.

Figure 126. MMC/SD Mode Write Sequence

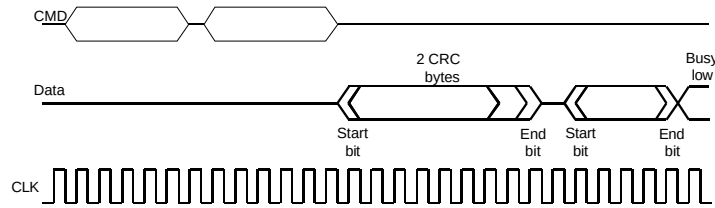


Table 34. MMC/SD Mode Write Sequence

Portion of the Sequence	Description
WR CMD	Write command: A 6-byte WRITE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response: The card sends a 6-byte response of type R1 to acknowledge the WRITE_BLOCK to the ARM.
DAT BLK	Data block: The ARM writes a block of data to the card. The data content is preceded by one start bit and is followed by two CRC bytes and one end bit.
CRC STAT	CRC status: The card sends a one byte CRC status information, which indicates to the ARM whether the data has been accepted by the card or rejected due to a CRC error. The CRC status information is preceded by one start bit and is followed by one end bit.
BSY	BUSY bit: The CRC status information is followed by a continuous stream of low busy bits until all of the data has been programmed into the flash memory on the card.

17.3.3 MMC/SD Mode Read Sequence

Figure 127 and Table 35. show the signal activity when the MMC Controller is in the MMC/SD mode and is reading data to a memory card. The same block length must be defined in the MMC Controller and in the card. In a successful read sequence:

- The Controller sends a read command to the card.
- The card drives responses to the command.
- The card sends a block of data to the ARM.

Figure 127. MMC/SD Mode Read Sequence

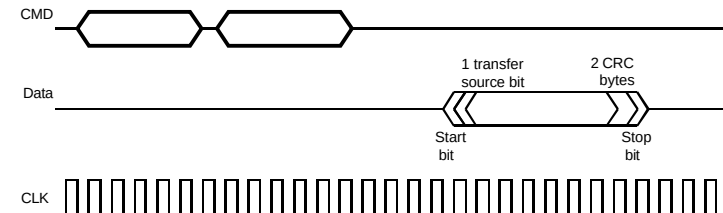


Table 35. MMC/SD Mode Read Sequence

Portion of the Sequence	Description
RD CMD	Read command: A 6-byte READ_SINGLE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response: The card sends a response of type R1 to acknowledge the READ_SINGLE_BLOCK command to the ARM.
DAT BLK	Data block: The card sends a block of data to the ARM. The data content is preceded by a start bit and is followed by two CRC byte and an end bit.

17.3.4 Card Identification Operation

In the MMC/SD mode, before the MMC Controller can start data transfers from memory cards, it has to first identify how many cards are present on the bus and configure them.

For each card that responds to the ALL_SEND_CID command, the controller reads that card's unique card identification address (CID) and then assigns it a relative address (RCA). This address is much shorter than the CID and is used by the controller to identify the card.

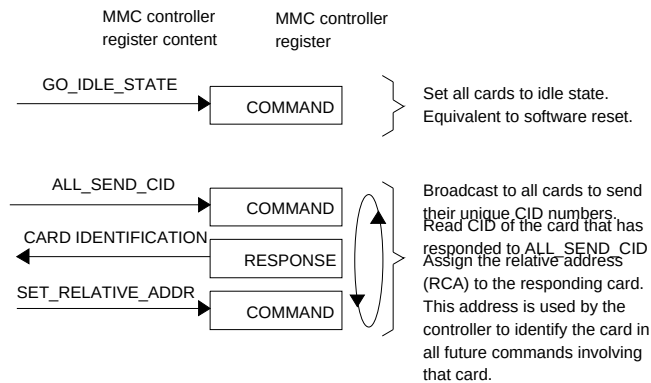
Only one card completes the response to ALL_SEND_CID at any one time. The absence of any response to ALL_SEND_CID indicates that all cards have been identified and configured.

The procedure for a card identification operation is as follows (see also Figure 128):

1. Use MMCCMD - MMC Command Register to send the GO_IDLE_STATE command to the cards. This puts all cards in the idle state.

2. Use MMCCMD - MMC Command Register to send the ALL_SEND_CID command to the cards. This tells all cards to identify themselves.
3. Wait for a card to respond. If a card responds, go to step 4). Otherwise, stop.
4. Read the CID from the response registers MMCRSP7 - MMC Response Register #7 - MMCRSP0 - MMC Response Register #0, and assign a relative address to the card by sending the SET_RELATIVE_ADDR command.

Figure 128. Card Identification (MMC/SD Mode)



17.3.5 MMC/SD Mode Single-Block Write Operation

In the MMC/SD mode, to write a single block of data to a memory card, use the following procedure. Figure 129 illustrates this procedure with some additional details. Note that the same block length must be defined in the MMC Controller and in the card. On the Controller side, set the byte length of one block to the MMCBLN - MMC Block Length Register and also set the number of transfer blocks or 1 to the MMCNBLK - MMC Number of Blocks Register.

1. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH - MMC Argument Register High and the lower address to the MMCARGL - MMC Argument Register Low.
2. Use MMCCMD - MMC Command Register to send a SELECT/DESELECT_CARD command. This selects the addressed card and deselects the others.

3. Write the destination start address to the argument registers. Load the upper address to the MMCARGH - MMC Argument Register High and the lower address to the MMCARGL - MMC Argument Register Low.
4. Write the first byte of the data block to the MMCDXR - MMC Data Transmit Register.
5. Send the WRITE_BLOCK command to the card.
6. Use MMCST0 - MMC Status Register #0 to check for errors and to determine if the byte has been successfully transmitted. If all of the data has not been written, go to step 7). If all of the data has been written, go to step 8).
7. Write the next byte of the data block to MMCDXR - MMC Data Transmit Register, and go to step 6).
8. Wait for the BSYDNE flag in MMCST0 - MMC Status Register #0 and complete writing to the card.

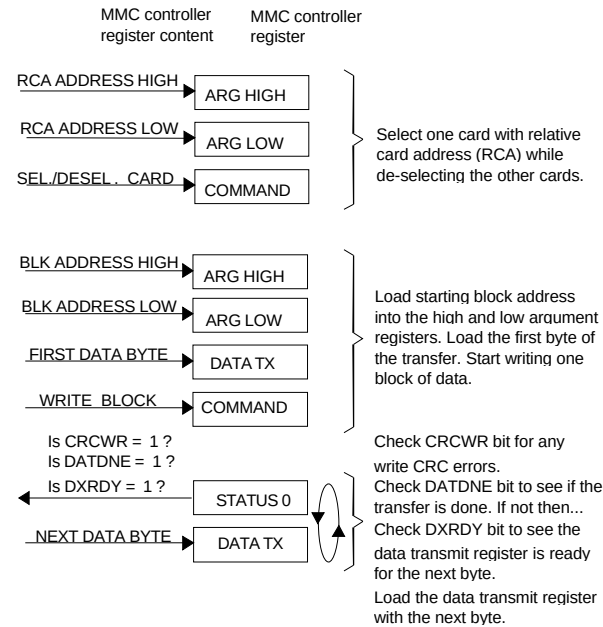
When using DMA transfer:

When using DMA transfer in Single-Block Write operation, observe the following items.

- As for the size of the data transfer, the value in the MMCBLN - MMC Block Length Register is applied as the block length. Set the number of transfer blocks to the MMCNBLK - MMC Number of Blocks Register as "1" here due to Single-Block transfer.
- Skip step 4).
- At step 5), set the following before issuing a BLOCK_WRITE command.
 - a. Set the transfer source SDRAM relative address of the data to the DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High. The address must be aligned in 4bytes units.
 - a. Set '1' to both DMAE and DIR bits in the DMAMODE - DMA Mode Register.
 - a. Set '1' to TRG bit in the DMATRG - DMA Trigger Register and trigger DMA.
- At step 5), set DCLR bit in the MMCCMD - MMC Command Register at the same time as issuing a BLOCK_WRITE command.
- At step 6), the DXRDY flag is not required to be checked. The DMA controller performs the operation of step 7) automatically.

Thus, no operation but waiting the DATDNE and BSYDNE flags is required after issuing a BLOCK_WRITE command. (Check the CRC's error flags as required.)

Figure 129. MMC/SD Mode Single-Block Write Operation



17.3.6 MMC/SD Mode Single-Block Read Operation

In the MMC/SD mode, to read a single block of data from a memory card, use the following procedure. Figure 130 illustrates this procedure with some additional details. Note that the same block length must be defined in the MMC Controller and in the card.

1. Write the card's relative address to the argument registers. Load the upper address to MMCARGH - MMC Argument Register High and the lower address to MMCARGL - MMC Argument Register Low.
2. Send a SELECT/ DESELECT_CARD command via MMCCMD - MMC Command Register. This selects the addressed card and deselects the others.

3. Write the source start address to the argument registers. Load the upper address to MMCARGH - MMC Argument Register High and the lower address to MMCARGL - MMC Argument Register Low.
4. Send a SET_BLOCKLEN command via MMCCMD - MMC Command Register (if the block length is different than the length used in the previous operation).
5. Send a READ_SINGLE_BLOCK command via MMCCMD - MMC Command Register.
6. Use MMCST0 - MMC Status Register #0 to check for errors and to determine if a new byte has been successfully received. If all of the data has still not been read and previous byte is being received, go to step 7). If all of the data has been read, stop.
7. Read the new byte of data from MMCDRR - MMC Data Receive Register, and go to step 6).

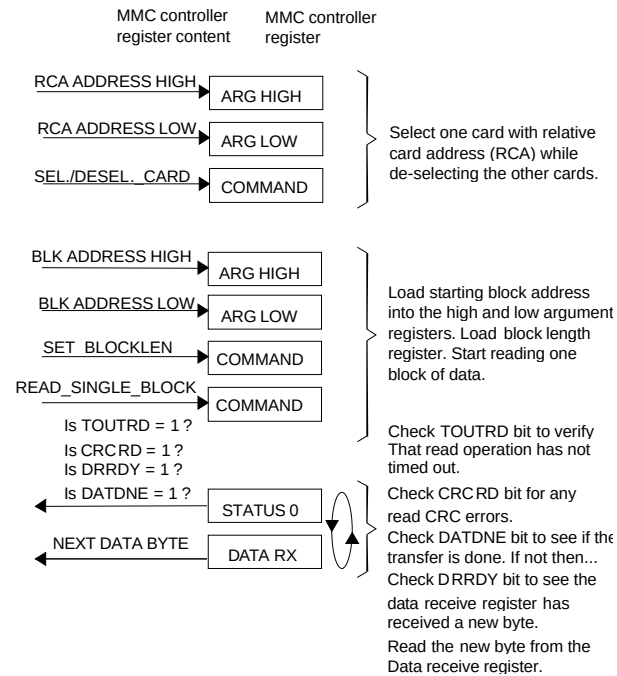
When using DMA transfer:

When using DMA transfer in Single-Block Read operation, observe the following items.

- o As for the size of the data transfer, the value in the MMCBLLEN - MMC Block Length Register is applied as the block length. Set the number of transfer blocks to the MMCNBLK - MMC Number of Blocks Register as "1" here due to Single-Block transfer.
- o At step 5), set the following before issuing a READ_SINGLE_BLOCK command.
 - n Set the transfer destination SDRAM relative address of the data to DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High. The address must be aligned in 4bytes units.
 - n Set '1' and '0' to DMAE bit and DIR bit respectively in the DMAMODE - DMA Mode Register.
 - n Set '1' to TRG bit in the DMATRG - DMA Trigger Register and trigger DMA.
- o At step 6), the DXRDY flag is not required to check. The DMA controller performs the operation of step 7) automatically.

Thus, no operation but waiting the DATDNE flag is required after issuing a READ_SINGLE_BLOCK command. (Check the error flags as required.)

Figure 130. MMC/SD Mode Single-Block Read Operation



17.3.7 MMC/SD Mode Multiple-Block Read Operation

In the MMC/SD mode, to read multiple blocks of data from a memory card, see the following procedure. The procedure is also illustrated in Figure 131 with some additional details. The same block length must be defined in the MMC Controller and in the card.

Note: The procedure in this section uses a STOP_TRANSMISSION command to end the block transfer. A multiple-block operation can terminate itself if you load MMCNBLK - MMC Number of Blocks Register with the exact number of blocks you want transferred. In addition, setting 0 to this register can continue to transfer until a STOP_TRANSMISSION command is issued.

1. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH - MMC Argument Register High and the lower address to the MMCARGL - MMC Argument Register Low.
2. Send a SET_BLOCKLEN command via MMCCMD - MMC Command Register.
3. Send a READ_MULT_BLOCKS command via MMCCMD - MMC Command Register.
4. Use MMCST0 - MMC Status Register #0 to check for errors and to determine if a new byte has been successfully received. If all of the data has still not been read, go to step 5). If all of the data has been read, go to step 6).
5. Read the new byte of data from MMCDRR - MMC Data Receive Register.
6. Send a STOP_TRANSMISSION command via MMCCMD - MMC Command Register.

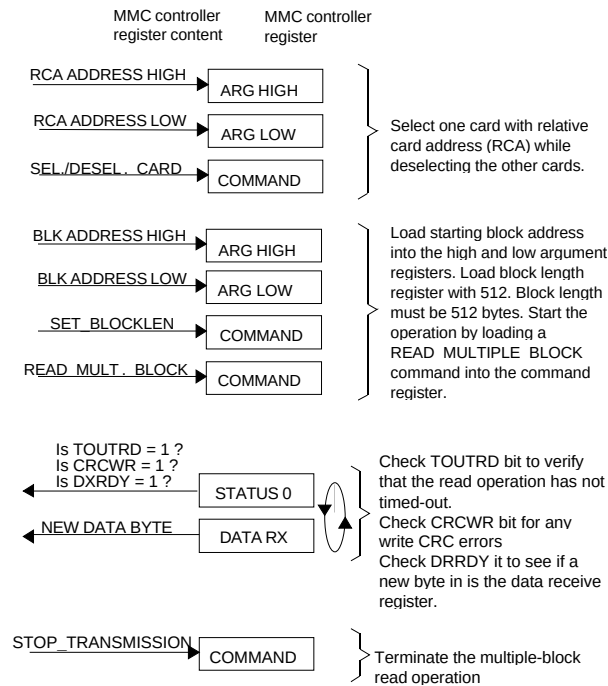
When using DMA transfer:

When using DMA transfer in Multi-Block Write operation, observe the following items.

- o As for the size of data transfer, the value in the MMCBLEN - MMC Block Length Register is applied as the block length. Set the number of transfer blocks to the MMCNBLK - MMC Number of Blocks Register.
- o At step 3), set the following before issuing a READ_MULT_BLOCKS command.
 - n Set the transfer destination SDRAM relative address of the data to the DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High. The address must be aligned in 4bytes units.
 - n Set '1' and '0' to DMAE bit and DIR bit respectively in the DMAMODE - DMA Mode Register.
 - n Set '1' to TRG bit in the DMATRG - DMA Trigger Register and trigger DMA.
- o At step 4), the DXRDY flag is not required to check. The DMA controller performs the operation of step 5) automatically.

Thus, no operation but waiting the DATDNE flag is required after issuing a READ_MULT_BLOCKS command. (Check the error flags as required.)

Figure 131. MMC/SD Mode Multiple-Block Read Operation



17.3.8 MMC/SD Mode Initialization

The general procedure for initializing the MMC Controller is given in the following steps. Details about the registers or register bit fields to be configured in the MMC/SD mode are in the subsections that follow the procedure.

1. Place the MMC Controller in its reset state by setting '1' to CMDRST bit and DATRST bit in the MMCCTL - MMC Function Clock Control Register. At this moment, other bits in MMCCTL - MMC Function Clock Control Register can be set.
2. Write the required values to other registers to complete the MMC Controller configuration.

3. Clear CMDRST bit and DATRST bit to release the MMC Controller from its reset state. Make sure you do not rewrite the values you wrote to the other bits of MMCCTL - MMC Function Clock Control Register in step 1.
4. Enable the CLK pin so that the memory clock is sent to the memory card. Enable the CLK pin by setting CLKEN bit in MMCCLK - MMC Control Register.

17.3.8.1 Initializing the MMC Control Register (MMCCTL)

When operating the MMC Controller in the MMC/SD mode, the bit fields named in the MMCCTL - MMC Function Clock Control Register affect the operation of the controller. The subsections that follow help you decide how to initialize each of control register's bit fields.

The DATED1-0 bits in MMCCTL - MMC Function Clock Control Register enable or disable edge detection on the DAT3 pin. If you enable edge detection and an edge is detected, the DATED flag bit of MMCST0 - MMC Status Register #0 is set. In addition, if EDATED bit = 1 in MMCIE - MMC Interrupt Enable Register, an interrupt request is generated.

The SPIEN bit of MMCCTL - MMC Function Clock Control Register determines whether the SPI mode is on. To enable the MMC/SD mode, turn the SPI mode off (SPIEN = 0).

In the MMC/SD mode, the MMC Controller must know how wide the data bus must be for the memory card that is connected. If an MMC is connected, specify a 1-bit data bus (WIDTH = 0). If an SD card is connected, specify a 4-bit data bus (WIDTH = 1).

To place the MMC Controller in its reset state and disable it, set the CMDRST and DATRST bits of MMCCTL - MMC Function Clock Control Register. The first step of the MMC Controller initialization process is to disable both sets of logic. When initialization is complete but before you enable the CLK pin, enable the MMC Controller by clearing the CMDRST and DATRST bits.

17.3.8.2 Initializing the Clock Controller Registers (MMCLK & MMCCLK)

The function clock is generated in the Clock Controller Module and configured by setting CMDV field (0x3088C [7:0]) of the CLKC's MMCFCLK register. The programmable clock divider in the CLKC module divides down the ARM clock to produce the function clock. The function clock is given by the following equation.

$$\text{Function clock frequency} = \text{ARM clock frequency} / (\text{CMDV} + 1)$$

A clock divider in the MMC Controller divides down the function clock to produce the memory clock. Load the divide-down value into the CLKRT field of MMCCLK - MMC Control Register. The resulting frequency is:

$$\text{Memory clock frequency} = \text{function clock frequency} / (2 * (\text{CLKRT} + 1))$$

The CLEN bit of MMCCLK - MMC Control Register determines whether the memory clock appears on the CLK pin anytime. If this bit is set as '0', the memory clock is not provided except when required.

For details of the clock setting, refer section "Function Clock and Memory Clock".

17.3.8.3 Initialize the Interrupt Mask Register (MMCIM)

The bits in MMCIE - MMC Interrupt Enable Register individually enable or disable the interrupt requests. Set one of the bits in MMCIE to enable the associated interrupt request. Clear one of the bits to disable the associated interrupt request. Load 0s into the bits not used in the MMC/SD mode.

17.3.8.4 Initialize the Time-Out Registers (MMCTOR and MMCTOD)

Specify the time-out period for responses using MMCTOR - MMC Response Time-Out Register and the time-out period for reading data using MMCTOD - MMC Data Read Time-Out Register.

When the MMC Controller sends a command to a memory card, it often must wait for a response. The controller can wait indefinitely or for up to 255 memory clock cycles. If you load 0 into MMCTOR - MMC Response Time-Out Register, the controller will wait for a response indefinitely. If you load a nonzero value into MMCTOR, the controller stops waiting after the specified number of cycles and then sets automatically a response time-out state to MMCST0 - MMC Status Register #0. If the associated interrupt request is enabled, the Controller also sends an interrupt request to the ARM.

When the MMC Controller requests data from a memory card, it can wait indefinitely for that data, or it can stop waiting after a programmable number of cycles. If you load 0 into MMCTOD - MMC Data Read Time-Out Register, the controller will wait indefinitely. If you load a nonzero value into MMCTOD, the controller will wait n memory clock cycles and then record a data-read time-out state in MMCST0 - MMC Status Register #0. If the associated interrupt request is enabled, the controller also sends an interrupt request to the ARM.

17.3.8.5 Initialize the Data Block Registers (MMCBLEN and MMCNBLK)

Specify the number of bytes in a data block in MMCBLEN - MMC Block Length Register and the number of blocks in a multiple-block transfer in MMCNBLK - MMC Number of Blocks Register.

In MMCBLEN - MMC Block Length Register, you must define the size for each block of data transferred between the MMC Controller and a memory card. The valid size depends on the type of read/write operation. A length of 0 bytes is prohibited.

For multiple-block transfers, you must specify how many blocks of data are to be transferred between the MMC Controller and a memory card. You can specify an infinite number of blocks by loading 0 into MMCNBLK - MMC Number of Blocks Register. When MMCNBLK = 0, the MMC Controller will continue to transfer data blocks until the transferring is stopped with a STOP_TRANSMISSION command. If you need a specific number of blocks transferred, load MMCNBLK - MMC Number of Blocks Register with a value from 1 through 65535.

17.3.9 Monitoring Activity in the MMC/SD Mode

This section describes registers and specific register bits that you can use to obtain the status of the MMC Controller in the MMC/SD mode. The status of the MMC Controller can be determined by reading the bits in MMCST0 - MMC Status Register #0 and MMCST1 - MMC Status Register #1.

17.3.9.1 Detecting Edges and Level Changes on the DAT3 Pin

Detecting edges. The MMC Controller sets the DATED flag of MMCST0 - MMC Status Register #0 if DAT3 edge detection is enabled (DATED1-0 are nonzero in MMCCTL - MMC Function Clock Control Register) and the specified edge is detected. The ARM can also be notified of the DAT3 by an interrupt if the interrupt request is enabled. (EDATED = 1 in MMCIE - MMC Interrupt Enable Register).

Detecting level changes. DAT3ST bit in the MMCST1 - MMC Status Register #1 tracks the signal level on the DAT3 pin.

17.3.9.2 Determining Whether New Data is Available in MMCDRR

The MMC Controller sets the DRRDY flag of MMCST0 - MMC Status Register #0 when new data arrives in the MMCDRR - MMC Data Receive Register. The ARM can also be notified of the event by an interrupt if the interrupt request is enabled. (EDRRDY = 1 in MMCIE - MMC Interrupt Enable Register).

17.3.9.3 Verifying that MMCDXR is Ready to Accept New Data

The MMC Controller sets the DXRDY flag of MMCST0 - MMC Status Register #0 when data leaves MMCDXR - MMC Data Transmit Register. The ARM can also be notified of the event by an interrupt if the interrupt request is enabled. (EDXRDY = 1 in MMCIE - MMC Interrupt Enable Register).

17.3.9.4 Checking for CRC Errors

The MMC Controller sets CRCRS/CRCRD/CRCWR bits in MMCST0 - MMC Status Register #0 in response to the corresponding CRC errors of command response/data read/data write. The ARM can also be notified of the CRC error by an interrupt if the interrupt request is enabled (ECRCRS/ECRCRD/ECRCWR = 1 in MMCIE - MMC Interrupt Enable Register).

17.3.9.5 Checking for Time-Out Events

The MMC Controller sets TOUTRS/TOUTRD bits in MMCST0 - MMC Status Register #0 in response to the corresponding command response/data read time-out event. The ARM can also be notified of the event by an interrupt if the interrupt request is enabled (ETOUTRS/ETOUTRD = 1 in MMCIE).

17.3.9.6 Determining When a Response/Command is Done

The MMC Controller sets the RSPDNE flag of MMCST0 - MMC Status Register #0 when the response is done (or, in the case of commands that do not require a response, when the command is done). The ARM can also be notified of the done condition by an interrupt if the interrupt request is enabled. (ERSPDNE = 1 in MMCIE - MMC Interrupt Enable Register).

17.3.9.7 Determining Whether the Memory Card is Busy

The card sends a busy signal either when waiting an R1b-type response or programming the last write data into its flash memory. The MMC Controller has two flags to tell you whether the memory card is sending a busy signal. The two flags are complements of each other:

- BSYDNE flag of MMCST0 - MMC Status Register #0 is set if the card did not send or is not sending a busy signal when the Controller is expecting a busy signal (BSYEXP = 1 in MMCCMD - MMC Command Register). The interrupt by this bit can be enabled by a corresponding interrupt enable bit (EBSYDNE bit in MMCIE - MMC Interrupt Enable Register).
- BUSY flag in MMCST1 - MMC Status Register #1 is set when a busy signal is received from the card.

17.3.9.8 Determining Whether a Data Transfer is Done

The MMC Controller sets the DATDNE flag when all the bytes of a data transfer have been transmitted/ received. You can poll this bit to determine when to stop writing to the data transmit register (for a write operation) or when to stop reading from the data receive register (for a read operation). The ARM can also be notified of the time-out event by an interrupt if the interrupt request is enabled (EDATDNE = 1 in MMCIE - MMC Interrupt Enable Register).

17.3.9.9 Checking For a Data Transmit Empty Condition

During transmission, a data value is passed from MMCDXR - MMC Data Transmit Register to the data transmit shift register. Then value is passed from this shift register to the memory card one bit at a time. The DXEMP bit in MMCST1 - MMC Status Register #1 indicates when this shift register is empty.

Typically, this bit is not used to control data transfers; rather, it is checked during recovery from an error condition. There is no interrupt associated with this bit.

17.3.9.10 Checking for a Data Receive Full Condition

During reception, the data receive shift register accepts a data value one bit at a time. Then the whole value is passed from this shift register to the MMCDRR - MMC Data Receive Register. The DRFUL bit in MMCST1 - MMC Status Register #1 indicates when this shift register is full; no new bits can be shifted in from the memory card.

Typically, this bit is not used to control data transfers; rather, it is checked during recovery from an error condition. There is no interrupt associated with this bit.

17.3.9.11 Checking the Status of the CLK Pin

Read CLKSTP bit in MMCST1 - MMC Status Register #1 to determine whether the memory clock has been stopped on the CLK pin.

17.3.9.12 Getting the Remaining Block Count During a Multiple-Block Transfer

During a transfer of multiple data blocks, MMCNBL - MMC Number of Blocks Counter tells you how many blocks are left to be transferred. You cannot write to this register.

17.4 SPI Mode

The interface of the MMC Controller can use an MMC/SD protocol or an SPI protocol. To use the SPI protocol, select the SPI mode by setting the SPIEN bit of MMCCTL - MMC Function Clock Control Register. In addition, make sure the memory card is in its SPI mode.

17.4.1 SPI Mode Interface

Figure 132 summarizes the SPI mode interface.

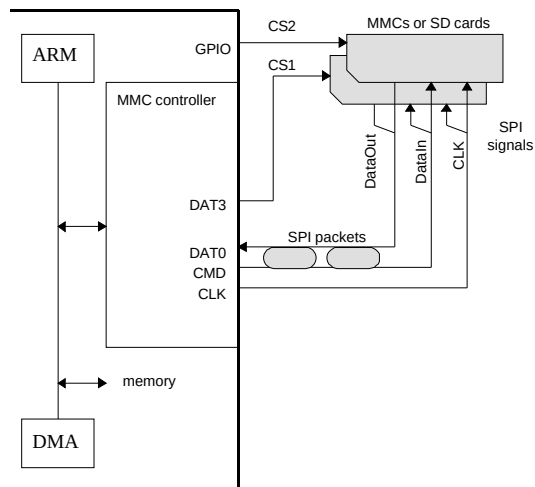
In the SPI mode:

- The MMC Controller supports one or more MultiMediaCards (MMCs) or Secure Digital Memory Cards (SD cards). The MMC Controller must access a given card using card select signals. One card select pin is available from the MMC Controller. If multiple cards are connected, use general-purpose I/O (GPIO) pins of the ARM to provide additional card select signals.

The following MMC Controller pins are used:

- DAT3 (for CS): This pin is available as a card select line. The signal on this pin can be controlled and monitored via MMC Controller register bits.
- DAT0 (for DataOut): The controller uses this pin to receive serial data and responses from the selected memory card.
- CMD (for DataIn): The controller uses this pin to transmit commands and serial data to the selected memory card.
- CLK: This pin provides a clock signal to time transfers between the Controller and the memory card(s).
- Because commands and data share the same line, they cannot overlap in time. Thus, the SPI mode does not support sequential transfer and multiple-block read/write operations. Only single-block read/ write operations can be performed.
- The SPI mode does not support reading the CID (Card Identification register) and CSD (Card Specific Data register) of the card. Read these registers in the MMC/SD mode.

Figure 132. SPI Mode Interface



17.4.2 SPI Mode Write Sequence

Figure 133 and Table 36. describe the write operation of the data to the memory card in the SPI mode. The same block length must be defined in the MMC Controller and in the card. In a successful write sequence:

- o The Controller sends a write command to the card.
- o The card sends a response to acknowledge the command.
- o The Controller sends a block of data to the card.
- o The card sends a response to indicate acceptance of the data.
- o The card sends BUSY tokens until all the data has been programmed into the flash memory inside the card.
- o The card select signal is active until the MMC Controller receives the data response.

Figure 133. SPI Mode Write Sequence

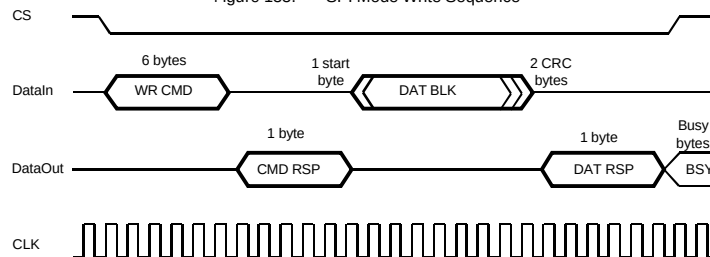


Table 36. SPI Mode Write Sequence

Portion of the Sequence	Description
WR CMD	Write command. A 6-byte WRITE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response. The card sends to the ARM a 1-byte command response of type R1 to acknowledge the WRITE_BLOCK command.
DAT BLK	Data block. The ARM writes a block of data to the card. The data content is preceded by one start block and is followed by two CRC bytes.
DAT RSP	Data response. The card sends to the ARM a data response token, indicating to the ARM whether the data has been accepted by the card or rejected due to a CRC error.
BSY	Busy token(s). The data response token is followed by 1-byte busy tokens until all of the data has been programmed into the flash memory inside the card.

17.4.3 SPI Mode Read Sequence

Figure 134 and Table 37. describe the read operation of the data from the memory card in the SPI mode. The same block length must be defined in the MMC Controller and in the card. In a successful read sequence:

- o The Controller sends a read command to the card.
- o The card sends a response to acknowledge the command.
- o The card sends a block of data or a data error token to the ARM.
- o The card select signal is active throughout the sequence.

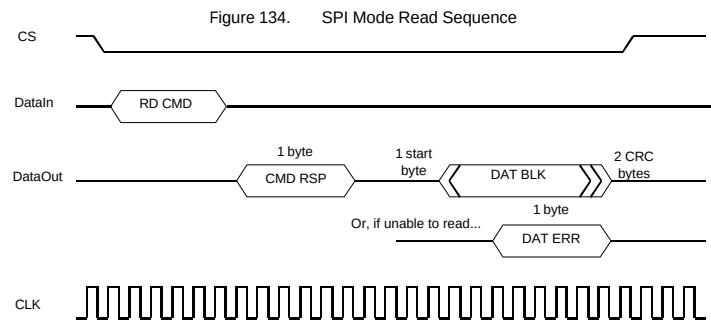


Table 37. SPI Mode Read Sequence

Portion of the Sequence	Description
RD CMD	Read command. A 6-byte READ_SINGLE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response. The card sends to the ARM a 1-byte command response of type R1 to acknowledge the READ_SINGLE_BLOCK command.
DAT BLK or DAT ERR	Data block or data error token. If the read operation is successful, the card sends a block of data to the ARM. The data content is preceded by one start block and is followed by two CRC bytes. If the read operation is not successful, the card instead sends a 1-byte data error token.

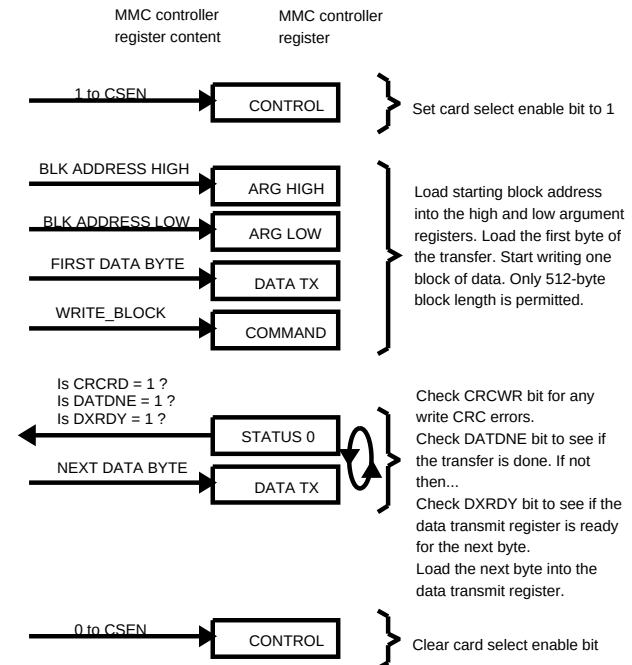
17.4.4 SPI Mode Single-Block Write Operation

In the SPI mode, to write a single block of data to a memory card, use the following procedure. Figure 135 illustrates this procedure with some additional details. The same block length must be defined in the MMC Controller and in the card.

1. Write 1 to the CSEN bit of MMCCTL - MMC Function Clock Control Register. This drives the card select signal low and selects a card. (If a general-purpose I/O pin is used as the card select pin, write to the appropriate GPIO bit.)
2. Write the destination start address to the argument registers. Load the upper address to the MMCARGH - MMC Argument Register High and the lower address to the MMCARGL - MMC Argument Register Low.
3. Write the first byte of the data block to MMCDXR - MMC Data Transmit Register.
4. Send the WRITE_BLOCK command to the card via MMCCMD - MMC Command Register.

5. Use MMCST0 - MMC Status Register #0 to check for errors and to determine when the byte has been successfully transmitted. If all of the data blocks have not been written and the previous byte has been transmitted, go to step 6). If the all of the data blocks have been written, stop. Also, deselect the card by clearing the CSEN bit.
6. Write the next byte of the data block to MMCDXR - MMC Data Transmit Register, and go to step 5.

Figure 135. SPI Mode Single-Block Write Operation

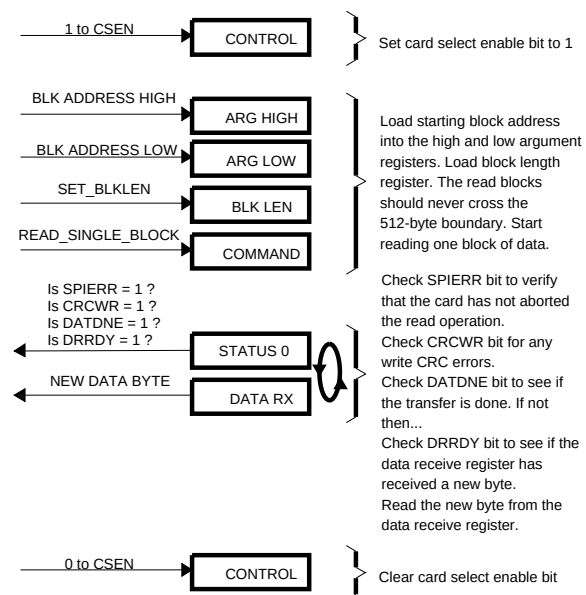


17.4.5 SPI Mode Single-Block Read Operation

In the SPI mode, to read a single block of data from a memory card, use the following procedure. The procedure is also illustrated in Figure 136 with some additional details. The same block length must be defined in the MMC Controller and in the card.

- Write 1 to the CSEN bit of MMCCTL - MMC Function Clock Control Register. This drives the card select signal low and selects a card. (If a general-purpose I/O pin is used as the card select pin, write to the appropriate GPIO bit.)
- Write the source start address to the argument registers. Load the upper address to the MMCARGH - MMC Argument Register High and the lower address to the MMCARGL - MMC Argument Register Low.
- Send a SET_BLOCKLEN command via MMCCMD - MMC Command Register (set at the first send and if the block length is different than that used in the previous operation).
- Send a READ_SINGLE_BLOCK command via MMCCMD - MMC Command Register.
- Use MMCST0 - MMC Status Register #0 to check for errors and to determine when a new byte has been successfully received. If all of the data blocks have not been read and a new byte has been received, go to step 6). If the all of the data blocks have been read, stop. Also, deselect the card by clearing the CSEN bit.
- Read the new byte of data from MMCDRR - MMC Data Receive Register, and go to step 5).

Figure 136. SPI Single-Block Read Operation
MMC controller register content MMC controller register



17.4.6 SPI Mode Initialization

The general procedure for initializing the MMC Controller is given in the following steps. Details about the registers or register bit fields to be configured in the SPI mode are in the subsections that follow the procedure.

- Place the MMC Controller in its reset state by setting CMDRST bit and DATRST bit in MMCCTL - MMC Function Clock Control Register. With the same register write operation, write the given values to other bits in MMCCTL.
- Write to other registers to complete the MMC Controller configuration.
- Clear CMDRST bit and DATRST bit in MMCCTL - MMC Function Clock Control Register to release the MMC Controller from its reset state. Make sure you do not rewrite the values you wrote to the other bits of MMCCTL - MMC Function Clock Control Register in step 1).
- Enable the clock of the CLK pin by setting CLKEN bit in MMCCLK - MMC Control Register so that the memory clock is sent to the memory card.

17.4.6.1 Initializing the MMC Control Register (MMCCTL)

When the MMC Controller is in the SPI mode, the bit fields in MMCCTL - MMC Function Clock Control Register set the operation of the Controller. The subsections that follow the figure help you decide how to initialize each of the fields.

The use of the DATED1-0 bits in MMCCTL - MMC Function Clock Control Register is applicable if the DAT3 pin is not used for the card select signal. The DATED control bits can be used for edge detection (rising/falling/both) on the DAT3 pin and setting of enable/ disable. If edge detection is enabled and an edge is detected, the DATED flag bit of MMCST0 - MMC Status Register #0 is set. In addition, if EDATED = 1 in MMCIE - MMC Interrupt Enable Register, an interrupt request is generated.

To enable the SPI mode, set the SPIEN bit in MMCCTL - MMC Function Clock Control Register

If you are using the DAT3 pin as a card select line, set CSEN (MMCCTL [4]) to select the memory card and clear CSEN to deselect the memory card. If you are using general-purpose I/O pins as card select lines, select and deselect the cards by writing to the appropriate GPIO bits.

To place the MMC Controller in its reset state and disable it, set CMDRST and DATRST bits. The first step of the MMC Controller initialization process is to set both bits and disable logic. When initialization is complete but before you enable the CLK pin, enable the MMC Controller by clearing the CMDRST and DATRST bits.

17.4.6.2 Initializing the Clock Control Registers (MMCLK and MMCCLK)

The function clock is generated in the Clock Controller Module (CLKC) and configured by setting CMDV field of the CLKC's MMCFCLK register. The programmable clock divider in the CLKC module divides down the ARM clock to produce the function clock. The function clock is given by the following equation.

$$\text{Function clock frequency} = \text{ARM clock frequency} / (\text{CMDV} + 1)$$

A clock divider in the MMC Controller divides down the function clock to produce the memory clock. Load the divide-down value into the CLKRT field of MMCCLK - MMC Control Register. The resulting frequency is:

$$\text{Memory clock frequency} = \text{function clock frequency} / (2^{(\text{CLKRT} + 1)})$$

The CLEN bit of MMCCLK - MMC Control Register determines whether the memory clock appears on the CLK pin anytime. If this bit is set as '0', the memory clock is not provided except when required.

17.4.6.3 Initialize the Interrupt Enable Register (MMCIM)

The bits in MMCIE - MMC Interrupt Enable Register individually enable or disable the interrupt requests. Set one of the bits in MMCIE - MMC Interrupt Enable Register to enable the associated interrupt request. Clear one of the bits to disable the associated interrupt request. Load 0s into the bits not used in the SPI mode.

17.4.6.4 Initialize the Block Length Register (MMBLEN)

In MMCBLEN - MMC Block Length Register, you must define the size for each block of data transferred between the MMC Controller and a memory card. The valid size depends on the type of read/write operation. A length of 0 bytes is prohibited.

17.4.7 Monitoring Activity in the SPI Mode

This section describes registers and specific register bits that you can use to obtain the status of the MMC Controller in the SPI mode. The status of the MMC Controller can be determined by reading the bits in MMCST0 - MMC Status Register #0 and MMCST1 - MMC Status Register #1.

17.4.7.1 Detecting Edges and Level Changes on the DAT3 Pin

Detecting edges. The MMC Controller sets the DATED flag of MMCST0 - MMC Status Register #0 if DAT3 edge detection is enabled (DATEG1-0 are nonzero in MMCCTL - MMC Function Clock Control Register) and the specified edge is detected. The ARM can also be notified of the DAT3 by an interrupt if the interrupt request is enabled (EDATED = 1 in MMCIE - MMC Interrupt Enable Register).

Detecting level changes. The DAT3ST bit of MMCST1 - MMC Status Register #1 tracks the signal level on the DAT3 pin.

17.4.7.2 Determining Whether New Data is Available in MMCDRR

The MMC Controller sets the DRRDY flag of MMCST0 - MMC Status Register #0 when new data arrives in MMCDRR - MMC Data Receive Register. The ARM can also be notified of the event by an interrupt if the interrupt request is enabled. (EDRRDY = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.3 Verifying that MMCDXR is Ready to Accept New Data

The MMC Controller sets the DXRDY flag of MMCST0 - MMC Status Register #0 when data leaves MMCDXR - MMC Data Transmit Register. The ARM can also be notified of the event by an interrupt if the interrupt request is enabled. (EDXRDY = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.4 Checking for an SPI Data Error

If an SPI read operation is successful, the card sends a block of data to the ARM. If the read operation is not successful, the card instead sends a 1-byte

data error token, which is stored in the MMCETOK - MMC SPI Error Token Register of the MMC Controller. When the token arrives, the SPIERR bit is set. The ARM can also be notified of the error by an interrupt if the interrupt request is enabled. (ESPIERR = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.5 Checking for CRC Errors

The MMC Controller sets CRCRS/CRCRD/CRCWR bits in MMCST0 - MMC Status Register #0 in response to the corresponding CRC errors of command response/data read/data write. The ARM can also be notified of the CRC error by an interrupt if the interrupt request is enabled (ECRCRS/ECRCRD/ECRCWR = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.6 Determining When a Response/Command is Done

The MMC Controller sets the RSPDNE flag in MMCST0 - MMC Status Register #0 when the response is done (or, in the case of commands that do not require a response, when the command is done). The ARM can also be notified of the done condition by an interrupt if the interrupt request is enabled (ERSPDNE = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.7 Determining When a Memory Card is Busy

The card sends a busy signal either when waiting an R1b-type response or programming the last write data into its flash memory. The MMC Controller has two flags to tell you whether the memory card is sending a busy signal. The two flags are complements of each other:

- BSYDNE flag of MMCST0 - MMC Status Register #0 is set if the card did not send or is not sending a busy signal when the Controller is expecting a busy signal (BSYEXP = 1 in MMCCMD - MMC Command Register). The interrupt by this bit can be enabled by a corresponding interrupt enable bit (EBSYDNE bit in MMCIE - MMC Interrupt Enable Register).
- BUSY flag MMCST1 - MMC Status Register #1 is set when a busy signal is received from the card.

17.4.7.8 Determining Whether a Data Transfer is Done

The MMC Controller sets the DATDNE flag in MMCST0 - MMC Status Register #0 when all the bytes of a data transfer have been transmitted/received. You can poll this bit to determine when to stop writing to the data transmit register (for a write operation) or when to stop reading from the data receive register (for a read operation). The ARM can also be notified of the time-out event by an interrupt if the interrupt request is enabled (EDATDNE = 1 in MMCIE - MMC Interrupt Enable Register).

17.4.7.9 Checking for a Data Transmit Empty Condition

During transmission, a data value is passed from MMCDXR - MMC Data Transmit Register to the data transmit shift register. Then value is passed from this shift register to the memory card one bit at a time. The DXEMP bit in MMCST1 - MMC Status Register #1 indicates when this shift register is empty.

Typically, this bit is not used to control data transfers; rather, it is checked during recovery from an error condition. There is no interrupt associated with this bit.

17.4.7.10 Checking for a Data Receive Full Condition

During reception, the data receive shift register accepts a data value one bit at a time. Then the whole value is passed from this shift register to the MMCDRR - MMC Data Receive Register. The DRFUL bit in MMCST1 - MMC Status Register #1 indicates that this shift register is full; no new bits can be shifted in from the memory card.

Typically, this bit is not used to control data transfers; rather, it is checked during recovery from an error condition. There is no interrupt associated with this bit.

17.4.7.11 Checking the Status of the CLK Pin

Read CLKSTP in MMCST1 - MMC Status Register #1 to determine whether the memory clock has been stopped on the CLK pin.

17.5 DMA Function

The MMC Controller has a block for Direct Memory Access. The DMA transfer is done using the bus for DMA. For the write operation to the SDRAM, the MMC Controller reads the data automatically from the memory card using MMCDRR - MMC Data Receive Register and sends it to the DMA controller via the bus for DMA. For the write operation to the memory card, the MMC Controller writes the data automatically from the DMA controller to the memory card via MMCDXR - MMC Data Transmit Register.

Case 1: From MMC Controller to SDRAM:

- Set the byte length, block length, argument and command of the MMC Controller (to determine the size of the DMA transfer).
- Set DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High. (The relative address from the top in the SDRAM area)
- Set 0 and 1 to DIR and DMAE respectively in DMAMODE - DMA Mode Register.
- Set 1 to TRG in DMATRG - DMA Trigger Register and send a request to the DMA controller.
- After completing DMA, the RUNST bit in DMASTAT1 - DMA Status Register #1 and the DATDNE bit in MMCST0 - MMC Status Register #0 are set to 0 and 1 respectively and then an interrupt is generated (when enabled).

Case 2: From SDRAM to MMC Controller:

- Set the byte length, block length, argument and command of the MMC Controller (to determine the size of the DMA transfer).
- Set DMAAD0 - DMA Address Low and DMAAD1 - DMA Address High. (The relative address from the top in the SDRAM area)
- Set 1 to DIR and DMAE in DMAMODE - DMA Mode Register.
- Set 1 to TRG in DMATRG - DMA Trigger Register and send a request to the DMA controller.
- After completing DMA, the RUNST bit in DMASTAT1 - DMA Status Register #1 and the DATDNE in MMCST0 - MMC Status Register #0 are set to 0 and 1 respectively and then an interrupt is generated (when enabled).

When the RUNST bit in DMASTAT1 - DMA Status Register #1 comes to 0, all of the bits in DMAMODE - DMA Mode Register return to default setting.

During the DMA transfer, the DCLR bit in MMCCMD - MMC Command Register is set as 1.

17.6 SDIO Card Function

To support the SDIO Card, the following features are included additionally to typical MMC/SD card controller (INT18).

- Read Wait Interrupt
- SDIO Card Interrupt

17.7 MMC/SD Card Interface Register Map (MMCSd)

Address	Register	Description
0003:0480	MMCCTL	MMC Function Clock Control Register
0003:0482	MMCCLK	MMC Control Register
0003:0484	MMCST0	MMC Status Register #0
0003:0486	MMCST1	MMC Status Register #1
0003:0488	MMCI E	MMC Interrupt Enable Register
0003:048A	MMCTOR	MMC Response Time-Out Register
0003:048C	MMCTOD	MMC Data Read Time-Out Register
0003:048E	MMCBLEN	MMC Block Length Register
0003:0490	MMCNBLK	MMC Number of Blocks Register
0003:0492	MMCNBLC	MMC Number of Blocks Counter
0003:0494	MMCDDR	MMC Data Receive Register
0003:0496	MMCDDR	MMC Data Transmit Register
0003:0498	MMCCMD	MMC Command Register
0003:049A	MMCARGL	MMC Argument Register Low
0003:049C	MMCARGH	MMC Argument Register High
0003:049E	MMCRSP0	MMC Response Register #0
0003:04A0	MMCRSP1	MMC Response Register #1
0003:04A2	MMCRSP2	MMC Response Register #2
0003:04A4	MMCRSP3	MMC Response Register #3
0003:04A6	MMCRSP4	MMC Response Register #4
0003:04A8	MMCRSP5	MMC Response Register #5
0003:04AA	MMCRSP6	MMC Response Register #6
0003:04AC	MMCRSP7	MMC Response Register #7
0003:04AE	MMCDSRSP	MMC SPI Data Response Register
0003:04B0	MMCETOK	MMC SPI Error Token Register
0003:04B2	MMCCIDX	MMC Command Index Register
0003:04B4	MMCKCK	MMC Test Register - Clock Start Phase Control
0003:04B6	MMCTORC	MMC Test register - TORC
0003:04B8	MMCTODC	MMC Test Register - TODC
0003:04BA	MMCBLNC	MMC Test Register - BLNC
0003:04BC	DMATRG	DMA Trigger Register
0003:04BE	DMAODE	DMA Mode Register
0003:04C0	DMAAD0	DMA Address Low
0003:04C2	DMAAD1	DMA Address High
0003:04C4	DMASTAT0	DMA Status Register #0
0003:04C6	DMASTAT1	DMA Status Register #1
0003:04C8	DMATOR	DMA Time-Out Register
0003:04CA	SDIOCTL	SDIO Control Register
0003:04CC	SDIOST0	SDIO Status Register #0
0003:04CE	SDIOEN	SDIO Enable Register
0003:04D0	SDIOIST	SDIO Status Interrupt Status Register

17.8 MMC/SD Card Interface Registers

17.8.1 MMCCTL - MMC Function Clock Control Register

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	PERMDX	PERMDR	RSV	DATED[1]	DATED[0]	SPIEN	CSEN	SPICRC	WIDTH	CMDRST	DATRST
R/W	-	-	-	-	-	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	PERMDX	0	R/W	Endian select Enable when writing » 0: Little Endian « » 1: Big Endian «
9	PERMDR	0	R/W	Endian select Enable when reading » 0: Little Endian « » 1: Big Endian «
8	RSV			Reserved
7:6	DATED	0	R/W	DAT3 Edge Detection Select Bits » 00: DAT3 edge detection is disabled « » 01: DAT3 rising edge detection is enabled « » 10: DAT3 falling edge detection is enabled « » 11: DAT3 dual edge detection is enabled «
5	SPIEN	0	R/W	SPI Mode Enable » 0: MMC mode select « » 1: SPI mode select «
4	CSEN	0	R/W	Card Select Enable when Writing (only in SPI mode) » 0: Drive CS (DAT3) high (inactive) to deselect card « » 1: Drive CS (DAT3) low (active) to select card «
3	SPI CRC	0	R/W	CRC Checking (only in SPI mode) » 0: Disable CRC checking « » 1: Enable CRC checking «
2	WIDTH	0	R/W	Data Bus Width (only in native MMC mode) » 0: Data bus has 1 bit (only DAT0 is used) « » 1: Data bus has 4 bits (DAT0-DAT3 are used) «
1	CMDRST	0	R/W	CMD (command) Logic Reset » 0: CMD line portion is enabled « » 1: CMD line portion is disabled and in reset state «
0	DATRST	0	R/W	DAT (data) Logic Reset » 0: DAT line portion is enabled « » 1: DAT line portion is disabled and in reset state «

17.8.2 MMCCLK - MMC Control Register

MMCCLK	0003:0482				offset:	0x02				default: 0x00FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CLEN	CLKRT[7]	CLKRT[6]	CLKRT[5]	CLKRT[4]	CLKRT[3]	CLKRT[2]	CLKRT[1]	CLKRT[0]			
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	0	1	1	1	1	1	1	1	1			

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	CLEN	0	R/W	Clock Enable » 0: Disable MMC Clock Output (MMC_CLK), set output low « » 1: Enable MMC Clock Output (MMC_CLK) «
7:0	CLKRT	255	R/W	MMC Clock Rate Divier rate for clock: Rate of MMC Clock Output $= (\text{Function Clock}) / (2 * (\text{CLKRT} + 1))$ The Function Clock rate is set in the MMCCLK register. The MMC clock is output on the pin named MMC_CLK.

17.8.3 MMCST0 - MMC Status Register #0

MMCST0	0003:0484				offset:	0x04				default: 0x0200									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	DATED	DRRDY	DXRDY	SPIERR	CRCRS	CRCRD	CRCWR	TOUTRS	TOUTRD	RSPDNE	BSYDNE	DATDNE			
R/W	-	-	-	-	R	R	R	R	R	R	R	R	R	R	R	R			
Default	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	DATED	0	R	Data Edge Detect Bit » 0: No DAT3 edge detected « » 1: DAT3 edge has been detected «
10	DRRDY	0	R	Data Receive Ready Bit » 0: MMCDRR not ready « » 1: MMCDRR ready - new data has arrived « and can be read by the CPU or by the DMA controller. DRRDY is cleared to 0 when the DAT logic is reset (DATRST=1), when a command is sent with data receive/transmit clear (DCLR=1), or when data is read from MMCDRR
9	DXRDY	1	R	Data Transmit Ready Bit » 0: MMCDXR not ready « » 1: MMCDXR ready - data in MMCDXR has been transmitted « MMCDXR can accept new data from the CPU or from the DMA controller. DXRDY is cleared to 0 when the DAT logic is reset (DATRST=1), when a command is sent with data receive/transmit clear (DCLR=1), or when data is written to MMCDXR.
8	SPIERR	0	R	Data Error Bit (in SPI mode) » 0: No data error is received « » 1: Data error token has been received «
7	CRCRS	0	R	Response CRC Error Bit » 0: No CRC error is detected « » 1: CRC error has been detected «
6	CRCRD	0	R	Read-Data CRC Error Bit » 0: No CRC error is detected « » 1: CRC error has been detected «
5	CRCWR	0	R	Write-Data CRC Error Bit » 0: No CRC error is detected « » 1: CRC error has been detected «
4	TOUTRS	0	R	Response Time-Out Bit » 0: No time-out event « » 1: A time-out event has occurred while the MMC controller was waiting fro a response to a command «
3	TOUTRD	0	R	Read-Data Time-Out Bit 0: No time-out event « 1: A time-out event has occurred while the MMC controller was waiting for data «
2	RSPDNE	0	R	Command/Response Done Bit If the command requires a response: 0: Response not done 1: Response fully received with no CRC error If no response required: 0: Command not done 1: Command has been sent
1	BSYDNE	0	R	Busy Done Bit » 0: The memory card is busy « » 1: The memory card is no longer sending a busy signal « The busy done bit is used for commands with R1b response (CMD38) to indicate card is busy. BSYDNE is set to indicate card is no longer busy.
0	DATDNE	0	R	Data Transfer Done Bit When reading from memory card with CPU or DMA: 0: Read operation not done 1: Data fully received with no CRC error When writing to memory card with CPU or DMA: 0: Write operation not done 1: Data fully transmitted

17.8.4 MMCST1 - MMC Status Register #1

MMCST1		0003:0486				offset: 0x06										default: 0x0002	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DAT3ST	DRFUL	DXEMP	CLKSTP	BUSY	
R/W	-	-	-	-	-	-	-	-	-	-	-	R	R	R	R	R	
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	1	0	

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	DAT3ST	0	R	DAT3 Status Bit » 0: Low signal level on DAT3 pin « » 1: High signal level on DAT3 pin «
3	DRFUL	0	R	DRRDY Data Receive Full » 0: MMCRRR (data receive register) is not full « » 1: MMCRRR (data receive register) is full «
2	DXEMP	0	R	DXRDY Data Transmit Empty » 0: MMCDCR (data transmit register) is not empty « » 1: MMCDCR (data transmit register) is empty «
1	CLKSTP	1	R	Clock Stop Status Bit » 0: CLK is active « » 1: CLK is held low « Possible reasons: Manual stop (CLKEN=0), data receive full condition, or data transmit empty condition.
0	BUSY	0	R	Busy Bit » 0: No busy signal detected from memory card « » 1: Busy signal detected (memory card is busy) «

17.8.5 MMCIE - MMC Interrupt Enable Register

MMCIE				0003:0488				offset: 0x08				default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	EDATED	EDRRDY	EDXRDY	ESPIERR	ECRCRS	ECRCRD	ECRCWR	ETOUTRS	ETOUTRD	ERSPDNE	EBSYDNE	EDATDNE
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	EDATED	0	R/W	DAT3 Edge Detect Interrupt Enable » 0: DAT3 edge detect interrupt is disabled « » 1: DAT3 edge detect interrupt is enabled «
10	EDRRDY	0	R/W	Data Receive Ready Interrupt Enable » 0: Data receive ready interrupt is disabled « » 1: Data receive ready interrupt is enabled «
9	EDXRDY	0	R/W	Data Transmit Ready Interrupt Enable » 0: Data transmit ready interrupt is disabled « » 1: Data transmit ready interrupt is enabled «
8	ESPIERR	0	R/W	SPI Data Error Interrupt Enable (in SPI mode) » 0: SPI data error interrupt is disabled « » 1: SPI data error interrupt is enabled «
7	ECRCRS	0	R/W	Response CRC Error Interrupt Enable » 0: Response CRC error interrupt is disabled « » 1: Response CRC error interrupt is enabled «
6	ECRCRD	0	R/W	Read-Data CRC Error Interrupt Enable » 0: Read-data CRC error interrupt is disabled « » 1: Read-data CRC error interrupt is enabled «
5	ECRCWR	0	R/W	Write-Data CRC Error Interrupt Enable » 0: Write-data CRC error interrupt is disabled « » 1: Write-data CRC error interrupt is enabled «
4	ETOUTRS	0	R/W	Response Time-Out Interrupt Enable » 0: Response time-out interrupt is disabled « » 1: Response time-out interrupt is enabled «
3	ETOUTRD	0	R/W	Read-Data Time-Out Interrupt Enable » 0: Read-data time-out interrupt is disabled « » 1: Read-data time-out interrupt is enabled «
2	ERSPDNE	0	R/W	Response/Command Done Interrupt Enable » 0: Response/command done interrupt is disabled « » 1: Response/command done interrupt is enabled «
1	EBSYDNE	0	R/W	Busy Done Interrupt Enable » 0: Busy done interrupt is disabled « » 1: Busy done interrupt is enabled «
0	EDATDNE	0	R/W	Data Transfer Done Interrupt Enable » 0: Data transfer done interrupt is disabled « » 1: Data transfer done interrupt is enabled «

17.8.6 MMCTOR - MMC Response Time-Out Register

MMCTOR					0003-048A		offset:	0x0A									default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TOR[7]	TOR[6]	TOR[5]	TOR[4]	TOR[3]	TOR[2]	TOR[1]	TOR[0]		
Default	-	-	-	-	-	-	-	-	R/W	R/W	R/W	r/d	r/d	r/d	R/W	r/d		

[illegible]

17.8.7 MMCTOD - MMC Data Read Time-Out Register

[illegible][illegible]

17.8.8 MMCBLLEN - MMC Block Length Register

MMCBLEN				0003:048E				offset:				0x0E								default:				0x0200			
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0											
Name	RSV	RSV	RSV	RSV	BLEN[11]	BLEN[10]	BLEN[9]	BLEN[8]	BLEN[7]	BLEN[6]	BLEN[5]	BLEN[4]	BLEN[3]	BLEN[2]	BLEN[1]	BLEN[0]											
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W											
Default	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0	0											

[illegible]

17.8.9 MMCNBLK - MMC Number of Blocks Register

[illegible][illegible]

17.8.10 MMCNBLC - MMC Number of Blocks Counter

[illegible][illegible]

17.8.11 MMCDRR - MMC Data Receive Register

[illegible][illegible]

17.8.12 MMCDXR - MMC Data Transmit Register

[illegible][illegible]

17.8.13 MMCCMD - MMC Command Register

MMCCMD																0003:0498		offset:		0x18																		default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																									
Name	DCLR	INT	DATA	STREAM	WRITE	RSPFMT[1]	RSPFMT[0]	BSYEXP	PPLEN	RSV	CMD[5]	CMD[4]	CMD[3]	CMD[2]	CMD[1]	CMD[0]																									
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																								
Default	0	0	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0																								

Bit	Name	Reset Value	R/W	Function
15	DCLR	0	R/W	<u>Data Receive/Transmit Clear</u> » 0: Do not clear data receive ready (DRRDY) and data transmit ready (DXRDY) bits « » 1: Clear DRRDY and DXRDY «
14	INT	0	R/W	<u>Initialization Clock Cycles</u> » 0: Do not insert initialization clock cycles « » 1: Insert initialization clock cycles « insert 80 CLK cycles before sending the command specified in the CMD field. These dummy clock cycles are required for resetting a card after power on.
13	DATA	0	R/W	<u>Data Transfer Indicator</u> » 0: No data transfer « » 1: A data transfer associated with the command «
12	STREAM	0	R/W	<u>Stream Enable</u> » 0: If DATA = 1, the transfer is a block transfer « » 1: If DATA = 1, the transfer is a stream transfer «
11	WRITE	0	R/W	<u>Write Enable</u> » 0: If DATA = 1, the data transfer is a read operation « » 1: If DATA = 1, the data transfer is a write operation «
10:9	RSPFMT	0	R/W	<u>Response Format (expected type of response to command)</u> 00: No response 01: R1/R4/R5/R6 Native mode: 48 bits with CRC SPI mode: 8 bits with CRC (if CRC enabled) 10: R2 Native mode: 136 bits with CRC SPI mode: 16 bits with CRC (if CRC enabled) 11: R3 Native mode: 48 bits with no CRC SPI mode: Not applicable
8	BSYEXP	0	R/W	<u>Busy Expected</u> » 0: No busy signal expected « » 1: Busy signal expected « If an R1b (R1 with busy) response is expected, set RSPFMT = 01 and BSYEXP = 1
7	PPLEN	0	R/W	<u>Push-Pull Enable</u> » 0: Push Pull Driver of CMD line is disabled « » 1: Push Pull Driver of CMD line is enabled «
6	RSV			Reserved
5:0	CMD	0	R/W	Command Index The command to be sent to the memory card CMD line. *Note: Writing to any bit in MMCCMD causes the MMC controller to send the command. Thus, the argument registers (ARGH and ARGL) have to be loaded properly before a write to MMCCMD. The CMD field specifies the type of command to be sent. The other fields in MMCCMD define the operation (command, response, additional activity) for the MMC controller. The contents of MMCCMD are retained until the next time MMCCMD is written.

17.8.14 MMCARGL - MMC Argument Register Low

MMCARGL																0003:049A		offset: 0x1A																		default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																					
Name	ARGL[15]	ARGL[14]	ARGL[13]	ARGL[12]	ARGL[11]	ARGL[10]	ARGL[9]	ARGL[8]	ARGL[7]	ARGL[6]	ARGL[5]	ARGL[4]	ARGL[3]	ARGL[2]	ARGL[1]	ARGL[0]																					
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																					
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																					

Bit	Name	Reset Value	R/W	Function
15:0	ARGL	0	R/W	<u>Argument Register, Low</u> Lower 16-bits of the command argument *Note: ARGH and ARGL are used for specifying the arguments to be sent with the command specified in CMD. Writing to MMCCMD causes the MMC controller to send a command; you must load ARGH and ARGL before writing the command to MMCCMD. Also, make sure you do not modify the argument registers while they are being used for an operation. The contents of the argument registers are retained until next time this register is written

17.8.15 MMCARGH - MMC Argument Register High

[illegible][illegible]

17.8.16 MMCRSP0 - MMC Response Register #0

[illegible][illegible]

17.8.17 MMCRSP1 - MMC Response Register #1

[illegible][illegible]

17.8.18 MMCRSP2 - MMC Response Register #2

[illegible][illegible]

17.8.19 MMCRSP3 - MMC Response Register #3

[illegible][illegible]

17.8.20 MMCRSP4 - MMC Response Register #4

[illegible][illegible]

17.8.21 MMCRSP5 - MMC Response Register #5

[illegible][illegible]

17.8.22 MMCRSP6 - MMC Response Register #6

[illegible][illegible]

17.8.23 MMCRSP7 - MMC Response Register #7

[illegible][illegible]

17.8.24 MMCDRSP - MMC SPI Data Response Register

[illegible][illegible]

17.8.25 MMCETOK - MMC SPI Error Token Register

MMCETOK									0003:04B0									offset:		0x30										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ETOK[7]	ETOK[6]	ETOK[5]	ETOK[4]	ETOK[3]	ETOK[2]	ETOK[1]	ETOK[0]																	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W																	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0																	

[illegible]

17.8.26 MMCCIDX - MMC Command Index Register

MMCCIDX				0003:04B2				offset:		0x32				default:				0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	START	XMT	CIDX[5]	CIDX[4]	CIDX[3]	CIDX[2]	CIDX[1]	CIDX[0]			
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	START	0	R/W	Start bit in command response, stored when it is received in the native mode.
6	XMT	0	R/W	Transmission bit in command response, stored when it is received in the native mode.
5:0	CIDX	0	R/W	<u>MMC Command Index</u> Command index bits of a response, stored when it is received in the native mode. NOTE: In R2, R3 response, command index bits are reserved and '111111' is received in the CIDX field.

17.8.27 MMCCCKC – MMC Test Register - Clock Start Phase Control

MMCCCKC				0003:04B4				offset:		0x34								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CPHA[7]	CPHA[6]	CPHA[5]	CPHA[4]	CPHA[3]	CPHA[2]	CPHA[1]	CPHA[0]					
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved (test bits, do not use)
				CLK start phase control
				If CPHA = ClockRate(in MMCCCLK),
				when MMCCCLK register is set up, MMC clock start immediately.
7:0	CPHA	0	R/W	If CPHA < ClockRate when ClockRate - CPHA = N, MMC clock start N*Tcycle(internal function_clock) delay. If CPHA > ClockRate when CPHA - ClockRate = M, MMC clock start (256-M)*Tcycle(internal function_clock) delay.

17.8.28 MMCTORC - MMC Test register – TORC

[illegible][illegible]

17.8.29 MMCTODC - MMC Test register – TODC

[illegible][illegible]

17.8.30 MMCBLNC - MMC Test register – BLNC

MMCBLCNC									0003:04BA offset: 0x3A									default: 0x0001		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BLCN[7]	BLCN[6]	BLCN[5]	BLCN[4]	BLCN[3]	BLCN[2]	BLCN[1]	BLCN[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	1				

[illegible]

17.8.31 DMATRG - DMA Trigger Register

DMATRG		0003:04BC				offset:		0x3C		default: 0x0000									
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BRK	TRG		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W	W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0		

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	BRK	0	W	<u>Break - DMA Transfer suspend</u> » 0: no effect « » 1: DMA transfer break « The current DMA transfer is suspended when 1 is written. Bit is cleared to 0 immediately after writing. Since the actual end of transfer is several cycles after the bit is set, please resume transfer only after confirming that transfer has completely ended; i.e., the RUNST flag of the DMASTAT1 register has been cleared to 0.
0	TRG	0	W	<u>DMA Transfer Start Trigger</u> » 0: no effect « » 1: DMA transfer start « The current DMA transfer is started when 1 is written. Bit is cleared to 0 immediately after writing.

17.8.32 DMAMODE - DMA Mode Register

DMAMODE		0003:04BE				offset:		0x3E		default: 0x0000									
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	DTIME	0	R/W	<u>DMA Transfer Timeout Interrupt Enable</u> » 0: DMA transfer timeout interrupt is disabled « » 1: DMA transfer timeout interrupt is enabled « Note: This bit is only valid in ES2 and thereafter. This bit is reserved in ES1.
13	DMAE	0	R/W	<u>DMA Enable</u> » 0: Disable DMA « » 1: Enable DMA «
12	DIR	0	R/W	<u>DMA Transfer Direction</u> » 0: MMCIF to SDRAM « » 1: SDRAM to MMCIF «
11	DPRMDX	0	R/W	<u>Word Swap Enable When Writing</u> » 0: Do not word-swap 32-bit wide data when writing data from the SDRAMC to the card (SDRAMC_out[31:0] -> CARD_in[31:0]) « » 1: Word-swap 32-bit wide data when writing data from the SDRAMC to the card (SDRAMC_out[31:0] -> CARD_in[15:0,31:16]) «
10	DPRMDR	0	R/W	<u>Word Swap Enable When Reading</u> » 0: Do not word-swap 32-bit wide data when reading data from the card to SDRAM (CARD_out[31:0] -> SDRAM_in[31:0]) « » 1: Word-swap 32-bit wide data when reading data from the card to the SDRAM (CARD_out[31:0] -> SDRAM_in[15:0,31:16]) «
9:0	RSV			Reserved

17.8.33 DMAAD0 - DMA Address Low

[illegible][illegible]

17.8.34 DMAAD1 - DMA Address High

[illegible][illegible]

17.8.35 DMASTAT0 - DMA Status Register #0

[illegible][illegible]

17.8.36 DMASTAT1 - DMA Status Register #1

[illegible][illegible]

17.8.37 DMATOR - DMA Time-Out Register

[illegible][illegible]

17.8.38 SDIOCTL - SDIO Control Register

[illegible][illegible]

17.8.39 SDIOST0 - SDIO Status Register #0

[illegible][illegible]

17.8.40 SDIOIEN - SDIO Enable Register

SDIOIEN		0003.04CE				offset:		0x4E								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RWSE	I0INTEN			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0			

[illegible]

17.8.41 SDIOIST - SDIO Status Interrupt Status Register

[illegible][illegible]

18 USB Interface

18.1 Introduction to USB Interface

DM270 incorporates a USB 1.1 compliant device controller. This chapter describes the USB module that can be found in the DM270.

18.2 References

Refer to the following sources for information on the USB specification. All of the following documents can be downloaded from <http://www.usb.org>

- o Universal Serial Bus Specification Revision 1.1
- o Mass Storage Class Control/Bulk/Interrupt (CBI) Transport Rev 1.0
- o Still Image Capture Device Definition Revision 1.0

18.3 Specifications

The DM270's USB interface has the following characteristics

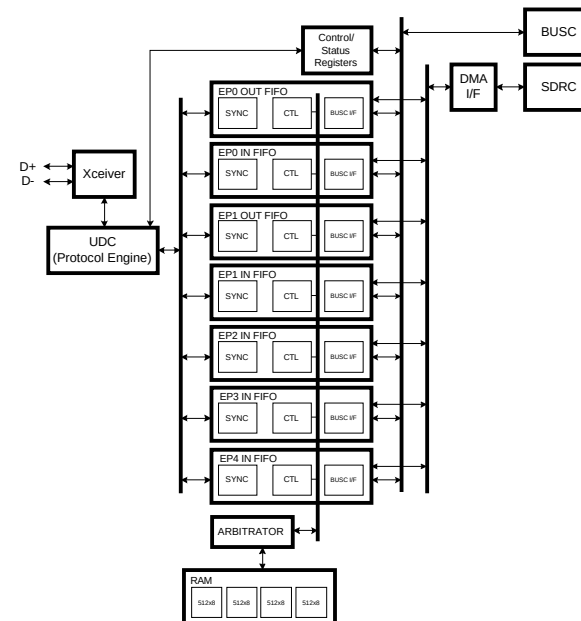
- o Compliant with USB Specification Revision 1.1
- o Involve UDC (USB Device Controller), the Protocol Engine
- o Support only for Full Speed Device
- o Built-in USB Xceiver
- o Built-in 2K RAM for Endpoint FIFO
- o Programmable MaxPktSize
- o Understanding and Decoding of Standard Requests Except For GetDescriptor, SetDescriptor and SynchFrame
- o Support of Device Remote Wakeup Feature
- o DMA transferring capability
- o Support for 8-bit as well 16-bit access to the End-Point FIFOs. (default access is 16-bit)

18.4 Block Diagram

Figure 137 shows the block diagram of the USB module. This module has a built-in transceiver and its own 2K bytes of RAM for the FIFOs. The module has a total of seven FIFOs, each associated with an Endpoint. The structure of each FIFO is the same except for the direction and the buffer size. The maximum buffer size is 256 bytes for EP0 and EP2 while it is 2048 bytes for EP1 and EP3 and EP4. Each FIFO buffer is allocated onto a unified RAM. User can program the base address and buffer size.

The ARM (through the Bus Controller) or the DMA interface accesses FIFO read/write port registers to read from or write to FIFOs.

Figure 137. USB module block diagram



18.5 Initialization Flow

The USB initialization must include the following points:

- o Set the size of the FIFOs
- o Set the base address within the dedicated RAM of the FIFOs
- o Set the threshold value used to generate an interrupt
- o Enable the interrupt that will be used
- o Perform a soft reset by writing a '1' in the USBRST - USB Reset register.
- o Initialize the USB core by writing a succession of configuration bytes into the USBCFG register.

After the USB initialization is successful, the USB device is ready for communication with the host

Universal Serial Bus Interface Register Map (USB) lists each USB register and the register's address within the ARM memory map. All of these registers are described in the sub-sections below.

Normally, when the ARM accesses a peripheral memory mapped register, it takes only 1 cycle. However, it takes 2+n cycles for the ARM to access (read or write) any of the USB registers, except EPxORDT and EPxIWDT. It takes 2 cycles to access the EPxORDT and EPxIWDT registers. The value 'n' is variable and depends on the current access condition.

18.6 FIFO Size Selection

The size of each FIFO can be set in the following (EPxxSZ) register:

- o EP0OSZ - USB Endpoint 0 OUT Buffer Size
- o EP0ISZ - USB Endpoint 0 IN Buffer Size
- o EP1OSZ - USB Endpoint 1 OUT Buffer Size
- o EP1ISZ - USB Endpoint 1 IN Buffer Size
- o EP2ISZ - USB Endpoint 2 IN Buffer Size
- o EP3ISZ - USB Endpoint 3 IN Buffer Size
- o EP4ISZ - USB Endpoint 4 IN Buffer Size

18.7 FIFO Base Address Selection

The RAM base address of each FIFO buffer is specified in the following (EPxxAD) register. This address must be set so as not to be duplicated among FIFOs. Since the built-in RAM is 2 Kbytes, this address must be assigned within an area of 2K (0000h-07FFh).

- o EP0OAD - USB Endpoint 0 OUT Buffer Address
- o EP0IAD - USB Endpoint 0 IN Buffer Address
- o EP1OAD - USB Endpoint 1 OUT Buffer Address
- o EP1IAD - USB Endpoint 1 IN Buffer Address
- o EP2IAD - USB Endpoint 2 IN Buffer Address
- o EP3IAD - USB Endpoint 3 IN Buffer Address
- o EP4IAD - USB Endpoint 4 IN Buffer Address

18.8 Interrupt Threshold

The threshold value of the FIFO data counter at which interrupt occurs is set for each FIFO in the following EPxxIRQSZ register.

- o EP0OIRQSZ - USB Endpoint 0 OUT Interrupt Size Threshold
- o EP0IIRQSZ - USB Endpoint 0 IN Interrupt Size
- o EP1OIRQSZ - USB Endpoint 1 OUT Interrupt Size Threshold
- o EP1IIRQSZ - USB Endpoint 1 IN Interrupt Size
- o EP2IIRQSZ - USB Endpoint 2 IN Interrupt Size Threshold
- o EP3IIRQSZ - USB Endpoint 3 IN Interrupt Size
- o EP4IIRQSZ - USB Endpoint 4 IN Interrupt Size

In IN endpoint (except ISO) FIFOs, when transfer ends by ACK, interrupt occurs if the number of bytes of data in the FIFO is less than or equal to the value set here. In ISO endpoints, when an IN transaction ends, interrupt occurs if the number of bytes of data in the FIFO is less than or equal to the value set here.

In OUT endpoints, when an OUT transaction ends by ACK, interrupt occurs if the number of bytes of data in the FIFO is more than or equal to the value set here. If this value is set to 0, it is possible to tell if there was reception of a packet of any number of transfer bytes including a short/null packet.

18.9 Interrupt control

USBINTMSK - USB Interrupt Mask register can be configured to individually enable or disable the end point IN/ OUT interrupts. To enable the interrupt, set each bit to one. To disable the interrupt, clear each bit to zero.

When an interrupt is generated by the USB, the source of it can be determined by reading the USBINTST - USB Interrupt Status/Reset register. The interrupt flag is automatically cleared when this register is read. The INT14 interrupt in the interrupt controller is used as USB interrupt (see section "Interrupt Controller").

18.10 Protocol Engine Reset

Writing a 1 into the MIRST bit of the USBRST - USB Reset register results in software reset of protocol engine.

18.11 Device Configuration

Figure 138, shows the configurations that USB module supports. There are two configurations that are supported in this module. CfgA supports all transport protocol modes CBI (Control-Bulk-Interrupt) Transport defined in Mass Storage Class like CBI (Control-Bulk-Interrupt) and Bulk-only sub-class. CfgB is provided to support the configuration defined in "Still Image Capture Device Definition".

You can configure following properties at device initialization;

- 1) Whether or not to use each Configuration
- 2) Each configuration number. A configuration number of 1 or 2 is assigned to CfgA and CfgB. If either is not used, set the unused configuration to 0, and set the used configuration to 1.
- 3) Whether or not to use each Endpoint for each AlternateSetting. The device does not respond to access from the host to an endpoint that is set to unused.
- 4) MaxPktSize of each Endpoint for each AlternateSetting

The above device initialization is done by writing the initialization data to the USBCFG - USB Configuration register. It must be done whenever the cable is attached. The format of the actual initialization data is described in the next subsection.

User cannot change the number of Interface, the interface value, the maximum number of AlternateSettings of each Interface, Endpoint number and Endpoint type.

Figure 138. Device Configuration

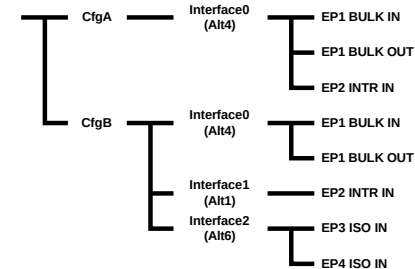


Table 38. Summary of possible Configurations

EP #	Dir	Type	Max FIFO Size
EP0	IN/OUT	CONTROL	256 bytes
EP1	IN/OUT	BULK	2048 bytes
EP2	IN	INTR	256 bytes
EP3	IN	ISO	2048 bytes
EP4	IN	ISO	2048 bytes

Each endpoint has a dedicated FIFO for IN and for OUT, and the endpoint is accessed from ARM via these FIFOs. The main USB module has 2 Kbytes of RAM built in, and each FIFO has a dedicated buffer within this memory area. User can set the RAM base address and buffer size for each FIFO in the memory size. For EP1 IN/OUT, EP3 IN, and EP4 IN a maximum FIFO size of 2048 bytes can be set (other FIFOs are a maximum of 256 bytes).

18.12 Device Initialization

The length of initialization data is 26 half-words (a half-word is 16 bit). You must write the 26 half-words from #0 to #25 into the USBCFG - USB Configuration register. When the device has been initialized successfully, the MIDID field in USBST - USB Status register is set to 1. The device will respond to control requests/commands from host only when the device has been successfully initialized.

The half-words that need to be written at initialization are described below in chronological order:

Table 39. Initialization half-words order

Half-word Number	Half-word Name	Function
#0	Configuration a and endpoint0	Associate Configuration number with CfgA and CfgB Configures endpoint 0
#1	Intf0-Alt0-EP1	Setting of alternate setting 0 of endpoint 1 in configuration CfgA
#2	Intf0-Alt1-EP1	Setting of alternate setting 1 of endpoint 1 in configuration CfgA
#3	Intf0-Alt2-EP1	Setting of alternate setting 2 of endpoint 1 in configuration CfgA
#4	Intf0-Alt3-EP1	Setting of alternate setting 3 of endpoint 1 in configuration CfgA
#5	Intf0-Alt0-EP2	Setting of alternate setting 0 of endpoint 2 in configuration CfgA
#6	Intf0-Alt1-EP2	Setting of alternate setting 1 of endpoint 2 in configuration CfgA
#7	Intf0-Alt2-EP2	Setting of alternate setting 2 of endpoint 2 in configuration CfgA
#8	Intf0-Alt3-EP2	Setting of alternate setting 3 of endpoint 2 in configuration CfgA
#9	Intf0-Alt0-EP1	Setting of alternate setting 0 of endpoint 1 in configuration CfgB
#10	Intf0-Alt1-EP1	Setting of alternate setting 1 of endpoint 1 in configuration CfgB
#11	Intf0-Alt2-EP1	Setting of alternate setting 2 of endpoint 1 in configuration CfgB
#12	Intf0-Alt3-EP1	Setting of alternate setting 3 of endpoint 1 in configuration CfgB
#13	Intf1-Alt0-EP2	Setting of alternate setting 0 of endpoint 2 in configuration CfgB
#14	Intf2-Alt0-EP3	Setting of alternate setting 0 of endpoint 3 in configuration CfgB
#15	Intf2-Alt1-EP3	Setting of alternate setting 1 of endpoint 3 in configuration CfgB
#16	Intf2-Alt2-EP3	Setting of alternate setting 2 of endpoint 3 in configuration CfgB
#17	Intf2-Alt3-EP3	Setting of alternate setting 3 of endpoint 3 in configuration CfgB
#18	Intf2-Alt4-EP3	Setting of alternate setting 4 of endpoint 3 in configuration CfgB
#19	Intf2-Alt5-EP3	Setting of alternate setting 5 of endpoint 3 in configuration CfgB
#20	Intf2-Alt0-EP4	Setting of alternate setting 0 of endpoint 4 in configuration CfgB
#21	Intf2-Alt1-EP4	Setting of alternate setting 1 of endpoint 4 in configuration CfgB
#22	Intf2-Alt2-EP4	Setting of alternate setting 2 of endpoint 4 in configuration CfgB
#23	Intf2-Alt3-EP4	Setting of alternate setting 3 of endpoint 4 in configuration CfgB
#24	Intf2-Alt4-EP4	Setting of alternate setting 4 of endpoint 4 in configuration CfgB
#25	Intf2-Alt5-EP4	Setting of alternate setting 5 of endpoint 4 in configuration CfgB

Table 40. Half-word #0 - Setting of Configuration and Endpoint0

Bits	Description
15-14	Configuration Number of CfgA
13-12	Configuration Number of CfgB
11-10	reserved
9-0	MaxPktSize of EP0

The configuration number can be "00", "01" or "10". The value "00" indicates that the configuration is not used. The configuration numbers for CfgA and CfgB cannot be identical if not "00".

Table 41. Half-word 1 to 4 - Setting of Endpoint1 in CfgA

Bits	Description
15	Enable for EP1 IN (1:enable, 0:disable)
14	Enable for EP1 OUT (1:enable, 0:disable)
13-10	Reserved
9-0	MaxPktSize of EP1

If the enable bit for an endpoint is set to 0, the functionality of the specified endpoint is disabled. Device does not respond to the transaction for the disabled endpoint.

Table 42. Half-words 5 to 8 - Setting of Endpoint2 in CfgA

Bits	Description
15	Enable for EP2 IN (1:enable, 0:disable)
14-10	reserved
9-0	MaxPktSize of EP2

Table 43. Half-word 9 to 12 - Setting of Endpoint1 in CfgB

Bits	Description
15	Enable for EP1 IN (1:enable, 0:disable)
14	Enable for EP1 OUT (1:enable, 0:disable)
13-10	Reserved
9-0	MaxPktSize of EP1

Table 44. Half-word 13 - Setting of Endpoint2 in CfgB

Bits	Description
15	Enable for EP2 IN (1:enable, 0:disable)
14-10	Reserved
9-0	MaxPktSize of EP2

Table 45. Half-word 14 to 19 - Setting of Endpoint3 in CfgB

Bits	Description
15	Enable for EP3 IN (1:enable, 0:disable)
14-10	Reserved
9-0	MaxPktSize of EP3

Table 46. Half-word 20 to 25 - Setting of Endpoint4 in CfgB

Bits	Description
15	Enable for EP 4 IN (1:enable, 0:disable)
14-10	Reserved
9-0	MaxPktSize of EP 4

18.13 Power Management

When the USB bus is suspended, all the clocks used in USB module except a clock used for resume detection can be cut-off by the register USBRSM - USB Resume for power saving. The clocks will be automatically activated as soon as the resume signal is detected.

If USB is not used, you can completely disable all the USB clocks by a register in CLKC module (Refer section "ARM Power Saving mode" for details). In that case, more power saving can be achieved by connecting VDD pins for USB I/O to GND.

18.14 USB Endpoint

The data to be sent is stored into the out port FIFO by writing their value inside the following EPxORDT register:

- EP0ORDT - USB Endpoint 0 OUT Read Data Port
- EP0IWDT - USB Endpoint 0 IN Write Data Port
- EP1ORDT - USB Endpoint 1 OUT Read Data Port
- EP1IWDT - USB Endpoint 1 IN Write Data Port
- EP2IWDT - USB Endpoint 2 IN Read Data Port
- EP3IWDT - USB Endpoint 3 IN Write Data Port
- EP4IWDT - USB Endpoint 4 IN Write Data Port

The number of bytes available in each buffer can be known by reading the following EPxxREST registers.

- EP0OREST - USB Endpoint 0 OUT Data Remaining
- EP1OREST - USB Endpoint 1 OUT Data Remaining
- EP2IREST - USB Endpoint 2 IN Data Remaining

When the OUT FIFO is full and more data is received from the host, the USB device sends NAKs for any new data received from the host. This acts as an indication to the host that the device has not accepted the recently sent data. This prevents loss of data by the device when the OUT FIFO is full

18.15 USB status

Following Read Only registers can be used to know the actual status of the USB module.

- USBST - USB Status
- USBFRM - USB Frame.

The register USBALT - USB Alternate indicates the current configuration and alternate setting value of each interface assigned by the HOST. Reset at USB Bus Reset, a soft reset by USBRST or when the cable is attached or detached. And the alternate setting values (bit14-12, bit10-8, bit6-4) are reset at receiving a SetConfiguration request.

18.16 Endpoint Control

Refer registers listed below for managing and controlling the USB Endpoints.

- EP0OCTL - USB Endpoint 0 OUT Control
- EP0ICTL - USB Endpoint 0 IN Control
- EP1OCTL - USB Endpoint 1 OUT Control
- EP1ICTL - USB Endpoint 1 IN Control
- EP2ICTL - USB Endpoint 2 IN Control
- EP3ICTL - USB Endpoint 3 IN Control
- EP4ICTL - USB Endpoint 4 IN Control

18.16.1 FIFO Status

Following is a list of registers that can be used to check the FIFO status for an Endpoint.

- EP0OST - USB Endpoint 0 OUT Status
- EP0IST - USB Endpoint 0 IN Status
- EP1OST - USB Endpoint 1 OUT Status
- EP1IST - USB Endpoint 1 IN Status
- EP2IST - USB Endpoint 2 IN Status
- EP3IST - USB Endpoint 3 IN Status
- EP4IST - USB Endpoint 4 IN Status

USBCTL - USB Control is used to control the rate feedback mode of EP2.

18.16.2 DMA Control

The internal DMA controller can be used to transfer data between SDRAM and USB endpoints without ARM intervention.

IMPORTANT: The SDRAM controller has two DMA channels, which are used to perform DMAs involving SDRAM as source or destination. A DMA channel between SDRAM and USB must be selected prior to triggering USB DMA. The DMA channel is selected using the REFCTL(0x309A8) register (See "Memory Traffic Controller (MTC)" Chapter). Also the DMA channel SDRAM access priority can be changed using the SDPRTY1-10 registers (see section "SDRAM priority access" in chapter "Memory Traffic Controller (MTC)" for details)

A uni-direction DMA transmission from/to one endpoint FIFO is allowed at one time. You have to choose one endpoint as the DMA target endpoint preceding the DMA transfer. You should also specify the SDRAM DMA source/destination

address and transfer length ([DMAAD0](#), [DMAAD1](#), [DMASZ](#)). When you write a '1' to the DSTART bit in [DMATRG](#) register, the DMA transfer starts. After the transmission of the specified amount of data is completed, the ARM is notified that the DMA transfer is complete by the USB interrupt. The USB interrupt is INT14. Refer to the "Interrupt Controller" module for information on INT14 in the Interrupt Controller.

For OUT endpoints, data stored in OUT FIFO can be transferred to SDRAM. If no data is received in FIFO, setting the DSTART bit does not start a DMA transfer. Also, if the number of data in OUT FIFO is less than DMA transfer length specified by user, the actual DMA transfer length is automatically clipped to the actual length.

For IN endpoints, data to be sent to the HOST can be transferred from SDRAM. If there is no space for the IN FIFO, setting DON bit has no effect. Also the room for the FIFO is less than the specified DMA length, the actual DMA transfer is clipped by the actual room length.

You can also change DMA endianness. Options are LSByte first, MSByte first or byte swap in half-word. Endianness is specified by the bit fields DEP and DENDI in the [DMACTL](#) register.

18.17 Universal Serial Bus Interface Register Map (USB)

Address	Register	Description
0003:0A00	EP0O0AD	Endpoint 0 OUT Buffer Address
0003:0A02	EP0OSZ	Endpoint 0 OUT Buffer Size
0003:0A04	EP0OCTL	Endpoint 0 OUT Control
0003:0A06	EP0OIRQSZ	Endpoint 0 OUT Interrupt Threshold
0003:0A08	EP0ORDT	Endpoint 0 OUT Read Data Port
0003:0A0C	EP0OREST	Endpoint 0 OUT Data Remaining
0003:0A0E	EP0OST	Endpoint 0 OUT Status
0003:0A90	EP0IAD	Endpoint 0 IN Buffer Address
0003:0A92	EP0ISZ	Endpoint 0 IN Buffer Size
0003:0A94	EP0ICTL	Endpoint 0 IN Control
0003:0A96	EP0IIRQSZ	Endpoint 0 IN Interrupt Threshold
0003:0A98	EP0IWDI	Endpoint 0 IN Write Data Port
0003:0A9C	EP0IREST	Endpoint 0 IN Remainder
0003:0A9E	EP0IST	Endpoint 0 IN Status
0003:0AA0	EP1O0AD	Endpoint 1 OUT Buffer Address
0003:0AA2	EP1OSZ	Endpoint 1 OUT Buffer Size
0003:0AA4	EP1OCTL	Endpoint 1 OUT Control
0003:0AA6	EP1OIRQSZ	Endpoint 1 OUT Interrupt Threshold
0003:0AAA	EP1ORDT	Endpoint 1 OUT Read Data Port
0003:0AAC	EP1OREST	Endpoint 1 OUT Data Remaining
0003:0AAE	EP1OST	Endpoint 1 OUT Status
0003:0AB0	EP1IAD	Endpoint 1 IN Buffer Address
0003:0AB2	EP1ISZ	Endpoint 1 IN Buffer Size
0003:0AB4	EP1ICTL	Endpoint 1 IN Control
0003:0AB6	EP1IIRQSZ	Endpoint 1 IN Interrupt Threshold
0003:0AB8	EP1IWDI	Endpoint 1 IN Write Data Port
0003:0ABC	EP1IREST	Endpoint 1 IN Remainder
0003:0ABE	EP1IST	Endpoint 1 IN Status
0003:0AC0	EP2IAD	Endpoint 2 IN Buffer Address
0003:0AC2	EP2ISZ	Endpoint 2 IN Buffer Size
0003:0AC4	EP2ICTL	Endpoint 2 IN Control
0003:0AC6	EP2IIRQSZ	Endpoint 2 IN Interrupt Threshold
0003:0AC8	EP2IWDI	Endpoint 2 IN Read Data Port
0003:0ACC	EP2IREST	Endpoint 2 IN Data Remaining
0003:0ACE	EP2IST	Endpoint 2 IN Status
0003:0AD0	EP3IAD	Endpoint 3 IN Buffer Address
0003:0AD2	EP3ISZ	Endpoint 3 IN Buffer Size
0003:0AD4	EP3ICTL	Endpoint 3 IN Control
0003:0AD6	EP3IIRQSZ	Endpoint 3 IN Interrupt Threshold
0003:0AD8	EP3IWDI	Endpoint 3 IN Write Data Port
0003:0ADC	EP3IREST	Endpoint 3 IN Remainder
0003:0ADE	EP3IST	Endpoint 3 IN Status
0003:0AE0	EP4IAD	Endpoint 4 IN Buffer Address
0003:0AE2	EP4ISZ	Endpoint 4 IN Buffer Size
0003:0AE4	EP4ICTL	Endpoint 4 IN Control
0003:0AE6	EP4IIRQSZ	Endpoint 4 IN Interrupt Threshold
0003:0AE8	EP4IWDI	Endpoint 4 IN Write Data Port
0003:0AEC	EP4IREST	Endpoint 4 IN Remainder
0003:0AEE	EP4IST	Endpoint 4 IN Status
0003:0B00	USBRST	USB Reset
0003:0B02	USBRSM	USB Resume
0003:0B04	USBINTMSK	USB Interrupt Mask
0003:0B06	USBCFG	USB Configuration
0003:0B08	USBST	USB Status
0003:0B0A	USBINTST	USB Interrupt Status/Reset
0003:0B0C	USBFrm	USB Frame
0003:0B0E	USBALT	USB Alternate
0003:0B10	USBCtl	USB Control
0003:0B12	DMAAD0	DMA Address 0 (low order bits)
0003:0B14	DMAAD1	DMA Address 1 (high order bits)
0003:0B16	DMASZ	DMA Size
0003:0B18	DMACTL	DMA Control
0003:0B1A	DMATRG	DMA Trigger
0003:0B1C	DMAREST	DMA Packets Remaining
0003:0B1E	DMAST	DMA Short Packet Received Status

18.18 USB Registers

18.18.1 EP0OAD - USB Endpoint 0 OUT Buffer Address

[illegible][illegible]

18.18.2 EP0OSZ - USB Endpoint 0 OUT Buffer Size

EP00SZ	0003.0A82								offset:	0x02								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	00SZ[7]	00SZ[6]	00SZ[5]	00SZ[4]	00SZ[3]	00SZ[2]	00SZ[1]	00SZ[0]					
R/W	-	-	-	-	-	-	-	-	w	w	w	w	w	w	w	w					
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0					

[illegible]

18.18.3 EP0OCTL - USB Endpoint 0 OUT Control

EPOCTL0003:0A84										offset: 0x04									default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	OEEMDE	OOBYTE	OEENDI	RSV	RSV	RSV	RSV	RSV	OOSTL	OORST	OENA					
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	-	R/W	W	R/W					
Default	-	-	-	-	-	0	0	0	-	-	-	-	-	0	0	0					

Bit	Name	Reset Value	RW	Function
15:11	RSV			Reserved
10	OEWMDE	0	R/W	FIFO ENABLE bit automatic clear mode select bit (See the description for bit 0 in EP0ORCTL)
				Byte Access Enable to EP0ORDT » 0: Enable 16-bit (word) access to EP0ORDT « » 1: Enable 8-bit (byte) access to EP0ORDT «
9	OOBYTE	0	R/W	When OOBYTE is set to '1', the lower 8-bits of EP0ORDT are valid for byte access.
				Endianess Select for EP0 OUT » 0: Little endian « » 1: Big endian « This bit sets the endianess when reading EP0ORDT.
7:3	RSV			Reserved
				STALL bit for EP0 OUT USB specifies the protocol stall in addition to the function stall of Endpoint by SetFeature command. The protocol stall is that when the data packet of the received setup transaction is illegal or unsupported command, the IN/OUT transaction of the successive data or status stage is stalled by the device. When set to one, EP0 OUT is stalled. The device returns a STALL handshake to the HOST after an OUT token is received. Since this bit is automatically cleared when the device receives the next SETUP token, users do not have to clear this bit. Note that do not set O6STL and O6ENA simultaneously.
2	O6STL	0	R/W	
				RESET bit for EP0 OUT FIFO When set to one, the status and the data stored in the EP0 OUT FIFO are cleared. This bit is automatically cleared and is write only. EP0OAB, EP0OSZ and EP0OIRGSZ are not cleared by this register. When a SETUP token is received, this bit is set automatically and the FIFO is reset to receive the DATA packet without a failure.
1	OORST	0	W	
				ENABLE bit for EP0 OUT When this bit is set, the device returns an ACK handshake after receiving a DATA packet from HOST in OUT transactions to EP0. When this bit is cleared, the device receives but ignores the DATA packet, and returns a NAK handshake. This bit is cleared by hardware when -The DEVICE sends an ACK handshake to the HOST (O6EMDE = 0) -The number of data sent from the HOST in the transaction is less than MaxPktSize when the DEVICE sends an ACK handshake to the HOST (O6EMDE = 1)
0	O6ENA	0	R/W	The enable bit auto clearing scheme can be selected by setting O6EMDE register (bit 0 of EP0ORCTL register) as for the SETUP transaction, after receiving a SETUP token the device always receives a DATA0 packet and returns an ACK handshake regardless of this bit.

18.18.4 EP0OIRQSZ - USB Endpoint 0 OUT Interrupt Size Threshold

[illegible][illegible]

18.18.5 EP0ORDT - USB Endpoint 0 OUT Read Data Port

[illegible]

Bit	Name	Reset Value	R/W	Function
				EP0 OUT Read Port Read port of EP0 OUT FIFO. The data field of the received data packet is stored in the FIFO. Read only.
15:0	OORDT	0	R	Note : By default FIFO read access is 16-bit and all the 16-bits in the read port register (EP0ORDT) are valid, However when FIFO read access is 8-bit, only the lower 8-bits in EP0ORDT are valid. 8-bits can be read from the FIFO by setting the O0BYTE field in EP0OCTL register.

18.18.6 EP0OREST - USB Endpoint 0 OUT Data Remaining

[illegible][illegible]

18.18.7 EP0OST - USB Endpoint 0 OUT Status

[illegible][illegible]

18.18.8 EP0IAD - USB Endpoint 0 IN Buffer Address

[illegible][illegible]

18.18.9 EP0ISZ - USB Endpoint 0 IN Buffer Size

EPOISZ	0003.0A92								offset: 0x12						default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	I0S2[7]	I0S2[6]	I0S2[5]	I0S2[4]	I0S2[3]	I0S2[2]	I0S2[1]	I0S2[0]		
R/W	-	-	-	-	-	-	-	-	w	w	w	w	w	w	w	w		
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0		

[illegible]

18.18.10 EP0ICTL - USB Endpoint 0 IN Control

EPOICTL													0003:0A94				offset: 0x14				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RSV	RSV	RSV	RSV	RSV	IOEMDE	IOBYTE	IOENDI	RSV	RSV	RSV	RSV	RSV	IOSTL	IOBST	IOENA								
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	-	R/W	W	R/W								
Default	-	-	-	-	-	0	0	0	-	-	-	-	-	0	0	0								

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	IOEMDE	0	R/W	FIFO ENABLE bit automatic clear mode select bit (See the description for bit 0 in EP0ICTL)
9	IOBYTE	0	R/W	Byte Access Enable to EP0IWDT » 0: Enable 16-bit (word) access to EP0IWDT « » 1: Enable 8-bit (byte) access to EP0IWDT « When IOBYTE is set to '1', the lower 8-bits of EP0IWDT are valid for byte access.
8	IOENDI	0	R/W	Endianness Select for EP0 IN » 0: Little endian « » 1: Big endian « This bit sets the endianness when writing EP0IWDT.
7:3	RSV			Reserved
2	I0STL	0	R/W	STALL bit for EP0 IN When set, the protocol stall condition for EP0 IN is activated. When an IN token to EP0 is received, the device returns a STALL handshake. This bit is automatically cleared after the device receives the next SETUP packet. Note that this stall bit and the enable bit (bit0) of this EP0ICTL, must not be set simultaneously.
1	I0RST	0	W	RESET bit for EP0 IN FIFO When set to one, the status and the data stored in EP0 OUT FIFO are cleared. This register is automatically cleared and write only. EP0IAD, EP0ISZ and EP0IIRQSZ are not cleared by this register.
0	IOENA	0	R/W	ENABLE bit for EP0 IN When this bit is set, the DEVICE returns a DATA0/DATA1 packet with data in the FIFO after receiving an IN token to EP0. When this bit is cleared, the DEVICE returns a NAK handshake for IN token. This bit is automatically cleared by hardware when -The DEVICE receives an ACK handshake from the HOST (IOEMDE = 0) -The rest of bytes in FIFO when the DEVICE receives an ACK handshake from the HOST is less than MaxPktSize (IOEMDE = 1) The ENABLE bit control scheme can be selected by setting IOEMDE. If a NAK handshake is received after sending the DATA packet, the data sent to the HOST is recovered. The FIFO status (EP0IST) and this bit are not changed from the status before the transaction.

18.18.11 EP0IRQSZ - USB Endpoint 0 IN Interrupt Size

[illegible][illegible]

18.18.12 EP0IWDT - USB Endpoint 0 IN Write Data Port

[illegible][illegible]

18.18.13 EP0IREST - USB Endpoint 0 IN Remainder

EPOIREST										0003:0A9C		offset:		0x1C												default:		0x0001	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0													
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	I[OREST]8	I[OREST]7	I[OREST]6	I[OREST]5	I[OREST]4	I[OREST]3	I[OREST]2	I[OREST]1	I[OREST]0													
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R													
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0													

[illegible]

18.18.14 EP0IST - USB Endpoint 0 IN Status

[illegible][illegible]

18.18.15 **EP10AD - USB Endpoint 1 OUT Buffer Address**

[illegible][illegible]

18.18.16 EP1OSZ - USB Endpoint 1 OUT Buffer Size

EP10SZ		0003:0AA2										offset:		0x22								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	RSV	RSV	RSV	RSV	O1SZ[10]	O1SZ[9]	O1SZ[8]	O1SZ[7]	O1SZ[6]	O1SZ[5]	O1SZ[4]	O1SZ[3]	O1SZ[2]	O1SZ[1]	O1SZ[0]								
R/W	-	-	-	-	-	-	w	w	w	w	w	w	w	w	w	w	w								
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0								

[illegible]

18.18.17 EP1OCTL - USB Endpoint 1 OUT Control

EP1OCTL	0003:0A44														offset:	0x24				default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0											
Name	RSV	RSV	RSV	RSV	RSV	O1EMDE	O1BYTE	O1ENDI	RSV	RSV	RSV	RSV	RSV	O1STL	O1RST	O1ENA											
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	-	R/W	W	R/W											
Default	-	-	-	-	-	0	0	0	-	-	-	-	-	0	0	0											

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	01EMDE	0	R/W	FIFO ENABLE bit automatic clear mode select bit (See the description for bit 0 in EP10CTL)
9	01BYTE	0	R/W	Byte Access Enable to EP10RDT » 0: 16-bit (byte) access to EP10RDT « » 1: Enable 8-bit access to EP10RDT « When 01BYTE is set to '1', the lower 8-bits of EP10RDT are valid for byte access.
8	01ENDI	0	R/W	Endianness Select for EP1 OUT » 0: Little endian « » 1: Big endian « This bit sets the endianness when reading EP10RDT.
7:3	RSV			Reserved
2	01STL	0	R/W	STALL bit for EP1 OUT When set, the function stall condition for EP1 OUT can be applied. On receiving an OUT token with setting 01STL, the internal state of EP1 OUT changes to HALT and the endpoint is stalled. The device receives and ignores the DATA packet, and sends a STALL handshake. Once the endpoint is stalled, the stall condition is not cleared until the HOST clears it by a ClearFeature(HALT) command even if this bit is cleared. If this bit is set with setting 01ENA when ClearFeature(HALT) is received, both bits are automatically cleared at that time. Setting STALL bit without setting 01ENA does not clear them automatically.
1	01RST	0	W	RESET bit for EP1 OUT FIFO When set to one, the status and the data stored in the EP1 OUT FIFO are cleared. This bit is automatically cleared and is write only. EP10AD, EP10SZ and EP10IRQSZ are not cleared by this register.
0	01ENA	0	R/W	ENABLE bit for EP1 OUT ENABLE bit for EP1 OUT. When this bit is set, the device returns an ACK handshake after receiving the DATA packet from the HOST in the OUT transactions to EP1. When this bit is cleared, the device receives but ignores the DATA packet, and returns a NAK handshake. This bit is cleared by hardware when -The DEVICE sends an ACK handshake to the HOST (01EMDE = 0) -The number of data sent from the HOST in the transaction is less than MaxPktSize when the DEVICE sends an ACK handshake to the HOST (01EMDE = 1) The ENABLE bit control scheme can be selected by setting 01EMDE.

18.18.18 EP10IRQSZ - USB Endpoint 1 OUT Interrupt Size Threshold

[illegible][illegible]

18.18.19 EP1ORDT - USB Endpoint 1 OUT Read Data Port

[illegible][illegible]

18.18.20 EP10REST - USB Endpoint 1 OUT Data Remaining

[illegible][illegible]

18.18.21 EP1OST - USB Endpoint 1 OUT Status

[illegible][illegible]

18.18.22 EP1IAD - USB Endpoint 1 IN Buffer Address

[illegible][illegible]

18.18.23 EP1ISZ - USB Endpoint 1 IN Buffer Size

[illegible][illegible]

18.18.24 EP1ICTL - USB Endpoint 1 IN Control

[illegible]

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	11EWDE	0	R/W	FIFO ENABLE bit automatic clear mode select bit (See the description for bit 0 in EPI1CTL) Byte Access Enable to EPI1WDT » 0: Enable 16-bit (word) access to EPI1WDT « » 1: Enable 8-bit (byte) access to EPI1WDT «
9	11BYTE	0	R/W	When 11BYTE is set to '1', the lower 8-bits of EPI1WDT are valid for byte access.
8	11ENDI	0	R/W	Endianness Select for EP1 IN » 0: Little endian « » 1: Big endian « This bit sets the endianness when writing EPI1WDT.
7:3	RSV			Reserved
2	11STL	0	R/W	EP1 IN STALL STALL bit for EP1 IN. When set, the function stall condition for EP1 IN can be applied. On receiving an IN token with setting 11STL, the internal state of the EP1 IN changes to HALT and the EP1 IN is stalled. The device returns a STALL handshake. Once the endpoint is stalled, the stall condition is not cleared until the HOST clears it by a ClearFeature(HALT) command even if this bit is cleared. If this bit is set with setting 11ENA when ClearFeature (HALT) is received, both bits are automatically cleared at that time. Setting 11STL without setting 11ENA does not clear them automatically.
1	11RST	0	W	RESET bit for EP1 IN FIFO When set to one, the status and the data stored in EP1 IN are cleared. This register is automatically cleared and write only. EPI1AD, EPI1SZ and EPI1IRQSZ are not cleared by this register.
0	11ENA	0	R/W	ENABLE bit for EP1 IN When this bit is set, the device returns a DATA0/DATA1 packet with the data in the FIFO after receiving an IN token to EP1. When this bit is cleared, the device returns a NAK handshake for the IN token. This bit is cleared by hardware when - The DEVICE receives an ACK handshake from the HOST (11EMDE = 0) - The rest of bytes in the FIFO when the DEVICE receives an ACK handshake from the HOST is less than MaxPktSize (11EMDE = 1) The enable bit control scheme can be selected by setting 11EMDE register (bit10 of EPI1CTL register). If a NAK handshake is received after sending a DATA packet, the data sent to the HOST is recovered, and the FIFO status (EPI1ST) and this bit are not changed from the status before the transaction.

18.18.25 EP1IRQSZ - USB Endpoint 1 IN Interrupt Size

[illegible][illegible]

18.18.26 EP1IWDTP - USB Endpoint 1 IN Write Data Port

[illegible][illegible]

18.18.27 EP1IREST - USB Endpoint 1 IN Remainder

[illegible][illegible]

18.18.28 EP1IST - USB Endpoint 1 IN Status

[illegible][illegible]

18.18.29 EP2IAD - USB Endpoint 2 IN Buffer Address

[illegible][illegible]

18.18.30 EP2ISZ - USB Endpoint 2 IN Buffer Size

EP2ISZ		0003.0AC2								offset:		0x42						default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	I2S[7]	I2S[6]	I2S[5]	I2S[4]	I2S[3]	I2S[2]	I2S[1]	I2S[0]					
R/W	-	-	-	-	-	-	-	-	w	w	w	w	w	w	w	w					
Default									0	0	0	0	0	0	0	0					

[illegible]

18.18.31 EP2ICTL - USB Endpoint 2 IN Control

EP2ICTL					0003:0AC4					offset: 0x44										default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RSV	RSV	RSV	RSV	RSV	IZEMDE	IZBYTE	IZENDI	RSV	RSV	RSV	RSV	RSV	IZ2STL	IZ2RST	IZENA								
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	-	R/W	W	R/W								
Default	-	-	-	-	-	0	0	0	-	-	-	-	-	0	0	0								

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	I2EMDE	0	R/W	FIFO ENABLE bit automatic clear mode select bit (See the description for bit 0 in EP2ICTL)
9	I2BYTE	0	R/W	Byte Access Enable to EP2IWDT » 0: Enable 16-bit (word) access to EP2IWDT « » 1: Enable 8-bit (byte) access to EP2IWDT «
8	I2ENDI	0	R/W	When I2BYTE is set to '1', the lower 8-bits of EP2IWDT are valid for byte access. Endianness Select for EP2 IN » 0: Little endian « » 1: Big endian « This bit sets the endianness when writing EP2IWDT.
7:3	RSV			Reserved
2	I2STL	0	R/W	EP2 In Stall STALL bit for EP2 IN. When set, the function stall condition for EP2 IN can be applied. On receiving an IN token with setting I2STL, the internal state of the EP2 IN changes to HALT and the EP2 IN is stalled. The device returns a STALL handshake. Once the endpoint is stalled, the stall condition is not cleared until the HOST clears it by a ClearFeature(HALT) command even if this bit is cleared. If this bit is set with setting I2ENA when ClearFeature (HALT) is received, both bits are automatically cleared at that time. Setting I2STL without setting I2ENA does not clear them automatically.
1	I2RST	0	W	RESET bit for EP2 IN FIFO When set to one, the status and the data stored in EP2 IN FIFO are cleared. This register is automatically cleared and write only. EP2IAD, EP2ISZ and EP2IIRQSZ are not cleared by this register.
0	I2ENA	0	R/W	ENABLE bit for EP2 IN When this bit is set, the device returns a DATA0/DATA1 packet with the data in the FIFO after receiving an IN token to EP2. When this bit is cleared, the device returns a NAK handshake for the IN token. This bit is cleared by hardware when -The DEVICE receives an ACK handshake from the HOST (I2EMDE = 0) -The rest of bytes in the FIFO when the DEVICE receives an ACK handshake from the HOST is less than MaxPktSize (I2EMDE = 1) The enable bit control scheme can be selected by setting I2EMDE register (bit10 of EP2ICTL register). If a NAK handshake is received after sending a DATA packet, the data sent to the HOST is recovered, and the FIFO status (EP2IST) and this bit are not changed from the status before the transaction.

18.18.32 EP2IIRQSZ - USB Endpoint 2 IN Interrupt Size Threshold

[illegible][illegible]

18.18.33 EP2IWDT - USB Endpoint 2 IN Read Data Port

[illegible][illegible]

18.18.34 EP2IREST - USB Endpoint 2 IN Data Remaining

[illegible][illegible]

18.18.35 EP2IST - USB Endpoint 2 IN Status

[illegible][illegible]

18.18.36 EP3IAD - USB Endpoint 3 IN Buffer Address

[illegible][illegible]

18.18.37 EP3ISZ - USB Endpoint 3 IN Buffer Size

[illegible][illegible]

18.18.38 EP3ICTL - USB Endpoint 3 IN Control

EP3IC1TL				0003:0AD4				offset: 0x54								default: 0x000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	13BYTE	13ENDI	RSV	RSV	RSV	RSV	RSV	13STL	13RST	13ENA			
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	-	-	R/W	W	R/W			
Default	-	-	-	-	-	-	0	0	-	-	-	-	-	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	RSV			Reserved
9	I3BYTE	0	R/W	Byte Access Enable to EP3IWDT » 0: Enable 16-bit (word) access to EP3IWDT « » 1: Enable 8-bit (byte) access to EP3IWDT « When I3BYTE is set to '1', the lower 8-bits of EP3IWDT are valid for byte access.
8	I3ENDI	0	R/W	Endianness Select for EP3 IN » 0: Little endian « » 1: Big endian « This bit sets the endianness when writing EP3IWDT.
7:3	RSV			Reserved
2	I3STL	0	R/W	EP3 IN Stall EP3 IN STALL. STALL bit for EP3 IN. On receiving an IN token with setting I3STL, the internal state of the EP3 IN changes to HALT and the endpoint is stalled. The device does not send any data packets to the HOST for IN tokens. Once the endpoint is stalled, the stall condition is not cleared until the HOST clears it by a ClearFeature(HALT) command even if this bit is cleared. If this bit is set with setting I3ENA when ClearFeature(HALT) is received, both bits are automatically cleared at that time. Setting I3STL without setting I3ENA does not clear them automatically.
1	I3RST	0	W	RESET bit for EP3 IN FIFO When set to one, the status and the data stored in EP3 IN FIFO are cleared. This register is automatically cleared and write only. EP3IAD, EP3ISZ and EP3IIRQSZ are not cleared by this register.
0	I3ENA	0	R/W	EP3 IN ENABLE. ENABLE bit for EP3 IN. When this bit is set, the device returns a DATA0 packet with the data in the FIFO after receiving an IN token to EP3. When this bit is cleared, the device returns a null DATA0 packet. This bit is not automatically cleared by hardware.

18.18.39 EP3IRQSZ - USB Endpoint 3 IN Interrupt Size

[illegible]

[illegible]

18.18.40 EP3IWDT - USB Endpoint 3 IN Write Data Port

[illegible][illegible]

18.18.41 EP3IREST - USB Endpoint 3 IN Remainder

[illegible][illegible]

18.18.42 EP3IST - USB Endpoint 3 IN Status

[illegible][illegible]

18.18.43 EP4IAD - USB Endpoint 4 IN Buffer Address

[illegible][illegible]

18.18.44 EP4ISZ - USB Endpoint 4 IN Buffer Size

[illegible][illegible]

18.18.45 EP4ICTL - USB Endpoint 4 IN Control

EP4/CTL														0003:0AE4				offset: 0x64				default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	RSV	RSV	RSV	RSV	14BYTE	14END1	RSV	RSV	RSV	RSV	RSV	14STL	14RT	14ENA									
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	-	-	R/W	W	R/W									
Default	-	-	-	-	-	-	0	0	-	-	-	-	-	0	0	0									

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	RSV			Reserved
9	I4BYTE	0	R/W	Byte Access Enable to EP4IWDT » 0: Enable 16-bit (word) access to EP4IWDT « » 1: Enable 8-bit (byte) access to EP4IWDT « When I4BYTE is set to '1', the lower 8-bits of EP4IWDT are valid for byte access.
8	I4ENDI	0	R/W	Endianness Select for EP4 IN » 0: Little endian « » 1: Big endian « This bit sets the endianness when writing EP4IWDT.
7:3	RSV			Reserved
2	I4STL	0	R/W	EP4 IN Stall EP4 IN STALL. STALL bit for EP4 IN. On receiving an IN token with setting I4STL, the internal state of the EP4 IN changes to HALT and the endpoint is stalled. The device does not send any data packets to the HOST for IN tokens. Once the endpoint is stalled, the stall condition is not cleared until the HOST clears it by a ClearFeature(HALT) command even if this bit is cleared. If this bit is set with setting I4ENA when ClearFeature(HALT) is received, both bits are automatically cleared at that time. Setting I4STL without setting I4ENA does not clear them automatically.
1	I4RST	0	W	RESET bit for EP3 IN FIFO When set to one, the status and the data stored in EP3 IN FIFO are cleared. This register is automatically cleared and write only. EP3IAD, EP3ISZ and EP3IIRQSZ are not cleared by this register.
0	I4ENA	0	R/W	EP4 IN ENABLE. ENABLE bit for EP4 IN. When this bit is set, the device returns a DATA0 packet with the data in the FIFO after receiving an IN token to EP4. When this bit is cleared, the device returns a null DATA0 packet. This bit is not automatically cleared by hardware.

18.18.46 EP4IRQSZ - USB Endpoint 4 IN Interrupt Size

[illegible][illegible]

18.18.47 EP4IWDT - USB Endpoint 4 IN Write Data Port

[illegible][illegible]

18.18.48 EP4IREST - USB Endpoint 4 IN Remainder

EP4IREST				0003:0AEC				offset:		0x6C										default:		0x0001	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	4REST[11]	4REST[10]	4REST[9]	4REST[8]	4REST[7]	4REST[6]	4REST[5]	4REST[4]	4REST[3]	4REST[2]	4REST[1]	4REST[0]							
R/W	-	-	-	-	R	R	R	R	R	R	R	R	R	R	R	R							
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0							

[illegible]

18.18.49 EP4IST - USB Endpoint 4 IN Status

[illegible][illegible]

18.18.50 USBRST - USB Reset

USBRST				0003:0B00				offset: 0x80				default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MIRST
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	MI RST	0	W	Soft Reset When set, the protocol engine (UDC) is reset. USBST and USBINTST registers are also reset. This bit is automatically cleared.

18.18.51 USBRSM - USB Resume

USBRSM				0003:0B02				offset:		0x82				default:				0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MICKOF	MIRSM			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0			

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	MI CKOF	0	R/W	Clock Off - Internal clock off control » 0: enable clock « » 1: disable clock « This can be used for power saving when USB is in the suspend state. Note that when the device is resumed (including the remote wakeup), this bit is automatically cleared, and the clocks are enabled.
0	MI RSM	0	W	Resume - Remote wakeup control register When USB bus is in the suspend condition, the device can resume the bus by setting this register to one. It is not until DEVICE_REMOTE_WAKEUP feature is enabled by SetFeature command from the HOST that remote wakeup function can be available. At power on reset, this function is disabled and setting this bit has no effect. When setting this register, do not set bit1 of USBRSM to one.

18.18.52 USBINTMSK - USB Interrupt Mask

[illegible]

Bit	Name	Reset Value	R/W	Function
15	MI DMAIE	0	R/W	DMA End Interrupt Enable
14	MI NCFIE	0	R/W	ClearFeature Interrupt Enable
13	MI NSPIE	0	R/W	No SOF Packet Interrupt Enable
12	MI SOFIE	0	R/W	SOF Interrupt Enable
11	MI SIE	0	R/W	SetInterface Interrupt Enable
10	MI SCIE	0	R/W	SetConfiguration Interrupt Enable
9	MI BRIE	0	R/W	USB Bus Reset Interrupt Enable
8	MI SETIE	0	R/W	Setup Interrupt Enable
7	MI SUSIE	0	R/W	Suspend Interrupt Enable
6	MI EP4IE	0	R/W	EP4 IN Interrupt Enable
5	MI EP3IE	0	R/W	EP3 IN Interrupt Enable
4	MI EP2IE	0	R/W	EP2 IN Interrupt Enable
3	MI EP1IE	0	R/W	EP1 IN Interrupt Enable
2	MI EP1OIE	0	R/W	EP1 OUT Interrupt Enable
1	MI EP0IE	0	R/W	EP0 IN Interrupt Enable
0	MI EP0OIE	0	R/W	EP0 OUT Interrupt Enable

18.18.53 USBCFG - USB Configuration

[illegible][illegible]

18.18.54 USBST - USB Status

USBST		0003:0B08										offset: 0x98		default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MIRMTF	RSV	MISPD	MISETUP	MIDID	
R/W	-	-	-	-	-	-	-	-	-	-	-	R	-	R	R	R	
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	0	0	0	

[illegible]

18.18.55 USBINTST - USB Interrupt Status/Reset

[illegible]

Bit	Name	Reset Value	R/W	Function
15	MI DMA	0	R	DMA End Interrupt Interrupt asserted when DMA transfer is completed.
14	MI NCFI	0	R	ClearFeature Interrupt Interrupt asserted when ClearFeature(ENDPOINT_HALT) is received
13	MI NSPI	0	R	No SOF Packet Interrupt Interrupt asserted when the device does not receive the next SOF packet within 1ms after a SOF packet is received.
12	MI SOFI	0	R	SOF Interrupt Interrupt asserted when a SOF packet is received.
11	MI SII	0	R	SetInterface Interrupt Interrupt asserted when SetInterface command is received. AlternateSetting set by HOST can be read from the USBALT register.
10	MI SCI	0	R	SetConfiguration Interrupt Interrupt asserted when SetConfiguration command is received. Configuration set by HOST can be read from bits 1:0 of USBALT register.
9	MI BRI	0	R	USB Bus Reset Interrupt Interrupt asserted when a USB Bus Reset is received.
8	MI SETI	0	R	Setup Interrupt Interrupt asserted when a SETUP transaction whose request is one of GetDescriptor, SetDescriptor and SynchFrame and the class/vendor requests is received.
7	MI SUSI	0	R	Suspend Interrupt Interrupt asserted when the USB Bus is turned into the suspend state or is resumed (including the device remote wakeup). The actual bus status can be known by reading bit2 of USBST register.
6	MI EP4I1	0	R/W	EP4 IN Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is less than or equals to EP4IIRQSZ at the completion of sending a DATA0 packet.
5	MI EP3I1	0	R/W	EP3 IN Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is less than or equals to EP3IIRQSZ at the completion of sending a DATA0 packet.
4	MI EP2I1	0	R/W	EP2 IN Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is less than or equals to EP2IIRQSZ at the completion of receiving of an ACK handshake.
3	MI EP1I1	0	R	EP1 IN Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is less than or equals to EP1IIRQSZ at the completion of receiving of an ACK handshake.
2	MI EP1O1	0	R	EP1 OUT Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is greater than or equals to EP1OIRQSZ or the received DATA packet is short or null at the completion of sending of an ACK handshake.
1	MI EP0I1	0	R	EP0 IN Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is less than or equals to EP0IIRQSZ at the completion of receiving of an ACK handshake.
0	MI EP0O1	0	R	EP0 OUT Interrupt Interrupt asserted when the number of bytes of data stored in FIFO is greater than or equals to EP0OIRQSZ or the received DATA packet is short or null at the completion of sending of an ACK handshake.

18.18.56 USBFRM - USB Frame

USBFrm			0003:0B0C								offset:		0x8C								default:				0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0										
R/W	RSV	RSV	RSV	RSV	RSV	MIFRN[10]	MIFRN[9]	MIFRN[8]	MIFRN[7]	MIFRN[6]	MIFRN[5]	MIFRN[4]	MIFRN[3]	MIFRN[2]	MIFRN[1]	MIFRN[0]										
Default	-	-	-	-	-	R	R	R	R	R	R	R	R	R	R	R										

[illegible]

18.18.57 **USBALT - USB Alternate**

USBALT		0003:0B0E				offset: 0x8E										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	MIA5[2]	MIA5[2]	MIA5[1]	MIA5[2]	RSV	MIA5[1]	MIA5[1]	MIA5[1]	RSV	MIA5[0]	MIA5[0]	MIA5[0]	RSV	RSV	MICON[1]	MICON[0]
R/W	-	R	R	R	R	-	R	R	R	-	R	R	R	-	-	R	R
Default	-	0	0	0	0	-	0	0	0	-	0	0	0	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:12	MI ASI 2	0	R	AlternateSetting of Interface2 AlternateSetting value set to Interface2 by SetInterface command.
11	RSV			Reserved
10:8	MI ASI 1	0	R	AlternateSetting of Interface1 AlternateSetting value set to Interface1 by SetInterface command.
7	RSV			Reserved
6:4	MI ASI 0	0	R	AlternateSetting of Interface0 AlternateSetting value set to Interface0 by SetInterface command.
3:2	RSV			Reserved
1:0	MI CONF	0	R	Configuration Current Configuration number selected by HOST.

18.18.58 USBCTL - USB Control

[illegible][illegible]

18.18.59 DMAAD0 - USB DMA Address 0 (low order bits)

[illegible][illegible]

18.18.60 DMAAD1 - USB DMA Address 1 (high order bits)

[illegible][illegible]

18.18.61 DMASZ - USB DMA Size

[illegible][illegible]

18.18.62 DMACTL - USB DMA Control

DMACTL		0003.0B18				offset:		0x98								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	DEP[2]	DEP[1]	DEP[0]	RSV	RSV	DENDI[1]	DENDI[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DMDE			
R/W	-	R/W	R/W	R/W	-	-	R/W	R/W	-	-	-	-	-	-	-	-			
Default	-	0	0	0	-	-	0	0	-	-	-	-	-	-	-	-			

[illegible]

18.18.63 DMATRG - USB DMA Trigger

[illegible][illegible]

18.18.64 DMAREST - USB DMA Packets Remaining

[illegible][illegible]

18.18.65 DMAST - USB DMA Short Packet Received Status

[illegible][illegible]

19 I²C Interface

19.1 Introduction

The DM270 includes an Inter Integrated Circuit (I²C™) serial interface that can act as a single master. Both 100kbps and 400kbps I²C-bus system is implemented. The interface uses 7bit/10bit addressing. The DM270 initiates and terminates transfers and generates clock signals.

NOTE: I²C is a registered trademark of PHILIPS.

19.2 Pseudo-I²C Interface

DM270 includes a 2-line serial interface and can perform I²C bus protocol-compliant synchronous serial communication as bus master. This module has following features.

- o Supports only single bus master
- o Supports transmit, receive and mixed transfer using the slave address of 7bits or 10bits. (CBUS address is unavailable)
- o Supports transfer rate of 100kbps or 400kbps.
- o Issues interrupt to ARM to indicate the completion of transfer.

Please observe following restrictions when this module is in use.

RESTRICTIONS on The Use of Pseudo-I²C Interface

1. The GPIO31 and GPIO32 ports are used as communication bus. Above two ports are not available as general-purpose I/O pin when this module is in use. Use FSEL1(0x305A6) to select the GPIO pin for I2C communication
2. Clock signals input to M48XI are used as clock generation source for data transfer clock. Thus, transfer rate is not as it appears in the specifications when a non-48MHz clock is input to this M48XI pin. When there is no clock input to this pin, this module does not run.

19.3 I²C Bus Configuration

19.3.1 Enabling DM270 Bus Line

I²C has the SCL line that transfers the clock and SDA line which transmits/receives data. On DM270, the SCL and SDA are assigned to GIO31 and GIO32 respectively. Writing "1" to the corresponding bit in the FSEL1(0x305A6) register in "GIO module" enables these bus line functions. (See Table 47.)

Bus line	GIO port	Corresponding bit in the FSEL1 (0x305A6)
SCL	31	GIO31SEL (bit8)
SDA	32	GIO32SEL (bit9)

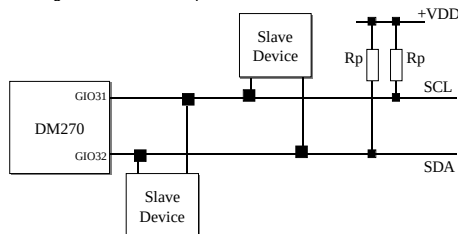
19.3.2 Connecting Slave Devices

Figure 139 shows an example I²C bus connection. DM270 is the master and other connecting devices are slaves. Each device should be an open-drain connection, hence each bus line should be connected with appropriate pull-up resistors. In addition, please observe the following restrictions concerning bus configuration.

RESTRICTIONS on Bus Configuration

1. Pseudo-I²C bus of DM270 does not support the multi bus master. DM270 itself is a unique master device on bus and another device as bus master cannot exist.
2. Pseudo-I²C bus I/O buffer of DM270 has low drivability. This may not meet the connection with many devices and a large amount of load capacity of bus.

Figure 139. Example of I²C bus connection



19.4 Data Transfer Sequence

Pseudo-I²C module of DM270 can perform data transmit to/ receive from slave device. It also supports mixed data transfer. 7bits or 10bits address can be specified as slave address. CBUS address is not supported.

Writing to/ reading from the register in pseudo-I²C module by ARM executes all procedures for data transfer. Description of actual data transfer sequence will follow that of basic operation for data transmit/ receive.

19.4.1 Setting data transfer rate

Data transfer rate is set prior to actual data transfer. The user can select 100kbps or 400kbps as data transfer rate. It is 100kbps by default. This switching is set with CLKSEL bit in the SCS – I2C Serial Control register. (See Table 48.)

The following restrictions concerning transfer rate should be observed.

RESTRICTIONS on Transfer Rate of Pseudo-I²C Module

Clock signals input to MXI48 are used as generation source of clocks for data transfer. Thus, transfer rate is not as it appears in the specifications when a non-48MHz clock is input to this MXI48 pin.

CLKSEL (0x30D84 [3])	Data transfer rate
0	100kbps
1	400kbps

19.4.2 Data Register for Transfer

There are two data transfer registers; the TXDATA – I2C Transmit Data Register and RXDATA – I2C Receive Data Register.

To transmit data to slave device, the data to be sent is set in TXD[7:0] bit of TXDATA – I2C Transmit Data Register. After sending, the data sent is shown in RXD[7:0] bit of RXDATA – I2C Receive Data Register. Check the status bit RXD of RXDATA – I2C Receive Data Register to determine if the data was sent properly on the bus or not.

To receive data from slave device, 0xFF should be set in TXD[7:0] bit of TXDATA – I2C Transmit Data Register. After completing reception, the received data can be read from RXD[7:0] bit of RXDATA – I2C Receive Data Register. (See Table 49.)

	Operation to TXD bit (0x30D80 [7:0])	Function of RXD bit (0x30D82 [7:0])
During data transmit	Write transmit data before sending	Indicates the sent data on bus after sending completed
During data receive	Set 0xFF before receiving	Indicates the received data after receiving completed

19.4.3 Acknowledgement bit

In I²C bus, an acknowledgement bit follows transfer of every data byte.

During data transmit to slave device, the slave device sets this bit. To receive Acknowledgement bit properly, ACK bit in the TXDATA – I2C Transmit Data Register should be set to “1” before sending data. After sending completed, Acknowledgement bit from slave device can be read from ACKS bit in the RXDATA – I2C Receive Data Register.

During data receive from slave device, master device or DM270 must set this ACK bit in TXDATA. Setting Acknowledgement bit to slave device (ACK bit) in the TXDATA – I2C Transmit Data Register is required before receiving data. In addition, after receiving completed, Acknowledgement bit actually sent on bus can be read from ACKS bit in the RXDATA – I2C Receive Data Register. (See Table 50.)

Table 50. Acknowledgemnt bit

	Operation to ACK bit (0x30D80 [8])	Function of ACKS bit (0x30D82 [8])
During data transmit	Set '1'	Indicates Acknowledgement bit from slave device after sending completed
During data receive	Returning to slave device Set Acknowledgement bit	Indicates Acknowledgement bit sent on bus after receiving completed

19.4.4 Issuing of Start, Stop and Restart conditions

The user can issue “Start” condition to start data transfer. After completing the data transfer, “Stop” condition can be issued. Furthermore, “Restart” condition can also be issued.

Setting '01' in COND[1:0] bit in the SCS – I2C Serial Control register issues “Start” condition. “Start” condition will be added automatically during actual data transfer start.

“Stop” condition is issued by setting '10' to COND[1:0] bit. Triggering data transfer with “Stop” condition set issues actual “Stop” condition on bus. In this case only issue of “Stop” condition is performed but data transfer is not done.

“Restart” condition can be issued by starting transfer with “Start” condition and then again starting transfer with “Start” condition prior to issuing “Stop” condition.

Setting '00' to COND[1:0] bit issues neither (Re)Start nor Stop conditions. (See Table 51.)

Table 51. Issuing Start, Stop and Restart conditions

	Start/Restart conditions	Stop condition	Transfer without condition
Setting of COND bit (0x30D84[1:0])	Set '01' before data transfer start to start data transfer	Set '10' to trigger stop condition	Set '00'

19.4.5 Triggering data transfer

All data transfers on pseudo-I²C module of DM270 are performed in 1 byte units. Consequently, ARM must trigger data transfer after every byte transfer.

The data transfer is triggered by XMIT bit in the SCS – I2C Serial Control register. Writing '1' to this bit starts data transfer of 1byte. (The settings of data transfer, Start/ Stop conditions and Acknowledgement bit should be executed in advance.)

XMIT bit shows '1' during transfer. When XMIT is “1”, writing '1' does not start new transfer. It is recommended to verify that this bit is set at '0' before writing “1” to start new data transfer. (See Table 52.)

Once a transfer starts, it cannot be suspended.

Table 52. Triggering data transfer

XMIT bit (0x30D84 [2]) state	Description
0	Stand-by state Writing '1' starts data transfer
1	During transfer Writing to this bit is when it is “1” should not be done .

19.4.6 Interrupt to interrupt controller

Pseudo-I²C module of DM270 has one port to interrupt controller in DM270. Port number is INT37. An interrupt is generated in any of the following cases.

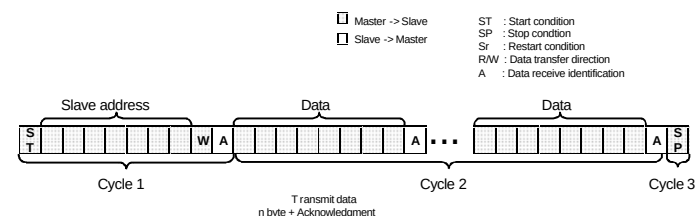
- When Acknowledgement bit is transmitted/ received
- When STOP condition is generated

For details of interrupt controller, refer to the section on “Interrupt controller”.

19.4.7 Transmit (7bit address)

Transfer data line to send n-byte data using the slave address of 7 bits is described in Figure 140. The sequence including specific register settings is also described.

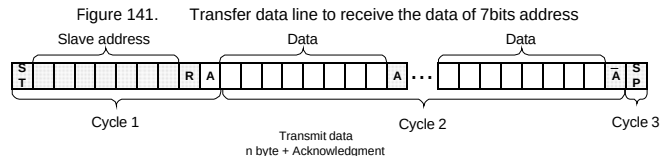
Figure 140. Transfer data line to send the data of 7bits address



- i. Write the slave address to TXD[7:1], '0' to TXD[0] and '1' to ACK in TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.
- iv. Cycle 1 is terminated. An interrupt is generated.
- v. Write the data to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- vii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5) – (8) n times.
- ix. Write "10" (stop condition) to COND [1:0] in the SCS – I2C Serial Control register.
- x. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 3 starts.
- xi. Cycle 3 is terminated. An interrupt is generated. Transmit is completed.

19.4.8 Receive (7bit address)

Transfer data line to send n-byte data using the slave address of 7bits is described in Figure 141. The sequence including specific register settings is also described.

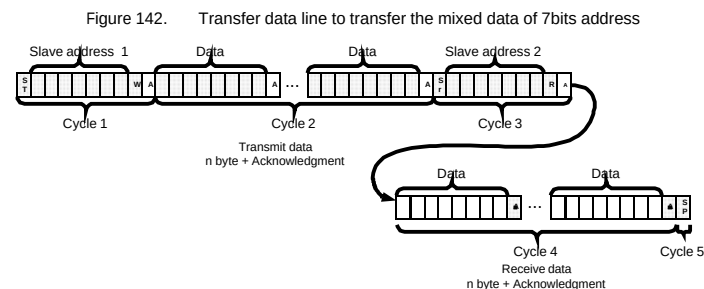


- i. Write the slave address to TXD[7:1], '1' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.
- iv. Cycle 1 is terminated. An interrupt is generated.

- v. Write '1' to all TXD[7:0] and '0' to ACK in the TXDATA – I2C Transmit Data Register (Write '1' to ACK in the th times).
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- vii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5)-(8) n times.
- ix. Write "10" (stop condition) to COND[1:0] in the SCS – I2C Serial Control register.
- x. Write "1" to XMIT in the SCS – I2C Serial Control register. Cycle 3 starts.
- xi. Cycle 3 is terminated. An interrupt is generated. Receive is completed.

19.4.9 Mixed transfer (7bit address)

Transfer data line to send n-byte data and then receive n-byte data using the slave address of 7bits is described in Figure 142. The sequence including specific register settings is also described.

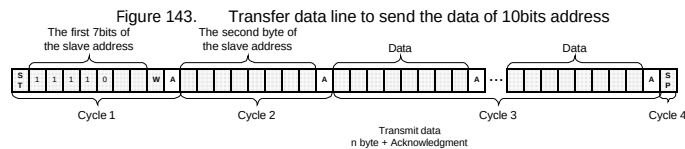


- i. Write the slave address 1 to TXD[7:1], "0" to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.
- iv. Cycle 1 is terminated. An interrupt is generated.
- v. Write the data to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.

- vii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5) – (8) n times.
- ix. Write the slave address 2 to TXD[7:1], '1' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- x. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- xi. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 3 starts.
- xii. Cycle 3 is terminated. An interrupt is generated.
- xiii. Write '1' to all TXD[7:0] and '0' to ACK in the TXDATA – I2C Transmit Data Register. (Write '1' to ACK in the nth times).
- xiv. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- xv. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 4 starts.
- xvi. Cycle 4 is terminated. An interrupt is generated. Repeat steps (13) – (16) n times.
- xvii. Write "10" (stop condition) to COND[1:0] in the SCS – I2C Serial Control register.
- xviii. Write '1' to XMIT in the SCS. Cycle 5 starts.
- xix. Cycle 5 is terminated. An interrupt is generated. Transfer is completed.

19.4.10 Transmit (10bit address)

Transfer data line to send n-byte data using the slave address of 10 bits is described in Figure 143. The sequence including specific register settings is also described.

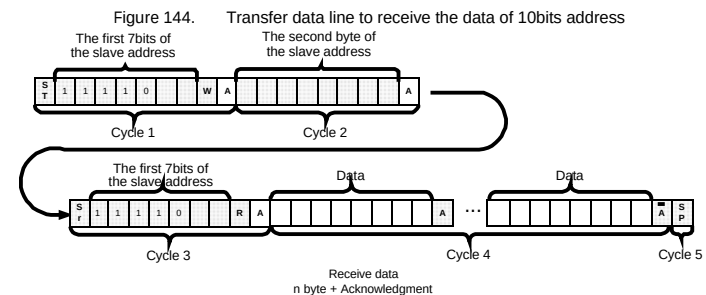


- i. Write the first 7 bits of the slave address to TXD[7:1], '0' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.
- iv. Cycle 1 is terminated. An interrupt is generated.

- v. Write the second byte of the slave address to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- vii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated.
- ix. Write the data to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- x. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- xi. Write '1' to XMIT in SCS – I2C Serial Control register. Cycle 3 starts.
- xii. Cycle 3 is terminated. An interrupt is generated. Repeat steps (9) – (12) n times.
- xiii. Write "10" (stop condition) to COND[1:0] in the SCS – I2C Serial Control register.
- xiv. Write '1' to XMIT in the SCS. Cycle 4 starts.
- xv. Cycle 4 is terminated. An interrupt is generated. Transmit is completed.

19.4.11 Receive (10bit address)

Transfer data line to receive n-byte data using the slave address of 10bits is described in Figure 6. The sequence including specific register settings is also described.

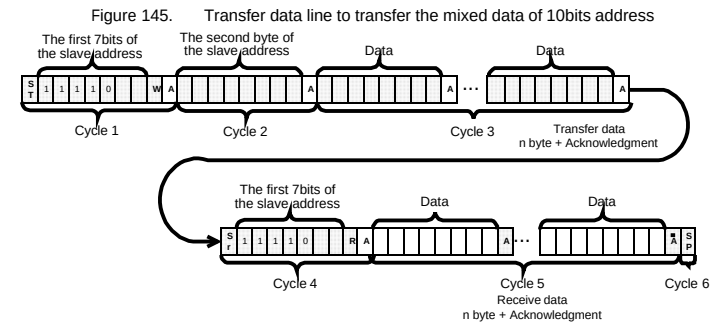


- i. Write the first 7bits of the slave address to TXD[7:1], '0' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.

- iv. Cycle 1 is terminated. An interrupt is generated.
- v. Write the second byte of the slave address to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- vii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated.
- ix. Write the first 7bits of the slave address to TXD[7:1], '1' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- x. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- xi. Write '1' to XMIT in SCS – I2C Serial Control register. Cycle 3 starts.
- xii. Cycle 3 is terminated. An interrupt is generated.
- xiii. Write '1' to all TXD[7:0] and '0' to ACK in the TXDATA (Write '1' to ACK in the nth times).
- xiv. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- xv. Write '1' to XMIT in the SCS. Cycle 4 starts.
- xvi. Cycle 4 is terminated. An interrupt is generated. Repeat steps (13) –(16) n times.
- xvii. Write "10" (stop condition) to COND[1:0] in the SCS.
- xviii. Write '1' to XMIT in the SCS. Cycle 5 starts.
- xix. Cycle 5 is terminated. An interrupt is generated. Receive is completed.

19.4.12 Mixed transfer (10bit address)

Transfer data line to send n-byte data and then receive n-byte data using the slave address of 10bits is described in Figure 145. The sequence including specific register settings is also described.



- i. Write the first 7bits of the slave address to TXD[7:1], '0' to TXD[0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- ii. Write "01" (start condition) to COND[1:0] in the SCS – I2C Serial Control register.
- iii. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 1 starts.
- iv. Cycle 1 is terminated. An interrupt is generated.
- v. Write the second byte of the slave address to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- vi. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- vii. Write '1' to XMIT in SCS – I2C Serial Control register. Cycle 2 starts.
- viii. Cycle 2 is terminated. An interrupt is generated.
- ix. Write the data to TXD[7:0] and '1' to ACK in the TXDATA – I2C Transmit Data Register.
- x. Write "00" (normal transfer) to COND[1:0] in the SCS – I2C Serial Control register.
- xi. Write '1' to XMIT in the SCS – I2C Serial Control register. Cycle 3 starts.
- xii. Cycle 3 is terminated. An interrupt is generated. Repeat steps (9) – (12) n times.
- xiii. Write the first 7bits of the slave address to TXD[7:1], '1' to TXD[0] and '1' to ACK in the TXDATA.
- xiv. Write "01" (start condition) to COND[1:0] in the SCS.
- xv. Write '1' to XMIT in the SCS. Cycle 4 starts.
- xvi. Cycle 4 is terminated. An interrupt is generated.

- xvii. Write '1' to all TXD[7:0] and "0" to ACK in the TXDATA (Write '1' to ACK in the nth times).
- xviii. Write "00" (normal transfer) to COND[1:0] in the SCS.
- xix. Write '1' to XMIT in SCS. Cycle 5 starts.
- xx. Cycle 5 is terminated. An interrupt is generated. Repeat 17) – 20) n times.
- xxi. Write "10" (stop condition) to COND[1:0] in the SCS.
- xxii. Write '1' to XMIT in SCS. Cycle 6 starts.
- xxiii. Cycle 6 is terminated. An interrupt is generated. Transfer is completed.

19.5 I2C Bus Interface Register Map (I2C)

Address	Register	Description
0003:0D80	TXDATA	Transmit Data Register
0003:0D82	RXDATA	Receive Data Register
0003:0D84	SCS	Serial Control

19.6 I2C Bus Interface Registers

19.6.1 TXDATA – I2C Transmit Data Register

TXDATA		0003:0D80				offset: 0x00												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ACK	TXD[7]	TXD[6]	TXD[5]	TXD[4]	TXD[3]	TXD[2]	TXD[1]	TXD[0]			
R/W	-	-	-	-	-	-	-	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	ACK	0	W	Acknowledge control (Setting of acknowledge level) » 0: Receiver sends Acknowledge « » 1: Receiver does not Acknowledge (when sending data) «
7:0	TXD	0	R/W	Transmission address/data When receiving from slave value is *xFF

19.6.2 RXDATA – I2C Receive Data Register

RXDATA				0003:0D82				offset: 0x02								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ACKS	RXD[7]	RXD[6]	RXD[5]	RXD[4]	RXD[3]	RXD[2]	RXD[1]	RXD[0]			
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R			
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	ACKS	0	R	Acknowledge Status After transfer completed (occurrence of interrupt), acknowledge value can be read.
7:0	RXD	0	R	Receive Data After transfer completed (occurrence of interrupt), received data can be read. At the time of transmitting, after transfer completed, values written in TXDATA can be read.

19.6.3 SCS – I2C Serial Control register

SCS	0003:0D84												offset:	0x04				default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CLKSEL	XMIT	COND[1]	COND[0]						
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W						
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0						

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	CLKSEL	0	R/W	Selection of transfer clock » 0: 100KHz « » 1: 400KHz «
2	XMIT	0	R/W	READ: Status on transfer can be read. 0: Transfer on standby 1: On transmitting WRITE: Trigger for transfer 0: No transfer 1: Transfer starts.
1:0	COND	0	R/W	Transfer Condition » 00: Normal transfer « » 01: Transfer with start condition « » 10: Transfer with stop condition « » 11: Reserved «

20 Memory Traffic Controller (MTC)

This chapter describes the Memory Traffic Controller (MTC) of DM270. The MTC consists of External Memory Interface (EMIF), SDRAM controller (SDRC), and DMA controller (DMAC). MTC enables DM270 to have access to large memory spaces. EMIF includes two specialized interfaces to control and access Compact Flash and Smart Media cards, whereas DMAC allows fast and efficient memory transfers between the different external memory regions.

20.1 Introduction

When processing data, it is often necessary to access large memory spaces. The internal memory of the DM270 chip is often too small to fit all the data and programs. To solve this problem, the external memory interface controller offers the possibility to access a large amount of external memory. Since the external memory interface is configurable, many different types of memories can be connected to the external interface. In fact the external memory interface includes two specialized interfaces to control and access CompactFlash and SmartMedia cards.

The Memory Traffic Controller (MTC) can be separated into three sub-blocks: the external memory interface (EMIF), the SDRAM controller, and the DMA controller (DMAC). All the three sub-blocks are described in this chapter.

DMAC consists of different DMA channels available with the MTC. These DMA channels allow fast and efficient memory transfers between the different external memory regions and the ARM.

20.1.1 Memory Map

The complete memory map of the DM270 is represented in the 0 This chapter describes the 128 Mbytes of external memory that starts at address 0010:0000h.

ARM memory address map

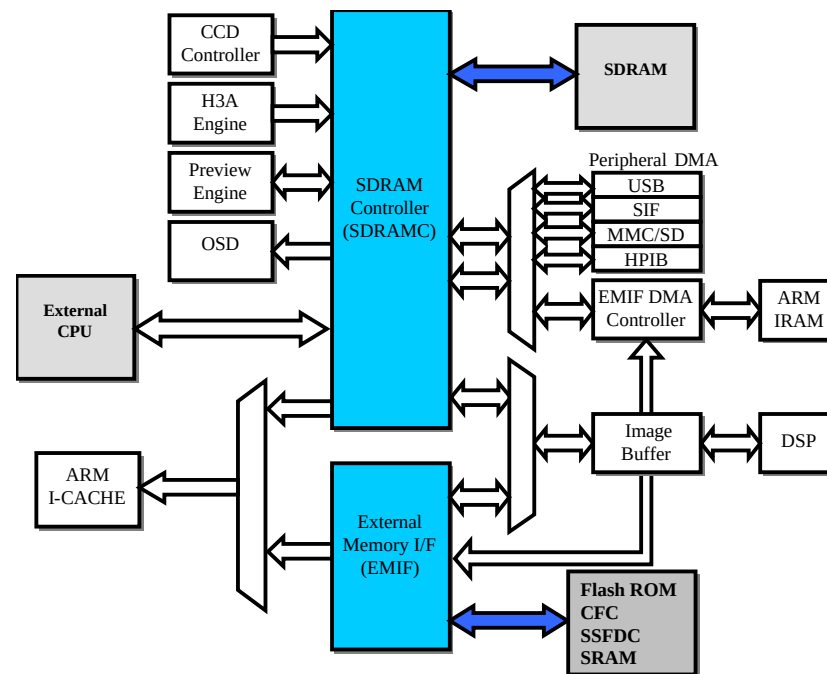
Region name	Start address	End address	Size (bytes)
Reset vector ROM	0x0000:0000	0x0000:0003	4
ARM internal memory (IRAM)	0x0000:0004	0x0000:7FFF	~32 K
Internal peripheral	0x0003:0000	0x0003:0FFF	4 K
DSP memory	0x0004:0000	0x0005:FFFF	128 K
External memory	0x0010:0000	0x080F:FFFF	128 M

After reset, the ARM program counter jumps to address 0 of the CS0 region.

20.1.2 Block Diagram

The SDRAM controller is the main point of communication between the different modules of the DM270. The CCD Controller, Preview Engine, OSD, Image Buffer, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. Depending on the memory location accessed, the ARM will either access the SDRAM controller, or the EMIF.

Figure 146. MTC Controller Block Diagram



20.2 Register Settings

As for any other peripheral, memory mapped registers control the MTC module. Refer to section 20.4 External Memory Interface Register Map (EMIF) and 20.7 SDRAM Controller Register Map (SDRC) for a listing of the registers and their addresses. You will find detailed descriptions of these registers in this chapter.

20.3 Access to EMIF Areas

20.3.1 Introduction to the EMIF regions

The MTC can access 128Mbytes of memory separated into 6 regions:

- o CS0, external memory region 0 (FLASH memory interface)
- o CS1, external memory region 1 (CompactFlash interface)
- o CS2, external memory region 2 (SmartMedia interface)
- o CS3, external memory region 3 (general purpose device)
- o CS4, external memory region 4 (general purpose device)
- o SDRAM memory region

Many different types of general-purpose memory can be attached to each region of the external memory interface, such as ROM, EPROM, and SRAM. There are five control output terminals from CS0 through CS4 that can be directly connected with these general-purpose memories without any external decoding circuit, and their timing can be controlled by software. For example, for a low-speed device in which output data is not immediately set to high impedance, a bus IDLE state cycle can be input between that read cycle and the next memory access cycle. Thus, data collision on the data bus can be avoided.

Except for the start address of external flash memory region 0 (CS0), the start address, and interface timing characteristics of each external memory region are configurable. Furthermore, the data busses of CS0, CS1, CS3, and CS4 can be set to 8-bits or 16-bits. The data bus width of CS2 is always 8-bits.

An SDRAM memory region is set up within a MTC region in the same way as the other external memory areas.

External memory regions 0, 1, and 2, have special features that allow them to easily be used as a FLASH memory interface, CompactFlash interface and SmartMedia interface, respectively. The CompactFlash and SmartMedia interfaces share some pins, therefore it is not possible to simultaneously access a CompactFlash and SmartMedia card.

20.3.2 Boundary Addresses and Sizes of the External Memory Regions

The external memory area has a total addressable length of 128 Mbytes in the ARM memory map. The external memory area can be divided into a maximum of six regions (SDRAM, CS0-CS4). The boundary address of each memory region is set, by specifying the physical address value of the ARM memory space. The boundary address is set in six different boundary address registers (DPSTR0 - EMIF Region 0 End Address to DPSTR5 - EMIF Region 4 End Address). The boundary addresses are specified in units of 1 Mbyte. The default start addresses, which are set after reset, are shown in 0

Initial values of offset

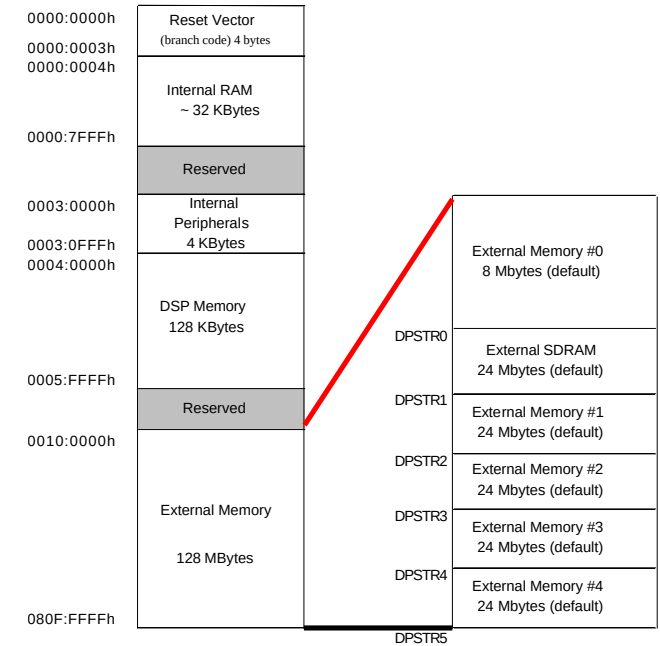
MTC region name	Initial boundary value (DPSTRn)	Corresponding boundary address	Corresponding memory	Corresponding Chip Select
External memory region 0	09h	0090:0000h	FLASH memory	FLS_CE
SDRAM region	21h	0210:0000h	SDRAM	SDR_CS
External memory region 1	39h	0390:0000h	Compact Flash	CFE1, CFE2
External memory region 2	51h	0510:0000h	Smart Media	CFE1
External memory region 3	69h	0690:0000h	General	EM_CE0
External memory region 4	81h	0810:0000h	General	EM_CE1

If 'N' represents the boundary address of a region, then 'N-1' is the largest address in that region and 'N' is the start address of the next subsequent region. Note that the start address of external memory region-0 is fixed at 0010:0000h. Size of a given region is obtained by subtracting the boundary address of previous region from current region. Lists the start and end addresses of external memory regions after reset (corresponding to Table 2).

Memory ranges after reset

MTC region name	Initial boundary value (DPSTRn)	Corresponding boundary address	Start address	End address
External memory region 0	09h	0090:0000h	0010:0000h	008F:FFFFh
SDRAM region	21h	0210:0000h	0090:0000h	020F:FFFFh
External memory region 1	39h	0390:0000h	0210:0000h	038F:FFFFh
External memory region 2	51h	0510:0000h	0390:0000h	050F:FFFFh
External memory region 3	69h	0690:0000h	0510:0000h	068F:FFFFh
External memory region 4	81h	0810:0000h	0690:0000h	080F:0000h

Figure 147. ARM Memory Map – DPSTRn register setting



20.3.3 Hardware interface

As external memory interfaces, DM270 has a general-purpose memory interface. The ARM can access SDRAM via the SDRAM controller module. The ARM can access general-purpose SRAM or flash ROM, a CompactFlash card, or a SmartMedia card via the external memory interface module. In access from ARM to external memory, a wait is automatically inserted, and its cycle varies depending on SDRAM frequency, external memory access time and so forth.

The functions of several pins in the external memory bus are shown in 0 and 0. For more information on the external memory and SDRAM bus pins refer to the TMS320DM270 Datasheet.

External Memory Bus Pins

Pin Name	I/O	Explanation
ARM_A[22:0]	O	ADDRESS bus Byte address for 8 bit device and half word address for 16 bit device is outputted. ARM_A[22]: -REG for CF/CF+
ARM_D[15:0]	I/O	DATA bus
EM_WE	O	Write enable code (FLSH_CE, CFE1, CFE2, EM_CS0, EM_CS1)
EM_RDWR	O	READ/WRITE Strobe code
EM_BEH	O	Upper byte enable code
EM_BEL	O	Lower byte enable code
EM_OE	O	Read enable code (FLSH_SE, EM_CS0, EM_CS1)
EM_WAIT	I	External wait code (EM_CS0 or EM_CS1)
CFE1	O	CF/CF+ [-CFE1] Smart Media [-CE]
CFE2	O	CF/CF+ [-CFE2] Smart Media [ALE]
CFOE	O	CF/CF+ [-OE] Smart Media [CLE]
CFIORD	O	CF/CF+ [-IORD] Smart Media [-RE]
CFIOWR	O	CF/CF+ [-IOWR] Smart Media [-WE]
CFRDY	I	CF/CF+ [READY/-BUSY] Smart Media [READY/-BUSY]
CFWAIT	I	CF/CF+ [-WAIT]
IOIS16	I	CF/CF+ [-IOIS16]

SDRAM Bus Pins

Pin Name	I/O	Explanation
SDR_DQ[31:16]	I/O	SDRAM data bus (32bit mode)
SDR_DQ[15:0]	I/O	SDRAM data bus (16/32bit mode)
SDR_A[12:0]	O	SDRAM address bus
SDR_A[14:13]	O	SDRAM bank address
SDR_RAS	O	SDRAM Row address strobe
SDR_CAS	O	SDRAM Column address strobe
SDR_WE	O	SDRAM Write enable
SDR_CS0	O	SDRAM Support chip select
SDR_CKE	O	SDRAM clock enable
SDR_DQMHH	O	SDRAM mask for DQ[31:24]
SDR_DQMHL	O	SDRAM mask for DQ[23:16]
SDR_DQMLH	O	SDRAM mask for DQ[15:8]
SDR_DQMLL	O	SDRAM mask for DQ[7:0]
SDR_CLK	O	SDRAM clock

A chip select (CS) signal is generated for each EMIF memory region. Correspondence between each memory region and CS is shown in 0

Initial Values of Chip Select Signals

Variable Area Name	Internal Chip Select Name	External Chip Select Pin Name
SDRAM area	SDR_CS	SDR_CS
External memory area 0	EM_CS0	FLS_CE
External memory area 1	EM_CS1	CFE1/CFE2
External memory area 2	EM_CS2	CFE1
External memory area 3	EM_CS3	EM_CE0
External memory area 4	EM_CS4	EM_CE1

If the regions are set so that they overlap, the chip select is valid for the region of higher priority ranking. If possible, try not to overlap memory areas.

Chip select signal priority order

Variable region name	Chip select	Priority
External memory region 0	FLS_CE	Low ↑ ↓ High
SDRAM region	SDR_CS	
External memory region 1	CFE1, CFE2	
External memory region 2	CFE1	
External memory region 3	EM_CE0	
External memory region 4	EM_CE1	High

20.3.4 Timing control

The timing for each external memory region can be configured by various register settings. This allows many different types of devices, which have different interface timing requirements, to be connected to each of the external memory regions. Also, an idle state can be inserted in order to prevent bus collision when performing continuous memory accesses. 0, 0, and 0 show the configurable timing characteristics for each external memory region, and Figure 148 shows the corresponding signal timing diagrams.

Configurable Timing Parameters for EM_CS0/EM_CS3/EM_CS4

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-16 clocks	1 CLK (ARM_CLK)	11
CS_SU	Number of Setup of CS	0-15 clocks	1 CLK	0
WE_SU	Number of Setup of WE	"	1 CLK	1
OE_SU	Number of Setup of OE	"	1 CLK	1
CS_WAIT	Number of Wait of CS	1-16 clocks	1 CLK	10
WE_WAIT	Number of Wait of WE	"	1 CLK	9
OE_WAIT	Number of Wait of OE	"	1 CLK	9
IDLE	Number of Idles	0-3 clocks	1 CLK	1

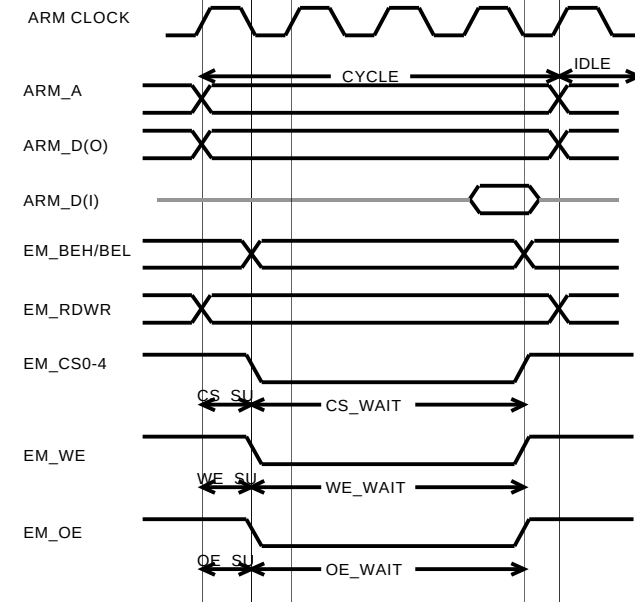
Configurable Timing Parameters for EM_CS1

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-32 clocks	1 CLK (ARM_CLK)	22
CS_SU	Number of Setup of CS	0-15 clocks	1 CLK	0
WE_SU	Number of Setup of WE	"	1 CLK	2
OE_SU	Number of Setup of OE	"	1 CLK	2
CS_WAIT	Number of Wait of CS	1-32 clocks	1 CLK	21
WE_WAIT	Number of Wait of WE	"	1 CLK	10
OE_WAIT	Number of Wait of OE	"	1 CLK	18
IDLE	Number of Idles	0-3 clocks	1 CLK	1

Configurable Timing Parameters for EM_CS2

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-16 clocks	1 CLK (ARM_CLK)	8
WE_SU	Number of Setup of WE	0-15 clocks	1 CLK	2
OE_SU	Number of Setup of OE	"	1 CLK	1
CS_WAIT	Number of Wait of CS	1-16 clocks	1 CLK	3
WE_WAIT	Number of Wait of WE	"	1 CLK	4
READ_SU	Number of Setup of READ	3-18 clocks	1 CLK	12
IDLE	Number of Idles	0-3 clocks	1 CLK	1

Figure 148. External Memory Bus Signals



The timing characteristics are controlled by the following registers:

- CS0CTRL1 - EMIF CS0 Control Register #1,
- CS0CTRL2 - EMIF CS0 Control Register #2,
- CS1CTRL1A - EMIF CS1 Control Register #1A,
- CS1CTRL1B - EMIF CS1 Control Register #1B,
- CS1CTRL2 - EMIF CS1 Control Register #2,
- CS2CTRL1 - EMIF CS2 Control Register #1,
- CS2CTRL2 - EMIF CS2 Control Register #2,
- CS3CTRL1 - EMIF CS3 Control Register #1,
- CS3CTRL2 - EMIF CS3 Control Register #2,
- CS4CTRL1 - EMIF CS4 Control Register #1 and,

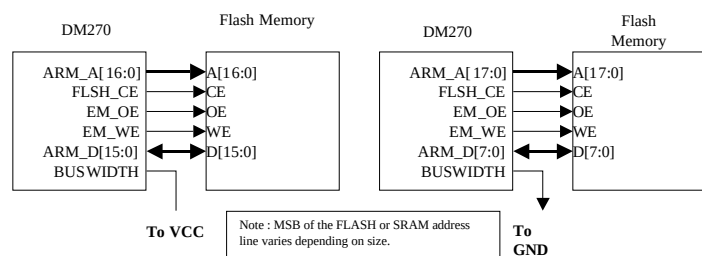
- CS4CTRL2 - EMIF CS4 Control Register #2

20.3.5 External Memory Region 0: The FLASH Memory Interface

Systems that use asynchronous FLASH ROM for program memory should attach the FLASH ROM to external memory region 0, since the ARM reset vector includes an instruction to jump to the start address of external memory region 0. The start address of external memory region 0 is 0x10:0000, and this address cannot be changed.

Figure 149 shows the FLASH memory interface. Refer to the TMS320DM270 Datasheet for more information on the FLASH interface signals. The FLASH interface uses ARM_A, FLASH_CE, EM_OE, EM_WE, ARM_D, and BUSWIDTH. The write enable signal (EM_WE) is used commonly by all devices connected to the external memory interface. When the ARM accesses the flash region, FLASH_CE becomes active. In addition to FLASH memory, it is also possible to connect SRAM or PROM devices to the FLASH interface.

Figure 149. FLASH Memory Interface



20.3.5.1 Flash Memory Interface Data Bus

External memory region 0 supports 8-bit and 16-bit wide FLASH. As shown in 0, the BUSWIDTH pin is used to set the data bus width of the region 0 to 8 or 16 bits.

FLASH_CE Bus Width

BUSWIDTH state	FLASH_CE Bus Width
Low	8-bit
High	16-bit

Although the FLASH data bus width can be set to 8 or 16-bits, a system may need to access 8, 16, and/or 32-bit data from the FLASH memory. For example, the LDR command will require a 32-bit read access. As shown in 0 and Figure 150, the external memory interface will automatically perform the correct number of accesses for an 8, 16, or 32-bit accesses, given an 8 or 16-bit wide FLASH

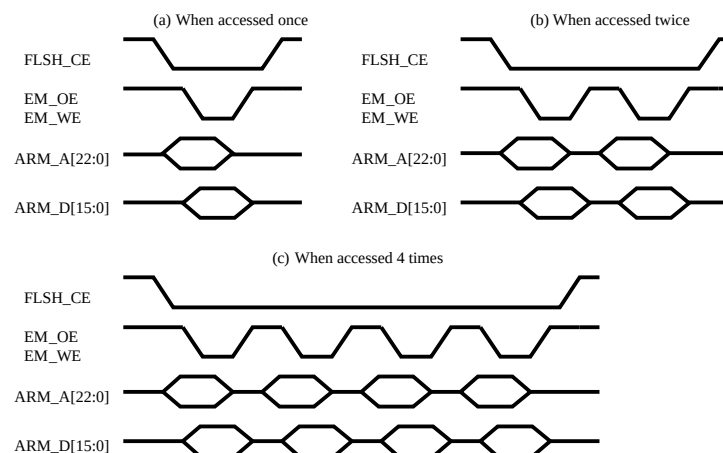
memory. When using an 8-bit wide FLASH memory, 1 access is required for an 8-bit access, 2 accesses are required for a 16-bit access, and 4 accesses are required for a 32-bit access. When using a 16-bit wide FLASH memory, 1 access is required for an 8-bit access, 1 access is required for a 16-bit access, and 2 accesses are required for a 32-bit access.

To perform an 8-bit read from a 16-bit wide FLASH memory, the required 8-bit portion is loaded and the remaining 8-bits are ignored.

Relationship Between Access Size and Flash Memory Data Bus Width

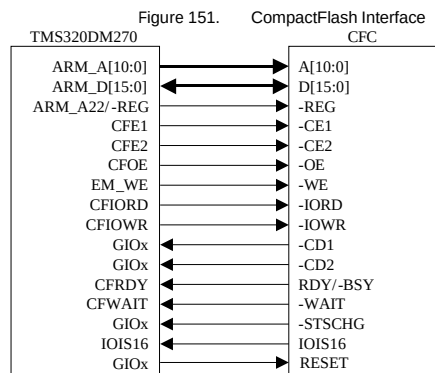
Access Size	Example of command from ARM	8 bits wide	16 bits wide
8-bit access	LDRB/STRB, etc.	1 time	1 time
16-bit access	LDRH/STRH, etc.	2 times	1 time
32-bit access	LDR/STR, etc.	4 times	2 times

Figure 150. Flash Memory Access Timing Diagrams



20.3.6 External Memory Region 1: The CompactFlash Interface

External memory region 1 can be used to interface CompactFlash cards. The CompactFlash interface is shown in Figure 151. For detailed information on the CompactFlash interface signals, refer to the TMS320DM270 Datasheet. The interface supports memory mode and IO mode, however, true IDE mode is not supported.



20.3.6.1 The RDY/-BSY Signal in the CompactFlash Interface

CompactFlash cards and SmartMedia cards use the RDY/-BSY signal to indicate busy status to the DM270. The RDY/-BSY signal is input to the DM270. RDY/-BSY is connected to the CFRDY pin. When the CFRDY signal is connected to a CompactFlash card, set bit CFRSL of BUSWAITMD - EMIF Bus Wait Mode Register to 0. However, when CFRDY is connected to a SmartMedia card, set CFRSL of BUSWAITMD - EMIF Bus Wait Mode Register to 1.

In a system that has been designed to have dual access to both a CompactFlash card and a SmartMedia card, connect CFRDY to the RDY/-BSY pin of either the CompactFlash card or SmartMedia card and connect an arbitrary GIO to the RDY/-BSY pin of the card not connected to the CFRDY pin.

When CFRW in the BUSWAITMD - EMIF Bus Wait Mode Register, is set to 1, the DM270 will wait to access the card until the CFRDY signal is low. When CFRW is set to 0, the DM270 will not wait to access the card; the bus cycle will start independent of the state of CFRDY.

20.3.6.2 CompactFlash Data Bus

The CompactFlash interface supports dynamic bus sizing for I/O mode. Data access will be either 8-bit or 16-bit mode depending on the level of the IOIS16 pin. When IOIS16 is low, the CompactFlash interface operates with an 8-bit bus, and when IOIS16 is high, the interface operates with a 16-bit bus. Dynamic bus sizing is enabled when BUSSZ is set in CFCTRL2 - EMIF CFC Control Register #2. When BUSSZ is cleared, the bus size is always 16-bits regardless of the IOIS16 signal. Similar to the FLASH interface, the correct number accesses are automatically performed when 32-bit, 16-bit, and 8-bit data is transferred, given that the data bus is set to be 8 or 16-bits wide (see 0 and Figure 150).

20.3.6.3 CompactFlash Interface Memory Map

The CompactFlash Interface Memory Map is shown in 0The table shows the offset address for the CFC Memory Space (Attribute and Common Memory Space) and the CFC IO Space. As shown in the table, the CFC Memory Space can be accessed through offset addresses 00:0000 – 00:07FF, and the CFC IO Space can be accessed through offset addresses 00:0800 – 00:0FFF. The addresses are offset from the start address of external memory region 1. Refer to the section on 20.3.2 Boundary Addresses and Sizes of the External Memory Regions for information on setting the start address of external memory regions.

CompactFlash Interface Memory Map			
External Memory Area	Offset Address	Size	CompactFlash Interface Function
EM_CS1	00:0000 – 00:07FF	2-Kbytes	CFC Memory Space (Attribute and Common)
	00:0800 – 00:0FFF	2-Kbytes	CFC IO Space

20.3.6.4 Accessing the CompactFlash memory space (attribute and common memory region)

The memory map in 0 shows the CompactFlash Card Memory Space. When the CompactFlash card Memory Space is accessed, the REG signal controls which region, Attribute or Common Memory, is accessed. The REG signal is output on pin ARM_A22. When REG is low, the attribute memory region is accessed. When REG is high, the common memory region is accessed. The REG signal can be controlled by CFMOD in CFCTRL2 - EMIF CFC Control Register #2.

Note: when I/O space is accessed, the REG signal is always output low regardless of the CFMOD bit setting.

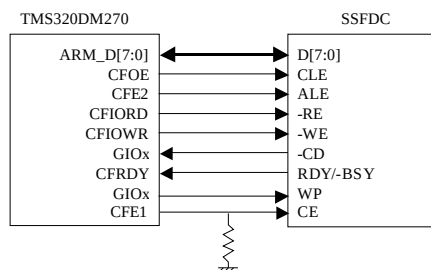
20.3.6.5 Resetting the CompactFlash

Furthermore, the CompactFlash reset signal can be controlled via a GIO pin. Refer to the GIO section for information on how to use GIO pins for simple control.

20.3.7 External Memory Region 2: The SmartMedia Interface

External memory interface 2 has a built-in SmartMedia interface. Figure 152 shows the SmartMedia interface. The interface includes ARM_D[7:0], ARM_A20, ARM_A21, CFIORD, CFIOWR, CFRDY or any available GIO pin. See the TMS320DM270 for more detail on these SmartMedia interface signals.

Figure 152. SmartMedia Interface



20.3.7.1 The RDY/-BSY Signal in the SmartMedia Interface

CompactFlash cards and SmartMedia cards use the RDY/-BSY signal to indicate busy status to the DM270. The RDY/-BSY signal is input to the DM270. RDY/-BSY is connected to the CFRDY pin. When the CFRDY signal is connected to a CompactFlash card, set bit CFRSL of BUSWAITMD - EMIF Bus Wait Mode Register to 0. However, when CFRDY is connected to a SmartMedia card, set CFRSL of BUSWAITMD - EMIF Bus Wait Mode Register to 1.

In a system that has been designed to have dual access to both a CompactFlash card and a SmartMedia card, connect CFRDY to the RDY/-BSY pin of either the CompactFlash card or SmartMedia card and connect an arbitrary GIO to the RDY/-BSY pin of the card not connected to the CFRDY pin.

When CFRW in the BUSWAITMD - EMIF Bus Wait Mode Register, is set to 1, the DM270 will wait to access the card until the CFRDY signal is low. When CFRW is set to 0, the DM270 will not wait to access the card; the bus cycle will start independent of the state of CFRDY.

20.3.7.2 SmartMedia Write Protect

You can control the SmartMedia Write Protect signal [WP] using any available, arbitrary GIO pin. Refer to the GIO section for information on how to use GIO pins for simple control. WP is controlled using a GIO pin configured to operate in normal GIO mode.

20.3.7.3 SmartMedia Data Bus

Since SmartMedia uses one 8-bit data bus for command, address, and data, external memory interface 2 is always an 8-bit wide interface. Similar to the FLASH interface, when 32-bit, 16-bit, and 8-bit data is transferred, an 8-bit access is automatically generated 4 times, 2 times, and 1 time respectively (see 0 and Figure 150).

20.3.7.4 SmartMedia Interface Memory Map

The SmartMedia Interface Memory Map is shown in 0. This table shows the offset address for command, address, and data. The ARM software should write SmartMedia commands to the address offset at 00:0000, write SmartMedia addresses to offset 00:0004, and should read and write SmartMedia data from offset 00:0008. The offset addresses are relative to the start address of external memory 2. Refer to the section "20.3.2 Boundary Addresses and Sizes of the External Memory Regions" for information on setting the start address of external memory regions.

External Memory Area	Offset Address	Size	Smart Media Interface Function
EM_CS2	00:0000	4-bytes	Smart Media Command Write
	00:0004	4-bytes	Smart Media Address Write
	00:0008	4-bytes	Smart Media Data Read/Write

20.3.7.5 Controlling the SmartMedia Chip Enable

The SmartMedia Chip Enable signal can be controlled by SCEH in SMCTRL - EMIF SmartMedia Control Register. Before accessing the SmartMedia Card, enable the SmartMedia Chip Enable signal by clearing bit SCEH in the register SMCTRL - EMIF SmartMedia Control Register. When SCEH is cleared, the chip enable signal, SCEH, is set low (active).

20.3.7.6 Controlling the SmartMedia Page Size

The SmartMedia page size can be set to 256+8-bytes per page or 512+16-bytes per page. This is set in PAGESZ - EMIF SmartMedia Page Size Register.

20.3.7.7 SmartMedia Interface Error Correction

The SmartMedia interface supports hardware error detection. During read and write, hardware error correction code (ECC) calculation is performed every 256-bytes. The generated ECC can be read from ECC1CP - EMIF ECC Area 1 CP Register, ECC1LP - EMIF ECC Area 1 LP Register, ECC2CP - EMIF ECC Area 2 CP Register, and ECC2LP - EMIF ECC Area 2 LP Register. ECC1CP and

ECC1LP are used for Storing Area -1 and ECC2CP and ECC2LP are used for Storing Area -2.

When the page size is configured to 256 + 8 bytes per page, Storing Area -1 corresponds to bytes 0-255 of the even numbered pages, and Storing Area -2 corresponds to bytes 0-255 of odd numbered pages.

When the page size is configure to 512 + 16 bytes per page, Storing Area -1 corresponds to bytes 0-255 and Storing Area -2 corresponds to bytes 256-511.

Although the hardware calculates ECC, the ARM software must perform ECC data comparison and correction. If 512+16 byte page SmartMedia is used, the calculated ECC is stored at ECC1CP and ECC1LP after the first half (256 bytes) of data is read, and then ECC2CP and ECC2LP are updated after the second half (256-bytes) of data is read. The procedure in the case where smart media is used is shown below.

512+16bytes/page mode

- o Write "1" to ECLR in ECCCLR - EMIF ECC Clear Register to clear the previous ECC results.
- o Read or Write the first half (0-255) of data -> ECC1CP & ECC1LP is updated.
- o Read or Write the second half (256-511) of data -> ECC2CP & ECC2LP is updated.
- o Check the ECC stored at ECC1CP, ECC1LP, ECC2CP and ECC2LP
- o Write "1" to ECLR in ECCCLR - EMIF ECC Clear Register to clear the ECC for the next page access.

256+8bytes/page mode

- o Write "1" to ECLR in ECCCLR - EMIF ECC Clear Register to clear the previous ECC results.
- o Read or Write the data area (0-255) of Even-numbered page -> ECC1CP & ECC1LP is updated.
- o Read or Write the data area (0-255) of Odd-numbered page -> ECC2CP & ECC2LP is updated.
- o Check the ECC stored at ECC1CP, ECC1LP, ECC2CP and ECC2LP
- o Write "1" to ECLR in ECCCLR - EMIF ECC Clear Register to clear the ECC for the next pair of pages.

"Error Detection & Correction" can be performed by comparing the calculated ECC (ECC1CP, ECC1LP, ECC2CP and ECC2LP) and the ECC stored at the redundant area of the page.

20.3.8 CompactFlash Interface and SmartMedia Interface Interrupts and Signal Status

20.3.8.1 Interface Interrupts

The external memory interface includes two interrupts, INT15, INT16. INT15 is generated when EMIF DMA completes, and this interrupt is described in the section "EMIF DMA Interrupt". INT16 is generated due to conditions specified in BUSINTEN - EMIF Bus Interrupt Enable Register, and this interrupt is described in this section.

CFC Busy interrupt can be enabled or disabled by setting the bits in BUSINTEN - EMIF Bus Interrupt Enable Register. The status of CFC Busy interrupt can be read from the bits in BUSSTS - EMIF Bus Status Register. CFC Busy interrupt corresponds to INT16.

20.3.8.2 External Memory Interface Signal Status

Read the bits in BUSSTS - EMIF Bus Status Register to monitor the state of various input signals used by the External Memory Interface.

20.3.9 External Memory Regions 3 and 4

20.3.9.1 Data Bus Width

The data width of external memory areas 3 and 4 can be either 8 or 16 bits. The BUSW3 and BUSW4 bits in CS3CTRL2 - EMIF CS3 Control Register #2 and CS4CTRL2 - EMIF CS4 Control Register #2 configure the data width of region 3 and 4, respectively. By default, after reset, the data width is set to 8 bits.

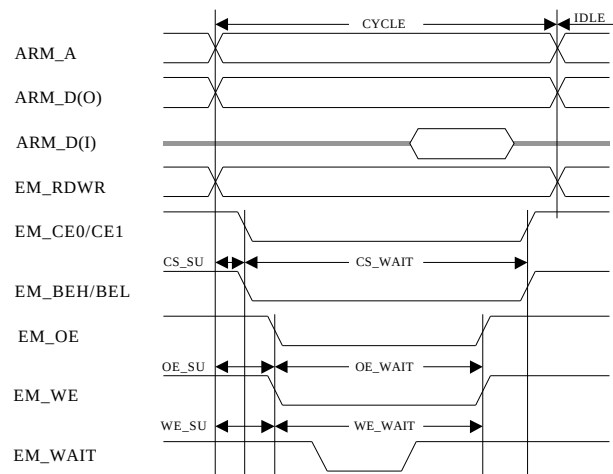
20.3.9.2 External Wait

The EM_WAIT signal is available for region 3 and/or region 4, which enables asynchronous device interface capability. Refer to the TMS320DM270 Datasheet for detailed information on the signal interface related to the external wait feature. To enable the EM_WAIT signal for region 3 or 4, set EMWE3 or EMWE4 in BUSCTRL - EMIF Bus Control Register, respectively.

When EM_WAIT is enabled and driven low during a bus cycle by an external device, an extra wait state (hardware wait-state) is added to the programmed wait states (software wait-states).

For instance, if CS_WAIT = 5 and EM_WAIT is held low for 5 clock cycles, the total number of wait states will be 10.

Figure 153. External memory bus timing with hardware wait-states



When the WAIT signal is used, the setup and wait time for the signals CS3/CS4 and OE must be configured such that the rising edges of these signals are less than 1.5 ARM clock cycle before the end of the address valid area (refer to the TMS320DM270 Datasheet for more information).

$$CS_SU + CS_WAIT \geq CYCLE - 1.5 * clocks$$

$$OE_SU + OE_WAIT \geq CYCLE - 1.5 * clocks$$

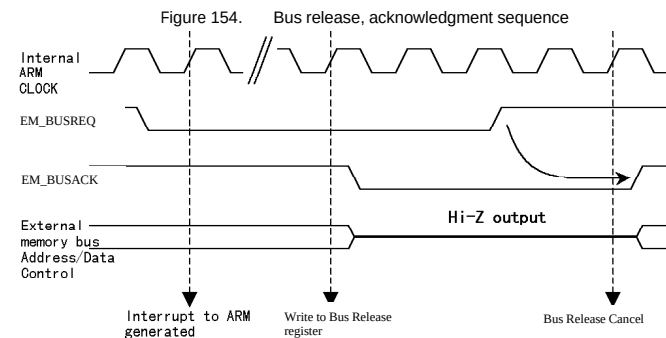
20.3.10 Bus Release

The EMIF bus and SDRAM bus can be released for an external device by using BUSRLS - EMIF Bus Release Control Register.

When EMIF bus is released all EMIF related signal (Address/Data/Chip select) become Hi-Z. Clearing the EMIF bus release (EMREL) turns on the EMIF buses.

Additionally, the SDRAM bus can be released to an external device. All SDRAM buses (ADDRESS/DATA/RAS/CAS/CS/WE/DQM/CKE/CLK) can be released (output Hi-Z) by setting the SDRAM bus release register, SDREL in BUSRLS - EMIF Bus Release Control Register, in the external memory interface module. SDRAM buses are turned on by clearing the SDRAM bus release register. By using this function, SDRAM can be shared between external devices. SDRAM bus outputs Hi-Z while the SDRAM bus open register is on.

Figure below shows an example sequence of bus release, acknowledge during release of EMIF bus. Since EM_BUSREQ, EM_BUSACK signals are not available in DM270, the user must use arbitrary GIO's to implement bus request/acknowledge type of handshake. Similarly for SDRAM bus release



20.3.11 EMIF access priority

The ARM, the Image Buffer's DMA controller (DSP), Host Port Interface Bus (DSP), and the EMIF's DMA controller can all access the EMIF. PRICTL - EMIF Priority Control Register allows you to configure EMIF access priority. By default the priority of EMIF access is as follows:
(highest) ARM > EMIF DMA > Image Buffer DMA > HPIB (DSP) (lowest)

20.4 External Memory Interface Register Map (EMIF)

Address	Register	Description
0003:0A00	CS0CTRL1	CS0 Control Register #1
0003:0A02	CS0CTRL2	CS0 Control Register #2
0003:0A04	CS1CTRL1A	CS1 Control Register #1A
0003:0A06	CS1CTRL1B	CS1 Control Register #1B
0003:0A08	CS1CTRL2	CS1 Control Register #2
0003:0A0A	CS2CTRL1	CS2 Control Register #1
0003:0A0C	CS2CTRL2	CS2 Control Register #2
0003:0A0E	CS3CTRL1	CS3 Control Register #1
0003:0A10	CS3CTRL2	CS3 Control Register #2
0003:0A12	CS4CTRL1	CS4 Control Register #1
0003:0A14	CS4CTRL2	CS4 Control Register #2
0003:0A16	BUSCTRL	Bus Control Register
0003:0A18	BUSRSL	Bus Release Control Register
0003:0A1A	CFCTRL1	CFC Control Register #1
0003:0A1C	CFCTRL2	CFC Control Register #2
0003:0A1E	SMCTRL	SmartMedia Control Register
0003:0A20	BUSINTEN	Bus Interrupt Enable Register
0003:0A22	RSVO	Reserved. Do not use
0003:0A24	BUSSTS	Bus Status Register
0003:0A26	BUSWAITMD	Bus Wait Mode Register
0003:0A28	ECC1CP	ECC Area 1 CP Register
0003:0A2A	ECC1LP	ECC Area 1 LP Register
0003:0A2C	ECC2CP	ECC Area 2 CP Register
0003:0A2E	ECC2LP	ECC Area 2 LP Register
0003:0A30	ECCCLR	ECC Clear Register
0003:0A32	PAGESZ	SmartMedia Page Size Register
0003:0A34	IMGDSPDEST	Image Buffer/DSP(HPIB) Device Select Register
0003:0A36	PRICTL	Priority Control Register
0003:0A38	SOURCEADDH	DMA Source Address High Register
0003:0A3A	SOURCEADDL	DMA Source Address Low Register
0003:0A3C	DESTADDH	DMA Destination Address High Register
0003:0A3E	DESTADDL	DMA Destination Address Low Register
0003:0A40	DMA SIZE	DMA Transfer Size Register
0003:0A42	DMADEVSEL	DMA Device Select Register
0003:0A44	DMACTL	DMA Control Register
0003:0A46	IMGDSPADDH	Image Buffer/DSP(HPIB) Destination Address High
0003:0A48	IMGDSPADDL	Image Buffer/DSP(HPIB) Destination Address Low
0003:0A4A	DPSTR0	EMIF Region 0 End Address
0003:0A4C	DPSTR1	SDRAM Region End Address
0003:0A4E	DPSTR2	EMIF Region 1 End Address
0003:0A50	DPSTR3	EMIF Region 2 End Address
0003:0A52	DPSTR4	EMIF Region 3 End Address
0003:0A54	DPSTR5	EMIF Region 4 End Address
0003:0A56	TEST	Test Register. Do not use

20.5 External Memory Interface Registers

20.5.1 CS0CTRL1 - EMIF CS0 Control Register #1

CS0CTRL1		0003:0A00				offset: 0x00										default: 0x889A	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	OEC0[3]	OEC0[2]	OEC0[1]	OEC0[0]	WEC0[3]	WEC0[2]	WEC0[1]	WEC0[0]	CEC0[3]	CEC0[2]	CEC0[1]	CEC0[0]	CYCL0[3]	CYCL0[2]	CYCL0[1]	CYCL0[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0	

Bit	Name	Reset Value	R/W	Function
15:12	OEC0	8	R/W	<u>Output Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (OEC + 1) ARM clock cycles.
11:8	WEC0	8	R/W	<u>Write Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (WEC + 1) ARM clock cycles.
7:4	CEC0	9	R/W	<u>Chip Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (CEC + 1) ARM clock cycles.
3:0	CYCL0	10	R/W	<u>Cycle Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The cycle width is (CYCL + 1) ARM clock cycles

20.5.2 CS0CTRL2 - EMIF CS0 Control Register #2

CS0CTRL2	0003:0A02			offset:	0x02			default: 0x1110										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	IDLE0[1]	IDLE0[0]	OESU0[3]	OESU0[2]	OESU0[1]	OESU0[0]	WESU0[3]	WESU0[2]	WESU0[1]	WESU0[0]	CESU0[3]	CESU0[2]	CESU0[1]	CESU0[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	0	0	0	1	0	0	0	1	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	IDLE0	1	R/W	Number of IDLE State Clocks 00: 0 01: 1 10: 2 11: 3
11:8	OESU0	1	R/W	Number of Clocks of Output Enable Setup 0000: 0 0001: 1 : 1110: 14 1111: 15
7:4	WESU0	1	R/W	Number Clocks of Write Enable Setup 0000: 0 0001: 1 : 1110: 14 1111: 15
3:0	CESU0	0	R/W	Number of Clocks of Chip Enable Setup 0000: 0 0001: 1 : 1110: 14 1111: 15

20.5.3 CS1CTRL1A - EMIF CS1 Control Register #1A

CS1CTRL1A	0003:0A04			offset:	0x04			default: 0x1415										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	CEC1[4]	CEC1[3]	CEC1[2]	CEC1[1]	CEC1[0]	RSV	RSV	RSV	CYCL1[4]	CYCL1[3]	CYCL1[2]	CYCL1[1]	CYCL1[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	1	0	1	0	0	-	-	-	1	0	1	0	1		

Bit	Name	Reset Value	R/W	Function
15:13	RSV			
12:8	CEC1	20	R/W	Chip Enable Width 0000: 1 0001: 2 : 1110: 31 1111: 32 The width is (CEC1 + 1) ARM clock cycles.
7:5	RSV			
4:0	CYCL1	21	R/W	Cycle Width 00000: 2 00001: 2 : 11110: 31 11111: 32 The width is (CYCL1 + 1) ARM clock cycles, except when set value is 0000, the width is 2 ARM clock cycles.

20.5.4 CS1CTRL1B - EMIF CS1 Control Register #1B

CS1CTRL1B	0003:0A06			offset:	0x06			default: 0x1109										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	OEC1[4]	OEC1[3]	OEC1[2]	OEC1[1]	OEC1[0]	RSV	RSV	RSV	WEC1[4]	WEC1[3]	WEC1[2]	WEC1[1]	WEC1[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	1	0	0	0	1	-	-	-	0	1	0	0	1		

Bit	Name	Reset Value	R/W	Function
15:13	RSV			
12:8	OEC1	17	R/W	Output Enable Width 00000: 1 00001: 2 : 11110: 31 11111: 32 The width is (set value + 1) ARM clock cycles.
7:5	RSV			
4:0	WEC1	9	R/W	Write Enable Width 00000: 1 00001: 2 : 11110: 31 11111: 32 The width is (set value + 1) ARM clock cycles.

20.5.5 CS1CTRL2 - EMIF CS1 Control Register #2

CS1CTRL2		0003:0A08				offset:		0x08								default:		0x1220	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	IDLE1[1]	IDLE1[0]	OESU1[3]	OESU1[2]	OESU1[1]	OESU1[0]	WESU1[3]	WESU1[2]	WESU1[1]	WESU1[0]	CESU1[3]	CESU1[2]	CESU1[1]	CESU1[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	1	0	0	1	0	0	0	0	1	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	IDLE1	1	R/W	<u>Number of IDLE State Clocks</u> 00: 0 01: 1 10: 2 11: 3
11:8	OESU1	2	R/W	<u>Number of Clocks of Output Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
7:4	WESU1	2	R/W	<u>Number Clocks of Write Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
3:0	CESU1	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15

20.5.6 CS2CTRL1 - EMIF CS2 Control Register #1

CS2CTRL1				0003:0A0A				offset: 0x0A												default: 0x3207			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	OEC2[3]	OEC2[2]	OEC2[1]	OEC2[0]	WEC2[3]	WEC2[2]	WEC2[1]	WEC2[0]	RSV	RSV	RSV	RSV	CYCL2[3]	CYCL2[2]	CYCL2[1]	CYCL2[0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W							
Default	0	0	1	1	0	0	1	0	-	-	-	-	0	1	1	1							

Bit	Name	Reset Value	R/W	Function
15:12	OEC2	3	R/W	<u>Output Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (OEC + 1) ARM clock cycles.
11:8	WEC2	2	R/W	<u>Write Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (WEC + 1) ARM clock cycles.
7:4	RSV			<u>Chip Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (CEC + 1) ARM clock cycles.
3:0	CYCL2	7	R/W	<u>Cycle Width</u> 0000: 2 0001: 2 0010: 3 : 1110: 15 1111: 16 The cycle width is (CYCL + 1) ARM clock cycles, except when set to 0, in which case the width is 2 ARM clock cycles.

20.5.7 CS2CTRL2 - EMIF CS2 Control Register #2

CS2CTRL2		0003:0A0C				offset: 0x0C										default: 0x1129	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	IDLE2[1]	IDLE2[0]	OESU2[3]	OESU2[2]	OESU2[1]	OESU2[0]	WESU2[3]	WESU2[2]	WESU2[1]	WESU2[0]	RESU2[3]	RESU2[2]	RESU2[1]	RESU2[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	1	0	0	0	1	0	0	1	0	1	0	0	1	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	IDLE2	1	R/W	<u>Number of IDLE State Clocks</u> 00: 0 01: 1 10: 2 11: 3
11:8	OESU2	1	R/W	<u>Number of Clocks of Output Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
7:4	WESU2	2	R/W	<u>Number Clocks of Write Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
3:0	RESU2	9	R/W	<u>Number of Clocks of Chip Enable Setup</u> 0000: 3 0001: 4 0010: 5 : 1110: 17 1111: 18 The width is (RESU + 3) ARM clock cycles.

20.5.8 CS3CTRL1 - EMIF CS3 Control Register #1

CS3CTRL1				0003:0A0E				offset: 0x0E												default: 0x889A			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	OEC3[3]	OEC3[2]	OEC3[1]	OEC3[0]	WEC3[3]	WEC3[2]	WEC3[1]	WEC3[0]	CEC3[3]	CEC3[2]	CEC3[1]	CEC3[0]	CYCL3[3]	CYCL3[2]	CYCL3[1]	CYCL3[0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W							
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0							

Bit	Name	Reset Value	R/W	Function
15:12	OEC3	8	R/W	<u>Output Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (OEC + 1) ARM clock cycles.
11:8	WEC3	8	R/W	<u>Write Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (WEC + 1) ARM clock cycles.
7:4	CEC3	9	R/W	<u>Chip Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (CEC + 1) ARM clock cycles.
3:0	CYCL3	10	R/W	<u>Cycle Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The cycle width is (CYCL + 1) ARM clock cycles

20.5.9 CS3CTRL2 - EMIF CS3 Control Register #2

CS3CTRL2		0003:0A10				offset: 0x10		default: 0x1110									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	BUSW3	IDLE3[1]	IDLE3[0]	OESU3[3]	OESU3[2]	OESU3[1]	OESU3[0]	WESU3[3]	WESU3[2]	WESU3[1]	WESU3[0]	CESU3[3]	CESU3[2]	CESU3[1]	CESU3[0]	
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	BUSW3	0	R/W	External Device Bus Width 0: 8-bits wide 1: 16-bits wide
13:12	IDLE3	1	R/W	<u>Number of IDLE State Clocks</u> 00: 0 01: 1 10: 2 11: 3
11:8	OESU3	1	R/W	<u>Number of Clocks of Output Enable Setup</u> 0000: 0 0001: 1 0010: 2 : 1110: 14 1111: 15
7:4	WESU3	1	R/W	<u>Number Clocks of Write Enable Setup</u> 0000: 0 0001: 1 0010: 2 : 1110: 14 1111: 15
3:0	CESU3	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> 0000: 0 0001: 1 0010: 2 : 1110: 14 1111: 15

20.5.10 CS4CTRL1 - EMIF CS4 Control Register #1

CS4CTRL1				0003:0A12				offset: 0x12												default: 0x889A			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	OEC4[3]	OEC4[2]	OEC4[1]	OEC4[0]	WEC4[3]	WEC4[2]	WEC4[1]	WEC4[0]	CEC4[3]	CEC4[2]	CEC4[1]	CEC4[0]	CYCL4[3]	CYCL4[2]	CYCL4[1]	CYCL4[0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W							
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0							

Bit	Name	Reset Value	R/W	Function
15:12	OEC4	8	R/W	<u>Output Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (OEC + 1) ARM clock cycles.
11:8	WEC4	8	R/W	<u>Write Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (WEC + 1) ARM clock cycles.
7:4	CEC4	9	R/W	<u>Chip Enable Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The width is (CEC + 1) ARM clock cycles.
3:0	CYCL4	10	R/W	<u>Cycle Width</u> 0000: 1 0001: 2 : 1110: 15 1111: 16 The cycle width is (CYCL + 1) ARM clock cycles

20.5.11 CS4CTRL2 - EMIF CS4 Control Register #2

CS4CTRL2		0003:0A14				offset: 0x14										default: 0x1110	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	CESL4	BUSW4	IDLE4[1]	IDLE4[0]	OESU4[3]	OESU4[2]	OESU4[1]	OESU4[0]	WESU4[3]	WESU4[2]	WESU4[1]	WESU4[0]	CESU4[3]	CESU4[2]	CESU4[1]	CESU4[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	CESL4	0	R/W	<u>Polarity of chip enable of memory area 4</u> » 0: Chip enable (CE1) is active LOW « » 1: Chip enable (CE1) is active HIGH «
14	BUSW4	0	R/W	<u>External Device Bus Width</u> » 0: 8-bits wide « » 1: 16-bits wide «
13:12	IDLE4	1	R/W	<u>Number of IDLE State Clocks</u> 00: 0 01: 1 10: 2 11: 3
11:8	OESU4	1	R/W	<u>Number of Clocks of Output Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
7:4	WESU4	1	R/W	<u>Number Clocks of Write Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15
3:0	CESU4	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> 0000: 0 0001: 1 : 1110: 14 1111: 15

20.5.12 BUSCTRL - EMIF Bus Control Register

BUSCTRL		0003:0A16				offset: 0x16												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	EMWE4	EMWE3			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0			

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	EMWE4	0	R/W	<u>CS4 External Wait (EM_WAIT) Enable</u> » 0: External wait when CS4 is accessed is invalid « » 1: External wait when CS4 is accessed is valid «
0	EMWE3	0	R/W	<u>CS4 External Wait (EM_WAIT) Enable</u> » 0: External wait when CS3 is accessed is invalid « » 1: External wait when CS3 is accessed is valid «

20.5.13 BUSRLS - EMIF Bus Release Control Register

BUSRLS		0003:0A18				offset: 0x18				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BUSSEL	SDREL	EMREL	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	BUSSEL	0	R/W	<u>SDRAM bus sharing mode</u> » 0: Normal Mode « » 1: SDRAM bus is shared for use (CS0-CS4) «
1	SDREL	0	R/W	<u>SDRAM Bus Open Control</u> » 0: SDRAM bus is not opened « Bus open is cleared by writing '0' » 1: SDRAM bus is opened «
0	EMREL	0	R/W	<u>External Memory Bus Open Control</u> » 0: External memory bus is not opened « Bus open is cleared when "0" is written or BUS_ACK becomes "1". » 1: External memory bus is opened «

20.5.14 CFCTRL1 - EMIF CFC Control Register #1

CFCTRL1		0003:0A1A				offset:		0x1A								default:		0x0000	
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CFSEL		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	9		

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	CFSEL	0	R/W	Select CF card interface or Smart Media Card interface » 0: CF « » 1: Smart Media «

20.5.15 CFCTRL2 - EMIF CFC Control Register #2

CFCTRL2		0003:0A1C				offset:		0x1C								default:		0x0000	
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BUSSZ	RSV	RSV	RSV	CFMOD			
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	-	-	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	-	0		

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	BUSSZ	0	R/W	CFC Dynamic Bus Sizing Enable » 0: Use 16-bit wide bus to access CFC « » 1: Use dynamic bus sizing when IO space of CF card is accessed « The IOIS16 pin is used for dynamic bus sizing.
3:1	RSV			Reserved
0	CFMOD	0	R/W	CFC Memory Mode REG Signal Control » 0: Output low REG (ARM_A22) when accessing CFC memory « » 1: Output high REG (ARM_A22) when accessing CFC memory « When the CF card memory space is accessed (in memory mode), the REG signal controls which region is accessed. When REG is low, the attribute memory region can be accessed. When REG is high, the common memory region can be accessed. When the I/O space is accessed, the REG signal is always output low regardless of the CFMOD bit setting. The REG signal is output on pin ARM_A22.

20.5.16 SMCTRL - EMIF SmartMedia Control Register

SMCTRL		0003:0A1E				offset:		0x1E								default:		0x0001	
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SCEH		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1		

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	SCEH	1	R/W	SmartMedia Chip Enable Pin (SCE) » 0: SCE pin is set low « » 1: SCE pin is set high «

20.5.17 BUSINTEN - EMIF Bus Interrupt Enable Register

BUSINTEN		0003:0A20				offset:		0x20								default:		0x0000	
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CBCND	CBIEN		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0		

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	CBCND	0	R/W	CFC Busy Interrupt Condition » 0: Interrupt generated when CFRDY pin changes '0'-'>'1' « » 1: Interrupt generated when CFRDY pin changes '1'-'>'0' «
0	CBIEN	0	R/W	CFC Busy Interrupt Enable » 0: Disable « » 1: Enable « Interrupt is generated when the CFRDY pin changes as specified by CBCND

20.5.18 RSV0 - EMIF Reserved - do not use

RSV0		0003:0A22				offset:		0x22								default:		0x0000	
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		

Bit	Name	Reset Value	R/W	Function
15:0	RSV			Reserved

20.5.19 BUSSTS - EMIF Bus Status Register

BUSSTS				0003:0A24				offset: 0x24				default: 0x000F				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	EM_WAIT	IOIS16	CFWAIT	CFRDY
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R	R	R	R
Default	-	-	-	-	-	-	-	-	-	-	-	-	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	EM_WAIT	1	R	EM_WAIT pin status
2	IOIS16	1	R	IOIS16 pin status
1	CFWAIT	1	R	CFWAIT pin status
0	CFRDY	1	R	CFRDY pin status

20.5.20 BUSWAITMD - EMIF Bus Wait Mode Register

BUSWAITMD		0003:0A26				offset: 0x26										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CFRSL	CFRW	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	CFRSL	0	R/W	CF Ready Select (Specify the kind of card connected to the CFRDY terminal) » 0: Connect CF card RDY/BSYn to CFRDY terminal « » 1: Connect SmartMedia R/Bn to CFRDY terminal «
0	CFRW	0	R/W	CFRDY Wait Defines whether or not to control the start of bus cycle by CFRDY terminal when the card specified in CFRSL (above) is accessed » 0: Access the card regardless of CFRDY terminal state « » 1: Do not access card while The CFRDY terminal is low «

20.5.21 ECC1CP - EMIF ECC Area 1 CP Register

ECC1CP		0003:0A28				offset: 0x28										default: 0x00FF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	1CP[5]	1CP[4]	1CP[3]	1CP[2]	1CP[1]	1CP[0]	1RD1[1]	1RD1[0]	
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	1CP	63	R	SSFDC Area1 ECC CP
1:0	1RD1	3	R	Always, one is read

20.5.22 ECC1LP - EMIF ECC Area 1 LP Register

ECC1LP		0003:0A2A				offset: 0x2A										default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	1LP[15]	1LP[14]	1LP[13]	1LP[12]	1LP[11]	1LP[10]	1LP[9]	1LP[8]	1LP[7]	1LP[6]	1LP[5]	1LP[4]	1LP[3]	1LP[2]	1LP[1]	1LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	1LP	65535	R	SSFDC Area1 ECC LP The result of the ECC calculation of Storing Region-1 of SmartMedia can be read.

20.5.23 ECC2CP - EMIF ECC Area 2 CP Register

ECC2CP		0003:0A2C				offset: 0x2C										default: 0x00FF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	2CP[5]	2CP[4]	2CP[3]	2CP[2]	2CP[1]	2CP[0]	2RD1[1]	2RD1[0]	
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	2CP	63	R	SSFDC Area2 ECC CP The result of the ECC calculation of Storing Region-2 of SmartMedia can be read.
1:0	2RD1	3	R	Fixed at "11"

20.5.24 ECC2LP - EMIF ECC Area 2 LP Register

ECC2LP		0003:0A2E				offset: 0x2E				default: 0xFFFF						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	2LP[15]	2LP[14]	2LP[13]	2LP[12]	2LP[11]	2LP[10]	2LP[9]	2LP[8]	2LP[7]	2LP[6]	2LP[5]	2LP[4]	2LP[3]	2LP[2]	2LP[1]	2LP[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:0	2LP	65535	R	SSFDC Area2 ECC LP The result of the ECC calculation of Storing Region-2 of SmartMedia can be read.

20.5.25 ECCCLR - EMIF ECC Clear Register

ECCCLR		0003:0A30				offset: 0x30								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ECLR	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	ECLR	0	R/W	ECC is cleared when 1 is written to this bit This bit is automatically cleared after writing

20.5.26 PAGESZ - EMIF SmartMedia Page Size Register

PAGESZ		0003:0A32				offset: 0x32								default: 0x0001			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PAGESZ	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	PAGESZ	1	R/W	SmartMedia Page Size » 0: 256*8bytes per page « » 1: 512*16bytes per page «

20.5.27 IMGDSPDEST - EMIF Image Buffer/DSP(HPIB) Device Select Register

IMGDSPDEST		0003:0A34				offset: 0x34				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	IDST[2]	IDST[1]	IDST[0]	RSV	DDST[2]	DDST[1]	DDST[0]	
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	0	0	0	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:4	IDST	0	R/W	Image Buffer Data Transfer Destination » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 «
3	RSV			Reserved
2:0	DDST	0	R/W	DSP (HPIB) Data Transfer Destination » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 «

20.5.28 PRIOTCTL - EMIF Priority Control Register

PRIOTCTL		0003:0A36				offset: 0x36				default: 0x001B							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PRY1[1]	PRY1[0]	PRY2[1]	PRY2[0]	PRY3[1]	PRY3[0]	PRY4[1]	PRY4[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	0	0	0	1	1	0	1	1	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:6	PRY1	0		Specifies 1st (highest) priority. » 00: ARM « » 01: DMA « » 10: Image Buffer « » 11: DSP (HPIB) «
5:4	PRY2	1		Specifies 2nd priority. » 00: ARM « » 01: DMA « » 10: Image Buffer « » 11: DSP (HPIB) «
3:2	PRY3	2		Specifies 3rd priority. » 00: ARM « » 01: DMA « » 10: Image Buffer « » 11: DSP (HPIB) «
1:0	PRY4	3		Specifies 4th priority. » 00: ARM « » 01: DMA « » 10: Image Buffer « » 11: DSP (HPIB) «

20.5.29 SOURCEADDH - EMIF DMA Source Address High Register

SOURCEADDH		0003:0A38				offset:		0x38				default:		0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	SAFIX	RSV	RSV	SAH[9]	SAH[8]	SAH[7]	SAH[6]	SAH[5]	SAH[4]	SAH[3]	SAH[2]	SAH[1]	SAH[0]	
R/W	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	0	-	-	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	SAFI X	0	R/W	Fix DMA Source Address » 0: Auto-increment address after each transfer « » 1: The address is fixed «
11:10	RSV			Reserved
9:0	SAH	0	R/W	DMA transfer source byte address - upper 10-bits Note: address is specified as offset from start of SDRAM/EMIF region

20.5.30 SOURCEADDL - EMIF DMA Source Address Low Register

SOURCEADDL		0003:0A3A				offset:		0x3A				default:		0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SAL[15]	SAL[14]	SAL[13]	SAL[12]	SAL[11]	SAL[10]	SAL[9]	SAL[8]	SAL[7]	SAL[6]	SAL[5]	SAL[4]	SAL[3]	SAL[2]	SAL[1]	SAL[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SAL	0	R/W	DMA transfer source byte address - lower 16-bits Address specified must be a multiple of 4 to align with a 32-bit address. The 2 LSBs are ignored even if non-zero values are written. Note : address is specified as offset from start of SDRAM/EMIF region

20.5.31 DESTADDH - EMIF DMA Destination Address High Register

DESTADDH		0003:0A3C				offset:		0x3C				default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	DAFIX	RSV	RSV	DAH[9]	DAH[8]	DAH[7]	DAH[6]	DAH[5]	DAH[4]	DAH[3]	DAH[2]	DAH[1]	DAH[0]			
R/W	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	0	-	-	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	DAFI X	0	R/W	Fix DMA Destination Address » 0: Auto-increment address after each transfer « » 1: The address is fixed «
11:10	RSV			Reserved
9:0	DAH	0	R/W	DMA transfer destination byte address - upper 10-bits Note : address is specified as offset from start of SDRAM/EMIF region

20.5.32 DESTADDL - EMIF DMA Destination Address Low Register

DESTADDL		0003:0A3E				offset:		0x3E				default:				0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	DAL[15]	DAL[14]	DAL[13]	DAL[12]	DAL[11]	DAL[10]	DAL[9]	DAL[8]	DAL[7]	DAL[6]	DAL[5]	DAL[4]	DAL[3]	DAL[2]	DAL[1]	DAL[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	DAL	0	R/W	DMA transfer destination byte address - lower 16-bits Address specified must be a multiple of 4 to align with a 32-bit address. The 2 LSBs are ignored even if non-zero values are written. Note : address is specified as offset from start of SDRAM/EMIF region

20.5.33 DMA SIZE - EMIF DMA Transfer Size Register

DMA SIZE		0003:0A40										offset: 0x40		default: 0x0000					
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name		TXB[15]	TXB[14]	TXB[13]	TXB[12]	TXB[11]	TXB[10]	TXB[9]	TXB[8]	TXB[7]	TXB[6]	TXB[5]	TXB[4]	TXB[3]	TXB[2]	TXB[1]	TXB[0]		
R/W		R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	TXB	0	R/W	DMA transfer size in bytes. Transfer size must always be specified in a multiple of 4. The 2 LSBs are ignored even non-zero values are written.

20.5.34 DMADEVSEL - EMIF DMA Device Select Register

DMADEVSEL		0003:0A42				offset: 0x42								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SDV[2]	SDV[1]	SDV[0]	RSV	DDV[2]	DDV[1]	DDV[0]	
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	0	0	0	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:4	SDV	0	R/W	DMA Transfer Source Device. » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 « » 101: SDRAM « » 110: AIM «
3	RSV			Reserved
2:0	DDV	0	R/W	DMA Transfer Destination Device. » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 « » 101: SDRAM « » 110: AIM «

20.5.35 DMACTL - EMIF DMA Control Register

DMACTL		0003:0A44								offset: 0x44		default: 0x000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	ENDI[1]	ENDI[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GO			
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	-	-	-	-	R/W			
Default	-	-	-	-	-	-	0	0	-	-	-	-	-	-	-	0			

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:8	ENDI	0	R/W	Endian Switching Control » 00: D3 D2 D1 D0 -> D3 D2 D1 D0 « » 01: D3 D2 D1 D0 -> D2 D0 D1 « » 10: D3 D2 D1 D0 -> D0 D1 D2 D3 « » 11: D3 D2 D1 D0 -> D1 D0 D3 D2 «
7:1	RSV			Reserved
0	GO	0	R/W	DMA Transfer Start Bit When "1" is set, DMA transfer is started. Automatically cleared to "0" after transfer is finished

20.5.36 IMGDSPADDH - EMIF Image Buffer/DSP(HPIB) Destination Address High

IMGDSPADDH		0003:0A46				offset: 0x46				default: 0x0000							
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CDAH[6]	CDAH[5]	CDAH[4]	CDAH[3]	CDAH[2]	CDAH[1]	CDAH[0]
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:0	CDAH	0	R/W	Upper 7 bits address of byte address of external memory when data of image buffer or DSP (HPIB) and external memory CS1 or CS2 is described. Specify the relative address from the header address of each bank. During data transfer between Image buffer or DSP (HPIB) and CS1 or CS2, the CS1 or CS2 address specified by IMGDSPADD1/2 acts as a fixed address, i.e., address is not incremented after transfer of each unit of data.

20.5.37 IMGDSPADDL - EMIF Image Buffer/DSP(HPIB) Destination Address Low

IMGDSPADDL	0003:0A48				offset: 0x48				default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	CDAL[15]	CDAL[14]	CDAL[13]	CDAL[12]	CDAL[11]	CDAL[10]	CDAL[9]	CDAL[8]	CDAL[7]	CDAL[6]	CDAL[5]	CDAL[4]	CDAL[3]	CDAL[2]	CDAL[1]	CDAL[0]				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:0	CDAL	0	R/W	Lower 16 bits address of byte address of external memory when data of image buffer or DSP (HPIB) and external memory CS1 or CS2 is described. Specify the relative address from the header address of each bank. When it is transferred, it becomes a fixed address.

20.5.38 DPSTR0 - EMIF Region 0 End Address

DPSTR0	0003:0A4A				offset: 0x4A				default: 0x0009											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST0[7]	DST0[6]	DST0[5]	DST0[4]	DST0[3]	DST0[2]	DST0[1]	DST0[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	0	0	0	1	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST0	9	R/W	CS0 area maximum address CS0 area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.39 DPSTR1 - EMIF SDRAM Region End Address

DPSTR1	0003:0A4C				offset: 0x4C				default: 0x0021											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST1[7]	DST1[6]	DST1[5]	DST1[4]	DST1[3]	DST1[2]	DST1[1]	DST1[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	0	1	0	0	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST1	33	R/W	SDRAM area maximum address SDRAM area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.40 DPSTR2 - EMIF Region 1 End Address

DPSTR2	0003:0A4E				offset: 0x4E				default: 0x0039											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST2[7]	DST2[6]	DST2[5]	DST2[4]	DST2[3]	DST2[2]	DST2[1]	DST2[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	0	1	1	1	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST2	57	R/W	CS1 area maximum address CS1 area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.41 DPSTR3 - EMIF Region 2 End Address

DPSTR3	0003:0A50				offset: 0x50				default: 0x0051											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST3[7]	DST3[6]	DST3[5]	DST3[4]	DST3[3]	DST3[2]	DST3[1]	DST3[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	1	0	1	0	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST3	81	R/W	CS2 area maximum address CS2 area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.42 DPSTR4 - EMIF Region 3 End Address

DPSTR4	0003:0A52				offset: 0x52				default: 0x0069											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST4[7]	DST4[6]	DST4[5]	DST4[4]	DST4[3]	DST4[2]	DST4[1]	DST4[0]				
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	-	-	0	1	1	0	1	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST4	105	R/W	CS3/SmartMedia area maximum address CS3/SmartMedia area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.43 DPSTR5 - EMIF Region 4 End Address

DPSTR5		0003:0A54				offset: 0x54										default: 0x0081	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DST5[7]	DST5[6]	DST5[5]	DST5[4]	DST5[3]	DST5[2]	DST5[1]	DST5[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	1	0	0	0	0	0	0	1	

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	DST5	129	R/W	CS4 area maximum address CS4 area is set by specifying absolute address in external memory area, units are 1M byte. Minimum value: 0 Maximum value: 255

20.5.44 TEST - EMIF Test Register - do not use

TEST		0003:0A56				offset: 0x56										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TEST	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	TEST	0	R/W	TEST mode » 0: Normal mode « » 1: Test mode « Internal ARM address is output to address bus when external memory access is IDLE.

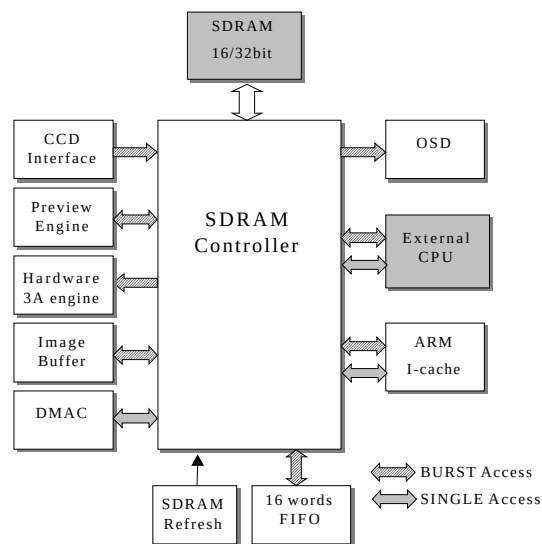
20.6 SDRAM controller

20.6.1 Features of the SDRAM controller

The SDRAM controller, shown in Figure 155, acts as the primary interface between SDRAM and all functional blocks such as the processors (ARM, DSP), CCDC, Preview Engine, OSD and DMA controller. This block supports SDRAM timing of a maximum of 102 MHz (refer to the DM270 data sheet for exact value), and provides continuous data access with low overhead. The start address and size of the SDRAM memory region are configurable within the external memory region of the ARM memory map. The SDRAM controller supports a DMA controller interface. The SDRAM access priority can be modified. The SDRAM controller supports refresh commands and can place the device in a self-refresh mode prior to going a power down mode. The refresh interval is programmable. The latency for the column access strobe (CAS) is programmable from 2 to 3.

- Supports 16/64/128/256/512M SDRAM
- Supports both 32-bit-wide SDRAM or 16-bit-wide SDRAM
- Maximum 102 MHz operation (see also DM270 datasheet)
- Access in WORD, HALF WORD, and BYTE units
- Mode setting, power down and self-refresh commands
- Programmable auto-refresh interval
- CAS latency 2 or 3 cycles can be selected
- Maximum SDRAM size 64 Mbytes
- DMA transfer with peripheral modules
- Variable SDRAM access priority
- SDRAM Controller Access

Figure 155. SDRAM Controller Block



20.6.2 Start Address and Size of the SDRAM Region

The SDRAM region can be set within the external memory region, similar to the other external memory regions 0-4. The SDRAM region can be set in EMIF by specifying the boundary addresses of EMIF region-0 and boundary address of SDRAM region. In fact, DPSTR0 - EMIF Region 0 End Address register can be used to mark the boundary between end of EMIF region-0 and beginning of SDRAM region. The end of SDRAM region can be set in DPSTR1 - EMIF SDRAM Region End Address register.

The boundary address of the SDRAM region can be specified in 1-Mbyte units, similar to external memory regions 0-4. The boundary address of SDRAM region is obtained by adding SDRAM size to the boundary address value of EMIF region-0. The reset value of SDRAM size is 24 Mbytes. Refer to the section titled "Boundary Addresses and Sizes of the External Memory Regions" in this chapter for more information on SDRAM start address and size configuration.

The SDRAM controller can access 512-Mbits (64-Mbytes) of SDRAM and supports both 16 and 32-bit SDRAM devices. Each access can be done in word (32-bit), half-word (16-bit) or byte units from the ARM side.

20.6.3 Memory Structure Setting

The DM270 can connect to SDRAM of a maximum of 512Mbits (64-Mbytes). The address that is connected differs depending on the memory structure of the SDRAM, as follows.

Correspondence between memory structure and address

Memory structure	ADDRESS	Number of banks	BANK
2k x 256 WORD	SDR_A10-SDR_A0	2	SDR_A13
		4	SDR_A14-SDR_A13
4k x 256 WORD	SDR_A11-SDR_A0	2	SDR_A13
		4	SDR_A14-SDR_A13
4k x 512 WORD	SDR_A11-SDR_A0	2	SDR_A13
		4	SDR_A14-SDR_A13
8k x 512 WORD	SDR_A12-SDR_A0	2	SDR_A13
		4	SDR_A14-SDR_A13

20.6.4 SDRAM configuration registers

Before using the SDRAM, the SDRAM controller clock must be turned on by setting the bit CSDRC of the MOD0 (0x30890) register in the clock controller module. When the module is on, you can configure it to match the SDRAM memory specifications. The SDMODE - SDRAM Mode Register lets you change the number of cycles to access the memory as well as the latency for the CAS. The memory can be configured to use 2 or 4 banks. The SDMODE - SDRAM Mode Register is used as well to write the commands that need to be sent to the SDRAM chip.

In addition to this register, REFCTL - Refresh Control Register allows you to specify the auto-refresh frequency used by the controller.

The bit SDBW of SDMODE - SDRAM Mode Register selects the bus width for SDRAM access.

20.6.5 SDRAM command

The ARM can issues the following commands to SDRAM by settings the registers in the SDRAM controller (SDCTL in SDMODE - SDRAM Mode Register):

- 5) MRS (mode register set)
- 6) PREA (precharge all)
- 7) REF (auto refresh)
- 8) SELF (self refresh entry)
- 9) SELFC (self refresh end)
- 10) PDN (power down mode)
- 11) DPDN (deep power down mode)
- 12) EMSR (Set extended mode register)

When an MRS command is issued, the following data settings are performed in the SDRAM mode register.

MRS command and SDMODE register

SDRAM Address pin	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
Meaning	Reserved				0	0	Read Latency			S/I	Burst length		
DM270 output value	0	0	0	0	0	0	0	CL	0	0	0	BST 1	BST 0

20.6.6 SDRAM priority access

The access priority to the SDRAM can be modified. The registers SDPRTY1 - SDRAM Priority Select #1 (CCD Controller) to SDPRTY10 - SDRAM Priority Select #10 (Refresh) control it. As shown in 0, by default the CCDC will have the highest priority and the Image Buffer (DSP) DMA request the lowest.

The binary value "1" cannot be written twice or more into any one priority register. If you do, the bit 9 has highest priority, and bit 0 has lowest priority. As well, if "1" is written in the same bit position in different registers, the request of the low register address takes priority, and the request of the high register address is ignored.

When the priority configuration is ready, you need to write a '1' into the bit USEPRTY of the PRTYON - SDRAM Priority On Register to enable it. Since SDRAM controller internal priority is changed automatically when access to SDRAM becomes IDLE (e.g. at the end of the current burst transfer), you do not need to pay attention to the change period.

If a request with a higher priority than the current transfer one occurs, the burst transfer in progress is completed prior to serving the received request. The size of a burst is 4 32-bit word in a 32 bits mode and 8 16-bit half-word in a 16 bits mode (8 16-bit half-words worth of data in both cases)

Default SDRAM Priority Access

Register name	Default Register setting	SDRAM controller input	Priority
SDPRTY1	0x200	CCDC DMA request	High
SDPRTY2	0x100	Preview Engine DMA request	
SDPRTY3	0x080	Hardware 3A request	
SDPRTY4	0x040	OSD DMA request	
SDPRTY5	0x020	External Host CPU request	
SDPRTY6	0x010	ARM CPU request	
SDPRTY7	0x008	External Memory I/F, Serial I/F, USB, and MMC/SD DMA request (channel 1)	Low
SDPRTY8	0x004	External Memory I/F, Serial I/F, USB, and MMC/SD DMA request (channel 2)	
SDPRTY10	0x002	SDRAM Auto Refresh request	
SDPRTY9	0x001	Image Buffer (DSP) DMA request	

20.6.7 SDRAM Initialization

SDRAM has a power on procedure. Since the procedure differs depending on the SDRAM that is used, SDRAM must be initialized by software using the SDRAM commands. Also, since the DQM (input/output mask) signal is high output as default, after the mode register set command (MRS) is finished, DQM must be set to low level. These SDRAM settings are performed by setting registers in the SDRAM controller.

A typical sequence is given below:

- 1) Apply power and start the clock. Attempt to maintain the NOP command at the input.
- 2) Maintain stable power, stable clock, and the NOP condition for a minimum of 100us or 200us. (Minimum time depends on devices. Please refer to the datasheet.)
- 3) Precharge all banks by the Precharge All (PREA) command.
- 4) Assert a minimum of 2 or 8 Auto-refresh commands. (Assert time depends on the devices. Please refer to the datasheet)
- 5) Program the mode register by the Mode Register Set command (MRS).
- 6) If required enable auto-refresh and set the appropriate refresh interval in register REFCTL - Refresh Control Register

20.6.8 ARM buffered access

In TMS320DM270, there are two types of access to SDRAM: burst access via a buffer, and ordinary memory access. SDRAM burst access is described in this section. Transfer to and from SDRAM is performed in 16 16-bit word units. There is a control register (SDBUFCTL - SDRAM Transfer Control) in the SDRAM controller for burst access as well as sixteen 16-bit data registers (SDBUFD0L - SDRAM Transfer Data Buffer 0L to SDBUFD7H - SDRAM Transfer Data Buffer 7) to store the data from 16-word burst transfer.

To write into the SDRAM, data needs to be set in these data registers. After the write, destination SDRAM address is set in the buffer address registers (SDBUFAD1 - SDRAM Transfer Address #1 - High, SDBUFAD2 - SDRAM Transfer Address #1 - Low,), a write command can be written in the SDBUFCTL - SDRAM Transfer Control register. When the proper bit (WM or WA) in this register is written, writing to the SDRAM is performed.

To read from SDRAM, the read origin from SDRAM address must be set first. This information is stored in the buffer address registers. When a read command is issued by writing the bit RSD of the control register (SDBUFCTL - SDRAM Transfer Control), the data is read from SDRAM and stored in the data registers.

You need to check the status of the WM, WA and RSD bit to know if a transfer is completed or not.

Auto-increment of SDRAM address is also available via register field AAI in register SDBUFCTL - SDRAM Transfer Control

IMPORTANT: For SDRAM buffered access, a SDRAM access port must be select using PTSEL field in SDBUFCTL - SDRAM Transfer Control. SDRAM access port can be Preview Engine port or 3A Engine port or External HOST I/F port. When a given peripheral port is selected SDRAM access of the peripheral (Preview Engine, or H3A or EHIF) is prohibited.

20.7 SDRAM Controller Register Map (SDRC)

Address	Register	Description
0003:0980	SDBUFD0L	SDRAM Transfer Data Buffer 0H
0003:0982	SDBUFD0H	SDRAM Transfer Data Buffer 0L
0003:0984	SDBUFD1L	SDRAM Transfer Data Buffer 1H
0003:0986	SDBUFD1H	SDRAM Transfer Data Buffer 1L
0003:0988	SDBUFD2L	SDRAM Transfer Data Buffer 2H
0003:098A	SDBUFD2H	SDRAM Transfer Data Buffer 2L
0003:098C	SDBUFD3L	SDRAM Transfer Data Buffer 3H
0003:098E	SDBUFD3H	SDRAM Transfer Data Buffer 3L
0003:0990	SDBUFD4L	SDRAM Transfer Data Buffer 4H
0003:0992	SDBUFD4H	SDRAM Transfer Data Buffer 4L
0003:0994	SDBUFD5L	SDRAM Transfer Data Buffer 5H
0003:0996	SDBUFD5H	SDRAM Transfer Data Buffer 5L
0003:0998	SDBUFD6L	SDRAM Transfer Data Buffer 6H
0003:099A	SDBUFD6H	SDRAM Transfer Data Buffer 6L
0003:099C	SDBUFD7L	SDRAM Transfer Data Buffer 7H
0003:099E	SDBUFD7H	SDRAM Transfer Data Buffer 7L
0003:09A0	SDBUFAD1	SDRAM Transfer Address #1 - Low
0003:09A2	SDBUFAD2	SDRAM Transfer Address #1 - High
0003:09A4	SDBUFCTL	SDRAM Transfer Control
0003:09A6	SDMODE	SDRAM Mode Register
0003:09A8	REFCTL	Refresh Control Register
0003:09AA	SDPRTY1	SDRAM Priority Select #1 (CCD Controller)
0003:09AC	SDPRTY2	SDRAM Priority Select #2 (Preview Engine)
0003:09AE	SDPRTY3	SDRAM Priority Select #3 (Hardware 3A)
0003:09B0	SDPRTY4	SDRAM Priority Select #4 (On Screen Display)
0003:09B2	SDPRTY5	SDRAM Priority Select #5 (External Host)
0003:09B4	SDPRTY6	SDRAM Priority Select #6 (ARM CPU)
0003:09B6	SDPRTY7	SDRAM Priority Select #7 (EMIF #1)
0003:09B8	SDPRTY8	SDRAM Priority Select #8 (EMIF #2)
0003:09BA	SDPRTY9	SDRAM Priority Select #9 (Image Buffer - DSP)
0003:09BC	SDPRTY10	SDRAM Priority Select #10 (Refresh)
0003:09BE	PRTYON	SDRAM Priority On Register
0003:09C0	SDRCTEST	SDRAM Test

20.8 SDRAM Controller Registers

20.8.1 SDBUFD0L - SDRAM Transfer Data Buffer 0L

SDBUFD0L		0003:0980				offset: 0x00										default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	SD0L[15]	SD0L[14]	SD0L[13]	SD0L[12]	SD0L[11]	SD0L[10]	SD0L[9]	SD0L[8]	SD0L[7]	SD0L[6]	SD0L[5]	SD0L[4]	SD0L[3]	SD0L[2]	SD0L[1]	SD0L[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	SD0L	0	R/W	SDRAM data register 0L

20.8.2 SDBUFD0H - SDRAM Transfer Data Buffer 0H

SDBUFD0H		0003:0982				offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD0H[15]	SD0H[14]	SD0H[13]	SD0H[12]	SD0H[11]	SD0H[10]	SD0H[9]	SD0H[8]	SD0H[7]	SD0H[6]	SD0H[5]	SD0H[4]	SD0H[3]	SD0H[2]	SD0H[1]	SD0H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	SD0H	0	R/W	SDRAM data register 0H

20.8.3 SDBUFD1L - SDRAM Transfer Data Buffer 1L

SDBUFD1L	0003:0984								offset: 0x04										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	SD1L[15]	SD1L[14]	SD1L[13]	SD1L[12]	SD1L[11]	SD1L[10]	SD1L[9]	SD1L[8]	SD1L[7]	SD1L[6]	SD1L[5]	SD1L[4]	SD1L[3]	SD1L[2]	SD1L[1]	SD1L[0]				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Bit	Name	Reset Value	R/W	Function
15:0	SD1L	0	R/W	SDRAM data register 1L

20.8.4 SDBUFD1H - SDRAM Transfer Data Buffer 1H

SDBUFD1H	0003:0986				offset:	0x06												default:	0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
Name	SD1H[15]	SD1H[14]	SD1H[13]	SD1H[12]	SD1H[11]	SD1H[10]	SD1H[9]	SD1H[8]	SD1H[7]	SD1H[6]	SD1H[5]	SD1H[4]	SD1H[3]	SD1H[2]	SD1H[1]	SD1H[0]						
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W						
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						

Bit	Name	Reset Value	R/W	Function
15:0	SD1H	0	R/W	SDRAM data register 1H

20.8.5 SDBUFD2L - SDRAM Transfer Data Buffer 2L

SDBUFD2L		0003:0988		offset: 0x08		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD2L[15]	SD2L[14]	SD2L[13]	SD2L[12]	SD2L[11]	SD2L[10]	SD2L[9]	SD2L[8]	SD2L[7]	SD2L[6]	SD2L[5]	SD2L[4]	SD2L[3]	SD2L[2]	SD2L[1]	SD2L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD2L	0	R/W	SDRAM data register 2L

20.8.6 SDBUFD2H - SDRAM Transfer Data Buffer 2H

SDBUFD2H		0003:098A		offset: 0x0A		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD2H[15]	SD2H[14]	SD2H[13]	SD2H[12]	SD2H[11]	SD2H[10]	SD2H[9]	SD2H[8]	SD2H[7]	SD2H[6]	SD2H[5]	SD2H[4]	SD2H[3]	SD2H[2]	SD2H[1]	SD2H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD2H	0	R/W	SDRAM data register 2H

20.8.7 SDBUFD3L - SDRAM Transfer Data Buffer 3L

SDBUFD3L		0003:098C		offset: 0x0C		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD3L[15]	SD3L[14]	SD3L[13]	SD3L[12]	SD3L[11]	SD3L[10]	SD3L[9]	SD3L[8]	SD3L[7]	SD3L[6]	SD3L[5]	SD3L[4]	SD3L[3]	SD3L[2]	SD3L[1]	SD3L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD3L	0	R/W	SDRAM data register 3L

20.8.8 SDBUFD3H - SDRAM Transfer Data Buffer 3H

SDBUFD3H		0003:098E		offset: 0x0E		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD3H[15]	SD3H[14]	SD3H[13]	SD3H[12]	SD3H[11]	SD3H[10]	SD3H[9]	SD3H[8]	SD3H[7]	SD3H[6]	SD3H[5]	SD3H[4]	SD3H[3]	SD3H[2]	SD3H[1]	SD3H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD3H	0	R/W	SDRAM data register 3H

20.8.9 SDBUFD4L - SDRAM Transfer Data Buffer 4L

SDBUFD4L		0003:0990		offset: 0x10		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD4L[15]	SD4L[14]	SD4L[13]	SD4L[12]	SD4L[11]	SD4L[10]	SD4L[9]	SD4L[8]	SD4L[7]	SD4L[6]	SD4L[5]	SD4L[4]	SD4L[3]	SD4L[2]	SD4L[1]	SD4L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD4L	0	R/W	SDRAM data register 4L

20.8.10 SDBUFD4H - SDRAM Transfer Data Buffer 4H

SDBUFD4H		0003:0992		offset: 0x12		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD4H[15]	SD4H[14]	SD4H[13]	SD4H[12]	SD4H[11]	SD4H[10]	SD4H[9]	SD4H[8]	SD4H[7]	SD4H[6]	SD4H[5]	SD4H[4]	SD4H[3]	SD4H[2]	SD4H[1]	SD4H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD4H	0	R/W	SDRAM data register 4H

20.8.11 SDBUFD5L - SDRAM Transfer Data Buffer 5L

SDBUFD5L		0003:0994		offset: 0x14		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD5L[15]	SD5L[14]	SD5L[13]	SD5L[12]	SD5L[11]	SD5L[10]	SD5L[9]	SD5L[8]	SD5L[7]	SD5L[6]	SD5L[5]	SD5L[4]	SD5L[3]	SD5L[2]	SD5L[1]	SD5L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD5L	0	R/W	SDRAM data register 5L

20.8.12 SDBUFD5H - SDRAM Transfer Data Buffer 5H

SDBUFD5H		0003:0996		offset: 0x16		default: 0x0000													
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD5H[15]	SD5H[14]	SD5H[13]	SD5H[12]	SD5H[11]	SD5H[10]	SD5H[9]	SD5H[8]	SD5H[7]	SD5H[6]	SD5H[5]	SD5H[4]	SD5H[3]	SD5H[2]	SD5H[1]	SD5H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15: 0	SD5H	0	R/W	SDRAM data register 5H

20.8.13 SDBUFD6L - SDRAM Transfer Data Buffer 6L

SDBUFD6L		0003:0998				offset: 0x18										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD6L[15]	SD6L[14]	SD6L[13]	SD6L[12]	SD6L[11]	SD6L[10]	SD6L[9]	SD6L[8]	SD6L[7]	SD6L[6]	SD6L[5]	SD6L[4]	SD6L[3]	SD6L[2]	SD6L[1]	SD6L[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD6L	0	R/W	SDRAM data register 6L

20.8.14 SDBUFD6H - SDRAM Transfer Data Buffer 6H

SDBUFD6H		0003:099A				offset: 0x1A										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD6H[15]	SD6H[14]	SD6H[13]	SD6H[12]	SD6H[11]	SD6H[10]	SD6H[9]	SD6H[8]	SD6H[7]	SD6H[6]	SD6H[5]	SD6H[4]	SD6H[3]	SD6H[2]	SD6H[1]	SD6H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD6H	0	R/W	SDRAM data register 6H

20.8.15 SDBUFD7L - SDRAM Transfer Data Buffer 7L

SDBUFD7L		0003:099C				offset:		0x1C								default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD7L[15]	SD7L[14]	SD7L[13]	SD7L[12]	SD7L[11]	SD7L[10]	SD7L[9]	SD7L[8]	SD7L[7]	SD7L[6]	SD7L[5]	SD7L[4]	SD7L[3]	SD7L[2]	SD7L[1]	SD7L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	SD7L	0	R/W	SDRAM data register 7L

20.8.16 SDBUFD7H - SDRAM Transfer Data Buffer 7H

SDBUFD7H		0003:099E				offset: 0x1E										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD7H[15]	SD7H[14]	SD7H[13]	SD7H[12]	SD7H[11]	SD7H[10]	SD7H[9]	SD7H[8]	SD7H[7]	SD7H[6]	SD7H[5]	SD7H[4]	SD7H[3]	SD7H[2]	SD7H[1]	SD7H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD7H	0	R/W	SDRAM data register 7H

20.8.17 SDBUFAD1 - SDRAM Transfer Address #1 - High

SDBUFAD1		0003:09A0				offset: 0x20										default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AAI	RSV	RSV	SDAH[5]	SDAH[4]	SDAH[3]	SDAH[2]	SDAH[1]	SDAH[0]			
R/W	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	0	-	-	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	AAI	0	R/W	Automatic address increment » 0: no autoincrement « » 1: After burst transfer, SDA01H/L is automatically incremented to the next burst boundary «
7:6	RSV			Reserved
5:0	SDAH	0	R/W	SDRAM Address (High) Upper 6 bits of SDRAM address #1 Address is the offset from the SDRAM base and is specified in 32-byte (burst) units

20.8.18 SDBUFAD2 - SDRAM Transfer Address #1 - Low

SDBUFAD2		0003:09A2				offset: 0x22										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SDAL[15]	SDAL[14]	SDAL[13]	SDAL[12]	SDAL[11]	SDAL[10]	SDAL[9]	SDAL[8]	SDAL[7]	SDAL[6]	SDAL[5]	SDAL[4]	SDAL[3]	SDAL[2]	SDAL[1]	SDAL[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SDAL	0	R/W	SDRAM Address (Low) Lower 16 bits of SDRAM address #1 Address is the offset from the SDRAM base and is specified in 32-byte (burst) units

20.8.19 SDBUFCTL - SDRAM Transfer Control

SDBUFCTL	0003:09A4				offset: 0x24				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PTSEL[1]	PTSEL[0]	BUFC	WM	WA	RSD
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:6	RSV			Reserved
5:4	PTSEL	0	R/W	<u>Port Selection</u> » 00: FIFO Access Function off « » 01: Preview Engine Port « » 10: Hardware #A Engine Port « » 11: Ext. CPU port « When a Port is selected, SDRAM access of the given module is disabled and and FIFO access is available
3	BUFC	0	W	<u>Buffer Clear</u> When set to '1', all data in data registers SDBUF* is cleared.
2	WM	0	R/W	<u>Write Modified Data</u> When set to "1", only the data in SDBUF* registers where write was performed since the last read are written into SDRAM at address specified by SDAD1L/H. Write is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer to SDRAM is finished.
1	WA	0	R/W	<u>Write All Data</u> When set to "1", all data in SDBUF* data registers is written to SDRAM at address specified by SDAD1L/H. Write is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer to SDRAM is finished.
0	RSD	0	R/W	<u>Read All Data</u> When set to "1", data is read into SDBUF* data registers from SDRAM at address specified by SDAD1L/H. Read is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer from SDRAM is finished.

20.8.20 SDMODE - SDRAM Mode Register

SDMODE	0003:09A6				offset: 0x26				default: 0x0080							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TRDL	SDBW	TRCD	BNS	RSV	CASL	MEMT[1]	MEMT[0]	DQMC	APO	SDCTL[5]	SDCTL[4]	SDCTL[3]	SDCTL[2]	SDCTL[1]	SDCTL[0]
R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	-	0	0	0	1	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	TRDL	0	R/W	<u>TRDL select</u> » 0: 1 cycle « » 1: 2 cycles « 1 or 2 cycles can be selected in SDRAM clock units
14	SDBW	0	R/W	<u>SDRAM bus width select</u> » 0: 32 bits « » 1: 16 bits «
13	TRCD	0	R/W	
12	BNS	0	R/W	<u>Bank select</u> » 0: 2 banks « » 1: 4 banks «
11	RSV			Reserved
10	CASL	2	R/W	<u>CAS Latency</u> » 0: 2 cycles « » 1: 3 cycles « CAS latency can be selected in SDRAM clock units
9:8	MEMT	0	R/W	<u>Memory Type</u> » 00: 2k x 256 words « » 01: 4k x 256 words « » 10: 4k x 512 words « » 11: 8k x 512 words «
7	DQMC	1	R/W	<u>DQM control</u> » 0: Normal « » 1: Forcibly sets DQM signal to '1' «
6	APO	0	R/W	<u>Automatic Power Down Mode</u> » 0: Disabled «
5:0	SDCTL	0	R/W	<u>SDRAM control select</u> » 000000: NOP No operation « » 000001: MSR Sets mode « » 000010: PREA Precharges all banks « » 000100: REF Issues auto refresh command « » 001000: SELF Enters self refresh mode « » 010000: SELF Cancels self refresh/power down mode « » 100000: PDN Enters power down mode « Following additional commands were added in ES1.1 for mobile SDRAM support » 110000: Deep power down mode (DPDN) « » 010001: Set extended mode register (EMSR) «

See also note on next page...

...

When EMSR is set, the user can set following mode simultaneously.
 Drive Strength
 MEMT[1:0](bit[9:8])
 00: Normal
 01: 1/2 strength
 10: 1/4 strength
 11: Reserved
 Temperature compensated self Refresh
 CASL(bit10), TRDC(bit13)
 00: 70 [C]
 01: 45 [C]
 10: 15 [C]
 11: 85 [C]
 Partical Array Self Refresh
 SDBW(bit14)
 0: All Banks
 1: Bank A and B
 Example: For EMSR setting
 SDMODE = 0x4411
 Drive Strength : 00 : Normal
 Temp. comp. self Refresh : 10 : 15[C]
 Partical Array Self Refresh : 1 : Bank A and B
 Note: Please use EMSR setting before SDRAM access. After setting EMSR command, the user will have to re-write MEMT,CASL,TRCD, SDBW bit with previous data. And set 000000(b) for SDCTL[5:0] (NOP command).

20.8.21 REFCTL - Refresh Control Register

REFCTL		0003:09A8								offset: 0x28		default: 0x0400							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SDMA1[2]	SDMA1[1]	SDMA1[0]	SDMA2[2]	SDMA2[1]	SDMA2[0]	RSV	REFEN	REFC[7]	REFC[6]	REFC[5]	REFC[4]	REFC[3]	REFC[2]	REFC[1]	REFC[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	1	-	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:13	SDMA1	0	R/W	DMA Channel Selection 1 » 000: Between External Memory Interface and SDRAM « » 001: Between Serial Interface and SDRAM « » 010: Between USB and SDRAM « » 011: Between MMC/SD and SDRAM « » 100: Between DSP(HPIB) and SDRAM « Note: it is not possible to set the same value in bits 15-14 and bits 13-12.
12:10	SDMA2	1	R/W	DMA Channel Selection 2 » 000: Between External Memory Interface and SDRAM « » 001: Between Serial Interface and SDRAM « » 010: Between USB and SDRAM « » 011: Between MMC/SD and SDRAM « » 100: Between DSP(HPIB) and SDRAM « Note: it is not possible to set the same value in bits 15-14 and bits 13-12.
9	RSV	0	R/W	Reserved
8	REFEN	0	R/W	Auto Refresh Enable » 0: Disabled « » 1: Enabled «
7:0	REFC	0	R/W	Refresh Cycle Sets interval of auto refresh. Interval is (REFC + 1) x 8 SDRAM clocks.

20.8.24 SDPRTY3 - SDRAM Priority Select #3 (Hardware 3A)

SDPRY3	0003:09AE														offset:	0x2E				default:	0x0080			
Name	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Bit	RSV	RSV	RSV	RSV	RSV	RSV	H3A[9]	H3A[8]	H3A[7]	H3A[6]	H3A[5]	H3A[4]	H3A[3]	H3A[2]	H3A[1]	H3A[0]								
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W								
Default	-	-	-	-	-	-	0	0	1	0	0	0	0	0	0	0								

[illegible]

20.8.25 SDPRTY4 - SDRAM Priority Select #4 (On Screen Display)

SDPRTY4										0003:09B0				offset: 0x30								default: 0x0040			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name	RSV	RSV	RSV	RSV	RSV	RSV	OSD[9]	OSD[8]	OSD[7]	OSD[6]	OSD[5]	OSD[4]	OSD[3]	OSD[2]	OSD[1]	OSD[0]									
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W									
Default	-	-	-	-	-	-	0	0	0	1	0	0	0	0	0	0									

[illegible]

20.8.26 SDPRTY5 - SDRAM Priority Select #5 (External Host)

SDPRTY5				0003:09B2				offset:		0x32				default:				0x0020			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	EXTHOST[9]	EXTHOST[8]	EXTHOST[7]	EXTHOST[6]	EXTHOST[5]	EXTHOST[4]	EXTHOST[3]	EXTHOST[2]	EXTHOST[1]	EXTHOST[0]					
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W					
Default	-	-	-	-	-	-	0	0	0	0	1	0	0	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	EXTHOST	32	R/W	SDRAM Access priority for External Host - Priority is set by writing "1" in any one of bits [9:0] - Bit 9 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.8.27 SDPRTY6 - SDRAM Priority Select #6 (ARM CPU)

SDPRTY6				0003:09B4				offset:		0x34				default:				0x0010	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	ARMCPU[9]	ARMCPU[8]	ARMCPU[7]	ARMCPU[6]	ARMCPU[5]	ARMCPU[4]	ARMCPU[3]	ARMCPU[2]	ARMCPU[1]	ARMCPU[0]			
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	0	0	0	0	0	1	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	ARMCPU	16	R/W	SDRAM Access priority for ARM CPU - Priority is set by writing "1" in any one of bits [9:0] - Bit 9 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.8.28 SDPRTY7 - SDRAM Priority Select #7 (EMIF #1)

SDPRTY7				0003:09B6				offset:		0x36				default:				0x0008			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	EM2[9]	EM2[8]	EM2[7]	EM2[6]	EM2[5]	EM2[4]	EM2[3]	EM2[2]	EM2[1]	EM2[0]					
R/W	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW					
Default	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0					

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	EM2	8	R/W	SDRAM Access priority for External Memory #2 - Priority is set by writing "1" in any one of bits [9:0] - Bit 9 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.8.29 SDPRTY8 - SDRAM Priority Select #8 (EMIF #2)

SDPRTY8				0003:09B8				offset:		0x38				default:				0x0004	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	EM1[9]	EM1[8]	EM1[7]	EM1[6]	EM1[5]	EM1[4]	EM1[3]	EM1[2]	EM1[1]	EM1[0]			
R/W	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW			
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	1	0	0			

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	EM1	4	R/W	SDRAM Access priority for External Memory #1 - Priority is set by writing "1" in any one of bits [9:0] - Bit 9 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.8.30 SDPRTY9 - SDRAM Priority Select #9 (Image Buffer - DSP)

SDPRTY9		0003:09BA				offset:		0x3A									default:		0x0001	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	DSPIB[9]	DSPIB[8]	DSPIB[7]	DSPIB[6]	DSPIB[5]	DSPIB[4]	DSPIB[3]	DSPIB[2]	DSPIB[1]	DSPIB[0]				
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	1				

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved

SDRAM Access priority for Coprocessor Image Buffer

- Priority is set by writing "1" in any one of bits [9:0]
- Bit 9 is highest priority, bit 0 lowest priority.
- Only one bit should be set to "1"
- If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

Note:

In ES1.0, default value is 2
In ES1.1, default value is 1

20.8.31 SDPRTY10 - SDRAM Priority Select #10 (Refresh)

SDPRTY10				0003:09BC				offset:		0x3C				default:				0x0002	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RFRSH[9]	RFRSH[8]	RFRSH[7]	RFRSH[6]	RFRSH[5]	RFRSH[4]	RFRSH[3]	RFRSH[2]	RFRSH[1]	RFRSH[0]			
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	0	1	0			

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved

SDRAM Access priority for SDRAM Refresh

- Priority is set by writing "1" in any one of bits [9:0]
- Bit 9 is highest priority, bit 0 lowest priority.
- Only one bit should be set to "1"
- If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

Note:

In ES1.0, default value is 1
In ES1.1, default value is 2

20.8.32 PRTYON - SDRAM Priority On Register

[illegible]

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
				<u>Use Programmed Priority</u>
				» 0: Use default fixed priority settings «
				Fixed at the default values in SDPRTY1 - SDPRTY9
				1: CCD Controller
				2: Preview Engine
				3: Hardware 3A
				4: OSD
				5: External Host CPU
				6: ARM CPU random access
0	USEPRTY	0	W	7: External Memory I/F, Serial I/F, MMC
				8: External Memory I/F, Serial I/F, MMC
				9: DSP Imager Buffer DMA
				10: SDRC Auto Refresh
				» 1: Use priority defined by SDPRTY1-SDPRTY10 «
				The SDRAM controller priority is changed automatically when access to SDRAM goes IDLE, the user does not need to pay attention to the change period.

20.8.33 SDRCTEST - SDRAM Test

[illegible][illegible]

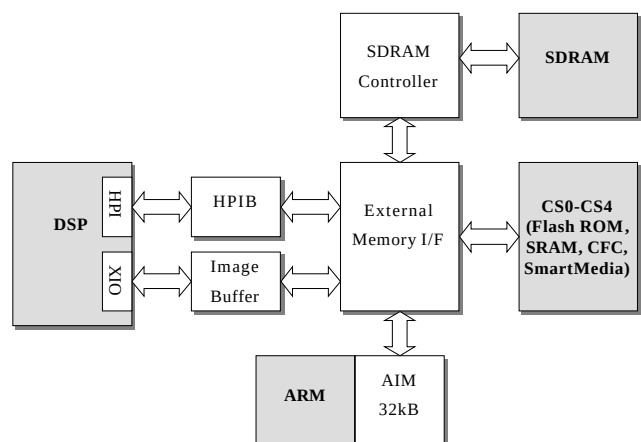
20.9 EMIF DMA Controller

20.9.1 Data Transfer Between EMIF regions

The External Memory Interface has a built-in DMA controller. As shown by the solid arrows in Figure 156, the EMIF's DMA controller can transfer data between SDRAM and external memory regions 0-4.

This diagram also shows that data can be transferred between external memory and the DSP's image buffer or HPIB. However, data is transferred between the image buffer and SDRAM/external memory by the Image Buffer's built-in DMA controller not the DMA controller built into the EMIF. Similarly data is transferred between the DSP internal memory and SDRAM/external memory (via the HPIB) by the DMA controller built into the HPIB interface in the DSP subsystem. See DSP subsystem, DSP co-processor subsystem for details of these DMA's

Figure 156. EMIF DMA



20.9.2 EMIF DMA Sources and destinations devices

To setup a DMA transfer using the EMIF's DMA controller you must configure the destination and source device, the destination and source addresses, and the transfer size. All possible sources and destinations are listed in Figure 157. Use DMADEVSEL - EMIF DMA Device Select Register to select any of these options. The addresses and transfer size registers are described in the following section.

IMPORTANT: The SDRAM controller has two DMA channels, which are used to perform DMAs involving SDRAM/external memory as source or destination. A DMA channel for EMIF data transfer must be selected prior to triggering EMIF DMA. The DMA channel is selected using the REFCTL - Refresh Control Register (SDMA1=0 or SDMA2=0).

Also the DMA channel SDRAM access priority can be changed using the SDPRTY7 - SDRAM Priority Select #7 (EMIF #1), SDPRTY8 - SDRAM Priority Select #8 (EMIF #2) (see section "SDRAM priority access" for details)

Figure 157. DMA transfer source, transfer destination devices

Transfer source device	Transfer destination device
External general-purpose memory CS0	External general-purpose memory CS0
External general-purpose memory CS1	External general-purpose memory CS1
External general-purpose memory CS2	External general-purpose memory CS2
External general-purpose memory CS3	External general-purpose memory CS3
External general-purpose memory CS4	External general-purpose memory CS4
SDRAM	SDRAM
ARM internal memory	ARM internal memory

20.9.3 EMIF DMA Sources/Destinations Addresses and Transfer Size

Configure the DMA source address in SOURCEADDH - EMIF DMA Source Address High Register and SOURCEADDL - EMIF DMA Source Address Low Register. Similarly configure the DMA destination address in DESTADDH - EMIF DMA Destination Address High Register and DESTADDL - EMIF DMA Destination Address Low Register. The transfer size is set in DMASIZE - EMIF DMA Transfer Size Register. The addresses are relative to the start of the EMIF region used, and the addresses must be a multiple of four. Since the addresses must be a multiple of 4, the lower 2 bits of the source and destination addresses are ignored.

The Transfer size must always be specified in a multiple of 4(DMASIZE - EMIF DMA Transfer Size Register). Therefore, the bit0, bit1 of DMASIZE are ignored.

20.9.4 EMIF DMA transfer control

When the previously mentioned registers are set-up, you can start the DMA transfer by setting the bit GO of the DMACTL - EMIF DMA Control Register (0x44). This bit is automatically cleared when the DMA transfer is finished. Writing to this bit while a transfer is in progress has no effect, and such a request is ignored.

The EMIF DMA controller can change the endianness property of the data transferred. This is controlled by the ENDI[1:0] bits of the DMACTL - EMIF DMA Control Register register.

20.9.5 EMIF DMA Interrupt

MTC interrupt #0, INT15 in the ARM's interrupt controller, is triggered upon completion of a DMA transfer. Refer to the section on the interrupt controller for more information on the ARM's interrupt controller.

20.9.6 DSP (Image Buffer) and DSP (HPIB) Access to the EMIF

It is possible to transfer DSP internal memory data via the HPIB to the EMIF/SDRAM, and it is possible to transfer data to and from the DSP's Image buffers and iMX Coefficient Memory to the EMIF/SDRAM. Refer to the "DSP Subsystem" section for information on how to transfer data from the DSP(HPIB) to the EMIF. Refer to the "DSP Coprocessor Subsystem" section for information on how to transfer data to and from the DSP's Image Buffer and iMX Coefficient Memory.

Three registers in the EMIF module are used to configure the DM270 to allow HPI data and/or image buffer data to be transferred between the EMIF:

- IMGDSPDEST - EMIF Image Buffer/DSP(HPIB) Device Select Register
- IMGDSPADDH - EMIF Image Buffer/DSP(HPIB) Destination Address High
- IMGDSPADDL - EMIF Image Buffer/DSP(HPIB) Destination Address Low

CS1 and CS2 are generally used to interface CompactFlash and SmartMedia cards, respectively. Data transfer to a CompactFlash or SmartMedia card is always performed through a fixed address (see table 0 and 0), so unlike other memories, the access address should not auto-increment after each transfer. Therefore, when transferring data between CS1 and CS2, IMGDSPADDH - EMIF Image Buffer/DSP(HPIB) Destination Address High and IMGDSPADDL - EMIF Image Buffer/DSP(HPIB) Destination Address Low are used to specify the source/destination address. When transferring data to CS0, CS3, or CS4; the same registers in the DSP(HPIB) and Image Buffer modules that used to specify SDRAM/EMIF transfer address are used to specify SDRAM/EMIF transfer addresses for CS0, CS3, CS4.

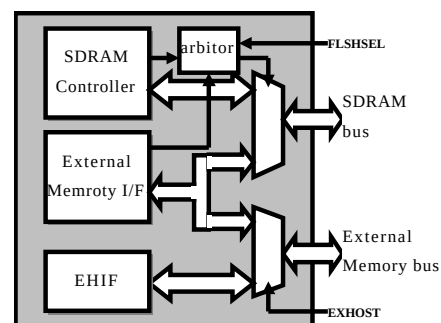
IDST and DDST in IMGDSPDEST - EMIF Image Buffer/DSP(HPIB) Device Select Register are used to specify which External Memory Region (CS0, CS1, CS2, CS3 or CS4) to involve in the transfer of DSP(Image Buffer) data and DSP(HPIB) data, respectively. This register needs to be set for all EMIF data transfer (CS0, CS1, CS2, CS3 or CS4)

IMPORTANT: When using DSP HPIB DMA, the DMA channel must be selected using register REFCTL - Refresh Control Register

20.10 SDRAM & External Memory I/F Pin Sharing

The SDRAM interface signals and EMIF interface signals can share pins on the DM270. The sharing of pins between SDRAM and flash memory is carried out on bus cycle by bus cycle basis. Each of SDRAM interface and flash memory interface has a separate chip select. When a bus cycle falls into the SDRAM's address space, the chip select of the SDRAM is asserted and the shared pins generate the SDRAM-related timing control signals to perform the cycle. Similarly for flash memory, if the cycle is its address space, its chip select is asserted and the shared pins generate the signals to perform the cycle. Note that it is possible that while ARM is accessing the flash memory (for example, to fetch an instruction), there may be concurrent DMA activities on the SDRAM. In such case, flash memory and the DMA will arbitrate for the control for the memory interface.

Figure 158. SDRAM and EMIF Pin sharing



Shows the different shared memory configurations of DM270. All the logic for multiplexing SDRAM interface signals and the External Bus (EXBUS) interface signals to the I/O pad is done by internal register and FLSHSEL/EXHOST pins. Figure 159, Figure 160, Figure 161, and Figure 162 illustrate shared memory configurations.

Shared Memory Configuration

	FLSHSEL*	BUSSEL*	SDRAM	Flash ROM	CFC/SSFDC/ SRAM
MODE1	Low	0	SDRAM bus	Ex. Memory bus	Ex. Memory bus
MODE2	High	0	SDRAM bus	SDRAM bus	Ex. Memory bus
MODE3	High	1	SDRAM bus	SDRAM bus	SDRAM bus

*FLSHSEL: pin name BUSSEL: CPU registers in EMIF module

There are two types of usages for memory resource in DM270. The first type is the normal usage for the SDRAM memory, which is the same as DSC25. In this type, SDRAM Controller (SDRC) is the master of the SDRAM memory. The second type is the usage of SDRAM interface pins by not only the SDRAM memory but other memories as well. In this type, SDRC and External Memory Interface (EMIF) are masters that use SDRAM bus pins and they are arbitrated. With this, a scheme can be implemented to allow the pins to be shared by the SDRAM external and EXBUS interface.

Pin sharing is required to support the optional media storage card hot plug within the constraint of pin count. The basic idea of hot plug is to have two separate buses for the media storage card (for example Compact Flash Card) and the flash memory so that plug and unplug of the media card does not interfere with the normal operation of flash memory.

Pin sharing method is also required to support external HOST CPU connection with external memory bus. The external CPU can be a bus master by releasing all the external memory bus to it.

Shows a list of pins that are shared between SDRAM and EXBUS

External HOST CPU Configuration

EXTHOST*	External Host CPU
Low	Not available
High	Available

*EXTHOST: pin name

Figure 159. Shared memory modes (MODE1)

MOI

Figure 160. Shared Memory Modes (MODE2)

MOI

Figure 161. Shared Memory Modes (MODE3)

M0E

Figure 162. Shared Memory Modes (MODE4)

M0E

Pins shared between SDRAM and EXBUS

Pin name	External Memory bus
SDR_DQ23	ARM_A22
SDR_DQ22	ARM_A21
SDR_DQ21	ARM_A20
SDR_DQ20	ARM_A19
SDR_DQ19	ARM_A18
SDR_DQ18	ARM_A17
SDR_DQ17	ARM_A16
SDR_DQ16	ARM_A15
SDR_A14	ARM_A14
SDR_A13	ARM_A13
SDR_A12	ARM_A12
SDR_A11	ARM_A11
SDR_A10	ARM_A10
SDR_A9	ARM_A9
SDR_A8	ARM_A8
SDR_A7	ARM_A7
SDR_A6	ARM_A6
SDR_A5	ARM_A5
SDR_A4	ARM_A4
SDR_A3	ARM_A3
SDR_A2	ARM_A2
SDR_A1	ARM_A1
SDR_A0	ARM_A0
SDR_DQ15	ARM_D15
SDR_DQ14	ARM_D14
SDR_DQ13	ARM_D13
SDR_DQ12	ARM_D12
SDR_DQ11	ARM_D11
SDR_DQ10	ARM_D10
SDR_DQ9	ARM_D9
SDR_DQ8	ARM_D8
SDR_DQ7	ARM_D7
SDR_DQ6	ARM_D6
SDR_DQ5	ARM_D5
SDR_DQ4	ARM_D4
SDR_DQ3	ARM_D3
SDR_DQ2	ARM_D2
SDR_DQ1	ARM_D1
SDR_DQ0	ARM_D0
SDR_RAS	EM_OE
SDR_CAS	EM_WE
SDR_DQ26	EM_RDWR
SDR_DQ25	EM_BEH
SDR_DQ24	EM_BEL

21 External Host Interface (EHIF)

21.1 Introduction

The external host CPU interface module (EHIF), shown in Figure 163, is a 16-bit parallel port used to interface a host CPU to the DM270. By setting the EXTHOST pin to high level, the DM270 is set in external CPU mode and ARM_A22-ARM_A0, EM_CE0, EM_WE and EM_OE become input pins. The EM_WAIT signal becomes an output pin. The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Both read and write are half-word (16 bit) access.

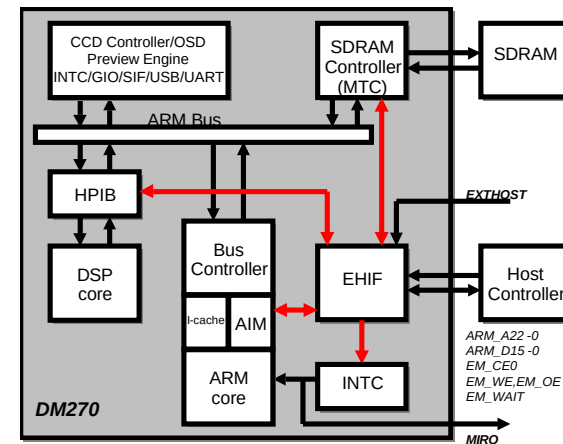
Even in external CPU mode, the ARM7 in DM270 can access all peripheral registers, internal memories and external memories. Data can be exchanged between the host CPU and the DM270 throughout the on chip memory such as ARM internal memory and DSP internal memory via EHIF module. The SDRAM, which is connected externally, is also accessible via EHIF and SDRAM controller.

The EHIF also provides the capability to access memory during reset. During reset, data or application code can be loaded via the EHIF, and the application can be initialized through the boot-up option by setting the BTSEL pin to high level (Refer "ARM Boot Up Modes" for details).

Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM270.

NOTE: DM270 External HOST mode and normal mode of operation cannot be switched dynamically, since DM270 operation mode is switched between external HOST or normal mode during power-on reset

Figure 163. External HOST Interface Mode Block Diagram



21.2 Memory Space

The memory areas that can be accessed from an external host controller are ARM internal memory (32 Kbytes), SDRAM (16.128Kbytes), and DSP internal memory (128 Kbytes). However, the memory map addresses differ from the usual ARM memory map. Table 53. shows the memory map of DM270 as seen from the external CPU.

Table 53. Memory Space

Memory Area	Start address	End address	size(byte)
EHIF registers	0x00:0000	0x00:007F	128
ARM Internal Memory	0x00:8000	0x00:FFFF	32 K
DSP Internal Memory	0x02:0000	0x03:FFFF	128 K
SDRAM (Single access)	0x04:0000	0xFF:FFFF	16.128 K
SDRAM (Burst access)	--	--	128M

ARM and DSP internal memory access on the DM270 are in shared-access mode, meaning both the DM270 and the host CPU can access memory. Asynchronous host accesses are resynchronized internally, and in the event that the ARM7 in DM270 and the host CPU request access to the same memory block, the host CPU has access priority. The access priority to the external SDRAM is configured by priority registers in SDRAM controller. The EM_WAIT pin can provide handshaking to the host during memory

21.3 Bus Access in External HOST Mode

- o The data bus is fixed at 16-bits and uses ARM_D15-ARM_D0. Both read and write are half-word (16 bit) access.
- o Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM270 ('0':Busy, '1':ready). WAITEN in register EHIFCTL (0000:0002) of the External CPU Interface must be set in advance to enable the EM_WAIT output.
- o The access time seen from the host device varies depending on ARM, DSP and SDRAM clock frequency settings
- o 32byte Burst/Continuous Burst access to SDRAM is supported via sixteen 16bit data registers.
- o If a read is performed continuously after read, EM_OE or EM_CEO must once return to high level. RTRG in register EHIFCTL (0000:0002) must be set in case EM_OE becomes high level and EM_CEO stays low.

21.3.1 Access Time

The External Host interface module operates using the ARM clock of the DM270. The access time seen from the host device varies depending on ARM, DSP and SDRAM clock frequency settings. The number of cycles of the ARM clock is shown below.

Memory Access	Timing
EHIF register/SDRAM burst access/AIM write	2 +a cycles (a:Input and/or Output delay)
ARM Internal memory read	3 +a cycles
SDRAM single access/DSP memory	3 + n +a cycles (n:Peripheral wait cycles)

21.4 Interrupts in External HOST Mode

- o Host device can issue an interrupt signal to the ARM by setting the register INTCTL(0000:0004). (EXTHOST interrupt is mapped to INT10 in ARM interrupt controller. See "Interrupt Controller" section for details)
- o In case the slave DM270 needs to interrupt external HOST, an arbitrary GIO pin can be used for this purpose

NOTE: DM270 ARM IRQ interrupt signal can be output over GIO25. Here, GIO25 has to be configured to function as the MIRQ (interrupt) signal using FSEL1 register in GIO module (refer GIO section for details). The IRQ interrupt can be generated for any of the interrupt sources available in INTC module. The HOST needs to check INTC registers to determine the source of the IRQ interrupt. Since external HOST cannot access INTC registers directly, DM270 ARM must save the INTC registers in some pre-designated memory location in ARM internal memory or SDRAM. (See also "Interrupt Controller" section)

21.5 SDRAM Access

Single/32byte Burst/Continuous Burst access to SDRAM is supported. Sixteen 16bit data registers are used for Burst/Continuous Burst access. When in Continuous Burst transfer mode, data of the specified length in 32byte units are transferred to and from SDRAM.

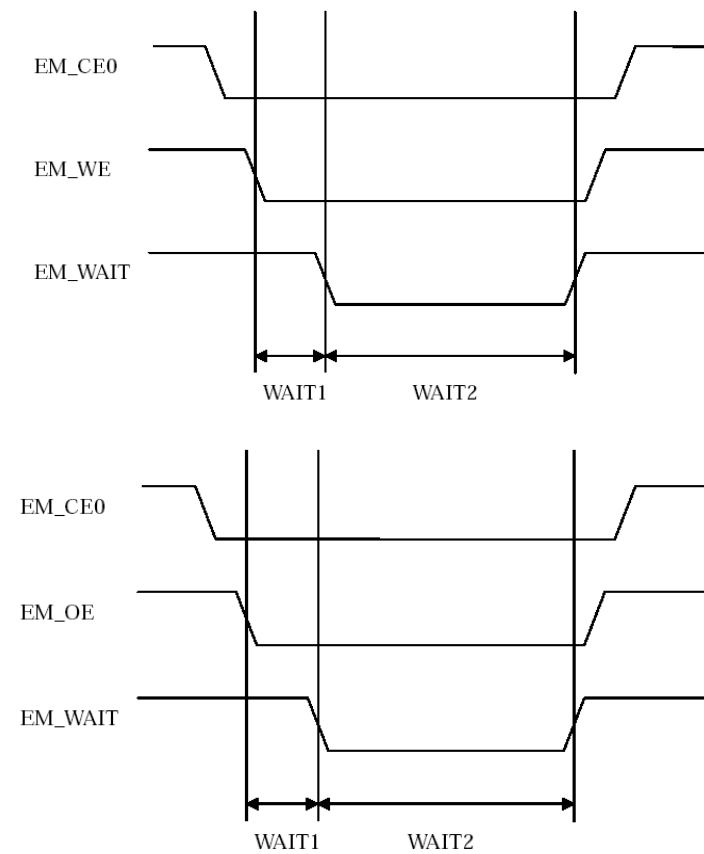
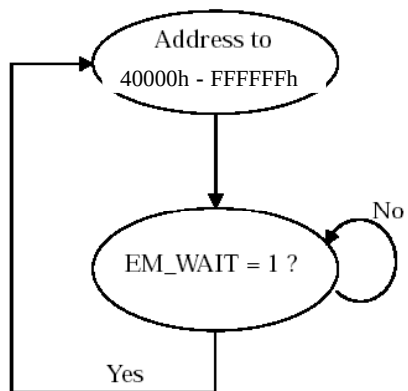
There are two modes for Continuous Burst access.

- o **MODE 1**
Address decode enabled. Burst transfer will be performed by writing or reading to/from a data register SDRBUF0 (0000:0010). Register, AIM and HPIB accesses are allowed during Continuous Burst access. There are some registers (EHIFSTS (0000:0030)) to monitor the status of EHIF during Continuous Burst access which are:
 - n Burst transfer wait flag: To check the Burst transfer status every 16 accesses. This is for a host device, which does not have a WAIT input.
 - n SDRAM access wait flag: To check the access status between EHIF and SDRC before you start access to AIM/HPIB during Continuous Burst transfer, or before you end the Continuous Burst transfer.
- o **MODE 2**
Address decoding will not be performed and every access from a host device will be treated as a burst transfer request once the data transfer starts, and until transfer of the specified length ends. No other access is allowed during Continuous Burst transfer.

21.5.1 Single Access (Random Access):

The host CPU can access to SDRAM randomly addressing to 4:0000h-FF:FFFFh (byte address). When SDRAM is accessed in single access mode, extra wait cycles are added. The host CPU can check the status of SDRAM access using EM_WAIT pin.

Figure 164. SDRAM Single Access



Write operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	2	-
WAIT2	No other request	32bit	3	7
		16bit	3	7
	One request at higher priority	32bit	3	15
		16bit	3	23
	N requests at higher priority	32bit	3	7+8N
		16bit	3	7+16N

Read operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	2	-
WAIT2	No other request	32bit	3	12
		16bit	3	13
	One request at higher priority	32bit	3	20
		16bit	3	29
	N requests at higher priority	32bit	3	12+8N
		16bit	3	13+16N

21.5.2 Single Burst Access (Buffered Access):

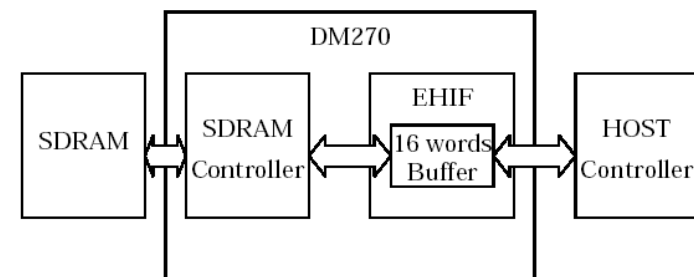
Transfer to and from SDRAM is performed in 16 16-bit word units. There is a control register in the EHIF for burst access as well as sixteen 16-bit data registers to store the data from 16-word burst transfer.

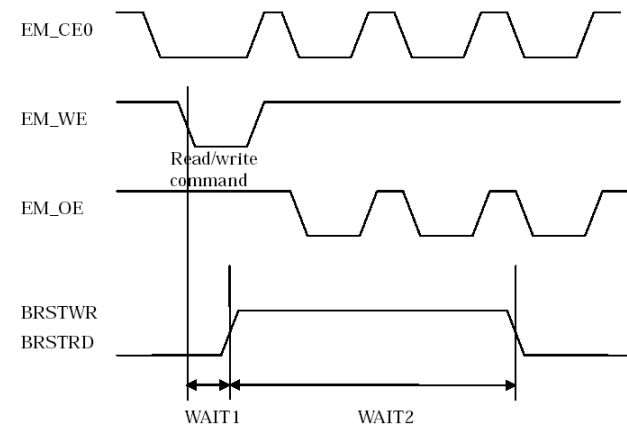
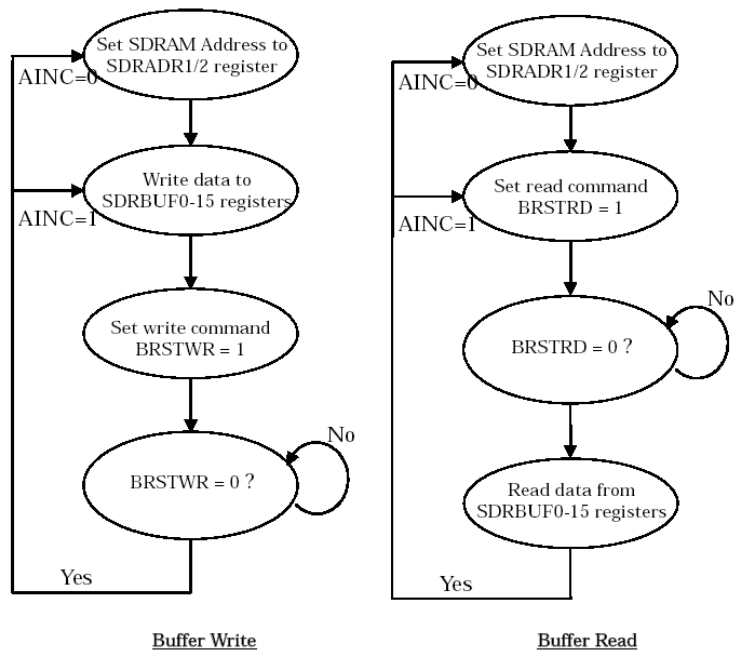
To write into the SDRAM, data needs to be set in these data registers. After the write destination SDRAM address is set in the buffer address registers, writing to the buffer transfer register (BRSTCTL1 (0000:0006)) can trigger a write. When the proper bit (BRSTWR or BRSTRD) in this register is written, writing to the SDRAM is performed.

To read from SDRAM, the read origin from SDRAM address must be set first. This information is stored in the buffer address registers. When a read command is issued by writing the bit BRSTRD of the control register, the data is read from SDRAM and stored in the data registers.

You need to check the status of the BRSTWR or BRSTRD bit to know if a transfer is completed or not.

Figure 165. SDRAM Buffered Access





Write operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	3	-
WAIT2	No other request	32bit	2	14
		16bit	2	14
	One request at higher priority	32bit	2	22
		16bit	2	30
	N requests at higher priority	32bit	2	14+8N
		16bit	2	14+16N

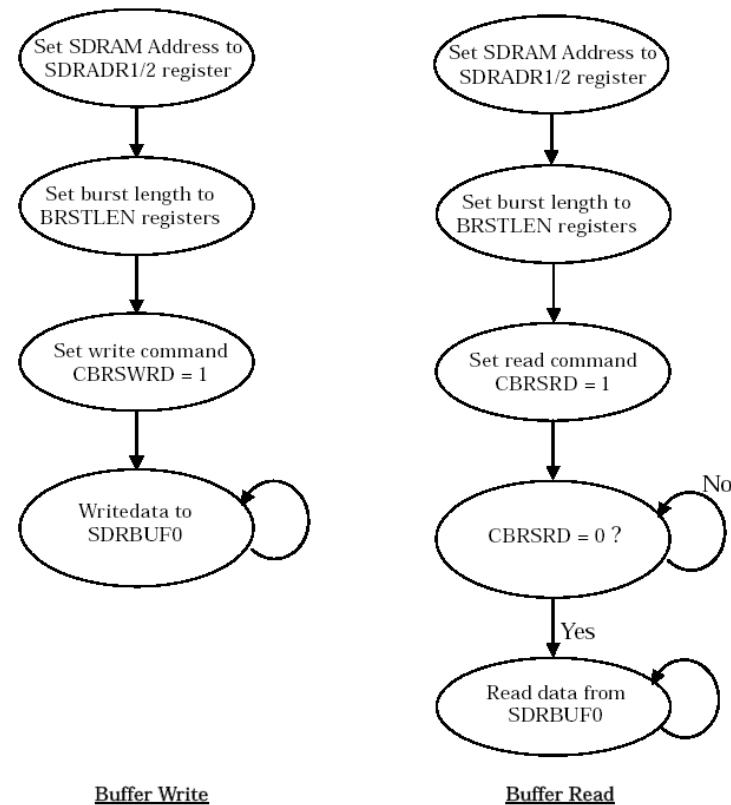
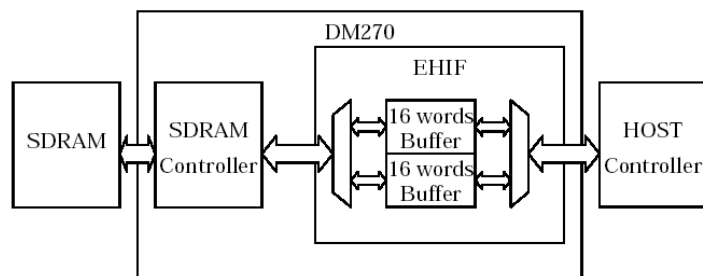
Read operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	3	-
WAIT2	No other request	32bit	2	19
		16bit	2	27
	One request at higher priority	32bit	2	27
		16bit	2	43
	N requests at higher priority	32bit	2	19+8N
		16bit	2	27+16N

21.5.3 Continuous Burst Access (Buffered Access):

Transfer to and from SDRAM is performed in 2 sets of 16 16-bit word units. To write into the SDRAM, the write origin to SDRAM address (SDRADR1 (0000:000C), SDRAMR2 (0000:000E)) and total transfer size (BRSTLEN (0000:000A)) must be set first. The host CPU writes a data continuously to SDBUF0 register. When it reaches at 16th write data, the SDRAM transfer is performed automatically and the host CPU can write the continuous data to another 16 word buffer. In this way the continuous burst write operation is achieved. When the proper bit (CBRSWR or CBRSRD) in register BRSTCTL2 (0000:0008) is written, writing to the SDRAM is performed.

To read from SDRAM, the read origin from SDRAM address and total transfer size must be set first. After this check the SDRBSY bit (EHIFSTS (0000:0030)) and confirm that the 16 word transfer end. The host CPU can read data from SDRBUF0 (0000:0010) register. When reaches at 16th read data, the SDRAM transfer is performed automatically and the host CPU can read the continuous data to another 16 word buffer. In this way the continuous burst read operation is achieved.

Figure 166. SDRAM Continuous Burst Access



Write operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	3	-
WAIT2	No other request	32bit	2	14
		16bit	2	14
	One request at higher priority	32bit	2	22
		16bit	2	30
	N requests at higher priority	32bit	2	14+8N
		16bit	2	14+16N

Read operation		SDRAM Bus	ARM (cycle)	SDRAM (cycle)
WAIT1		-	3	-
WAIT2	No other request	32bit	2	19
		16bit	2	27
	One request at higher priority	32bit	2	27
		16bit	2	43
	N requests at higher priority	32bit	2	19+8N
		16bit	2	27+16N

21.6 EHIF Control Register Map

Table 54. EHIF Register List

Register Address	Register Name	Description
0x00	REVR	Device revision ID
0x02	EHIFCTL	EHIF control signals
0x04	INTCTL	Interrupt control
0x06	BRSTCTL1	SDRAM Burst transfer control 1
0x08	BRSTCTL2	SDRAM Burst transfer control 2
0x0A	BRSTLEN	SDRAM Burst transfer length
0x0C	SDRADR1	SDRAM address for Burst transfer
0x0E	SDRADR2	SDRAM address for Burst transfer
0x10	SDRBUF0	Data registers for SDRAM Burst transfer
0x12	SDRBUF1	Data registers for SDRAM Burst transfer
0x14	SDRBUF2	Data registers for SDRAM Burst transfer
0x16	SDRBUF3	Data registers for SDRAM Burst transfer
0x18	SDRBUF4	Data registers for SDRAM Burst transfer
0x1A	SDRBUF5	Data registers for SDRAM Burst transfer
0x1C	SDRBUF6	Data registers for SDRAM Burst transfer
0x1E	SDRBUF7	Data registers for SDRAM Burst transfer
0x20	SDRBUF8	Data registers for SDRAM Burst transfer
0x22	SDRBUF9	Data registers for SDRAM Burst transfer
0x24	SDRBUF10	Data registers for SDRAM Burst transfer
0x26	SDRBUF11	Data registers for SDRAM Burst transfer
0x28	SDRBUF12	Data registers for SDRAM Burst transfer
0x2A	SDRBUF13	Data registers for SDRAM Burst transfer
0x2C	SDRBUF14	Data registers for SDRAM Burst transfer
0x2E	SDRBUF15	Data registers for SDRAM Burst transfer
0x30	EHIFSTS	EHIF status

21.7 Detailed Register Description

21.7.1 REVR (0000:0000)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REV7	REV6	REV5	REV4	REV3	REV2	REV1	REV0
RAW	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	0	0	0	1	0	0	0	1

Bits	Register Name	Description
15:8	RSV	Reserved bit.
7:0	REV7-REV0	Device revision ID. eg.) ES1.0 => 0x10, ES1.1 => 0x11, Revision ID of DM270 can be read out from this register.

21.7.2 EHIFCTL (0000:0002)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	ENDIW1	ENDIW0	ENDIR1	ENDIR0	RSV	RSV	RSV	RTRG	RSV	RSV	RSV	WAITEN
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	-	0	0	0	0	-	-	-	0	-	-	-	0

Bits	Register Name	Description
15:12	RSV	Reserved bit.
11:10	ENDIW	Endian select for Write access Bit1: half-word swap enable(AIM, SDRAM) Bit0: byte swap enable 00 : D0 D1 D2 D3 à D0 D1 D2 D3 (no swap) 01 : D0 D1 D2 D3 à D1 D0 D3 D2 (byte swap enable) 10 : D0 D1 D2 D3 à D2 D3 D0 D1 (half word swap enable) 11 : D0 D1 D2 D3 à D3 D2 D1 D0 (both enable)
9:8	ENDIR	Endian select for Read access Bit1: half-word swap enable(AIM, SDRAM) Bit0: byte swap enable 00 : D0 D1 D2 D3 à D0 D1 D2 D3 (no swap) 01 : D0 D1 D2 D3 à D1 D0 D3 D2 (byte swap enable) 10 : D0 D1 D2 D3 à D2 D3 D0 D1 (half word swap enable) 11 : D0 D1 D2 D3 à D3 D2 D1 D0 (both enable)
7:5	RSV	Reserved bit.
4	RTRG	Data Read trigger select bit. 0: nCS falling edge 1: nOE falling edge RTRG selects the trigger to start a Read access. A Read access is triggered by the CE0 falling edge in case RTRG='0', or by the EM_OE falling edge in case RTRG='1'.
3:1	RSV	Reserved bit.
0	WAITEN	EM_WAIT output enable. 0: EM_WAIT='Z' 1: EM_WAIT= OUTPUT(Busy/Ready status) Host device can monitor the status of DM270 through EM_WAIT. It allows the host device to dynamically access DM270.

21.7.3 INTCTL(0000:0004)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	INT
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bits	Register Name	Description
15:1	RSV	Reserved bit.
0	INT	Issue an interrupt to ARM Writing '1' to this bit issues an interrupt to ARM. This bit will be cleared automatically (EXTHOST interrupt is mapped to INT10 in ARM interrupt controller. See "Interrupt Controller" section for details).

21.7.4 BRSTCTL1 (0000:0006)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BFCLR	RSV	RSV	BRSTWR	BRSTRD
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	0

Bits	Register Name	Description
15:5	RSV	Reserved bit.
4	BFCLR	SDRAM data buffer clear. Writing '1' to this bit clears the content of SDRBUF15 to SDRBUF0. This bit will be cleared automatically after data buffer clear is done.
3:1	RSV	Reserved bit.
1	BRSTWR	SDRAM Burst Write enable. Writing '1' to this bit starts 32byte Burst Write transfer from SDRBUF15-SDRBUF0 to SDRAM. This bit will be automatically cleared when the transfer is completed.
0	BRSTRD	SDRAM Burst Read enable. Writing '1' to this bit starts 32byte Burst Read transfer from SDRAM to SDRBUF15-SDRBUF0. This bit will be automatically cleared when the transfer is completed.

21.7.5 BRSTCTL2 (0000:0008)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CBRMD	RSV	RSV	CBRSWR	CBRSRD
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	0

Bits	Register Name	Description
15:5	RSV	Reserved bit.
4	CBRSMD	Specify a mode of SDRAM Continuous Burst transfer. 0: mode1 – Burst transfer will be performed by writing or reading to/from a data register SDRBUF0. 1: mode2 – Address decoding will not be performed and every access from a host device will be treated as a burst transfer request once the data transfer starts, and until transfer of the specified length ends.
3:1	RSV	Reserved bit.
1	CBRSWR	SDRAM Continuous Burst Write enable. Writing '1' to this bit starts Continuous Burst Write transfer of the specified length set in BRSTLEN. Writing '0' to this bit ends data transfer, or otherwise, it will be automatically cleared when the transfer is completed.
0	CBRSRD	SDRAM Continuous Burst Read enable. Writing '1' to this bit starts Continuous Burst Read transfer of the specified length set in BRSTLEN. Writing '0' to this bit ends data transfer, or otherwise, it will be automatically cleared when the transfer is completed.

21.7.6 BRSTLEN (0000:000A)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRSTL15	BRSTL14	BRSTL13	BRSTL12	BRSTL11	BRSTL10	BRSTL9	BRSTL8	BRSTL7	BRSTL6	BRSTL5	BRSTL4	BRSTL3	BRSTL2	BRSTL1	BRSTL0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	BRSTL	Burst Length (32Byte unit).

21.7.7 SDRADR1 (0000:000C)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AINC	RSV	RSV	SDRA21	SDRA20	SDRA19	SDRA18	SDRA17	SDRA16
R/W	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	-	-	0	0	0	0	0	0

Bits	Register Name	Description
15:9	RSV	Reserved bit.
8	AINC	SDRAM Address auto increment. Writing '1' to this bit enables auto address increment after a burst transfer is completed.
7:6	RSV	Reserved bit.
5:0	SDRA(21:16)	Upper 6 bits of SDRAM Address for burst transfer. Address is the offset from the SDRAM base and is specified in 32byte units.

21.7.8 SDRADR2 (0000:000E)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDRA15	SDRA14	SDRA13	SDRA12	SDRA11	SDRA10	SDRA9	SDRA8	SDRA7	SDRA6	SDRA5	SDRA4	SDRA3	SDRA2	SDRA1	SDRA0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDRA(15:0)	Lower 16 bits of SDRAM Address for burst transfer. Address is the offset from the SDRAM base and is specified in 32byte units.

21.7.9 SDRBUF0 (0000:0010)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB0	Buffer0 for SDRAM burst transfer

21.7.10 SDRBUF1 (0000:0012)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB115	SDB114	SDB113	SDB112	SDB111	SDB110	SDB19	SDB18	SDB17	SDB16	SDB15	SDB14	SDB13	SDB12	SDB11	SDB10
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB1	Buffer1 for SDRAM burst transfer

21.7.11 SDRBUF2 (0000:0014)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB215	SDB214	SDB213	SDB212	SDB211	SDB210	SDB29	SDB28	SDB27	SDB26	SDB25	SDB24	SDB23	SDB22	SDB21	SDB20
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB2	Buffer2 for SDRAM burst transfer

21.7.12 SDRBUF3 (0000:0016)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB315	SDB314	SDB313	SDB312	SDB311	SDB310	SDB39	SDB38	SDB37	SDB36	SDB35	SDB34	SDB33	SDB32	SDB31	SDB30
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB3	Buffer3 for SDRAM burst transfer

21.7.13 SDRBUF4 (0000:0018)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB4	Buffer4 for SDRAM burst transfer

21.7.14 SDRBUF5 (0000:001A)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB5	Buffer5 for SDRAM burst transfer

21.7.15 SDRBUF6 (0000:001C)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB6	Buffer6 for SDRAM burst transfer

21.7.16 SDRBUF7 (0000:001E)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB7	Buffer7 for SDRAM burst transfer

21.7.17 SDRBUF8 (0000:0020)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB8	Buffer8 for SDRAM burst transfer

21.7.18 SDRBUF9 (0000:0022)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB9	Buffer9 for SDRAM burst transfer

21.7.19 SDRBUF10 (0000:0024)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB10	Buffer10 for SDRAM burst transfer

21.7.20 SDRBUF11 (0000:0026)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB11	Buffer11 for SDRAM burst transfer

21.7.21 SDRBUF12 (0000:0028)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB12	Buffer12 for SDRAM burst transfer

21.7.22 SDRBUF13 (0000:002A)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB13	Buffer13 for SDRAM burst transfer

21.7.23 SDRBUF14 (0000:002C)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB14	Buffer14 for SDRAM burst transfer

21.7.24 SDRBUF15 (0000:002E)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDB015	SDB014	SDB013	SDB012	SDB011	SDB010	SDB09	SDB08	SDB07	SDB06	SDB05	SDB04	SDB03	SDB02	SDB01	SDB00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Register Name	Description
15:0	SDB15	Buffer15 for SDRAM burst transfer

21.7.25 EHIFSTS (0000:0030)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PLLLK	RSV	RSV	RSV	SDRBSY	RSV	RSV	RSV	BRSWT
RW	-	-	-	-	-	-	-	R	-	-	-	R	-	-	-	R
Default	-	-	-	-	-	-	-	0	-	-	-	0	-	-	-	0

Bits	Register Name	Description
15:9	RSV	Reserved bit.
8	PLLLK	PLL lock status '1' at this bit indicates PLL is locked.
7:5	RSV	Reserved bit.
4	SDRBSY	SDRAM Access busy flag. '1' at this bit indicates that Data transfer between EHIF and SDRC is busy. HPIB and AIM can be accessed during SDRAM Continuous Burst transfer (mode0) after this bit becomes '0'.
3:1	RSV	Reserved bit.
0	BRSWT	Burst transfer access wait flag '1' at this bit indicates that wait state is asserted for Burst transfer access, so that any access to the data register (SDRBUF) is prohibited.

22 ARM Boot-Up Modes

DM270 provides three different ARM boot-up sequences. There are

- o Self boot-up mode,
- o Serial boot-up mode and
- o External HOST boot-up mode

Serial boot-up mode and external HOST boot-up mode makes it possible to download an ARM boot code externally.

The BTSEL pin defines how ARM will operate after power-on reset. Table 55. shows pin configuration.

Table 55. ARM boot-up configuration

Pin	Status	Function
BTSEL	0	Self boot up Jump to external Flash memory at address 0x100000
	1	Serial/External Host boot up Jump to ARM internal memory

22.1 Self boot-up mode

Upon power up and reset, the ARM program counter is reset to 0x0000 (ARM reset vector). This address resides at the start of ARM Internal memory. When BTSEL pin is set to "0" the address location from 0x0000 to 0x0004 is ROM and the ARM reset vector is hard coded in ROM at this location with an instruction to branch to 0x0010:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM270 system. After turn-off, the power on reset signal to first access to CS0(FLASH_CE) region takes about 10 ARM clock cycles. In this mode, the external pin of BTSEL should be tied to low(ground) level.

22.2 Serial boot-up mode

DM270 program/data can downloaded into ARM internal memory externally through serial interface (SIF0) and UART (UART0) pins. During reset, the external CPU can gain access to ARM internal memory by using SIF/UART. This interface consists of four signals serial clock input(SCLK0), enable signal input(SDEN0), serial data input(RXD0) and serial data output(TXD0). The external CPU can write the necessary data to ARM internal memory(AIM), for instance boot code, and read out the data from AIM.

The followings are the features in this mode:

- Full addressing of ARM internal memory (AIM) from 0x0 to 0x7FFF
- Access mode is a byte mode only
- Address can be incremented automatically after sending first address
- Supports both write and read operation
- Effective when power on reset signal is low level, eg. during DM270 is on reset.
- Effective when SDEN0 is low level

Figure 167 and Figure 168 show the timing diagram for reading and writing data via the serial interface.

The first received bit indicates read/write command and it is followed by a 15-bit address. When the command bit is low, a write operation is to be performed and when the command bit is high, a read operation is to be performed.

Following the command and address bits, data of 8 bits length is sent to the AIM in the write mode via the RXD0 pin. In the read mode, data of 8 bits length read from AIM will be transferred through data out (TXD0) pin.

The enable signal is active low during this process. The address to be accessed is incremented automatically when enable signal is low.

The minimum unit of the transfer is defined 8 bits. If the data transferred is less than 8 bits it will be discarded.

The interface has the below protocol:

- The address/data to be transferred is MSB first.
- The maximum frequency of SCLK0 is 10MHz.
- Performed while the power on reset signal is in assertion

IMPORTANT: When program/data is to be downloaded via the serial interface, BSTEL pin must be HIGH and reset signal must be asserted. When BSTEL pin HIGH, the area of reset vector ROM (0x0 to 0x0004) is replaced by the first 4 byte of RAM. When the data is transferred to the AIM, the reset signal is deasserted and ARM program counter (PC) is loaded with address 0x0000 (ARM reset vector). The user must make sure a branch to user initialization routine is set at this location (0x0 to 0x0004) during the downloading of program via the serial interface

Figure 169 shows the detail timing sequence of ARM boot via serial interface

Figure 167. Serial data read timing

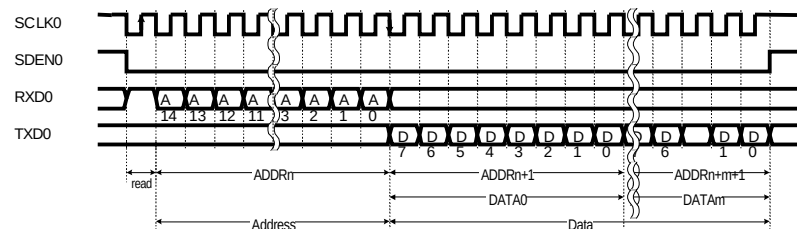


Figure 168. Serial data write timing

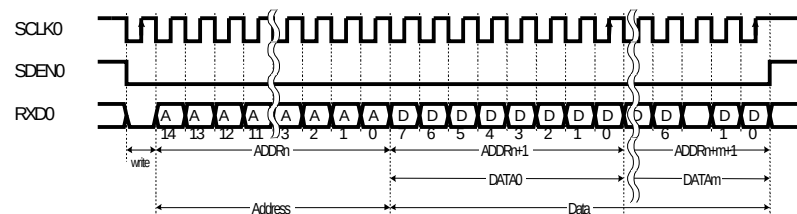
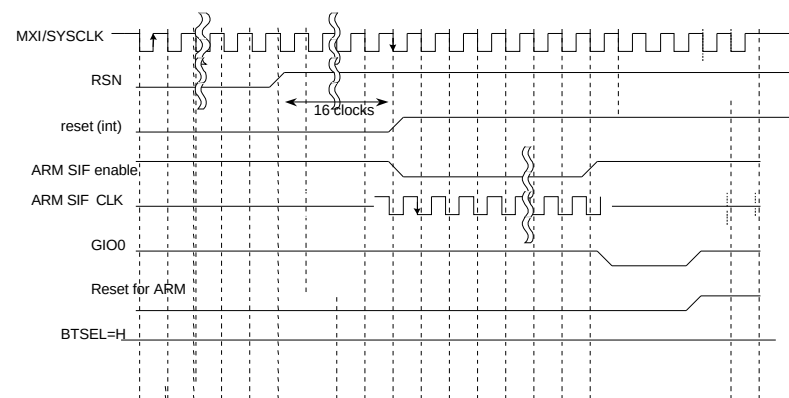


Figure 169. Timing Sequence of ARM boot via serial interface

DM270 Sequence of ARM Boot -up via AIM SIF:

1. BSTSEL=H, RSN=L, (EXTHOST=L)
2. RSN=H
3. Wait 16 cycles at MXI/SYSCLK
4. Load ARM code to AIM via AIMSIF
5. To deassert system reset, make GIO0=H -> L (2 cycles)



22.3 External HOST boot-up mode

External HOST can access ARM internal memory directly through EMIF bus pins. See chapter "External HOST Interface" for details about memory access in external HOST mode. This makes it possible for the external HOST to download an ARM boot code externally.

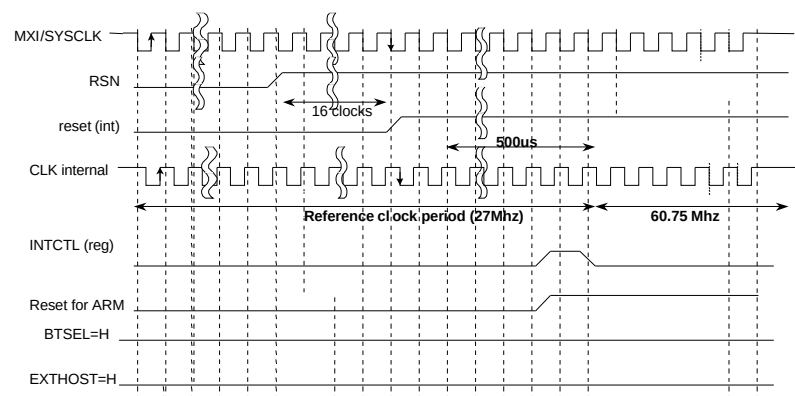
IMPORTANT: When program/data is to be downloaded via the external HOST boot mode, BSTEL pin must be HIGH and reset signal must be asserted (EXTHOST is also high in external HOST mode). When BSTEL pin HIGH, the area of reset vector ROM (0x0 to 0x0004) is replaced by the first 4 byte of RAM. When the program/data is transferred to the AIM, the reset signal is deasserted and ARM program counter (PC) is loaded with address 0x0000 (ARM reset vector). The user must make sure a branch to user initialization routine is set at this location (0x0 to 0x0004) during the downloading of program by the external HOST.

Figure 170 shows the detail timing sequence of ARM boot via external HOST boot-up mode.

Figure 170. Timing sequence of ARM boot via external HOST boot-up mode.

DM270 Sequence of ARM Boot -up via EXTHOST:

1. EXTHOST=H, BTSEL=H, RSN= L
2. RSN=H
3. Wait 16 cycles at MXI/SYSCLK
4. Load ARM code to AIM via external host
5. To deassert system reset, write 0x0001 into INTCTL(0x04) register of EHIF



23 ARM Power Saving Modes

The DM270 consumes a low amount of power during normal operation; however, it is possible to further reduce the DM270's power consumption via the DM270's various power saving modes.

The ARM7 subsystem used in the DM270 features several power-saving modes to lengthen the operating time on battery-powered systems. The content of the CPU and the registers settings are maintained when the device is in one of these modes.

Depending on the power down mode chosen, there are different ways to enter and exit it. Each one is described in this section

Refer to the DSP subsystem chapter for information on DSP-side power saving modes.

23.1 Disabling Module Clocks

The first method to reduce the power consumption in the ARM Subsystem is to disable the clock of any module whose function is not needed. For example, you can disable the clock supplied to the Preview Engine when your system does not require the Preview Engine module.

Use the registers MOD0 (0x30890), MOD1(0x30892) and MOD2(0x30894) in the Clock Controller module to disable module clocks. Refer to the section titled "Clock Controller" for more information on these registers.

If clock supply to the ARM core, ARM internal memory and bus controller is stopped, DM270 is completely stopped. Therefore, these clocks must be stopped last.

Even if all clocks in DM270 are stopped, RAM contents are saved as long as power is not lost, and it is not necessary to reload ARM and DSP programs. If you wish to save SDRAM contents, the SDRAM must be set to self-refresh mode before stopping the SDRAM clock.

Recovery from sleep is performed by GIO0. You need to apply a pulse signal on the GIO0 pin (active low) at least the width of two clocks of SYSCLK or MXI clock. Since the PLLs still operate, the full recovery is quick.

23.2 USB Power Saving Modes

The DM270's USB module automatically enters a power saving mode after the USB module is idle for a period 6ms.

Furthermore, if your system does not use the USB peripheral, you can manually disable the USB peripheral by connecting the USB Vdd (USB_VDD), the USB D+ pin (USB_DP), and the USB D- pin (USB_DM) ground (GND).

23.2.1 Disabling the 48MHz Oscillator

The bit OSC48 in the LPCTL1 (0x30898) register allows you to disable the oscillator connected to the 48MHz input (M48XI). Disable the 48MHz oscillator when your system does not require the 48MHz clock.

23.3 Sleep Mode

The DM270 is said to be in sleep mode when all module clocks are disabled and all PLLs are stopped.

23.3.1 Entering Sleep Mode

The Sleep Mode bit, SLPMD, in LPCTL0 (0x30896) allows you to enter sleep mode. When SLPMD is set to "1" all PLL's are stopped and all module clocks are disabled. Although all module clocks are disabled in sleep mode, the bits in MOD0 (0x30890), MOD1(0x30892) and MOD2(0x30894) are not affected. Therefore, any module clock that was enabled before entering sleep mode will be re-enabled upon wake-up from sleep mode.

23.3.2 Wake-up From Sleep Mode

GIO0 is used to wake-up from sleep mode. No special configuration is required to use GIO0 to wake-up from sleep mode. In DM270, GIO0 is dedicated to act as wakeup signal from power down mode.

Pull GIO0 low for at least the width of one clock cycle to wake-up from sleep mode. Upon wake-up from sleep mode, the PLL's start their locking process and each clock enabled by bits in MOD0 (0x30890), MOD1(0x30892) and MOD2(0x30894) become active. The PLL's will lock in 8192 cycles. Note, SLPMD bit is automatically cleared upon wakeup from sleep mode

23.4 Power Down Mode

The DM270 is said to be in power down mode when the DM270 is in sleep mode and the MXI oscillators are stopped.

23.4.1 PWDWN Pin

As shown in Table 56. , when the PWDWN pin is high (pulled-up), the oscillator connected to MXI, M48XI will stop, and the oscillator will run when PWDWN is low (pulled-down).

Note when PWDWN is low, the oscillators run even if the RESET signal is asserted.

Table 56. PWDWN pin

PWDWN state	Oscillators
Low	Running
High	Stopped

23.4.2 Entering Power Down Mode

Power Down mode can be entered by two methods:

- o Normal Power Down Mode
 - n When power down mode is to be entered. Set the Sleep Mode bit, SLPMD, in LPCTL0 (0x30896) to enter sleep mode
 - n After this, pull-up the PWDWN pin to stop the oscillators
- o Auto-power down mode,
 - n Set bit 1 (PDMD) in LPCTL1 (0x30898)
 - n When power down mode is to be entered, just pull-up the PWDWN pin. This will automatically stop PLL, clock to all modules (sleep mode). After which it stops oscillation of crystal connected to MXI, M48XI pins

23.4.3 Wake-up From Power Down Mode

To wake the device from a power down state, the following needs to be done:

- o Pull-down the PWDWN pin to start the oscillators
- o Wait appropriate time to stable the oscillator (about same time as power-on reset)
- o Pull GIO0 low for atleast one MXI clock cycle to wake-up from sleep mode
- o Wait 8192 cycles for the PLLs to lock

23.5 Disabling DAC Power

If you want to turn off the power to unused DACs, you must do as follows:

The following conditions must always be met:

- Connect VSSA and VSS3V to ground
- Connect VDD3V to power (3.3V)

The voltage of pins IREF, DAOOUT, BIAS, and VREF_RGB must satisfy following formula:

$$\text{Voltage (VDDA) } \geq \text{Voltage (IREF,DAOOUT,BIAS,VREF_RGB)}$$

Thus, IREF, BIAS, and VREF_RGB must be LOW.

Because of terminal resistance connected to VSSA, DAOOUT goes to GND. DAOOUT is an output pin, so the user need not worry about DAOOUT.

23.6 Disabling Power to SDRAM

It is possible to disable power to SDRAM. Disabling power to SDRAM can further reduce system power.

To remove power to the SDRAM:

- o Release the SDRAM bus by setting the bit SDREL in the EMIF register BUSRLS (0x30A18)
- o Turn off the SDRAM using any arbitrary GIO port

When the SDRAM bus is released, the SDRAM address, data, and control lines are all set to a high impedance state.

To power the SDRAM on:

- o Turn on the SDRAM through any arbitrary GIO port and wait until VCC is stable
- o Take the SDRAM bus out of the release state by clearing the bit SDREL in the EMIF register BUSRLS (0x30A18)
- o Perform the SDRAM initialization sequence.

23.7 Bypass bits in BYP (0x3088A)

The register BYP (0x3088A) in the Clock Controller module allows you to bypass each of the PLL for AXL, SDRAM, DSP, ARM clock. When appropriate bits in register BYP(0x3088A) are set to "1", the PLL's are bypassed. This makes the electric power consumption of PLL minimum. When a PLL is bypassed the PLL reference clock is used to drive modules in place of the PLL output. The PLL reference clock is either SYSCLK or MXI. Once a PLL is completely bypassed, power consumption can be further reduced by entering PLL powerdown mode (bit 12 in register PLLA(0x30880), PLLB(0x30882))

24 DSP Subsystem

24.1 Introduction

The DSP subsystem is based on the TMS320C5409 DSP from TI catalog. This DSP core and its peripherals are fully code-compatible with other C54x products. The DSP software development tools for the C54x are very mature and can be used to develop, compile and debug the DSP code.

The DSP subsystem includes the C54x core, several blocks of volatile memories, an imaging extension coprocessor (iMX), and a variable length coding/decoding accelerator (VLCD).

This chapter gives only an overview of the DSP core and focuses more on its peripherals. For detailed information on the architecture of this 16-bit, DSP core and/or on the associated tools, refer to the C54x user's guides and tools documentation.

To realize image processing, image compression and voice processing, the DSP controls SDRAM DMA transfers to/from external memory and the iMX and VLCD accelerators. Since programming of the DSP is flexible and easy, users can improve the image processing pipeline and add differentiated functions to a system.

24.2 DSP core

C54x devices are fixed-point digital signal processors (DSPs) from the Texas Instruments TMS320 family. The C54x CPU, with its advanced modified Harvard architecture, features minimized power consumption and a high degree of parallelism.

This processor has one program memory bus and three data memory buses. It also provides an arithmetic logic unit (ALU) that has a high degree of parallelism, application-specific hardware logic, on-chip memory, and additional on-chip peripherals. The basis of the operational flexibility and speed of this DSP is a highly specialized instruction set.

Separate program and data spaces allow simultaneous access to program instructions and data, providing the high degree of parallelism. Two read operations and one write operation can be performed in a single cycle. Instructions with parallel store and application-specific instructions can fully utilize this architecture. In addition, data can be transferred between data and program spaces. Such parallelism supports a powerful set of arithmetic, logic, and bit manipulation operations that can all be performed in a single machine cycle. In addition, this processor includes the control mechanisms to manage interrupts, repeated operations, and function calls.

24.2.1 Features

- o Low-power C54x DSP CPU
- o Software-programmable wait-state generator with bank-switching wait state logic
- o External memory interface
- o Program space
- o Data space
- o I/O space
- o Scan-based emulation logic

In the DM270, the wait state generator and the external memory interface are used to connect the DSP core to the expansion memory of the DSP subsystem and to the image buffers.

The maximum DSP cycle frequency is programmable up to 100 MHz (refer to the DM270 data sheet for the exact value).

24.3 DSP peripherals

The DSP CPU core is associated with an HPI interface, an interrupt handler, a parallel XIO interface, two multi-channel buffered serial ports (McBSPs), and a JTAG interface.

The JTAG interface is connected to the JTAG interface of the ARM core through some logic.

The parallel interface is used to connect the core to the expanded memory, the shared blocks of memory, but as well, this interface is used to control the VLCD engine and iMX coprocessor. Refer to the appropriate chapter to get more details on each one of these points.

The host port interface (HPI) of the DSP is connected to the DSP Controller of the ARM subsystem. This connection cannot be modified. The HPI cannot serve any other purpose than to communicate back and forth with the ARM. The HPI bridge is described in the ARM-DSP Communication chapter.

The DSP core features also a DMA controller that can be used to transfer data between the image buffers, the expanded memory block, the internal memory and the McBSPs.

The PLL that can normally be found on a standard C54x device has been removed. The DSP PLL is now part of the ARM Subsystem clock controller module and fully controlled by the ARM.

The only peripherals that can be used by the DSP to communicate directly with external devices to the DM270 are the two multi-channel buffered serial ports and the XF pin. The DSP can control the XF signal by writing into the proper

memory map register. This signal is multiplexed with the ARM GIO pin. This signal can be read as well by the ARM through the DSP controller MMRs.

24.3.1 Multi-channel buffered serial ports

The DM270 has two high-speed, full-duplex multi-channel buffered serial ports (McBSPs), one of which (McBSP1) has its pins multiplexed with the ARM GIO pins (Refer GIO section for details). The McBSP ports allow direct interface to other C54x devices, audio codecs, and other devices in a system. The McBSPs used in the DSP are based on the standard serial port interface found on other TMS320C54x devices. The two McBSPs that are accessible to the DSP are named McBSP0 and McBSP1.

A brief description of the McBSP included in the DM270's DSP core is given in this section. For detailed information on the DSP's McBSP, refer to the C54x peripheral manual; The *C54x reference set volume 1 (spru131)*.

The McBSPs provide:

- o Full-duplex communication
- o Double-buffered data registers that allow a continuous data stream
- o Independent framing and clocking for receive and transmit

24.3.1.1 Capabilities

In addition, the McBSPs have the following capabilities:

- o Direct interface to:
 - n T1/E1 framers
 - n MVIP switching-compatible and ST-BUS-compliant device
 - n IOM-2-compliant devices
 - n AC97-compliant devices
 - n Some IIS-compliant devices
 - n Serial peripheral interface (SPI) devices
- o Multichannel transmit and receive of up to 32 channels
- o Direct interface to industry-standard codecs, analog interface chips (AICs), and other serially connected A/D and D/A devices
- o A wide selection of data sizes, including 8, 12, 16, 20, 24, and 32 bits
- o μ -law and A-law companding
- o External shift clock or an internal, programmable-frequency shift clock for data transfer
- o Autobuffering capability through the six-channel DMA controller

- Programmable polarity for both frame synchronization and data clocks

24.3.1.2 External Interface

The McBSPs consist of separate transmit and receive channels that operate independently. The external interface of each McBSP consists of the following pins:

- BCLKX: Transmit reference clock
- BDY: Transmit data
- BFSX: Transmit frame synchronization
- BCLKR: Receive reference clock
- BDR: Receive data
- BFSR: Receive frame synchronization
- CLKS: External clock reference for the programmable clock generator

24.3.1.3 Transmitter/Receiver Pins

The seven pins listed are functionally equivalent to the pins of previous serial port interface pins in the C54x generation of DSPs. On the transmitter, transmit frame synchronization and clocking are indicated by the BFSX and BCLKX pins, respectively. The CPU or DMA can initiate transmission of data by writing to the data transmit register (DXR). Data written to DXR is shifted out on the BDY pin through a transmit shift register (XSR). This structure allows DXR to be loaded with the next word to be sent while the transmission of the current word is in progress.

On the receiver, receive frame synchronization and clocking are indicated by the BFSR and BCLKR pins, respectively. The CPU or DMA can read received data from the data receive register (DRR). Data received on the BDR pin is shifted into a receive shift register (RSR) and then buffered in the receive buffer register (RBR). If the DRR is empty, the RBR contents are copied into the DRR. If not, the RBR holds the data until the DRR is available. This structure allows storage of the two previous words while the reception of the current word is in progress.

24.3.1.4 Data Movement

The CPU and DMA can move data to and from the McBSPs and can synchronize transfers based on McBSP interrupts, event signals, and status flags. The DMA is capable of handling data movement between the McBSPs and memory with no intervention from the CPU.

24.3.1.5 Programmable Functions

In addition to the standard serial port functions, the McBSP provides programmable clock and frame sync generation. Among the programmable functions are:

- Frame synchronization pulse width
- Frame period
- Frame synchronization delay
- Clock reference (internal vs. external)
- Clock division
- Clock and frame sync polarity

The on-chip companding hardware allows compression and expansion of data in either μ -law or A-law format. When companding is used, transmit data is encoded according to specified companding law and received data is decoded to 2s-complement format.

24.3.1.6 Multiple Channel Selection

The McBSPs allow multiple channels to be independently selected for the transmitter and receiver. When multiple channels are selected, each frame represents a time-division multiplexed (TDM) data stream. In using TDM data streams, the CPU may only need to process a few of them. Thus, to save memory and bus bandwidth, multichannel selection allows independent enabling of particular channels for transmission and reception. Up to 32 channels can be enabled.

24.3.1.7 Clock-Stop Mode

The clock-stop mode (CLKSTP) in the McBSP provides compatibility with the serial peripheral interface (SPI) protocol. Clock-stop mode works with only single-phase frames and one word per frame. The word sizes supported by the McBSP are programmable for 8-, 12-, 16-, 20-, 24-, or 32-bit operation. When the McBSP is configured to operate in SPI mode, both the transmitter and the receiver operate together as a master or as a slave.

The McBSP is fully static and operates at arbitrarily low clock frequencies. The maximum frequency is CPU clock frequency divided by two.

24.3.1.8 Data Clock Generation

You can select from a variety of data bit-clocks independently for the receiver and transmitter. These options include:

- The input clock to the sample rate generator can be either the CPU clock or an external clock input (CLKS).
- The input clock (CPU clock or external clock CLKS) source to the sample rate generator can be divided down by a programmable value.

24.3.2 Hardware Timer

The DSP subsystem features one independent software-programmable timer. Three memory-mapped registers (MMRs) control the operation of the timer. These registers are: the timer control register (TCR), the timer period register (PRD), and the timer counter register (TIM). The timer resolution is the CPU clock rate of the DSP. The timer has a 20-bit dynamic range. The timer consists of a programmable 16-bit main counter (TIM), and a programmable 4-bit prescaler.

The main counter is driven by the 4-bit prescaler, which decrements by one at every CPU clock. Once the prescaler reaches zero, the 16-bit counter decrements by one. When the 16-bit counter decrements to zero, a maskable interrupt (TINT) is generated, and the counter is reloaded with the period value defined in the PRD. The timer can be stopped, restarted, reset, or disabled via the bits of the timer control register (TCR).

Refer to the C54x reference set volume 1 (spru131) document for more information.

24.3.3 Direct Memory Access Controller (DMAC)

A brief description of the DMAC included in the DM270's DSP core is given in this section. For detailed information on the DSP's DMAC, refer to the C54x datasheet, *TMS320VC5409 Fixed-Point Digital Signal Processor (SPRS082C)*.

24.3.3.1 Features

The DSP subsystem includes a six-channel DMA controller, for performing data transfers independent of the CPU.

This DMA controller is separate from the HPIB DMA Controller and Image Buffer DMA controller. Refer to "ARM-DSP Communication section" and "DSP Coprocessor subsystem" respectively for details about these DMA controllers

The DMA Controller includes the following features:

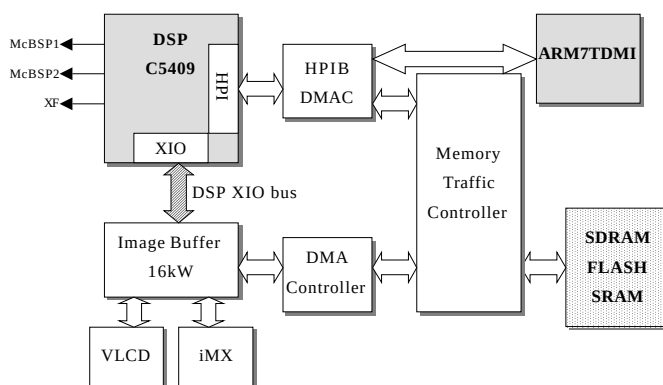
- Operates independently of the CPU
- Six independent channels (DMA controller can keep track of the context of six independent block transfers)
- Higher priority for memory accesses than CPU
- Each channel has an independently programmable priority level
- Each channel's source and destination address registers include configurable indexing modes. The address can remain constant, be post-incremented/decremented, or be adjusted by a programmable value.

- Each read or write transfer may be initialized by selected events (internally only)
- Each DMA channel can interrupt the CPU upon completion of a half or entire block transfer
- Supports double word transfers; i.e., a 32-bit transfer of two 16-bit words
- The DSP DMA supports the following data transfers:
DARAM $\longleftrightarrow$ DARAM
SARAM(data) $\longleftrightarrow$ DARAM
SARAM(data) $\longleftrightarrow$ SARAM (data)
DARAM $\longleftrightarrow$ IO
SARAM(data) $\longleftrightarrow$ IO
Note: DARAM is 0x80 to 0x7FFF, SARAM(data) is 0x8000 to 0xBFFF
(See also Figure 172 for DSP Memory Map)

24.4 DSP subsystem memory

Figure 171 shows the DSP subsystem. Shared memory for image processing (Image Buffer, 16 KW) is connected to the DSP expansion memory bus (XIO 16-bit data bus). The DSP has 64 KW of DSP program and data space memory. In addition to the DSP, the VLCD, iMX, image processing hardware accelerator, and DMA controller (in the Image Buffer module) can access shared memory (Image Buffer). Since the shared buffer is mapped to DSP memory addresses, it can be accessed directly. DSP memory is constructed from three independent spaces: program, data, and I/O. Control and configuration registers for the Image Buffer, iMX and VLCD are mapped to I/O space.

Figure 171. DSP subsystem block diagram



24.4.1 Program data and IO spaces

The DSP memory is organized into three individually selectable spaces: program, data, and I/O. Within any of these spaces, DARAM, ROM, SARAM, or memory-mapped peripherals can reside either within the DSP core or off the core.

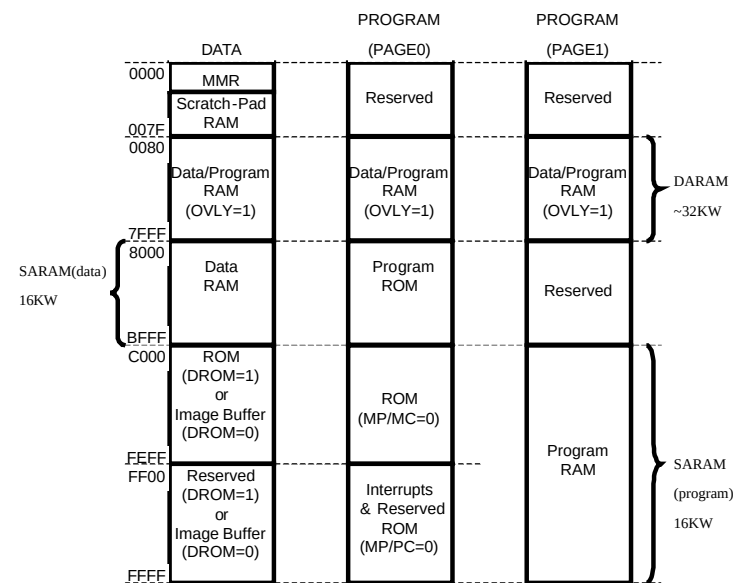
The program memory space contains the instructions to execute, as well as tables used in execution. The data-memory space stores data used by instructions. The I/O memory space interfaces to the iMX, VLCD engines and to the image buffer controller.

There are three CPU status register bits that affect memory configuration. The MP/MC, OVLY, and DROM bits are located in the processor mode status register (PMST).

24.4.2 DSP Memory map

The DSP subsystem offers 32 K words (1 word = 16 bits) of DSP internal DARAM memory, 32 K words of DSP internal SARAM memory, 32 K words of ROM memory, and 16 K words of shared RAM memory. Figure 172 shows the DSP memory map after reset. The program space and the data memory spaces span the addresses 0x0h to 0x1FFFFh. Each block of memory has a width of 16 bits. Some blocks are available for both program and data spaces, such as the DARAM block when OVLY=1.

Figure 172. DSP Memory Map



Note: Figure is not to scale

24.4.3 I/O memory space

The C54x devices offer an I/O-memory space in addition to the program memory and data-memory spaces. The I/O-memory space is a 64K-word address space (0000h–FFFFh) and exists only external to the DSP device. Two instructions, PORTR and PORTW, are used to access this space. Read timings vary from those of the program-memory and data-memory spaces to facilitate access to individual I/O-mapped devices rather than to memories.

On the DM270, only some addresses from this space are used. Image Buffer registers, iMX registers, and VLCD registers are mapped to I/O spaces. Signals to control the Image buffer clock, iMX buffer clock, and VLCD buffer clock are mapped to an I/O space. A signal to interrupt the ARM from the DSP is also mapped to an I/O space. Table 57. shows the address allocation of each module.

Table 57. I/O space address map

I/O address	Module name
0x0000h-0x0009h	Image Buffer
0x0010h-0x0017h	iMX
0x0020h-0x005Fh	VLCD
0x0080h-0x0087h	HPIB
0xFFFFh	CLOCK Control
0xFFFFh	DSP to ARM Interrupt

24.5 DSP internal memory

24.5.1 Organization of the memory

The DSP subsystem features 32K x 16 bits of on-chip DARAM as well as **32K x 16 bits of on-chip SARAM**.

The internal memory configuration is controlled by the OVLY bit. After a reset, this bit is set to '1'. In this mode, the DARAM memory block is shared between data and program space.

It is possible to change this bit to '0' by software, however, it is not recommended, since the internal memory will not be available anymore to store program data.

The DSP internal memory can be accessed by the ARM through the HPI bridge. This capability is explained in greater detail in the ARM-DSP Communication chapter.

24.5.2 DARAM properties

The internal memory of the DSP is built from 4 blocks of 8K x 16-bit DARAM. The DSP CPU can perform two accesses to a DARAM in one machine cycle (two reads in one cycle, or a read and a write in one cycle). It can also perform multiple accesses to separate memory blocks in one machine cycle.

The figure below shows the RAM block organization for the DM270 DSP DARAM memory. The gray lines in the figure indicate block boundaries. The organization of the first 1K of DARAM includes the memory-mapped CPU and peripheral registers, 32 words of scratch-pad DARAM, and 896 words of DARAM.

Figure 173. Internal DSP DARAM block organization

0000-007F
0080-1FFF
2000-3FFF
4000-5FFF
6000-7FFF

24.5.3 SARAM Properties

DM270 DSP subsystem contains 16KW SARAM for data (0x8000 to 0xBFFF) and 16KW SARAM for program (Page 1: 0x1C000 to 0x1FFFF).

No wait state is required to access SARAM. However when there is conflict during access of memory location in SARAM, wait state is automatically inserted by the DSP.

24.5.4 Access of DSP internal memory from ARM side

The ARM can access DSP internal memory (32 KW) through the DSP HPI port. The HPI operates in 16-bit non-multiplexed mode for this purpose. The C54x in the DM270 does not support 8-bit HPI modes. Refer to "ARM-DSP Communication" chapter for details.

24.5.5 DMA from DSP internal Memory to SDRAM/External Memory

DM270 DSP subsystem supports DMA transfer between DSP internal memory and SDRAM/external memory. Refer to "ARM-DSP Communication" chapter for details.

24.5.6 Reset and relocatable interrupt vector table

Synchronization between DSP and DSP peripheral modules (Image Buffer, iMX, and VLCD) and the ARM is performed using interrupts. When the operation of each DSP peripheral module ends, an interrupt is generated to the DSP. The ARM can interrupt the DSP at any time (provided this interrupt is enabled by the DSP). The interrupts of the DSP interrupt ports are as shown in the table below:

Table 58. DSP interrupt ports

Port name	Module name
INT0	ARM
INT1	iMX
INT2	VLCD
INT3	SDRAM-DMA Controller (Image Buffer)

The reset, interrupt, and trap vectors are addressed in program space. These vectors are soft—meaning that the processor, when taking the trap, loads the program counter (PC) with the trap address and executes the code at the vector.

location. Four words are reserved at each vector location to accommodate a delayed branch instruction—either two 1-word instructions or one 2-word instruction—which allows branching to the appropriate interrupt service routine with minimal overhead. For more information refer to the datasheet, *TMS320VC5409 Fixed-Point Digital Signal Processor (SPRS082C)*.

At device reset, the reset, interrupt, and trap vectors are mapped to address FF80h in ROM space. However, these vectors can be remapped to the beginning of any 128-word page in program space after device reset. This is done by loading the interrupt vector pointer (IPTR) bits in the PMST register with the appropriate 128-word page boundary address. The ROM based DSP code change the IPTR value for the interrupt vector table to be located at address 0x7F80h.

24.5.7 Memory mapped registers and scratch pad RAM

Most of the DSP control registers are located within the address range 0x00 and 0x5F in the data space of the DSP subsystem memory map. The first 20h MMRs control the CPU, the MMRs after 20h are used to control the DSP peripherals. The registers that control the iMX, VLCD and image buffer are mapped into the IO space and do not appear in the table below.

The McBSPs need more than two registers each to be controlled. To workaround the limited space in the MMR area, a sub-bank scheme is used to control the McBSP. This is described with further details in the *TMS320C54x DSP reference set vol5 - Enhanced Peripherals (spru 302)*.

The tables below give the name and a short description of the different memory mapped registers.

Table 59. CPU Memory-Mapped Registers

Address (Hex)	Name	Description
0	IMR	Interrupt mask register
1	IFR	Interrupt flag register
2–5	–	Reserved for testing
6	ST0	Status register 0
7	ST1	Status register 1
8	AL	Accumulator A low word (bits 15–0)
9	AH	Accumulator A high word (bits 31–16)
A	AG	Accumulator A guard bits (bits 39–32)
B	BL	Accumulator B low word (bits 15–0)
C	BH	Accumulator B high word (bits 31–16)
D	BG	Accumulator B guard bits (bits 39–32)
E	T	Temporary register
F	TRN	Transition register
10	AR0	Auxiliary register 0
11	AR1	Auxiliary register 1
12	AR2	Auxiliary register 2
13	AR3	Auxiliary register 3
14	AR4	Auxiliary register 4
15	AR5	Auxiliary register 5
16	AR6	Auxiliary register 6
17	AR7	Auxiliary register 7
18	SP	Stack pointer
19	BK	Circular-buffer size register

1A	BRC	Block-repeat counter
1B	RSA	Block-repeat start address
1C	REA	Block-repeat end address
1D	PMST	Processor mode status register
1E	XPC	Program counter extension register
1E–1F	–	Reserved

Table 60. DSP Peripheral Memory-Mapped Registers

Register	Address	Description	Type
DDR20	20h	Data receive register 2	McBSP #0
DDR10	21h	Data receive register 1	McBSP #0
DXR20	22h	Data transmit register 2	McBSP #0
DXR10	23h	Data transmit register 1	McBSP #0
TIM	24h	Timer register	Timer
PRD	25h	Timer period counter	Timer
TCR	26h	Timer control register	Timer
–	27h	Reserved	
SWWSR	28h	Software wait -state register	External Bus
BSCR	29h	Bank-switching control register	External Bus
–	2Ah	Reserved	
SWCR	2Bh	Software wait -state control register	External Bus
HPIC	2Ch	HPI control register	HPI
–	2Dh–37h	Reserved	
SPSA0	38h	McBSP0 subbank address register	McBSP #0
SPSD0	39h	McBSP0 subbank data register	McBSP #0
–	3Ah–3Fh	Reserved	
DDR21	40h	Data receive register 2	McBSP #1
DDR11	41h	Data receive register 1	McBSP #1
DXR21	42h	Data transmit register 2	McBSP #1
DXR11	43h	Data transmit register 1	McBSP #1
–	44h–47h	Reserved	
SPSA1	48h	McBSP1 subbank address register	McBSP #1
SPSD1	49h	McBSP1 subbank data register	McBSP #1
–	4Ah–53h	Reserved	
DMPREC	54h	DMA channel priority and enable control register	DMA
DMSA	55h	DMA subbank address register	DMA
DMSDI	56h	DMA subbank data register with autoincrement	DMA
DMSDN	57h	DMA subbank data register	DMA
–	58h–5Fh	Reserved	

The advantage of using the scratch pad RAM that is located between address 0x60 and 0x7F is that it can be accessed anytime through an immediate memory access regardless of the data page pointer value. Some special assembly instruction like STM let you do this. Additionally, since writing to control registers can incur latencies, instructions like STM operate early and avoid most pipeline problems.

24.5.8 ROM usage

Please contact TI for ROM details

24.5.9 Access properties

When shared memory is accessed, a 1-cycle WAIT must be inserted.

No wait state is required for the other DSP subsystem memories : DARAM, SARAM(data), SARAM(program), ROM

Since the DSP has programmable software WAIT generator, software WAIT can be generated by setting the number of WAITs in the WAIT control register (SWWSR register) in the DSP.

Example: SWWSR = 0x0200h

24.6 DSP - Power Saving Modes

24.6.1 DSP Idle modes

The C54x DSP embedded in the DM270 has power-down modes in which it enters a dormant state and dissipates less power than normal operation while maintaining the CPU contents. This allows operations to continue unaltered when the power-down mode is terminated.

You can invoke one of the power-down modes either by executing the IDLE 1 IDLE 2 instructions, or by using the ARM to drive the HOLD signal low with the HM status bit set to 1.

Power-down operation is summarized in the table below and described in detail in the following sub-sections

Table 61. Operation During the Three Power-Down Modes

Operation/Feature	IDLE1	IDLE2	HOLD
CPU halted	Yes	Yes	Yes
CPU clock stopped	Yes	Yes	No
Peripheral clock stopped	No	Yes	No
Power-down terminated by:			
HOLD driven high	No	No	Yes
Unmasked internal hardware interrupts	Yes	No	No
Unmasked interrupts from ARM/IMX/VLC/DMA	Yes	Yes	No
RS signal controlled by ARM DSP Controller module	Yes	Yes	No

Depending on the state of the HM bit, the CPU continues to execute unless the execution requires an external memory access.

24.6.1.1 IDLE1 Mode

The IDLE1 mode halts all CPU activities except the system clock. Because the system clock remains applied to the peripheral modules, the peripheral circuits continue operating and the CLKOUT pin remains active. Thus, peripherals such as McBSP and timers can take the CPU out of its power-down state.

Use the IDLE 1 instruction to enter the IDLE1 mode. To terminate IDLE1, use a wake-up interrupt. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE1 is terminated. If INTM = 1, the C54x DSP continues with the instruction following the IDLE 1 instruction. All wake-up interrupts must set to enable the corresponding bits in the IMR register regardless of the INTM value. The only exceptions are the nonmaskable interrupts, RS and NMI.

24.6.1.2 IDLE2 Mode

The IDLE2 mode halts the on-chip peripherals as well as the CPU. Because the on-chip peripherals are stopped in this mode, they cannot be used to generate the interrupt to wake up the DSP as with IDLE1. However, power is significantly reduced because the device is completely stopped.

Use the IDLE 2 instruction to enter the IDLE2 mode. To terminate IDLE2, activate any of the external interrupt signals (RS, NMI, and INTO) with a 10-ns minimum pulse. If INTM = 0 when the wake-up interrupt takes place, the C54x DSP enters the ISR when IDLE2 is terminated. If INTM = 1, the C54x DSP continues with the instruction following IDLE 2 instruction. All wake-up interrupts must be set to enable the corresponding bits in the IMR register regardless of the INTM value. Reset all peripherals when IDLE2 terminates, especially the McBSP when the CLKS pin is used.

When RS is the wake-up interrupt in IDLE2, a 10-ns minimum pulse of RS can activate the reset sequence.

24.6.1.3 Hold Mode

The Hold mode is another power-down mode. Depending on the value of the HM bit (register ST1), you can use this mode to halt the CPU.

This power-down mode is initiated by the HOLD signal. This signal is controlled by the ARM within the DSP controller module. The effect of HOLD depends on the value of HM. If HM = 1, the CPU stops executing. If HM = 0, the CPU continues to execute internally. You can use HM = 0 with the HOLD signal when your system does not require the use of iMX or VLCD co-processors. The C54x DSP continues to operate normally unless an off-chip access is required by an instruction; then the processor halts until HOLD is released.

This mode does not stop the operation of on-chip peripherals (such as timers and serial ports); they continue to operate regardless of the HOLD level or the condition of the HM bit.

This mode is terminated when HOLD becomes inactive.

24.6.2 Other Power-Down Capability

Switching the internal CLKOUT signal can reduce the DM270 DSP core power consumption. This internal signal is coming out of the DSP core but it is not used within the DM270 chip.

The CLKOUT off function allows the C54x DSP to disable CLKOUT using software instructions. The CLKOFF bit of PMST determines whether CLKOUT is enabled or disabled. At reset, CLKOUT is enabled. To gain a little bit on power consumption, you can disable this signal.

25 DSP Coprocessor Subsystem

25.1 Introduction to coprocessor subsystem

The coprocessor subsystem is part of the DSP subsystem. It is designed to enhance DSP's capability in image processing and in other computation-intensive applications. The coprocessor subsystem consists of the following components:

- o Coprocessors (iMX and VLCD)
- o DMA controller (built into the Image Buffer module)
- o Shared memory
- o Shared memory access control
- o The DSP's Interrupt controller
- o Clock and coprocessor manager

All components reside in DSP's data page and/or I/O space, and all components are controlled by the DSP. Figure 174 shows the architecture of the Coprocessor Subsystem. There are two coprocessors in the system, Image Extension (iMX) and Variable Length Coder/Decoder (VLCD). The iMX accelerates computation-intensive imaging processing functions such as filtering, interpolation and DCT/IDCT. The VLCD provides acceleration for Huffman coding and encoding as well as quantization and inverse quantization required in various still image and motion picture compression standards.

The DMA controller provides a pathway to the ARM subsystem's SDRAM, allowing high-speed data transfer between the ARM subsystem and the DSP Subsystem. The DMA controller is part of the image buffer module. The DMA controller referred to in this section is the DMA controller built into the image buffer module not the DSP's DMA controller. However, the DSP's DMA controller can be used to transfer data between the DSP and each of the shared memories. The DSP's DMA controller cannot be used to transfer data to SDRAM, this can be done by the image buffers DMA controller or HPIB DMA controller (See section titled "HPIB").

Each coprocessor and the DMA controller have their own private memory to store data structures and instructions unique to each functional unit. In addition, the DSP, iMX, VLCD and DMA controller also communicate to each other in a cooperative manner in order to perform useful functions. The communication is made possible by a shared memory, shared memory access control, and DSP interrupts.

Figure 175 shows the shared memory blocks. There are three types of coprocessor memory: Image buffers, iMX coefficient/command buffer, and VLCD table buffer. Image buffers consist of Buffer A and Buffer B. Buffer A and Buffer B memory are shared among the coprocessors and the DMA controller. The iMX coefficient/command buffer is used to store the coefficients and

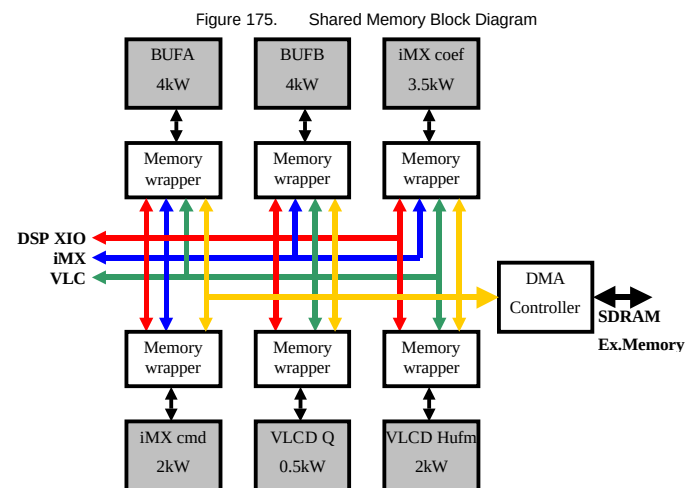
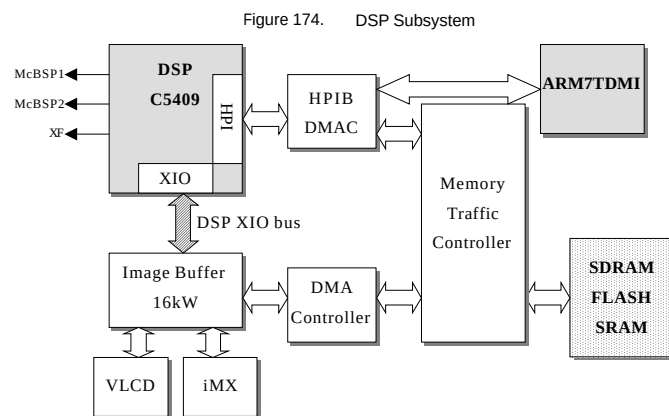
command for the iMX. The VLCD buffer is used to store Huffman tables, quantization coefficients, and other data structures for the VLCD.

NOTE: The user needs to configure the device to add one wait state when accessing these memory blocks (Image Buffer A, Image Buffer B, iMX coefficient/command buffer, and VLCD table buffer). No wait state is required to access the control registers in the I/O space.

The DSP software through registers in the Image Buffer module controls shared memory access. When a coprocessor, DMA controller, or DSP is given access to a shared memory buffer that module has a dedicated link to the buffer.

Each coprocessor and the image buffer DMA controller can interrupt the DSP.

The Clock and Coprocessor Manager manages the clocks and provides top-level control to coprocessors and other functional units in the Coprocessor Subsystem.



BUFA and BUFB are used as the working memory for the image processing, and are shared either by DSP, iMX or VLCD. The memory wrapper is doing this switch logic for the shared memory. Note: a buffer can't be shared by all of access unit at same time, and access switch is controlled by DSP.

iMX coefficient memory is used as the coefficient and temporary working space for iMX. DSP needs to initialize the filter coefficients into this memory. The buffer switch is controlled by DSP.

iMX command memory is used as the program command memory for the iMX. Before iMX is active, DSP needs to download the program into this memory. The buffer switch is control by DSP.

VLCD quantization table memory is used as the quantization table for the VLCD engine. Table can be updated by DSP. The buffer switch is control by DSP.

VLCD Huffman table memory is used as the Huffman table for the VLCD engine. Table can be updated by DSP. The buffer switch is control by DSP.

The image buffer module manages access to each of the shared memory blocks and manages DMA transfers from the image buffers to SDRAM or External Memory.

25.4 Shared Memory Access Switching and Memory Map

Table 63. shows which modules can access each buffer. The addresses, in the table, correspond to the DSP memory map addresses. The DSP controls which module has access to each buffer by configuring the [BUF_CTRL \(0x08\)](#). [BUF_CTRL \(0x08\)](#) is mapped to the DSP I/O space and is part of the image buffer hardware module. The DSP can have access to both Image Buffers, simultaneously. However, the iMX, VLCD, and DMA controller should only be connected to a single Image Buffer at a time (see also 25.5.4 Capabilities and Restrictions). It takes one DSP cycle to switch the access of a buffer from one module to another.

Table 63. Shared Memory Access Control

Memory	Access module				DSP Address	SIZE (WORDS)
	DSP	iMX	VLCD	DMAC		
Image BUFA	X	X	X	X	C000H-CFFFH	4k
Image BUFB	X	X	X	X	D000H-DFFFH	4k
iMX coefficient	X	X	X	X	E000H-EDFFFH	3.5k
iMX command	X	X	-	X	EE00H-F5FFFH	2k
VLCD Q table	X	-	X	X	F600H-F7FFFH	512
VLCD Huffman	X	-	X	X	F800H-FFFFFH	2k

25.5 Image Buffer DMA Controller

The Image Buffer module has a DMA controller that allows the DSP to initiate data transfers between the image buffers (Buffer A and Buffer B) and SDRAM or external memory or between the iMX coefficient buffer and SDRAM or external memory. It can also transfer the data to/from iMX cmd, VLCD Q and VLCD Huffman memory. The DMA controller will handle the handshaking sequence to/from SDRAM or the External Memory controller automatically.

The DMA controller supports 2-dimensional data transfers. As shown in Figure 176, [DMA_XNUM](#) specifies the number of pixels in the horizontal direction, and [DMA_YNUM](#) specifies the number of lines in the vertical direction. By setting each memory lead address ([BUF_ADDR](#), [SDR_ADDR](#)) and line offset value ([BUF_LOFST](#), [SDR_LOFST](#)), block transfer of second-order image data is possible. This function is effective in cases where processing in MACRO block units is required, such as in JPEG compression.

Also, when the line offset value ([BUF_LOFST](#), or [SDR_LOFST](#)) is less than the value of [DMA_XNUM](#), it is overwritten as shown in Figure 177.

Figure 176. 2-D Data Transfer using Image Buffer DMA Controller

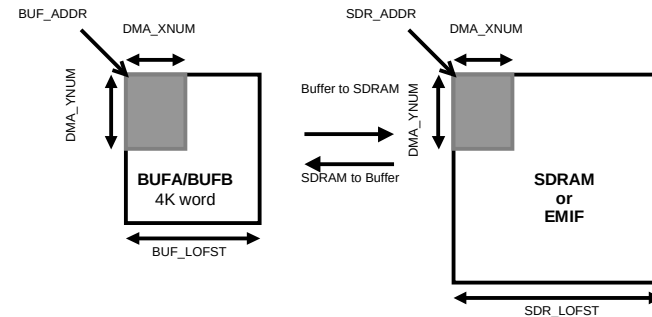
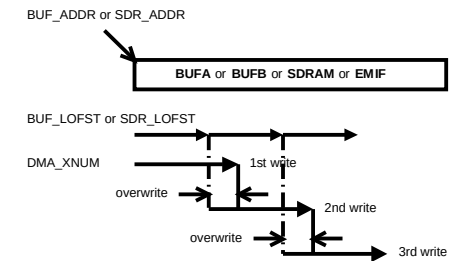


Figure 177. Data overwrite when LOFST < DMA_XNUM



25.5.1 Interrupting the DSP after DMA Transfer Completion

When the DMA completes, the DMA controller can interrupt the DSP. The DMA controller is connected to the DSP's INT3. The DSP can enable this interrupt (through interrupt mask register IMR) so that an interrupt service routine (ISR) is called upon transfer completion. Alternatively, the DSP can disable this interrupt and poll the bit 1 in the [DMA_CTRL \(0x07\)](#) register.

25.5.2 Byte to Word Conversion

The Image buffer DMA controller also has a function mode of byte-to-word conversion. In this mode, the 32bit(4 bytes) read data from SDRAM is converted

to 64bit(4 words), and stored into image buffer. Table 64. shows data format in image buffer when byte –to-word conversion is enabled. Byte to word conversion can be enabled using WORD field in [IMG_MODE \(0x9\)](#)

Table 64. Byte to word conversion

SDRAM read data	0(LSB)				31(MSB)			
	byte0	byte1	byte2	byte3				
Image Buffer Address	4n (word)				4n+1			
Write data	byte0	0	byte1	0	byte2	0	byte3	0

25.5.3 Steps to Configure the DMA Controller

In general, to transfer data using the DMA controller, the DSP should setup the DMA transfer registers as follows:

- o Perform the following steps depending on the SDRAM or EMIF data source/destination device:
 - n To transfer data between SDRAM, set the SDRAM source/destination address in [SDR_ADDRH \(0x00\)](#) and [SDR_ADDRL \(0x01\)](#)
 - n To transfer data between CS0, CS3, or CS4; set the EMIF destination/source address in [SDR_ADDRH \(0x00\)](#) and [SDR_ADDRL \(0x01\)](#)
 - n To transfer data between CS1 or CS2, set the EMIF destination/source address in the EMIF module registers IMGDSPADDH (0x30A46) and IMGDSPADDL (0x30A48).
 - n For EMIF data transfer, select the EMIF source/destination region (CS0/CS1/CS2/CS3/CS4) by configuring IDST in IMGDSPDEST (0x30A34)
 - n Select whether SDRAM or EMIF would be used for data transfer using DESTSEL bit in [DMA_CTRL \(0x7\)](#)
 - n NOTE: SDRAM, EMIF address is specified as word offset from start of respective SDRAM, EMIF region. See EMIF section for EMIF register details
- o Set the SDRAM data offset in [SDR_LOFST \(0x02\)](#)
- o Set the shared memory source/destination address for BUFA/BUFB or iMX coefficient memory or VLCD Huff memory or VLCD Q memory using [BUF_ADDR \(0x03\)](#). Address is specified as offset from start of respective memory. (NOTE: The shared memory to be used in DMA transfer is selected using [BUF_CTRL\(0x8\)](#))
- o Set the shared memory buffer data offset in [BUF_LOFST\(0x04\)](#)

- o Set the access block size in the X and Y directions in [DMA_XNUM \(0x05\)](#) and [DMA_YNUM \(0x06\)](#)
- o Set the bits in [IMG_MODE \(0x9\)](#) to specify whether you want data byte swapping, data shifting and/or byte-to-word executed during the DMA transfer.
- o Select the direction of the DMA transfer by configuring bit 0 of [DMA_CTRL \(0x07\)](#). Switch the appropriate shared memory to DMA controller using [BUF_CTRL \(0x8\)](#)
- o Initiate the DMA request by writing a "1" to bit 1 of [DMA_CTRL \(0x07\)](#)
- o Check for DMA completion by either using bit 1 of [DMA_CTRL \(0x7\)](#) or DSP INT3 interrupt

25.5.4 Capabilities and Restrictions

The DMA controller has certain capabilities and restrictions:

- o When the DMA controller writes data to SDRAM/EMIF, the DMA controller can only access one shared memory (image buffer A, image buffer B, iMX coefficient memory, VLCD huff memory or VLCD Q memory). Use [BUF_CTRL\(0x8\)](#) for buffer access control
- o When the DMA controller reads data from SDRAM/EMIF, the DMA controller can access both image buffer A and B at the same time. In this case the same data would be stored at the same address offset location, in both image buffer A and B. Use [BUF_CTRL\(0x8\)](#) for buffer access control
- o DMA transfers are most efficient when done in units of SDRAM/EMIF bursts. A burst consists of 16 word units. It is possible to transfer data in word units, however, the timing will be equal to the timing required for a burst transfer. For example, you can transfer one word from image buffer A to SDRAM, however, the time to complete this transfer will be equal to the time required to transfer one burst (16 words).
- o DMA transfers are most efficient when the starting SDRAM/EMIF address is a 256-bit (256-bit = 16 words) burst address. When the starting address is not a 256-bit burst address, then the DMA controller requires up to 2 (1 additional) burst transfer to account for the unaligned data.
- o The DMA controller supports 2-dimensional SDRAM/EMIF accesses. The DMA controller can read an NxM block of data out of a larger block of data, however, the transfer is most efficient when the block is a multiple of 256-bits wide. (see also Figure 176)

25.7.2 SDR_ADDRL - ImgBuf SDRAM Address Low Register

[illegible][illegible]

25.7.3 SDR_LOFST - ImgBuf SDRAM Offset Register

[illegible][illegible]

25.7.4 BUF_ADDR - ImgBuf Buffer Address Register

[illegible][illegible]

25.7.5 BUF_LOFST - ImgBuf Buffer Offset Register

[illegible][illegible]

25.7.6 DMA_XNUM - ImgBuf DMA X Direction Transfer Number Register

[illegible][illegible]

25.7.7 DMA_YNUM - ImgBuf DMA Y Direction Transfer Number Register

[illegible][illegible]

25.7.8 DMA_CTRL - ImgBuf DMA Control Register

DMA_CTRL		IO 0007 offset: 0x0E								default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DESTSEL	RSV	RSV	RSV	RSV	RSV	RSV	DMAEN	IMG2MEM	
R/W	-	-	-	-	-	-	-	R/W	-	-	-	-	-	-	R/W	R/W	
Default	-	-	-	-	-	-	-	0	-	-	-	-	-	-	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	DESTSEL	0	R/W	<u>DMA Destination Select</u> » 0: SDRAM « » 1: External Memory IF «
7:2	RSV			Reserved
				<u>DMA Request</u> 1 : DMA go
1	DMAON	0	R/W	Writing "1" initiates the DMA transfer is in progress. The request will be cleared automatically when the DMA transfer completes.
0	IMG2MEM	0	R/W	<u>Read or Write</u> » 0: Image Buffer to SDRAM/External Memory « » 1: SDRAM/External Memory to Image Buffer «

25.7.9 BUF_CTRL - ImgBuf Buffer Control Register

BUF_CTRL		IO 0008				offset: 0x10								default: 0x0000			
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	VLCDHUFF[0]	VLCDHUFF[0]	VLCDQUANT[0]	VLCDQUANT[0]	IMXCMD[0]	IMXCMD[0]	IMXCOEF[0]	IMXCOEF[0]	IMXCOEF[0]	BUFBE[0]	BUFB[0]	BUFA[0]	BUFA[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:10	VLCDHUFF	0	R/W	Switch for VLCD Huffman Shared Memory Buffer » 00: DSP « » 01: Reserved « » 10: VLCD « » 11: DMA «
9:8	VLCDQUANT	0	R/W	Switch for VLCD quantization Shared Memory Buffer » 00: DSP « » 01: Reserved « » 10: VLCD « » 11: DMA «
7:6	IMXCMD	0	R/W	Switch for IMX Command Shared Memory Buffer » 00: DSP « » 01: IMX « » 10: Reserved « » 11: DMA «
5:4	IMXCOEF	0	R/W	Switch for IMX Coefficient shared Memory Buffer » 00: DSP « » 01: IMX « » 10: VLCD « » 11: DMA «
3:2	BUFB	0	R/W	Switch for BUFB Shared Memory Buffer » 00: DSP « » 01: IMX « » 10: VLCD « » 11: DMA « When IMG_MODE:bit3=0 The IMX, VLCD, and the image buffer's DMA controller each can only access BUFA, BUFB, or none, at the same time. Setting the same value (01, 10, 11) to bits (1:0) and (3:2) will result in undefined behavior. The DSP and DSP DMA controller can access both buffers, so setting both fields to 00 is safe. When IMG_MODE:bit3=1 IMX, VLCD, and DMA can access both BUFA, BUFB, hence same value (01, 10, 11) can be set to bits (1:0) and (3:2)
1:0	BUFA	0	R/W	Switch for BUFA Shared Memory Buffer » 00: DSP « » 01: IMX « » 10: VLCD « » 11: DMA « When IMG_MODE:bit3=0 The IMX, VLCD, and the image buffers DMA controller each can only access BUFA, BUFB, or none, at the same time. Setting the same value (01, 10, 11) to bits (1:0) and (3:2) will result in undefined behavior. The DSP and DSP DMA controller can access both buffers, so setting both fields to 00 is safe. When IMG_MODE:bit3=1 IMX, VLCD, and DMA can access both BUFA, BUFB, hence same value (01, 10, 11) can be set to bits (1:0) and (3:2)

25.7.10 IMG_MODE - ImgBuf Image Buffer Mode Register

IMG_MODE		IO 0009				offset: 0x12								default: 0x0000			
Bit		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SWAPRD	SWAPWR	SEQ	WORD	SHIFT[0]	SHIFT[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:6	RSV			Reserved
5	SWAPRD	0	R/W	Byte SWAP for SDRAM/External Memory Reading (via DMA) » 0: No SWAP « » 1: Byte SWAP «
4	SWAPWR	0	R/W	Byte SWAP for SDRAM/External Memory Writing (via DMA) » 0: No SWAP « » 1: Byte SWAP «
3	SEQ	0	R/W	Buffer A/B sequential access mode » 0: Disable « » 1: Enable «
2	WORD	0	R/W	Byte data to word data transform » 0: No transformation « » 1: Byte data in SDRAM/ExMem stored as Words in ImgBuf « NOTE: - This feature is applicable only when BUFA, BUFB or IMX coeff. memory is involved in the DMA - In this case, the size of data in X-direction in image buffer A/B or IMX coeff memory will be 2 times the size of data in X-direction in SDRAM/External memory.
1:0	SHIFT	0	R/W	(DMA) Data Bus Shift Down » 00: 0-bits « » 01: 4-bits « » 10: 6-bits « » 11: 8-bits «

25.8 Imaging Extension Engine (iMX)

25.8.1 Introduction

Details describing the internal architecture of the iMX co-processor are generally not available to the user. The description of the iMX given in this chapter is very limited. To use the iMX, you should use the C callable iMX API provided by Texas Instruments. Refer to the documentation provided with the C callable API library for more information on these APIs.

The iMX is a parallel MAC engine for increasing DSP image processing performance. The iMX hardware module consists of a flexible control and memory interface and four MACs. The iMX works well in a shared memory configuration with the DSP, so that regular block-based image processing tasks can be accelerated by the iMX, and the DSP can handle irregular processing as well as task-level control.

The iMX can be used to increase the computation speed of many computations, such as the following computations:

- o 2-D FIR filtering
- o CFA interpolation
- o Color space conversion
- o Chroma down-sampling
- o Edge enhancement
- o False color suppression
- o DCT/IDCT
- o Table lookup
- o Motion estimation
- o Generic matrix multiplication
- o Generic linear transform

25.8.2 Features new to DM270

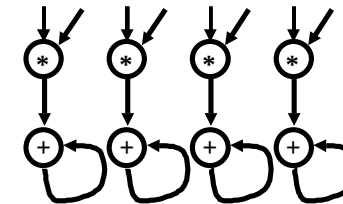
DM270 iMX is a 4-MAC version based on DSC25 iMX, with the following enhancements:

- o Logical AND/OR/XOR function to support extracting lower bits, for post-lookup interpolation
- o MIN-ID and MAX-ID functions. Basically preserves the pointer value to which the minimal value resides. This helps motion estimation.
- o New "CALL" command which enables re-use a iMX command module
- o Sequential access to image buffer A and B for non concurrent process

25.8.3 iMX Structure

As shown in Figure 178, the iMX consists of 4 parallel multiply-accumulate units (MACs). For most of the calculations mentioned above, the iMX processes 720(180x4) million MACs/sec (NOTE: iMX can run upto clock frequency of about 180Mhz. Refer datasheet for exact maximum frequency). Each MAC can perform multiplication, addition, subtraction, absolute value, and accumulate.

Figure 178. iMX Structure : four parallel MACs



25.8.4 DSP/iMX Shared Memory Interface

Figure 179 shows the shared memory configuration in the DSP and iMX memory maps. The iMX co-processor shares iMX coefficient memory, iMX command memory, image buffer A, and image buffer B. All shared memory blocks shown are single-ported, and the DSP software must set which module has access to the shared memory block (no cycle-by-cycle arbitration is implemented). The DSP sets which module has access to each shared memory block by setting register BUF_CTRL (0x8).

In a typical application, the DSP places input data in buffer A and buffer B and iMX commands, filter coefficients, DCT/IDCT cosine constants, and lookup tables in the iMX coefficient/command memory. After placing data and commands in buffer A, buffer B, coefficient memory and command memory, the DSP configures BUF_CTRL (0x8) so that the iMX has access to each of these memory blocks, and then initiates iMX execution. The iMX accesses the memory blocks, computes the commands specified in the command buffer, and writes the result back to the data buffer.

The two general-purpose data buffers, buffer A and buffer B, can be used for efficient concurrent processing. For example, while the iMX is accessing data in buffer A, the data in buffer B can be transferred to SDRAM through the image buffer's DMA controller. When the iMX finishes computation and the result computation result is stored into buffer A, buffer A and buffer B can be swapped so that the iMX can access data in buffer B and the processed data in buffer A can be sent to SDRAM through the image buffer's DMA controller.

To simplify the software that implements buffer swapping, the shared memory interface is designed such that iMX cannot access both buffer A and B at the same time. Buffer A and buffer B have the same memory map with respect to the iMX, and the DSP controls which buffer the iMX can access.

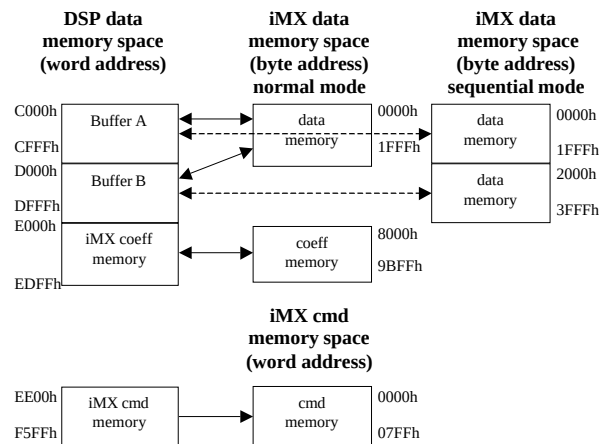
Buffer A, buffer B, the coefficient buffer, and the command buffer are 16-bit wide from DSP's point of view. Also the iMX can access data of type short or byte

from buffer A, buffer B, the coefficient buffer, and the command buffer. Short accesses must be aligned to even 16-bit alignments.

NOTE: Only iMX commands can be stored in iMX command memory.

DM270 has another memory interface to shared memory. It enables a sequential memory access to buffer A and B (bit3 in IMG_MODE(0x9)). In this mode iMX can access to image buffer A and B directly. Also DSP has to switch both buffer A and B to iMX access.

Figure 179. Shared Memory Between iMX and DSP



25.8.5 IMX Control and Status Register Interface

The registers listed in section 25.9 IMX Control Register Map (IMX) control the operation of the iMX. Each bit in these registers is described in the section that follow section 25.10 IMX Control Registers. The iMX control registers reside in the DSP's I/O space, starting at address 0x0010.

Writing an address to **START** initiates iMX execution. The iMX will begin execution from the address written to **START**. When the iMX is executing, bit 0 of **BUSY** is set to 1. **BUSY** is used to determine the busy status of the iMX.

[INTR_EN](#) is used to enable the DSP's iMX interrupt. After the iMX completes execution, the iMX will interrupt the DSP if bit 0 in [INTR_EN](#) is set to 1. The iMX interrupt is DSP interrupt INT1.

CMDPTR shows the address of the next instruction to be decoded by the iMX. **CMDPTR** provides feedback on the progress of iMX. However, due to the pipelining nature of iMX hardware, the returned pointer value will be ahead of actual execution by roughly one command.

ABORT is used to abort iMX when handling errors. Note: since there is no way to know exactly where the iMX stops, it may be very difficult to restart iMX computation after aborting.

25.9 IMX Control Register Map (IMX)

Page	Address	Register	Description
IO	0010	START	iMX Start Register
IO	0011	INTR_EN	iMX INTR Enable Register
IO	0012	BUSY	iMX Busy Register
IO	0013	CMDBPTR	iMX Command Pointer Register
IO	0014	ABORT	iMX Abort Register
IO	0015	CLKCNTRL	iMX Clock Control
IO	0016	MINIMUM_ID	Minimum SAD Block - ID
IO	0017	MINIMUM_VALUE	Minimum SAD Block - Value

25.10 IMX Control Registers

25.10.1 START - iMX Start Register

[illegible][illegible]

25.10.2 INTR_EN - iMX INTR Enable Register

[illegible][illegible]

25.10.3 BUSY - iMX Busy Register

[illegible][illegible]

25.10.4 CMDPTR - iMX Command Pointer Register

[illegible][illegible]

25.10.5 ABORT - iMX Abort Register

[illegible][illegible]

25.10.6 CLKCNTL - iMX Clock Control

[illegible][illegible]

25.10.7 MINIMUM_ID - Minimum SAD Block - ID

[illegible][illegible]

25.10.8 MINIMUM_VALUE - Minimum SAD Block – Value

[illegible][illegible]

25.11 Variable Length Coder/Decoder (VLCD)

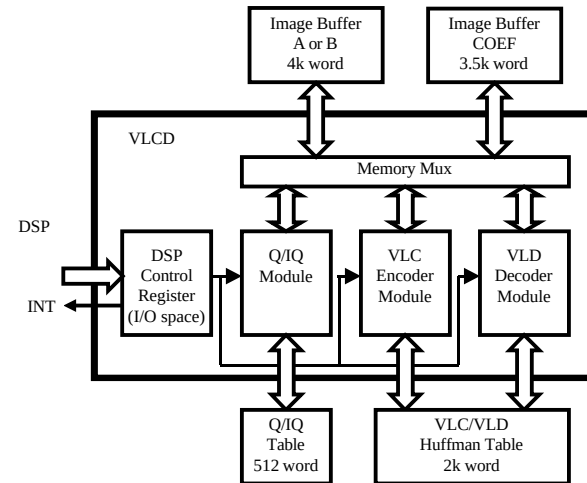
DM270 VLCD module has QIQ, VLC and VLD functions for JPEG, MPEG standard. Quantizer and Inverse Quantizer (Q/IQ) module of the DM270 DSP subsystem is a high-performance, intelligent and highly programmable hardware accelerator for the quantization and de-quantization of DCT coefficients. The Q/IQ is highly autonomous and is capable of carrying out almost all the steps of quantization and de-quantization process with little or no help from the DSP.

25.11.1 Features

Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263

- o Quantization, zigzag (& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263.
- o Quantization, zigzag(& h/v alternated) scan for JPEG, MPEG1/2, MPEG4 and H.263
- o Huffman encode for JPEG, MPEG1/2, MPEG4 and H.263
- o Huffman decode for JPEG, MPEG1/2, MPEG4 and H.263
- o Quantization, zigzag scan for JPEG, MPEG1/2, MPEG4 and H.263
- o CBP (Coded Block Pattern) detection
- o Auto block skip for un-coded block (CBP)
- o Auto insertion of Macro Block Header or SYNC Markers

Figure 180. VLCD Block Diagram



25.11.2 Q/IQ Engine

With highly pipelined architecture, at its peak speed, Q/IQ module is capable of processing 180M DCT coefficients per second.

Q/IQ interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, Q/IQ Table and a set of control registers.

Q/IQ can access the Image Buffers(A/B) or Coefficient memory. Q/IQ can also transfer data to and from the Image Buffer(A/B) or Coefficient memory. Input DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the quantization or de-quantization process. Q/IQ reads the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory.

Q/IQ Table contains more configuration parameters in addition to what is already specified in the control registers. During the Q/IQ process, Q/IQ engine reads the parameters from the Q/IQ Table. The additional parameters provide more flexible control over the Q/IQ process. The parameters include quantization/de-quantization coefficients, DC predictor data, zigzag scan path and block operators.

25.11.3 VLCD Engine

The VLCD coprocessor handles Huffman encode and decode operation for image compression algorithms such as JPEG, MPEG-1/2/4, and H.263. It operates with Huffman tables preloaded by DSP, via shared memory blocks. The VLCD is highly autonomous and is capable of carrying out all the steps of Huffman coding and decoding process with little or no help from the DSP.

VLC/VLD interfaces with the rest of the DSP subsystem via Image Buffers(A/B), Coefficient memory, VLC/VLD Huffman Table and a set of control registers. VLC/VLD can access the Image Buffer(A/B) or Coefficient memory. VLC/VLD can also transfers data to and from the Image Buffer(A/B) or Coefficient memory. The quantized DCT data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLC. The JPEG/MPEG bitstream data must be placed in the image buffers(A/B) or Coefficient memory prior to the start of the VLD. VLC/VLD read the input data from and writes output data to the Image Buffer(A/B) or Coefficient memory. VLC/VLD Huffman Table contains more configuration parameters in addition to what is already specified in the control registers. During the VLC/VLD process, VLC/VLD engine reads the parameters from the VLCD Huffman Table.

DSP configures and controls VLCD module via the control registers, which are mapped into DSP's IO space. The control registers allow the DSP to program Q/IQ and VLC/VLD or various industrial or user-defined standards, start the quantization or/and de-quantization process and VLC or VLD process and monitor its progress. The control registers also contain a set of address pointers to identify the location of input data and output data in the Image buffers(A/B) or coefficient memory and the location of additional configuration parameters in the Q/IQ Table and VLC/VLD Huffman Table.

25.12 Variable Length Coder Decoder Register Map (VLCD)

Page	Address	Register	Description
IO	0020	START	VLCD Start Register
IO	0021	MODE	VLCD Mode Register
IO	0022	QIN_ADDR	Quantization Input Address Register
IO	0023	QOUT_ADDR	Quantization Output Address Register
IO	0024	IQIN_ADDR	Inverse Quantization Input Address Register
IO	0025	IQOUT_ADDR	Inverse Quantization Output Address Register
IO	0026	VLCDIN_ADDR	VLCD Input Address
IO	0027	VLCDOUT_ADDR	VLCD Output Address
IO	0028	DC_PRED0	Quantization DC Predictor 0 Register
IO	0029	DC_PRED1	Quantization DC Predictor 1 Register
IO	002A	DC_PRED2	Quantization DC Predictor 2 Register
IO	002B	DC_PRED3	Quantization DC Predictor 3 Register
IO	002C	DC_PRED4	Quantization DC Predictor 4 Register
IO	002D	DC_PRED5	Quantization DC Predictor 5 Register
IO	002E	IDC_PRED0	Inverse Quantization DC Predictor 0 Register
IO	002F	IDC_PRED1	Inverse Quantization DC Predictor 1 Register
IO	0030	IDC_PRED2	Inverse Quantization DC Predictor 2 Register
IO	0031	IDC_PRED3	Inverse Quantization DC Predictor 3 Register
IO	0032	IDC_PRED4	Inverse Quantization DC Predictor 4 Register
IO	0033	IDC_PRED5	Inverse Quantization DC Predictor 5 Register
IO	0034	MPEG_INVQ	MPEG Inverse Quantization Scale Register
IO	0035	MPEG_Q	MPEG Quantization Scale Register
IO	0036	MPEG_DELTA_Q	MPEG Quantization Delta Register
IO	0037	MPEG_DELTA_IQ	MPEG Inverse Quantization Delta Register
IO	0038	MPEG_THRED	MPEG Thred Register
IO	0039	MPEG_CBP	MPEG coded Block Pattern Register
IO	003A	LUMA_VECTOR	LUMA Bit Vector Register
IO	003B	HUFFTAB_DCY	Huffman DC Y Table Base Address Register
IO	003C	HUFFTAB_DCUV	Huffman DC UV Table Base Address Register
IO	003D	HUFFTAB_AC0	Huffman AC0 Table Base Address Register
IO	003E	HUFFTAB_AC1	Huffman AC1 Table Base Address Register
IO	003F	OFLEV_MAX0TAB	MPEG Max 0 Level Table Base Address Register
IO	0040	OFLEV_MAX1TAB	MPEG Max 1 Level Table Base Address Register
IO	0041	CTLTAB_DCY	DC Y Control Lookup Table Base Address Register
IO	0042	CTLTAB_DCUV	DC UV Control Lookup Table Base Address Register
IO	0043	CTLTAB_AC0	AC0 Control Lookup Table Base Address Register
IO	0044	CTLTAB_AC1	AC1 Control Lookup Table Base Address Register
IO	0045	OFFSET_DCY	DC Y Symbol Lookup Table Address Offset Register
IO	0046	OFFSET_DCUV	DC UV Symbol Lookup Table Address Offset Register
IO	0047	OFFSET_AC0	AC0 Symbol Lookup Table Address Offset Register
IO	0048	OFFSET_AC1	AC1 Symbol Lookup Table Address Offset Register
IO	0049	SYMTAB_DCY	DC Y Symbol Lookup Table Base Address Register
IO	004A	SYMTAB_DCUV	DC UV Symbol Lookup Table Base Address Register
IO	004B	SYMTAB_AC0	AC0 Symbol Lookup Table Base Address Register
IO	004C	SYMTAB_AC1	AC1 Symbol Lookup Table Base Address Register
IO	004D	VLD_CTL	VLD Control Register
IO	004E	VLD_NRBIT_DC	DC Number of Bits Register
IO	004F	VLD_NRBIT_AC	AC Number of Bits Register
IO	0050	BITS_BPTR	Bits Pointer Register
IO	0051	BITS_WORD	Bits Word Register
IO	0052	BYTE_ALIGN	Byte Align Register
IO	0053	HEAD_ADDR	Header Address Register
IO	0054	HEAD_NUM	Number of Header Data Register
IO	0055	QIQ_CONFIG0	QIQ Configuration Register #0
IO	0056	QIQ_CONFIG1	QIQ Configuration Register #1
IO	0057	QIQ_CONFIG2	QIQ Configuration Register #2
IO	0058	QIQ_CONFIG3	QIQ Configuration Register #3
IO	0059	QIQ_CONFIG4	QIQ Configuration Register #4
IO	005A	QIQ_CONFIG5	QIQ Configuration Register #5
IO	005B	VLD_ERRCTL	VLD Error Control Register
IO	005C	VLD_ERRSTAT	VLD Error Status Register
IO	005D	RING_START	Ring Buffer Start Address Register
IO	005E	RING_END	Ring Buffer End Address Register
IO	005F	CLKCTRL	VLCD Clock Control
IO	0020	0	

25.13 VLCD Registers

25.13.1 START - ImgBuf VLCD Start Register

[illegible][illegible]

25.13.2 MODE - ImgBuf VLCD Mode Register

[illegible]

Bit	Name	Reset Value	R/W	Function
15	MPEGRND	0	R/W	<u>Round control for MPEG INVO</u> » 0: round off « » 1: round on «
14:12	NUMBLKS	0	R/W	NOTE : This bit is reserved in ES1 and is available only in ES2 and later
11	JPEGSYNC	0	R/W	<u>JPEG Sync Insertion</u> » 0: Off « » 1: On «
10	MPEGTYPE	0	R/W	Writing "1" enables inserting 00 byte after FF byte MPEG Picture Coding Type » 0: I/P/B picture coding « » 1: D picture coding «
9:8	SCANMODE	0	R/W	<u>Scan Mode (VLC/VLD)</u> » 00: Linear scan « » 01: Zig-Zag scan « » 10: Alternate-vertical scan « » 11: Alternate-horizontal scan «
7	MPEGM INTRA	0	R/W	<u>Intra Macro Block Select</u> » 0: Non-intra macro block « » 1: Intra macro block (In JPEG mode should be '1') « Selects whether blocks are encoded as Intra Macro blocks
6	INTRAVLC	0	R/W	<u>MPEG Intra VLC Format</u> <u>Codec Mode Select</u> » 00: JPEG « » 01: MPEG1 « » 10: MPEG4 « » 11: H.263 «
5:4	MODESEL	0	R/W	Type of quantization to use for Encode and Decode use in MPEG4 0: TYPE 0 « 1: TYPE 1 «
3	OMPEGTYPE	0	R/W	<u>Function Select</u> » 000: Q « » 001: IQ « » 010: VLC « » 011: VLD « » 100: Q + IQ « » 101: Q + VLC « » 110: Q + IQ + VLC « » 111: VLD + IQ «
2:0	FUNC	0	R/W	

25.13.3 QIN_ADDR - ImgBuf Quantization Input Address Register

[illegible][illegible]

25.13.4 QOUT_ADDR - ImgBuf Quantization Output Address Register

[illegible][illegible]

25.13.5 IQIN_ADDR - ImgBuf Inverse Quantization Input Address Register

[illegible][illegible]

25.13.6 IQOUT_ADDR - ImgBuf Inverse Quantization Output Address Register

[illegible][illegible]

25.13.7 VLCDIN_ADDR - ImgBuf VLCD Input Address

[illegible][illegible]

25.13.8 VLCDOUT_ADDR - ImgBuf VLCD Output Address

[illegible][illegible]

25.13.9 DC_PRED0 - ImgBuf Quantization DC Predictor 0 Register

[illegible][illegible]

25.13.10 DC_PRED1 - ImgBuf Quantization DC Predictor 1 Register

[illegible][illegible]

25.13.11 DC_PRED2 - ImgBuf Quantization DC Predictor 2 Register

[illegible][illegible]

25.13.12 DC_PRED3 - ImgBuf Quantization DC Predictor 3 Register

[illegible][illegible]

25.13.13 DC_PRED4 - ImgBuf Quantization DC Predictor 4 Register

[illegible][illegible]

25.13.14 DC_PRED5 - ImgBuf Quantization DC Predictor 5 Register

[illegible][illegible]

25.13.15 IDC_PRED0 - ImgBuf Inverse Quantization DC Predictor 0 Register

[illegible][illegible]

25.13.16 IDC_PRED1 - ImgBuf Inverse Quantization DC Predictor 1 Register

[illegible][illegible]

25.13.17 IDC_PRED2 - ImgBuf Inverse Quantization DC Predictor 2 Register

[illegible][illegible]

25.13.18 IDC_PRED3 - ImgBuf Inverse Quantization DC Predictor 3 Register

[illegible][illegible]

25.13.19 IDC_PRED4 - ImgBuf Inverse Quantization DC Predictor 4 Register

[illegible][illegible]

25.13.20 IDC_PRED5 - ImgBuf Inverse Quantization DC Predictor 5 Register

[illegible][illegible]

25.13.21 MPEG_INVQ - ImgBuf MPEG Inverse Quantization Scale Register

[illegible][illegible]

25.13.22 MPEG_Q - ImgBuf MPEG Quantization Scale Register

MPEG_Q	IO 0035 offset: 0x2A															default: 0x0000
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MQ[7]	MQ[6]	MQ[5]	MQ[4]	MQ[3]	MQ[2]	MQ[1]	MQ[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

[illegible]

25.13.23 MPEG_DELTA_Q - ImgBuf MPEG Quantization Delta Register

[illegible][illegible]

25.13.24 MPEG_DELTA_IQ - ImgBuf MPEG Inverse Quantization Delta Register

[illegible][illegible]

25.13.25 MPEG_THRED - ImgBuf MPEG Thred Register

[illegible][illegible]

25.13.26 MPEG_CBP - ImgBuf MPEG coded Block Pattern Register

MPEG_CBP		IO		0039		offset:		0x32						default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CPBON	RSV	RSV	MCBP[5]	MCBP[4]	MCBP[3]	MCBP[2]	MCBP[1]	MCBP[0]	
R/W	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	-	-	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	CPBON	0	R/W	CBP Detection Controll 0: CBP detect off (JPEG) 1: CBP detect on
7:6	RSV			Reserved
5:0	MCBP	0	R/W	Coded block pattern, indicating at least one nonzero in a block. For the standard six block macro block, CBP of the first block would be in bit position five. The second block's CBP bit in position 4, down to the last block being in bit position 0.

25.13.27 LUMA_VECTOR - ImgBuf LUMA Bit Vector Register

LUMA_VECTOR		IO		003A		offset:		0x34		default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	LUMVECT[5]	LUMVECT[4]	LUMVECT[3]	LUMVECT[2]	LUMVECT[1]	LUMVECT[0]	
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:6	RSV			Reserved
5:0	LUMVECT	0	R/W	Luma Bit Vector This register contains the Luma bit vector. Each block will have a 1 bit entry into this vector specifying whether the block has luminance data or chrominance data. Setting this bit high indicates that block has luminance data. This bit vector shall be arranged such that the Luma bit for each block shall be in the bit position of number of blocks minus the block count number. Thus for the standard six block macro block, the first block's Luma bit would be in bit position five. The second block's coded bit in position 4, down to the last block being in bit position 0.

25.13.28 HUFFTAB_DCY - ImgBuf Huffman DC Y Table Base Address Register

[illegible][illegible]

25.13.29 HUFFTAB_DCUIV - ImgBuf Huffman DC UV Table Base Address Register

[illegible][illegible]

25.13.30 HUFFTAB_AC0 - ImgBuf Huffman AC0 Table Base Address Register

[illegible][illegible]

25.13.31 HUFFTAB_AC1 - ImgBuf Huffman AC1 Table Base Address Register

[illegible][illegible]

25.13.32 OFLEV_MAXOTAB - ImgBuf MPEG Max 0 Level Table Base Address Register

OFLEV	MAXOTAB	IO	003F	offset:	0x3E														default:	0x0000
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	MAXO[10]	MAXO[9]	MAXO[8]	MAXO[7]	MAXO[6]	MAXO[5]	MAXO[4]	MAXO[3]	MAXO[2]	MAXO[1]	MAXO[0]				
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0				

[illegible]

25.13.33 OFLEV_MAX1TAB - ImgBuf MPEG Max 1 Level Table Base Address Register

[illegible][illegible]

25.13.34 CTLTAB_DCY - ImgBuf DC Y Control Lookup Table Base Address Register

[illegible][illegible]

25.13.35 CTLTAB_DCUV - ImgBuf DC UV Control Lookup Table Base Address Register

[illegible][illegible]

25.13.36 CTLTAB_AC0 - ImgBuf AC0 Control Lookup Table Base Address Register

[illegible][illegible]

25.13.37 CTLTAB_AC1 - ImgBuf AC1 Control Lookup Table Base Address Register

CTLTAB AC1		IO		0044		offset:		0x48												default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2			1	0					
Name	RSV	RSV	RSV	RSV	RSV	CACI[10]	CACI[9]	CACI[8]	CACI[7]	CACI[6]	CACI[5]	CACI[4]	CACI[3]	CACI[2]			CACI[1]	CACI[0]					
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		R/W	R/W					
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0		0	0					

[illegible]

25.13.38 OFFSET_DCY - ImgBuf DC Y Symbol Lookup Table Address Offset Register

[illegible][illegible]

25.13.39 OFFSET_DCUV - ImgBuf DC UV Symbol Lookup Table Address Offset Register

[illegible][illegible]

25.13.40 OFFSET_AC0 - ImgBuf AC0 Symbol Lookup Table Address Offset Register

[illegible][illegible]

25.13.41 OFFSET_AC1 - ImgBuf AC1 Symbol Lookup Table Address Offset Register

[illegible][illegible]

25.13.42 SYMTAB_DCY - ImgBuf DC Y Symbol Lookup Table Base Address Register

[illegible][illegible]

25.13.43 SYMTAB_DCUIV - ImgBuf DC UV Symbol Lookup Table Base Address Register

[illegible][illegible]

25.13.44 SYMTAB_AC0 - ImgBuf AC0 Symbol Lookup Table Base Address Register

[illegible][illegible]

25.13.45 SYMTAB_AC1 - ImgBuf AC1 Symbol Lookup Table Base Address Register

[illegible][illegible]

25.13.46 VLD_CTL - ImgBuf VLD Control Register

[illegible]

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	DCYLEAD	0	R/W	This bit signifies that the Uvld is expecting a 1 leading Huffman Table for the DC Y Tables
6	DCUVLEAD	0	R/W	This bit signifies that the Uvld is expecting a 1 leading Huffman Table for the DC UV Tables
5	ACOLEAD	0	R/W	This bit signifies that the Uvld is expecting a 1 leading Huffman Table for the AC 0 Tables
4	AC1LEAD	0	R/W	This bit signifies that the Uvld is expecting a 1 leading Huffman Table for the AC 1 Tables
3	DCYSYMLN	0	R/W	This bit signifies the bit length of the decoded symbol in the DC Y Symbol Table. Setting this bit to 1 signifies a 11 bit symbol, while 0 signifies a 12 bit table.
2	DCUVSYMLN	0	R/W	This bit signifies the bit length of the decoded symbol in the DC UV Symbol Table. Setting this bit to 1 signifies a 11 bit symbol
1	AC0SYMLN	0	R/W	This bit signifies the bit length of the decoded symbol in the AC 0 Symbol Table. Setting this bit to 1 signifies a 11 bit symbol
0	AC1SYMLN	0	R/W	This bit signifies the bit length of the decoded symbol in the AC 1 Symbol Table. Setting this bit to 1 signifies a 11 bit symbol

25.13.47 VLD_NRBIT_DC - ImgBuf DC Number of Bits Register

VLD NRBIT DC		IO		004E		offset:		0x5C												default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	Y[4]	Y[3]	Y[2]	Y[1]	Y[0]	RSV	RSV	RSV	uv[4]	uv[3]	uv[2]	uv[1]	uv[0]							
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W							
Default	-	-	-	0	0	0	0	0	-	-	-	0	0	0	0	0							

[illegible]

25.13.48 VLD_NRBIT_AC - ImgBuf AC Number of Bits Register

VLD	NRBIT	AC	IO	004F	offset:	0x5E											default:	0x0000
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	AC0[4]	AC0[3]	AC0[2]	AC0[1]	AC0[0]	RSV	RSV	RSV	AC1[4]	AC1[3]	AC1[2]	AC1[1]	AC1[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	0	0	0	0	0	-	-	-	R/W	R/W	0	0	0		

[illegible]

25.13.49 BITS_BPTR - ImgBuf Bits Pointer Register

BITS BPTR													IO		0050		offset:		0x60		default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BPTR[3]	BPTR[2]	BPTR[1]	BPTR[0]								
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W								
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0								

[illegible]

25.13.50 BITS_WORD - ImgBuf Bits Word Register

[illegible][illegible]

25.13.51 BYTE_ALIGN - ImgBuf Byte Align Register

BYTE ALIGN		IO		0052		offset:		0x64										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ENABLE	BIT					
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W					
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0					

[illegible]

25.13.52 HEAD_ADDR - ImgBuf Header Address Register

[illegible][illegible]

25.13.53 HEAD_NUM - ImgBuf Number of Header Data Register

HEAD NUM		IO 0054 offset: 0x68										default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	HNDUM[5]	HNDUM[4]	HNDUM[3]	HNDUM[2]	HNDUM[1]	HNDUM[0]	
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	

[illegible]

25.13.54 QIQ_CONFIG0 - ImgBuf QIQ Configuration Register #0

[illegible][illegible]

25.13.55 **QIQ_CONFIG1 - ImgBuf QIQ Configuration Register #1**

QIQ_CONFIG1		IO		0056		offset:		0x6C		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	CPRESEL	CPRESEL	CPRESEL			
R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	QMATSEL			Quant matrix selector (1 of 8 banks)
7	RSV			Reserved
6:4	I QMATSEL			Inverse Quantization matrix selector (1 of 8 banks)
3	RSV			Reserved
2:0	DCPRESEL			DC predictor selector (1 of 6)

25.13.56 **QIQ_CONFIG2 - ImgBuf QIQ Configuration Register #2**

QIQ_CONFIG2		IO		0057		offset:		0x6E		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	CPRESEL	CPRESEL	CPRESEL			
R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	QMATSEL			Quant matrix selector (1 of 8 banks)
7	RSV			Reserved
6:4	I QMATSEL			Inverse Quantization matrix selector (1 of 8 banks)
3	RSV			Reserved
2:0	DCPRESEL			DC predictor selector (1 of 6)

25.13.57 **QIQ_CONFIG3 - ImgBuf QIQ Configuration Register #3**

QIQ_CONFIG3		IO		0058		offset:		0x70		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	CPRESEL	CPRESEL	CPRESEL			
R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	QMATSEL			Quant matrix selector (1 of 8 banks)
7	RSV			Reserved
6:4	I QMATSEL			Inverse Quantization matrix selector (1 of 8 banks)
3	RSV			Reserved
2:0	DCPRESEL			DC predictor selector (1 of 6)

25.13.58 **QIQ_CONFIG4 - ImgBuf QIQ Configuration Register #4**

QIQ_CONFIG4		IO		0059		offset:		0x72		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	QMATSEL	QMATSEL	QMATSEL	RSV	CPRESEL	CPRESEL	CPRESEL			
R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	QMATSEL			Quant matrix selector (1 of 8 banks)
7	RSV			Reserved
6:4	I QMATSEL			Inverse Quantization matrix selector (1 of 8 banks)
3	RSV			Reserved
2:0	DCPRESEL			DC predictor selector (1 of 6)

25.13.59 QIQ_CONFIG5 - ImgBuf QIQ Configuration Register #5

[illegible][illegible]

25.13.60 VLD_ERRCTL - ImgBuf VLD Error Control Register

VLD_ERRCTL		IO		005B		offset:		0x76								default: 0x0004	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	EOB	RSV	UVLD	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	1	-	0	

[illegible]

25.13.61 VLD_ERRSTAT - ImgBuf VLD Error Status Register

VLD_ERRSTAT		IO		005C		offset:		0x78								default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RINGBUFF	EOB	VLD	UVLD	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R	R	R	R	
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0	

[illegible]

25.13.62 RING_START - ImgBuf Ring Buffer Start Address Register

[illegible][illegible]

25.13.63 RING_END - ImgBuf Ring Buffer End Address Register

RING_END		IO		005E	offset: 0x7C												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	REND[12]	REND[11]	REND[10]	REND[9]	REND[8]	REND[7]	REND[6]	REND[5]	REND[4]	REND[3]	REND[2]	REND[1]	REND[0]		
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:0	REND	0	R/W	Ring buffer end address. VLC: When output pointer reaches end address, output pointer starts at start address automatically. VLD: When input pointer reaches end address, input pointer starts at start pointer automatically.

25.13.64 CLKCTRL - ImgBuf VLCD Clock Control

CLKCTRL		IO		005F	offset: 0x7E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MPEG2	ESC	CLKOFF		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	MPEG2	0	R/W	MPEG2 encode/decode » 0: MPEG1, VLCD_MODE(5:4)=MPEG1 « » 1: MPEG2, VLCD_MODE(5:4)=MPEG1 «
1	ESC	0	R/W	MPEG4 escape encoding » 0: escape 3 only (DSC25) « » 1: escape 1, 2, 3 «
0	CLKOFF	0	R/W	Enable/Disable Clock » 0: ON « » 1: OFF «

25.14 VLC – Huffman Table Configuration

25.14.1 VLC – Huffman Table Configuration

The table below provides more information on Huffman table configuration for VLC (encoding).

Addr	Register	Bit(s)	Description
0021	MODE	10	Flag to indicate the picture coding type. picture_coding_type = 1 => D-picture in MPEG-1.
		6	Flag to indicate the coding table selection, in the range of [0, 1]. The following rules are applied to determine value of this flag: IF (H263 MPEG1) intra_vlc_format = 0; IF (MPEG4 && DC tables are used for DC coding) intra_vlc_format = 1; IF (MPEG4 && AC tables are used for DC coding) intra_vlc_format = 0;
003B	Huffman DC Y Table Pointer		Starting address of the DC coding table for luminance. JPEG: each entry is in 32-bit, the first 16 MSBs are used to store the code length, the 16 LSBs are used to store the code word. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
003C	Huffman DC UV Table Pointer		Starting address of the DC coding table for chrominance. JPEG: each entry is in 32-bit, the first 16 MSBs are used to store the code length, the 16 LSBs are used to store the code word. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
003D	Huffman AC #0 Table Pointer		Starting address of the AC coding table 0 JPEG: each entry is in 32-bit, the first 16 MSBs are used to store the code length, the 16 LSBs are used to store the code word. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
003E	Huffman AC #1 Table Pointer		Starting address of the AC coding table 1 JPEG: each entry is in 32-bit, the first 16 MSBs are used to store the code length, the 16 LSBs are used to store the code word. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
003F	MPEG Max 0 Level Table Pointer		Pointer to array of 64 (for MPEG-1) or 128 (for H.263/MPEG-4) elements, in the range of [0x000 - 0xFFFF]. Each entry has 16 bits, the 8 MSBs are used to store the (offset, levelmax), which is used for addressing the AC coding table and deciding coding mode chosen for a coding symbol Let (run, level) or (last, run, level) be a coding symbol in MPEG1 and H.263/MPEG-4, respectively. The following steps demonstrate how this field is utilized during the coding: IF (H263 MPEG4) run +=64*last; offset = offset_levelmax0[run]>>8; levelmax = offset_levelmax0[run]&0xff; IF (level > levelmax) the symbol is coded in ESCAPE mode ELSE codtab_ac0[offset+level] is used for encoding the symbol in normal mode
0040	MPEG Max 1 Level Table Pointer		Pointer to array of 64 (for MPEG-1) or 128 for (H.263/MPEG-4) elements, in the range of [0x000 - 0xFFFF]. Each entry has 16 bits, the 8 MSBs are used to store the (offset, levelmax), which is used for addressing the AC coding table and deciding coding mode chosen for a coding symbol Let (run, level) or (last, run, level) be a coding symbol in MPEG1 and H.263/MPEG-4, respectively. The following steps demonstrate how this field is utilized during the coding: IF (H263 MPEG4) run +=64*last; offset = offset_levelmax1[run]>>8; levelmax = offset_levelmax1[run]&0xff; IF (level > levelmax) the symbol is coded in ESCAPE mode ELSE codtab_ac1[offset+level] is used for encoding the symbol in normal mode

25.14.2 VLC – Huffman Table Configuration Example – MPEG-1

The table below provides more information on Huffman table configuration for VLC (encoding) of MPEG-1

Addr	Register	Bit(s)	Description
0021	MODE	10	0: I, P, B picture coding
		6	0: H.263 or MPEG-1 or MPEG-4 and AC tables used for DC coding
003B	Huffman DC Y Table	1804 1000 1001 1805 1806 200e 281e 303e 387e 40fe 49fe 49ff	
003C	Huffman DC UV Table	1000 1001 1002 1806 200e 281e 303e 387e 40fe 49fe 53fe 53ff	
003D	Huffman AC #0 Table	0000 1003 2004 2805 3806 4026 4021 500a 601d 6018 6013 6010 681a 6819 6818 6817 701f 701e 701d 701c 701b 701a 7019 7018 7017 7016 7015 7014 7013 7012 7011 7010 7818 7817 7816 7815 7814 7813 7812 7811 7810 1803 3006 4025 500c 601b 6816 6815 781f 781e 781d 781c 781b 781a 7819 8013 8012 8011 8010 2005 3804 500b 6014 6814 2807 4024 601c 6813 2806 500f 6012 3007 5009 6812 3005 601e 8014 3004 6015 3807 6011 3805 6811 4027 6810 4023 801a 4022 8019 4020 8018 500e 8017 500d 8016 5008 8015 601f 601a 6019 6017 6016 681f 681e 681d 681c 681b 801f 801e 801d 801c 801b	
003E	Huffman AC #1 Table	0000 1003 2004 2805 3806 4026 4021 500a 601d 6018 6013 6010 681a 6819 6818 6817 701f 701e 701d 701c 701b 701a 7019 7018 7017 7016 7015 7014 7013 7012 7011 7010 7818 7817 7816 7815 7814 7813 7812 7811 7810 1803 3006 4025 500c 601b 6816 6815 781f 781e 781d 781c 781b 781a 7819 8013 8012 8011 8010 2005 3804 500b 6014 6814 2807 4024 601c 6813 2806 500f 6012 3007 5009 6812 3005 601e 8014 3004 6015 3807 6011 3805 6811 4027 6810 4023 801a 4022 8019 4020 8018 500e 8017 500d 8016 5008 8015 601f 601a 6019 6017 6016 681f 681e 681d 681c 681b 801f 801e 801d 801c 801b	
003F	MPEG Max 0 Level Table	0028 2812 3a05 3f04 4303 4603 4903 4c02 4e02 5002 5202 5402 5602 5802 5a02 5c02 5e02 6001 6101 6201 6301 6401 6501 6601 6701 6801 6901 6a01 6b01 6c01 6d01 6e01 0000	
0040	MPEG Max 1 Level Table	0028 2812 3a05 3f04 4303 4603 4903 4c02 4e02 5002 5202 5402 5602 5802 5a02 5c02 5e02 6001 6101 6201 6301 6401 6501 6601 6701 6801 6901 6a01 6b01 6c01 6d01 6e01 0000	

25.15 VLC – Header Insertion

25.15.1 VLC – Header Insertion Configuration

Addr	Register	Bit(s)	Description
0053	HEAD_ADDR	10:0	Specifies the start address of bitstream to be appended. This macro block header should be stored in Huffman table (0xB800h-0xBFFFh), and HEAD_ADDR represents a relative address (0x0000h-0x07FFh) within this table. Each entry is in 32-bit, the 16 MSBs are used to store the code length and the 16 LSBs are used to store the code word.
0054	HEAD_NUM	5:0	Specifies the total number of entries. When a non-zero value is written to this register, the stored code word is inserted in the bitstream before starting the DCT block Huffman encoding.

25.15.2 VLC – Header Insertion Example

Addr	Register	Bit(s)	Description
0053	HEAD_ADDR	10:0	0x700 à header starts at 0xB800+0x0700 = 0xBF00
0054	HEAD_NUM	5:0	3

Addr	Code Length	Code Word
BF00	0x0003	0x0006 à 110
BF02	0x0004	0x0009 à 1001
BF04	0x0002	0x0002 à 10

The output bitstream is:

Previous bitstream	Macroblock header	Current DCT bitstream
---	110, 1001, 10	***

25.16 VLD – Huffman Table Configuration

25.16.1 VLD – Huffman Table Configuration

The table below provides more information on Huffman table configuration for VLD (decoding).

Addr	Register	Bit(s)	Description
0021	MODE	10	Flag to indicate the picture coding type. picture_coding_type = 1 => D -picture in MPEG-1.
		6	Flag to indicate the coding table selection, in the range of [0, 1]. The following rules are applied to determine value of this flag: IF (H263 MPEG1) intra_vlc_format = 0; IF (MPEG4 && DC tables are used for DC coding) intra_vlc_format = 1; IF (MPEG4 && AC tables are used for DC coding) intra_vlc_format = 0;
0041	DC Y Control Table Pointer		Starting address of the DC decoding table for luminance. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
0042	DC UV Control Table Pointer		Starting address of the DC decoding table for chrominance. MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
0043	AC #0 Control Table Pointer		Starting address of the AC decoding table 0 MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
0044	AC #1 Control Table Pointer		Starting address of the AC decoding table 1 MPEG: each entry is 16-bit, the first 5 MSBs are used to store the code length, the 11 LSBs are used to store the code word.
0045	Control Table offset DC Y		16-bit global offset for the DC decoding control table of luminance
0046	Control Table offset DC UV		16-bit global offset for the DC decoding control table of chrominance
0047	Control Table offset AC 0		16-bit global offset for the AC decoding control table 0
0048	Control Table offset AC 1		16-bit global offset for the AC decoding control table 1
0049	Symbol table DC Y pointer		Starting address of the DC decoding table for luminance, each entry is in 16 -bit, the first 4/5 MSBs contain the code length, 12/11 LSBs contain the code symbol.
004A	Symbol table DC UV pointer		Starting address of the DC decoding table for chrominance, each entry is 16 -bits, the first 4/5 MSBs contain the code length, 12/11 LSBs contain the code symbol.
004B	Symbol table AC 0 pointer		Starting address of the AC decoding table 0, each entry is 16 -bits, the first 4/5 MSBs contain the code length, the 12/11 LSBs contain the code symbol.
004C	Symbol table AC 1 pointer		Starting address of the AC decoding table 1, each entry is 16-bit, the first 4/5 MSBs contain the code length, the 12/11 LSBs contain the code symbol.
004D	VLC_CTL	7	Flag to indicate the type of DC Y decoding table, 1: one leading table, 0: zero leading table
		6	Flag to indicate the type of DC UV decoding table, 1: one leading table, 0: zero leading table
		5	Flag to indicate the type of AC 0 decoding table, 1: one leading table, 0: zero leading table
		4	Flag to indicate the type of AC 1 decoding table, 1: one leading table, 0: zero leading table
		3	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table DC Y". 0:12bit, 1:11bit
		2	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table DC UV". 0:12bit, 1:11bit
		1	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table AC 0". 0:12bit, 1:11bit
		0	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table AC 1". 0:12bit, 1:11bit
004E	VLD_NRBIT_DC	12:8 4:0	Max code length of the luminance DC decoding table, in the range of [1 -16].
004F	VLD_NRBIT_AC	12:8	Max code length of the chrominance DC decoding table, in the range of [1 -16].
		4:0	Max code length of the chrominance AC decoding table 1, in the range of [1 -16].

25.16.2 VLD – Huffman Table Example – MPEG-1

The table below provides more information on Huffman table configuration for VLD (decoding).

Addr	Register	Bit(s)	Description
0021	MODE	10	0: I, P, B picture coding
		6	0: H.263 or MPEG-1 or MPEG-4 and AC tables used for DC coding
0041	DC Y Control Table Pointer		0000 0000 0001 1002 1803 2004 2805 3006 3006 3808
0042	DC UV Control Table Pointer		0000 0000 0801 1002 1803 2004 2805 3006 3807 4008 4008
0043	AC #0 Control Table Pointer		0000 0000 0000 0000 0000 0000 0810 1020 1830 2040 3058 5067 4865 5069 4051 608d 7093
0044	AC #1 Control Table Pointer		0000 0000 0000 0000 0000 0000 0810 1020 1830 2040 3058 5067 4865 5069 4051 608d 4015
0045	Control Table offset DC Y		0
0046	Control Table offset DC UV		0
0047	Control Table offset AC 0		0
0048	Control Table offset AC 1		0
0049	Symbol table DC Y pointer		480b 480a 4009 3808 3007 2806 2005 1804 1803 1800 1002 1001
004A	Symbol table DC UV pointer		500b 500a 4809 4008 3807 3006 2805 2004 1803 1002 1001 1000
004B	Symbol table AC 0 pointer		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 8052 8051 8050 804f 8183 8402 83c2 8382 8342 8302 82c2 87c1 8781 8741 8701 86c1 7828 7827 7826 7825 7824 7823 7822 7821 7820 784e 784d 784c 784b 784a 7849 7848 701f 701e 701d 701c 701b 701a 7019 7018 7017 7016 7015 7014 7013 7012 7011 7010 6a82 6a42 6943 68c4 6885 6847 6846 680f 680e 680d 680c 6e81 6e41 6e01 6dc1 6d81 600b 6202 6103 600a 6084 61c2 6541 6501 6009 64c1 6481 6045 60c3 6008 6182 6441 5401 5142 5007 5083 5044 53c1 5381 5102 3058 3882 3a41 3804 3a01 31c1 3181 3042 3141 4341 4006 4301 42c1 40c2 4043 4005 4281 2803 2803 2803 2803 2803 2803 2803 2803 2901 2901 2901 2901 2901 2901 2901 2901 28c1 28c1 28c1 28c1 28c1 28c1 28c1 28c1 2002 2081 1841 1841 17d0 1001

Addr	Register	Bit(s)	Description
004C	Symbol table AC 1 pointer		0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 8052 8051 8050 804f 8183 8402 83c2 8382 8342 8302 82c2 87c1 8781 8741 8701 86c1 7828 7827 7826 7825 7824 7823 7822 7821 7820 784e 784d 784c 784b 784a 7849 7848 701f 701e 701d 701c 701b 701a 7019 7018 7017 7016 7015 7014 7013 7012 7011 7010 6a82 6a42 6943 68c4 6885 6847 6846 0000 0000 0000 0000 6e81 6e41 6e01 6dc1 6d81 0000 6202 6103 0000 0000 61c2 6541 6501 0000 64c1 6481 0000 60c3 0000 6182 6441 4942 4942 4b81 4b81 5084 5401 4bc1 4bc1 3058 39c1 3a01 3 981 3882 3007 3006 3101 3141 4045 42c1 400b 400a 4341 4301 40c2 4044 2881 2881 2881 2881 2881 2881 2881 2881 2842 2842 2842 2842 2842 2842 2842 2842 28c1 28c1 28c1 28c1 28c1 28c1 28c1 28c1 1841 1841 27d0 2003 1001 1001 1001 1001 1001 1001 1001 1001 1001 10 01 1001 1802 2804 2804 2804 2804 2804 2804 2804 2804 2805 2805 2805 2805 2805 2805 2805 2805 3a41 3a41 3843 3843 3a81 3a81 3808 3808 3809 3809 400c 400d 4083 4102 400e 400f
			0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 8052 8051 8050 804f 8183 8402 83c2 8382 8342 8302 82c2 87c1 8781 8741 8701 86c1 7828 7827 7826 7825 7824 7823 7822 7821 7820 784e 784d 784c 784b 784a 7849 7848 701f 701e 701d 701c 701b 701a 7019 7018 7017 7016 7015 7014 7013 7012 7011 7010 6a82 6a42 6943 68c4 6885 6847 6846 0000 0000 0000 0000 6e81 6e41 6e01 6dc1 6d81 0000 6202 6103 0000 0000 61c2 6541 6501 0000 64c1 6481 0000 60c3 0000 6182 6441 4942 4942 4b81 4b81 5084 5401 4bc1 4bc1 3058 39c1 3a01 3 981 3882 3007 3006 3101 3141 4045 42c1 400b 400a 4341 4301 40c2 4044 2881 2881 2881 2881 2881 2881 2881 2881 2842 2842 2842 2842 2842 2842 2842 2842 28c1 28c1 28c1 28c1 28c1 28c1 28c1 28c1 1841 1841 27d0 2003 1001 1001 1001 1001 1001 1001 1001 1001 1001 10 01 1001 1802 2804 2804 2804 2804 2804 2804 2804 2804 2805 2805 2805 2805 2805 2805 2805 2805 3a41 3a41 3843 3843 3a81 3a81 3808 3808 3809 3809 400c 400d 4083 4102 400e 400f
004D	VLC_CTL	7	Flag to indicate the type of DC Y decoding table, 1: one leading table
		6	Flag to indicate the type of DC UV decoding table, 1: one leading table
		5	Flag to indicate the type of AC 0 decoding table, 0: zero leading table
		4	Flag to indicate the type of AC 1 decoding table, 0: zero leading table
		3	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table DC Y". 1:11bit
		2	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table DC UV". 1:11bit
		1	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table AC 0". 1:11bit
		0	Number of LSBs used for storing the code symbol in each 16 -bit entry of the luminance DC decoding table "Symbol Table AC 1". 1:11bit
004E	VLD_NRBIT_DC	12:8	0x09
		4:0	0x0A
004F	VLD_NRBIT_AC	12:8	0x10
		4:0	0x10

25.17 QIQ – Scanning paths and quantization matrices used

25.17.1 Zigzag scanning path, used in JPEG,H.263, MPEG-1 and MPEG4

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

25.17.2 Vertical scanning path, used in MPEG4

0	4	6	20	22	36	38	52
1	5	7	21	23	37	39	53
2	8	19	24	34	40	50	54
3	9	18	25	35	41	51	55
10	17	26	30	42	46	56	60
11	16	27	31	43	47	57	61
12	15	28	32	44	48	58	62
13	14	29	33	45	49	59	63

25.17.3 Horizontal scanning path, used in MPEG4

0	1	2	3	10	11	12	13
4	5	8	9	17	16	15	14
6	7	19	18	26	27	28	29
20	21	24	25	30	31	32	33
22	23	34	35	42	43	44	45
36	37	40	41	46	47	48	49
38	39	50	51	56	57	58	59
52	53	54	55	60	61	62	63

25.17.4 Default intra quantiser matrix for MPEG1 and MPEG4

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	46	56	69
27	29	35	38	46	56	69	83

25.17.5 Default inter quantiser matrix for MPEG1

16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16

25.17.6 Default inter quantiser matrix for MPEG4

16	17	18	19	20	21	22	23
17	18	19	20	21	22	23	24
18	19	20	21	22	23	24	25
19	20	21	22	23	24	26	27
20	21	22	23	25	26	27	28
21	22	23	24	26	27	28	30
22	23	24	26	27	28	30	31
23	24	25	27	28	30	31	33

25.18 VLC Tables used**25.18.1 VLC table for MPEG1 dct_dc_size_luminance, 12 entries**

```
0x1804, 0x1000, 0x1001, 0x1805, 0x1806, 0x200e, 0x281e, 0x303e,
0x387e, 0x40fe, 0x49fe, 0x49ff
```

25.18.2 VLC table for MPEG1 dct_dc_size_chrominance, 12 entries

```
0x1000, 0x1001, 0x1002, 0x1806, 0x200e, 0x281e, 0x303e, 0x387e,
0x40fe, 0x49fe, 0x53fe, 0x53ff
```

25.18.3 VLC table for MPEG1 DCT coefficients table zero, 112 entries

```
0x0000, /* dummy code */
0x1003, 0x2004, 0x2805, 0x3806, 0x4026, 0x4021, 0x500a, 0x601d,
0x6018, 0x6013, 0x6010, 0x681a, 0x6819, 0x6818, 0x6817, 0x701f,
0x701e, 0x701d, 0x701c, 0x701b, 0x701a, 0x7019, 0x7018, 0x7017,
0x7016, 0x7015, 0x7014, 0x7013, 0x7012, 0x7011, 0x7010, 0x7818,
0x7817, 0x7816, 0x7815, 0x7814, 0x7813, 0x7812, 0x7811, 0x7810,
0x1803, 0x3006, 0x4025, 0x500c, 0x601b, 0x6816, 0x6815, 0x781f,
0x781e, 0x781d, 0x781c, 0x781b, 0x781a, 0x7819, 0x8013, 0x8012,
0x8011, 0x8010, 0x2005, 0x3804, 0x500b, 0x6014, 0x6814, 0x2807,
0x4024, 0x601c, 0x6813, 0x2806, 0x500f, 0x6012, 0x3007, 0x5009,
0x6812, 0x3005, 0x601e, 0x8014, 0x3004, 0x6015, 0x3807, 0x6011,
0x3805, 0x6811, 0x4027, 0x6810, 0x4023, 0x801a, 0x4022, 0x8019,
0x4020, 0x8018, 0x500e, 0x8017, 0x500d, 0x8016, 0x5008, 0x8015,
0x601f, 0x601a, 0x6019, 0x6017, 0x6016, 0x681f, 0x681e, 0x681d,
0x681c, 0x681b, 0x801f, 0x801e, 0x801d, 0x801c, 0x801b
```

25.18.4 VLC table for MPEG2 DCT coefficients table one, 112 entries

```
0x0000, /* dummy code */
0x1002, 0x1806, 0x2007, 0x281c, 0x281d, 0x3005, 0x3004, 0x387b,
0x387c, 0x4023, 0x4022, 0x40fa, 0x40fb, 0x40fe, 0x40ff, 0x701f,
0x701e, 0x701d, 0x701c, 0x701b, 0x701a, 0x7019, 0x7018, 0x7017,
0x7016, 0x7015, 0x7014, 0x7013, 0x7012, 0x7011, 0x7010, 0x7818,
0x7817, 0x7816, 0x7815, 0x7814, 0x7813, 0x7812, 0x7811, 0x7810,
0x1802, 0x2806, 0x3879, 0x4027, 0x4020, 0x6816, 0x6815, 0x781f,
0x781e, 0x781d, 0x781c, 0x781b, 0x781a, 0x7819, 0x8013, 0x8012,
0x8011, 0x8010, 0x2805, 0x3807, 0x40fc, 0x500c, 0x6814, 0x2807,
0x4026, 0x601c, 0x6813, 0x3006, 0x40fd, 0x6012, 0x3007, 0x4804,
0x6812, 0x3806, 0x601e, 0x8014, 0x3804, 0x6015, 0x3805, 0x6011,
0x3878, 0x6811, 0x387a, 0x6810, 0x4021, 0x801a, 0x4025, 0x8019,
0x4024, 0x8018, 0x4805, 0x8017, 0x4807, 0x8016, 0x500d, 0x8015,
0x601f, 0x601a, 0x6019, 0x6017, 0x6016, 0x681f, 0x681e, 0x681d,
0x681c, 0x681b, 0x801f, 0x801e, 0x801d, 0x801c, 0x801b
```

25.18.5 offset_levelmax table for MPEG1/2 DCT coefficients table zero and one, 64 entries

0x0028, 0x2812, 0x3a05, 0x3f04, 0x4303, 0x4603, 0x4903, 0x4c02,
0x4e02, 0x5002, 0x5202, 0x5402, 0x5602, 0x5802, 0x5a02, 0x5c02,
0x5e02, 0x6001, 0x6101, 0x6201, 0x6301, 0x6401, 0x6501, 0x6601,
0x6701, 0x6801, 0x6901, 0x6a01, 0x6b01, 0x6c01, 0x6d01, 0x6e01,
0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000

25.19 VLD Tables used**25.19.1 VLD decoding table for MPEG1/2 – dct_dc_size_luminance (12 entries, nrbits_dc_y=9, oneleading_dc_y=1, size_offset_global_dc_y = 0, symbol is size)**

0x480b, 0x480a, 0x4009, 0x3808, 0x3007, 0x2806, 0x2005, 0x1804, 0x1803, 0x1800, 0x1002, 0x1001

25.19.2 VLD decoding control table for MPEG1/2 dct_dc_size_luminance (10 entries, nrbits_dc_y=9, oneleading_dc_y=1, size_offset_global_dc_y = 0)

0x0000, 0x0000, 0x0801, 0x1002, 0x1803, 0x2004, 0x2805, 0x3006, 0x3006, 0x3808

25.19.3 VLD decoding table for MPEG1/2 dct_dc_size_chrominance (12 entries, nrbits_dc_uv=10, oneleading_dc_uv=1, size_offset_global_dc_uv = 0, symbol is size)

0x500b, 0x500a, 0x4809, 0x4008, 0x3807, 0x3006, 0x2805, 0x2004, 0x1803, 0x1002, 0x1001, 0x1000

25.19.4 VLD decoding control table for MPEG1/2 dct_dc_size_chrominance (11 entries, nrbits_dc_uv=10, oneleading_dc_uv=1, size_offset_global_dc_uv = 0)

0x0000, 0x0000, 0x0801, 0x1002, 0x1803, 0x2004, 0x2805, 0x3006, 0x3807, 0x4008, 0x4008

25.19.5 VLD decoding table for MPEG1/2 DCT coefficients table zero (151 entries, nrbits_ac0=16, oneleading_ac0=0, offset_global_ac0 = 0, symbol is 64*run + level, symbol 0x58 denotes ESCAPE, entry value 0x07d0 means EOB)

0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x0000, 0x8052, 0x8051, 0x8050, 0x804f, 0x8183, 0x8402, 0x83c2, 0x8382, 0x8342, 0x8302, 0x82c2, 0x87c1, 0x8781, 0x8741, 0x8701, 0x86c1, 0x7828, 0x7827, 0x7826, 0x7825, 0x7824, 0x7823, 0x7822, 0x7821, 0x7820, 0x784e, 0x784d, 0x784c, 0x784b, 0x784a, 0x7849, 0x7848, 0x701f, 0x701e, 0x701d, 0x701c, 0x701b, 0x701a, 0x7019, 0x7018, 0x7017, 0x7016, 0x7015, 0x7014, 0x7013, 0x7012, 0x7011, 0x7010, 0x6a82, 0x6a42, 0x6943, 0x68c4, 0x6885, 0x6847, 0x6846, 0x680f, 0x680e, 0x680d, 0x680c, 0x6e81, 0x6e41, 0x6e01, 0x6dc1, 0x6d81, 0x600b, 0x6202, 0x6103, 0x600a, 0x6084, 0x61c2, 0x6541, 0x6501, 0x6009, 0x64c1, 0x6481, 0x6045, 0x60c3, 0x6008, 0x6182, 0x6441, 0x5401, 0x5142, 0x5007, 0x5083, 0x5044, 0x53c1, 0x5381, 0x5102, 0x3058, 0x3882, 0x3a41, 0x3804, 0x3a01, 0x31c1, 0x3181, 0x3042, 0x3141, 0x4341, 0x4006, 0x4301, 0x42c1, 0x40c2, 0x4043, 0x4005, 0x4281, 0x2803, 0x2803, 0x2803, 0x2803, 0x2803, 0x2803, 0x2803, 0x2803, 0x2901, 0x2901, 0x2901, 0x2901, 0x2901, 0x2901, 0x2901, 0x2901, 0x28c1, 0x28c1, 0x28c1, 0x28c1, 0x28c1, 0x28c1, 0x28c1, 0x28c1, 0x2002, 0x2081, 0x1841, 0x1841, 0x17d0, 0x1001
--

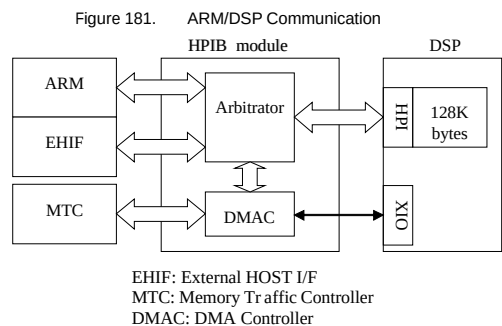
26 ARM-DSP Communication

26.1 Introduction

The ARM's DSP Controller in the DM270 controls data transfer and interrupts between the ARM and DSP, via the Host Port Interface Bridge (HPIB). DM270 has the Texas Instruments TMS320VC5409-equivalent DSP built into it. This DSP has an Host Port Interface (HPI), which allows communication between a host CPU, such as the ARM7 core used in the DM270.

As shown in Figure 181, the ARM and DSP can communicate through the HPI. The ARM (or external host) can access the DSP memory through a 128-Kbyte window into the DSP memory map. Software designer can define a communication protocol and data structures understood by the ARM and DSP to pass command requests, acknowledgements, and datagram between the ARM and DSP. HPIB module also has DMA controller inside, it enables data transfer from/to DSP internal memory to/from external memory (SDRAM, Flash ROM etc.) through memory traffic controller (MTC) module. This DMA controller is managed by DSP through XIO bus of DSP.

NOTE: This HPIB DMAC is separate from the DMA controller built into the shared memory interface of the DSP subsystem (image buffer)



26.2 HPI-16 mode

Normally, the HPI in the TMS320VC5409 DSP can be used in two modes, HPI-8 mode and HPI-16 mode. However the TMS320VC5409-equivalent DSP built into the DM270 can be used only in non-multiplexed HPI16 mode. In non-multiplexed HPI-16 mode, the ARM can read and write the DSP memory through a 16-bit bus. However, in this mode handshaking is not possible like it is in HPI-8 mode. Although handshaking is not possible, the ARM can still use the INT0 and NMI interrupts to interrupt the DSP, and the DSP can use an I/O port mapped to DSP address 0xFFFF to interrupt the ARM. For additional information on non-multiplexed HPI16 mode, refer to TMS320C54x DSP Reference Set, Volume 5: Enhanced Peripherals.

26.3 Memory Access

The DSP internal memory (32-Kwords of DARAM, 32-Kwords of SARAM) can be read and written by the ARM or an external host through the HPI-Bridge.

Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0004:0000-0005:FFFF in the ARM memory map.

Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM(data) and 0xC000-0xFFFFh of DSP SARAM(PAGE1:program) correspond to addresses 0002:0000-0003:FFFF in the external host memory map.

The memory maps of the ARM and external HOST are shown in Table 65. and Table 66. , respectively. When the endianness of the ARM and DSP do not match, EXCHG in HPIBCTL – DSPC HPIB Control Register can be used to swap the upper and lower 8-bits of data transferred between the ARM and DSP.

Table 65. ARM Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x40000-0x4FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x50000-0x57FFF	0x8000-0xBFFF	Data	32k
0x58000-0x5FFFF	0xC000-0xFFFF	Program(Page1)	32k

Table 66. External HOST Memory Map

ARM address(byte)	DSP address(word)	DSP memory	Size(byte)
0x20000-0x2FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x30000-0x37FFF	0x8000-0xBFFF	Data	32k
0x38000-0x3FFFF	0xC000-0xFFFF	Program(Page1)	32k

26.4 Interrupts between ARM and DSP

The sections that follow explain how the host and the DSP interrupt each other.

26.4.1 ARM to DSP Interrupt

The host (ARM) can interrupt the DSP by writing "0" to bit DINT0 or by writing "0" to bit HPNMI of HPIBCTL – DSPC HPIB Control Register. This generates INTO or NMI to DSP respectively. After an interrupt is triggered to DSP, ARM must write "1" to DINT0 or HPNMI of HPIBCTL – DSPC HPIB Control Register respectively. A low level signal having width of atleast two DSP clocks is detected as an interrupt by the DSP

26.4.2 DSP to ARM Interrupt

The DSP can interrupt the ARM through the memory mapped I/O port, 0xFFFF. The DSP's HINT pin is not used to interrupt the ARM, since the HPI is operating in a mode that cannot control the HINT pin (non-multiplexed HPI16 mode). The DSP to ARM interrupt signal is connected to the ARM's interrupt controller and corresponds to INT11 in the interrupt controller.

26.5 Boot loader

26.5.1 ARM boot sequence

Upon power up and reset, the ARM program counter is reset to 0x0000, which resides at the start of ARM Internal RAM. The ARM reset vector is hard coded in ROM at this location with an instruction to branch to 0x0100:0000 (start of the CS0 address space). The ARM should then execute a boot sequence from CS0 region to start the DM270 system.

The ARM boot sequence should initialize the different peripheral registers, such as the register for the clock controller module, the SDRAM controller, etc.

(See also section "ARM Boot-up Modes")

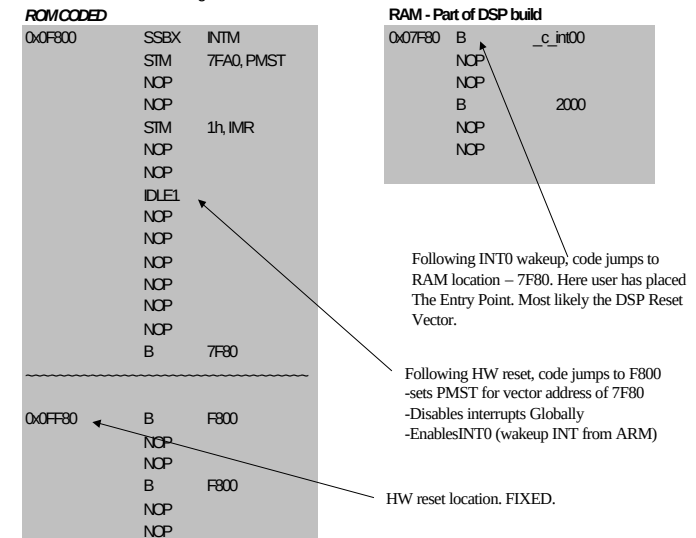
Also, the ARM boot phase should reset the DSP via the DRST bit of HPIBCTL – DSPC HPIB Control Register. When done, the ARM should take the DSP out of reset. At this stage the DSP begins execution at 0xFF80, a hard coded address in ROM, which places the DSP into IDLE 1 mode (a power down mode). This gives the ARM the opportunity to download the DSP code by using the HPI-16 interface. After it completes downloading the DSP image, the ARM can send an interrupt to the DSP, which wakes it up from IDLE 1 mode and jump to address 0x7F80 where it starts running its application code loaded by the ARM.

26.5.2 DSP Boot Sequence

On the 'C54x DSP, the reset vector is located at address 0x0FF80. Within the DM270, this memory location has been ROM'ed with a small boot loader. Figure 182 describes this process. It is achieved in 6 steps:

- The DSP goes out of reset and load its program counter (PC) register with 0x0FF80
- The ROM code found at this location branches the DSP to address 0x0F800 where the initialization routine resides.
- At address 0x0F800, the DSP status register PMST is initialized to move the vector table to 0x07F80, all the interrupts is disabled except for INTO and finally, the DSP is sent to a power down mode (Idle 1).
- While in this mode, the ARM loads the DSP internal 64k words of Program/Data memory with the DSP code. The code being loaded from the ARM side is not predefined
- When the ARM is done downloading the DSP code, it wakes it up from IDLE mode by asserting INTO.
- The DSP then branches to address 0x7F80 where the new interrupt vector table is located. The ARM should have loaded this location with at least a branch to the start of your code.

Figure 182. Host Interface Mode



26.6 ARM-Side HPI Control and Status Registers

On the ARM side, the DSP Controller module has two registers, listed in Table 67, that control the HPIB.

Address	Register	Description
0x30600	HPIBCTL	HPIB Control Register
0x30602	HPIBSTAT	HPIB Status Register

The registers HPIBCTL – DSPC HPIB Control Register and HPIBSTAT – DSPC HPIB Status Register contain DSP and HPIB control and status bits, which facilitate communication between the ARM and DSP. HPIBCTL – DSPC HPIB Control Register is used to control various DSP signals and the HPIB as well as generate interrupts to the DSP. HPIBSTAT – DSPC HPIB Status Register is a read only register that provides status information on the various DSP signals and HPI bridge control signals and interrupts.

Figure 183 shows ARM & DSP Communication Block Diagram. Table 68. lists DSP signals that have corresponding control and/or status bits in HPIBCTL and/or HPIBSTAT. Refer to the TMS320VC5409 Datasheet for detailed information on each signal listed in Table 68.

Figure 183. ARM & DSP Communication Block Diagram

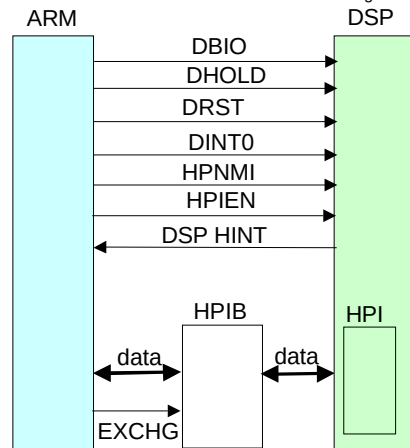


Table 68. DSP Signals Corresponding to Bits in HPIBCTL and/or HPISTAT

Pin Name	Module name
BIO	Branch control. A branch can be conditionally executed when BIO is active. If low, the processor executes the conditional instruction. For the XC instruction, the BIO condition is sampled during the decode phase of the pipeline; all other instructions sample BIO during the read phase of the pipeline.
HOLDN	HOLD is asserted to request control of the address, data, and control lines. When acknowledged by the C54x, these lines go into the high-impedance state.
HOLDA	Hold acknowledge. HOLDA indicates that the 5409 is in a hold state and that the address, data, and control lines are in the high-impedance state, allowing the external memory interface to be accessed by other devices. HOLDA also goes into the high-impedance state when OFF is low.
RSN	Reset. RS causes the DSP to terminate execution and causes a reinitialization of the CPU and peripherals. When RS is brought to a high level, execution begins at location 0FF80h of program memory. RS affects various registers and status bits.
INT0	External user interrupt. INT0 is prioritized and maskable by the interrupt mask register and the interrupt mode bit. INT0 can be polled and reset by way of the interrupt flag register.
HPNMI	Nonmaskable interrupt. NMI is an external interrupt that cannot be masked by way of the INTM or the IMR. When NMI is activated, the processor traps to the appropriate vector location.
HPIEN	HPIENA must be driven high during reset to enable the HPI. An internal pulldown resistor is always active and the HPIENA pin is sampled on the rising edge of RS. If HPIENA is left open or is driven low during reset, the HPI module is disabled. Once the HPI is disabled, the HPIENA pin has no effect until the 5409 is reset.
XF	External flag output (latched software-programmable signal). XF is set high by the SSBX XF instruction, set low by the RSBX XF instruction or by loading ST1. XF is used for signaling other processors in multiprocessor configurations or used as a general-purpose output pin. XF goes into the high-impedance state when OFF is low, and is set high at reset.
HINT	Host Port Interrupt. Status bit that shows the state of the DSP I/O port corresponding to bit zero at address 0xFFFF. This I/O port is internally tied to the ARM's interrupt controller, and is used to allow the DSP to interrupt the ARM.

26.7 ARM Subsystem - DSP Controller (DSPC)

Address	Register	Description
0003:0600	HPIBCTL	HPIB Control Register
0003:0602	HPIBSTAT	HPIB Status Register

26.8 ARM Subsystem - DSP Controller Registers

26.8.1 HPIBCTL – DSPC HPIB Control Register

HPIBCTL				0003:0600				offset: 0x00												default: 0x0789			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name	RSV	RSV	RSV	RSV	RSV	DBIO	DHOLD	DRST	DINT0	RSV	EXCHG	RSV	HPNMI	RSV	RSV	HPIEN							
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	-	R/W	-	R/W	-	-	R/W							
Default	-	-	-	-	-	1	1	1	1	-	0	-	1	-	-	1							

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10	DBIO	1	R/W	<u>DSP B10 Control</u> » 0: B10 asserted « » 1: Normal « DBIO controls the state of the DSP's B10n pin. When the DSP is the B10 state, it can branch and control the program.
9	DHOLD	1	R/W	<u>DSP Hold Request Control</u> » 0: Hold « » 1: Normal « DHOLD controls the state of the DSP's HOLDn pin.
8	DRST	1	R/W	<u>DSP Reset</u> » 0: DSP reset « » 1: Normal « DRST controls the state of the DSP's RSn pin, and is used to reset the DSP.
7	DINT0	1	R/W	<u>DSP INT0 Control (ARM-To-DSP Interrupt)</u> » 0: INT0 « » 1: Normal « DINT0 controls the state of the DSP's INT0n pin, and it is used interrupt the DSP from the ARM. The INT0 interrupt is generated in the DSP when the the ARM sets this bit to "0". After the interrupt is generated, the ARM must return this bit to "1".
6	RSV			Reserved
5	EXCHG	0	R/W	<u>Byte Exchange</u> » 0: No exchange « » 1: Exchange upper/lower bytes of 16-bit HPI16 transfers « The exchange is done when data is transferred in both directions ARM to DSP and DSP to ARM (write and read).
4	RSV			Reserved
3	HPNMI	1	R/W	<u>HPI NMI Control (NMI Interrupt)</u> » 0: NMI « » 1: Normal « HPNMI controls the state of the DSP's NMIn pin, and it is used to interrupt the DSP from the ARM. The NMI interrupt is generated in the DSP when the ARM sets this bit to '0'. After the NMI interrupt is generated, the ARM must return this bit to '1'.
2:1	RSV			Reserved
0	HPIEN	1	R/W	<u>Enables HPI Control</u> » 0: Disabled « » 1: Enabled « HPIEN controls the DSP's HPIEN pin. When HPIEN is disabled, all HPI functions are invalid.

26.8.2 HPIBSTAT – DSPC HPIB Status Register

HPIBSTAT	0003:0602													offset:	0x02			default:	0x1101		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Name	RSV	RSV	RSV	DXF	RSV	RSV	RSV	HOLDA	RSV	RSV	RSV	RSV	RSV	RSV	RSV	HINT					
R/W	-	-	-	R	-	-	-	R	-	-	-	-	-	-	-	R					
Default	-	-	-	1	-	-	-	1	-	-	-	-	-	-	-	1					

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	DXF	1	R	<u>DSP_XF Status</u> Indicates status of DSP XF signal output
11:9	RSV			Reserved
8	HOLDA	1	R	<u>DSP_Hold Acknowledge</u> » 0: DSP hold status « » 1: Normal « Indicates status of the DSP's HOLDAn signal output. After a hold request is sent to the DSP by DHOLD in HPIBCLT, the acknowledgement can be checked by this bit.
7:1	RSV			Reserved
0	HINT	1	R	<u>Host Port Interrupt Status</u> » 0: DSP sent interrupt request to host « » 1: Normal « Indicates status of DSP HINT signal output from DSP to host.

26.9 DSP Subsystem - HPIB DMAC

DM270 has capability to perform DMA from DSP internal memory to SDRAM, EMIF via the Memory Traffic Controller. This DMA is controlled by the DSP

NOTE: This HPIB DMAC is separate from the DMA controller built into the shared memory interface of the DSP subsystem (image buffer).

NOTE: This HPIB DMA would be slower than ImageBuffer DMA since DMA access is not done in bursts of 32bytes and hence HPIB DMA takes more cycles to complete.

To transfer data using the HPIB DMA controller, the DSP should setup the DMA transfer registers as follows:

- o **IMPORTANT:** Reserve Memory Traffic Controller(MTC) DMA Channel 1 or Channel 2, for HPIB DMA using register REFCTL(0x309A8), field SDMA1 or SDMA2. (See chapter MTC, section SDRAM Controller registers)
- o Perform the following steps depending on the SDRAM or EMIF data source/destination device:
 - n To transfer data between SDRAM, set the SDRAM source/destination address in [DMASADL\(0x80\)](#) and [DMASADH\(0x81\)](#)
 - n To transfer data between EMIF region CS0, CS3, or CS4; set the EMIF destination/source address in [DMASADL\(0x80\)](#) and [DMASADH\(0x81\)](#)
 - n To transfer data between EMIF region CS1 or CS2, set the EMIF destination/source address in the EMIF module registers IMGDSPADDH (0x30A46) and IMGDSPADDL (0x30A48).
 - n For EMIF data transfer, select the EMIF source/destination region (CS0/CS1/CS2/CS3/CS4) by configuring DDST in IMGDSPDEST (0x30A34)
 - n Select whether SDRAM or EMIF would be used for data transfer using DEST bit in [DMACTL\(0x85\)](#)
 - n **NOTE:** SDRAM, EMIF address is specified as byte offset from start of respective SDRAM, EMIF region. See EMIF section for EMIF register details. Address must be 4 byte aligned

26.11.2 DMASADH - HPIB DMA SDRAM Address High Register

[illegible][illegible]

26.11.3 DMADADL - HPIB DMA DSP Address Low Register

[illegible][illegible]

26.11.4 DMADADH - HPIB DMA DSP Address Start Register

[illegible][illegible]

26.11.5 DMASZ - HPIB DMA transfer size

[illegible][illegible]

26.11.6 DMACTL - HPIB DMA Control

DMAC TL	IO		0085		offset:		0x0A										default:		0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	RSV	RSV	SWAP[1]	SWAP[0]	RSV	RSV	RSV	DEST	RSV	RSV	RSV	DIR				
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	R/W	-	-	-	-	R/W			
Default	-	-	-	-	-	-	0	0	-	-	-	0	-	-	-	-	0			

[illegible]

26.11.7 DMATRG - HPIB DMA Trigger

DMATRG		IO		0086		offset:		0x0C										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RUN			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0			

[illegible]

26.11.8 DMAREST - HPIB DMA Remaining bytes

[illegible][illegible]

27 Appendix

27.1 Appendix A : TMX320DM270GHK Pin Assignment

PIN#	CSP BALL #	Pin Name	Group	Type	During reset Nrm	During reset Eho	Pin Function	Pull up/down
1	B2	DSP_BDX0	Audio I/F	O	Z	Z	DSP Serial-port Transmit 0 OPEN when not in use. Pull up or pull down, however, in case that level fixing during HiZ is needed with audio CODEC to be connected.	-
2	B1	DSP_BCLKX0	Audio I/F	I/O	I	I	DSP Transmit Clock 0 OPEN when output is set. Pull up or pull down, however, in case that CODEC side is in HiZ when input is set. Pull up or pull down also when not in use. Note that tying this pin directly to GND means that output pin is tied directly to GND in case that this pin is output when McBSP is set.	-
3	E5	DSP_BFSX0	Audio I/F	I/O	I	I	DSP Frame Sync Pulse Transmit 0 OPEN when output is set. Pull up or pull down, however, in case that CODEC side is in HiZ when input is set. Pull up or pull down also when not in use. Note that tying this pin directly to GND means that output pin is tied directly to GND in case that this pin is output when McBSP is set.	-
4	C2	DSP_BDR0	Audio I/F	I/O	I	I	DSP Serial Data Receive 0 Input only. Pull up, pull down or tie directly to GND when not in use.	-
5	D3	DSP_BCLKR0	Audio I/F	I/O	I	I	DSP Receive Clock 0 OPEN when output is set. Pull up or pull down, however, in case that CODEC side is in HiZ when input is set. Pull up or pull down also when not in use. Note that tying this pin directly to GND means that output pin is tied directly to GND in case that this pin is output when McBSP is set.	-
6	F6	DSP_BFSR0	Audio I/F	I/O	I	I	DSP Frame Sync Pulse Receive 0 OPEN when output is set. Pull up or pull down, however, in case that CODEC side is in HiZ when input is set. Pull up or pull down also when not in use. Note that tying this pin directly to GND means that output pin is tied directly to GND in case that this pin is output when McBSP is set.	-
7	C1	DVDD1	Power	-	-	-	VDD for I/O	-
8	D2	DSP_CLKS0	Audio I/F	I	I	I	DSP Serial Clock Source 0 CLKS input for McBSP0. This is for input only. Pull up or tie directly to GND when not in use.	-
9	H8	DGND1	Power	-	-	-	GND for I/O	-
10	E3	RXD0	Communication I/F	I	I	I	UART RXD 0 / AIMSIF Data in Tie directly to GND or pull up with R when not in use.	-
11	F5	TXD0/SDO	Communication I/F	O	H	H	UART TXD 0 / AIMSIF Data out Output pin. This pin is never in HiZ.	-
12	D1	SDEN0	Communication I/F	I/O	I	I	Serial Data Enable 0 / AIMSIF Enable I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
13	E2	SCLK0	Communication I/F	I/O	I	I	Serial Clock 0 / AIMSIF Clock I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
14	E1	SDIO	Communication I/F	I	I	I	Serial Data Input 0 Tie directly to GND or pull up with R when not in use.	-
15	F3	SDO0	Communication I/F	O	L	L	Serial Data Output 0 Output pin. This pin is never in HiZ.	-
16	F1	GIO33	General I/O	I/O	I	I	GIO_EXT_LINE_ID NTSC/PAL Encoder	-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset Nrm	During reset Eho	Pin Function	Pull up/down
							These GIO33-GIO1 pins are input by default. Pull up or pull down with R. Do not tie GIO pin directly to VDD or GND because GIO may be output depending on program.	-
17	G5	GIO32	General I/O	I/O	I	I	GIO_SDA I2C Data	-
18	F2	GIO31	General I/O	I/O	I	I	GIO_SCL I2C Clock	-
19	H7	GIO30	General I/O	I/O	I	I	GIO_PWM1 Pulse Width Modulation	-
20	G3	GIO29	General I/O	I/O	I	I	GIO_PWM0 Pulse Width Modulation	-
21	G1	CVDD1	Power	-	-	-	VDD for core	-
22	H10	CGND1	Power	-	-	-	GND for core	-
23	H5	GIO28	General I/O	I/O	I	I	GIO_FIELD Field (NTSC/PAL Enc.)	-
24	G2	GIO27	General I/O	I/O	I	I	GIO_CCDC_SHUT CCD CCDC_SHUT	-
25	J8	GIO26	General I/O	I/O	I	I	GIO_C_WEN CCD In Write Enable	-
26	H3	GIO25	General I/O	I/O	I	I	GIO_MIRQ IRQ Monitor MRST Master Reset from WDT	-
27	H1	GIO24	General I/O	I/O	I	I	GIO_TXD1 UART TXD 1	-
28	H2	GIO23	General I/O	I/O	I	I	GIO_RXD1 UART RXD 1	-
29	J6	GIO22	General I/O	I/O	I	I	GIO_SDEN1 I/O Serial Data Enable 1	-
30	J5	GIO21	General I/O	I/O	I	I	GIO_SCLK1 I/O Serial Clock 1	-
31	J3	GIO20	General I/O	I/O	I	I	GIO_SDI1 I Serial Data input 1	-
32	J1	GIO19	General I/O	I/O	I	I	GIO_SDO1 O Serial Data Output 1	-
33	J2	GIO18	General I/O	I/O	I	I	GIO_CLKOUT2 O General Clock Out 2	-
34	J7	GIO17	General I/O	I/O	I	I	GIO_CLKOUT1 O General Clock Out 1	-
35	K5	GIO16	General I/O	I/O	I	I	GIO_CLKOUT0 O General Clock Out 0	-
36	K3	GIO15	General I/O	I/O	I	I	GIO_INT15 I External Interrupt DSP_XF O External Flag	-
37	K1	DVDD2	Power	-	-	-	VDD for I/O	-
38	K2	GIO14	General I/O	I/O	I	I	GIO_INT14 I External Interrupt DSP_CLKS1 I DSP Serial Clock 1	-
39	H11	DGND2	Power	-	-	-	GND for I/O	-
40	K7	GIO13	General I/O	I/O	I	I	GIO_INT13 I External Interrupt DSP_BDX1 O DSP Serial-port Transmit 1	-
41	L1	GIO12	General I/O	I/O	I	I	GIO_INT12 I External Interrupt DSP_BCLKX1 I/O DSP Transmit Clock 1	-
42	L3	GIO11	General I/O	I/O	I	I	GIO_INT11 I External Interrupt DSP_BFSX1 I/O DSP Frame Sync Pulse Transmit 1	-
43	L5	GIO10	General I/O	I/O	I	I	GIO_INT10 I External Interrupt DSP_BDR1 I/O DSP Serial Data Receive	-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function						Pull up/down	
					Nrml	Eho st								
44	L2	GIO9	General I/O	I/O	I	I	GIO	INT9	I	External Interrupt	DSP_BCLKR1	I/O	DSP Receive Clock 1	-
45	L6	GIO8	General I/O	I/O	I	I	GIO	INT8	I	External Interrupt	DSP_BFSR1	I/O	DSP Frame Sync Pulse Receive 1	-
46	M1	GIO7	General I/O	I/O	I	I	GIO	INT7	I	External Interrupt				-
47	M3	GIO6	General I/O	I/O	I	I	GIO	INT6	I	External Interrupt				-
48	M2	GIO5	General I/O	I/O	I	I	GIO	INT5	I	External Interrupt				-
49	L7	GIO4	General I/O	I/O	I	I	GIO	INT4	I	External Interrupt				-
50	N1	GIO3	General I/O	I/O	I	I	GIO	INT3	I	External Interrupt				-
51	M5	GIO2	General I/O	I/O	I	I	GIO	INT2	I	External Interrupt				-
52	N3	GIO1	General I/O	I/O	I	I	GIO	INT1	I	External Interrupt				-
53	N2	WAKEUP		I	I	I								-
54	P1	CVDD2	Power	-	-	-	VDD for core							-
55	L8	RSN	Misc.	I	I	I	Reset for Main Land Reset input signal. By applying L level, device comes to reset state.							-
56	P2	PWDWN	Sleep Controller	I	I	I	Power Down Mode (with built-in pull-down resistance)							Dn
57	H12	CGND2	Power	-	-	-	GND for core							-
58	P3	TCK	Misc.	I	I	I	JTAG Clock (with built-in pull-up resistance)							Up
59	N5	TDI	Misc.	I	I	I	JTAG Data Input (with built-in pull-up resistance)							Up
60	R1	TMS	Misc.	I	I	I	JTAG Test Mode Select (with built-in pull-up resistance)							Up
61	M7	TRST	Misc.	I	I	I	JTAG Test Reset (with pull-down resistance)							Dn
62	R2	TEST0	Misc.	I	I	I	Test Input0 (with built-in pull-down resistance)							Dn
63	R3	TEST1	Misc.	I	I	I	Test Input1 (with built-in pull-down resistance)							Dn
64	T1	TEST2	Misc.	I	I	I	Test Input2 (with built-in pull-down resistance)							Dn
65	T2	TEST3	Misc.	I	I	I	Test Input3 (with built-in pull-down resistance)							Dn
66	U1	SCANEN	Misc.	I	I	I	Test Scan Enable (with built-in pull-down resistance)							Dn
67	P5	TDO	Misc.	O	Z	Z	JTAG Data Output							-
68	T3	VCLK	Video Output	O	L	L	Video Out Clock							-
69	V1	HSYNC	Video Output	O	L	L	H-Sync for RGB output							-
70	U2	VSNC	Video Output	O	L	L	V-Sync/Composite - Sync output							-
71	N7	YOUT7	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-
72	U3	YOUT6	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-
73	V2	YOUT5	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-
74	W2	YOUT4	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-
75	V3	YOUT3	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-
76	U4	YOUT2	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.							-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
77	W3	YOUT1	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
78	V4	YOUT0	Video Output	I/O	L	L	Y OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
79	P6	DVDD3	Power	-	-	-	VDD for I/O	-
80	J9	DGND3	Power	-	-	-	GND for I/O	-
81	W4	COUT7	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
82	U5	COUT6	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
83	V5	COUT5	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
84	R6	COUT4	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
85	W5	COUT3	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
86	R7	COUT2	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
87	U6	COUT1	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
88	V6	COUT0	Video Output	I/O	L	L	C OUT Signal I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
89	W6	CVDD3	Power	-	-	-	VDD for core	-
90	J10	CGND3	Power	-	-	-	GND for core	-
91	N8	YI7	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
92	R8	YI6	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
93	U7	YI5	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
94	V7	YI4	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
95	W7	YI3	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
96	N9	YI2	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
97	P9	YI1	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
98	U8	YI0	Video Input	I	I	I	Y IN Signal (with built-in pull-down resistance)	Dn
99	J11	DGND4	Power	-	-	-	GND for I/O	-
100	V8	PCLK	Video Input	I	I	I	Pixel Clock Pull up, pull down or tie directly to GND when not in use.	-
101	W8	DVDD4	Power	-	-	-	VDD for I/O	-
102	R9	HD	Video Input	I/O	I	I	H-Sync I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
103	M10	VD	Video Input	I/O	I	I	V-Sync I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
104	U9	FID	Video Input	I/O	I	I	Field Indicator I/O pin. Since this may be output, never fail to put an R when pull up or pull down.	-
105	V9	CI7	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
106	W9	CI6	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
107	N10	CI5	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
108	R10	CI4	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
109	U10	CI3	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
110	V10	CI2	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
111	W10	CI1	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
112	P12	CIO	Video Input	I	I	I	C IN Signal (with built-in pull-down resistance)	Dn
113	R11	PLLVD01	Power	-	-	-	VDD for PLL	-
114	U11	PLLGND1	Power	-	-	-	GND for PLL	-
115	J12	CGND4	Power	-	-	-	GND for core	-
116	V11	M48XO	Misc.	O	O	O	48MHz X'tal output OPEN when not in use.	-
117	W11	M48XI	Misc.	I/O	I	I	48MHz X'tal Input Pull down when not in use. When this pin goes to power -down, it comes to output and outputs L level. Therefore, do not tie directly to GND.	-
118	U12	CVDD4	Power	-	-	-	VDD for core	-
119	K8	DGND5	Power	-	-	-	GND for I/O	-
120	V12	SYSCLK	Misc.	I	I	I	External Clock Input Tie directly to GND or pull up with R when not in use.	-
121	N11	DVDD5	Power	-	-	-	VDD for I/O	-
122	W12	BTSEL	Misc.	I	I	I	ARM Boot Selector When boot from external memory connecting to SELF BOOTUP:FLSH_CE, tie directly this pin to GND or pull down with R. When boot from Serial/HOST BOOT UP: ARM internal memory, pull up this pin with R.	-
123	R12	FLSHSEL	Misc.	I	I	I	Flash ROM Bus Selector When FLASHROM is connected to ARM bus, tie directly this pin to GND or pull down with R. When FLASHROM is connected to SDRAM bus, pull up this pin with R.	-
124	U13	EXTHOST	Misc.	I	I	I	EMIF/EHIF Selector When only EMIF is used, tie directly this pin to GND or pull down with R. When external host interface EHIF is used, pull up this pin with R.	-
125	V13	IREF	Video DAC	I	I	I	DAC Current Control	-
126	W13	DAOUT	Video DAC	O	O	O	DAC Composite Output	-
127	W14	VSSA	Power	-	-	-	analog GND for DAC	-
128	V14	VDDA	Power	-	-	-	analog VDD for DAC	-
129	U14	BIAS	Video DAC	I	I	I	DAC Bias with Compensation Capacitor	-
130	W15	VREFIN	Video DAC	I	I	I	DAC Reference Voltage	-
131	R13	VSS3V	Power	-	-	-	digital GND for DAC	-
132	V15	VDD3V	Power	-	-	-	digital VDD for DAC	-
133	U15	USB_VDD	Power	-	-	-	VDD for USB	-
							Tie directly to GND when not in use.	-
134	W16	USB_DP	Communication I/F	I/O	I	I	USB Data Plus Tie directly to GND when not in use.	-
135	W17	USB_DM	Communication I/F	I/O	I	I	USB Data Minus Tie directly to GND when not in use.	-
136	V16	CVDD5	Power	-	-	-	VDD for core	-
137	K9	CGND5	Power	-	-	-	GND for core	-
138	R14	SDR_A14	SDRAM I/F	O	L	L	SDRAM Bank 0	-
139	U16	SDR_A13	SDRAM I/F	O	L	L	SDRAM Bank 1	-
140	V17	SDR_A12	SDRAM I/F	O	L	L	SDRAM Address (MSB)	-
141	W18	SDR_A11	SDRAM I/F	O	L	L	SDRAM Address	-
142	N12	SDR_A10	SDRAM I/F	O	L	L	SDRAM Address	-
143	U17	SDR_A9	SDRAM I/F	O	L	L	SDRAM Address	-
144	V18	SDR_A8	SDRAM I/F	O	L	L	SDRAM Address	-
145	N13	SDR_A7	SDRAM I/F	O	L	L	SDRAM Address	-
146	V19	SDR_A6	SDRAM I/F	O	L	L	SDRAM Address	-
147	U18	SDR_A5	SDRAM I/F	O	L	L	SDRAM Address	-
148	T17	SDR_A4	SDRAM I/F	O	L	L	SDRAM Address	-
149	U19	DVDDSD1	Power	-	-	-	VDD for SDRAM I/O	-
150	T19	DGNDSD1	Power	-	-	-	GND for SDRAM I/O	-
151	P14	SDR_A3	SDRAM I/F	O	L	L	SDRAM Address	-
152	T18	SDR_A2	SDRAM I/F	O	L	L	SDRAM Address	-
153	R17	SDR_A1	SDRAM I/F	O	L	L	SDRAM Address	-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
154	P15	SDR_A0	SDRAM I/F	O	L	L	SDRAM Address (LSB)	-
155	R19	SDR_DQ31	SDRAM I/F	I/O	I	I	SDRAM Data (MSB) (Pulled down by internal circuit only during reset.)	Dn
156	M12	SDR_DQ30	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
157	R18	SDR_DQ29	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
158	P17	SDR_DQ28	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
159	N15	SDR_DQ27	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
160	P19	SDR_DQ26	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
161	M13	SDR_DQ25	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
162	P18	SDR_DQ24	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
163	N17	CVDD6	Power	-	-	-	VDD for core	-
164	K10	CGND6	Power	-	-	-	GND for core	-
165	M15	SDR_DQ23	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
166	N19	SDR_DQ22	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
167	M14	SDR_DQ21	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
168	N18	SDR_DQ20	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
169	M17	SDR_DQ19	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
170	L13	SDR_DQ18	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
171	M19	SDR_DQ17	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
172	M18	SDR_DQ16	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
173	L15	SDR_DQ15	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
174	L17	SDR_DQ14	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
175	L18	SDR_DQ13	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
176	L19	SDR_DQ12	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
177	K18	DVDDSD2	Power	-	-	-	VDD for SDRAM I/O	-
178	K19	DGNDSD2	Power	-	-	-	GND for SDRAM I/O	-
179	K17	SDR_DQ11	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
180	K15	SDR_DQ10	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
181	K14	SDR_DQ9	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
182	J18	SDR_DQ8	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
183	J19	SDR_DQ7	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
184	J17	SDR_DQ6	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
185	K13	SDR_DQ5	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
186	J15	SDR_DQ4	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
187	H18	SDR_DQ3	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
188	H19	SDR_DQ2	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
189	H17	SDR_DQ1	SDRAM I/F	I/O	I	I	SDRAM Data (Pulled down by internal circuit only during reset.)	Dn
190	G19	SDR_DQ0	SDRAM I/F	I/O	I	I	SDRAM Data (LSB)	Dn
191	J13	CVDD7	Power	-	-	-	VDD for core	-
192	K11	CGND7	Power	-	-	-	GND for core	-
193	G18	SDR_RAS	SDRAM I/F	O	H	H	SDRAM RAS Strobe	-
194	H15	SDR_CAS	SDRAM I/F	O	H	H	SDRAM CAS Strobe	-
195	G17	SDR_WE	SDRAM I/F	O	H	H	SDRAM WE	-
196	F19	SDR_CLK	SDRAM I/F	I/O	L	L	SDRAM I/F	-
197	H14	SDR_CS	SDRAM I/F	O	H	H	SDRAM CS	-
198	F18	SDR_CKE	SDRAM I/F	O	H	H	SDRAM CKE	-
199	G15	SDR_DQMH	SDRAM I/F	O	H	H	DQMH for SDR_DQ[31:24]	-
200	F17	SDR_DQML	SDRAM I/F	O	H	H	DQMH for SDR_DQ[23:16]	-
201	E19	SDR_DQMLH	SDRAM I/F	O	H	H	DQMH for SDR_DQ[15:8]	-
202	E18	SDR_DQMLL	SDRAM I/F	O	H	H	DQMH for SDR_DQ[7:0]	-
203	H13	CFOE	External Memory I/F	O	H	H	CFC Output Enable	-
204	D19	CFE1	External Memory I/F	O	H	H	CFC Enable 1	-
205	E17	CFE2	External Memory I/F	O	H	H	CFC Enable 2	-
206	D18	CFWAIT	External Memory I/F	I	I	I	CFC Wait Signal Input (with built-in pull-up resistance)	Up
207	F15	CFRDY	External Memory I/F	I	I	I	CFC Ready (with built-in pull-up resistance)	Up

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
208	C19	IOIS16	External Memory I/F	I	I	I	CFC I/O Select (with built-in pull-up resistance)	Up
209	C18	CFIORD	External Memory I/F	O	H	H	CFC I/O Read Strobe	-
210	D17	CFIOWR	External Memory I/F	O	H	H	CFC I/O Write Strobe	-
211	B19	DVDD6	Power	-	-	-	VDD for I/O	-
212	K12	DGND6	Power	-	-	-	GND for I/O	-
213	B18	SD_DATA1	SDMS Card I/F	I/O	I	I	SD Data 1 Pull up with R.	-
214	C17	SD_DATA0	SDMS Card I/F	I/O	I	I	SD Data 0 Pull up with R.	-
215	A18	SD_CLK	SDMS Card I/F	O	L	L	SD Clock	-
216	B17	SD_CMD	SDMS Card I/F	I/O	I	I	SD Command Pull up with R.	-
217	G13	SD_DATA3	SDMS Card I/F	I/O	I	I	SD Data 3 Pull up with R.	-
218	C16	SD_DATA2	SDMS Card I/F	I/O	I	I	SD Data 2 Pull up with R.	-
219	A17	CVDD8	Power	-	-	-	VDD for core	-
220	L9	CGND8	Power	-	-	-	GND for core	-
221	B16	EM_WE	External Memory I/F	I/O	H	I	External Memory Write Enable Pull up with R.	-
222	A16	EM_OE	External Memory I/F	I/O	H	I	External Memory Output Enable Pull up with R.	-
223	F14	EM_WAIT	External Memory I/F	I/O	I	Z	External Memory Wait Pull up with R.	-
224	C15	EM_BEL	External Memory I/F	O	H	H	External Memory Byte Enable Low	-
225	E14	EM_BEH	External Memory I/F	O	H	H	External Memory Byte Enable High	-
226	B15	EM_CEO	External Memory I/F	I/O	H	I	External Memory Chip Enable #1 Pull up with R.	-
227	A15	EM_CEO	External Memory I/F	O	H	H	External Memory Chip Enable #2	-
228	C14	EM_RDWR	External Memory I/F	O	H	H	External Memory Read/Write	-
229	E13	DVDD7	Power	-	-	-	VDD for I/O	-
230	B14	MXO	Misc.	O	O	O	X'tal Output OPEN when not in use.	-
231	A14	MXI	Misc.	I/O	I	I	X'tal Input Pull down when not in use. When this pin goes to power -down, it comes to output and outputs L level. Therefore, do not tie directly to GND.	-
232	L10	DGND7	Power	-	-	-	GND for I/O	-
233	B13	PLLVD2	Power	-	-	-	VDD for PLL	-
234	C13	PLLGND2	Power	-	-	-	GND for PLL	-
235	G12	TEST4	Misc.	I	I	I	Test Input4 (INITZ) (with built-in pull-up resistance)	Up
236	A13	FLSH_CE	External Memory I/F	O	H	H	Flash ROM Chip Enable	-
237	E12	BUSWIDTH	External Memory I/F	I	I	I	8/16 bit select for FLSH_CEH Pull up when FLSH_CE is used in 16bit mode. Pull down or tie directly to GND when FLSH_CE is used in 8bit mode.	-
238	B12	CVDD9	Power	-	-	-	VDD for core	-
239	L11	CGND9	Power	-	-	-	GND for core	-
240	C12	ARM_D15	External Memory I/F	I/O	I	I	External Memory Data (MSB) Pull up or pull down with R.	-
241	G11	ARM_D14	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
242	A12	ARM_D13	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
243	F12	ARM_D12	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
244	B11	ARM_D11	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
245	C11	ARM_D10	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
246	A11	ARM_D9	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
247	E11	ARM_D8	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
248	A10	ARM_D7	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
249	B10	DVDD8	Power	-	-	-	VDD for I/O	-
250	L12	DGND8	Power	-	-	-	GND for I/O	-
251	C10	ARM_D6	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
251	C10	ARM_D6	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
252	G10	ARM_D5	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
253	E10	ARM_D4	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
254	A9	ARM_D3	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
255	B9	ARM_D2	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
256	C9	ARM_D1	External Memory I/F	I/O	I	I	External Memory Data Pull up or pull down with R.	-
257	F10	ARM_D0	External Memory I/F	I/O	I	I	External Memory Data (LSB) Pull up or pull down with R.	-
258	A8	CVDD10	Power	-	-	-	VDD for core	-
259	M8	CGND10	Power	-	-	-	GND for core	-
260	E9	ARM_A22	External Memory I/F	I/O	L	I	External Memory Address (MSB) Pull up or pull down with R.	-
261	B8	ARM_A21	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
262	C8	ARM_A20	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
263	F9	ARM_A19	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
264	A7	ARM_A18	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
265	G9	ARM_A17	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
266	B7	ARM_A16	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
267	C7	ARM_A15	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
268	E8	ARM_A14	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
269	A6	DVDD9	Power	-	-	-	VDD for I/O	-
270	M9	DGND9	Power	-	-	-	GND for I/O	-
271	H9	ARM_A13	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
272	B6	ARM_A12	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
273	C6	ARM_A11	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
274	A5	ARM_A10	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-
275	E7	ARM_A9	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	-

PIN#	CSP BALL #	Pin Name	Group	Type	During reset		Pin Function	Pull up/down
					Nrml	Eho st		
276	B5	ARM_A8	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
277	G8	ARM_A7	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
278	C5	ARM_A6	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
279	A4	CVDD11	Power	-	-	-	VDD for core	
280	M11	CGND11	Power	-	-	-	GND for core	
281	E6	ARM_A5	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
282	B4	ARM_A4	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
283	A3	ARM_A3	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
284	C4	ARM_A2	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
285	G7	ARM_A1	External Memory I/F	I/O	L	I	External Memory Address Pull up or pull down with R.	
286	B3	ARM_A0	External Memory I/F	I/O	L	I	External Memory Address (LSB) Pull up or pull down with R.	
287	A2	EMU0	Misc.	I/O	I	I	Emulator Interrupt0 Pull up with 4.7K?	-
288	C3	EMU1	Misc.	I/O	I	I	Emulator Interrupt1 Pull up with 4.7K?	-

Dn

Pulled down during reset

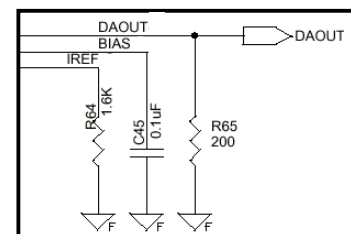
27.1.1 Unused Pin Connection Notes

Management for built-in DAC pins (VREF, BIAS, IREF and DAOUT)

- When DAC is not used, the pins are managed as below.

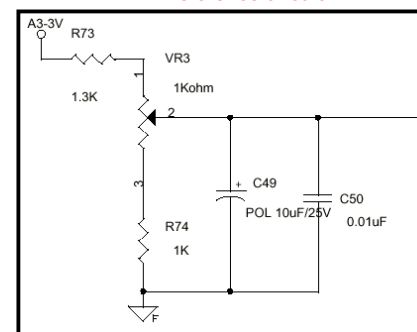
VREF=GND
BIAS=Open
IREF=Open
AOUT=Open

Management for pins (DAOUT, BIAS and IREF) of DAC reference circuit



Lowpass filter and amplifier are put in later level of this DAOUT in EVM.

VREF Reference circuit



Management for USB related pins (USB_DP, USB_DM, USB_VDD)

- When USB is not used, the pins are managed as below.

USB_DP=GND
USB_DM=GND
USB_VDD=GND

The power consumption can be reduced significantly due to these connections.

FYI, as the power supply for USB related circuit is separated from others, it causes no problem with USB_VDD=GND.

Management for clock related pins (MXI, MXO, M48XI, M48XO, SYSCLK)

In case that MXI is not used	In case that M48XI is not used	In case that SYS_CLK is not used
MXI: Pulled-down with R. (Do not tie directly to GND) MXO: Open	M48XI: Pulled-down with R. (Do not tie directly to GND) M48XO: Open	SYS_CLK: Tie directly to GND.

Management for GIO pins (GIO33-GIO1)

GIOs are set to input by default and have no internal pull-up/pull-down resistance. Therefore, it is necessary to put external resistance when pull up or pull down.

As these GIOs may be output depending on the setting of resistor, do not tie directly them to VDD or GND.

Management by binding them should be also avoided.

When directly connected GIO happens to be output by incorrect setting in software, the device can be damaged.

Management for ARM data bus pins (ARM_D15-ARM_D0)

Data buses of ARM are neither pulled up nor pulled down.

In case that they are reset for many hours and used in host mode, pull up them with R.

Management for TEST pins (TEST0, TEST1, TEST2, TEST3 and TEST4)

- Refer to the table below to conduct circuit connection for each pin.
Note that setting of TEST pins have an impact on operation of DM270GHK device.

TEST pin name	Description
TEST0	EMULATION MODE (Refer to the following table)
TEST1	EMULATION MODE (Refer to the following table)
TEST2	MXI/SYSCLK switching 0: Crystal oscillator connecting to MXI/MXO is used. 1: External clock connecting to SYSCLK is used.
TEST3	Open (DM270 has an internal pulled-DOWN register)
TEST4	Open (DM270 has an internal pulled-UP register)

- Emulation mode of DM270 (TEST0 and TEST1 are used.)

TEST1	TEST0	Emulation mode
0	0	ARM -> DSP (for multi CPU debugging)
0	1	ARM only
1	0	DSP only
1	1	Not available

Set the level of TEST0 and TEST1 pins in accordance with CCD (Code Composer Studio) debugging mode.

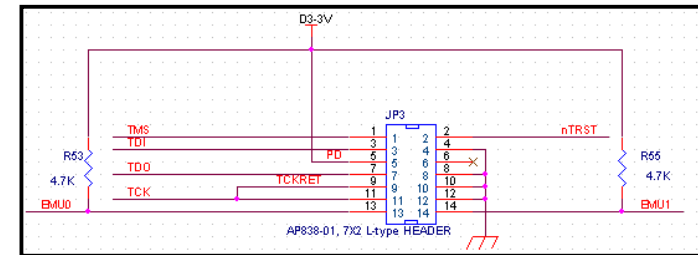
Management for JTAG related signal pins (TCK, TDI, TDO, TMS, TRST, EMU0 and EMU1)

LVT244 buffer IC is put to avoid the distortion of signal quality between JTAG header on EVM board and the device caused by problem of wiring length. To include or not such an IC for buffer is determined by wiring length between 14-pin JTAG header and DM270 device. When the wiring is 6 inch (about 15.24mm) long and more, put the buffer IC.

Our DM270EVM includes the buffer IC because it is large in size and the wiring length of JTAG pins is longish.

When the board is small in size and the wiring of JTAG pin is 6 inch long or less, the buffer IC can be excluded as figure below.

Example for the circuit without buffer



Among JTAG related pins in DM270, only EMU0 and EMU1 are not pulled up within the DM270 device. Management for pins when buffer is not put is as follows.

Pin	Management
TCK	Not required (with built-in pull-up resistance)
TDI	Not required (with built-in pull-up resistance)
TDO	Open because of output pin
TMS	Not required (with built-in pull-up resistance)
TRST	Not required (with built-in pull-down resistance)
EMU0	Pulled-up with 4.7KO resistance
EMU1	Pulled-up with 4.7KO resistance

27.2 Appendix B : Chip Support Library (CSL) Macro Notation

27.2.1 Introduction

The CSL macro notation is used in many sections in the DM270 TRM.

This section introduces the ARM CSL (Chip Support Library) macros, and describes the procedure to use these macros to program the various peripheral registers present in the ARM sub-system of the DM270.

The DM270 ARM CSL includes a set of macros to manipulate registers and fields within peripheral registers present in the ARM sub-system. It uses the same standard protocol to program those peripherals as the TMS320C54x Chip Support Library (CSL).

27.2.2 ARM Peripheral Modules

The DM270 ARM CSL provides set of macros to program the registers in the following peripheral modules.

Table 69. ARM Peripheral list

Peripheral	CSL Peripheral Prefix (PER)
Timer 0 - 3	TMR0 TMR1 TMR2 TMR3
Serial Port Interface 0, 1	SP0 SP1
UART 0, 1	UART0 UART1
Watch Dog Timer	WDT
MMC/SD Card Interface	MMCS
Interrupt Controller	INTC
General I/O	GIO
DSP Controller	DSPC
On-Screen Display	OSD
CCD Controller	CCDC
Preview Engine	PREV
Video Encoder	VENC
Clock Controller	CLKC
Bus Controller	BUSC
SDRAM Controller	SDRC
External Memory Interface	EMIF
USB	USB
3A Function	H3A
Memory Stick Controller	MS
Address Trace Module	ATM
I ² C Bus Interface	I2C
Instruction Cache	ICACHE

27.2.3 Using the CSL Macros

The ARM CSL provides macros to read, write peripheral registers and its individual fields. It provides efficient and consistent interface to access registers/fields of different peripherals. The macros take care of the field masking ('and'-ing, 'or'-ing) of fields and thus keep the user code clean.

The table below provides a generic description of the ARM CSL macros used on DM270 where,

- *PER*, indicates a peripheral, as listed in Table 69. (eg. CCDC, TMR0...)
- *REG*, indicates, if applicable, a register. The register name is as defined in the DM270 TRM (eg. SYNCEN, MODESET...)
- *FIELD*, indicates, if applicable, a register field. The register field name is as defined in the DM270 TRM (eg. WEN, VHDEN...)
- *regval*, indicates an integer constant or integer variable
- *fieldval*, indicates an integer constant or integer variable; all field values are right justified

Table 70. CSL Macros

Macro	Description
<i>PER_RGET(REG)</i>	Register GET - Returns the value in the peripheral register.
<i>PER_RSET(REG, regval)</i>	Register SET - Writes the value to the peripheral register.
<i>PER_FGET(REG, FIELD)</i>	Field Get - Returns the value of the specified <i>FIELD</i> in the peripheral register.
<i>PER_FSET(REG, FIELD, fieldval)</i>	Field SET - Writes <i>fieldval</i> to the specified <i>FIELD</i> in the peripheral register.

27.2.4 Example usage of CSL macros

To enable CCDC HD/VD and CCDC raw data write to SDRAM use the macros as follows:

```
CCDC_FSET(SYNCEN, WEN, 1);
CCDC_FSET(SYNCEN, HDVDEN, 1);
```

The register SYNCEN and field names WEN, HDVDEN are as defined in DM270 TRM

To set the OSD video window 1 start address 'vwin1_start' use the macros as follows:

```
OSD_FSET(VIDWINADH, V1AH, vwin1_start >> 16 );
OSD_RSET(VINWIN1AL, vwin1_start & 0xFFFF );
```

The register VIDWINADH, VIDWIN1ADL and field name V1AH are as defined in DM270 TRM

To wait for MMCSD DMA to complete use the macro as follows:

```
while ( MMCSD_FGET( DMASTAT1, RUNST ) == 1 );
```

The register DMASTAT1 and field name RUNST are as defined in DM270 TRM